

*DB2 9 Database  
Administration Workshop for  
Windows*

(Course code CF23)

**Instructor Guide**

ERC 8.3

IBM certified course material

## Trademarks

The reader should recognize that the following terms, which appear in the content of this training document, are official trademarks of IBM or other companies:

IBM® is a registered trademark of International Business Machines Corporation.

The following are trademarks of International Business Machines Corporation in the United States, or other countries, or both:

|             |                        |                                              |
|-------------|------------------------|----------------------------------------------|
| AIX         | AIX 5L                 | Approach                                     |
| CICS        | ClearCase              | DB2                                          |
| DB2 Connect | DB2 Universal Database | Distributed Relational Database Architecture |
| DRDA        | Encina                 | Enterprise Storage Server                    |
| Informix    | iSeries                | Lotus                                        |
| MQSeries    | MVS                    | MVS/ESA                                      |
| OS/2        | OS/390                 | OS/400                                       |
| pSeries     | pureXML                | QMF                                          |
| QBIC        | QuickPlace             | RAMAC                                        |
| S/390       | Tivoli                 | z/OS                                         |
| zSeries     | 1-2-3                  |                                              |

Alerts® is a registered trademark of Alphablox Corporation in the United States, other countries, or both.

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.

Microsoft, Windows and Windows NT are trademarks of Microsoft Corporation in the United States, other countries, or both.

Intel is a trademark or registered trademark of Intel Corporation or its subsidiaries in the United States and other countries.

UNIX® is a registered trademark of The Open Group in the United States and other countries.

Linux® is a registered trademark of Linus Torvalds in the United States, other countries, or both.

Other company, product, or service names may be trademarks or service marks of others.

## April 2007 edition

The information contained in this document has not been submitted to any formal IBM test and is distributed on an "as is" basis without any warranty either express or implied. The use of this information or the implementation of any of these techniques is a customer responsibility and depends on the customer's ability to evaluate and integrate them into the customer's operational environment. While each item may have been reviewed by IBM for accuracy in a specific situation, there is no guarantee that the same or similar results will result elsewhere. Customers attempting to adapt these techniques to their own environments do so at their own risk.

© Copyright International Business Machines Corporation 1999, 2007. All rights reserved.

This document may not be reproduced in whole or in part without the prior written permission of IBM.

Note to U.S. Government Users — Documentation related to restricted rights — Use, duplication or disclosure is subject to restrictions set forth in GSA ADP Schedule Contract with IBM Corp.

# Contents

|                                                             |             |
|-------------------------------------------------------------|-------------|
| <b>Trademarks</b>                                           | <b>xiii</b> |
| <b>Instructor course overview</b>                           | <b>xv</b>   |
| <b>Course description</b>                                   | <b>xvii</b> |
| <b>Agenda</b>                                               | <b>xxi</b>  |
| <b>Unit 1. Overview of DB2 9 on Linux, UNIX and Windows</b> | <b>1-1</b>  |
| Unit objectives                                             | 1-3         |
| 1.1 Overview of DB2                                         | 1-5         |
| DB2 products                                                | 1-6         |
| DB2 — The scalable DB                                       | 1-8         |
| Accessing DB2                                               | 1-11        |
| DB2 product components                                      | 1-15        |
| DRDA Application Requester (AR) support                     | 1-19        |
| DRDA Application Server (AS) support                        | 1-22        |
| Installation prerequisites                                  | 1-24        |
| Installation steps                                          | 1-27        |
| For more information                                        | 1-30        |
| Unit summary                                                | 1-32        |
| <b>Unit 2. Command Line Processor (CLP) and GUI usage</b>   | <b>2-1</b>  |
| Unit objectives                                             | 2-2         |
| 2.1 Command Line Processor (CLP)                            | 2-5         |
| CLP Command Line Processor                                  | 2-6         |
| CLP syntax                                                  | 2-8         |
| Online reference                                            | 2-10        |
| Using the CLP                                               | 2-12        |
| QUIT/TERMINATE/CONNECT RESET differences                    | 2-15        |
| CLP command options                                         | 2-17        |
| Modify CLP options                                          | 2-20        |
| Input file — no operating system commands                   | 2-23        |
| Input file — operating system commands                      | 2-25        |
| 2.2 Getting started with DB2 GUI tools                      | 2-27        |
| Roadmap to the GUI tools                                    | 2-28        |
| Control Center                                              | 2-30        |
| Command Editor                                              | 2-33        |
| SQL Assist window                                           | 2-36        |
| Administration tools                                        | 2-39        |
| Using the wizards                                           | 2-42        |
| Health Center                                               | 2-46        |
| Journal                                                     | 2-49        |
| Configuration Assistant (1 of 2)                            | 2-52        |

|                                                      |            |
|------------------------------------------------------|------------|
| Configuration Assistant (2 of 2)                     | 2-55       |
| Tool settings                                        | 2-58       |
| 2.3 Overview of the Administration Server            | 2-61       |
| DB2 Administration Server                            | 2-62       |
| Using the Administration Server                      | 2-65       |
| Unit summary                                         | 2-68       |
| <b>Unit 3. The DB2 environment</b>                   | <b>3-1</b> |
| Unit objectives                                      | 3-2        |
| What is an Instance?                                 | 3-4        |
| The Database Manager instance                        | 3-7        |
| Create and drop an instance                          | 3-9        |
| db2icrt command                                      | 3-12       |
| db2idrop command                                     | 3-15       |
| Starting and stopping an instance                    | 3-18       |
| Types of configuration                               | 3-21       |
| Database Manager configuration                       | 3-23       |
| Unit summary                                         | 3-26       |
| <b>Unit 4. Creating databases and data placement</b> | <b>4-1</b> |
| Unit objectives                                      | 4-3        |
| 4.1 Create database overview                         | 4-5        |
| Create database overview                             | 4-6        |
| Steps for database creation                          | 4-8        |
| Create Database Wizard                               | 4-11       |
| Create database syntax (1 of 3)                      | 4-14       |
| Create database syntax (2 of 3)                      | 4-18       |
| Create database syntax (3 of 3)                      | 4-22       |
| Automatic storage — database                         | 4-26       |
| Automatic storage syntax — database                  | 4-28       |
| Automatic storage — create database examples         | 4-30       |
| Automatic Storage provisioning — syntax              | 4-32       |
| Create bufferpool syntax                             | 4-35       |
| Default table spaces and buffer pool                 | 4-37       |
| System Catalog tables and views                      | 4-39       |
| 4.2 Table spaces and containers                      | 4-45       |
| DMS versus SMS table spaces                          | 4-46       |
| Writing to containers                                | 4-49       |
| Table space, container, extent, page                 | 4-52       |
| DMS table space — minimum requirements               | 4-54       |
| Defining table spaces                                | 4-56       |
| Create Table Space Wizard                            | 4-58       |
| CREATE TABLESPACE syntax (1 of 2)                    | 4-62       |
| CREATE TABLESPACE syntax (2 of 2)                    | 4-66       |
| Automatic storage — table space examples             | 4-69       |
| 4.3 Gathering system information                     | 4-73       |
| Get snapshot for tablespaces command                 | 4-74       |

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| Using the SNAPCONTAINER view . . . . .                                | 4-78       |
| Alter Table Space Wizard . . . . .                                    | 4-80       |
| ALTER TABLESPACE syntax . . . . .                                     | 4-82       |
| Placement considerations . . . . .                                    | 4-86       |
| 4.4 Describe and modify the Database Configuration . . . . .          | 4-89       |
| Database Configuration . . . . .                                      | 4-90       |
| Database Manager Configuration . . . . .                              | 4-92       |
| Database directories overview . . . . .                               | 4-94       |
| Checkpoint . . . . .                                                  | 4-97       |
| Unit summary . . . . .                                                | 4-98       |
| <b>Unit 5. Creating database objects . . . . .</b>                    | <b>5-1</b> |
| Unit objectives . . . . .                                             | 5-2        |
| 5.1 DB2 object hierarchy and physical directories and files . . . . . | 5-5        |
| DB2 object hierarchy . . . . .                                        | 5-6        |
| Database physical directories and files . . . . .                     | 5-9        |
| 5.2 Creating objects . . . . .                                        | 5-13       |
| Create Schema . . . . .                                               | 5-14       |
| Set current schema . . . . .                                          | 5-17       |
| Create Table . . . . .                                                | 5-20       |
| Create Table statement . . . . .                                      | 5-23       |
| Table partitioning . . . . .                                          | 5-28       |
| Create View . . . . .                                                 | 5-31       |
| Create View statement . . . . .                                       | 5-33       |
| Create Alias . . . . .                                                | 5-36       |
| Create Alias statement . . . . .                                      | 5-38       |
| Overview of Declared Temporary Tables . . . . .                       | 5-40       |
| Create Index . . . . .                                                | 5-44       |
| Create Index statement . . . . .                                      | 5-46       |
| Overview of Referential Integrity . . . . .                           | 5-50       |
| RI — Add a foreign key . . . . .                                      | 5-54       |
| Referential Integrity — Create Table statement . . . . .              | 5-56       |
| Unique Key considerations . . . . .                                   | 5-59       |
| Overview of Check Constraints: The need . . . . .                     | 5-62       |
| Check Constraints — definition . . . . .                              | 5-64       |
| Implementation of Check Constraints . . . . .                         | 5-67       |
| Overview of triggers: The need . . . . .                              | 5-69       |
| Implementation of triggers (1 of 2) . . . . .                         | 5-71       |
| Implementation of triggers (2 of 2) . . . . .                         | 5-73       |
| Create Trigger statement . . . . .                                    | 5-75       |
| Large objects: The need . . . . .                                     | 5-79       |
| 5.3 Creating tables with XML columns . . . . .                        | 5-83       |
| What is XML? (1 of 2) . . . . .                                       | 5-84       |
| What is XML? (2 of 2) . . . . .                                       | 5-86       |
| The XML document tree . . . . .                                       | 5-88       |
| PureXML datastore . . . . .                                           | 5-91       |
| Integration of XML and relational capabilities . . . . .              | 5-94       |

|                                                            |            |
|------------------------------------------------------------|------------|
| Two ways to query XML data . . . . .                       | 5-97       |
| For more information . . . . .                             | 5-101      |
| Unit summary . . . . .                                     | 5-105      |
| <b>Unit 6. Moving data . . . . .</b>                       | <b>6-1</b> |
| Unit objectives . . . . .                                  | 6-2        |
| Review INSERT statement . . . . .                          | 6-4        |
| 6.1 Export/Import utilities . . . . .                      | 6-7        |
| Differences between IMPORT and LOAD . . . . .              | 6-8        |
| IMPORT/EXPORT overview . . . . .                           | 6-10       |
| EXPORT command . . . . .                                   | 6-13       |
| EXPORT command syntax (basic) . . . . .                    | 6-15       |
| EXPORT command example . . . . .                           | 6-20       |
| IMPORT command . . . . .                                   | 6-22       |
| IMPORT command syntax (basic) . . . . .                    | 6-24       |
| IMPORT command example . . . . .                           | 6-29       |
| Import command... CREATE... IN TABLESPACE . . . . .        | 6-32       |
| 6.2 LOAD utility . . . . .                                 | 6-35       |
| Four phases of Load . . . . .                              | 6-36       |
| LOAD: Load phase . . . . .                                 | 6-39       |
| LOAD: Build phase . . . . .                                | 6-41       |
| LOAD: Delete phase . . . . .                               | 6-43       |
| LOAD: Index Copy phase . . . . .                           | 6-45       |
| Load command . . . . .                                     | 6-47       |
| LOAD command syntax (basic) . . . . .                      | 6-52       |
| LOAD scenario . . . . .                                    | 6-56       |
| Rules for creating Exception Table . . . . .               | 6-60       |
| Suggested methods for creating Exception Tables . . . . .  | 6-63       |
| Offline versus Online Load . . . . .                       | 6-65       |
| Table states . . . . .                                     | 6-68       |
| Load Pending state: Recovering from LOAD failure . . . . . | 6-71       |
| Backup Pending state — COPY options . . . . .              | 6-74       |
| Set Integrity Pending table state . . . . .                | 6-77       |
| SET INTEGRITY command overview . . . . .                   | 6-80       |
| SET INTEGRITY command syntax ((basic) . . . . .            | 6-82       |
| Set Integrity Pending state . . . . .                      | 6-85       |
| Meaningful steps for LOAD . . . . .                        | 6-88       |
| Overview of the db2move tool . . . . .                     | 6-90       |
| db2move tool syntax . . . . .                              | 6-92       |
| db2move tool examples . . . . .                            | 6-96       |
| Overview of the db2look tool . . . . .                     | 6-98       |
| db2look tool syntax . . . . .                              | 6-101      |
| db2look tool examples . . . . .                            | 6-105      |
| Unit summary . . . . .                                     | 6-109      |
| <b>Unit 7. Backup and recovery . . . . .</b>               | <b>7-1</b> |
| Unit objectives . . . . .                                  | 7-2        |

|     |                                                          |            |
|-----|----------------------------------------------------------|------------|
| 7.1 | Introduction to backup and recovery . . . . .            | 7-5        |
|     | Backup and recovery . . . . .                            | 7-6        |
|     | Types of failure . . . . .                               | 7-9        |
|     | Database objects . . . . .                               | 7-11       |
|     | Forms of backup . . . . .                                | 7-14       |
| 7.2 | Methods of recovery and logging . . . . .                | 7-17       |
|     | Methods of recovery . . . . .                            | 7-18       |
|     | Introduction to logging . . . . .                        | 7-21       |
|     | Log file use during concurrent transactions . . . . .    | 7-24       |
|     | Circular logging . . . . .                               | 7-26       |
|     | Archival logging . . . . .                               | 7-29       |
|     | Mirrored logging . . . . .                               | 7-33       |
|     | Log file information . . . . .                           | 7-35       |
|     | Location of log files . . . . .                          | 7-37       |
|     | Configure database logs — parameters . . . . .           | 7-40       |
|     | Configure database logs — GUI (Control Center) . . . . . | 7-48       |
|     | Configure database logs — Wizard (CC) . . . . .          | 7-50       |
|     | Configure database logs — Command line . . . . .         | 7-52       |
|     | Recovery history file . . . . .                          | 7-54       |
| 7.3 | Backup and restore utilities . . . . .                   | 7-59       |
|     | BACKUP command . . . . .                                 | 7-60       |
|     | Syntax of the BACKUP command . . . . .                   | 7-63       |
|     | BACKUP using the GUI interface . . . . .                 | 7-68       |
|     | Online versus offline BACKUP . . . . .                   | 7-71       |
|     | The backup files . . . . .                               | 7-73       |
|     | RESTORE command . . . . .                                | 7-77       |
|     | Syntax of the RESTORE command . . . . .                  | 7-80       |
|     | RESTORE using the GUI interface . . . . .                | 7-84       |
|     | Backup/restore table space considerations . . . . .      | 7-87       |
|     | Roll forward pending . . . . .                           | 7-90       |
|     | Syntax of the ROLLFORWARD command . . . . .              | 7-92       |
|     | ROLLFORWARD — GUI interface . . . . .                    | 7-97       |
|     | ROLLFORWARD — how far? . . . . .                         | 7-100      |
|     | Log file naming . . . . .                                | 7-103      |
|     | Disaster recovery considerations . . . . .               | 7-106      |
|     | Questions to consider . . . . .                          | 7-109      |
|     | Logging/backup requirements summary . . . . .            | 7-111      |
|     | Advanced recovery features . . . . .                     | 7-113      |
|     | DB2 Advanced Recovery and HA Workshop . . . . .          | 7-120      |
|     | Checkpoint . . . . .                                     | 7-123      |
|     | Unit summary . . . . .                                   | 7-124      |
|     | <b>Unit 8. Locking and concurrency . . . . .</b>         | <b>8-1</b> |
|     | Unit objectives . . . . .                                | 8-2        |
| 8.1 | Locking objects, strategies, and modes . . . . .         | 8-5        |
|     | Why are locks needed? . . . . .                          | 8-6        |
|     | Classification of locks . . . . .                        | 8-9        |

|                                       |            |
|---------------------------------------|------------|
| Objects of locks                      | 8-11       |
| Locking strategies                    | 8-13       |
| Table lock modes                      | 8-16       |
| Row lock modes                        | 8-20       |
| Lock mode compatibility               | 8-23       |
| 8.2 Isolation levels                  | 8-31       |
| Introduction to isolation levels      | 8-32       |
| Isolation levels                      | 8-34       |
| Repeatable Read versus Read Stability | 8-37       |
| Summary of isolation levels           | 8-39       |
| SET CURRENT ISOLATION                 | 8-41       |
| 8.3 Explicit versus implicit locking  | 8-45       |
| Explicit versus implicit locking      | 8-46       |
| LOCK TABLE statement                  | 8-49       |
| Lock conversion                       | 8-52       |
| Lock escalation                       | 8-55       |
| 8.4 Lock timeouts and deadlock        | 8-59       |
| Lock wait and timeout                 | 8-60       |
| SET CURRENT LOCK TIMEOUT              | 8-63       |
| Deadlock causes and detection         | 8-67       |
| Checkpoint                            | 8-75       |
| Unit summary                          | 8-78       |
| <b>Unit 9. Problem determination</b>  | <b>9-1</b> |
| Unit objectives                       | 9-2        |
| 9.1 Problem solving                   | 9-5        |
| How to solve a problem                | 9-6        |
| Diagnostic data and data checklist    | 9-9        |
| First Failure Data Capture (FFDC)     | 9-11       |
| FFDC structure                        | 9-16       |
| Interpreting the Notification Log     | 9-19       |
| Diagnostic Log Analysis Tool: db2diag | 9-24       |
| 9.2 Database monitoring               | 9-31       |
| Why use database monitors?            | 9-32       |
| Ways to monitor the database          | 9-35       |
| Snapshot Monitor — who and how?       | 9-39       |
| Snapshot Monitor — when?              | 9-41       |
| Snapshot Monitor — what?              | 9-43       |
| Snapshot Monitor — CLP commands       | 9-45       |
| Snapshot Monitor — taking snapshots   | 9-48       |
| Event Monitor                         | 9-51       |
| Filtering Event Monitor output        | 9-54       |
| Examining Event Monitor output        | 9-56       |
| Activity Monitor                      | 9-58       |
| Health Monitor                        | 9-61       |
| Health indicator settings             | 9-64       |
| Health Monitor — alerts view          | 9-69       |



|                                                    |             |
|----------------------------------------------------|-------------|
| Alarm details                                      | 9-72        |
| Independent trace facility — db2trc                | 9-75        |
| 9.3 EXPLAIN                                        | 9-79        |
| Explain tool                                       | 9-80        |
| EXPLAIN facility tools                             | 9-82        |
| Visual explain access plan graph                   | 9-84        |
| 9.4 Additional commands                            | 9-87        |
| Additional commands                                | 9-88        |
| LIST ACTIVE DATABASES command                      | 9-90        |
| LIST APPLICATIONS command                          | 9-92        |
| FORCE APPLICATION command                          | 9-94        |
| The db2set command                                 | 9-97        |
| Retrieve stats from a running instance: db2pd      | 9-99        |
| Performance tuning and monitoring workshop         | 9-105       |
| Checkpoint                                         | 9-107       |
| Unit summary                                       | 9-108       |
| <b>Unit 10. Application issues and performance</b> | <b>10-1</b> |
| Unit objectives                                    | 10-3        |
| 10.1 Application alternatives                      | 10-5        |
| Application alternatives                           | 10-6        |
| 10.2 Database engine access                        | 10-9        |
| Database engine access                             | 10-10       |
| Performance — dynamic versus static SQL            | 10-13       |
| Database Manager APIs                              | 10-16       |
| 10.3 Embedded SQL program preparation              | 10-21       |
| Programming interfaces                             | 10-22       |
| Embedded SQL applications                          | 10-24       |
| Write the program                                  | 10-27       |
| Program preparation steps                          | 10-30       |
| Associating load modules and packages              | 10-34       |
| Precompiling versus binding                        | 10-37       |
| Adjusting qualifier and owner                      | 10-39       |
| Precompile command syntax (basic)                  | 10-41       |
| Bind command syntax (basic)                        | 10-45       |
| Package catalog views                              | 10-48       |
| Application process                                | 10-51       |
| Developer Workbench and SQL PL                     | 10-53       |
| 10.4 Application performance                       | 10-57       |
| DB2 Optimizer                                      | 10-58       |
| Optimization classes                               | 10-61       |
| Adjusting the QUERY OPTIMIZATION class             | 10-65       |
| Design Advisor                                     | 10-67       |
| Implicit rebind                                    | 10-70       |
| Explicit rebind                                    | 10-73       |
| Sorts — piped versus non-piped                     | 10-75       |
| Sorts — Sort passes                                | 10-77       |

|                                                   |             |
|---------------------------------------------------|-------------|
| Concurrent application tuning introduction        | 10-79       |
| Asynchronous Page Cleaner                         | 10-82       |
| 10.5 Minimizing communication overhead            | 10-85       |
| Minimizing communication overhead                 | 10-86       |
| Levels of data transmission                       | 10-88       |
| Stored procedures                                 | 10-91       |
| 10.6 Utilities and tools                          | 10-95       |
| Operational utilities                             | 10-96       |
| Clustered and non-clustered indexes               | 10-100      |
| RUNSTATS command syntax (basic)                   | 10-103      |
| Determine if table reorganization is required     | 10-107      |
| REORGCHK statistics                               | 10-111      |
| Reorganize table / index command syntax           | 10-114      |
| Statistical Catalog views                         | 10-118      |
| The db2look mimic mode                            | 10-121      |
| 10.7 Additional learning opportunities            | 10-125      |
| Need more information? (1 of 3)                   | 10-126      |
| More information (2 of 3)                         | 10-128      |
| More information (3 of 3)                         | 10-131      |
| Checkpoint                                        | 10-135      |
| Unit summary                                      | 10-139      |
| <b>Unit 11. Security</b>                          | <b>11-1</b> |
| Unit objectives                                   | 11-2        |
| 11.1 Authentication versus authorization          | 11-5        |
| Authentication versus authorization               | 11-6        |
| 11.2 Access control — privileges                  | 11-15       |
| DB2 access control hierarchy                      | 11-16       |
| Database Authority summary                        | 11-19       |
| Authorities and privileges                        | 11-22       |
| Controlling use of schemas                        | 11-28       |
| Protecting resources through programs             | 11-31       |
| 11.3 Explicit privileges                          | 11-33       |
| Explicit authorization                            | 11-34       |
| Grant statement — table / view privileges syntax  | 11-36       |
| Individual IDs, group IDs, and public             | 11-40       |
| Using group IDs                                   | 11-42       |
| Public privilege support for static SQL and views | 11-45       |
| Privileges required for programming               | 11-48       |
| Dynamic rules and dynamic SQL                     | 11-50       |
| DB2 BIND GRANT option                             | 11-53       |
| 11.4 Implicit privileges                          | 11-55       |
| Implicit privileges (1 of 2)                      | 11-56       |
| Implicit privileges (2 of 2)                      | 11-58       |
| Implicit privileges scenarios                     | 11-61       |
| Grant / Revoke scenarios                          | 11-63       |
| 11.5 Querying privileges/authorities              | 11-65       |

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| Query who has privileges .....                                      | 11-66      |
| Query privileges granted to current ID .....                        | 11-69      |
| 11.6 Label Based Access Control (LBAC) .....                        | 11-71      |
| Label Based Access Control (LBAC) overview .....                    | 11-72      |
| Security model extensions — SECADM Authority .....                  | 11-74      |
| Checkpoint .....                                                    | 11-77      |
| Unit summary .....                                                  | 11-78      |
| <b>Appendix A. LOBs, UDFs, UDTs, and Relational Extenders .....</b> | <b>A-1</b> |
| Appendix objectives .....                                           | A-2        |
| LOBs .....                                                          | A-4        |
| Large objects — memory consideration .....                          | A-5        |
| Large objects — manipulation .....                                  | A-7        |
| DB2 Relational Extenders .....                                      | A-10       |
| User-Defined distinct types and User-Defined functions .....        | A-15       |
| User-Defined Types — the need .....                                 | A-16       |
| User-Defined Types — definition .....                               | A-18       |
| Create Distinct Type — GUI .....                                    | A-21       |
| User-Defined Functions — the need .....                             | A-23       |
| User-Defined Function — definition .....                            | A-25       |
| User-Defined Function — sourced .....                               | A-28       |
| User-Defined Functions — fencing .....                              | A-31       |
| Checkpoint .....                                                    | A-35       |
| Appendix summary .....                                              | A-36       |
| <b>Appendix B. Checkpoint solutions .....</b>                       | <b>B-1</b> |
| <b>Index .....</b>                                                  | <b>X-1</b> |



# Trademarks

The reader should recognize that the following terms, which appear in the content of this training document, are official trademarks of IBM or other companies:

IBM® is a registered trademark of International Business Machines Corporation.

The following are trademarks of International Business Machines Corporation in the United States, or other countries, or both:

|              |                         |                                               |
|--------------|-------------------------|-----------------------------------------------|
| AIX®         | AIX 5L™                 | Approach®                                     |
| CICS®        | ClearCase®              | DB2®                                          |
| DB2 Connect™ | DB2 Universal Database™ | Distributed Relational Database Architecture™ |
| DRDA®        | Encina®                 | Enterprise Storage Server®                    |
| Informix®    | iSeries™                | Lotus®                                        |
| MQSeries®    | MVS™                    | MVS/ESA™                                      |
| OS/2®        | OS/390®                 | OS/400®                                       |
| pSeries®     | pureXML™                | QMF™                                          |
| QBIC®        | QuickPlace®             | RAMAC®                                        |
| S/390®       | Tivoli®                 | z/OS®                                         |
| zSeries®     | 1-2-3®                  |                                               |

Alerts® is a registered trademark of Alphablox Corporation in the United States, other countries, or both.

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

UNIX® is a registered trademark of The Open Group in the United States and other countries.

Linux® is a registered trademark of Linus Torvalds in the United States, other countries, or both.

Other company, product, or service names may be trademarks or service marks of others.



# Instructor course overview

This course will teach the students how to perform basic database administration of a DB2 system.

**NOTE:** If you are using the source for this course, when formatting the *Instructor Guide* and *Student Notebook* for CF21, Plat\_A, Plat\_A\_Inst, Plat\_B and Plat\_B\_Inst conditions must be turned **ON**.

To make sure they're turned ON, go to **Special -> Conditional Text**. Only **instr**, **Plat\_A**, **Plat\_A\_Inst**, **Plat\_B**, and **Plat\_B\_Inst** (for *Instructor Guide*) or **Plat\_A**, **Plat\_B** and **student** (for *Student Notebook*) should be listed in the **In** box depending on which book you are printing.

## Course strategy

The strategy in the development of this course was to keep together units common to *DB2 9 Database Administration Workshop for Linux*, *DB2 9 Database Administration Workshop for UNIX*, and *DB2 9 for Windows Database Administration Workshop*, so that maintenance, updates, and product upgrades could be accomplished with minimum effort. The lab files for the AIX, Linux, and Solaris labs are in one lab document. The FastPath to DB2 for experienced relational database administrators is also built from these softcopy files.





# Course description

## DB2 9 Database Administration Workshop for Windows

**Duration:** 4 days

### Purpose

This course is designed to teach you how to perform database administration tasks using DB2 9. These tasks include creating DB2 instances, creating and populating databases, and using logical design to support concurrency and recovery requirements. New features, such as range partitioning, data row compression, and pureXML (native XML storage) will be introduced.

### Audience

System administrators, database administrators, and technical personnel involved in planning, implementing, and maintaining DB2 databases.

### Prerequisites

Before taking this course you should be able to:

- Use basic OS functions such as utilities, file permissions, hierarchical file system, commands, and editor
- State the functions of the Structured Query Language (SQL), and be able to construct DDL, DML, and authorization statements
- Discuss basic relational database concepts and objects such as tables, indexes, views, and joins

These skills can be developed by taking:

- OS Training
  - AIX 5L Basics
  - Linux Basics and Administration
  - Windows Systems Administration
  - Or by having equivalent HP-UX or Solaris administration experience
- DB2 SQL Workshop
- DB2 Fundamentals

## Objectives

After completing this course, you should be able to:

- Administer a DB2 database system using commands and GUI tools
- Implement DB2 security
- Manage System Managed Storage (SMS) and Database Managed Storage (DMS) databases and apply data placement principles
- Implement a given logical database design using DB2 to support integrity and concurrency requirements
- List and describe the components of DB2
- Define a DB2 recovery strategy and perform the tasks necessary to support the strategy
- Describe the application development process with respect to DB2 considerations
- Use autonomic features of DB2

## Contents

Overview of DB2 9 on Linux, UNIX and Windows

Command Line Processor (CLP) and GUI usage

The DB2 environment

Creating databases and data placement

Creating database objects

Moving data

Backup and recovery

Locking and concurrency

Problem determination

Application issues and performance

Security

## Curriculum relationship

|             |                                                  |
|-------------|--------------------------------------------------|
| <b>CF03</b> | DB2 Family Fundamentals                          |
| <b>CF53</b> | DB2 Fundamentals                                 |
| <b>CF20</b> | DB2 9 Database Administration Workshop for Linux |
| <b>CF21</b> | DB2 9 Database Administration Workshop for UNIX  |
| <b>CF24</b> | DB2 Multi Partition DBA Workshop                 |

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| <b>CF28</b> | Fast Path to DB2 9 for Experienced Relational DBAs                            |
| <b>CF50</b> | DB2 V6 Administrator Certification Crammer                                    |
| <b>CF10</b> | DB2 Programming Fundamentals                                                  |
| <b>CF11</b> | DB2 Advanced Programming                                                      |
| <b>CG11</b> | DB2 for Linux, UNIX, and Windows Programming using Java                       |
| <b>CF41</b> | DB2 Performance Monitoring and Tuning Workshop                                |
| <b>CF44</b> | DB2 Multi Partitioned Performance Monitoring and Tuning Workshop for UNIX     |
| <b>CF46</b> | DB2 for Linux, UNIX, and Windows Advanced Database Administration for Experts |
| <b>CF15</b> | Web Enabled DB2: Net.Data                                                     |
| <b>CF60</b> | DB2 Connect DRDA Implementation with TCP/IP                                   |
| <b>CF63</b> | DB2 Connect to DB2 for OS/390 Problem Determination and Performance           |
| <b>CF49</b> | DB2 Advanced Recovery and High Availability Workshop                          |
| <b>CG07</b> | DB2 UDB for UNIX, Windows and OS/2 V7 Transition                              |
| <b>CG24</b> | DB2 Multi Partition Environment for Single Partition DBAs                     |
| <b>CF48</b> | DB2 9 for Linux, UNIX, and Windows Quickstart for Experienced Relational DBAs |
| <b>DW48</b> | DB2 Federated Database with DB2 Information Integrator: Up and Running        |
| <b>DW49</b> | DB2 Federated Database Performance, Tuning, and Advanced Topics Workshop      |



# Agenda

The planned agenda follows. Here are the considerations:

- The first five units must be taught in the order specified:
  - Overview of DB2
  - Command Line Processor (CLP) and GUI usage
  - The DB2 environment
  - Creating databases and data placement
  - Create database objects
- Moving data
- Backup and recovery
- Problem determination
- Locking and concurrency
- Application issues and performance
- Security

## Day 1

(00:30) Welcome  
(00:45) Unit 1 - Overview of DB2 9 on Linux, UNIX and Windows  
(00:10) Lab 1 (Starting your lab environment)  
(01:15) Unit 2 - Command Line Processor (CLP) and GUI usage  
(00:55) Lab 2 (DB2 Customization)  
(01:00) Unit 3 - The DB2 environment  
(01:00) Lab 3 (DB2 environment)

## Day 2

(01:00) Unit 4 - Creating databases and data placement  
(01:00) Lab 4 (Creating databases and data placement)  
(01:20) Unit 5 - Creating database objects  
(01:00) Lab 5 (Create objects)  
(01:50) Unit 6 - Moving data

## Day 3

(01:15) Lab 6 (Moving data)  
(01:50) Unit 7 - Backup and recovery  
(00:40) Lab 7 (Backup and recovery)  
(01:10) Unit 8 - Locking and concurrency  
(00:45) Lab 8 (Investigating DB2 locking)

## Day 4

(01:40) Unit 9 - Problem determination  
(00:30) Lab 9 (Problem determination)  
(01:30) Unit 10 - Application issues and performance  
(00:30) Lab 10 (Application performance tools)  
(01:10) Unit 11 - Security  
(01:00) Lab 11 (Security)

# Unit 1. Overview of DB2 9 on Linux, UNIX and Windows

## Estimated time

00:45

## What this unit is about

This unit describes the overall DB2 system, including a discussion on the components of the system. A high-level overview of the installation of DB2 is also covered.

The DB2 products that run on Windows and Linux/UNIX platforms are collectively known as *DB2 for Linux, UNIX, and Windows*. The rest of this document discusses DB2 on these platforms, unless otherwise noted. References to “DB2” refer to the DB2 Version 9.1. When a specific reference is needed to a distinct product, such as DB2 for AIX, DB2 for Windows, or DB2 for Linux/UNIX, it will be stated explicitly.

Also, there will be times when an indication will be made of a difference between Linux/UNIX and Windows. The comments indicated for Linux/UNIX apply to all Linux/UNIX products including those that run on Intel hardware. The comments indicated for Windows apply to Windows NT, Windows 2000, Windows XP, Windows ME, and Windows .NET.

## What you should be able to do

After completing this unit, you should be able to:

- Contrast the DB2 Family of products
- Identify the DB2 Products
- Describe the functions of DB2 components
- Explore installation and parameters

## How you will check your progress

Accountability:

- Machine exercises

## References

*Administration Guide*

*Quick Beginnings for DB2 Servers*

*Quick Beginnings for DB2 Clients*

*Getting started with DB2 installation and administration on Linux and Windows*

<http://www.ibm.com/software/data/db2/udb/support/manualsv9.html>



## Unit objectives

After completing this unit, you should be able to:

- Contrast the DB2 family of products
- Identify the DB2 products
- Describe the functions of DB2 components
- Explore installation and parameters

© Copyright IBM Corporation 2007

Figure 1-1. Unit objectives

CF238.3

### **Notes:**

***Instructor notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Identify what the students should learn in this unit.

**Additional information** —

**Transition statement** — Let's start by looking at the DB2 family.

## 1.1 Overview of DB2

### Instructor topic introduction

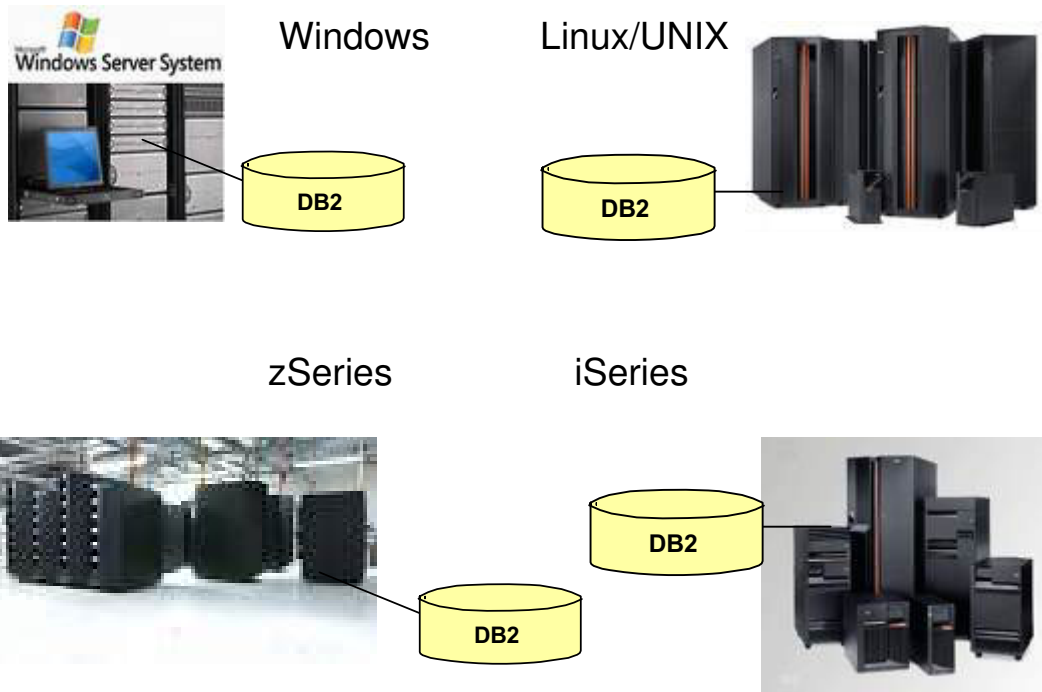
**What students will do** — List the products that make up the DB2 family of products and name the DB2 components.

**How students will do it** — Lecture and lab.

**What students will learn** — The DB2 family of products and the DB2 components.

**How this will help students on their job** — They will be able to determine what product needs to be installed on each workstation to accomplish the job needed at that workstation.

## DB2 products



© Copyright IBM Corporation 2007

Figure 1-2. DB2 products

CF238.3

### Notes:

DB2 is a relational database management system (RDBMS) that enables users to create, update, and control relational databases using Structured Query Language (SQL). Designed to meet the information needs of small and large businesses alike, it is available on a variety of platforms, including large systems such as z/OS, OS/390, VM, and VSE; mid-sized systems such as iSeries, pSeries, Linux, HP-UX, and Solaris; and single or LAN-based systems such as Windows 2003 and Windows XP. **Clients running V8 or V9 can access DB2 9 servers. Clients running Version 7 are supported with some restrictions.**

All members of the DB2 family have similar external architecture and use many of the same key algorithms, thereby ensuring portability for customers across platforms.

***Instructor notes:***

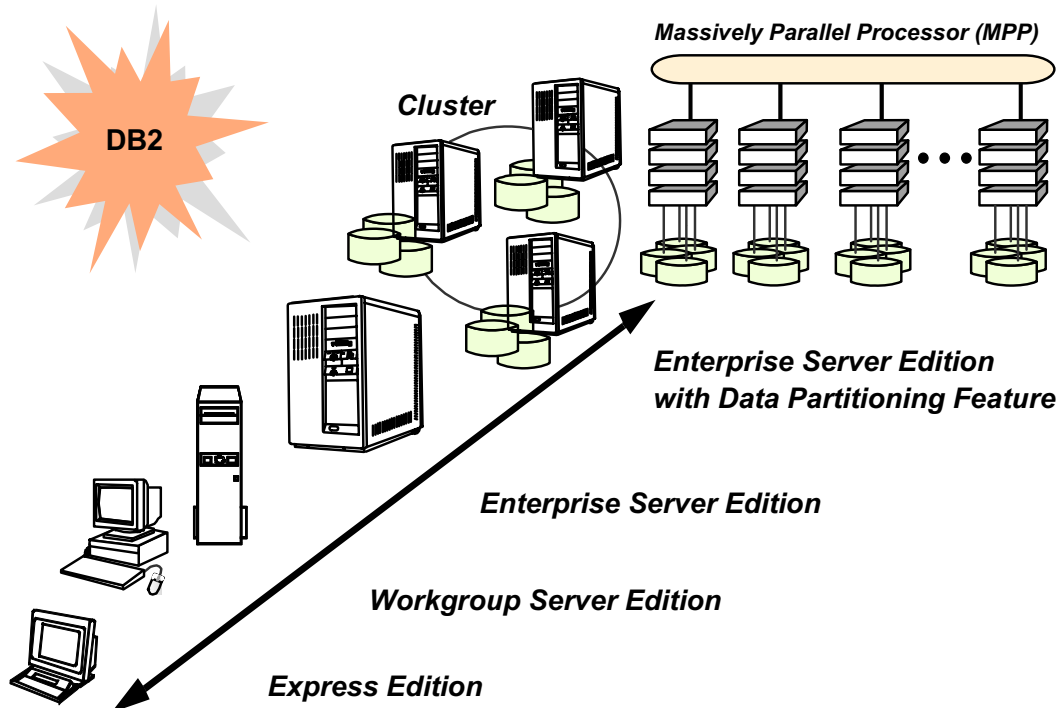
**Purpose** — Introduce IBM DB2 products.

**Details** — IBM has DB2 products on multiple platforms. Though the products on each platform have been customized for their platform, they have features common across all of the products.

**Additional information** —

**Transition statement** — Let's look at the server products that are available.

## DB2 — The scalable DB



© Copyright IBM Corporation 2007

Figure 1-3. DB2 — The scalable DB

CF238.3

### Notes:

The graphic shows the scalability of the DB2 database server. DB2 is capable of supporting hardware platforms from uniprocessor laptops to massively parallel systems with hundreds of nodes and many processors per node. In between, it can support SMP machines or clusters of SMP machines. This provides both extensive and granular growth.

- **DB2 Express Edition (EE)** is the ideal entry level data server. Suitable for transaction processing or complex query workloads on servers with up to two processors. Your workload may be smaller, but your business data is as critical to you as to the largest enterprise. DB2 Express 9 provides many of Enterprise capabilities as value add features to allow you to control costs by purchasing only the capability you need. EE includes all core DB2 capabilities, packaged in two media packs — the Core CD and the ECF (electronic certification file), being for either Authorized User or CPU.
- **DB2 Workgroup Server Edition (WSE)** provides full DB2 database server capacity for your departmental LANs running Windows NT, Windows 2000, Windows XP, Windows .NET, AIX, HP-UX, Linux, and Solaris operating environments. It is the database server designed for deployment in a departmental or small business

environment that involves a small number of internal users. Workgroup Server Edition uses a licensing model designed to provide an attractive price point for smaller installations while still providing a full function database server. Workgroup Server Edition can be deployed in Linux, UNIX, and Windows server environments on systems with up to four CPUs. WSE is packaged in two media packs, which includes the Core CD and the ECF, being for either Authorized User or CPU.

- **DB2 Enterprise Server Edition (ESE) meets the database server needs of midsize to large businesses.** It can be deployed in Linux, UNIX, and Windows server environments on any size server. Enterprise Server Edition is the ideal foundation for building data warehouses, transaction processing, or Web-based solutions as well as a backend for packaged solutions like Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), and Supply Chain Management (SCM). **Additionally, Enterprise Server Edition offers connectivity and integration for other enterprise DB2 and Informix data sources.** Platforms supported include Windows NT, Windows 2000, Windows .NET, Linux, Linux/390, AIX, HP-UX, and Solaris. ESE is packaged in two media packs, including the Core CD and the ECF (being for either Authorized User or CPU).
- **DB2 Database Partitioning Feature (DPF) allows ESE customers to partition a database within a single system or across a cluster of system.** The DPF capability provides the customer with multiple benefits, including scalability to support very large databases or complex workloads and increased parallelism for administration tasks. The DPF feature is only available in the CPU Option, contained in a separate ECF.
- The ECFs for each of the Features contain the entitlements for both the Authorized User and CPU Option, with the exception of DPF and Homogeneous Federation Feature, which are only available in the CPU Option.

## ***Instructor notes:***

**Purpose** — Introduce DB2 products.

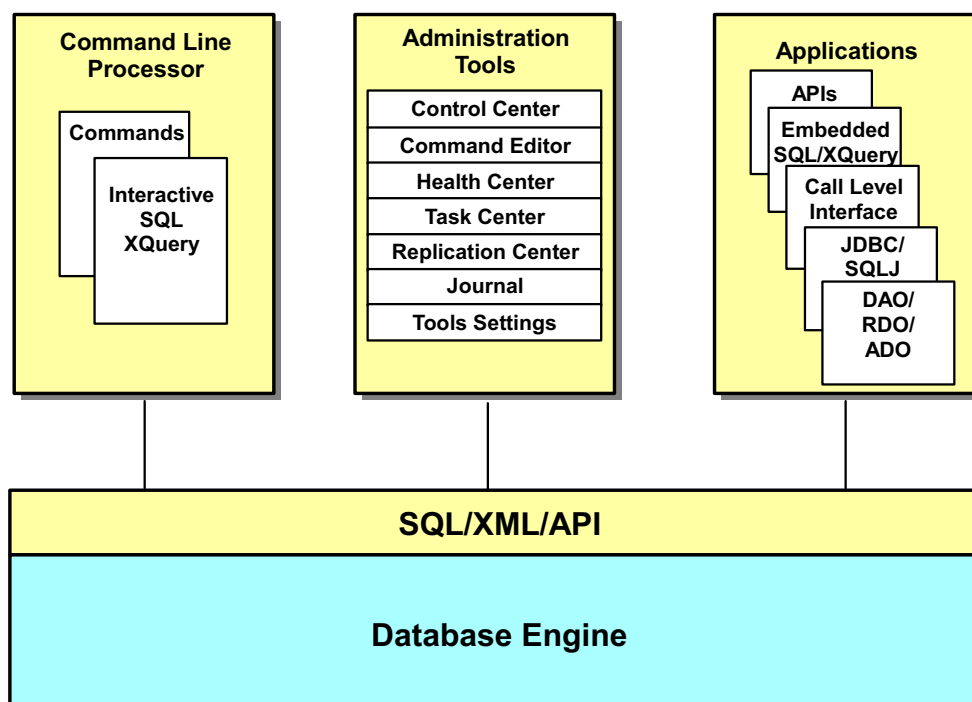
**Details** —

**Additional information** — DB2 Database is the database management system that delivers a flexible and cost-effective database platform to build robust on demand business applications. DB2 further leverages your resources with broad support for open standards and popular development platforms like J2EE and Microsoft .NET. The DB2 family also includes solutions tailored for specific needs like business intelligence and advanced tooling. Whether your business is large or small, DB2 has a solution built and priced to meet your unique needs.

**Transition statement** — Let's look at the client products in more detail.



# Accessing DB2



© Copyright IBM Corporation 2007

Figure 1-4. Accessing DB2

CF238.3

## Notes:

The main DB2 software components are:

- **Database Engine** — Provides the base functions of DB2. It manages and controls all access to data. It generates packages (stored access paths to the data), provides transaction management, ensures data integrity and data protection, and provides concurrency control.  
The basic elements of the Database Engine are database objects, system catalogs, directories, and configuration files. All access to data takes place through the SQL or XML interface. For all commands the API interface is used.
- **Command Line Processor (CLP)** — CLP is used to dynamically execute SQL requests, XQuery requests, DB2 commands, or all of them. CLP may be used to access local workstation databases, remote workstation databases, or remote Distributed Relational Database Architecture Application Server (DRDA AS) databases via DB2 Connect Personal Edition or DB2 Connect Enterprise Edition.

- **Administration Tools** — A collection of GUI tools that help to manage and administer databases, including:
  - **Control Center** — Manage systems, DB2 instances, DB2 for OS/390 and z/OS subsystems, databases, and database objects such as tables and views; display all of your systems, databases, and database objects and perform administration tasks on them; from the Control Center, open other centers and tools to help you optimize queries, jobs, and scripts, perform data warehousing tasks, create stored procedures, and work with DB2 commands.
  - **Replication Center** — Administer relational data between DB2 servers or databases; define replication environments, copy designated changes from one location to another, and synchronize the data in both locations.
  - **Command Editor** — Execute DB2 commands, XQuery and SQL statements, execute z/OS or OS/390 host system console commands, work with command scripts, and view a graphical representation of the access plan for explained SQL and XQuery statements.
  - **Task Center** — Organize task flow, schedule tasks, and distribute notifications about the status of completed tasks. A task can be created from a script that contains DB2, operating system, or MVS JCL commands.
  - **Health Center** — Identifies key performance and resource allocation problems within DB2, through notifications such as alarms or warnings, and provides recommended actions that can help to resolve the problems.
  - **Journal** — Focal point for viewing historical information generated within the Control Center and its components; views include: Task History, Database History, Performance Monitor, Alerts, Messages, Notification Log messages.
  - **License Center** — Display license status and usage information for DB2 products installed on your system; configure your system for license monitoring.
  - **Development Center** — Work with stored procedures, user-defined functions (UDFs), and structured types; modify existing routines and rebuild them, and run the routines on the database server for testing and debugging; deploy routines from a development project or database to a production server, and export or import routines to and from your project.
  - **Tools Settings** — For setting termination characters, fonts, OS/390 and z/OS options, Health Center Status Beacon options, and scheduler settings.
- **Applications** — To access the database, you can:
  - Use DB2 APIs to perform administrative functions such as backing up and restoring databases.
  - Embed static and dynamic SQL statements as well as XQuery statements in your applications.
  - Code DB2 Call Level Interface (DB2 CLI) function calls in your applications to invoke dynamic SQL statements. DB2 CLI is based on the Microsoft Open Database Connectivity (ODBC) specification, and the ISO CLI standard.
  - Develop Java applications and applets that call the Java Database Connectivity application programming interface (JDBC API). DB2 implements two

standards-based Java programming APIs: Java Database Connectivity (JDBC) and embedded SQL for Java (SQLJ).

- Develop Microsoft Visual Basic and Visual C++ applications that conform to Data Access Object (DAO) and Remote Data Object (RDO) specifications, and ActiveX Data Object (ADO) applications that use the Object Linking and Embedding Database (OLE DB) Bridge.
- Develop applications using IBM or third-party tools such as Net.Data, Excel, Perl, and Open Database Connectivity (ODBC) end-user tools such as Lotus Approach, and its programming language, LotusScript.

## ***Instructor notes:***

**Purpose** — Introduce the components of DB2.

**Details** — All access to DB2 data is through SQL or XQuery. APIs give access to perform database environment manipulation.

The SQL and XQuery statements or API commands can be issued through a supplied tool in the Command Line Processor. It allows interactive SQL/XQuery statements to be issued, or commands which invoke the API functions.

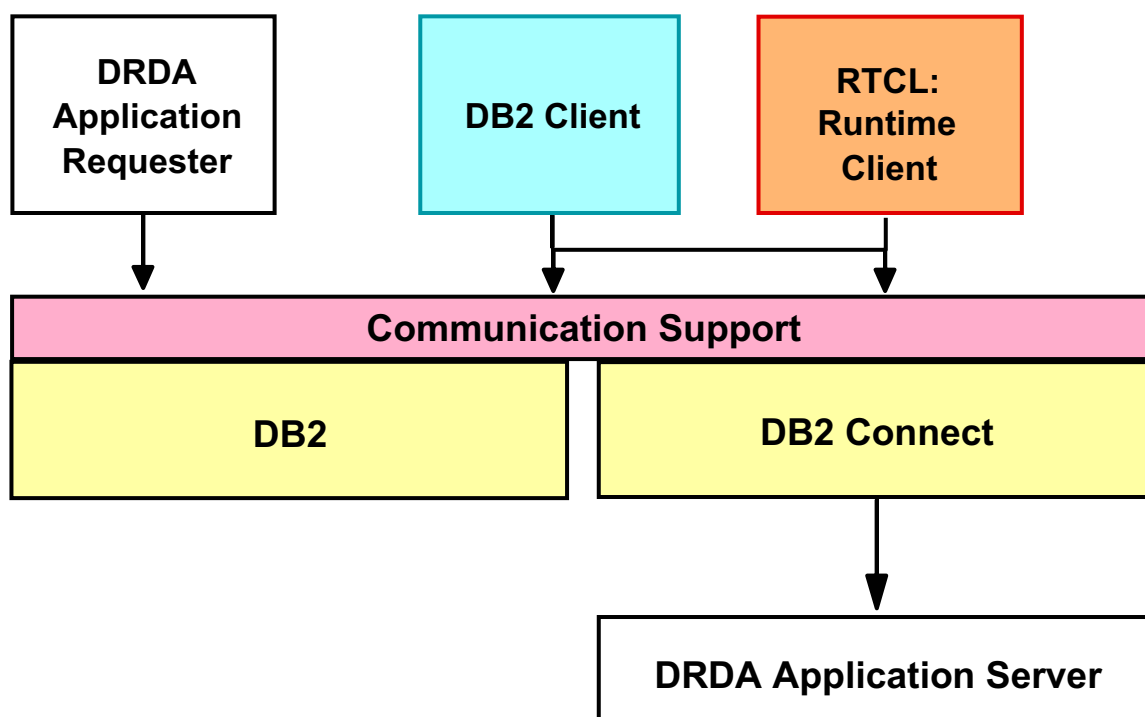
The Administrative Tools are applications provided by the database product. We will see more about the Administration tools in the GUI tools lecture.

Finally, customers can write their own applications. We will examine this in more detail in the Application Alternatives unit, but applications can be written in embedded SQL (either static or dynamic), using the Call Level Interface (also known as CLI, ODBC applications use the CLI capability), or using APIs to supply command-type functionality.

**Additional information** — Since version 9 of DB2 access to data can be done with SQL, XQuery, SQL statements with XQuery elements and XQuery with SQL elements.

**Transition statement** — [Let's look at the product components.](#)

## DB2 product components



© Copyright IBM Corporation 2007

Figure 1-5. DB2 product components

CF238.3

### Notes:

DB2 provides a full-function, robust, relational database management system (RDBMS) along with a set of related products designed for the DB2 platforms. DB2 brings to these platforms the support of the IBM SQL technology that is available on all members of the DB2 family.

**A Communication Support product must be installed on the server to enable remote client access.**

**Distributed Relational Database Architecture (DRDA)** Application Requesters represent DRDA requests coming from, for example, z/OS, VM, and OS/400. No special product or enablement, other than support of remote clients, is required on the DB2 Server to support DRDA Application Requesters.

**The DB2 Application Development Client and DB2 Administration Client have been combined and renamed to DB2 Client.**

The DB2 Client provides the ability for workstations from a variety of platforms to access and administer DB2 databases. The DB2 Client has all the features of the DB2 Runtime

Client and also includes all the DB2 administration tools and support for Thin Clients. The DB2 Client is a collection of graphical and non-graphical tools and components for developing character-based, multimedia, and object-oriented applications. Special features include a plug-in to the Developer Workbench (was Development Center) and sample applications for all supported programming languages.

The DB2 Runtime Client (RTCL) is a lightweight client that provides the functionality required for an application to access DB2 servers and DB2 Connect servers. Functionality includes communication protocol support and support for application interfaces such as JDBC, SQLJ, ODBC, CLI, and OLE DB. Compared to previous versions, due to the removal of most of the previous Run-Time Client GUI facilities, the current Runtime Client has diminished disk requirements.

DB2 Connect provides DRDA application requester (AR) function. With Communications Support, DB2 Connect enables applications running on DB2 for Linux, UNIX, and Windows platforms to access and update data on DB2 for OS/390, DB2 for z/OS, DB2 for VM and VSE, DB2 for iSeries, and other database management systems that are DRDA-compliant. The DB2 Connect product incorporates the following utilities and functions of DB2:

- Command Line Processor (CLP), which allows you to issue SQL statements to access host platforms
- Database system monitor utility, which is an aid in the problem determination area
- The bind command, which allows you to bind packages to the host platforms
- The IMPORT and EXPORT utilities, which allow you to move data between host platforms and the DB2 Connect workstations
- The ability to catalog databases for the host platform

**Instructor notes:**

**Purpose** — Introduce the companion products.

**Details** — Starting with the box labeled DB2: this represents the database engine. All you **must** have to use DB2 is the database engine. Users who want to access the database could telnet to the server that contains the database, and issue a request from locally.

But, more commonly, users access DB2 from remote clients: often Windows workstations running client applications. To enable remote client access, one of the Communication Support protocols needs to be enabled.

Available Communication Support protocols:

- AIX: TCP/IP
- HP-UX: TCP/IP
- Linux: TCP/IP
- Solaris: TCP/IP
- Windows: TCP/IP, Named Pipes, NetBIOS

This then allows access from remote clients running the Runtime client or the DB2 Client. Starting with V8, the only way of remotely administering databases is to use TCP/IP.

The Runtime client is a run-time environment only; it provides the capability for a remote client to run an application against DB2. It is not licensed; you can install the Runtime Client on every machine in your network.

The DB2 Client allows for the development of applications. It includes the functionality of the Runtime client and Administration functions, but it also includes the components necessary to develop applications: precompilers, include files, libraries.

We will see a bit more on both of these on the next page.

DB2 provides a set of administrative tools that can be installed in the client environment, called the DB2 Client. These include tools for the DBA to manage and administer the DB2 system via GUI tools. We will see more on these tools in the next unit on GUI tools.

Moving to the box labeled DB2 Connect: this is a separate product that allows for access via a DRDA connection to a DRDA Application Server host. Examples of DRDA AS hosts include DB2 for z/OS, DB2 for OS/390, DB2 for MVS/ESA, and DB2 for iSeries.

Finally, the DB2 engine has the DRDA Application Server host functionality built into the product. This means that DRDA Application Requester products can access DB2 data with no additional products being required on the workstation. This means that tools, for example, on MVS, like QMF, could be used to query your DB2 database.

You may also want to mention here that you can perform joins between tables that exist in the DB2 products (Linux, UNIX, Windows, and mainframe databases) by defining the table as a nickname. A nickname refers to an object in another database. This is referred to as federated support. Cross database joins are supported through nicknames; the ability to define nicknames on IBM objects (including Informix) is provided in the DB2 product.

There is also an ability to join to non-DB2 objects (such as Oracle, Sybase, SQL Server); using Information Integrator.

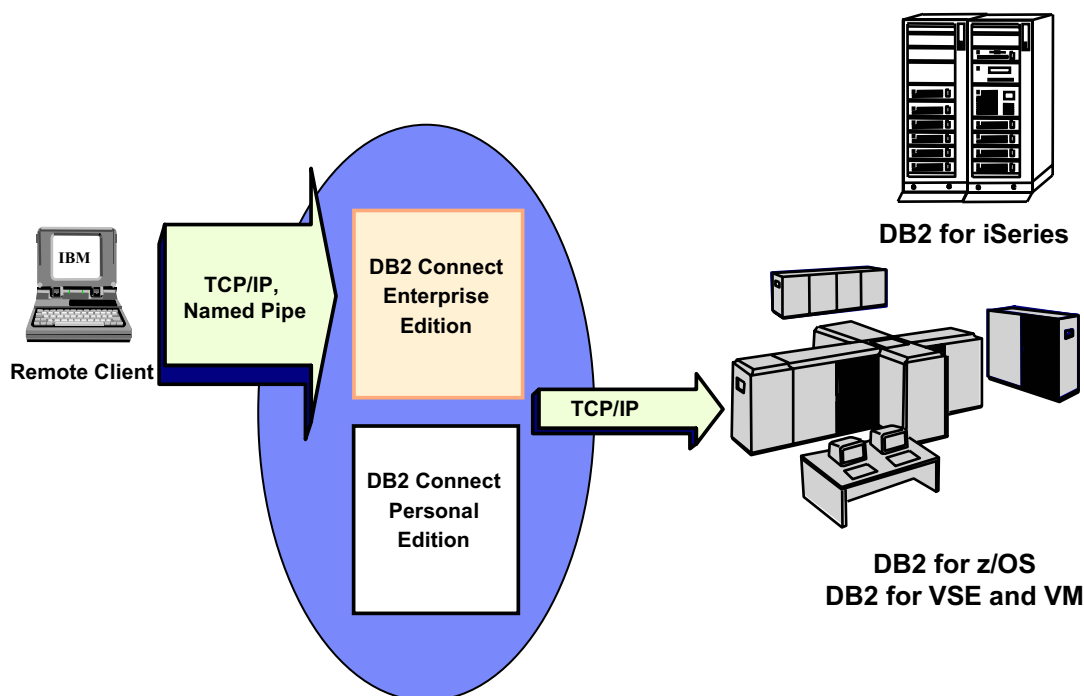
**Additional information** — Point out that Communication Support is only needed for remote client access, whether from the Runtime Client, DB2 Client, or DRDA Application Requester.

In the student notes, there is a statement: *DRDA (Distributed Relational Database Architecture) Application Requesters represent DRDA requests coming from, for example, z/OS, MVS, VM, and OS/400.* Really, all client requests are DRDA AR now. However, this statement remains so you can discuss here that access **from** a mainframe host application does not require DB2 Connect.

**Transition statement** — More information on accessing the host environment from the LAN environment.



## DRDA Application Requester (AR) support



© Copyright IBM Corporation 2007

Figure 1-6. DRDA Application Requester (AR) support

CF238.3

### Notes:

**DRDA Application Requester (AR) support is provided by DB2 Connect.** DB2 Connect is available either in DB2 Connect Personal Edition, DB2 Connect Enterprise Edition, or DB2 Connect Unlimited Edition.

DB2 Connect Personal Edition provides access from a single workstation to DB2 databases residing on servers such as OS/390, z/OS, OS/400, VM, and VSE, as well as to DB2 servers on Linux/UNIX and Windows operating systems. DB2 Connect Personal Edition provides the same rich set of APIs as DB2 Connect Enterprise Edition. This product is currently available for Linux and Windows operating systems.

DB2 Connect Personal Edition is used to connect a single Windows operating system, or Linux workstation, to a host or iSeries database. DB2 Connect Personal Edition is best suited for environments where native TCP/IP support is provided by the database servers, and the application being deployed is a traditional 2-tier client/server application.

DB2 Connect Enterprise Edition is a connectivity server that concentrates and manages connections from multiple desktop clients and web applications to DB2 database servers running on host or iSeries systems. IBM's DB2 for iSeries, DB2 for OS/390 and z/OS, and

DB2 for VSE and VM databases continue to be the systems of choice for managing most critical data for the world's largest organizations. While these host and iSeries databases manage the data, there is a great demand to integrate this data with applications running on Windows and Linux/UNIX workstations.

DB2 Connect Enterprise Edition enables local and remote client applications to create, update, control, and manage DB2 databases and host systems using Structured Query Language (SQL), DB2 APIs (Application Programming Interfaces), ODBC (Open Database Connectivity), JDBC (Java Database Connectivity), SQLJ (Embedded SQLJ for Java), or DB2 CLI (Call Level Interface). In addition, DB2 Connect supports Microsoft Windows data interfaces such as ActiveX Data Objects (ADO), Remote Data Objects (RDO), and Object Linking and Embedding (OLE) DB.

DB2 Connect Enterprise Edition is currently available for AIX, HP-UX, Linux, Solaris, and Windows operating systems. These servers provide support for applications running on Linux, UNIX (AIX, HP-UX, and Solaris operating environments), and Windows workstations.

**DB2 Connect Enterprise Edition is often installed on an intermediate server to connect DB2 clients to a host or iSeries database.** It can also be used on machines where multiple local users want to access the host or iSeries servers directly.

DB2 Connect Enterprise Edition is most appropriate for environments where:

- Web servers run Web-based applications.
- Web servers run Web-based application using data-aware Java applications.
- A middle-tier application server is used.
- TP monitors, such as CICS, Encina, Microsoft Transaction Server (MTS), Tuxedo, Component Broker, and MQSeries, are used.

DB2 Connect Unlimited Edition and DB2 Connect Application Server Edition are unique package offerings that allows complete flexibility of DB2 Connect deployment and simplifies product selection and licensing. This product contains both DB2 Connect Personal Edition and DB2 Connect Enterprise Edition with license terms and conditions that allow the unlimited deployment of any DB2 Connect product. License charges are based on the size of the S/390 or zSeries server that DB2 Connect users will be working with.

This package offering is only available for OS/390 and z/OS systems, and licensing is only valid for DB2 for OS/390 and z/OS data sources.

**Instructor notes:**

**Purpose** — Give more information on accessing host databases from the LAN environment.

**Details** — Access to host databases is provided via the DB2 Connect product. DB2 Connect comes in two alternatives: Personal Edition to allow a single workstation to access host databases, or Enterprise Edition to provide gateway services to host databases.

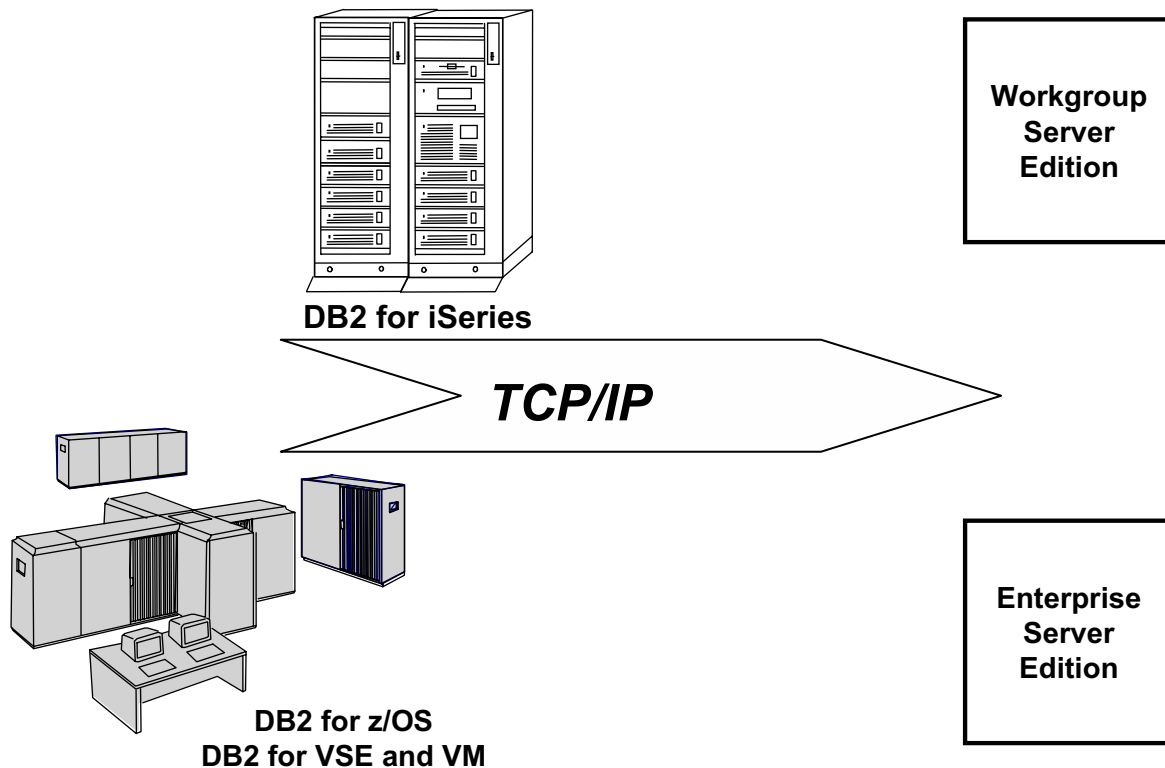
Access to the host is via TCP/IP.

**Additional information** — Two-phase commit is supported from the LAN environment against host databases over either TCP/IP.

*DB2 Connect Unlimited Edition* — As an alternative to user counting, IBM introduced an MSU model in V7 which allows unlimited user access, as long as the host machine is fully entitled for its processing capacity. An MSU entitlement must be obtained for each MSU on the DB2 host system. When this option is used, the customer need not count users of the DB2 Connect product. Only one server install is required for each customer utilizing the MSU licensing.

**Transition statement** — To get LAN database data from mainframe hosts, the DRDA AS capability is built in to the database engine.

# DRDA Application Server (AS) support



© Copyright IBM Corporation 2007

Figure 1-7. DRDA Application Server (AS) support

CF238.3

## Notes:

DRDA Application Server (AS) support is provided in Workgroup Server Edition and Enterprise Server Edition. This allows a host client access to data on a DB2 server.

DRDA two-phase commit over TCP/IP is supported on all platforms for the DRDA AS.

***Instructor notes:***

**Purpose** — Identify support of mainframe applications accessing databases on Linux/UNIX or Windows.

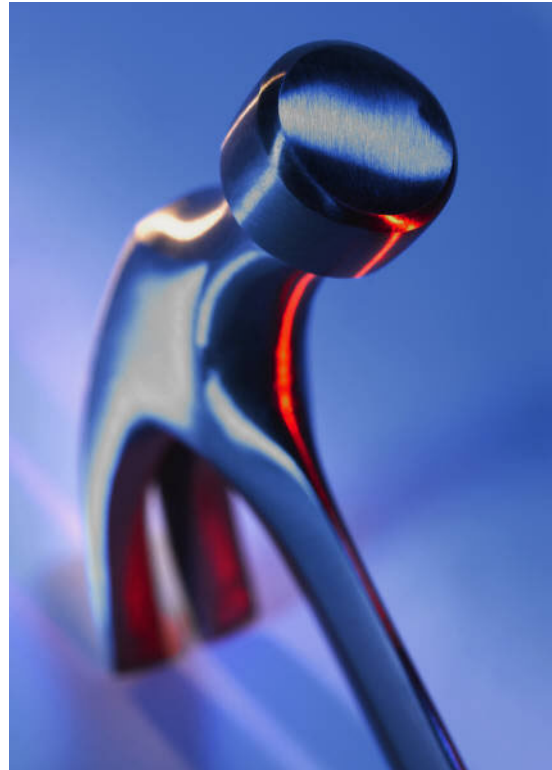
**Details** — This is looking at the situation where you have an application running on the mainframe, and the data now resides on a database on your Linux/UNIX or Windows machine. The ability to access data on the Linux/UNIX or Windows DB2 database is built into the DB2 server; you do not need DB2 Connect to provide this functionality.

**Additional information** —

**Transition statement** — Let us now have a look to some installation prerequisites.

## Installation prerequisites

- Disk space
- Enough memory
- Available Internet browser
- Security (Authorities) concept for SYSADM / Admin-Server
- Specific requirements depending on operating system



© Copyright IBM Corporation 2007

Figure 1-8. Installation prerequisites

CF238.3

### Notes:

The disk space required for your product depends on the type of installation you choose and the type of file system you have. The DB2 Setup wizard provides dynamic size estimates based on the components selected during a typical, compact, or custom installation.

At a minimum, a DB2 database system requires 256 MB of RAM. For a system running just DB2 and the DB2 GUI tools, a minimum of 512 MB of RAM is required. However, 1 GB of RAM is recommended for improved performance. These requirements do not include any additional memory requirements for other software that is running on your system.

One of the following browsers is required to view online help, run the DB2 install launchpad (setup.exe), and to run First Steps (db2fs): — Internet Explorer 5.5 and up — Mozilla 1.4 and up — Firefox 1.0 and up — Netscape 7.0 and up.

Security considerations are important to the DB2 administrator from the moment the product is installed. To complete the installation of the DB2 database manager, a user ID, a group name, and a password are required. The GUI-based DB2 database manager install program creates default values for different user IDs and the group. Different defaults are created, depending on whether you are installing on Linux/UNIX or Windows platforms.

- For the most up-to-date prerequisite information, see <http://www.ibm.com/software/data/db2/udb/sysreqs.html>.
- Getting Started Manuals

***Instructor notes:***

**Purpose** — Give the students an high level overview of the installation prerequisites.

**Details** — All the relevant details could be found on the indicated internet page or in the Getting started manuals.

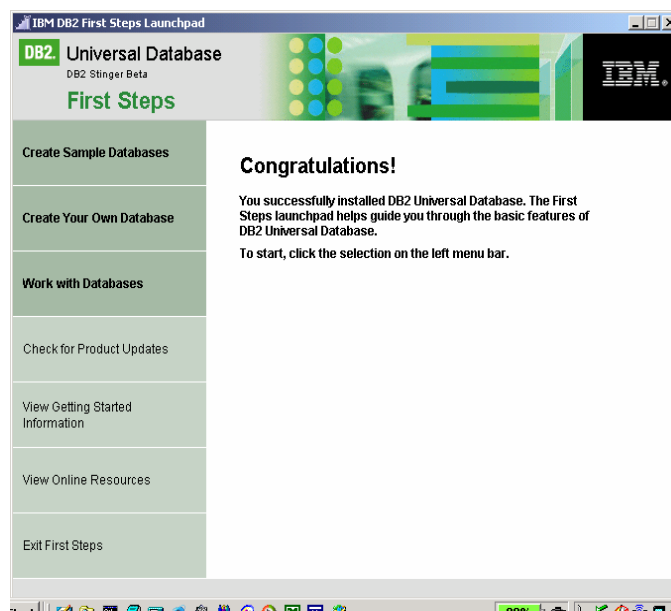
**Additional information** —

**Transition statement** — Let's now have an overview of the installation steps.



## Installation steps

- GUI or non-GUI
- Customizing?
- Default instance?
- Admin-Server?
- Notification?
- Installation verification



© Copyright IBM Corporation 2007

Figure 1-9. Installation steps

CF238.3

### Notes:

You can decide to run the installation with a command or using the GUI, where you will be guided step by step. You must have a local Administrator account (Windows) or root authority (Linux, UNIX) to successfully start the installation.

During both installation processes you can select typical or customized installation, whereby in the customized installation you would be able to select the components you want to install. Furthermore you need to decide during the installation if you want the installation process having create the DB2 default instance and the Administration server for you.

In an Windows environment the DB2 product will be installed, by default, in the `x:\Program Files\IBM\sqllib` directory, where x: represent the drive letter of the drive where you have installed your DB2 product.

If you are installing on a system where this directory is already being used, the DB2 product installation path will have `_xx` added to it, where `_xx` are digits, starting at 01 and increasing depending on how many DB2 copies you have installed. You can also specify your own DB2 product installation path.

For information on errors encountered during installation, review the installation log file located in the My Documents\DB2LOG\ directory. The log file uses the following format: *DB2-ProductAbbrev-DateTime.log*, for example, **DB2-ESE-Tue Apr 04 17\_04\_45 2006.log**.

In an Linux environment the DB2 product will be installed, by default, in the following directory: **/opt/ibm/db2/V9.1**. Also here the same naming rules applies for installation of several DB2 copies, as well as that you can specify your own installation path.

The installation logs, **db2setup.log** and **db2setup.err** will be located, by default, in the **/tmp** directory. You can specify the location of the log files.

After verification of the relevant installation logs, you might need to update the relevant registries.

After a successful installation, you can use the FIRST STEPS GUI to create for example the SAMPLE database.

***Instructor notes:***

**Purpose** — Quickly discuss with the students the installation and post-installation steps. Don't go into too much detail.

**Details** —

**Additional information** — Even if the students will not perform the installation during the LAB, they will do some of the customization steps necessary after the installation.

Do Certification objective: 1J Ability to obtain and modify registry variables.

**Transition statement** — Do you need more information? Let's see the opportunities to get it.

## For more information



- CF60 DB2 Connect DRDA Implementation
- CF63 DB2 Connect Problem Determination and Performance
- CF24 DB2 Multi-Partition Database Administration
- CG24 DB2 Multi-Partitioned Environment for Single Partition DBAs

© Copyright IBM Corporation 2007

Figure 1-10. For more information

CF238.3

### **Notes:**

You, or other people in your organization, may desire additional details. Please visit [www.ibm.com/services/learning](http://www.ibm.com/services/learning) for the most current information and schedules of these courses in your country.

***Instructor notes:***

**Purpose** — Identify additional courses to get more in depth education.

**Details** —

**Additional information** —

**Transition statement** —

## Unit summary

Having completed this unit, you should be able to:

- Contrast the DB2 family of products
- Identify the DB2 products
- Describe the functions of DB2 components
- Explore installation and parameters

© Copyright IBM Corporation 2007

Figure 1-11. Unit summary

CF238.3

### **Notes:**

***Instructor notes:*****Purpose —****Details —****Additional information —**

**Transition statement —** Now, let's look at your first lab. You will be starting up the VMware image of your operating environment.





## Unit 2. Command Line Processor (CLP) and GUI usage

### Estimated time

01:15

### What this unit is about

This unit explains the two methods of interacting with the DB2 database server system. Practice use of the CLP and GUI tools will be explored, using common database commands. The need for the DAS Administration Server will also be explained.

### What you should be able to do

After completing this unit, you should be able to:

- Use the Command Line Processor
- Explore the GUI environment
- Describe the DAS role with GUI tools

### How you will check your progress

Accountability:

- Machine exercises

## Unit objectives

After completing this unit, you should be able to:

- Use the Command Line Processor
- Explore the GUI environment
- Describe the DAS role with GUI tools

© Copyright IBM Corporation 2007

Figure 2-1. Unit objectives

CF238.3

### Notes:

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**



## 2.1 Command Line Processor (CLP)

### Instructor topic introduction

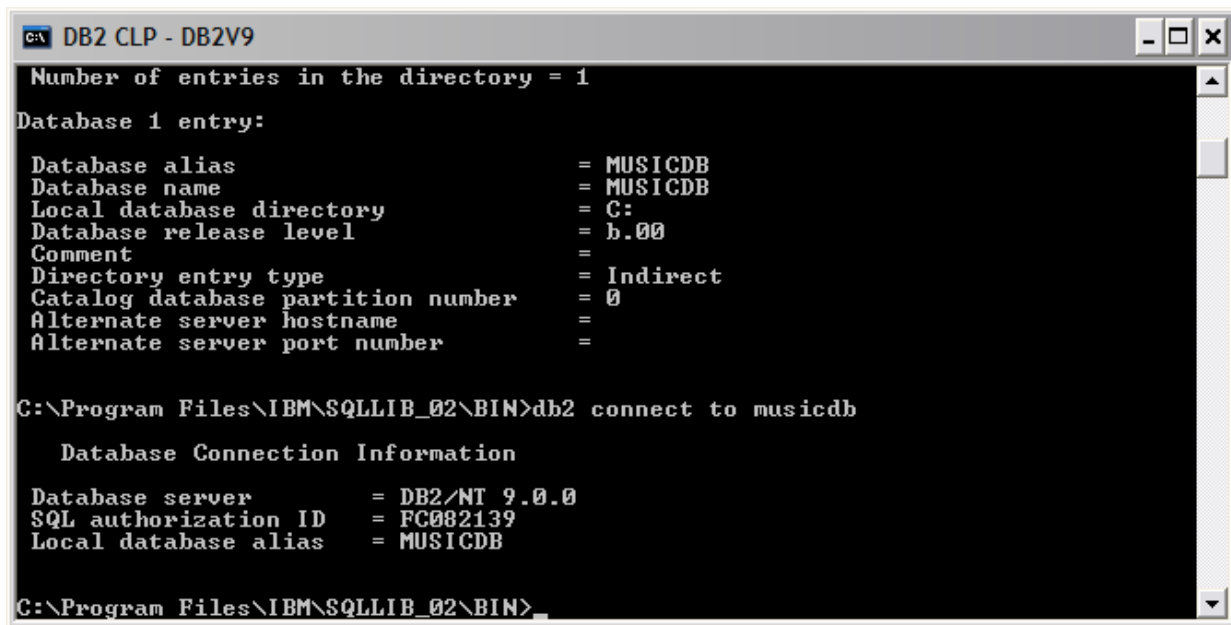
What students will do —

How students will do it —

What students will learn —

How this will help students on their job —

# CLP Command Line Processor



```
C:\ DB2 CLP - DB2V9
Number of entries in the directory = 1
Database 1 entry:
Database alias           = MUSICDB
Database name           = MUSICDB
Local database directory = C:
Database release level  = b.00
Comment                 =
Directory entry type     = Indirect
Catalog database partition number = 0
Alternate server hostname =
Alternate server port number =

C:\Program Files\IBM\SQLLIB_02\BIN>db2 connect to musicdb

Database Connection Information

Database server           = DB2/NT 9.0.0
SQL authorization ID      = FC082139
Local database alias      = MUSICDB

C:\Program Files\IBM\SQLLIB_02\BIN>
```

© Copyright IBM Corporation 2007

Figure 2-2. CLP Command Line Processor

CF238.3

## Notes:

Through the Command Line Processor you can issue:

- SQL statements
- XQuery statements (with the prefix XQUERY)
- DB2 commands
- OS system commands

In an Windows environment, the CLP can be found under the Command Line Processor as well as the *Command Window* and in an Linux/UNIX environment, it would be activated with the **db2profile**, so a *normal* terminal could be used.

***Instructor notes:***

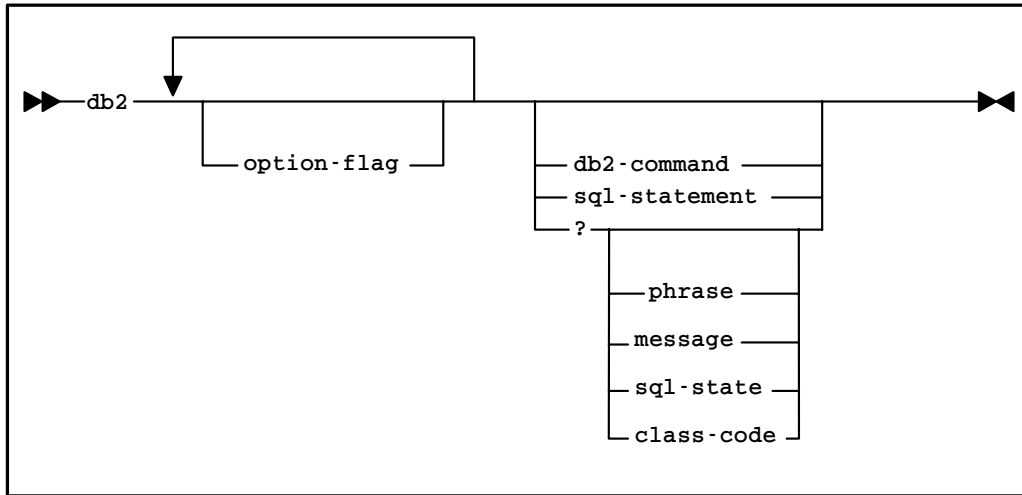
**Purpose** — Introduce the CLP to the students.

**Details** — Make the students aware of the two different scenarios WINDOWS and Linux/UNIX. Even if in Windows the CLP looks like a Command Prompt, we need the relevant run time environment. The **db2profile** is used in the Linux/UNIX environment.

**Additional information** —

**Transition statement** — Let's look at the CLP syntax.

# CLP syntax



© Copyright IBM Corporation 2007

Figure 2-3. CLP syntax

CF238.3

## Notes:

The options may be entered after the **db2** and before the command or statement. If the command or statement will not fit on one line, either:

- Continue typing and allow your typing to wrap and continue on the next line, or
- Use the \ (backslash) character for continuation.

The **class-code** option in the above syntax diagram means that you can request help for a message specified by a valid class-code.



***Instructor notes:***

**Purpose** — Introduce CLP syntax.

**Details** —

**Additional information** — Lab covers in detail — keep discussion short and tell students lab covers in detail.

**Transition statement** — Now let's examine the online reference possibility.

## Online reference



- Online Command Reference

```
db2 ?  
db2 ? command string  
db2 ? SQLnnnn    (nnnn = 4 or 5 digit SQLCODE)  
db2 ? nnnnn      (nnnnn = 5 digit SQLSTATE)
```

- Online Reference Manuals

© Copyright IBM Corporation 2007

Figure 2-4. Online reference

CF238.3

### Notes:

The Online Command Reference contains the syntax and explanations of all DB2 commands that may be executed through CLP. The Online Command Reference is invoked by:

1. **db2 ?** — List of all DB2 commands
2. **db2 ? command** — Information about specific commands
3. **db2 ? SQLnnnn** — Information about a specific SQLCODE generated by database manager. SQLCODE must be four or five digits in length.
4. **db2 ? nnnnn** — Information about a specific SQLSTATE generated by database manager. SQLSTATE must be five digits in length.

A set of reference manuals are also available online. For information on installation and setting up online reference manuals, consult your *Installing and Configuring DB2 Clients* manual.

***Instructor notes:***

**Purpose** — Give the students an short overview of the online reference.

**Details** — See student notes.

**Additional information** —

**Transition statement** — Now let us look to the two ways you can use the CLP.

## Using the CLP

### Non-interactive mode

db2 connect to musicdb  
 db2 "select \* from syscat.tables" | more  
 ↗ (double quotes may be required)

### Interactive mode

db2  
 db2=> connect to musicdb  
 db2=> select \* from syscat.tables

© Copyright IBM Corporation 2007

Figure 2-5. Using the CLP

CF238.3

### Notes:

Prefix all CLP commands or requests with **db2**, or use CLP in interactive mode by typing **db2** and then pressing Enter. In the interactive mode, the user can type CLP commands without prefixing them by db2.

**When using CLP without being in interactive mode on a Linux/UNIX platform, remember to put quotes around the statement or command if special characters are included.**

To issue XQuery statements in CLP, prefix the statements with the XQUERY keyword.

Interactive mode does not allow you to do piping or other operating system functions at the interactive CLP command level. To execute operating system commands without quitting interactive mode, issue **!<operating-system-command>**.

In the interactive mode, the history of commands is displayed by entering **history** (or shortened form **h**). Two variations are allowed:

- **reverse (r)** list in reverse order, in other words most recent first
- **num** (such as 5) limits the list to the last *num* history entries

Examples, assuming that you first entered `db2start`, then a “list active databases” command, and then you entered `? sql11111`:

```
db2 => h
      1 db2start
      2 list active databases
      3 ? sql11111
      4 h
db2 => h r
      5 h r
      4 h
      3 ? sql11111
      2 list active databases
      1 db2start
db2 => h r 3
      6 h r 3
      5 h r
      4 h
```

There is also an `edit (e)` command which enables editing of a previous command. You can edit the command using the number in the history list, for example `e 2`. If no number is entered, the last command would be edited.

After having edited the command and closed the editor, the edited command is displayed in the CLP window, and you will be asked:

```
Do you want to execute the above command ? (y/n)
```

If you enter `y`, the command as edited is executed.

With `runcmd (r)` you can re-run a previously executed command. You can do this using the number from the history list, for example `r 3`. If no number is entered, the last command is re-executed.

If a command that you are entering exceeds the limit allowed at the command prompt, use a `\` (backslash) as the line continuation character. If you want a blank space between the last character on the current line and the first character on the next line, don't forget to code a blank space before the backslash. You may wish to use the CLP flag `-t` instead of the `\` (backslash). It says that the CLP statement terminates with a `;` (semicolon).

In its output, CLP represents a SQL NULL value as a hyphen (-). If the column is numeric, the hyphen is placed at the right of the column. If the column is not numeric, the hyphen is at the left.

### ***Instructor notes:***

**Purpose** — Introduce the two different ways to use the CLP.

**Details** — Maybe you want to point out, that in the interactive mode OS commands can not be used because of the prefix **db2 =>**.

**Additional information** —

**Transition statement** — Now let us examine the differences between QUIT, TERMINATE and CONNECT RESET.

## QUIT/TERMINATE/CONNECT RESET differences

| <i>CLP<br/>COMMAND</i> | <i>Terminate CLP<br/>Back-end Process</i> | <i>Disconnect<br/>database Connection</i> |
|------------------------|-------------------------------------------|-------------------------------------------|
| quit                   | No                                        | No                                        |
| terminate              | Yes                                       | Yes                                       |
| connect reset          | No                                        | Yes if<br>CONNECT=1<br>(RUOW)             |

© Copyright IBM Corporation 2007

Figure 2-6. QUIT/TERMINATE/CONNECT RESET differences

CF238.3

### Notes:

To connect to a local or remote database, specify *db2 connect to dbname* where dbname maps to the alias name specified in the System Database Directory. After the *connect* is issued, all SQL requests are executed against the database to which you are connected.

If **CONNECT=1 (RUOW)**, **db2 connect reset** terminates the connection, and a subsequent SQL statement will cause connection to the default database, if it is defined. **The default database is defined in the environment variable DB2DBDFT.** If **CONNECT=2 (DUOW)**, **db2 connect reset** puts the current connection in a dormant state and establishes a connection with the default database if it is defined. DUOW and CONNECT types will be defined in detail in the Distributed Management Topic.

**db2 terminate** issues a disconnect and also terminates the CLP back-end process.

The **quit** command ends the input mode and returns the user to the command prompt, but quit does not terminate CLP nor disconnect the database connection.

To end the CLP background process and disconnect the database connection issue the terminate command.

***Instructor notes:***

**Purpose** — Point out the differences.

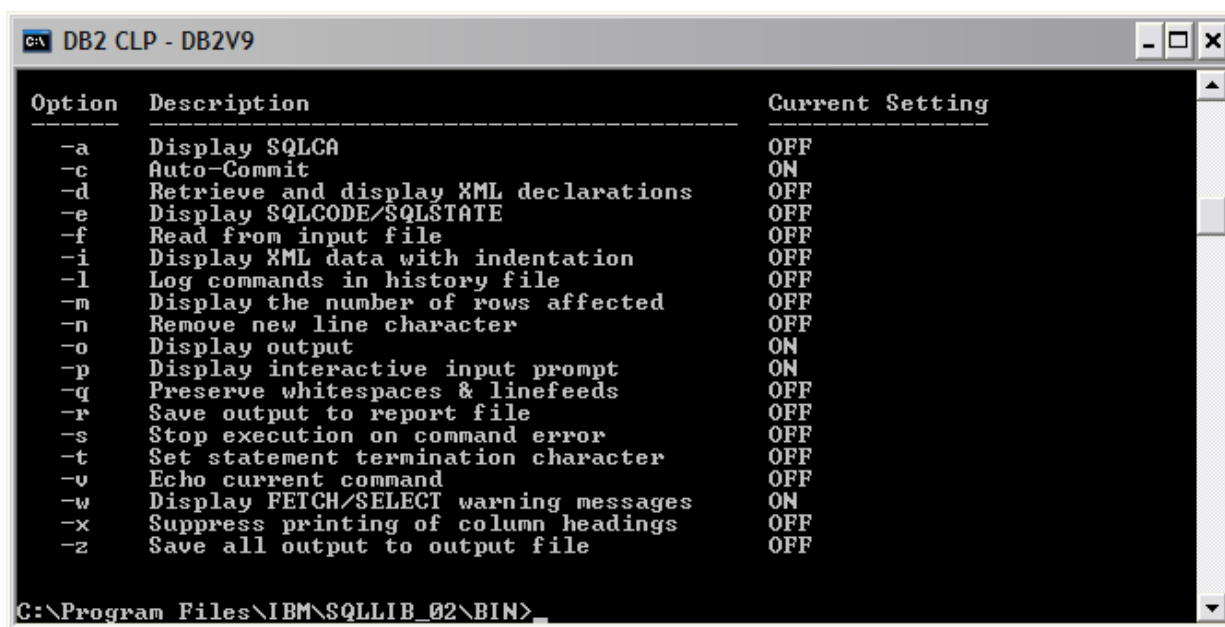
**Details** — See student notes.

**Additional information** —

**Transition statement** — Now let's examine the different Command Line options.



## CLP command options



| Option | Description                           | Current Setting |
|--------|---------------------------------------|-----------------|
| -a     | Display SQLCA                         | OFF             |
| -c     | Auto-Commit                           | ON              |
| -d     | Retrieve and display XML declarations | OFF             |
| -e     | Display SQLCODE/SQLSTATE              | OFF             |
| -f     | Read from input file                  | OFF             |
| -i     | Display XML data with indentation     | OFF             |
| -l     | Log commands in history file          | OFF             |
| -m     | Display the number of rows affected   | OFF             |
| -n     | Remove new line character             | OFF             |
| -o     | Display output                        | ON              |
| -p     | Display interactive input prompt      | ON              |
| -q     | Preserve whitespaces & linefeeds      | OFF             |
| -r     | Save output to report file            | OFF             |
| -s     | Stop execution on command error       | OFF             |
| -t     | Set statement termination character   | OFF             |
| -v     | Echo current command                  | OFF             |
| -w     | Display FETCH/SELECT warning messages | ON              |
| -x     | Suppress printing of column headings  | OFF             |
| -z     | Save all output to output file        | OFF             |

C:\Program Files\IBM\SQLLIB\_02\BIN>

`db2 list command options`

© Copyright IBM Corporation 2007

Figure 2-7. CLP command options

CF238.3

### Notes:

Issue the `db2 list command options` command to view the current settings for the command line flags and the value of DB2OPTIONS.

The following shows the option flag, description, and default setting.

- **a** — Displays SQLCA data — OFF
- **c** — Autocommit SQL statements — ON
- **e{c | s}** — Display SQLCODE or SQLSTATE — OFF
- **f filename** — Read command input from a file instead of standard input — OFF  
You may wish to issue `db2 < filename` instead of `db2 -f filename`.
- **i** — Display XML data with indentation — OFF
- **l filename** — Log commands in a history file — OFF
- **m** — Display the number of rows affected — OFF
- **n** — Remove new line character — OFF

- **o** — Display output data and messages to standard output — ON
- **q** — Preserver wildspaces and linfeeds — OFF
- **p** — Display CLP prompt when in interactive mode — ON
- **r filename** — Write output to a file — OFF
- **s** — Stop execution if errors occur while executing commands in a batch file or in interactive mode — OFF
- **t** — Set the statement termination symbol — OFF
- **v** — Echo command text to standard output — OFF
- **w** — Display FETCH/SELECT warning messages — ON
- **z filename** — Save all output to output file — OFF

CLP consists of two processes, the front-end process and the back-end process, which are required to maintain the database connection between each CLP invocation. These two processes communicate via three message queues: a request queue, an input queue and an output queue.

**DB2BQTIME**, **DB2BQTRY**, **DB2RQTIME**, and **DB2IQTIME** offer a means of configuring communication between the two processes. When CLP is invoked, the front-end process checks to see if the back-end process is already active. If it is, the front-end process reestablishes a connection with it. If not, the front-end process sleeps for the duration of time specified by the **DB2BQTIME** variable and then checks again. The front-end will perform this checking for the number of times specified by the **DB2BQTRY** variable. If the back-end process is still not active, it is activated.

Once the back-end process has been started, it waits on its request queue for a request from the front-end. It also waits on the request queue in between requests initiated from the command prompt. The **DB2RQTIME** variable specifies the length of time the back-end process waits for a request from the front-end process.

When the back-end process receives a request from the front-end process, it sends an acknowledgement to the front-end process indicating that it is ready to receive input via the input queue. The back-end process then waits on its input queue. The back-end process also waits on the input queue while a batch file is executing and while the user is in interactive mode. The **DB2IQTIME** variable specifies the length of time the back-end process waits on the input queue for the front-end process to pass commands.

Users can terminate the back-end process explicitly by issuing the **TERMINATE** command.

***Instructor notes:***

**Purpose** — List the different options.

**Details** — Explain the students the most important options (like **-tvf** and so forth).

**Additional information** —

**Transition statement** — Now let's examine how the options can be changed.

# Modify CLP options

- 1 **Temporary for Command**

```
db2 -r options.rep list command options
db2 -svtf create.tab3
db2 +c "update tab3 set salary=salary + 100"
```
- 2 **Temporary for Interactive CLP Session**

```
db2=>update command options using c off a on
```
- 3 **Temporary for non-interactive CLP Session**

```
export DB2OPTIONS="-svt" (UNIX)
set DB2OPTIONS="-svt" (Windows)
db2 -f create.tab3
```
- 4 **Every session**

put point 3 in UNIX **db2profile**  
or System Program Group in Windows

© Copyright IBM Corporation 2007

Figure 2-8. Modify CLP options

CF238.3

## Notes:

The CLP options can be used in any sequence and combination. To turn the option on, prefix the option with a minus sign (-). To turn an option off, prefix the option with a minus sign (-) and follow the option letter with another minus sign (-) or prefix the option with a plus sign (+). For example, either use -c- or +c.

These options can also be specified by setting the DB2OPTIONS environment variable. ((Windows) set DB2OPTIONS='+c -a', (Linux/UNIX) export DB2OPTIONS='+c -a').

CLP option flags temporarily override DB2OPTIONS settings.

**db2 update command options** command allows the user to change an option setting from the interactive input mode or a command file.

### db2 update command options using c off

Another way to redirect the CLP output from the screen to a file instead of using the **-r option** is to use the **>** symbol.

Some operating systems use the > or < symbols in SQL statements, and do not want them interpreted as redirection symbols by CLP. The quotation marks should surround all text to be processed by the CLP, but does not include any options.

- **db2 -r sample.rep "select \* from org"**
- **db2 "select \* from org where deptnumb > 38" > sample.rep**

This can be avoided by using a DB2 interactive CLP session.

***Instructor notes:***

**Purpose** — Explain how the options can be changed in different ways.

**Details** — See student notes.

**Additional information** —

**Transition statement** — Now let us examine an example of an input file without OS commands.

## Input file — no operating system commands



Edit create.tab

```
-- comment:  db2 -svtf create.tab

connect to sample;

create table tab3
  (name varchar(20) not null,
   phone char(40),
   salary dec(7,2));

select * from tab3;

commit work;

connect reset;
```

db2 -svtf create.tab

© Copyright IBM Corporation 2007

Figure 2-9. Input file — no operating system commands

CF238.3

### Notes:

Use an editor to create a file called **create.tab**.

Comments are denoted with a line that starts with two hyphens (--).

A semicolon may be used to denote the end of a SQL statement if the file is executed with the **-t** command option.

In non-interactive mode, execute the file with **db2 -svtf create.tab**. Since *db2* was not coded in the input file, only DB2 commands may be coded in the input file. In non-interactive mode, the command to execute the commands in the input file must be started with *db2*. The **-s** option says to stop execution if an error occurs. The **-v** option says to echo the current command on the monitor screen. The **-t** option says the statements end with a *semicolon*. The **-f** option says the command input is read from an input file called **create.tab**.

***Instructor notes:***

**Purpose** — Go with the students quick through this example.

**Details** — See student notes.

**Additional information** —

**Transition statement** — Next we will examine an example with OS commands.



# Input file — operating system commands

vi seltab

edit seltab.cmd

echo "Table Name Is" \$1 > out.sel  
db2 "select \* from \$1" >> out.sel

echo 'Table Name Is' %1 > out.sel  
db2 Select \* from %1 >> out.sel

seltab org

out.sel  
contents

Table Name Is org

| DEPTNUMB | DEPTNAME       | MANAGER | DIVISION  | LOCATION      |
|----------|----------------|---------|-----------|---------------|
| 10       | Head Office    | 160     | Corporate | New York      |
| 15       | New England    | 50      | Eastern   | Boston        |
| 20       | Mid Atlantic   | 10      | Eastern   | Washington    |
| 38       | South Atlantic | 30      | Eastern   | Atlanta       |
| 42       | Great Lakes    | 100     | Midwest   | Chicago       |
| 51       | Plains         | 140     | Midwest   | Dallas        |
| 66       | Pacific        | 270     | Western   | San Francisco |
| 84       | Mountain       | 290     | Western   | Denver        |

© Copyright IBM Corporation 2007

Figure 2-10. Input file — operating system commands

CF238.3

## Notes:

When including DB2 commands or SQL statements in an input file with operating system commands, place **db2** before the command or SQL statement and enclose the command or statement in **double quotation marks**.

**db2 -r out.sel "select \* from \$1"** may also be used to redirect the output to the file **out.sel**. The output will still echo on the screen unless the **+o** option is used. The **-r** option does not append output to the file.

In Linux/UNIX, the user must have execute authority to execute the CLP input file. The following steps may be executed to do so:

```
$chmod 744 sel.tab
$ls -l sel.tab
```

```
-rwxr--r--  1  inst31  adm31   58   Jul  15  13:53  sel.tab
```

***Instructor notes:***

**Purpose** — Go with the students through this example.

**Details** — See student notes.

**Additional information** —

**Transition statement** — Now let's start with the DB2 GUIs.

## 2.2 Getting started with DB2 GUI tools

### Instructor topic introduction

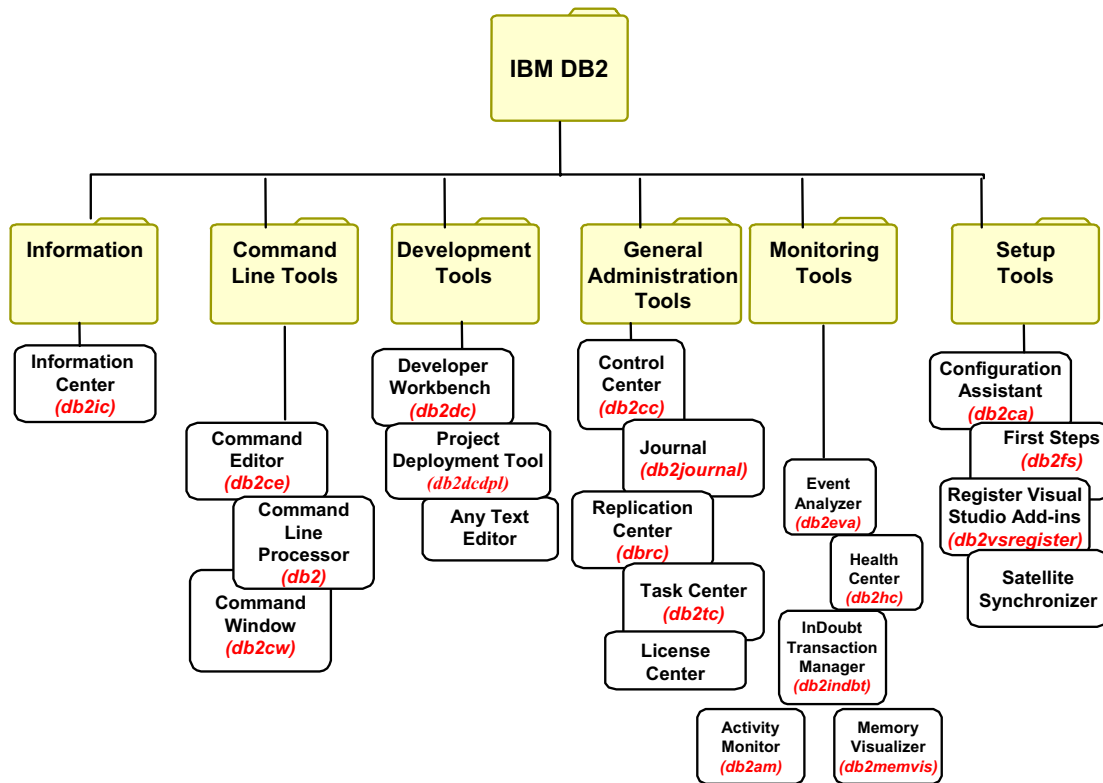
**What students will do** — The graphical tools can be very useful when performing a task that you don't do very frequently, particularly when the wizards provide assistance. But, the command line interface may be of more use when more commands need to be issued together: putting the commands into a script, to be issued from the command line, or from the Command Editor or Task Center.

**How students will do it** — Listen to the lecture and try the exercises.

**What students will learn** — The roadmaps and how the tools can be invoked.

**How this will help students on their job** — Many task can be done more easy with the appropriate GUIs.

# Roadmap to the GUI tools



© Copyright IBM Corporation 2007

Figure 2-11. Roadmap to the GUI tools

CF238.3

## Notes:

The graphic shows an overall view, or roadmap, for accessing the graphical tools.

There are some tools that can be started from a command prompt, in addition to starting them from the GUI interface. These are shown on the graphic, if they exist, in italics and red color. For example, by entering the command `db2cmd` from a command prompt, you can start the DB2 Command Line Processor. The Configuration Assistant (CA) can be started using the command `db2ca`, as well as from the DB2 Desktop folder.

You can access many of the other graphical tools from within the Control Center. Most of the graphical tools have a menu bar option, Tools, and an icon bar, that will allow you to easily access any of the other graphical tools.

**Instructor notes:**

**Purpose** — Give an overview of the graphical tools and their interfaces.

**Details** — Point out that the tools are available from the DB2 Desktop folder. They will be using the Control Center extensively during class. We will be looking at the tools under the Control Center with more emphasis on following pages.

Note that if you open any of the tools that can be opened from the Control Center, they will also be able to open the other tools that could have been opened from the Control Center

**Additional information** — At the top of the roadmap is the DB2 Desktop folder, which contains:

- Information folder
- Command Line Tools folder
- Development Tools folder
- General Administration Tools folder
- Monitoring Tools folder
- Setup Tools folder

Certification objective: 3E Ability to use the GUI Tools for administration.

**Transition statement** — First let's look at the Control Center GUI.

# Control Center

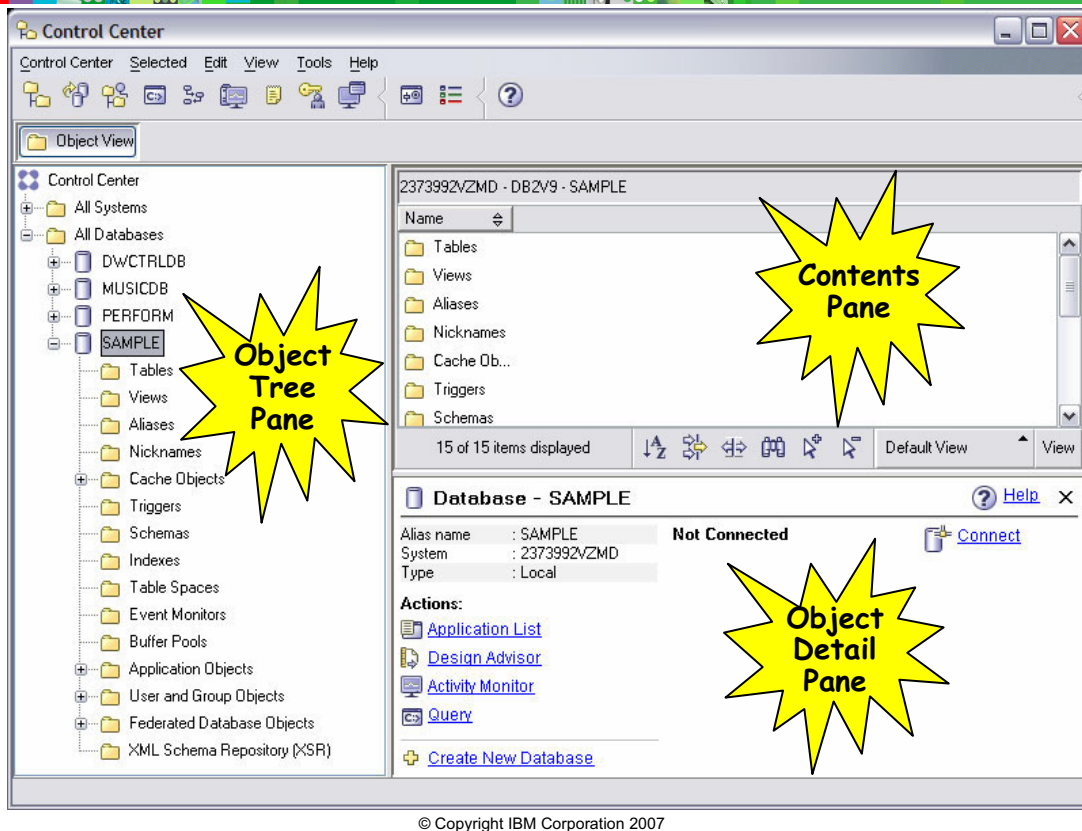


Figure 2-12. Control Center

CF238.3

## Notes:

The Control Center is the central point of administration for DB2. There are three views that you can choose from:

- **Basic View:** Is a simplified view of DB2 core functionality, and omits features that are generally used by the DB2 system administrator but not by the DBA. This includes essential objects such as databases, tables, views, triggers, and stored procedures.
- **Advanced View:** Is the default and lists the standard set of objects.
- **Custom View:** Here you can select those features which you wish to see.

The Control Center is divided into three areas which have contents depending on your chosen view:

- **Object tree Pane**
- **Contents Pane**, showing the contents of the selected object.
- **Object Detail Pane**, which provides context-sensitive access to the main features of the object focused and selected in the other panes, like, for example, the Backup Database tool, the Storage Management tool, or Automatic Maintenance.

Use the Control Center to manage systems (different OSs), DB2 instances, DB2 for OS/390 and z/OS subsystems, databases, and database objects such as tables and views.

In general: The most often used parameters and options are included in UIF panels. If a special parameter or option needs to be used, use the command line interface instead.

In the Control Center, you can administer all of your systems, databases, and database objects. From the Control Center, you can also open other centers and tools to help you optimize queries, jobs, and scripts; perform data warehousing tasks; create stored procedures or UDFs; and work with DB2 commands.

The following are some of the key tasks that you can perform with the Control Center:

- Add DB2 systems, federated systems, DB2 for z/OS and OS/390 systems, instances, databases, and database objects to the object tree.
- Manage database objects. You can create, alter, and drop databases, table spaces, tables, views, indexes, triggers, and schemas. You can also manage users.
- Manage data. You can load, import, export, and reorganize data. You can also gather statistics.
- Perform preventive maintenance by backing up and restoring databases or table spaces.
- Monitor performance and troubleshoot problems.
- Configure and tune instances and databases.
- Manage database connections, such as DB2 Connect servers and subsystems.
- Manage applications.
- Analyze queries using Visual Explain to look at access plans.
- Launch other tools such as the Command Editor and the Health Center.
- Automatic maintenance of your databases.

In many cases, wizards and launchpads are available to help you perform these tasks more quickly and easily. In many cases, the wizard will launch automatically when you select a task to be performed on an object; for example, the Create action on the Tables object will launch the Create Table wizard.

Other components of the Control Center are the following:

- Menu Bar — It is used to access Control Center functions and online help using menus.
- Tool Bar — These icons are used to access all the other administration tools.
- Hover Help — Provides a short description for each icon on the toolbars.

The Systems object represents machines. You can see all of the systems that your system has catalogued by looking at the list in the Control Center. The local workstation is represented by a System icon labeled with the machine name of the local workstation; other remote systems defined are labeled as they are cataloged.

To display the instances on each system, expand the object tree by clicking the plus sign (+) on the local or remote system name.

## ***Instructor notes:***

**Purpose** — Give an overview of the Control Center.

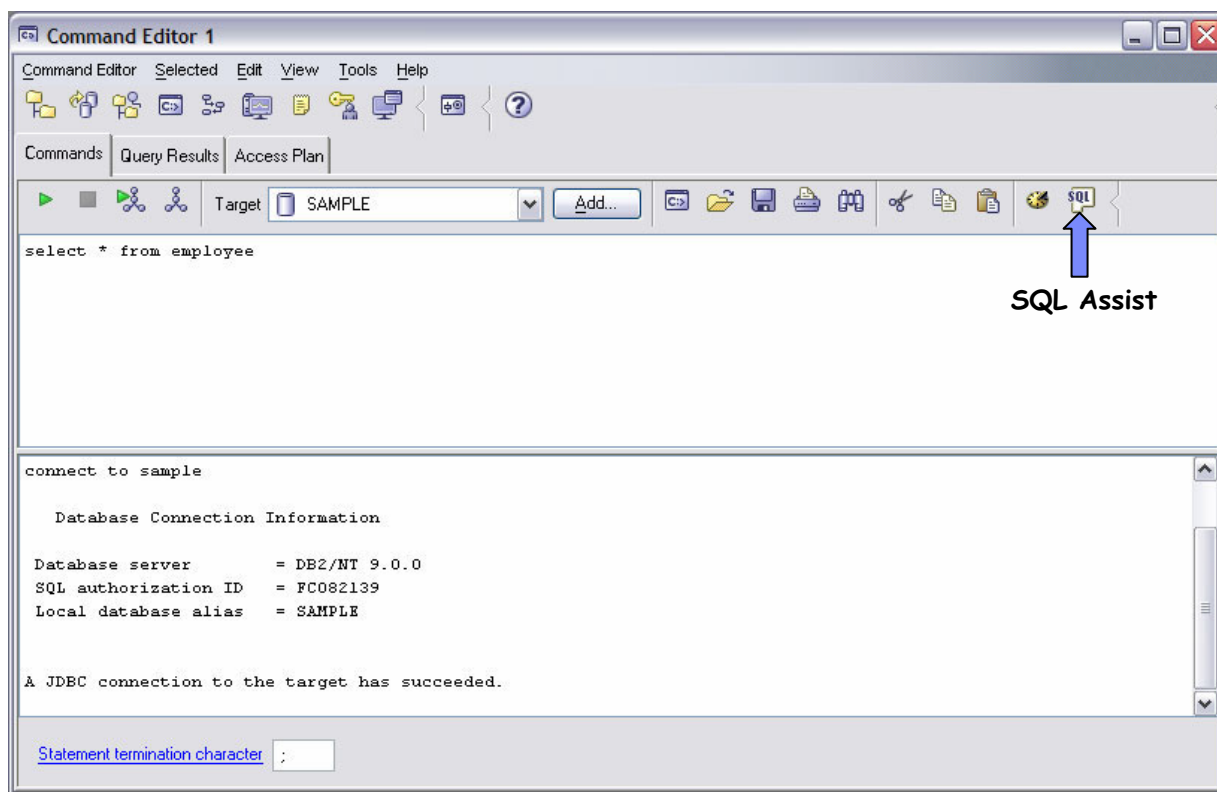
**Details** — Identify the different components of the Control Center. Point out the Objects Pane, Contents Pane, and Detail Pane, as well as the fact that the contents of the Detail Pane is dependent on the selected objects in the Objects Pane. Identify that what you can do depends on the object you have selected; for example, if you have the database selected, you could do performance monitoring or configure the database. If you have a table selected, you could reorganize the table or run statistics on it.

**Additional information** — Certification objective: 5H Ability to use the DB2 Control Center.

**Transition statement** — Next let us have a look at the Command Editor.



# Command Editor



© Copyright IBM Corporation 2007

Figure 2-13. Command Editor

CF238.3

## Notes:

Use the Command Editor to execute DB2 commands and SQL/XQUERY statements; to execute MVS, OS/390, or z/OS commands; to work with command scripts; and to view a graphical representation of the access plan for explained SQL statements.

On the **command** page, you can perform the following actions:

- Execute an SQL statement or DB2 CLP command. You do not need to precede the command by *DB2*.
- Run an existing script, by clicking the Execute icon (at the left of the tool bar).
- Execute the commands in sequence.
- Create and save a script. You can optionally store a saved script in the Task Center, where you can schedule the script to run at a specific time.
- Run and/or modify an existing script.

On the **Query Results** page, you can see the results of the queries. You can also save the results or edit the contents of the table.

On the **Access Plan** page, you can see the access plan for any explainable statement that you specified on the Interactive page or the Script page. DB2 generates the access plan when it compiles the SQL or XQUERY statement. You can use this information to tune your queries for better performance.

To invoke the SQL Assist tool, click the **SQL Assist** button on the Interactive page. To invoke the Visual Explain tool, execute an explainable statement on the Interactive page or the Script page.

The Web version of the Command Editor includes most of these features but it *does not* currently include Visual Explain or SQL Assist.

Both the GUI and traditional CLP are installed and shown as icons in the DB2 folder. For Windows, there is also a Command Window to enable users to execute SQL statements and DB2 commands from an MS-DOS command prompt.

The example commands shown in the Command Editor assume that a semi-colon (;) has been set in the Tools (can be specified at the bottom of the command editor window) as the statement termination character.

***Instructor notes:***

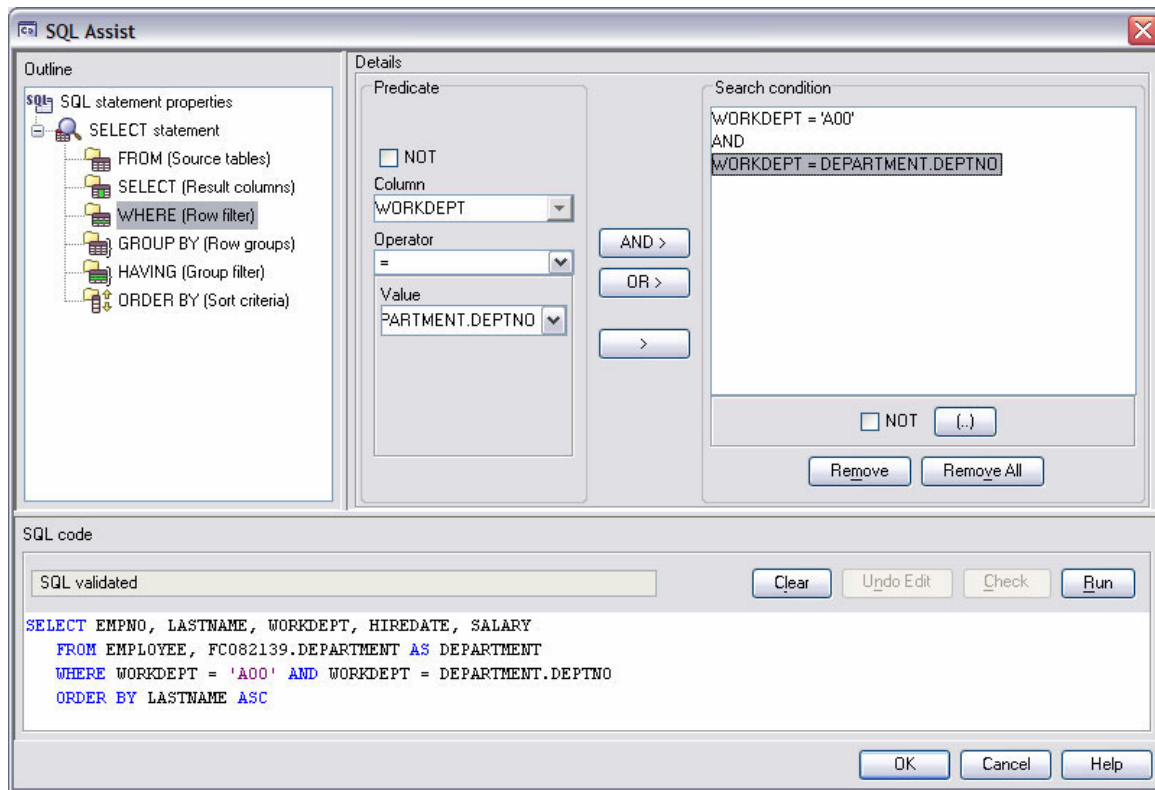
**Purpose** — Identify the functionality of the Command Editor.

**Details** — The Command Editor provides a GUI interface to issuing SQL/XQUERY statements and DB2 commands. The Interactive tab may be used when a single statement or command is being entered. The Script tab must be used if more than one statement or command is being entered. The Results area of the window allows you to see the results of executing those statements or commands.

**Additional information** —

**Transition statement** — Let's look at the SQL Assist window.

## SQL Assist window



© Copyright IBM Corporation 2007

Figure 2-14. SQL Assist window

CF238.3

### Notes:

Use SQL Assist to create SQL statements. With SQL Assist and some knowledge of SQL, you can create SQL SELECT statements. In some environments, you can also use SQL Assist to create INSERT, UPDATE, or DELETE statements.

The SQL Assist main window comprises three areas:

- Outline view
- Details view
- SQL code view

The Outline view contains a high-level representation of the current SQL statement. With the Outline view, you can visually examine an outline of the SQL statement and navigate through the steps of building the SQL statement. Select a node in the Outline view to view details for this element in the Details area.

Use the Details area to add elements to the SQL statement. The Details area changes based on what node you select in the Outline view. When you select a node in the Outline

view and make changes in the Details area, the SQL code is generated in the SQL code view.

The SQL code view contains the SQL code that is generated based on the contents of the Outline view and changes that you made in the Details area. The code is syntax-highlighted. In some environments, you can edit the SQL code in the SQL code view.

Refer to the Online Help for more information.

***Instructor notes:***

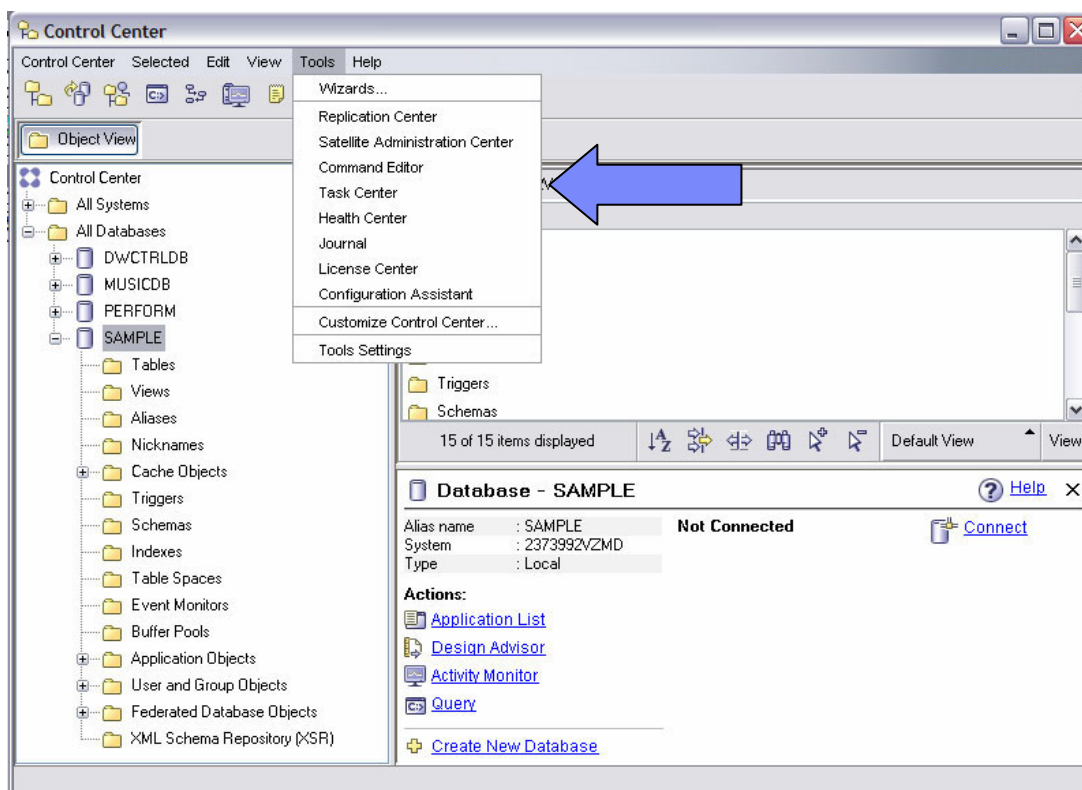
**Purpose** — Introduce the SQL Assist window.

**Details** — This shows the graphical tool. The statement will be built in the area at the bottom of the window. This might help not so experienced users to build SQL statements.

**Additional information** —

**Transition statement** — Next let's examine the other Administration tools.

# Administration tools



© Copyright IBM Corporation 2007

Figure 2-15. Administration tools

CF238.3

## Notes:

From the DB2 Desktop folder, you can access the Control Center. From the Control Center, there is a set of database server administration tools, designed to help a database administrator manage and administer DB2 databases.

The administration tools consist of the following:

- Wizards that helps you perform tasks like backups, creation of tables or tablespaces, and so on.
- Control Center — The primary point of control for the administrator.
- Replication Center — To administer replication between a DB2 database and another relational database.
- Satellite Administration Center — To set up and administer a group of DB2 servers that perform the same business function.
- Command Editor — The same GUI command and statement processor available from the DB2 Desktop Folder.
- Task Center — To run tasks, either immediately or according to a schedule, and to notify people about the status of completed tasks.

- Health Center — To monitor the state of the database environment and make any necessary changes.
- Journal — Displays historical information about tasks, database actions and operations, Control Center actions, messages, and alerts.
- License Center — To display license status and usage information for DB2 products that are installed on the system.
- Development Center — To develop stored procedures, user-defined functions (UDFs), and structured types.
- Tools Settings — To modify the interface with the GUI tools.
- Configuration Assistant
- Customize Control Center

The administration tools are part of the Administration Client installation.

If installed on a remote client, the administration tools can be used from the remote machine to configure a dedicated database administrator's system. This allows remote administration of DB2 databases on all DB2 server platforms.

Some of these administration tools are explained in detail in the next several pages.

All dialogs that run from the Control Center that generate an SQL statement or a command have a button (option) to SHOW SQL or SHOW COMMAND. A displayed statement or command may be saved as a script.



***Instructor notes:***

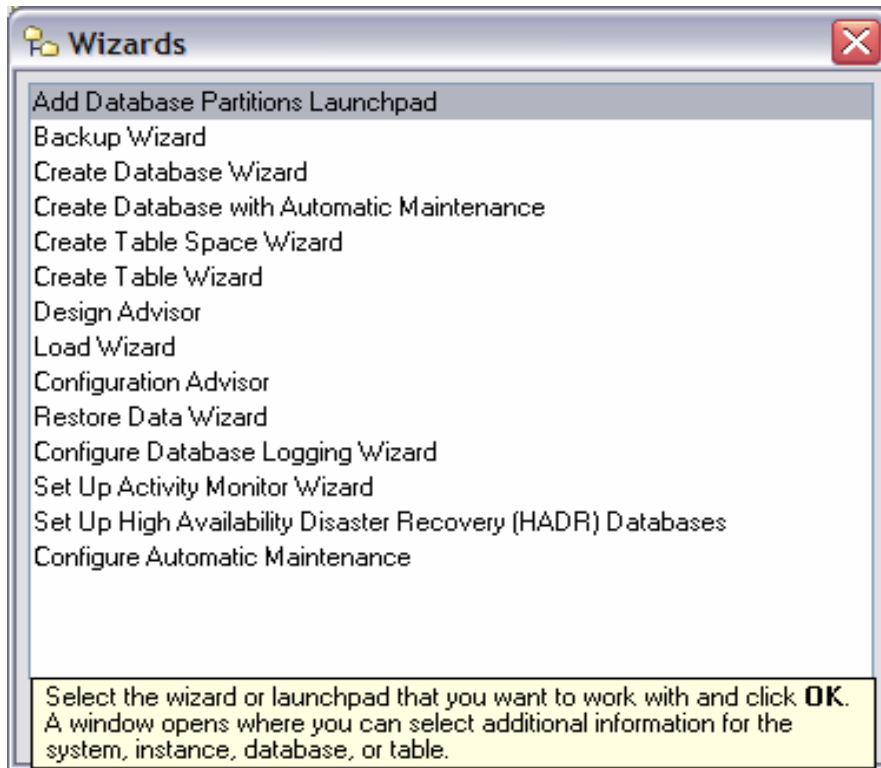
**Purpose** — Introduce the administration tools.

**Details** — The administration tools are available via icons or via the Tools menu bar option from the Control Center. We will look at some tools in more detail on the following pages.

**Additional information** —

**Transition statement** — Let us now examine the available wizards.

## Using the wizards



© Copyright IBM Corporation 2007

Figure 2-16. Using the wizards

CF238.3

### Notes:

*Wizards*, *Advisors*, and *Launchpads* are integrated into the administration tools.

*Wizards* assist you in completing a single task by stepping you through the task. The wizard task overview on the first page of the wizard lists any prerequisite steps and briefly describes every page of the wizard. Other pages of the wizard may contain links to conceptual or reference information to help you understand the function of the wizard.

*Advisors* assist you with more complex tasks, such as tuning and performance tasks, by gathering information and recommending options that you might not consider. You can accept or reject the advice of the advisor. Advisors can be called from the GUI as well as from APIs and the command line interface. Conceptual and reference information is available to help you understand the function of the advisor.

*Launchpads* assist you in completing high-level tasks by stepping you through a set of tasks in the right order. Launchpads can call wizards or other dialogs to accomplish the high-level task. The launchpad task overview on the first page of the launchpad lists any prerequisite steps and briefly describes every page of the launchpad. Other pages of the

launchpad may contain links to conceptual or reference information to help you understand the function of the launchpad.

To select a wizard, advisor, or launchpad, from the Control Center window, select **Tools — > Wizards**. The **Wizards** window opens. Select the wizard, advisor, or launchpad you want to use. Select the object for which you want help and follow the instructions to complete the task.

The following wizards are available:

### Create Database Wizard

Use the Create Database wizard to create a database with default or tailored settings.

- To create a database with default settings, complete only the first page of the wizard.
- To create a database with specific table spaces (containers), custom performance parameters, and local language attributes, complete all the pages of the wizard.
- To create a database with automatic maintenance, use the Create Database with automatic Maintenance wizard.

### Create Table Space Wizard

Use the Create Table Space wizard to create new table spaces. First you name your table space and specify the type of table space you want to create, then you make more advanced choices.

### Create Table Wizard

Use the Create Table wizard to create new tables in a database. First you name the table, determine the columns that you want, and specify space for storing the table data. From there, you continue on to make selections about keys, dimensions, and constraints.

### Backup Database Wizard

Use the Backup wizard to back up the objects in a database or database partition. First you specify if you want to back up the entire database or database partition, or if you want to back up only selected table spaces or table space partitions, then you make more advanced choices.

### Restore Database Wizard

Use the Restore wizard to perform any of the following tasks:

- Restore a database or database partition
- Roll forward a database or database partition
- Restore a database backup image to a new database
- Restore the history file for a database
- Restore a table space or table space partition
- Roll forward a table space or table space partition

First you specify the objects you want to restore and select the image that you want to use, then you make more advanced choices.

## **Load Wizard**

Use the Load wizard to load data into a selected table. The Load wizard guides you through load configuration and the selection of options. The Load wizard also lets you copy an existing load task and use the values that are set in the existing load task for your new load task.

If you want to use exception tables with the load, you must create the exception tables before running the load task.

## **Design Advisor**

Use the Design Advisor to optimize workload performance by suggesting a set of indexes. This advisor recommends which DB2 objects to create in your database to improve performance for a given set of SQL statements in a workload.

## **Configuration Advisor**

Use the Configure Performance wizard to calculate suggested configuration parameters. A series of pages collects information about the server, workload, transaction type, priority, population characteristic, number of connections, and isolation level.

## **Configure Database Logging**

Use the Configure Database Logging wizard to configure data logging options for a database. First, you specify whether you want to use circular logging or archive logging; then, you indicate how you want to handle your log files; then, you make more advanced choices.

## **Add Partitions Launchpad**

This launchpad provides access to the Add Partitions wizard and the Redistribute Data wizard.

Use the Add Partitions wizard to create a partition and add it to one or more database partition groups. First you add a new partition to your instance and assign the partition to one or more database partition groups, then you make more advanced choices.

Use the Redistribute Data wizard to create an effective redistribution plan for your database partition group and redistribute your data. First, you select your redistribution method and strategy, then you make more advanced choices.

***Instructor notes:***

**Purpose** — Identify the wizards provided in the product.

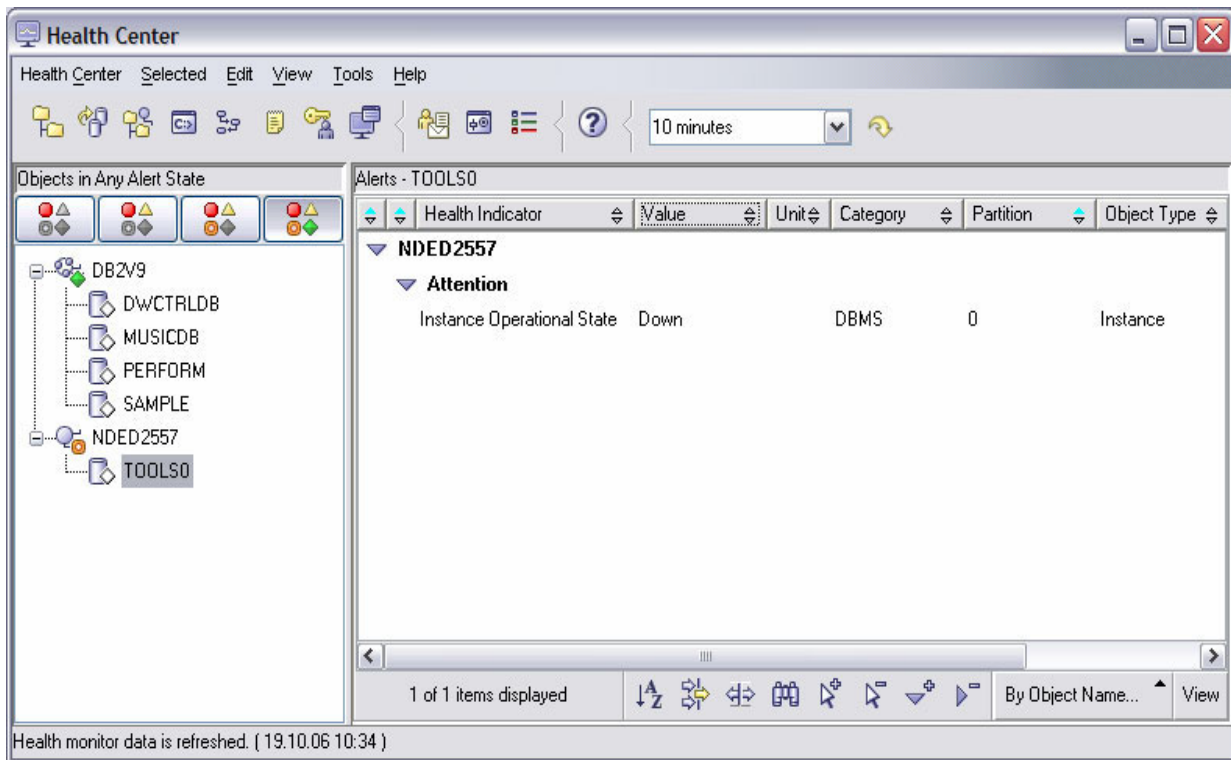
**Details** —

**Additional information** — DB2 provides wizards to help the administrator in performing tasks that may not be done frequently and for which assistance may be required.

Certification objective: 5G Knowledge of the functionality of the DB2 Advisors (db2advis utility).

**Transition statement** — Next we will explore the Health Center.

# Health Center



© Copyright IBM Corporation 2007

Figure 2-17. Health Center

CF238.3

## Notes:

Use the Health Center to monitor the state of the database environment and make any necessary changes.

When you use DB2, a health monitor continuously monitors a set of health indicators. If the current value of a health indicator is worse than the appropriate threshold value, the health monitor generates an alert. DB2 comes with a set of predefined threshold values, which you can later customize. For example, you can customize the alarm and warning thresholds for the amount of free memory on the system.

Depending on your configuration, the following actions can occur when the health monitor generates an alert:

- An entry is written in the administration notification log, which you can read from the Journal.
- The health center status beacon appears in the lower right corner of the DB2 GUI Tools window.
- An e-mail or pager message is sent to the contacts that you specify for this instance.

The following are some of the key tasks that you can perform with the Health Center:

- View the status of the database environment. Beside each object in the navigation tree, an icon indicates the most severe alert for the object (or for any objects contained by that object). For example, a green diamond icon beside an instance means that the instance and the databases contained in the instance do not have any alerts.
- View the alerts for an instance or a database. When you select an object in the navigation tree, the alerts for that object are shown in the pane to the right.
- View detailed information about an alert, and recommended actions. When you double-click an alert, a notebook appears. The first page shows the details for the alert. The second page shows any recommended actions.
- Configure the health monitor settings for a specific object, and the default settings for an object type or for all objects within an instance.
- Select which contacts will be notified of alerts.
- Review the history of alerts for an instance.

***Instructor notes:***

**Purpose** — Identify the purpose of the Health Center.

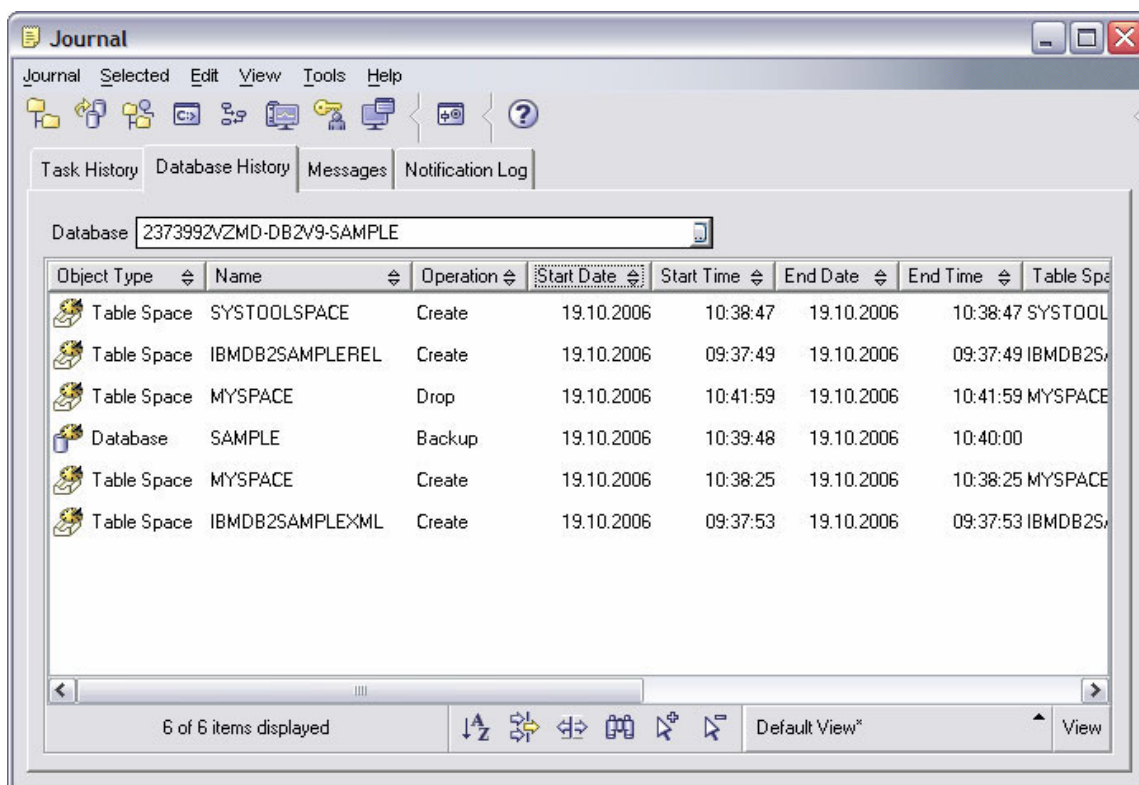
**Details** — Use the Health Center to monitor the state of the database environment and make any necessary changes. The Health Center includes functionality from the Alert Center in previous versions of DB2, plus additional functionality.

**Additional information** —

**Transition statement** — Now let's see the Journal.



# Journal



© Copyright IBM Corporation 2007

Figure 2-18. Journal

CF238.3

## Notes:

The Journal displays historical information about tasks, database actions and operations, Control Center actions, messages, and alerts.

To see the most recent information, click the Refresh button.

The **Task History** page shows the results of tasks that were previously executed. You can use this information to estimate how long future tasks will run.

The Task History page contains one row for each execution of a task. The Task Center contains only one row for each task no matter how many times the task is executed. Each row in the Task Center could be directly related to multiple rows in the Task History page of the Journal.

For each completed execution of a task, you can perform the following actions:

- View the execution results
- View the task that was executed
- Edit the task that was executed
- View the task execution statistics

- Remove the task execution object from the Journal

To perform one of these actions, right-click a completed task execution and select the corresponding action from the pop-up menu that appears.

The **Database History** page shows information from the recovery history file. This file is updated when various operations are performed, including:

- Backup
- Restore
- Roll forward
- Load
- Reorg

This information could be useful if you need to restore a database or table space.

The **Messages** page shows messages that were previously issued from the Control Center and other GUI tools.

The **Notification Log** page shows information from the administration notification log. You can filter the list, for example to show only alerts from the health monitor.

Several customizable saved views are available via the pull-up menu at the bottom of the Journal.

***Instructor notes:***

**Purpose** — Identify the capabilities of the Journal.

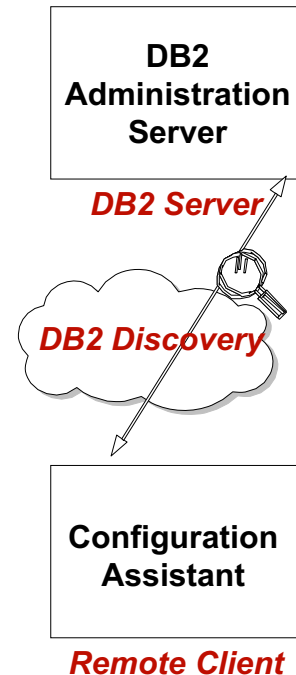
**Details** — The Journal allows you to check on output of jobs, look up information stored in the recovery history file, and review messages that DB2 has issued. The graphic shows the database history, where you can see information from the recovery history file.

**Additional information** —

**Transition statement** — Next let's determine how the Configuration Assistant can help you.

## Configuration Assistant (1 of 2)

- Used to configure client connections
  - Clients can be configured:
    - Manually
    - Automatically using DB2 Discovery
    - Using imported profiles
    - Search the network
- DB2 CA can also:
  - Connect to databases, both LAN and DRDA
  - Perform CLI/ODBC administration
  - Bind applications to database



© Copyright IBM Corporation 2007

Figure 2-19. Configuration Assistant (1 of 2)

CF238.3

### Notes:

The Configuration Assistant (CA) is the GUI tool that is used to configure access to remote databases. It can be invoked from the Control Center, DB2 Desktop folder, or from the command line with the `db2ca` command.

Use the Configuration Assistant to configure your clients. You can also use it as a lightweight alternative to the Control Center, in situations where you do not want to install the complete set of GUI tools.

CA can also be used to change DB2 registry parameters and DB2 instance parameters but, not all available parameters are shown in the lists.

**You must configure your DB2 clients so that they can work with the available objects, as follows:**

- To access an instance or database on another server or system, DB2 must catalog that system in the node directory of the client.
- To access a database, DB2 needs information about the node where the database resides.

- Each database that will be accessed must be configured at the DB2 client.

From the Configuration Assistant, you can work with existing database objects, add new ones, bind applications, set database manager configuration parameters, and import and export configuration information. The graphical interface makes these complex tasks easier through:

- Wizards that help you perform certain tasks
- Dynamic fields that are activated based on your input choices
- Hints that help you make configuration decisions
- Discovery that can retrieve information about selected database objects

The Configuration Assistant displays a list of the databases to which your applications can connect. Each database is identified by its database alias. You can use the Add Database wizard to add databases to the list. You can use the Change Database wizard to alter the information that is associated with databases in the list.

From the View menu, you can select an advanced view, which uses a notebook to organize connection information by object: Systems, Instance Nodes, Databases, Database Connection Services (DCS), and Data Sources.

You can use the notebook pages to perform object-specific actions.

## ***Instructor notes:***

**Purpose** — Provide an overview of the purpose of the CA.

**Details** — One of the concerns that can plague the life of a database administrator is the requirement to define their databases on all of the client machines that need to access the database. This is complicated by the fact that you cannot perform this configuration remotely; **it must be performed on the client machine.**

The Configuration Assistant helps with this. It provides an easy-to-use interface to define access to remote databases. The user of the client can choose to configure the client connection in one of the following ways:

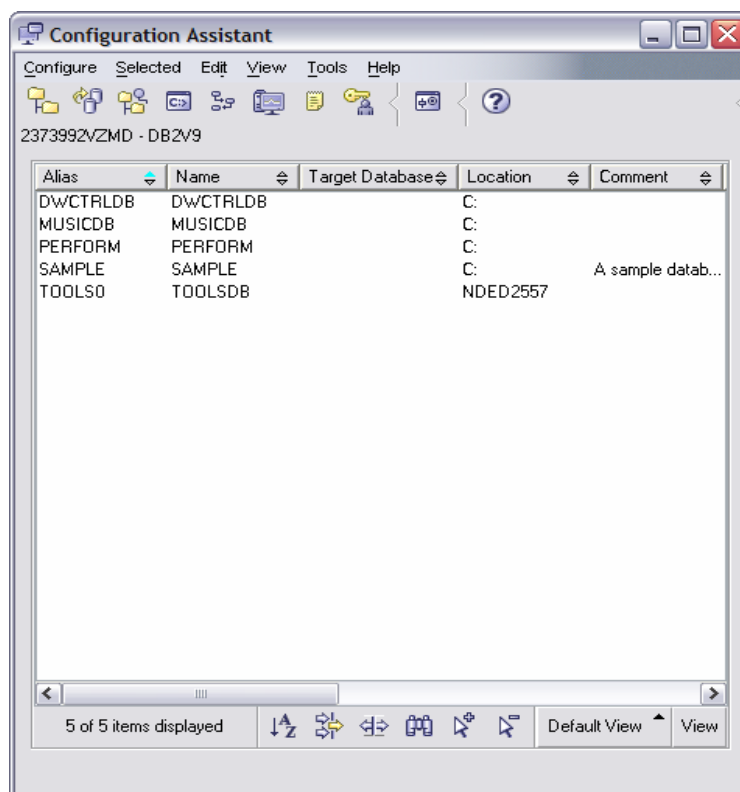
- Manually, requiring the person doing the configuration to know specific protocol information about how the database server was configured;
- Using DB2 Discovery, which can find databases across the network as long as they have been defined as discoverable;
- Using profiles:
  - Either using client profiles: if you have many clients that are clones of each other in terms of the databases that they need to access, you can define the client configuration on one of them, export that profile, and then use it on the others.
  - Or using server profiles: if the clients are heterogeneous in terms of their database access, then you could provide a server profile for each of the servers to which clients may wish to configure access and then let each client just configure those connections appropriate for them.

**Additional information** — The student notes provide much more information than you will want to address in CF2x.

Certification Objective: 1E Ability to configure client / server connectivity, 1H Ability to configure client server connectivity using Discovery.

**Transition statement** — Let's look at the window used for CA.

## Configuration Assistant (2 of 2)



© Copyright IBM Corporation 2007

Figure 2-20. Configuration Assistant (2 of 2)

CF238.3

### Notes:

The graphic shows an example of using the Configuration Assistant (CA). To configure a client connection click **Selected**, **Add Database Using Wizard**, and the **Add Database Wizard** appears. On this page, you indicate how you will be adding the database to which you want to connect. Each method involves a slightly different set of wizard pages.

- **Use a profile** invokes:
  - Select a database from a profile.
  - Specify an alias for the database.
  - Register this database as a data source.
- **Search the network** invokes:
  - Select a database from the network search result.
  - Specify an alias for the database.
  - Register this database as a data source.

- **Manually configure a connection to a database** invokes:
  - Specify catalog options. (Only appears if the Lightweight Directory Access Protocol (LDAP) is enabled.)
  - Select a communications protocol.
  - Specify communication parameters. (A page tailored to the protocol specified on the previous page.)
  - Specify information for a database on this system. (Only appears if the database is local.)
  - Specify information for a database on a remote system. (Only appears if the database is remote.)
  - Register this database as a data source.
  - Specify the node options.
  - Specify the system options.
  - Specify the security options.
  - Specify the DCS options when adding a DCS entry. (Only appears if TCP/IP or APPN is the communications protocol.)

In order to use the discovery facility to search the network for remote databases, you must do the following:

- Ensure that protocol stacks on the client and server are preinstalled and configured so that they are fully functional at discovery time.
- On every server where a database exists which you want the option of discovering:
  - The DB2 Administration Server service must be installed, configured, and active on the server workstation to support one or more protocols.
  - The database manager configuration parameter of DISCOVER must be set to either KNOWN or SEARCH. DISCOVER is only used when the system is acting as a client.
  - The parameter DISCOVER\_INST is used to specify whether this instance can be detected by DB2 discovery. The default, enable, specifies that the instance can be detected, while disable prevents the instance from being discovered.
  - The database parameter of DISCOVER must be enabled for each database to be discovered.
- The code that enables discovery on the client must be installed on the client workstation to support one or more protocols.



**Instructor notes:**

**Purpose** — Show an example of the use of CA.

**Details** — The graphic shows an example of using the CA to begin to define a client connection. The background shows that one database has already been defined. The foreground shows that you can make selections of using an access profile, searching the network (discovery), or performing a manual configuration.

**Additional information** — In many cases, if the server is remote across bridges or routers from the client, you will want to use the *known* discovery, where you provide the name of the server, and DB2 just looks on that server for a DAS, other DB2 instances, and databases.

Certification Objective: 1E Ability to configure client / server connectivity, 1H Ability to configure client server connectivity using Discovery.

**Transition statement** — Now let us have a look at the tool settings.

## Tool settings

- 
- **General**
    - Option to disable hover help, infopops
    - Option to perform **db2start** at tool startup time
    - Define statement termination character
    - Filtering based on number of rows
    - Maximum size for Sample Contents, Command Center
  - **Fonts**
    - Specify font, size, and color for Menu and Text
  - **OS/390 and z/OS**
    - Specify to use system catalog column names
    - Specify Utility execution options
  - **Health Center Status Beacons**
    - Option to specify to notify through pop-up message, status line, or both
  - **Scheduler Settings**
    - Define default scheduling scheme
    - Define Tools Catalog

© Copyright IBM Corporation 2007

Figure 2-21. Tool settings

CF238.3

### Notes:

Using Tool Settings, preferences for the Control Center can be changed. From the Control Center toolbar, click **Tool Settings** icon or choose **Tool Settings** from the *Tools* menu bar option.

This opens the Tool Settings notebook where you can change settings for:

- **General** — Enable or disable hover help, whether to automatically display infopops, whether to start (local) DB2 automatically on tools startup or not, define the statement terminator for SQL statements, and to set the filtering based on number of rows. You can also set the maximum size of rows in Sample Contents and Command Editor.
- **Fonts** — Change the font in which text and menus appear in the DB2 user interface.
- **OS/390 and z/OS** — Identify whether to use system catalog column names as column headings. Specify utility execution options: whether to edit options each time a utility runs, specify the online execution utility ID template, specify if execution should continue if an error is encountered, specify if grouping of objects should occur for parallel utility execution, indicate the maximum number of objects to process in parallel

for online execution, indicate the maximum number of jobs to run in parallel for batch execution, and indicate the maximum number of objects per batch job.

- **Health Center Status Beacons** — Indicate if notification should take place through a popup message or through the status line.
- **Scheduler Settings** — Identify if the default scheduling scheme should be server scheduling or centralized scheduling. Define the tools catalog.

After modifying the settings, close the Tool Settings window and most changes will take place immediately. Some changes, such as those made to fonts, will not take effect until the Control Center is restarted.

## ***Instructor notes:***

**Purpose** — Identify what can be customized on the Tool Settings window.

**Details** — The student notes indicate information about what can be customized using the Tool Settings. Infopops are information windows that are displayed when the mouse cursor is over a field or control in a window or notebook or in a Wizard.

**Additional information** — In order to use the scheduling capabilities in DB2 UDB V8, you must have a DB2 tools catalog database. Creating this database will enable the Task Center feature. The Task Center provides support for complex scheduling schemes to manage different tasks and scripts.

Certification objective: 1F Ability to schedule jobs.

**Transition statement** — Now let's have a look at a component necessary for all this GUI to run in full functionality, the ADMINISTRATION SERVER.

## 2.3 Overview of the Administration Server

### Instructor topic introduction

**What students will do** — They will get an overview of the Administration Server and what it is used for.

**How students will do it** — Listen to the lecture and explore later in the exercise.

**What students will learn** — When they need the Admin Server and how to obtain relevant information.

**How this will help students on their job** —

## DB2 Administration Server

- 
- DB2 control point to assist with server tasks
  - Must have running DAS on server to use Configuration Assistant, Control Center, Task Center, or Development Center
  - Might be created and configured at installation time
  - Assists with:
    - Enabling remote administration of DB2 servers
    - Providing the facility for job management, including scheduling
    - Providing a means for discovering information about the configuration of DB2 instances, databases, and other DB2 Administration Servers in conjunction with DB2 Discovery

© Copyright IBM Corporation 2007

Figure 2-22. DB2 Administration Server

CF238.3

### Notes:

The DB2 Administration Server (DAS) is a control point used only to assist with tasks on DB2 servers. You must have a running DAS if you want to use available tools like the Configuration Assistant, the Control Center, the Task Center, or the Development Center. DAS assists the Control Center and Configuration Assistant when working on the following administration tasks:

- Enabling remote administration of DB2 servers.
- Providing the facility for job management, including the ability to schedule the running of both DB2 and operating system command scripts. These command scripts are user-defined.
- Defining the scheduling of jobs, viewing the results of completed jobs, and performing other administrative tasks against jobs located either remotely or locally to the DAS using the Task Center.
- Providing a means for discovering information about the configuration of DB2 instances, databases, and other DB2 administration servers in conjunction with the DB2 Discovery utility. This information is used by the Configuration Assistant and the Control

Center to simplify and automate the configuration of client connections to DB2 databases.

You can only have one DAS on a machine. DAS is configured during installation to start when the operating system is booted.

DAS is used to perform remote tasks on the server system and the host system on behalf of a client request from the Control Center, the Configuration Assistant, or any of the other available tools.

The DAS on Windows and Linux/UNIX includes a scheduler to run tasks (such as DB2 and operating system command scripts) defined using the Task Center. Task information such as the commands to be run; schedule, notification, and completion actions associated with the task, and run results are stored in a DB2 database called the Tools Catalog database. The Tools Catalog database is created as part of the setup. It can also be created and activated through the Control Center, or through the CLP using the CREATE TOOLS CATALOG command.

### ***Instructor notes:***

**Purpose** — Identify what DAS is needed for.

**Details** — One DAS is required on a server to support remote administration tasks. The graphic and student notes identify what items require DAS to be created and running.

**Additional information** —

**Transition statement** — Let's look at commands associated with DAS.



## Using the Administration Server

- Creating the Administration Server
  - `dasct ASName` (UNIX)
  - `db2admin create` (Windows)
- Starting and stopping the Administration Server
  - `db2admin start`
  - `db2admin stop`
- Listing the Administration Server
  - `db2admin`
- Configuring the Administration Server
  - `db2 get admin cfg`
  - `db2 update admin cfg using ...`
- Removing the Administration Server
  - `dasdrop ASName` (UNIX)
  - `db2admin drop` (Windows)

© Copyright IBM Corporation 2007

Figure 2-23. Using the Administration Server

CF238.3

### Notes:

When you install and configure the DB2 database product, the Administration Server instance is automatically created. You can also manually create an Administration Server instance, start, stop, list, and remove the Administration Server.

On a Linux/UNIX system, to create an Administration Server, you must have root authority to execute the `dasct` command. The syntax is as follows:

```
dasct ASName      (Linux/UNIX)
db2admin create   (Windows)
```

where `ASName` is the name of the Administration Server instance. This is a string of up to eight alphanumeric characters. You use the name of the Administration Server to set up the directory structure and access permissions. You can only start the newly-created Administration Server:

- Following a system reboot after installation
- Using a manual start (`db2admin start`)

To manually start or stop the DAS on Windows, you must first log on to the machine using an account or user ID that belongs to either Administrators, Server Operators, or Power Users groups. To manually start or stop the DAS on Linux/UNIX, the account or user ID must be made part of the *dasadm\_group*. The *dasadm\_group* is specified in the DAS configuration parameters. You can access the DAS configuration parameters by issuing:

```
db2 get admin cfg
db2 update admin cfg using ...
```

To start or stop the DAS, issue:

```
db2admin start
db2admin stop
```

The Administration Server is automatically started after each system reboot.

To obtain the name of the Administration Server on your system, use the `db2admin` command:

```
db2admin
```

To remove the Administration Server, you must log in as the Administration Server owner and issue the following command:

```
db2admin stop
```

If you wish to preserve the files stored for the DAS, you must first back up the files:

- On Windows, in the **db2das00** subdirectory under the **sqllib** subdirectory.
- On Linux/UNIX, in the **das** subdirectory under the **home** directory of the DAS.

On Windows, issue:

```
db2admin drop
```

On Linux/UNIX, log out as the Administration Server owner. Then log in as root and remove the Administration Server instance using the `dasdrop` command as follows:

```
dasdrop ASName
```

where `ASName` is the name of the instance being removed. The `dasdrop` command removes the **sqllib** directory under the home directory of the Administration Server.

***Instructor notes:***

**Purpose** — Identify the commands that can be used with DAS.

**Details** — The creation of the DAS is usually done during install, but if it was not done during install (for example, if you used **db2\_install** on Linux/UNIX), you can create the DAS explicitly using the **dascrt** or **db2admin** create commands. These must be issued by a user ID that has system administration authority (root on Linux/UNIX).

Starting and stopping the DAS can be issued from the DAS ID or from an instance ID.

Listing the DAS can be issued from the DAS ID or from an instance ID.

Looking at and configuring the DAS must be done from an instance ID.

Dropping the DAS must be issued by an ID that has system administration authority.

**Additional information** —

**Transition statement** —

## Unit summary

Having completed this unit, you should be able to:

- Use the Command Line Processor
- Explore the GUI environment
- Describe the DAS role with GUI tools

© Copyright IBM Corporation 2007

Figure 2-24. Unit summary

CF238.3

### Notes:

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Let's look at your next lab. You will be customizing your operating environment.



## Unit 3. The DB2 environment

### Estimated time

01:00

### What this unit is about

This unit describes the environment, including how to create and drop a database server system (Instance). The ability to start and stop the server system will be explained, and the configuration mechanism and parameters for the database system will also be covered.

### What you should be able to do

After completing this unit, you should be able to:

- Specify the key features of an Instance
- Create and drop an Instance
- Use **db2start** and **db2stop**
- Distinguish between types of configuration
- Describe and modify the Database Manager Configuration

### How you will check your progress

Accountability:

- Machine exercises

### References

*Administration Guide: Planning*

*Administration Guide: Implementation*

*Command Reference*

<http://www.ibm.com/software/data/db2/udb/support/manualsv9.html>  
*DB2 V9 Manuals*

## Unit objectives

After completing this unit, you should be able to:

- Specify the key features of an instance
- Create and drop an instance
- Use **db2start** and **db2stop**
- Distinguish between types of configuration
- Describe and modify the Database Manager Configuration

© Copyright IBM Corporation 2007

Figure 3-1. Unit objectives

CF238.3

### **Notes:**



***Instructor notes:***

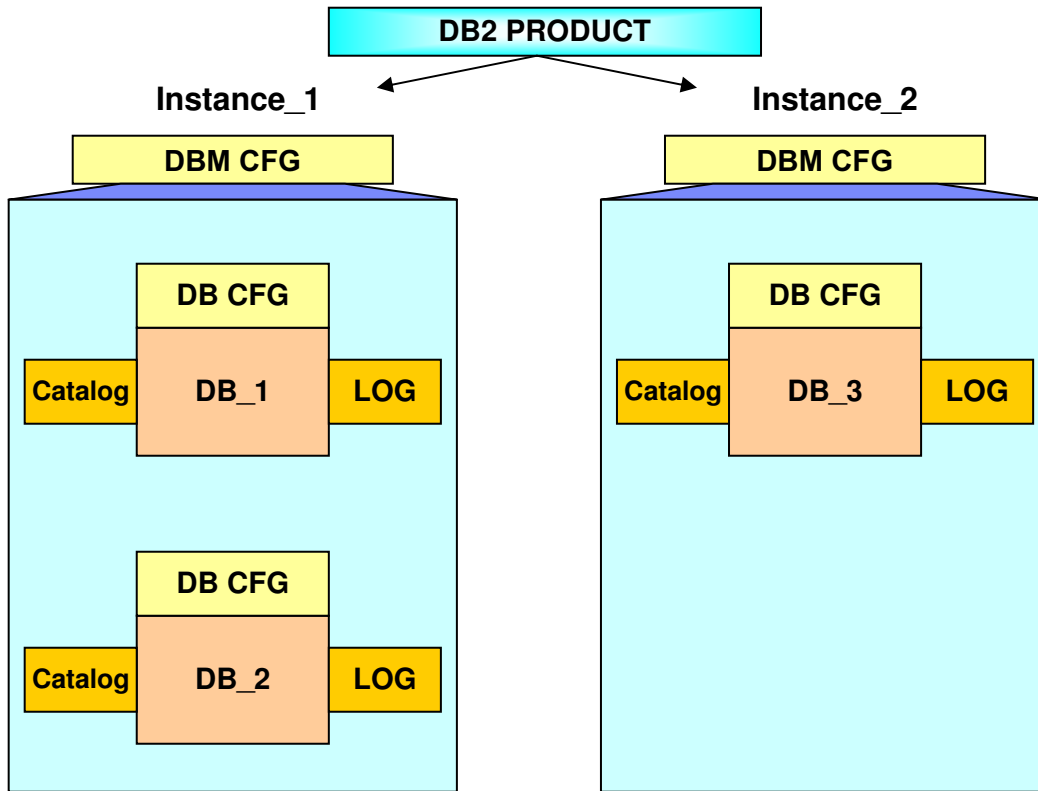
**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**

## What is an instance?



© Copyright IBM Corporation 2007

Figure 3-2. What is an Instance?

CF238.3

### Notes:

Multiple instances of the database manager may be created on a single workstation. This means that you can create several instances of the same product on a physical machine, and have them running concurrently.

*DB2 configuration files* contain parameter values that define the resources allocated to DB2 and individual databases. There are two types of configuration files: the *database manager configuration file*, for the DB2 instance as a whole, and the *database configuration file*, for each individual database.

The *database manager configuration file* is created when an *instance* of DB2 is created, and affects the system resources allocated to DB2 for an individual instance. Its parameters have global applicability, independent of any one database stored in the system.

A *database configuration file* for an individual database is created when the database is created. There is one configuration file per database. The parameters in this configuration file specify the amount of resources to be allocated to that database. Many of these

parameters can be changed to improve the performance or increase capacity. Different changes may be required depending on the type of activity in that specific database.

DB2 creates and maintains a set of system catalog tables for EACH database. These tables contain information such as descriptions of tables, views, and packages.

On Windows and Linux/UNIX-based platforms, each *instance* is a virtual copy of the code using symbolic links to the physical code. This allows a separate physical copy of a new version of code to be installed and instances to be created from the new code.

DB2 users have the power of distributed query across any DB2 family database or OLE DB source. This means that users and applications can use the DB2 SQL syntax and APIs to access data that resides at heterogeneous data sources. With this functionality, users and applications have the capability of referencing multiple data sources in a single SQL statement.

## ***Instructor notes:***

**Purpose** — Introduce the concept of an instance.

- **Details** — If you have any DB2 for MVS or DB2 for OS/390 folks, warn them when coming to this graphic that this is different from what they are used to.
- Point out that multiple instances can be defined on one physical machine. Each instance has its own database manager configuration file that contains parameters that can be configured on the instance level.
- Each instance can have multiple databases defined within it. Each database has its own catalog table, log files, and configuration parameters. (This is similar to a subsystem on DB2 for z/OS, DB2 for MVS, and DB2 for OS/390.)

**Note:** Joins can be specified between tables in different DB2 databases if nicknames are defined that point to the object in the remote database.

- Ask the students: Why do you think you might have multiple instances?
  - Possible answers: unique configuration parameters (test and production instances); one instance on one version of DB2, the other on another version of DB2; different requirements on who should administer the system.

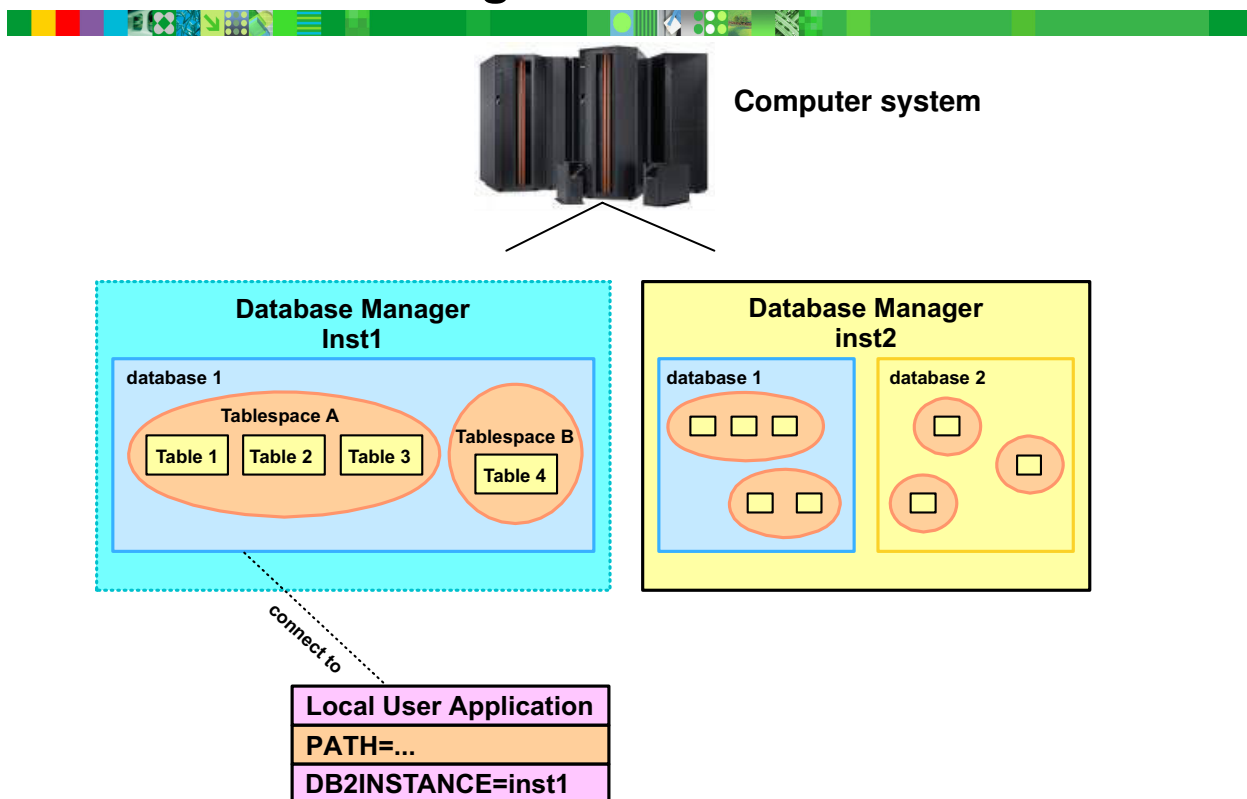
**Additional information — Note:** You can have the same database name on multiple instances (so you could have a production instance and a development instance and they could each have the same database names). The database name must be unique within an instance.

In V8, instances on Linux/UNIX could be running different fixpacks of the DB2 code. Prior versions only allowed different instances to run different *versions* of DB2; if two instances were both on V7, they had to be on the same fixpack level. In V8, they could now be on different fixpack levels.

Also starting with Version 9, there are several DB2 installations on one server allowed (also on Windows).

**Transition statement** — Finally, let's just take a look at an instance.

# The Database Manager instance



© Copyright IBM Corporation 2007

Figure 3-3. The Database Manager instance

CF238.3

## Notes:

Each node can have several instances.

Each instance can manage several databases. However, a given database belongs to only one instance.

A database is a collection of tables.

The tables are logical grouped in table spaces.

Access to a particular database or instance will be determined by the PATH and DB2INSTANCE environment variables or through an CONNECT or ATTACH command in the application.

### ***Instructor notes:***

**Purpose** — Explain the students the *big picture*, also mentioning here the first time the environment variables needed for *configuration*.

**Details** — See student notes.

**Additional information** —

**Transition statement** — Now let us examine what happens when you create or drop such an instance.

## Create and drop an instance

- CREATE (different on Linux/UNIX and Windows)
  - Prerequisites met?
  - Creates the Instance
  - Creates the SQLLIB subdirectory
  - Creates needed directories in the SQLLIB subdirectory
  - Creates the DBM CFG with defaults
- DROP
  - Instance must be stopped and no applications are allowed to be connected to the databases in this instance
  - Does not remove databases (catalog entries)
  - Removes the Instance

© Copyright IBM Corporation 2007

Figure 3-4. Create and drop an instance

CF238.3

### Notes:

To create an Instance, we must distinguish between the Operating Systems Linux/UNIX and Windows.

#### Linux/UNIX environments:

- We must take care from which path we issue the **db2icrt** command (because of possible different installations and fixpacks). Furthermore we need an Instance-Owner (User) and also we need another user, to run fenced processes like UDFs and Stored Procedures.
- Because the authorities of an instance need a Group and not a User, we additionally need to create two groups: one for the administrator and instance owner and one for the fenced user.
- During the instance creation, there will be created different directories and subdirectories for configuration and information in the \$HOME directory of the instance owner:
  - /sqllib/

- adm
- cfg
- ctrl
- db2cshrc
- db2dump
- db2nodes.cfg
- db2profile
- db2system
- security
- sqldbdir
- and so forth. Furthermore, there will be a lot of *links* to the original installation files of DB2.

### Windows environments:

- There is no need to create special users or groups, but it is recommended to do it for security reasons (SYSADM).
- The instance creation creates a directory in the root directory for example, **c:\IBM\SQLLIB\InstanceName**. Here all other subdirectories and files for configuration and information can be found:
  - ctrl
  - db2system
  - log
  - security
  - TMP

Before dropping an instance, you must:

- If needed, backup all databases or if not needed, drop those databases. Dropping an instance does not remove the databases (uncatalog).
- Force all applications connected to any database in the instance.
- Stop the instance.
- All relevant directories and subdirectories will be removed (please verify).



***Instructor notes:***

**Purpose** — Explain the students the steps performed during the instance creation.

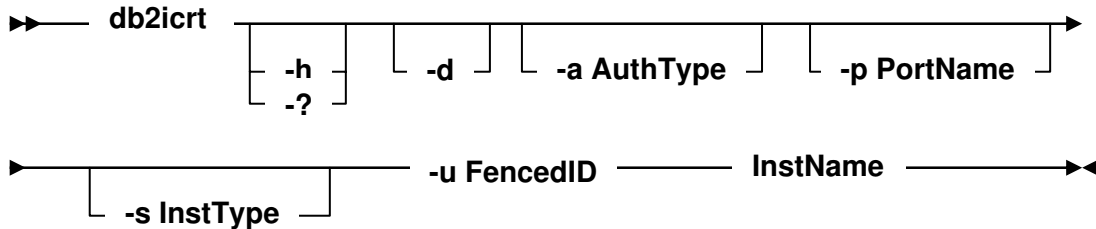
**Details** — Also point out that this is the only command, which is really different on Windows and Linux/UNIX platforms.

**Additional information** —

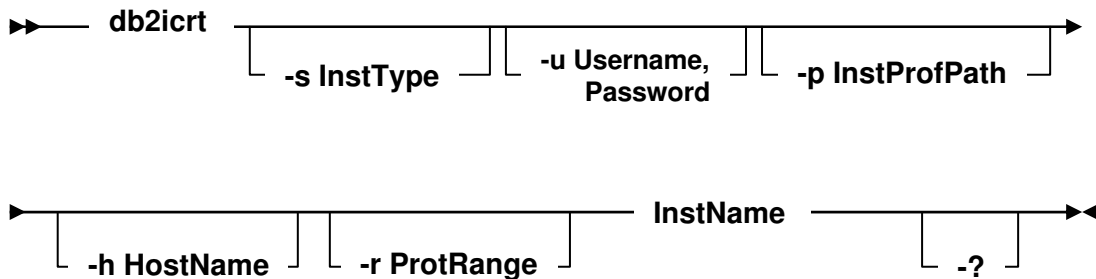
**Transition statement** — Let's look at the **db2icrt** command in detail now.

## db2icrt command

### Linux/UNIX



### Windows



© Copyright IBM Corporation 2007

Figure 3-5. db2icrt command

CF238.3

### Notes:

On Linux and UNIX-based systems, this utility is located in the **DB2DIR/instance** directory, where DB2DIR represents the installation location where the current version of the DB2 database system is installed. On Windows operating systems, this utility is located under the **DB2PATH\bin** directory where DB2PATH is the location where the DB2 copy is installed.

You must have Root access on Linux and UNIX-based systems or Local Administrator authority on Windows operating systems.

Command parameters for Linux and UNIX operating systems:

- **-h or -?** — Displays the usage information.
- **-d** — Turns debug mode on. Use this option only when instructed by DB2 Support.
- **-a AuthType** — Specifies the authentication type (SERVER, CLIENT or SERVER\_ENCRYPT) for the instance. The default is SERVER.
- **-p PortName** — Specifies the port name or number used by the instance. This option does not apply to client instances.

- **-s *InstType*** — Specifies the type of instance to create. Use the **-s** option only when you are creating an instance other than the default for your system. Valid values are:
  - **client** — Used to create an instance for a client. Use this value if you are using DB2 Connect Personal Edition.
  - **ese** — Used to create an instance for a database server with local and remote clients.
  - **wse** — Used to create an instance for DB2 Workgroup Server Edition, DB2 Express Edition and DB2 Connect Enterprise Edition.
- **-u *Fenced ID*** — Specifies the name of the user ID under which fenced user-defined functions and fenced stored procedures will run. The **-u** option is required if you are not creating a client instance.
- ***InstName*** — Specifies the name of the instance which is also the name of an existing user in the operating system.

Command parameters for Windows operating systems:

- ***InstName*** — Specifies the name of the instance.
- **-s *InstType*** — Specifies the type of instance to create. Valid values are:
  - **client** — Used to create an instance for a client. Use this value if you are using DB2 Connect Personal Edition.
  - **standalone** — Used to create an instance for a database server with local clients.
  - **ese** — Used to create an instance for a database server with local and remote clients.
  - **wse** — Used to create an instance for DB2 Workgroup Server Edition, DB2 Express Edition and DB2 Connect Enterprise Edition.
- **-u *Username, Password*** — Specifies the account name and password for the DB2 service. This option is required when creating a partitioned database instance.
- **-p *InstProfPath*** — Specifies the instance profile path.
- **-h *HostName*** — Overrides the default TCP/IP host name if there is more than one for the current machine. The TCP/IP host name is used when creating the default database partition (database partition 0). This option is only valid for partitioned database instances.
- **-r *PortRange*** — Specifies a range of TCP/IP ports to be used by the partitioned database instance when running in MPP mode. For example, -r 50000,50007. The services file of the local machine will be updated with the following entries if this option is specified:
  - DB2\_*InstName* baseport/tcp
  - DB2\_*InstName*\_END endport/tcp
- **-?** — Displays usage information.

## ***Instructor notes:***

**Purpose** — Go with the students through the most important syntax points.

**Details** — It could be helpful, to point out (maybe on the white board) the differences in a short way like:

Linux and UNIX: ***DB2DIR/instance/db2icrt -u db2fenc1 db2inst1***

Windows: ***db2icrt inst1***

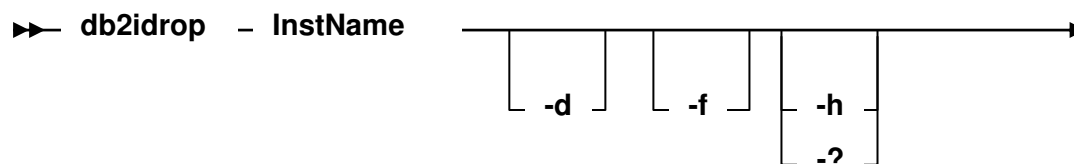
**Additional information** —

**Transition statement** — Next let's examine the ***db2idrop*** command.

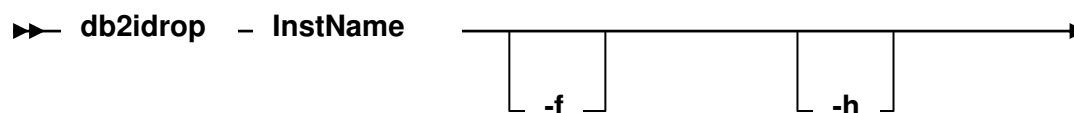
**Transition statement** — Certification objective: 1A Ability to configure / maintain DB2 instances (for example, scope).

## db2idrop command

### Linux/UNIX



### Windows



© Copyright IBM Corporation 2007

Figure 3-6. db2idrop command

CF238.3

### Notes:

Removes a DB2 instance that was created by db2icrt. You can only drop instances that are listed by db2ilist for the same DB2 copy where you are issuing db2idrop from.

On Linux and UNIX-based systems, this utility is located in the **DB2DIR/instance** directory, where DB2DIR represents the installation location where the current version of the DB2 database system is installed. On Windows operating systems, this utility is located under the **DB2PATH\bin** directory where DB2PATH is the location where the DB2 copy is installed.

You must have Root access on Linux and UNIX-based systems or Local Administrator on Windows operating systems.

Command parameters for Linux and UNIX-based systems:

**InstName** — Specifies the name of the instance.

- **-d** — Enters debug mode, for use by DB2 Service.
- **-f** — Specifies the force applications flag. If this flag is specified, all the applications using the instance will be forced to terminate.

- **-h or -?** — Displays the usage information.

Command parameters for Windows-based systems:

- **InstName** — Specifies the name of the instance.
- **-f** — Specifies the force applications flag. If this flag is specified, all the applications using the instance will be forced to terminate.
- **-h** — Displays usage information.

***Instructor notes:***

**Purpose** — Explain the students how an instance can be dropped.

**Details** — see student notes. Also here it might be helpful to show the *little* difference between Linux/UNIX and Windows:

Windows: **db2idrop inst1....**

Linux/UNIX: ***DB2DIR/instance/db2idrop db2inst1***

**Additional information** —

**Transition statement** — Next lets have a look how to start and stop an instance.

## Starting and stopping an instance



**db2start**



**db2stop**

© Copyright IBM Corporation 2007

Figure 3-7. Starting and stopping an instance

CF238.3

### **Notes:**

#### **db2start**

Starts the current database manager instance background processes on a single database partition or on all the database partitions defined in a partitioned database environment. Start DB2 at the server before connecting to a database, precompiling an application, or binding a package to a database. **db2start** can be executed as a system command or a CLP command.

The **db2start** command launches the DB2 product installation as a Windows service. The DB2 product installation on Windows can still be run as a process by specifying the **/D** switch when invoking **db2start**. The DB2 product installation can also be started as a service using the Control Panel or the **NET START** command.

Since **db2start** launches a Windows service, you must meet Windows requirements for starting a service. If Extended Security is disabled, you must be a member of the Administrators, Server Operators or Power Users group. If Extended Security is enabled, you must be a member of either the Administrators group or the DB2ADMNS group to start the database.



If a **db2start** operation in a multi-partition database is not completed within the value specified by the *start\_stop\_timeout* database manager configuration parameter, the database partitions that have timed out will be killed internally (all resources associated with the database partition will be removed). Environments with many database partitions with a low value for *start\_stop\_timeout* might experience this behavior. To resolve this behavior, increase the value of *start\_stop\_timeout*.

Alternatively, you can also issue a *start dbm* or *start database manager* command.

## **db2stop**

Stops the current database manager instance. **db2stop** can be executed as a system command or a CLP command.

If a **db2stop** operation in a multi-partition database is not completed within the value specified by the *start\_stop\_timeout* database manager configuration parameter, the database partitions that have timed out will be killed internally (all resources associated with the database partition will be removed). Environments with many database partitions with a low value for *start\_stop\_timeout* might experience this behavior. To resolve this behavior, increase the value of *start\_stop\_timeout*.

If a **db2stop** command is issued and there are active databases or connections to the databases in the instance, an error is returned. If you need to force the stop (so applications will be forced, not committed work rolled back) you can issue **db2stop force**.

Alternatively, you can also issue a *stop dbm* or *stop database manager* command.

## ***Instructor notes:***

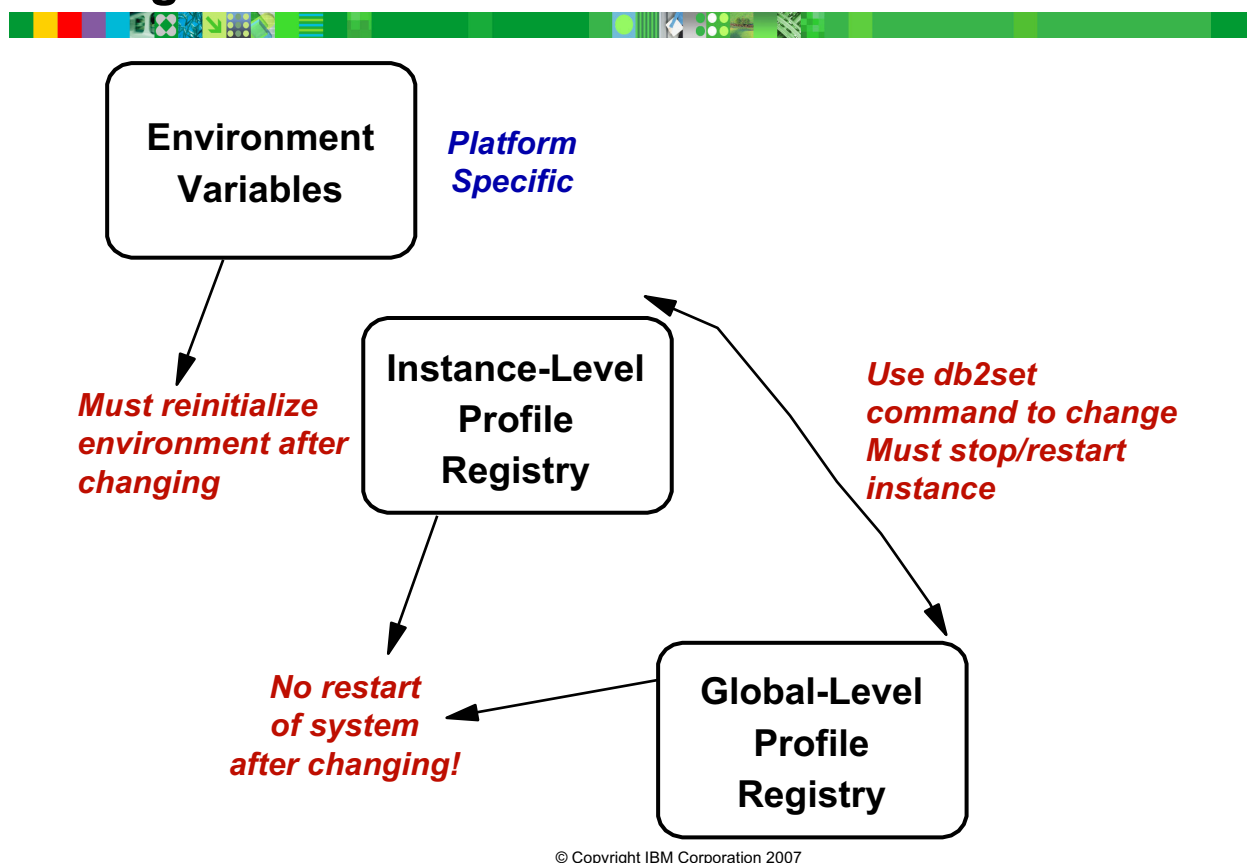
**Purpose** — Explain the students how to start and stop an instance.

**Details** — With this command per default the instance mentioned in DB2INSTANCE variable will be started or stopped. If the students need to do this with other instances the need to issue the **db2set** command.

**Additional information** — The instance remains active until it is stopped explicitly.

**Transition statement** — Next let's examine the different types of configuration.

## Setting DB2 variable values



© Copyright IBM Corporation 2007

Figure 3-8. Types of configuration

CF238.3

### Notes:

There are different types of configurations related to the Instance:

- Registry settings like DB2PATH, DB2COMM, and so forth. These settings can be changed using the *db2set* command.
- Configuration settings in the Database Manager Configuration file. These settings can be changed using the GUI in the Control Center, selecting the instance, right-clicking and selecting Configure parameters, or with the command *update dbm cfg using ParameterName ParameterValue*.
- Some of the DBM CFG parameters can be changed online, which means the change takes effect immediately. Some of them can be changed, but the change will be effective after an instance restart. Using the GUI, you might see which of them are updatable online and which offline.
- There are also several parameters which are or can be set to AUTOMATIC so you don't have to care about the values.

## ***Instructor notes:***

**Purpose** — Explain the students the different types of configuration.

**Details** —

**Additional information** —

**Transition statement** — Let's have now a closer look the DBM CFG.

**Transition statement** — Certification objective: 1B Ability to manage / modify Database Manager configuration information , 6J Knowledge of and ability to perform Dynamic Configuration Parameters.

## The db2set command

- Command line tool for DB2 Profile Registry administration
- Displays, sets, resets, or removes profile variables

```
db2set variable=value
-g
-e
-i instance [db-partition-number]
-n DAS node [[-u user id] [-p password]]
-u
-ul
-ur
-p
-r
-l
-lr
-v
-? (or -h)
-all
```

© Copyright IBM Corporation 2007

Figure 3-9. Database Manager configuration

CF238.3

### Notes:

You can see the database manager configuration in different ways. The DBM configuration GUI can be accessed via the CC (Control Center) and CA (Configuration Assistant). Not all possible parameters are listed, but the command line can be used to change others.

- Using the CLP or Command Editor and then issuing *db2 get dbm cfg (show detail)*.
- Using the GUI from the Control Center: Select the instance, right-click with the mouse and select Configure parameters. This method has advantages:
  - The parameters are grouped
  - You can see if they could be changed online or offline
  - You will get a hint (short description) if you highlight a specific parameter

The DBM CFG parameter groups are the following:

- Administration related parameters
- Application related parameters

- Communication related parameters
- Diagnostic related parameters
- Environment related parameters
- Miscellaneous related parameters
- Monitoring related parameters
- Parallel functionality related parameters
- Performance related parameters

To change the value of any of those parameters, you can:

Use the CLP or Command Editor with the command *update dbm cfg using ParameterName ParameterValue*

Use the GUI from the Control Center: Select the instance, right-click with the mouse and select Configure parameters. Here you might also have some advantages:

- You will get a hint (short description) if you highlight a specific parameter
- When you click to the VALUE field you will see:
  - Actual value
  - Possible range of values
  - If this parameter could be changed online
  - Hint (short description) of this parameter

***Instructor notes:***

**Purpose** — Explain the students the different parameter groups, how they can get information and how the parameters could be changed.

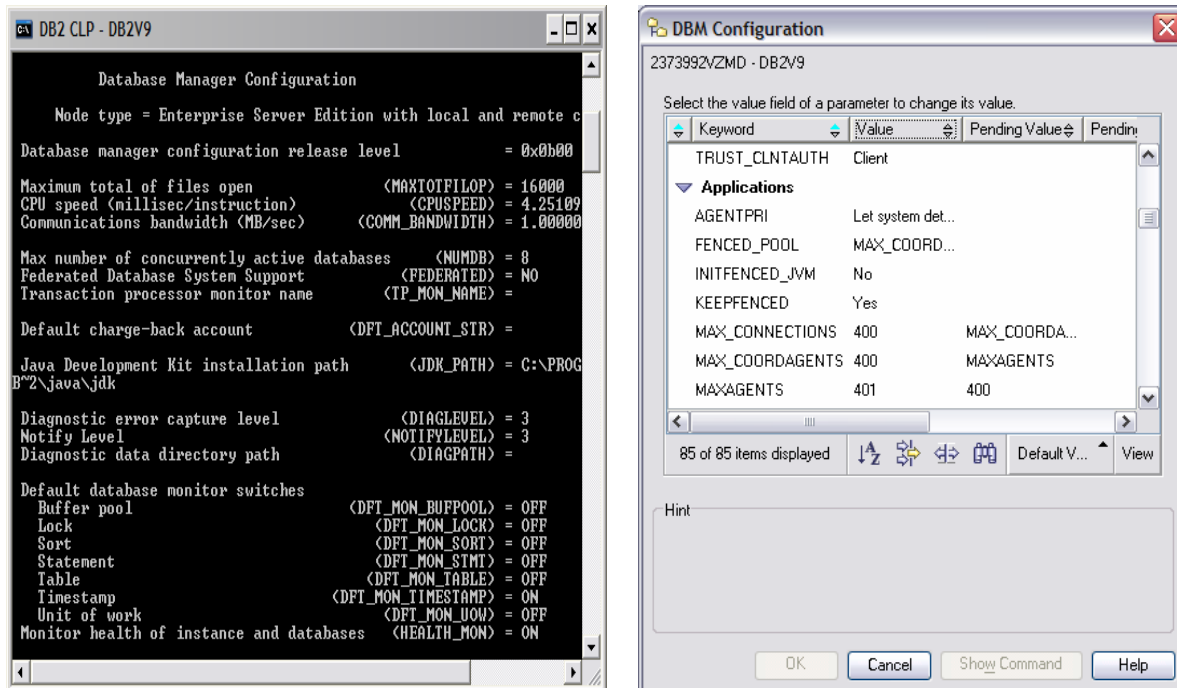
**Details** —

**Additional information** —

**Transition statement** —

**Transition statement** — Certification objective: 1B Ability to manage / modify Database Manager configuration information.

# Database Manager configuration



© Copyright IBM Corporation 2007

Figure 3-10. Unit summary

CF238.3

## Notes:



***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**



# Unit 4. Creating databases and data placement

## Estimated time

01:15

## What this unit is about

When you are ready to implement your database, there are a number of factors which should be considered about the physical environment in which the database will be implemented. These factors include choosing an instance in which you will create your database, how much space will be required to store your data, where data should be physically located, and what kind of table space should be used for storing data. This unit will address considerations in these areas.

## What you should be able to do

After completing this unit, you should be able to:

- Review specifics of creating a database
- Explore the System Catalog tables and views
- Compare DMS versus SMS table spaces
- Describe how to setup and manage a DB2 database with Automatic Storage enabled
- Differentiate between table spaces, containers, extents, and pages
- Define table spaces
- Use the get snapshot for tablespaces command to display table space statistics
- Explore Database configuration parameters

## How you will check your progress

Accountability:

- Checkpoint
- Machine exercises

## References

*Administration Guide*

## Unit objectives

After completing this unit, you should be able to:

- Review specifics of creating a database
- Explore the System Catalog tables and views
- Compare DMS versus SMS table spaces
- Describe how to setup and manage a DB2 database with Automatic Storage enabled
- Differentiate between table spaces, containers, extents, and pages
- Define table spaces
- Use the **get snapshot for tablespaces** to display table space statistics
- Explore database configuration parameters

© Copyright IBM Corporation 2007

Figure 4-1. Unit objectives

CF238.3

### Notes:

### ***Instructor notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Identify what we are going to address in this unit.

**Transition Statement** — You have just completed your lab. In the lab, you created your instance, and used the GUIs provided with the DB2. Now, let's examine creating databases and table spaces.

## 4.1 Create database overview

### Instructor topic introduction

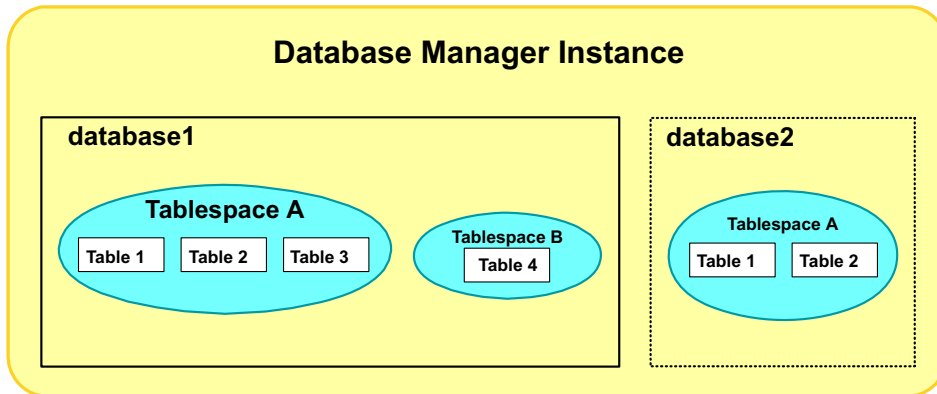
**What students will do** — Listen to a short lecture and then create a simple database in a lab.

**How students will do it** — Listen to the lecture and practice in the lab.

**What students will learn** — An overview of the creation of a database. They will learn the steps that are completed when a database is created.

**How this will help students on their job** — They will know how to create a database using the default options for table spaces (SMS table spaces for three default table spaces).

## Create database overview



- Databases are created within a Database Manager instance
- Table spaces are a logical layer created within database
- Tables are created within table spaces

© Copyright IBM Corporation 2007

Figure 4-2. Create database overview

CF238.3

### Notes:

Each instance can have one or more databases defined in it.

Each database can have three or more table spaces associated with it. The table spaces are a logical level between the database and the tables stored in that database. Table spaces are created within a database, and tables are created within table spaces.



***Instructor notes:***

**Purpose** — Identify relationship between instance, database, table spaces, and tables.

**Details** — As discussed previously, one or more databases can reside in an instance. DB2 creates three default table spaces in a database when it is created, and you can create additional table spaces for the storage of your table, index, and long objects.

**Additional Information** —

**Transition Statement** — Let's look at what is completed when a database is created.

## Steps for database creation

- 
1. Create database
  2. Create database configuration file and set default values
  3. Bind database utilities to the database (db2ubind.lst)
  4. Define SYSCATSPACE, TEMPSPACE1, and USERSPACE1 table spaces
  5. Set up all of the system catalog tables and allocate the database recovery log
  6. Catalog database in local database directory and system database directory
  7. Assign codeset, territory, and collating sequence
  8. Create SYSCAT, SYSFUN, SYSIBM, SYSSTAT schemata
  9. Grant the following privileges to database Creator:
    - DBADM authority with CONNECT, CREATETAB, BINDADD, IMPLICIT\_SCHEMA, CREATE\_NOT\_FENCED\_ROUTINE, CREATE\_EXTERNAL\_ROUTINE, QUIESCE\_CONNECT, and LOAD authorities
  10. Grant the following privileges to Public (If RESTRICTIVE is not specified):
    - SELECT privilege on system catalog tables and views
    - BIND and EXECUTE privileges to PUBLIC for each successfully bound utility
    - CREATETAB, BINDADD, IMPLICIT\_SCHEMA, and CONNECT authorities
    - USE privilege on USERSPACE1 table space

© Copyright IBM Corporation 2007

Figure 4-3. Steps for database creation

CF238.3

### Notes:

When you create a database, the database manager:

1. Creates a database in the specified subdirectory.
2. Creates the database configuration file and sets the default values.
3. Binds previously defined database manager bind files to the database (these are listed in db2ubind.lst).
4. Creates SYSCATSPACE, TEMPSPACE1, AND USERSPACE1 table spaces.
5. Sets up all of the system catalog tables needed by the database and allocates the database recovery log.
6. Catalogs the database in the following database directories:
  - Server's local database directory on the path indicated by path or, if the path is not specified, the default database path defined in the database manager system configuration file. A local database directory resides on each operating system file system (UNIX) or drive (Windows) that contains a database.
  - Server's system database directory for the attached instance. The resulting directory entry will contain the database name and a database alias. If the command was

issued from a remote client, the client's system database directory is also updated with the database name and alias.

- Creates a system or a local database directory if neither exists. If specified, the comment and code set values are placed in both directories.
7. Stores the specified codeset, territory, and collating sequence. A flag is set in the database configuration file if the collating sequence consists of unique weights, or if it is the identity sequence.
  8. Creates the SYSCAT, SYSFUN, SYSIBM, and SYSSTAT schemata with SYSIBM as the owner.
  9. Grants the following privileges:
    - DBADM authority with CONNECT, CREATETAB, BINDADD, IMPLICIT\_SCHEMA, CREATE\_NOT\_FENCED\_ROUTINE (for user-defined functions and stored procedures), CREATE\_EXTERNAL\_ROUTINE (for user-defined functions and stored procedures that are implemented using external code), QUIESCE\_CONNECT and LOAD authorities to database creator
    - SELECT privileges on each system catalog to PUBLIC
    - BIND and EXECUTE privileges to PUBLIC for each successfully bound utility
    - CREATETAB, BINDADD, IMPLICIT\_SCHEMA, and CONNECT authorities to PUBLIC
    - USE privilege on USERSPACE1 table space to PUBLIC

## ***Instructor notes:***

**Purpose** — Identify what occurs when a database is created.

**Details** — The graphic supplies most all the information you need to lecture.

**Additional Information** — You may want to mention that the grants that are indicated as the last point on the graphic are what DB2 does when the database is created. You may not agree with these grants, I may not agree with these grants, but it really doesn't matter whether we agree with them or not, they are the grants DB2 does when the database is created. They make the database rather wide open (PUBLIC is given connect and create table authorities!). You can revoke the privileges after the database is created, if you want.

A student may ask, "Why does DB2 leave the database so wide open like this?", particularly if they are coming from a DB2 for z/OS background, where the database is initially closed down, and an administrator must explicitly grant privileges to allow any access. My (Patti's) answer to this is as follows: On other databases, for example, on DB2 for z/OS, you always have a professional DBA staff. On DB2 for Linux, UNIX, and Windows, there are many shops where there is no professional DBA staff (probably not for any of the students we see in classes, but for *Mom 'n' Pop Grocery Stores*, they may not have a professional DBA). The goal with DB2, as I have heard it, is that it be an "out of the box" experience, in other words, that it be immediately usable right out of the box. Imagine how disappointed you would be if you were *Mom* or *Pop* who just bought this new database for your grocery store, and you installed it, but before you could start using your slick financial application, you had to figure out how to grant privileges on the database.

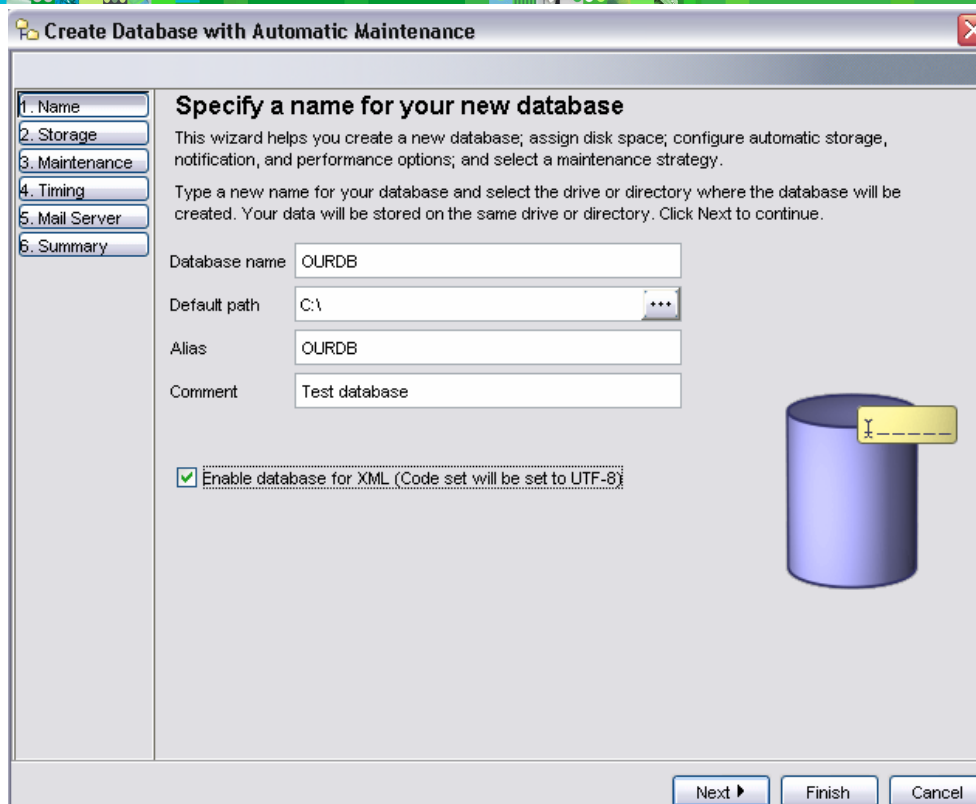
Now, the fact that the database is wide open may not be a problem for *Mom 'n' Pop Grocery Stores*; they control access to the database not through database security, but through preventing physical access to the machine where the database is located.

More discussion on this at the end of the class when we look at security.

Certification objective: 2A Ability to create a database.

**Transition statement** — Let's look at the create database wizard.

# Create Database Wizard



© Copyright IBM Corporation 2007

Figure 4-4. Create Database Wizard

CF238.3

## Notes:

Only a user with SYSADM or SYSCTRL authority may create databases. A database can be created using CLP, or the Control Center, or through an application program invoking an API. The Create Database Wizard Graphical User Interface or the Create Database Wizard with Automatic Maintenance (GUI) can be displayed through the Control Center.

From the Control Center, expand the object tree until you find the **Databases** folder. Right-click the Databases folder, and then select **Create Database--> Automatic Maintenance** from the pop-up menu. The Create Database wizard opens.

Complete each of the applicable wizard pages. Each page is discussed below. The **Finish** push button is available when you complete enough information for the wizard to create your database. Click **Finish** to create your database.

## Name

On this page, you specify a name for your database and select a drive on which to store it. Use the following pages if you want to customize the database. You can also specify whether you want to enable the database for XML.

## User Tables

Use this page to select how your user tables should be managed. If you use System Managed Space (SMS), the operating system's file system manager allocates and manages the space where the user tables are stored. The storage model typically consists of many files, representing table objects, stored in the file system space. If you use Database Managed Space (DMS), the database manager controls the storage space. The storage model consists of a limited number of devices, whose space is managed by DB2.

## Catalog Tables

Use this page to select how your catalog tables should be managed. If you use System Managed Space (SMS), the operating system's file system manager allocates and manages the space where the catalog tables are stored. If you use Database Managed Space (DMS), the database manager controls the storage space.

## Temporary Tables

Use this page to select how your temporary tables should be managed. If you use System Managed Space (SMS), the operating system's file system manager allocates and manages the space where the temporary tables are stored. If you use Database Managed Space (DMS), the database manager controls the storage space.

## Performance

This page allows you to set performance parameters for your database.

## Region

Use this page to specify the locale of the database in order to determine the character set that the database will use. Also use this page to indicate the collating sequence, which determines how character strings are to be sorted. You cannot change the collating sequence after the database has been created.

## Summary

This page allows you to review the decisions that you made for your create database operation, and to view the command that will create your database.

In the **Wizard with Automatic Maintenance**, you can additionally indicate such things as: specifying if you will have an offline maintenance window or not, the start time and day for your maintenance window, and contacts for notification.

***Instructor notes:***

**Purpose** — Explain the Create Database Wizard and its tabs and mention also the Automatic Maintenance wizard and its possibilities.

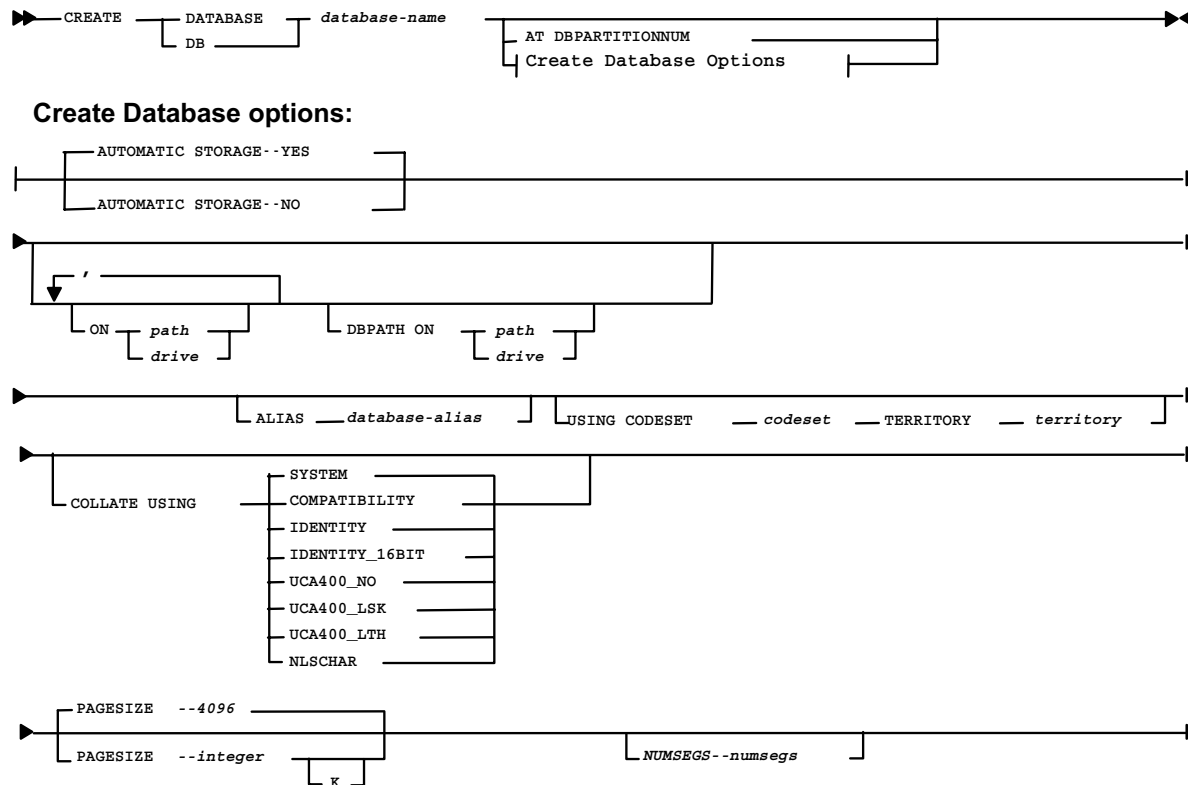
**Additional Information** — The next foil shows the CREATE DATABASE syntax. Both of these foils should be up at once. You will not need to talk about the database syntax diagram. There are several places where both the GUI and the syntax diagrams are included.

The instructor should use discretion and emphasize what is appropriate for the student audience. In most cases, you should not take the time to cover both GUI and syntax charts for each topic. For example, it may be advisable to cover CREATE INDEX with the syntax page and CREATE TRIGGER with the GUI page, and alter topics throughout the lecture.

The student notes are derived from the help screens for the wizard (Task Description).

**Transition Statement** — Let's look at the database syntax diagram in conjunction with the Create Database Wizard.

# Create database syntax (1 of 3)



© Copyright IBM Corporation 2007

Figure 4-5. Create database syntax (1 of 3)

CF238.3

## Notes:

The CREATE DATABASE command creates a database at the location defined by the ON parameter or at the default location. Only a user with SYSADM or SYSCTRL authority may create databases. A database can be created using CLP, or the Control Center, or through an application program invoking an API.

The CREATE DATABASE command initializes a new database, creates three initial table spaces, creates the system catalog tables, and allocates recovery logs.

## Command Parameters:

### database-name

Unique name to be assigned to the new database. 1-8 characters long.

- Must begin with alphabetic character, @, #, or \$
- May contain a-z, A-Z, 0-9, @, #, \$, or \_ (underscore)



## AT DBPARTITIONNUM

Specifies that the database is to be created only on the database partition that issues the command. You do not specify this option when you create a new database. You can use it to recreate a database partition that you dropped because it was damaged.

## AUTOMATIC STORAGE

Specifies that automatic storage is being explicitly disabled or enabled for the database. The default value is YES. If the AUTOMATIC STORAGE clause is not specified, automatic storage is implicitly enabled by default.

- Must be NO or YES

## path/drive

On UNIX platforms, the path on which to create the database can be specified. By default, the **DFTDBPATH** parameter specified in the database manager configuration file is used. On Windows platforms, the letter of the drive or path on which to create the database can be specified.

## database-alias

An alias for the database. Defaults to the database name. The alias name is used by applications to connect to the database.

## codeset

Specifies the code set to be used for data entered into this database.

## territory

Specifies the territory to be used for data entered into this database.

## COLLATE USING

Identifies the type of collating sequence to be used for this database.

**SYSTEM** indicates that the collating sequence is based on the current territory.

**IDENTITY** indicates that the collating sequence is the identity sequence, where strings are compared byte by byte.

## dft-extentsize

Specifies the default extent size of table spaces in the database.

## tblspace-defn

Specifies the definition of the table space which will be used to hold the catalog tables (CATALOG TABLESPACE — SYSCATSPACE), initial user table space (USER TABLESPACE — USERSPACE1), and initial temporary table space (TEMPORARY TABLESPACE — TEMPSPACE1).

## “comment string”

Up to 30-character comment describing the database.

## **PAGESIZE**

Specifies the page size of the default buffer pool along with the initial table spaces (SYSCATSPACE, TEMPSPACE1, USERSPACE1) when the database is created. This also represents the default page size for all future CREATE BUFFERPOOL and CREATE TABLESPACE statements.

- Valid values for integer without the suffix K are 4096, 8192, 16384, or 32768.
- Valid values for integer with the suffix K are 4, 8, 16, or 32.
- At least one space is required between the integer and the suffix K. The default is a page size of 4096 bytes (4 K).

**Instructor notes:**

**Purpose** — Identify the syntax for CREATE DATABASE.

**Details** — Most of the syntax for CREATE DATABASE is shown. Talk about the major items here. Database name must be provided. Path will default to the path in the DBM CFG if not specified, which by default is the instance's home directory: probably not desirable. The alias will default to being the same as the database name.

DB2 will pick up the codeset and territory from the operating system, but you can override if you need to. However, the codeset and territory that you indicate must be available from the operating system (you can't create a Hebrew database if the Hebrew codeset/territory is not known to the operating system).

The collate sequence will default to SYSTEM if not specified. You can also specify IDENTITY to indicate that strings should be compared byte for byte.

Skip default extent size for now.

Three default table spaces are created when the database is created. The definition below allows you to specify exactly what storage to use. We will look at that later as we look at the syntax for creating table spaces.

You can put a comment on to a database. There are also auto-configuration settings that can be specified.

**Additional Information** — Certification objective: 2A Ability to create a database.

**Transition statement** — Let's look at more of the database syntax diagram.

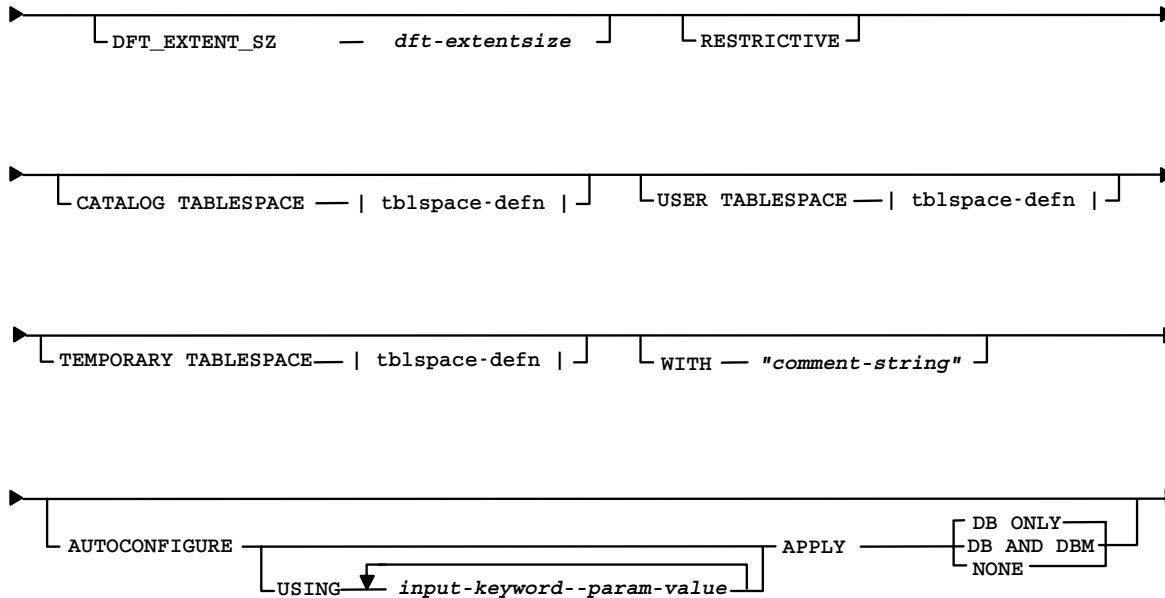


Figure 4-6. Create database syntax (2 of 3) CF238.3

More parts of the CREATE DATABASE statement are explained on this page.

Specifies the default extent size of table spaces in the database.

If the RESTRICTIVE option is present, it causes the RESTRICT\_ACCESS database configuration parameter to be set to YES and no privileges are automatically granted to PUBLIC. If the RESTRICTIVE option is not present, then the RESTRICT\_ACCESS database configuration parameter is set to NO and all of the following privileges are automatically granted to PUBLIC.

- CREATETAB
- BINDADD
- CONNECT
- IMPLSCHEMA
- EXECUTE with GRANT on all procedures in schema SQLJ

- EXECUTE with GRANT on all functions and procedures in schema SYSPROC
- BIND on all packages created in the NULLID schema
- EXECUTE on all packages created in the NULLID schema
- CREATEIN on schema SQLJ
- CREATEIN on schema NULLID
- USE on table space USERSPACE1
- SELECT access to the SYSIBM catalog tables
- SELECT access to the SYSCAT catalog views
- SELECT access to the SYSSTAT catalog views
- UPDATE access to the SYSSTAT catalog views

## CATALOG TABLESPACE

Specifies the definition of the table space that will hold the catalog tables, SYSCATSPACE. If not specified and automatic storage is not enabled for the database, SYSCATSPACE is created as a System Managed Space (SMS) table space with *numsegs* number of directories as containers, and with an extent size of *dft\_extentsize*.

If not specified and automatic storage is enabled for the database, SYSCATSPACE is created as an automatic storage table space with its containers created on the defined storage paths. The extent size of this table space is 4. Appropriate values for AUTORESIZE, INITIALSIZE, INCREASESIZE, and MAXSIZE are set automatically.

## USER TABLESPACE

Specifies the definition of the initial user table space, USERSPACE1. If not specified and automatic storage is not enabled for the database, USERSPACE1 is created as an SMS table space with *numsegs* number of directories as containers and with an extent size of *dft\_extentsize*.

If not specified and automatic storage is enabled for the database, USERSPACE1 is created as an automatic storage table space with its containers created on the defined storage paths. The extent size of this table space will be *dft\_extentsize*. Appropriate values for AUTORESIZE, INITIALSIZE, INCREASESIZE, and MAXSIZE are set automatically.

## TEMPORARY TABLESPACE

Specifies the definition of the initial system temporary table space, TEMPSPACE1. If not specified and automatic storage is not enabled for the database, TEMPSPACE1 is created as an SMS table space with *numsegs* number of directories as containers and with an extent size of *dft\_extentsize*.

If not specified and automatic storage is enabled for the database, TEMPSPACE1 is created as an automatic storage table space with its containers created on the defined storage paths. The extent size of this table space is *dft\_extentsize*.

## WITH

Describes the database entry in the database directory. Any comment that helps to describe the database can be entered. Maximum length is 30 characters. A carriage

return or a line feed character is not permitted. The comment text must be enclosed by single or double quotation marks.

## **AUTOCONFIGURE**

Based on user input, calculates the recommended settings for buffer pool size, database configuration, and database manager configuration and optionally applies them. The Configuration Advisor is run by default when the CREATE DATABASE command is issued. The AUTOCONFIGURE option is needed only if you want to tweak the recommendations.

**Instructor notes:****Purpose —****Details —**

**Additional information —** AUTOCONFIGURE — Based on user input, calculates the recommended settings for buffer pool size, database configuration, and database manager configuration, and optionally applies them. USING input-keyword param-value — mem\_percent, workload\_type, num\_stmts, tpm, admin\_priority, num\_local\_apps, num\_remote\_apps, isolation, bp\_resizeable. Addressed in CF41/CF44.

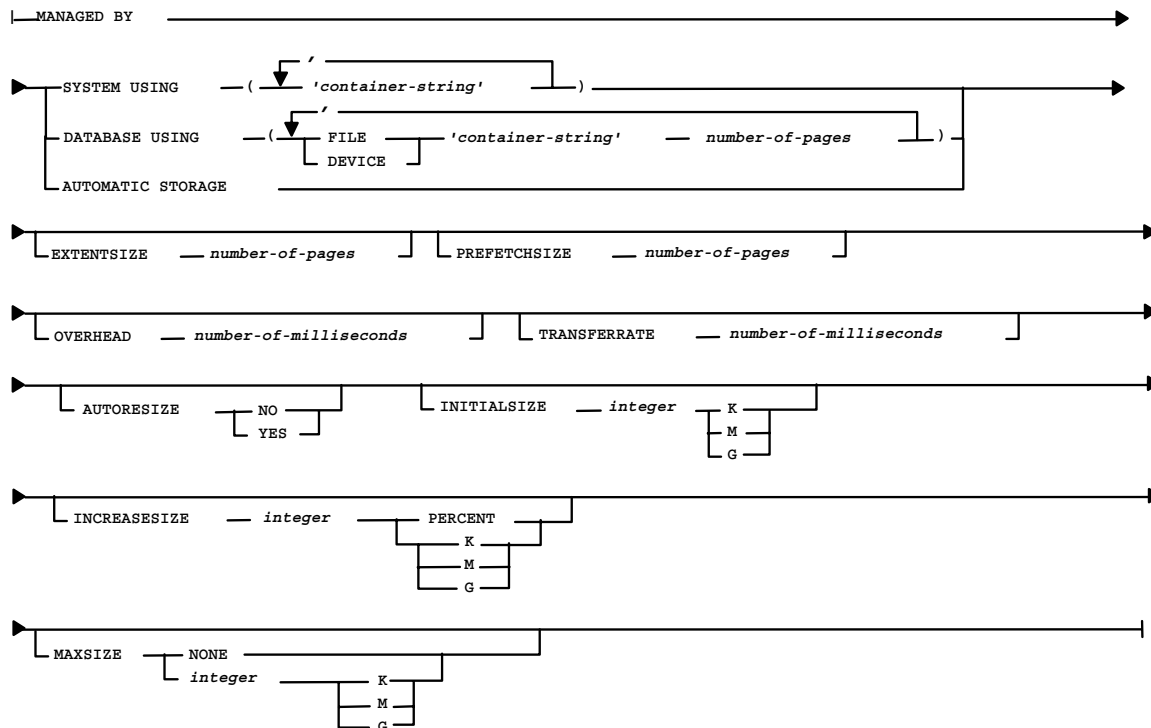
Certification objective: 1K Ability to use self-tuning memory management and autoconfig.

Certification objective: 2A Ability to create a database.

**Transition statement —** Let's look at more of the database syntax diagram.

## Create database syntax (3 of 3)

### tblspace-defn:



© Copyright IBM Corporation 2007

Figure 4-7. Create database syntax (3 of 3)

CF238.3

### Notes:

Part of the syntax of the CREATE DATABASE statement relation to the table space definitions is explained here, referencing the syntax diagram in the visual.

#### MANAGED BY

##### SYSTEM

Specifies that the table space is to be an SMS table space. When the type of table space is not specified, the default behavior is to create a regular table space.

##### DATABASE

Specifies that the table space is to be a DMS table space. When the type of table space is not specified, the default behavior is to create a large table space.

##### AUTOMATIC STORAGE

Specifies that the table space is to be an automatic storage table space. If automatic storage is not defined for the database, an error is returned (SQLSTATE 55060).



An automatic storage table space is created as either a system managed space (SMS) table space or a database managed space (DMS) table space. When DMS is chosen and the type of table space is not specified, the default behavior is to create a large table space. With an automatic storage table space, the database manager determines which containers are to be assigned to the table space, based upon the storage paths that are associated with the database.

## **EXTENTSIZE**

Specifies the number of PAGESIZE pages that will be written to a container before skipping to the next container. The extent size value can also be specified as an integer value followed by K (for kilobytes) or M (for megabytes). If specified in this way, the floor of the number of bytes divided by the page size is used to determine the value for the extent size. The database manager cycles repeatedly through the containers as data is stored.

The default value is provided by the DFT\_EXTENT\_SZ database configuration parameter, which has a valid range of 2-256 pages.

## **PREFETCHSIZE**

Specifies to read in data needed by a query prior to it being referenced by the query, so that the query need not wait for I/O to be performed.

The default value is provided by the DFT\_PREFETCH\_SZ database configuration parameter.

## **OVERHEAD**

Specifies the I/O controller overhead and disk seek and latency time. This value is used to determine the cost of I/O during query optimization. The value of number-of-milliseconds is any numeric literal (integer, decimal, or floating point). If this value is not the same for all containers, the number should be the average for all containers that belong to the table space.

For a database that was created in Version 9 or later, the default I/O controller overhead and disk seek and latency time is 7.5 milliseconds. For a database that was migrated from a previous version of DB2 to Version 9 or later, the default is 12.67 milliseconds.

## **TRANSFERRATE**

Specifies the time to read one page into memory. This value is used to determine the cost of I/O during query optimization. The value of number-of-milliseconds is any numeric literal (integer, decimal, or floating point). If this value is not the same for all containers, the number should be the average for all containers that belong to the table space.

For a database that was created in Version 9 or later, the default time to read one page into memory is 0.06 milliseconds. For a database that was migrated from a previous version of DB2 to Version 9 or later, the default is 0.18 milliseconds.

## AUTORESIZE

Specifies whether or not the auto-resize capability of a DMS table space or an automatic storage table space is to be enabled. Auto-resizeable table spaces automatically increase in size when they become full. The default is NO for DMS table spaces and YES for automatic storage table spaces.

## INITIALSIZE

Specifies the initial size, per database partition, of an automatic storage table space. This option is only valid for automatic storage table spaces. The integer value must be followed by K (for kilobytes), M (for megabytes), or G (for gigabytes). Note that the actual value used might be slightly smaller than what was specified, because the database manager strives to maintain a consistent size across containers in the table space. Moreover, if the table space is auto-resizeable and the initial size is not large enough to contain meta-data that must be added to the new table space, DB2 will continue to extend the table space by INCREASESIZE until there is enough space. If the table space is auto-resizeable, but the INITIALSIZE clause is not specified, the database manager determines an appropriate value.

## INCREASESIZE

Specifies the amount, per database partition, by which a table space that is enabled for auto-resize will automatically be increased when the table space is full, and a request for space has been made. The integer value must be followed by:

- **PERCENT** to specify the amount as a percentage of the table space size at the time that a request for space is made. When PERCENT is specified, the integer value must be between 0 and 100 (SQLSTATE 42615).
- **K** (for kilobytes), **M** (for megabytes), or **G** (for gigabytes) to specify the amount in bytes.



### Note

Note that the actual value used might be slightly smaller or larger than what was specified, because the database manager strives to maintain consistent growth across containers in the table space. If the table space is auto-resizeable, but the INCREASESIZE clause is not specified, the database manager determines an appropriate value.

## MAXSIZE

Specifies the maximum size to which a table space that is enabled for auto-resize can automatically be increased. If the table space is auto-resizeable, but the MAXSIZE clause is not specified, the default is NONE.

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —** Certification objective: 2A Ability to create a database.

**Transition statement —** Let's check Automatic storage for a database.

## Automatic storage — database

- 
- Intended as a *single point of storage management* for table spaces
  - Create a database and associate a set of storage paths with it
    - Create AUTOMATIC STORAGE table spaces
    - No explicit container definitions are provided
    - Containers automatically created across the storage paths
    - Growth of existing containers and addition of new ones managed by DB2
  - Add storage paths to the database afterwards
  - Redefine those storage paths during a database restore

© Copyright IBM Corporation 2007

Figure 4-8. Automatic storage — database

CF238.3

### **Notes:**

By creating a database with AUTOMATIC STORAGE, the need for constant monitoring of space is eliminated. If maintenance needs to be performed on the storage spaces, storage paths can be added as needed.

***Instructor notes:*****Purpose —****Details —****Additional information —** Certification objective: 2F Ability to create and discuss the characteristics of an automated managed table space.**Transition statement —** Let's check the syntax for Automatic storage for a database.

## Automatic storage syntax — database

### Automatic storage databases

- Created as AUTOMATIC STORAGE YES by default (only on non-DPF servers)
- Disable automatic storage during database creation explicitly

Examples of automatic storage being disabled explicitly:

Examples of automatic storage being enabled either explicitly or implicitly:

```
CREATE DATABASE ASNODB1 AUTOMATIC STORAGE NO  
CREATE DATABASE ASNODB2 AUTOMATIC STORAGE NO ON X:
```

```
CREATE DATABASE DB1  
CREATE DATABASE DB2 AUTOMATIC STORAGE YES ON X:  
CREATE DATABASE DB3 ON /data/path1, /data/path2  
CREATE DATABASE DB4 ON D:\StoragePath DBPATH ON C:
```

© Copyright IBM Corporation 2007

Figure 4-9. Automatic storage syntax — database

CF238.3

### Notes:

By default, automatic storage is enable when creating a database. To disable this feature, set AUTOMATIC STORAGE to NO.

***Instructor notes:***

**Purpose** —

**Details** —

**Additional information** —

**Transition statement** — Let's view a few examples of creating databases with automatic storage.

## Automatic storage — create database examples

### Examples:

CREATE DATABASE TESTDB1

- Automatic storage enabled: **Yes**
- Database and Storage path: **dftdbpath**

CREATE DATABASE TESTDB2

- ON /testdb2
- Automatic storage enabled: **Yes**
- Database and Storage path: **/testdb2**

CREATE DATABASE TESTDB3  
AUTOMATIC STORAGE YES

- Automatic storage enabled: **Yes**
- Database path: **dftdbpath**
- Storage path: **dftdbpath**

CREATE DATABASE TESTDB4  
AUTOMATIC STORAGE YES ON /dbdir

- Automatic storage enabled: **Yes**
- Database path: **/dbdir**
- Storage path: **/dbdir**

CREATE DATABASE TESTDB5  
ON /db2/dir1,/db2/dir2,/db2/dir3

- Automatic storage enabled: **Yes**
- Database path: **/db2/dir1**
- Storage paths: **/db2/dir1, /db2/dir2, /db2/dir3**

CREATE DATABASE TESTDB6  
ON D: \AS\_PATH DBPATH ON C:

- Automatic storage enabled: **Yes**
- Database path: **C:**
- Storage path: **D:\AS\_PATH**

© Copyright IBM Corporation 2007

Figure 4-10. Automatic storage — create database examples

CF238.3

### Notes:

Examples of creating databases with and without automatic storage are shown above:

- **TESTDB1** — Set to automatic storage, and uses a default database path
- **TESTDB2** — Set to automatic storage, but uses a specified database path
- **TESTDB3** — Set to automatic storage, using the default database path and a default storage path
- **TESTDB4** — Set to automatic storage, using the same specified database path and storage path
- **TESTDB5** — Set to automatic storage, using the specified database path and specified storage paths
- **TESTDB6** — Set to automatic storage, using the specified database path and a different specified storage path



***Instructor notes:*****Purpose —****Details —****Additional information —**

**Transition statement —** Next, we check out the results of storage locations when using automatic storage.

## Automatic Storage provisioning — syntax

```

1. CREATE DATABASE DB1
    AUTOMATIC STORAGE YES

2. CREATE DATABASE DB3
    AUTOMATIC STORAGE YES
    ON 'C:\', 'C:\DB2\data\path1',
    'C:\DB2\data\path2'
    DBPATH ON 'C:\'

3. CREATE TABLESPACE TS2
    MANAGED BY AUTOMATIC STORAGE

4. CREATE TEMPORARY TABLESPACE TEMPTS

5. CREATE USER TEMPORARY TABLESPACE USRTMP
    MANAGED BY AUTOMATIC STORAGE

6. CREATE TABLESPACE TS1
    INITIALSIZE 500 K
    INCREASESIZE 100 K
    MAXSIZE 100 M

```



© Copyright IBM Corporation 2007

Figure 4-11. Automatic Storage provisioning — syntax

CF238.3

### Notes:

In the information shown above, the first statement is the simplest, declaring AUTOMATIC STORAGE for the whole database.

The following statements provide a variety of syntax for both databases and table spaces, and paths created are shown in the table.

**Table 1: Example storage paths**

|   | Table space  | Path                         | Type             |
|---|--------------|------------------------------|------------------|
| 1 | SYSCATSPACE  | C:\DB2\NODE0000\DB1\T0000000 | Regular          |
|   | TEMPSPACE1   | C:\DB2\NODE0000\DB1\T0000001 | System temporary |
|   | USERSPACE1   | C:\DB2\NODE0000\DB1\T0000002 | Large            |
|   | SYSTOOLSPACE | C:\DB2\NODE0000\DB1\T0000003 | Large            |

**Table 1: Example storage paths**

|   |             |                                             |                  |
|---|-------------|---------------------------------------------|------------------|
| 2 | SYSCATSPACE | C:\DB2\NODE0000\DB3\T0000000                | Regular          |
|   |             | C:\DB2\data\path1\DB2\NODE0000\DB3\T0000000 |                  |
|   |             | C:\DB2\data\path2\DB2\NODE0000\DB3\T0000000 |                  |
|   | TEMPSPACE1  | C:\DB2\NODE0000\DB3\T0000001                | System temporary |
|   |             | C:\DB2\data\path1\DB2\NODE0000\DB3\T0000001 |                  |
|   |             | C:\DB2\data\path2\DB2\NODE0000\DB3\T0000001 |                  |
|   | USERSPACE1  | C:\DB2\NODE0000\DB3\T0000002                | Large            |
|   |             | C:\DB2\data\path1\DB2\NODE0000\DB3\T0000002 |                  |
|   |             | C:\DB2\data\path2\DB2\NODE0000\DB3\T0000002 |                  |
| 3 | TS2         | C:\DB2\NODE0000\SAMPLE\T0000006             | Large            |
| 4 | TEMPTS      | C:\DB2\NODE0000\SAMPLE\T0000008             | System temporary |
| 5 | UDRTMP      | C:\DB2\NODE0000\SAMPLE\T0000009             | User temporary   |
| 6 | TS1         | C:\DB2\NODE0000\SAMPLE\T0000010             | Large            |

Note that in the second example, several locations are used for storage:

C:\DB2\NODE0000\DB3, C:\DB2\data\path1\DB2\NODE0000\DB3, and C:\DB2\data\path2\DB2\NODE0000\DB3.

***Instructor notes:***

**Purpose —**

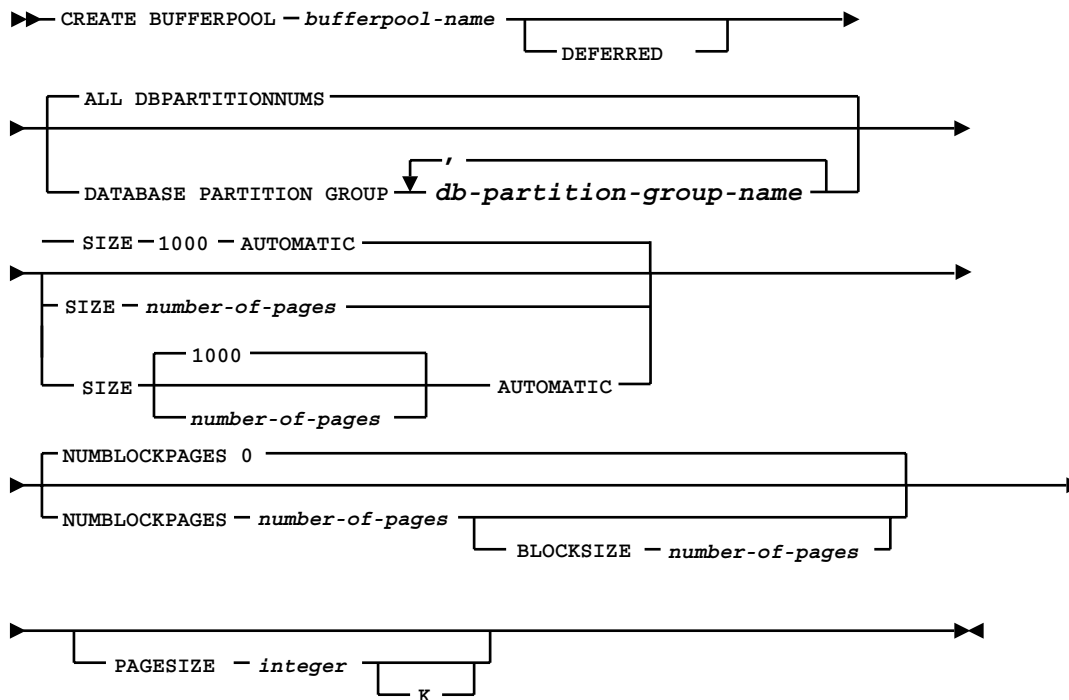
**Details —**

**Additional information —**

**Transition statement —** Let's look at the syntax needed for creating buffer pools.

## Create bufferpool syntax

- Requires SYSCTRL or SYSADM authority



© Copyright IBM Corporation 2007

Figure 4-12. Create bufferpool syntax

CF238.3

### Notes:

The CREATE BUFFERPOOL statement creates a new buffer pool to be used by the database manager.

Once a buffer pool has been created, it can be assigned to a table space using the ALTER TABLESPACE command. When creating a table space, a buffer pool can be assigned to the table space.

The authorization ID of the statement must have SYSCTRL or SYSADM authority.

***Instructor notes:***

**Purpose** —

**Details** —

**Additional information** —

**Transition statement** — Next, a quick view of what the various default table spaces and buffer pool are used for.

## Default table spaces and buffer pool

- Default table spaces
  - SYSCATSPACE
    - Stores system catalog tables and views
  - TEMPSPACE1
    - Used by the system to store temporary activity, like sorts
  - USERSPACE1
    - First table space for users to create data and index structures
- Default buffer pool
  - IBMDEFAULTBP
    - Created when the database is created, and used by data from all default table spaces

© Copyright IBM Corporation 2007

Figure 4-13. Default table spaces and buffer pool

CF238.3

### Notes:

When the database is initially created, a set of three table spaces will be defined:

- SYSCATSPACE for the system catalog tables
- USERSPACE1 for user-defined tables
- TEMPSPACE1 for temporary work area

If you do not specify any table space parameters on the CREATE DATABASE command, the database manager will create DMS table spaces for SYSCATSPACE and USERSPACE1, and SMS table space for TEMPSPACE1. The default extent size for SYSCATSPACE is 4 pages, and 32 pages for TEMPSPACE1 and USERSPACE1.

## ***Instructor notes:***

**Purpose —**

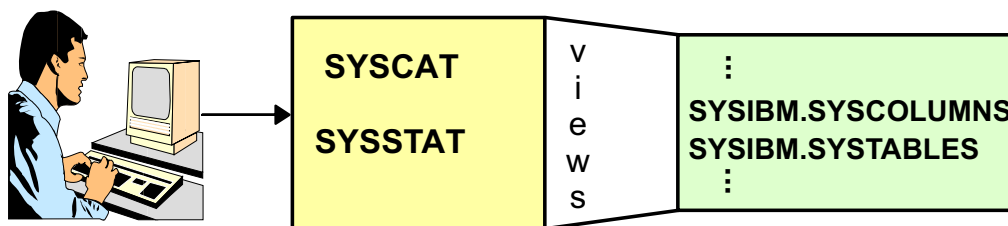
**Details —**

**Additional information —** Certification objective: 2E Ability to create and discuss the characteristics of an SMS table space.

**Transition statement —** Let's look at the catalog tables and views.



## System Catalog tables and views



© Copyright IBM Corporation 2007

Figure 4-14. System Catalog tables and views

CF238.3

### Notes:

A set of system catalog tables is created and maintained for each database. These tables contain information about the definitions of the database objects and also security information about the type of access users have to these objects.

The database manager creates and maintains two sets of catalog views. All of the system catalog views are created when a database is created with the CREATE DATABASE command. The catalog views cannot be explicitly created or dropped. The views are within the SYSCAT schema and select privilege on all views is granted to PUBLIC by default. A second set of views formed from a subset of those within the SYSCAT schema contains statistical information used by the SQL optimizer. The views within the SYSSTAT schema contain some updatable columns.

**Table 3-1. Catalog Views as documented in *SQL Reference***

| <b>Catalog View</b>           | <b>Description</b>                                    |
|-------------------------------|-------------------------------------------------------|
| SYSCAT.ATTRIBUTES             | attributes of structured data types                   |
| SYSCAT.BUFFERPOOLDBPARTITIONS | buffer pool size on database partition                |
| SYSCAT.BUFFERPOOLS            | buffer pool configuration on database partition group |
| SYSCAT.CASTFUNCTIONS          | cast functions                                        |
| SYSCAT.CHECKS                 | check constraints                                     |
| SYSCAT.COLAUTH                | column privileges                                     |
| SYSCAT.COLCHECKS              | columns referenced by check constraints               |
| SYSCAT.COLDIST                | detailed column options                               |
| SYSCAT.COLGROUPCOLS           | detailed column group columns                         |
| SYSCAT.COLGROUPS              | detailed column group statistics                      |
| SYSCAT.COLIDENTATTRIBUTES     | attributes of identity column                         |
| SYSCAT.COLOPTIONS             | detailed column options                               |
| SYSCAT.COLUMNS                | columns                                               |
| SYSCAT.COLUSE                 | columns used in dimensions                            |
| SYSCAT.CONSTDEP               | constraint dependencies                               |
| SYSCAT.DATATYPES              | data types                                            |
| SYSCAT.DBAUTH                 | authorities on database                               |
| SYSCAT.DBPARTITIONGROUPDEF    | database partition group database partitions          |
| SYSCAT.DBPARTITIONGROUPS      | database partition group definitions                  |
| SYSCAT.EVENTMONITORS          | event monitor definitions                             |
| SYSCAT.EVENTS                 | events currently monitored                            |
| SYSCAT.EVENTTABLES            | events currently monitored                            |
| SYSCAT.FULLHIERARCHIES        | hierarchies (types, tables, views)                    |
| SYSCAT.FUNCMAPOPTIONS         | function mapping options                              |
| SYSCAT.FUNCMAPPARAMOPTIONS    | function mapping parameter options                    |
| SYSCAT.FUNCMAPPINGS           | function mapping                                      |
| SYSCAT.HIERARCHIES            | hierarchies (types, tables, views)                    |
| SYSCAT.INDEXAUTH              | index privileges                                      |
| SYSCAT.INDEXCOLUSE            | index columns                                         |
| SYSCAT.INDEXDEP               | index dependencies                                    |
| SYSCAT.INDEXES                | indexes                                               |
| SYSCAT.INDEXEXPLOITRULES      | index exploitation                                    |
| SYSCAT.INDEXEXTENSIONDEP      | index extension dependencies                          |

**Table 3-1. Catalog Views as documented in *SQL Reference***

|                              |                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------|
| SYSCAT.INDEXEXTENSIONMETHODS | index extension search methods                                                      |
| SYSCAT.INDEXEXTENSIONPARMS   | index extension parameters                                                          |
| SYSCAT.INDEXEXTENSIONS       | index extensions                                                                    |
| SYSCAT.INDEXOPTIONS          | index options                                                                       |
| SYSCAT.KEYCOLUSE             | columns used in keys                                                                |
| SYSCAT.NAMEMAPPINGS          | object mapping                                                                      |
| SYSCAT.PACKAGEAUTH           | package privileges                                                                  |
| SYSCAT.PACKAGEDEP            | package dependencies                                                                |
| SYSCAT.PACKAGES              | packages                                                                            |
| SYSCAT.PARTITIONMAPS         | partitioning maps                                                                   |
| SYSCAT.PASSTHRUAUTH          | pass-through privileges                                                             |
| SYSCAT.PREDICATESPEC         | predicate specifications                                                            |
| SYSCAT.PROCOPTIONS           | procedure options                                                                   |
| SYSCAT.PROCPARMOPTIONS       | procedure parameter options                                                         |
| SYSCAT.REFERENCES            | referential constraints                                                             |
| SYSCAT.REVTYPEMAPPINGS       | reverse data type mapping                                                           |
| SYSCAT.ROUTINEAUTH           | routine privileges                                                                  |
| SYSCAT.ROUTINEDEP            | function, method, routine dependencies                                              |
| SYSCAT.ROUTINEPARMS          | function, method, procedure, routine parameters                                     |
| SYSCAT.ROUTINES              | functions, methods, procedures, routines, stored procedures, user-defined functions |
| SYSCAT.SCHEMAAUTH            | schema privileges                                                                   |
| SYSCAT.SCHEMATA              | schemas                                                                             |
| SYSCAT.SEQUENCEAUTH          | sequence privileges                                                                 |
| SYSCAT.SEQUENCES             | sequences                                                                           |
| SYSCAT.SERVEROPTIONS         | server options                                                                      |
| SYSCAT.SERVERS               | system servers                                                                      |
| SYSCAT.STATEMENTS            | statements in packages                                                              |
| SYSCAT.TABAUTH               | table privileges                                                                    |
| SYSCAT.TABCONST              | table constraints                                                                   |
| SYSCAT.TABDEP                | table dependencies                                                                  |
| SYSCAT.TABLES                | tables                                                                              |
| SYSCAT.TABLESPACES           | table spaces                                                                        |
| SYSCAT.TABOPTIONS            | remote table options                                                                |
| SYSCAT.TBSPACEAUTH           | table spaces use privileges                                                         |
| SYSCAT.TRANSFORMS            | transforms                                                                          |

**Table 3-1. Catalog Views as documented in *SQL Reference***

|                     |                                           |
|---------------------|-------------------------------------------|
| SYSCAT.TRIGDEP      | trigger dependencies                      |
| SYSCAT.TRIGGERS     | triggers                                  |
| SYSCAT.TYPEMAPPINGS | type mapping                              |
| SYSCAT.USEROPTIONS  | server options values                     |
| SYSCAT.VIEWS        | views                                     |
| SYSCAT.WRAPOPTIONS  | wrapper options                           |
| SYSCAT.WRAPPERS     | wrappers                                  |
| SYSIBM.SYSDUMMY1    | provides DB2 UDB for OS/390 compatibility |

**Table 3-2. Updatable Catalog Views as documented in *SQL Reference***

| Catalog View      | Description                      |
|-------------------|----------------------------------|
| SYSSTAT.COLDIST   | detailed column statistics       |
| SYSSTAT.COLGROUPS | detailed column group statistics |
| SYSSTAT.COLUMNS   | columns                          |
| SYSSTAT.INDEXES   | indexes                          |
| SYSSTAT.ROUTINES  | routines                         |
| SYSSTAT.TABLES    | tables                           |

**Instructor notes:**

**Purpose** — Identify the system catalog tables and two sets of views.

**Details** — When the database is created, DB2 creates the system catalog tables which contain all of the information about objects defined in the database. The system catalog tables exist “behind the scenes”; there is no documentation about the tables themselves, though you could see them if you query SYSCAT.TABLES. The “real” tables have names like SYSIBM.SYSCOLUMNS and SYSIBM.SYSTABLES. However, you are not to access the system catalog through these definitions. Instead, a set of views are supplied to allow you to access the underlying system catalog information. The SYSCAT views are read-only views; the SYSSTAT views allow access to a subset of the catalog: specifically those tables and columns that carry statistical information. With the SYSCAT views, you can access all the necessary system catalog information. With the SYSSTAT views, you have an option to be able to manually update statistics if necessary (for example, if you are mimicking a production environment on your test system).

**Additional Information** — Documentation for the SYSCAT and SYSSTAT tables can be found in an Appendix in the *SQL Reference*.

Certification objective: 3D Skill in examining the contents of the System Catalog tables.

**Transition statement** —



## 4.2 Table spaces and containers

### Instructor topic introduction

**What students will do** — Learn the high-level differences between SMS and DMS table spaces.

**How students will do it** — Listen to the lecture.

**What students will learn** — The difference (high-level) between SMS and DMS table spaces.

**How this will help students on their job** — They will have an introductory understanding of the difference between SMS and DMS table spaces.

## DMS versus SMS table spaces

- SMS - Database Manager manages data using the operating system
- DMS - Managed by Database Manager directly



© Copyright IBM Corporation 2007

Figure 4-15. DMS versus SMS table spaces

CF238.3

### Notes:

Table spaces are a logical level between the database and the tables stored in that database. There are two types of table spaces that can be created, SMS and DMS. A database can have any combination of SMS and DMS table spaces.

In an **SMS (System Managed Space)** table space, the operating system's file system manager allocates and manages the space where the table is stored. The storage model typically consists of many files, representing table objects, stored in the file system space. The user decides on the location of the files; DB2 controls their names; and the file system is responsible for managing them. By controlling the amount of data written to each file, the database manager distributes the data evenly across the table space containers. By default, the initial table spaces created at database creation time are SMS.

In a **DMS (Database Managed Space)** table space, the database manager controls the storage space. The storage model consists of a limited number of devices or files whose space is managed by DB2. The database administrator decides which devices and files to use, and DB2 manages the space on those devices and files. The table space is essentially an implementation of a special-purpose file system designed to best meet the needs of the database manager.



In DB2 9, auto growth and Automatic Storage are enabled as default.

### **Auto growth**

A table space will auto-extend when it is full and more space is needed. However, only those containers that are part of the last range in the table space map will grow.

This ensures that a rebalance will never take place as part of an autoresize.

- Autoresize is a characteristic of DMS tablespaces, which also applies to the REGULAR and LARGE tablespaces using AUTOMATIC STORAGE.

Auto-growth will stop when any of the following happen:

- The value specified for MAXSIZE is reached -or-
- DB2's table space size limit has been reached (if MAXSIZE is NONE) -or-
- One of the containers in the last range cannot grow any further
  - DB2 will not extend the others in the last range

To continue growth, you can add space to the file system, or add a new stripe set.

### **Automatic Storage**

In DB2 9, automatic storage support is added for multi-partition databases. In addition, the Control Center has been enhanced to let you create databases that use automatic storage, and to enable you to add storage paths to existing databases.

By creating a database with AUTOMATIC STORAGE, the need for constant monitoring of space is eliminated. If maintenance needs to be performed on the storage spaces, storage paths can be added as needed.

### **Default table spaces**

The type of table space for the three default table spaces has changed with DB2 9.

The Syscatspace and Userspace1 table spaces are created as DMS table spaces by default, while the TempSPACE1 table space is created as SMS.

The default page size for these table spaces is 4 K.

Default storage paths (Windows):

- Syscatspace — C:\DB2\NODE0000\SAMPLE\T0000000 (DMS-file)
- Userspace1 — C:\DB2\NODE0000\SAMPLE\T0000002 (DMS-file)
- TempSPACE1 — C:\DB2\NODE0000\SAMPLE\T0000001 (SMS-folder)

## ***Instructor notes:***

**Purpose** — Define containers.

**Details** — When a table space is created, you will specify the containers to be associated with it. The containers are the mapping to an object in the operating system.

Containers may be files, directories, or devices.

Recall that DMS table spaces take space away from the operating system and hand it over to DB2 to manage. Do directories represent an allocation of so many pages of physical space? No. So, directories are not a valid container type with DMS table spaces; directories are used with SMS table spaces. DMS table spaces require file or device containers, representing an allocation of physical space reserved for this table space.

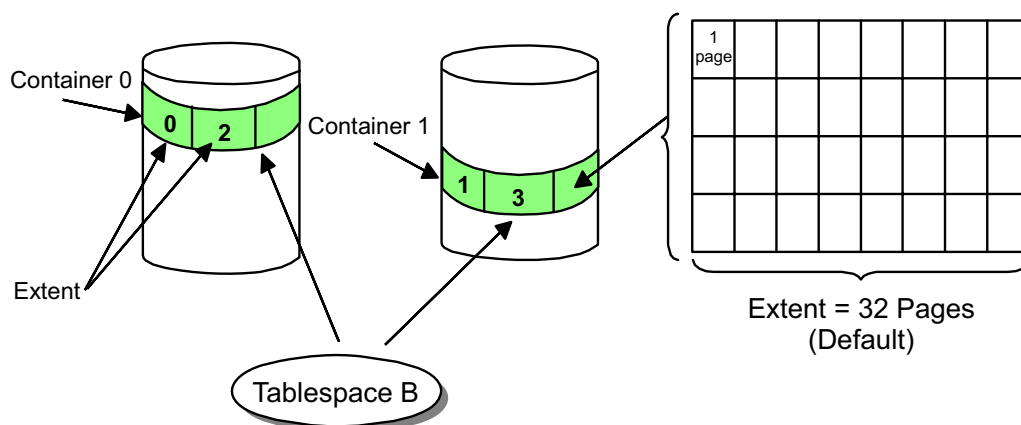
**Additional Information** — Note the GUID information provided in the student notes for Windows 2000 and above.

Certification objective: 2E Ability to create and discuss the characteristics of an SMS table space.

**Transition Statement** — Let's look at how DB2 will write to the containers if more than one is provided.

## Writing to containers

- DFT\_EXTENT\_SZ Defined at database level
- EXTENTSIZE Defined at table space level
- Data written in round-robin manner



© Copyright IBM Corporation 2007

Figure 4-16. Writing to containers

CF238.3

### Notes:

An extent is an allocation of contiguous space within a container of a table space. Extents are allocated to a single database object and consist of multiple pages. The default extent size is 32 pages, but a different value can be specified when creating a table space.

Data for an object is stored on a container by extents. When data for an object is written, the database system stripes the data across all the containers in the table space based on the extent size.

DFT\_EXTENT\_SZ is a database configuration parameter. If you do not specify an extent size when the table space is created, DFT\_EXTENT\_SZ will be used. The default size for DFT\_EXTENT\_SZ is 32 pages. If you do not alter this value or explicitly indicate an extent size when you create a table space, all your table spaces within the database will have this default value. The range of values for DFT\_EXTENT\_SZ is between two and 256 pages.

You can specify extent size at the table space level. This can be done at table space creation with the parameter EXTENTSIZE. Carefully determine the correct size, as once it is set for a table space, it cannot be altered. This size can have an impact on space utilization and performance.

The database manager will try to evenly distribute the table among containers. In doing so, the database manager writes an extent of pages to each container before writing to the next container. Once the database manager has written an extent to all the containers allocated to a table space, it will write the next extent to the first container written to in that table space. This round-robin process of writing to the containers is designed to balance the workload across the containers of the table space.

**Instructor notes:**

**Purpose** — Identify how the extent size is determined, and its impact on writing to containers.

**Details** — When a table space is created, its extent size is determined either explicitly or implicitly. If no extent size is provided on the CREATE TABLE SPACE statement, then the value of DFT\_EXTENT\_SZ in the database configuration is used. The default value of DFT\_EXTENT\_SZ is 32 pages. You can also explicitly provide an extent size on the creation of a table space.

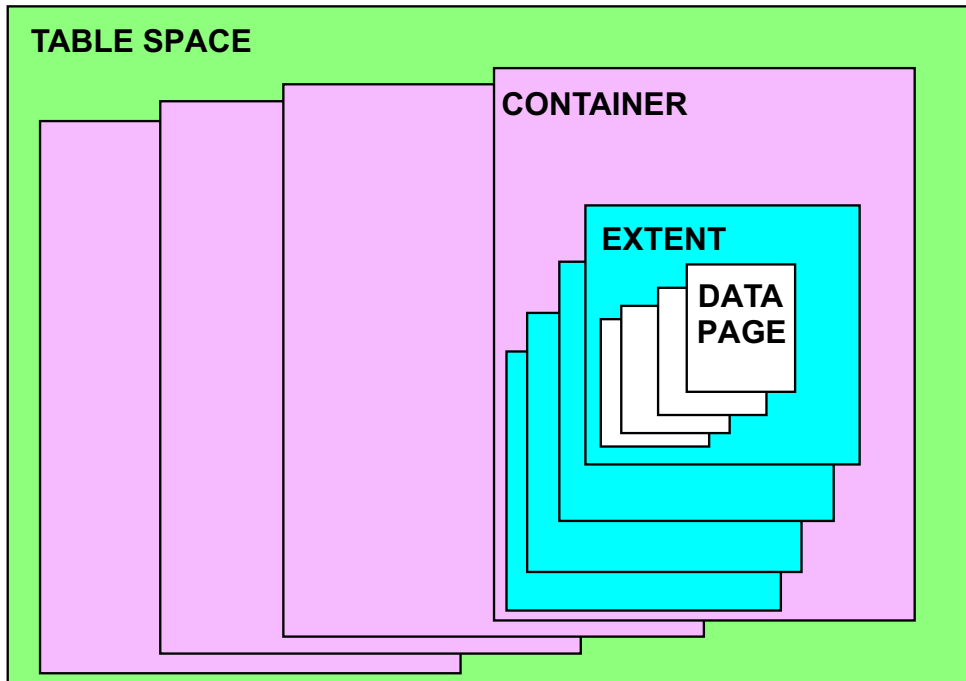
The impact of the extent size of a table space is this: when a large insert of data is being done on the table space, DB2 will write one extent of data to the first container of the table space, then the next extent of data is written to the next container of the table space, and then to the next, until each container has one extent of the table's data. Then DB2 will wrap around and write to the first container again. In this way, the table's data will be spread across as many containers as possible, thus giving the potential for a performance advantage on a table scan, as DB2 can use parallel I/O.

**Additional Information** — To help distribute data across the containers more evenly, the database manager determines which container to use first by taking the table identifier modulo the number of containers. Containers are numbered sequentially, starting at 0.

Certification objective: 2E Ability to create and discuss the characteristics of an SMS table space.

**Transition Statement** — Let's look at the relationship between table spaces, containers, extents, and data pages.

## Table space, container, extent, page



© Copyright IBM Corporation 2007

Figure 4-17. Table space, container, extent, page

CF238.3

### Notes:

A table space is made up of containers.

A container is made up of extents.

An extent is made up of data pages.

***Instructor notes:***

**Purpose** — Show a graphical representation of the relationship between table spaces, containers, extents, and data pages.

**Details** — Use this page to make sure students understand the relationships between these four objects.

**Additional Information** —

**Transition Statement** — Because DMS table spaces must manage the storage within the containers more directly, some overhead is required. Let's look at the minimum requirements for DMS table spaces.

## DMS table space — minimum requirements

|                                                                                                 |                        |                        |
|-------------------------------------------------------------------------------------------------|------------------------|------------------------|
| Table Space Creation                                                                            | container tag          | 1 extent per container |
| table space meta data                                                                           | header for table space | 1 extent               |
|                                                                                                 | space map              | 1 extent               |
|                                                                                                 | object table data      | 1 extent               |
|                                                                                                 |                        | 4 extents              |
| <hr/>                                                                                           |                        |                        |
| Table (Object) Creation                                                                         | extent map             | 1 extent               |
|                                                                                                 | extent for data        | 1 extent               |
| Minimum space required in containers =<br>(for first object)                                    |                        | 6 extents              |
| If EXTENTSIZE = 32 pages, then $6 \times 32 = 192$ pages<br>needed to be provided in containers |                        |                        |

© Copyright IBM Corporation 2007

Figure 4-18. DMS table space — minimum requirements

CF238.3

### Notes:

All indexes for a table are considered to be a single object.

A certain amount of overhead is required in a DMS table space. The space required is:

- One extent per container for a container tag
- Three extents per table space for the table space meta data
- Two initial extents per object in the table space: one for the extent map for the object, one for the data for the object.

When a DMS table space is created, there must be enough space in the containers for six extents of the table space, otherwise the table space will not be created.

**Note:** Once DB2 removes the one extent per container for the container tag, it will make the number of usable pages a multiple of the extent size. So, for optimal space utilization, allocate an even multiple of the extent size in pages in each container so no space is wasted.



**Instructor notes:**

**Purpose** — Identify the overhead required for management of DMS table spaces.

**Details** — Because DMS table spaces are managed by DB2, they require some additional space for keeping track of the information stored within them. DB2, in V8, requires a full extent per container for the container tag (previous versions required a page). Three extents are required for the table space meta data. For storing a simple table object, two additional extents are required.

When creating a table space, you need to think about the overhead required. This is usually not a big issue when you are creating table spaces for production tables (you usually are not sizing so close to what is absolutely required). But, when you are first starting out with DB2, you may be playing around, and trying to create a DMS table space, and you are getting `SQL0289N Unable to allocate new pages in table space`. This indicates that you have not allocated enough space in the containers to store the overhead, and the two extents for the table data. A total of six extents must be allocated in the table space (and that is if there is only one container; each container requires an extent for the container tag). With the default extent size of 32 pages, a minimum of 192 pages must be allocated in a single container to successfully create the table space.

To ensure this makes sense, ask the students, out of that 192 pages, how many pages are available for storing table data? The answer is 32 pages. Now, you would never really create a table space with this size characteristic, but it is good to understand the calculation required.

**Additional Information** — Registry variable `DB2_USE_PAGE_CONTAINER_TAG=YES` will cause DB2 to use just a page for the container tag. If you have existing table spaces and a new container is added, in V8, DB2 will use a full extent by default.

Note also that the table space meta data requirements are the initial allocation requirements. Additional table space meta data may be required as the data in the table space grows (for example, there may need to be more space map pages).

**Transition statement** — Let's look at what you can specify when defining table spaces.

## Defining table spaces

- 
- When defining a table space, you can
    - Specify how the table space will be used
    - Specify the management of the table space
    - Specify containers and their description
    - Specify extent size
    - Specify performance characteristics

© Copyright IBM Corporation 2007

Figure 4-19. Defining table spaces

CF238.3

### Notes:

When defining a table space, the following information can be supplied:

- How the table space will be used: REGULAR, LARGE (DMS only), or TEMPORARY
- The management of the table space: SMS or DMS
- Whether or not to use AUTOMATIC STORAGE
- The type, location, and size (DMS) of the containers
- The extent size
- Performance characteristics, such as buffer pool assignment, overhead, transfer rate, and prefetch size

The next pages will show how you can create table spaces within a database. We will show both SMS and DMS table spaces being created in the *ourdb* database. The examples will illustrate these different ways of creating table spaces. Some values will be allowed to default.

***Instructor notes:***

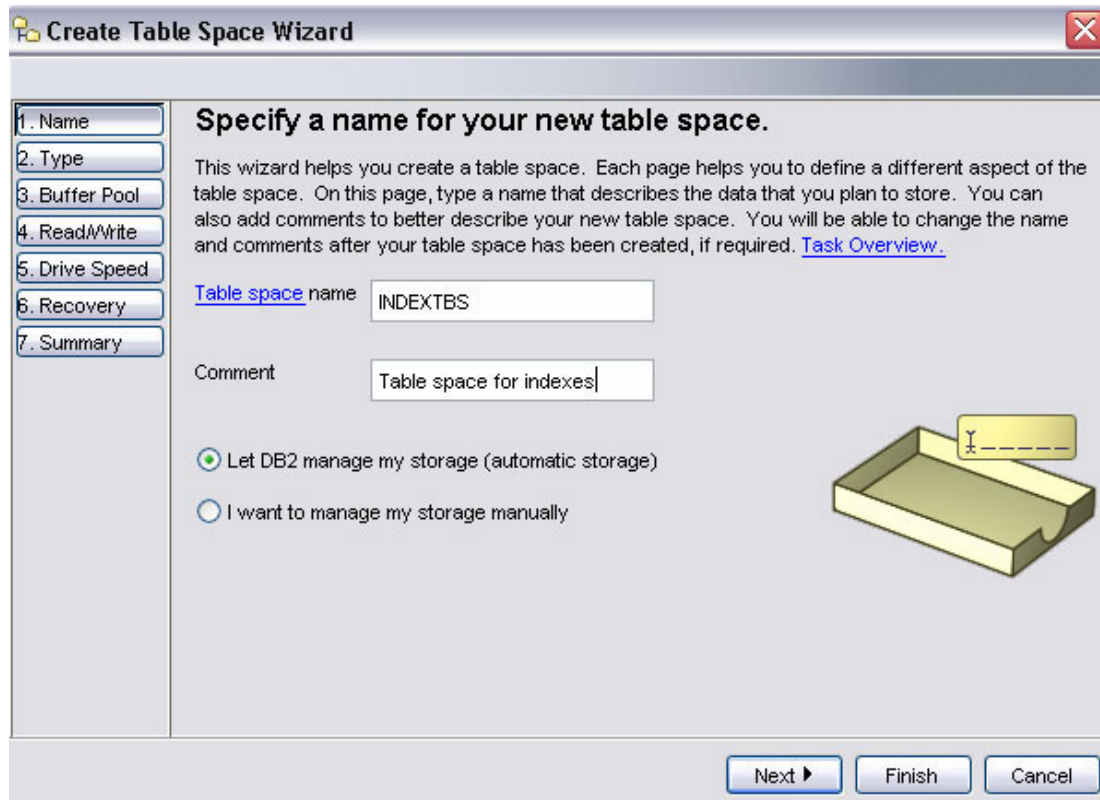
**Purpose** — Identify the characteristics that can be set when the table space is created.

**Details** — We are about to see the syntax for creating table spaces. This page just provides information about the characteristics that can be specified when the table space is created.

**Additional information** —

**Transition statement** — Let's look at how we could create the table space using the Control Center.

# Create Table Space Wizard



© Copyright IBM Corporation 2007

Figure 4-20. Create Table Space Wizard

CF238.3

## Notes:

The Create Table Space Wizard makes creating a table space easy. There are a set of wizard pages that you fill out:

### Name

On this page, you provide a name for your new table space. You can also provide a comment to describe the table space.

### Type

Use this page to select the type of table space you want to create. You can use a regular table space for any type of data except temporary tables. However, you will want a large table space if you are going to store long or LOB data such as images, audio, video, or long text fields. You can also choose to create a table space that can be used by the database manager for temporary tables created during sorts, joins, and other operations, or to create a table space that will be used to store user-defined temporary tables.

## Database Partition Group

This page is only available if you are creating a table space on a partitioned database system. Use this page to select the database partition group that includes the database partitions on which the table space will reside. You can also launch the Create Database Partition Group window if you need to create a new one.

## Buffer Pool

Use this page to specify the name of the buffer pool to be used for tables in this table space. You can choose from a list of existing buffer pools or you can launch the Create Buffer Pool window to create a new one. The page size of the buffer pool must match the page size specified for the table space. The database partition group of the table space must be defined for the buffer pool.

## Space Management

On this page you select how your table space should be managed. In a System Managed Space (SMS) table space, the operating system's file system manager allocates and manages the space where the table is to be stored. The storage model typically consists of many files, representing table objects, stored in the file system space. In a Database Managed Space (DMS) table space, the database manager controls the storage space. The storage model consists of a limited number of devices or files, whose space is managed by DB2.

## Containers

Use this page to identify one or more containers that will belong to the table space and into which the table space's data will be stored. All containers must be unique across all databases; a container can belong to only one table space. The size of the containers can differ; however, optimal performance is achieved when all containers are the same size.

## Read/Write

Based on the information you provide about the average size of a table to be stored in the table space, the wizard will recommend the prefetch size and extent size settings for your table space. You can modify these settings. Note that you can modify the prefetch size after the table space has been created, but you cannot modify the extent size.

## Drive Speed

Use this page to specify the type of system you are working on (for example, server, RAID environment, or storage area network). Based on the information you provide, the wizard will estimate your hard drive specifications, such as the average seek time, rotation speed, and transfer rate. If you know the exact specifications, you can enter them instead of using the wizard's estimates. Based on the specifications, the wizard will calculate values for overhead and transfer.

## **Recovery**

This page allows you to enable dropped table recovery. When this option is enabled, you can recover a dropped table by restoring your table space and rolling forward. If you do not enable this option, you can still recover a dropped table by restoring the entire database and rolling forward.

## **Summary**

This page allows you to review the decisions that you made for your table space, and to view the command that will create the table space.

***Instructor notes:***

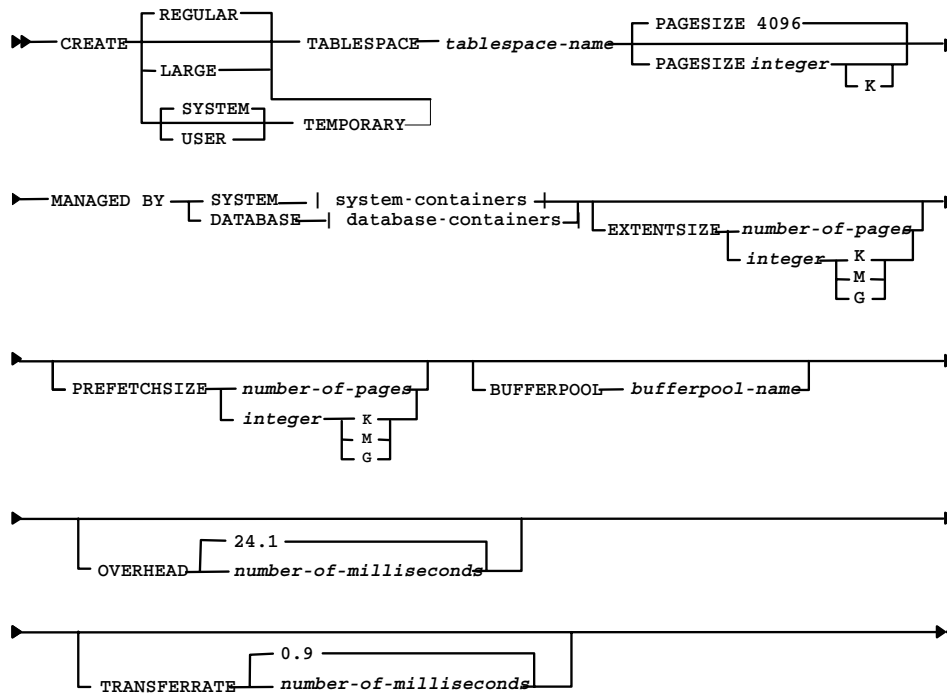
**Purpose** — Introduce the Table Space Wizard.

**Details** — Identify that table spaces can be created through the Create Table Space wizard.

**Additional information** —

**Transition statement** — It is a good idea to understand the Create Table Space syntax even if you will be using the GUI to create the table spaces, as the syntax identifies all of the choices and the interrelationships.

# CREATE TABLESPACE syntax (1 of 2)



© Copyright IBM Corporation 2007

Figure 4-21. CREATE TABLESPACE syntax (1 of 2)

CF238.3

## Notes:

The CREATE TABLESPACE syntax diagram starts on this visual and ends on the next visual.

The CREATE TABLESPACE statement creates a new table space within the database, assigns containers to the table space, and records the table space definition and attributes in the catalog. Only users with SYSADM or SYSCTRL authority can create table spaces.

## REGULAR

Stores all data except for temporary tables.

## LARGE

Stores long or LOB table columns, or indexes. Can only be defined as a DMS table space.

In DB2 9, with large RID support, regular data can be stored in Large table spaces.



## SYSTEM TEMPORARY

Stores temporary tables (work areas used by the database manager to perform operations such as sorts or joins). The keyword **SYSTEM** is optional. A database must always have at least one **SYSTEM TEMPORARY** table space, as temporary tables can only be stored in such a table space. A **SYSTEM TEMPORARY** table space is created when a database is created. A database can have more than one temporary table space. Temporary objects are allocated between the temporary table spaces in a round-robin fashion.

**Recommendation:** Create as few temporary table spaces as possible. It is better to put all of the containers into a single table space rather than dividing them up among multiple table spaces. Any given activity that uses a temporary table space (for example, a sort) will use only one temporary table space. If the temporary table space it uses is too small, the activity will fail.

## USER TEMPORARY

Stores declared global temporary tables. No **USER TEMPORARY** table spaces exist when a database is created. At least one user temporary table space should be created with appropriate **USE** privileges to allow definition of declared temporary tables.

### tablespace-name

Names the table space. This is a one-part name. It can be up to 18 characters in length.

### PAGESIZE

Defines the size of pages used for the table space. Valid values are 4 KB, 8 KB, 16 KB, and 32 KB, or the equivalent integer values.

### EXTENTSIZEnumber-of-pages

Specifies the number of **PAGESIZE** pages that will be written to a container before skipping to the next container. The database manager cycles repeatedly through the containers as data is stored.

The default value is provided by the **DFT\_EXTENT\_SZ** configuration parameter.

### PREFETCHSIZEnumber-of-pages

Specifies the number of **PAGESIZE** pages that will be read from the table space when data prefetching is being performed. Prefetching reads in data needed by a query prior to it being referenced by the query, so the query need not wait for I/O to be performed.

The default value is provided by the **DFT\_PREFETCH\_SZ** configuration parameter.

### BUFFERPOOLbufferpool-name

The name of the buffer pool used for tables in this table space. The buffer pool must exist. If not specified, the default buffer pool, **IBMDEFAULTBP**, is used.

### **OVERHEAD number-of-milliseconds**

Any numeric literal that specifies the I/O controller overhead and disk seek and latency time, in milliseconds. The number should be an average for all containers that belong to the table space, if not the same for all containers.

### **TRANSFERRATE number-of-milliseconds**

Any numeric literal that specifies the time to read one page into memory, in milliseconds. The number should be an average for all containers that belong to the table space, if not the same for all containers.

**Instructor notes:**

**Purpose** — Identify the syntax of the CREATE TABLESPACE statement.

**Details** — Identify at the top that you specify the type of data, the table space, name, page size, whether it is managed by SYSTEM or DATABASE, and its extentsize, either explicitly or implicitly when the table space is created. I like to draw a line above the PREFETCHSIZE/BUFFERPOOL line and say, the characteristics of the table space above the line are set when you create the table space and cannot be changed later; you cannot change whether the table space is for REGULAR data or TEMPORARY data, you cannot change the page size, you cannot change whether it is SMS or DMS, and you cannot change the extent size, once it is created. The characteristics below the line can be changed through an ALTER TABLESPACE statement.

Note: you can change the name of a table space, through a RENAME TABLESPACE statement. You can also change the table space containers for a DMS table space through an ALTER TABLESPACE statement, but the containers are really addressed on the next page. In other words: don't dig yourself into a hole by drawing the line, but it can help to defer some of the discussion until a little later in the unit. At this point, you want to get ahead and get into the examples.

**Additional Information** —

**Transition Statement** —

## CREATE TABLESPACE syntax (2 of 2)

### system-containers:

```
|----- USING ----- ( ----- 'container-string' ----- ) -----|
```

### database-containers:

```
|----- USING ----- ( ----- FILE ----- 'container-string' ----- number-of-pages ----- ) -----|
                        |----- DEVICE -----|
                        |----- integer -----|
                                |----- K -----|
                                |----- M -----|
                                |----- G -----|
```

© Copyright IBM Corporation 2007

Figure 4-22. CREATE TABLESPACE syntax (2 of 2)

CF238.3

### Notes:

#### MANAGED BY SYSTEM

Specifies that the table space is to be a SMS table space.

#### USING ('container-string')

Identifies one or more containers that will belong to the table space and into which the table space's data will be stored. Each container string can be an absolute or relative directory name.

#### MANAGED BY DATABASE

Specifies that the table space is to be a DMS table space.

#### USING (FILE|DEVICE 'container-string' number-of-pages)

Identifies one or more containers that will belong to the table space and into which the table space's data will be stored. The type of the container (either FILE or DEVICE) and its size (in PAGESIZE pages) are specified.

For a FILE container, the string must be an absolute or relative file name.

For a DEVICE container, the string must be a device name. The device must already exist. Note that device containers are not supported on Windows 98, or on Linux on OS/390.

All containers must be unique across all databases; a container can belong to only one table space. The size of the containers can differ; however, optimal performance is achieved when all containers are the same size.

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Let's look at Automatic storage table space examples.

## Automatic storage — table space examples

- Syntax for CREATE and ALTER TABLESPACE:

```
CREATE TABLESPACE <tsName> [MANAGED BY AUTOMATIC STORAGE]
[INITIALSIZE integer {K|M|G}]
[AUTORESIZE {NO|YES}] [INCREASESIZE integer {PERCENT|K|M|G}]
[MAXSIZE {NONE | integer {K|M|G}}]
```

- Default initial size is 32 MB
- Default max size is none
- Default increase size is determined by DB2, which might change over the life of the table space
- Examples:

```
CREATE TABLESPACE USER1
CREATE TEMPORARY TABLESPACE TEMPTS
CREATE TABLESPACE MYTS INITIALSIZE 100 M MAXSIZE 1 G
CREATE LARGE TABLESPACE LRGTs INITIALSIZE 5 G AUTORESIZE NO
CREATE REGULAR TABLESPACE USER2 INITIALSIZE 500 M
```

© Copyright IBM Corporation 2007

Figure 4-23. Automatic storage — table space examples

CF238.3

### Notes:

When creating a table space in a database that is not enabled for automatic storage, the **MANAGED BY SYSTEM** or **MANAGED BY DATABASE** clause must be specified. Using these clauses results in the creation of a system managed space (SMS) table space or database managed space (DMS) table space respectively. An explicit list of containers must be provided in both cases.

If a database is enabled for automatic storage, another choice exists. The **MANAGED BY AUTOMATIC STORAGE** clause might be specified, or the **MANAGED BY** clause might be left out completely (which implies automatic storage). No container definitions are provided in this case because the DB2 database manager assigns the containers automatically.

In the **CREATE TABLESPACE** statements shown above:

**USER1** — Created with automatic storage, with an initial size of 32 MB and will auto-resize.

**TEMPTS** — Created with automatic storage, as a SMS managed temporary tablespace with one directory on each automatic storage path. SMS managed tablespaces do not have a MAXSIZE limit.

**MYTS** — Created with automatic storage, with an initial size of 100 MB and will auto-resize until it reaches 1 GB.

**LRGTS** — Created with automatic storage, with an initial size of 5 GB and will not auto-resize.

**USER2** — Created the automatic storage with an initial size of 500 MB and will auto-resize.



***Instructor notes:*****Purpose —****Details —****Additional information —** Certification objective: 2F Ability to create and discuss the characteristics of an automated managed table space.**Transition statement —**



## 4.3 Gathering system information

### Instructor topic introduction

What students will do —

How students will do it —

What students will learn —

How this will help students on their job —

## Get snapshot for tablespaces command

- Use GET SNAPSHOT FOR TABLESPACES ON
  - [LIST TABLESPACES](#) does not provide all of the information available
- Example of GET SNAPSHOT FOR TABLESPACES ON SAMPLE

```
<database> command:
Tablespace name = SYSCATSPACE
Tablespace ID = 0

...
Automatic Prefetch size enabled = Yes
Buffer pool ID currently in use = 1
Buffer pool ID next startup = 1
Using automatic storage = Yes
Auto-resize enabled = Yes

...
Increase size (bytes) = AUTOMATIC
```

- Note that Using automatic storage = Yes and Increase size (bytes) = AUTOMATIC are shown in the snapshot.

© Copyright IBM Corporation 2007

Figure 4-24. Get snapshot for tablespaces command

CF238.3

### Notes:

To identify which table spaces are enabled for automatic storage, use snapshots such as GET SNAPSHOT FOR TABLESPACES ON. Do not use the LIST TABLESPACES command as it identifies a table space as SMS or DMS, but does not identify which table spaces are enabled for automatic storage.

Snapshot monitors may be set up by:

- **Set monitor switches on/off:**

UPDATE DBM CFG USING DFT\_MON\_BUFPOOL ON (instance level)

or

UPDATE MONITOR SWITCHES USING BEFFERPOOL ON (application level)

- **Determine how the monitor switches are set:**

db2 get dbm monitor switches (instance level)

or

db2 get monitor switches (application level)

- **To gather tablespace information:**

get snapshot for tablespaces on <database\_name>

- **Reset monitor switches:**

reset monitor for database <database\_name>

or

reset monitor all

Example of LIST TABLESPACES command:

```

Tablespace ID          = 0
Name                   = SYSCATSPACE
Type                   = Database managed space
Contents               = All permanent data. Regular table space.
State                  = 0x0000
Detailed explanation:
Normal

```

**Note that no information about automatic storage is specified.**

Example of GET SNAPSHOT FOR TABLESPACES ON <database> command:

```

Tablespace name        = SYSCATSPACE
Tablespace ID          = 0
Tablespace Type        = Database managed space
Tablespace Content Type = All permanent data. Regular table s
pace.
Tablespace Page size (bytes) = 4096
Tablespace Extent size (pages) = 4
Automatic Prefetch size enabled = Yes
Buffer pool ID currently in use = 1
Buffer pool ID next startup = 1
Using automatic storage = Yes
Auto-resize enabled = Yes
File system caching = Yes
Tablespace State       = 0x'00000000'
Detailed explanation:
Normal
Tablespace Prefetch size (pages) = 4
Total number of pages = 16384
Number of usable pages = 16380
Number of used pages = 8868
Number of pending free pages = 0
Number of free pages = 7512
High water mark (pages) = 8868
Initial tablespace size (bytes) = 33554432

```

|                                 |                  |
|---------------------------------|------------------|
| Current tablespace size (bytes) | = 67108864       |
| Maximum tablespace size (bytes) | = NONE           |
| Increase size (bytes)           | = AUTOMATIC      |
| Time of last successful resize  | =                |
| Last resize attempt failed      | = No             |
| Rebalancer Mode                 | = No Rebalancing |
| Minimum Recovery Time           | =                |
| Number of quiescers             | = 0              |
| Number of containers            | = 1              |

**Note that Using *automatic storage* = *Yes* and *Increase size (bytes)* = *AUTOMATIC* are shown in the snapshot.**

***Instructor notes:***

**Purpose** — Identify the commands that can be used to query table space information.

**Details** — Just give high-level information on this page; examples will follow.

**Additional Information** — The LIST commands are being deprecated, so the direction is that the snapshot commands are the preferred way to get this information.

Certification objective: 4B Ability to capture snapshots using Get Snapshots or SQL functions.

**Transition statement** — Let's check out the SNAPCONTAINER view.

## Using the SNAPCONTAINER view

- Use the SYSIBMADM.SNAPCONTAINER view to retrieve container information

```
SELECT SNAPSHOT_TIMESTAMP, TBSP_NAME, TBSP_ID,
CONTAINER_NAME, CONTAINER_ID, CONTAINER_TYPE,
ACCESSIBLE, TOTAL_PAGES, USABLE_PAGES,
DBPARTITIONNUM FROM SYSIBMADM.SNAPCONTAINER ORDER
BY DBPARTITIONNUM
```

| TBSP_NAME    | TBSP_ID | CONTAINER_NAME       | CONTAINER_ID | CONTAINER_TYPE  | TOTAL_PAGES | USABLE_PAGES |
|--------------|---------|----------------------|--------------|-----------------|-------------|--------------|
| SYSCATSPACE  | 0       | C:\DB2\NODE0000\M... | 0            | FILE_EXTENT_TAG | 8192        | 8188         |
| TEMPSPACE1   | 1       | C:\DB2\NODE0000\M... | 0            | PATH            | 0           | 0            |
| USERSPACE1   | 2       | C:\DB2\NODE0000\M... | 0            | FILE_EXTENT_TAG | 8192        | 8160         |
| SYSTOOLSPACE | 3       | C:\DB2\NODE0000\M... | 0            | FILE_EXTENT_TAG | 8192        | 8188         |
| DMS01        | 4       | C:\DB2\dms\dms01     | 0            | FILE_EXTENT_TAG | 106         | 100          |
| DMS02        | 5       | C:\DB2\dms\dms02     | 0            | FILE_EXTENT_TAG | 14          | 12           |
| DMS03        | 6       | C:\DB2\dms\dms03     | 0            | FILE_EXTENT_TAG | 728         | 720          |
| DMS04        | 7       | C:\DB2\dms\dms04     | 0            | FILE_EXTENT_TAG | 22          | 20           |
| DMS05        | 8       | C:\DB2\dms\dms05     | 0            | FILE_EXTENT_TAG | 16          | 14           |
| DMS06        | 9       | C:\DB2\dms\dms06     | 0            | FILE_EXTENT_TAG | 40          | 36           |
| SMS01        | 10      | C:\DB2\sms\sms01     | 0            | PATH            | 0           | 0            |

© Copyright IBM Corporation 2007

Figure 4-25. Using the SNAPCONTAINER view

CF238.3

### Notes:

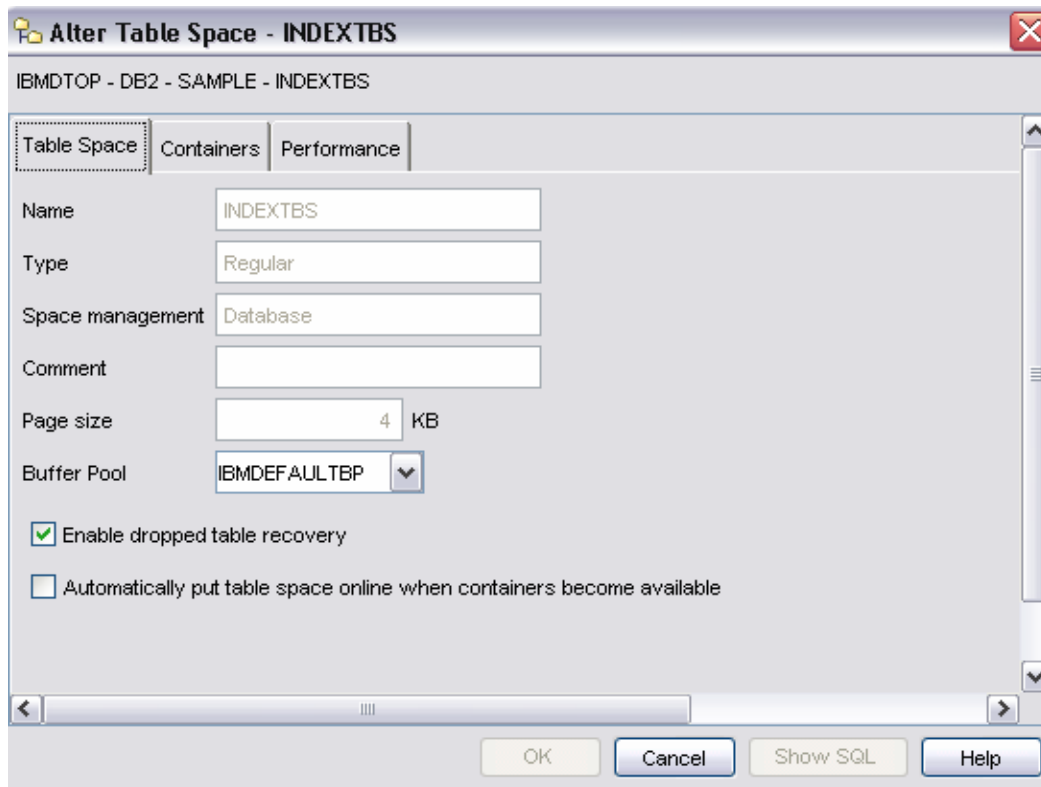
This administrative view allows you to retrieve tablespace\_container logical data group snapshot information for the currently connected database.

Used with the SNAPTbsp, SNAPTbsp\_PART, SNAPTbsp\_QUIESCER and SNAPTbsp\_RANGE administrative views, the SNAPCONTAINER administrative view returns data equivalent to the GET SNAPSHOT FOR TABLESPACES ON database-alias CLP command.



***Instructor notes:*****Purpose —****Details —****Additional information —****Transition statement —** Let's look at the Alter Table space wizard.

# Alter Table Space Wizard



© Copyright IBM Corporation 2007

Figure 4-26. Alter Table Space Wizard

CF238.3

## Notes:

What can be altered from the Alter Tablespace GUI?

- On **Table Space** tab:
  - Comment for table space
  - Buffer pool assignment
  - Setting for dropped table recovery
  - Setting for bringing table space online when containers are available
- On **Containers** tab:
  - Add a container to a DMS table space
  - Drop a container from a DMS table space
  - Resize a DMS table space container
  - Manage stripe sets
- On **Performance** tab:
  - Prefetch size
  - Overhead
  - Transfer rate

***Instructor notes:***

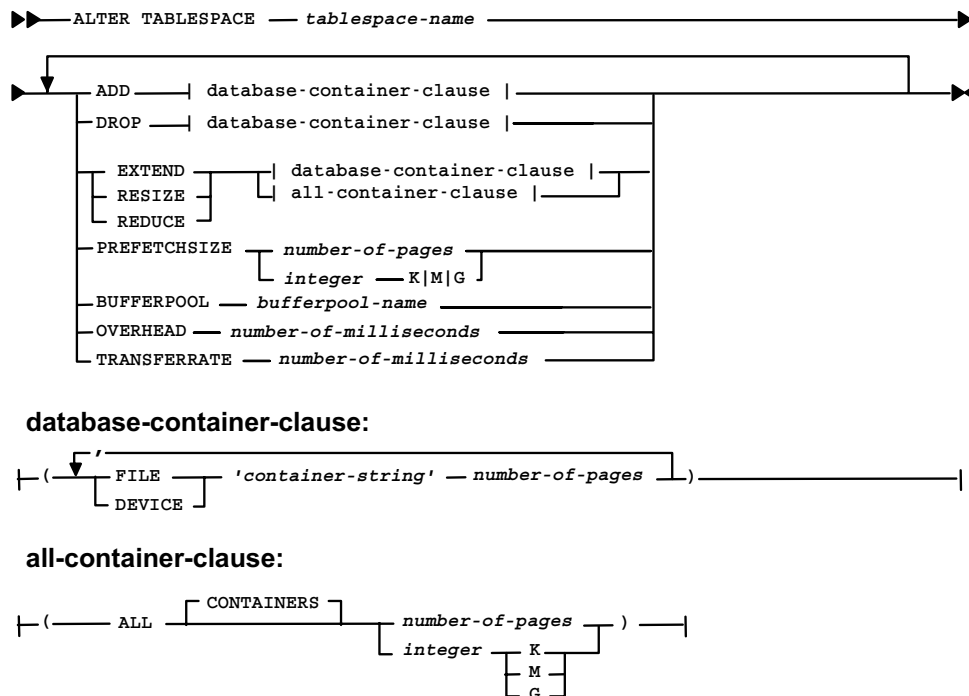
**Purpose** — Identify the Alter Table Space GUI.

**Details** — There are three tabs that can be explored with the Alter Table Space GUI.

**Additional information** —

**Transition statement** — Next, we view the syntax for the Alter table space command.

# ALTER TABLESPACE syntax



© Copyright IBM Corporation 2007

Figure 4-27. ALTER TABLESPACE syntax

CF238.3

## Notes:

The ALTER TABLESPACE statement is used to do the following:

- Add or drop a container in a DMS table space
- Extend, reduce, or resize DMS table space containers
- Modify the PREFETCHSIZE setting for a table space
- Modify the OVERHEAD setting for a table space
- Modify the TRANSFERRATE setting for a table space
- Modify the BUFFERPOOL assigned for a table space

When adding or removing space from the table space, the following rules must be followed:

- EXTEND and RESIZE can be used in the same statement, provided that the size of each container is increasing.
- REDUCE and RESIZE can be used in the same statement, provided that the size of each container is decreasing.
- EXTEND and REDUCE cannot be used in the same statement, unless they are being directed to different partitions.

- ADD cannot be used with REDUCE or DROP in the same statement, unless they are being directed to different partitions.
- DROP cannot be used with EXTEND or ADD in the same statement, unless they are being directed to different partitions.

## **ADD**

Specifies that a new container is to be added to the table space. Once the new container has been added and the transaction is committed, the contents of the table space are automatically rebalanced across the containers. Access to the table space is not restricted during the rebalancing.

If you are adding more than one container to a table space, they should be added at the same time, in the same ALTER TABLESPACE statement, so that the cost of rebalancing is incurred only once.

## **DROP**

Specifies that one or more containers are to be dropped from the table space.

## **EXTEND**

Specifies that existing containers are being increased in size. The size specified is the size by which the existing container is increased. If the `all-containers-clause` is specified, then all containers on the table space will increase by this size.

## **REDUCE**

Specifies that existing containers are to be reduced in size. The size specified is the size by which the existing container is decreased. If the `all-containers-clause` is specified, all containers in the tablespace will decrease by this size.

## **RESIZE**

Specifies that the size of existing containers is being changed. The size specified is the new size for the container. If the `all-containers-clause` is specified, then all containers in the table space will be changed to this size. If the operation affects more than one container, these containers must all either increase in size, or decrease in size. It is not possible to increase some while decreasing others.

## **PREFETCHSIZE**

Specifies the number of PAGESIZE pages that will be read from the table space when data prefetching is being performed.

## **BUFFERPOOL**

Specifies the name of the buffer pool to be used for tables in this table space.

## **OVERHEAD**

A number that specifies the I/O controller overhead and disk seek and latency time, in milliseconds. The number should be an average for all containers that belong to the table space, if not the same for all containers. This value is used to determine the cost of I/O during query optimization.

## TRANSFERRATE

A number that specifies the time to read one page into memory, in milliseconds. The number should be an average for all containers that belong to the table space, if not the same for all containers. This value is used to determine the cost of I/O during query optimization.



### Important

**Caution:** When using RESIZE or EXTEND with device containers, ensure that you do not overallocate the space in the underlying device.

The RENAME TABLESPACE statement allows you to rename an existing table space.

You can turn AUTORESIZE off by specifying AUTORESIZE NO in the ALTER TABLESPACE command.

This specifies that the auto-resize capability of a DMS table space or an automatic storage table space is to be disabled. If the auto-resize capability is disabled, any values that have been previously specified for INCREASESIZE or MAXSIZE will not be kept.

***Instructor notes:***

**Purpose** — Identify the ALTER TABLESPACE syntax.

**Details** — Identify the alterable characteristics of a table space. DMS table spaces have more alterable options than SMS. For a DMS table space you can:

- Add additional containers
- Drop existing containers
- Extend the size of existing containers
- Resize existing containers (either to make them larger or smaller, but all must be changing in the same direction)
- Reduce the size of existing containers

For any type of table space, you can alter the prefetchsize, buffer pool, overhead, or transfer rate. We will be addressing prefetchsize and buffer pool on the following pages.

Regarding overhead and transfer rate: these values communicate to the optimizer how fast it can perform transfers of data from the I/O devices associated with the table space, and what the seek time is. The default values are probably very pessimistic compared to any currently used devices. However, if you leave them set to the defaults, it probably is not a big problem — it just implies to the optimizer that I/O is more expensive than it is — but I/O **is** more expensive than other operations, so it is probably not too bad.

**Additional Information** — Student notes used to say to alter the table space in the same transaction, but SQL Reference says that cannot be done.

**Transition Statement** — Let's look at data placement considerations.

## Placement considerations

- 
- If index scans are required:
    - Consider separate table spaces for index and data
  - Consider placing index table space on fastest media available
  - Consider placing entire table onto separate table space
    - If accessed frequently
  - Consider EXTENTSIZE
    - Trade-off between space (very small tables) and performance
  - I/O Prefetch

© Copyright IBM Corporation 2007

Figure 4-28. Placement considerations

CF238.3

### Notes:

Having the ability to logically group tables in table spaces opens up many new possibilities for the database and system administrator. It also creates the necessity of planning to be done before the database creation is done, and to determine the optimal table space configuration. One question to ask is, “Where will I keep my catalogs, data objects, and indexes?”

Consider, as in our example, that you will have a very large database. We will keep employee resumes and pictures in our database; possibly these are types of data we had not considered storing in our database in the past. For performance, we chose to separate the indexes onto a separate table space (and disk). Multiple queries can read data and indexes in parallel, searching for the index and table data at the same time. However, for a single query, the database manager must read the index before reading the data. The performance benefit in this case is the reduction of the read/write head movement that would occur when index and data I/O requests go to the same disk.

Consider placing a table onto a separate table space if that table will be accessed frequently. You may be able to control the placement onto disk for that table. For example,



placing a frequently accessed table closer to the middle of the disk on a UNIX platform could ensure faster access.

The EXTENTSIZE parameter is the number of pages written before writing continues in another extent or segment. The EXTENTSIZE is a trade-off between the potential for wasting space and the cost of extending a database object. If you choose a small EXTENTSIZE (for example, two 4 KB pages or 8 KB), then every time an object grows by 8 KB, you will need to add a new extent. On the other hand, if you choose a large EXTENTSIZE (for example, 256 4 KB pages or 1024 pages), then there can be many unused pages in an extent that contains a small table or the last extent of a large table.

The objective of I/O prefetch is to improve the performance of queries that need a large amount of I/O. This is done by creating a separate process that actually performs the reads of large amounts of data.

***Instructor notes:***

**Purpose** — Identify data placement considerations.

**Details** — Some of the items for consideration are listed. See student notes for some discussion points.

**Additional information** —

**Transition statement** —

## 4.4 Describe and modify the Database Configuration

### Instructor topic introduction

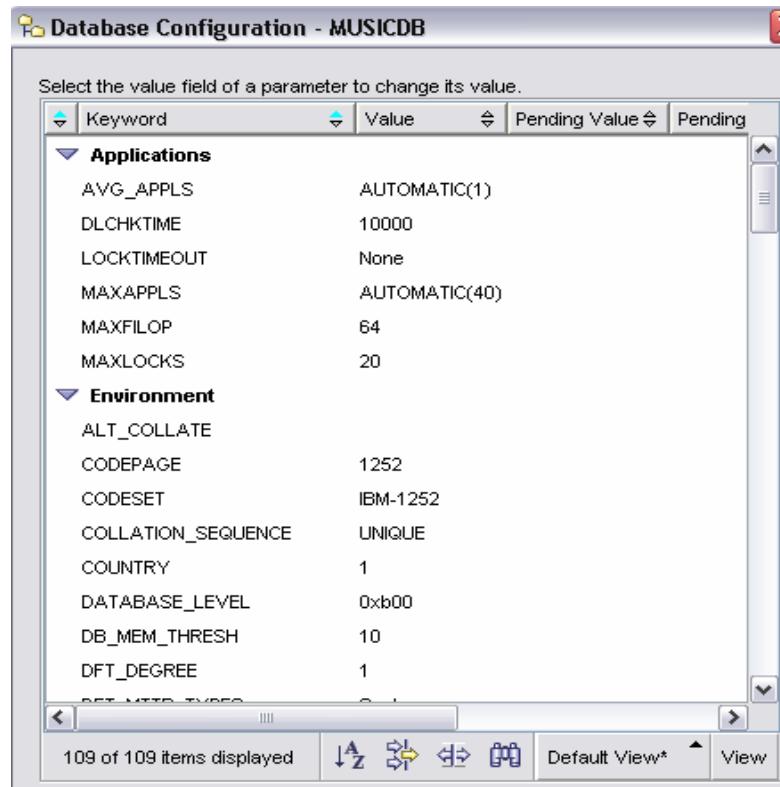
**What students will do** — Learn how to access the db cfg, dbm cfg, and db dir.

**How students will do it** — Listen to the lecture and practice in the lab.

**What students will learn** — As part of the things created when a database is created, students will learn how to access the database configuration file, the database manager configuration file, and the database directory.

**How this will help students on their job** — They will know how to access the db cfg, the dbm cfg, and the database directory.

# Database Configuration



© Copyright IBM Corporation 2007

Figure 4-29. Database Configuration

CF238.3

## Notes:

Use the Database Configuration window to view or change the database configuration parameter values. The database configuration parameters (**Keyword**) and their values for this database are displayed. You can use the options available from **View** to rearrange the display.

The database configuration file is created when a DB2 database is created. The parameters it contains affect resources at the database level. Values for many of these parameters can be changed from the default values to improve performance.

CLP or the Control Center may be used to get a listing of the database configuration file. The GET DB CFG command will list the parameters contained in the database configuration file.

The updatable parameters in the database configuration file can be changed using the UPDATE DB CFG command, or via the database configuration GUI.

The database territory, code set, country code, and code page are recorded in the database configuration file. However, these parameters cannot be changed.

***Instructor notes:***

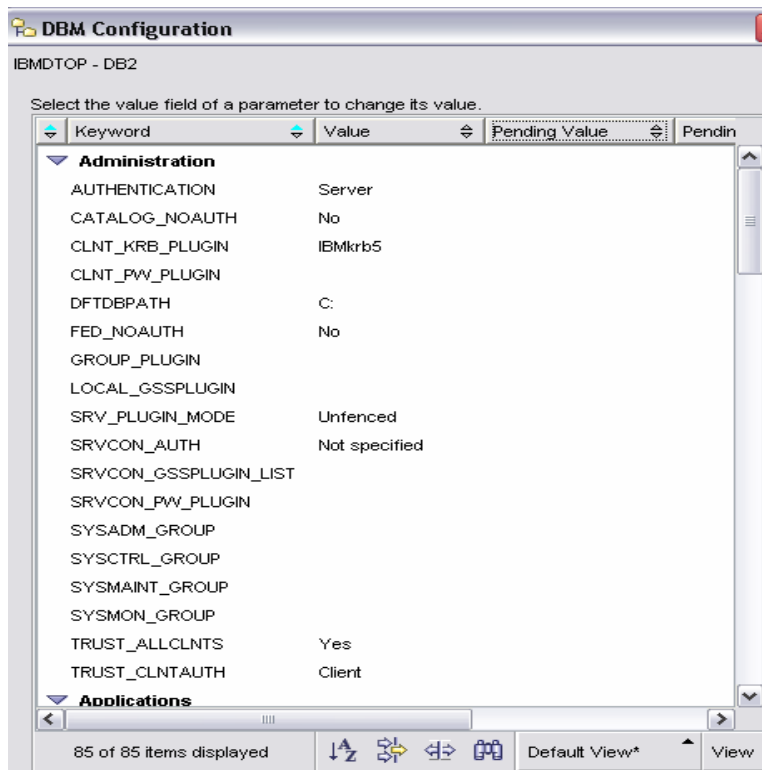
**Purpose** — Introduction to Configuring DB2 databases from the GUI.

**Details** — Show the GUI screen that can be used to view and change the database configuration parameters.

**Additional Information** — Certification objective: 1C Ability to manage / modify Database configuration information.

**Transition Statement** — Now, let's look at configuring the database manager from the GUI.

# Database Manager Configuration



© Copyright IBM Corporation 2007

Figure 4-30. Database Manager Configuration

CF238.3

## Notes:

Use the DBM Configuration window to view or change the database manager configuration parameter values.

The database manager configuration file is created when a DB2 instance is created. The parameters it contains affect system resources at the instance level, independently of any one database that is part of that instance. Values for many of these parameters can be changed from the system default values to improve performance or increase capacity, depending on your system's configuration.

The database manager configuration variables (**Keyword**) and their values for this instance are displayed. You can use the options available from **View** to rearrange the display.

Every instance has a database manager configuration file, specifying global parameters that affect the instance. The GET DBM CFG command can be used to list the database manager configuration file.

Values of updatable parameters can be changed using the UPDATE DBM CFG command.

***Instructor notes:***

**Purpose** — Identify the GUI screen for viewing and changing the instance level configuration parameters.

**Details** — The updatable database manager configuration parameters can be changed using the GUI or by using the UPDATE DBM CFG command.

**Additional Information** —

**Transition Statement** — Let's look at the database directory.

## Database directories overview

### ■ db2 create database ourdb

```
db2 list db directory
```

```
System Database Directory
```

```
Number of entries in the directory = 2
```

```
Database 1 entry:
```

```
Database alias           = OURDB
Database name            = OURDB
Local database directory = C:
Database release level   = b.00
Comment                  =
Directory entry type     = Indirect
Catalog database partition number = 0
Alternate server hostname =
Alternate server port number =
```

```
Database 2 entry:
```

```
Database alias           = SAMPLE
Database name            = SAMPLE
Local database directory = C:
Database release level   = b.00
Comment                  = A sample database
Directory entry type     = Indirect
Catalog database partition number = 0
Alternate server hostname =
Alternate server port number =
```

```
db2 catalog db sample as testdb on c:
DB20000I The CATALOG DATABASE command
completed successfully.
DB21056W Directory changes may not be
effective until the directory cache is
refreshed.
```

```
db2 uncatalog db testdb
DB20000I The UNCATALOG DATABASE command
completed successfully.
DB21056W Directory changes may not be
effective until the directory cache is
refreshed.
```

© Copyright IBM Corporation 2007

Figure 4-31. Database directories overview

CF238.3

### Notes:

The database directory maintains a list of databases that can be connected to.

Either CLP or the Control Center may be used to change directory entries. The Control Center allows a change to be made without uncataloging and recataloging the directory. There is no CHANGE DIRECTORY CLP command. If you wish to change a catalog entry using CLP, you must uncatalog the directory entry and then recatalog the entry using the CATALOG command.

The database alias must be unique. The database name does not have to be unique. A reason to catalog a local database is to change its alias, which is the name by which users and programs identify the database. When a database is created, its alias is the same as its name, unless specified otherwise on the CREATE DATABASE command. The name by which users and programs refer to a database can be changed without having to DROP and re-CREATE the database.

You may want a database cataloged with more than one alias name.



**Instructor notes:**

**Purpose** — Identify how to view and change the database directory.

**Details** — You can view the contents of the database directory with the LIST DB DIRECTORY command. This will show you a listing of the databases cataloged on your system.

In our example, we created a database named OURDB. Since we did not give an alias name, the alias was set the same as the database name. When users are connecting to the database, they connect using the alias name. Unfortunately, when we were telling users what the name of this new database was, they thought we were saying RDB. So, we decided we needed to give OURDB a different alias, TESTDB.

We could just run the command at the middle of the page. There is no need to uncatalog the old alias; it can stay around.

Notice that, once we catalog the new alias, and we do a LIST DB DIRECTORY, we see that the database whose name is OURDB now has an alias of TESTDB.

**Additional information** —

**Transition statement** —



# Checkpoint

## Exercise — Unit Checkpoint

\_\_\_ 1. Name the two types of table spaces:

\_\_\_\_\_

\_\_\_\_\_

**SMS**  
**DMS**

\_\_\_ 2. Which type of table space must be used if you want to store the index data separately from the table data?

\_\_\_\_\_

**DMS**

\_\_\_ 3. Which type of table space should be used to allow the operating system to allocate pages to the table space as needed?

\_\_\_\_\_

**SMS**

## Unit summary

Having completed this unit, you should be able to:

- Review specifics of creating a database
- Explore the System Catalog tables and views
- Compare DMS versus SMS table spaces
- Describe how to setup and manage a DB2 database with Automatic Storage enabled
- Differentiate between table spaces, containers, extents, and pages
- Define table spaces
- Use the **get snapshot for tablespaces** command to display table space statistics
- Explore database configuration parameters

© Copyright IBM Corporation 2007

Figure 4-32. Unit summary

CF238.3

### Notes:

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**



# Unit 5. Creating database objects

## Estimated time

00:40

## What this unit is about

This unit provides information about DB2's physical structure (files and directories), tables, views, aliases, referential integrity, check constraints, triggers, and LOBs.

Database objects can be created, updated, and listed with the Command Line Processor (CLP) or the Control Center graphical tool.

## What you should be able to do

After completing this unit, you should be able to:

- List DB2 object hierarchy and physical directories and files
- Create the following objects:
  - Schema, Table, View, Alias, Index
- Explore the use of table partitioning
- Review the use of Temporary Tables
- Explore the use and implementation of Check Constraints, Referential Integrity and Triggers
- Exploring the need for and the use of Large Objects
- Recognize XML and its native store as critical infrastructure for emerging technologies

## How you will check your progress

Accountability:

- Machine exercises

## References

*Command Reference*  
*Administration Guide*

## Unit objectives

After completing this unit, you should be able to:

- List DB2 object hierarchy and physical directories and files
- Learn to create the following objects:
  - [Schema](#), [Table](#), [View](#), [Alias](#), [Index](#)
- Explore the use of table partitioning
- Review the use of Temporary Tables
- Explore the use and implementation of Check Constraints, Referential Integrity and Triggers
- Exploring Large Objects: the need and usage
- Study an overview of XML as a native store

© Copyright IBM Corporation 2007

Figure 5-1. Unit objectives

CF238.3

### Notes:



***Instructor notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Identify the objectives for the unit.

**Transition Statement** — Let's look at the database objects in a DB2 Database.



## 5.1 DB2 object hierarchy and physical directories and files

### Instructor topic introduction

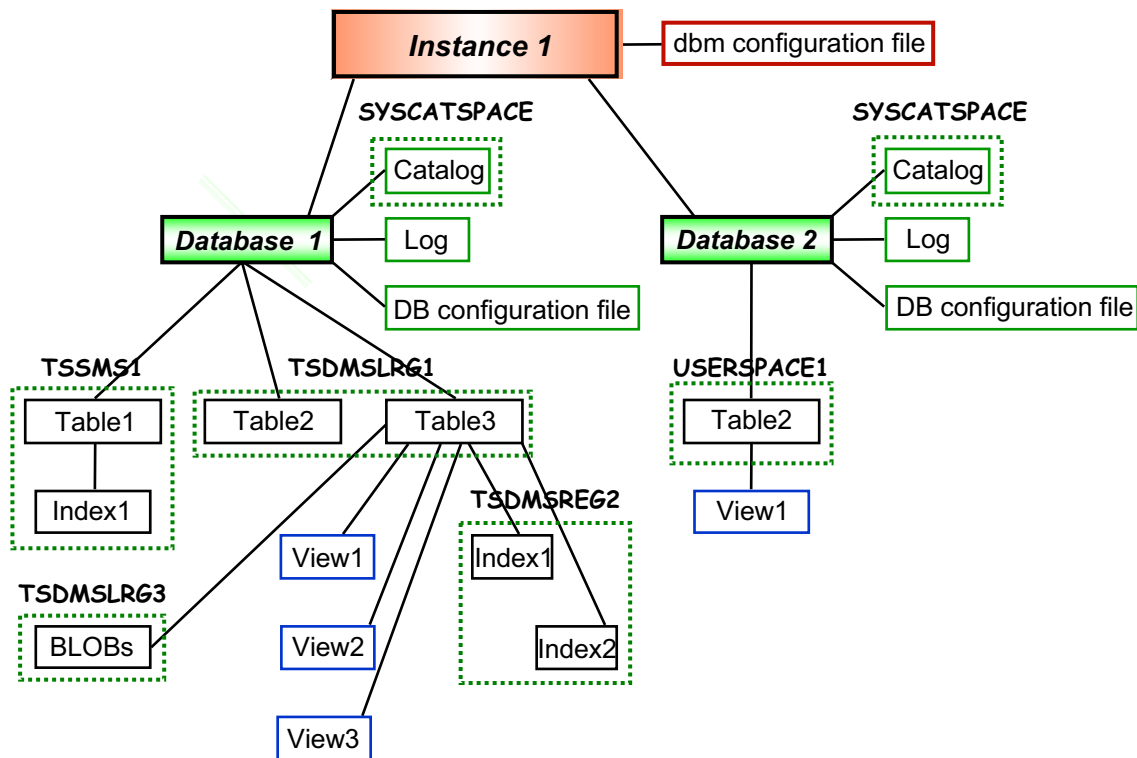
**What students will do** — Learn about the object hierarchy (instance, database, table space, tables, views) in DB2.

**How students will do it** — Listen to the lecture.

**What students will learn** — The object hierarchy in DB2.

**How this will help students on their job** — They will understand the relationship of database objects to each other.

## DB2 object hierarchy



© Copyright IBM Corporation 2007

Figure 5-2. DB2 object hierarchy

CF238.3

### Notes:

Each DB2 instance has its own database manager configuration file. Its global parameters affect the system resources allocated to DB2 for an individual instance. Its parameters may be changed from the system default values to improve performance or increase capacity, depending on the workstation configuration.

Each instance may have multiple databases. A relational database presents data as a collection of tables. A table consists of a defined number of columns and any number of rows. Each database includes a set of system catalog tables, which describe the logical and physical structure of the data (like a table or view), or contain statistics of the data distribution; a configuration file containing the parameter values allocated for the database; and a recovery log with ongoing transactions and archiveable transactions.

Each table may have multiple indexes. Indexes may provide a faster way to access table data. Each table may have multiple views. Views may be associated with more than one base table.

The physical objects in a database are assigned to table spaces. When creating a table, you can decide to have certain objects such as indexes and large object (LOB) data kept separately from the rest of the table data. By default, all objects referencing a table reside in the same table space where the table itself resides. A table space can also be spread over one or more physical storage devices.

In the visual, two databases are shown.

**For Database 1:**

- The system catalog tables are in table space SYSCATSPACE.
- Table 1 and its one Index are in a SMS table space named TSSMS1.
- Tables 2 and Table 3 are both assigned to the Large DMS table space TSDMSLRG1.
- Two Indexes for Table 3 are assigned to the Regular table space TSDMSREG2.
- The Large Object data columns from Table 3 are assigned to the Large table space TSDMSLRG3.

**For Database 2:**

- The system catalog tables are in table space SYSCATSPACE.
- Table 2 is assigned to the table space USERSPACE1. By default, USERSPACE1 would be an Automatic Storage managed table space.

## ***Instructor notes:***

**Purpose** — Identify the relationship between instances, database, configuration files.

**Details** — As discussed previously, instances have a database manager configuration file that allows you to configure how resources should be allocated for the instance.

An instance can have one or more databases defined within it. Each database has a database configuration file, which allows you to configure how resources should be allocated for the database. Each database also has a set of system catalog tables that contain information about all of the objects defined in the database.

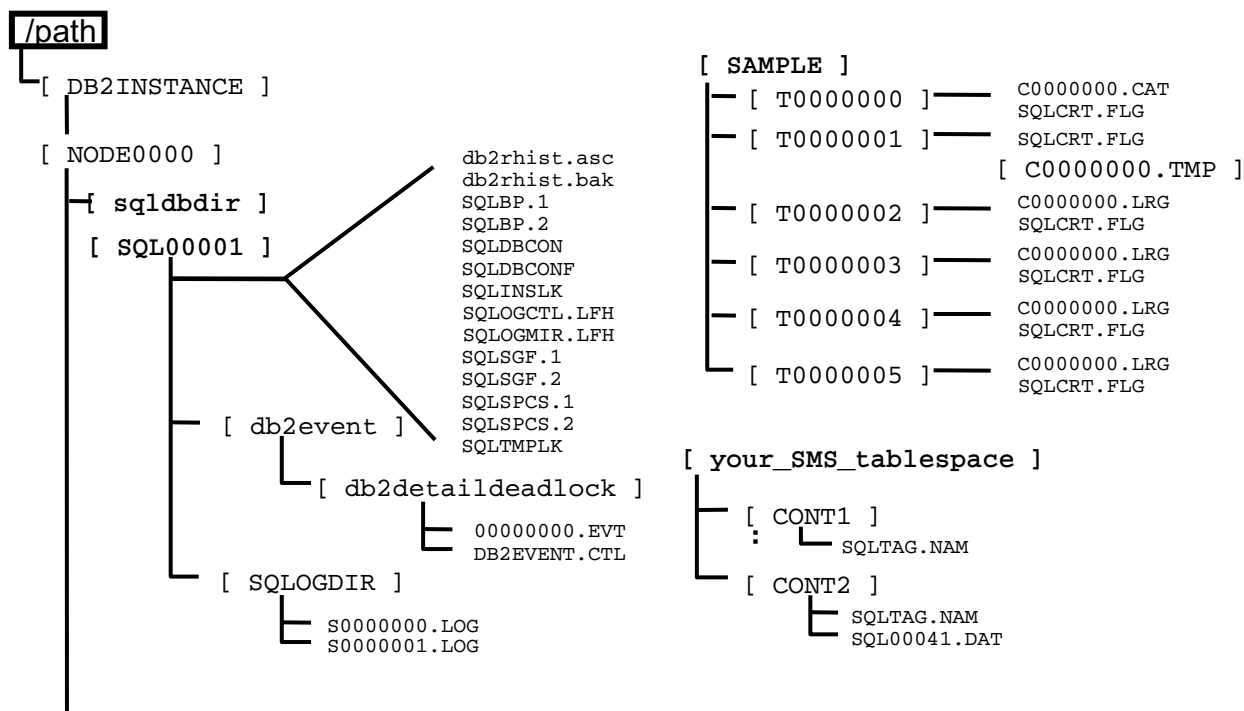
Each database also has a recovery log that contains information about transactions that have completed and those which are in progress on the database. The recovery log is used to maintain the integrity of the database in case of application or system failure.

Finally, the database has a set of user tables defined to it. Tables are defined in table spaces which themselves reference containers that identify physically where the data should be located on disk. As we saw in the previous unit, you can separate components of tables (indexes and long data) into separate table spaces.

**Additional Information** —

**Transition Statement** — Let's look at how these items appear in physical storage.

## Database physical directories and files



© Copyright IBM Corporation 2007

Figure 5-3. Database physical directories and files

CF238.3

### Notes:

**Do not directly edit or alter these files. Do not move these files.** The only supported means of database access is through the use of DB2 administration tools, such as the Control Center.

When an instance is created, the following directory structure is created:

```

$HOME/sqllib (UNIX)
X:\install_location\%DB2INSTANCE%\SQLLIB (Windows)

```

After you create the first database for an instance, the following directory structure exists:

```

/path/%DB2INSTANCE%/NODE0000/sqldbdir/sqldbdir
/path/%DB2INSTANCE%/NODE0000/SQL00001

```

When a database is created, a local database directory is created. The name of the file is **sqldbdir**.

When a database is created, the database manager creates a separate subdirectory to store the database objects associated with that database. The naming scheme used by the database manager is `/path/%DB2INSTANCE%/NODE0000/SQL00001`, where SQL00001

contains the objects associated with the first database created. On the CREATE DATABASE command a path (for example, /path) is specified. The database manager maintains these subdirectories automatically. You can determine which subdirectory is related to each database by using the command `LIST DB DIRECTORY` or `LIST DB DIRECTORY ON <path>`.

The database directory contains the following files that are created as part of the CREATE DATABASE command.

- The files SQLBP.1 and SQLBP.2 contain buffer pool information. Each file has a duplicate copy to provide a backup.
- The files SQLSPCS.1 and SQLSPCS.2 contain table space information. Each file has a duplicate copy to provide a backup.
- The files SQLSGF.1 and SQLSGF.2 contain storage path information associated with the database's automatic storage. Each file has a duplicate copy to provide a backup.
- The SQLDBCON file contains database configuration information. Do not edit this file. To change configuration parameters, use either the Control Center or the command-line statements `UPDATE DATABASE CONFIGURATION` and `RESET DATABASE CONFIGURATION`.
- The DB2RHIST.ASC history file and its backup DB2RHIST.BAK contain history information about backups, restores, loading of tables, reorganization of tables, altering of a table space, and other changes to a database.
  - The contents of the DB history file can be viewed using the DB2 Journal or by using the `LIST HISTORY` command.
- The DB2TSCHNG.HIS file contains a history of table space changes at a log-file level. For each log file, DB2TSCHG.HIS contains information that helps to identify which table spaces are affected by the log file. Table space recovery uses information from this file to determine which log files to process during table space recovery. You can examine the contents of both history files in a text editor.
- The log control files, SQLOGCTL.LFH and SQLOGMIR.LFH, contain information about the active logs.
  - Recovery processing uses information from this file to determine how far back in the logs to begin recovery. The SQLOGDIR subdirectory contains the actual log files.
- The SQLINSLK file helps to ensure that a database is used by only one instance of the database manager.

## **SMS table spaces**

In SMS table spaces, tables are stored in files at the operating system level. The first table data file (**SQL00001.DAT**) is created in a container (segment), for example SQLT0000.0. DB2 will extend the DAT file one extent at a time, using the extent size for the table space (defaults to **DFT\_EXTENT\_SZ** if not explicitly indicated). The next extent of data would be written to the SQL00001.DAT in the next container, for example SQLT0000.1, if more than



one container has been defined for the table space. The same mechanism is followed for INX, LF, LOB, and LBA files.

Each subdirectory or container has a file created in it called SQLTAG.NAM. This file marks the subdirectory as being in use so that subsequent table space creation does not attempt to otherwise use these subdirectories.

In addition, a file called SQL\*.DAT stores information about each table that the subdirectory or container contains. The asterisk (\*) is replaced by a unique set of digits that identifies each table. For each SQL\*.DAT file there might be one or more of the following files, depending on the table type, the reorganization status of the table, or whether indexes, LOB, or LONG fields exist for the table:

- SQL\*.BKM (contains block allocation information if it is an MDC table)
- SQL\*.LF (contains LONG VARCHAR or LONG VARGRAPHIC data)
- SQL\*.LB (contains BLOB, CLOB, or DBCLOB data)
- SQL\*.XDA (contains XML data)
- SQL\*.LBA (contains allocation and free space information about SQL\*.LB files)
- SQL\*.INX (contains index table data)
- SQL\*.IN1 (contains index table data)
- SQL\*.DTR (contains temporary data for a reorganization of an SQL\*.DAT file)
- SQL\*.LFR (contains temporary data for a reorganization of an SQL\*.LF file)
- SQL\*.RLB (contains temporary data for a reorganization of an SQL\*.LB file)
- SQL\*.RBA (contains temporary data for a reorganization of an SQL\*.LBA file)

On the visual, you also see two user-defined table space directories:

- **SMS** directory, for the SMS table space, and the files it includes has the same structure as the SMS table spaces that were created under SQL00001. Note that .DAT or .INX or .LBA files could be found under this directory.
- **DMS** directory, for the DMS table space, the specified disk space is preallocated. Every DMS table space has its internal metadata that is used by DB2 to manage the disk space among the objects assigned to the tablespace.

Temporary transient files that are not shown on the visual include the following:

- **SQLnnnnn.DTR** is a temporary data file for a reorg of a DAT file.
- **SQLnnnnn.LFR** is a temporary data file for a reorg of a LF file.
- **SQLnnnnn.RLB** is a temporary data file for a reorg of a LB file.
- **SQLnnnnn.RBA** is a temporary data file for a reorg of a LBA file.

Temporary files used by REORG should never be erased; they may be the only place your data still exists if there is a failure in the midst of a REORG process.

**SQLTMPLK** and **SQLINSLK** are files used to help ensure that a database is only used by one instance of the database manager.

### ***Instructor notes:***

**Purpose** — Identify the physical directories and files created by DB2.

**Details** — Sometimes DBAs or system administrators will be puzzled by all of the files that are created for the storing of DB2 data. This graphic allows you to indicate what physical items DBAs will see from the operating system's point of view.

Make sure you emphasize the first two sentences in the student notes: **Do not directly edit or alter these files. Do not move these files.**

**Additional Information** —

**Transition Statement** — Let's look at the syntax needed for creating schema.

## 5.2 Creating objects

### Instructor topic introduction

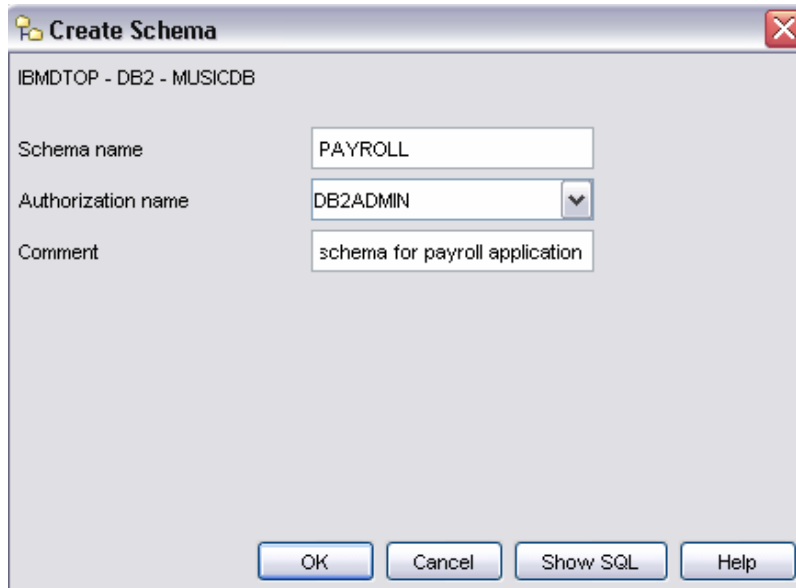
What students will do —

How students will do it —

What students will learn —

How this will help students on their job —

## Create Schema



```
CREATE SCHEMA PAYROLL AUTHORIZATION DB2ADMIN;  
COMMENT ON Schema PAYROLL IS 'schema for  
payroll application';
```

© Copyright IBM Corporation 2007

Figure 5-4. Create Schema

CF238.3

### Notes:

To access the Create Schema GUI from the Control Center:

- Select **Schemas** (right-click the DB node *Schemas*)
  - Select **Create**

The Create Schema GUI has the following:

- Schema name
- Authorization name
- Comment

The **Show SQL** or **Show Command** button is available on all Control Center dialogs that generate SQL or DB2 commands. You can display the SQL statement save it as a script.

When the schema is explicitly created with the CREATE SCHEMA statement, the schema owner is granted CREATEIN, DROPIN, ALTERIN privileges on the schema with the ability to grant these privileges to other users.

A schema name or authorization name cannot begin with **SYS**.

While organizing your data into tables, it may also be beneficial to group tables (and other related objects) together. This is done by defining a schema. Information about the schema is kept in the system catalog tables of the database to which you are connected. As other objects are created, they can be placed within this schema.

An authorization ID that holds SYSADM or DBADM authority can create a schema with any valid schema name or authorization name. Any ID can explicitly create a schema that matches the authorization ID of the statement.

When the schema is explicitly created with the CREATE SCHEMA statement, the schema owner is granted CREATEIN, DROPIN, ALTERIN, GRANTIN privileges on the schema with ability to grant to other users.

You can create a schema and include certain SQL statements with it (CREATE TABLE, excluding typed tables and materialized query tables; CREATE VIEW statement, excluding typed views; CREATE INDEX statement; COMMENT statement; GRANT statement). For example, the following is a *single* statement:

```
CREATE SCHEMA pers
CREATE TABLE ORG (
    deptnumb SMALLINT NOT NULL,
    deptname VARCHAR(14),
    manager SMALLINT,
    division VARCHAR(10),
    location VARCHAR(13),
    CONSTRAINT pkeydno PRIMARY KEY (deptnumb),
    CONSTRAINT fkeymgr FOREIGN KEY (manager)
    REFERENCES staff (id)
)
CREATE TABLE STAFF (
    id SMALLINT NOT NULL,
    name VARCHAR(9),
    dept SMALLINT,
    job VARCHAR(5),
    years SMALLINT,
    salary DECIMAL(7,2),
    comm DECIMAL(7,2),
    CONSTRAINT pkeyid PRIMARY KEY (id),
    CONSTRAINT fkeydno FOREIGN KEY (dept)
    REFERENCES org (deptnumb)
)
```

Thus, you can use a single statement to create two tables that are dependent on each other, rather than having to create the first with primary key, the second with primary and foreign key, and then alter the first to add foreign key.

Unqualified object names in any SQL statement within the CREATE SCHEMA statement are implicitly qualified by the name of the created schema.

## ***Instructor notes:***

**Purpose** — Identify how a schema can be created through the GUI. Show an example of the statements that could be issued to accomplish the same purpose as the GUI shown on the previous screen.

**Details** — Show the GUI screen for creating a schema. The statements on the screen (CREATE SCHEMA and COMMENT ON) are the statements you would see if you pressed SHOW STATEMENT on the GUI on the previous screen.

**Additional Information** — You will need to make the decision based on your mix of students in class; if you have primarily Windows DBAs, then throughout this unit, an emphasis on the GUI screens is probably appropriate. However, if you have primarily UNIX DBAs, they may have more interest in the command line interface. In addition, in many cases, the functionality of the GUI (the capabilities of the statement that the GUI allows you to take advantage of) lags behind the ability of the product. Thus, using the GUI, you cannot always take advantage of the latest function of the product.

Consider as you determine what to emphasize as you teach: The GUI is easy to use in most cases. If you know how to execute a statement or command from the command line interface, you will have no problem using the GUI. On the other hand, if you only know the GUI, you may have difficulty if you are ever in a situation where the GUI is not available to you. It is important that, at least as an instructor, you are familiar with the command line interface. For UNIX students, it is also critical that they know how to use statements and commands from the command line interface, as there will be times that they will not have the GUI interface available to them.

You may want to call students' attention to the additional example in the student notes; this is (in V8) not available to be done through the GUI, only through the statement interface.

You may want to note the permissions granted to the authorization name specified; but this will be addressed in more detail in the Security unit.

Certification objective: 2B Skill in discussing use of schemas, 2D Ability to create and manipulate the various DB2 objects.

**Transition Statement** — Let's look at how the schema name can be used.

## Set current schema



```
connect to musicdb user Keith;  
select * from employee;
```

- will select from KEITH.EMPLOYEE

```
set current schema = 'PAYROLL';  
select * from employee;
```

- will select from PAYROLL.EMPLOYEE

© Copyright IBM Corporation 2007

Figure 5-5. Set current schema

CF238.3

### Notes:

When accessing data within DB2, unqualified references will be implicitly qualified with the authorization ID that was used to connect to the database. You can override this by setting the CURRENT SCHEMA. The initial value of the CURRENT SCHEMA special register is equivalent to USER.

The example on the graphic shows that a user *KEITH* is connecting to the database. If Keith issues a select against the EMPLOYEE table, the table that will be accessed will be KEITH.EMPLOYEE. If he sets his current schema to *PAYROLL*, then a select against the EMPLOYEE table will be directed against the PAYROLL.EMPLOYEE table.

Alternative syntax includes:

```
SET CURRENT SCHEMA = 'PAYROLL'  
SET SCHEMA 'PAYROLL'  
SET CURRENT SQLID 'PAYROLL'
```

Note that the use of the = is optional in all of these statements.

The value of the CURRENT SCHEMA special register is used as the schema name in all dynamic SQL statements where an unqualified reference to a database object exists.



***Instructor notes:***

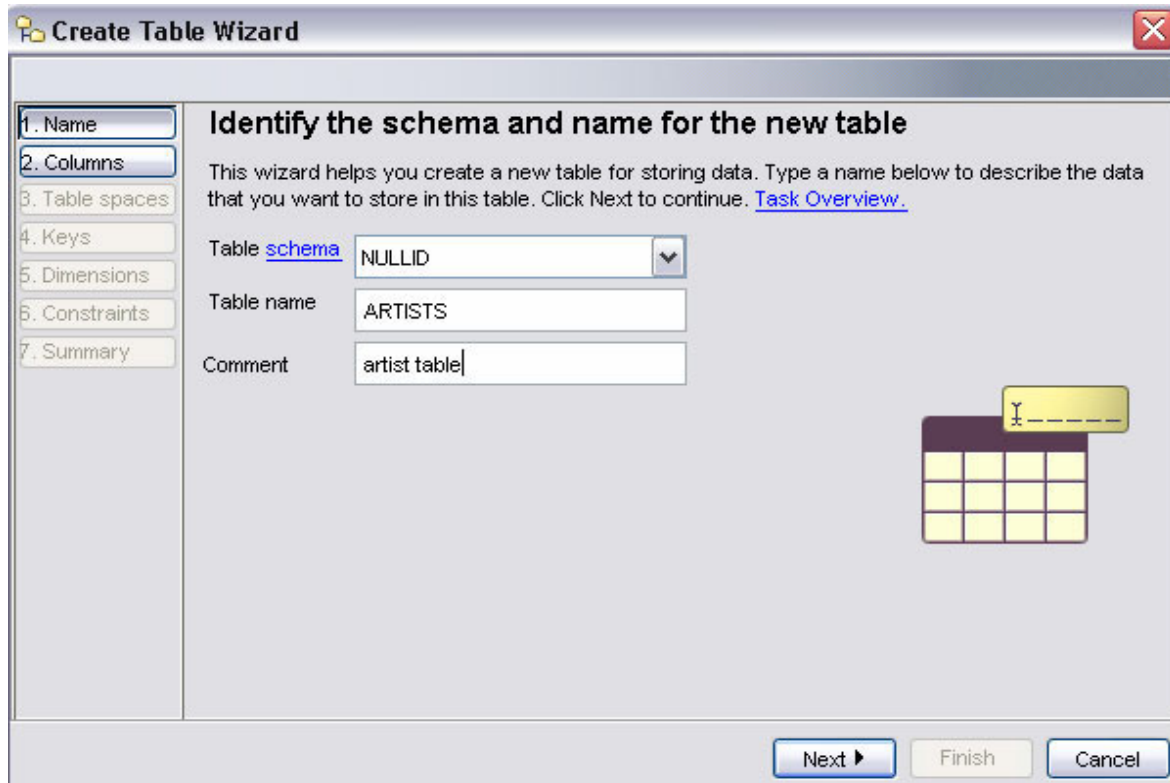
**Purpose** — Identify how the CURRENT SCHEMA special register can be used.

**Details** — Even though the default value is set by the USER special register, you can override the value by setting the CURRENT SCHEMA special register. This will set the value to be used for any unqualified dynamic SQL statements.

**Additional Information** — The Qualifier option on the BIND will be addressed in the Application Programming unit.

**Transition Statement** — Let's look at creating tables.

# Create Table



© Copyright IBM Corporation 2007

Figure 5-6. Create Table

CF238.3

## Notes:

To access the Create Table GUI from the Control Center:

- Select **Tables** (right-click the DB node *Tables*)
  - Select **Create**

The Create Table Wizard has the following pages:

- **Name**

On this page, you specify the schema and name for the table.

- **Columns**

This page displays the columns of the table. On this page, you can add, edit, remove, or rearrange the columns. You can also choose from a list of predefined columns and add them to the table.

---

- **Table Space**

Use this page to specify the table space that will store the data. On this page, you can also launch the Create Table Space wizard if you need to create a table space.

- **Keys**

Use this page to define the primary, unique, and foreign keys for the table. You can add, change, or remove keys. If you are creating the table on a partitioned database, then this page allows you to define the partitioning key.

- A primary key is a column or set of columns that is used to uniquely identify each row on a table. It is used by the database to increase the efficiency of table operations. It is also used in the creation of foreign keys. A table cannot have more than one primary key, and no column of the primary key can contain a NULL value.
- A unique key is a key that is constrained so that no two of its values are equal. The columns of a unique key can contain one NULL value.
- A foreign key is a column or set of columns whose values are required to match at least one primary key value in another table.
- A partitioning key is a key that is part of the definition of a table in a partitioned database. The partitioning key is used to determine the partition on which the row of data is stored. If a partitioning key is defined, unique keys and primary keys must include the same columns as the partitioning key (they may have more columns). A table cannot have more than one partitioning key.

- **Dimensions**

If you are creating a multidimensional clustered table, then use this page to define the dimensions for the table. This is your only opportunity to define the table dimensions. The only way to change the dimensions for a table is to redefine the table and move the data into the new table.

- **Constraints**

This page lists the check constraints for the table, and allows you to add, change, or remove a check constraint. A check constraint sets restrictions on the data in a column that can be added to the table, and is enforced whenever the table has rows inserted or updated.

- **Summary**

This page allows you to review the decisions you have made for the table and to view the command that will create the table.

The items under these tabs will be covered later in this unit.

### ***Instructor notes:***

**Purpose** — Show the GUI Create Table Wizard.

**Details** — The Create Table Wizard can be used to create tables with DB2's help.

Use the Create Table wizard to create new tables in a database. First you name the table, determine the columns that you want, and specify space for storing the table data. From there, you continue on to make selections about keys, dimensions, and constraints.

**Additional Information** — Note that in previous versions of DB2 UDB, there was a GUI that was not the wizard; but now the GUI Create Table only supports the wizard.

**Transition Statement** — Let's look at a command line interface example of creating a table.

## Create Table statement

```
connect to musicdb;
create table artists
(artno          smallint not null,
 name           varchar(50) with default 'abc',
 classification  char(1) not null,
 bio            clob(100K) logged,
 picture        blob( 10M) not logged compact)

in dms01

index in dms02

long  in dms03;
```

© Copyright IBM Corporation 2007

Figure 5-7. Create Table statement

CF238.3

### Notes:

A table consists of data logically arranged in columns and rows. DB2 supports page sizes of 4, 8, 16, and 32 KB. The number of columns, maximum row length, and maximum table size vary by page size. Regular tablespaces use a 4-byte Row ID, which allows 16 million pages with up to 255 rows per page. Non-temporary SMS-managed table spaces are all Regular table spaces. DMS-managed table spaces can be either Regular or Large table spaces. Large table spaces use a 6-byte Row ID, which allows up to 512 million pages to be addressed.

A table with a 4K page could be as large as 64 GB in a regular table space or 2 Terabytes in a Large table space. Using a 32K page size would allow a table in a Regular table space to be as large as 512GB or 16TB in a Large table space.

To create a table, you must be connected to a database, either implicitly or explicitly. You must have SYSADM or DBADM authority, or, you must have CREATETAB privilege and one of either IMPLICIT\_SCHEMA or CREATEIN privilege on the schema. You must also have USE privilege over all table spaces referenced.

Tables are created using the SQL statement CREATE TABLE.

If no schema name is supplied with the table name, the value of the CURRENT SCHEMA special register is used as the schema name.

A table, view, or alias may be from one to 128 characters in length. A column name may be from one to 30 characters in length. An index name may be from one to 18 characters in length.

The data types may be INTEGER, INT, SMALLINT, BIGINT, DOUBLE, DOUBLE PRECISION, FLOAT, REAL, DECIMAL (precision-integer, scale-integer), DEC (precision-integer, scale-integer), NUMERIC (precision-integer, scale-integer), NUM (precision-integer, scale-integer), CHARACTER (integer), CHAR(integer), VARCHAR(integer), CHARACTER VARYING (integer), CHAR VARYING (integer), LONG VARCHAR, BLOB (integer), BINARY LARGE OBJECT (integer), CLOB (integer), CHARACTER LARGE OBJECT (integer), CHAR LARGE OBJECT (integer), DBCLOB (integer), GRAPHIC (integer), VARGRAPHIC (integer), LONG VARGRAPHIC, DATE, TIME, TIMESTAMP, DATALINK, or a user-defined distinct or structured type. More information about user-defined distinct types can be found in the appendix.

**FOR BIT DATA** specifies that the contents of the character or varying character column are to be treated as bit (binary) data.

- The following built-in function is available to use this type of column:
  - *GENERATE\_UNIQUE()* which returns a unique value that can be used to provide unique values in a table. The type of unique value is CHAR(13) FOR BIT DATA.

**NOT NULL** prevents the column from containing null values. If you omit NOT NULL, the column can contain null values, and its default value is the null value.

**NOT NULL WITH DEFAULT** prevents a column from containing null values while allowing system defaults such as:

- 0 for a numeric data type
- Blanks for a fixed-length character string data type
- Current DATE, TIME, and TIMESTAMP for DATE, TIME, and TIMESTAMP data types (respectively).

**WITH DEFAULT value** allows a user-defined default value to be specified, whether or not NOT NULL is specified. This allows the table definer to select the specific default value for use when a column is not specified in an INSERT statement, or when DEFAULT is specified in an INSERT or UPDATE statement.

The CREATE TABLE and ALTER TABLE statements allow a constant value; special registers: CURRENT DATE, CURRENT TIME, CURRENT TIMESTAMP, CURRENT SCHEMA, or USER; or NULL to be specified following the DEFAULT keyword.

Referential integrity syntax is discussed later.

**IN** identifies the table space in which the table will be created. The table space must exist and be a REGULAR table space over which the authorization ID of the statement has USE privilege. If no other table space is specified, then all table parts will be stored in this table space.

If the **IN** clause is not specified, then if the table space IBMDEFAULTGROUP exists and it has sufficient page size, and if the user has USE privilege over the table space IBMDEFAULTGROUP, the table will be created in IBMDEFAULTGROUP. Otherwise, if a table space over which the user has USE privilege exists with sufficient page size, it will be chosen (if multiples exist, see next paragraph). If no table space exists that meets these criteria, the CREATE TABLE statement will fail.

If multiple table spaces exist that meet the page size and USE privilege requirements, the one with the smallest sufficient page size is selected. Preference is given according to who was granted USE privilege:

1. The authorization ID
2. A group to which the authorization ID belongs
3. PUBLIC

If more than one table space still qualifies, the final choice is made by the database manager.

**INDEX IN** identifies the table space in which any index on the table will be stored. This option is allowed only when the primary table space specified in the IN clause is a DMS table space. The specified table space must exist, and be a REGULAR or LARGE DMS table space over which the authorization ID of the statement has USE privilege.

Note that specifying which table space will contain a table's index can only be done when the table is created. The checking of USE privilege over the table space for the index is only carried out at table creation time. The database manager will not require that the authorization ID of a CREATE INDEX statement have USE privilege on the table space when an index is created later.

**LONG IN** identifies the table space in which the values of any long columns (LONG VARCHAR, LONG VARGRAPHIC or LOB data types) will be stored. This option is allowed only when the primary table specified in the IN clause is a DMS table space. The table space must exist and be a LARGE DMS table space over which the authorization ID of the statement has USE privilege.

A table may be created similar to an existing table by using the LIKE clause, for example: `CREATE TABLE painters LIKE artists.` The PAINTERS table will have the same columns (name and description) as the ARTISTS table.

The RENAME TABLE statement renames an existing table. Privileges needed to rename a table are SYSADM, DBADM authority or CONTROL privilege.

For more information, refer to the *SQL Reference* manual.

**Instructor notes:**

**Purpose** — Identify how a table may be created.

**Details** — On the graphic, an illustration is given of creating a table named ARTISTS. Note first that you must be connected to a database to create a table. The CREATE TABLE statement is taught in the prerequisite SQL Workshop, so there is no need to go into it in detail here, but you should point out the items that may not be familiar to the students. Suggested items to address include: WITH DEFAULT does not require the column to be not nullable; LOB columns are for large amounts of data (addressed more on the next page); and table spaces can be defined for regular, index, and long data. Note that you can only specify index and long table spaces if the regular table space is DMS, and all of the table spaces must be DMS.

**Additional Information** — The data type, BIGINT, was introduced in V5.2. A big integer is an 8-byte integer with a precision of 19 digits. The range of big integer is -9223372036854775808 to +9223372036854775807.

Some interesting table alteration options:

- Increase column length for VARCHAR columns (ALTER colname SET DATA TYPE VARCHAR (integer)).
- Activating not logging for a unit of work (ACTIVATE NOT LOGGED INITIALLY [ WITH EMPTY TABLE ] ). The table must have originally been created with the NOT LOGGED INITIALLY attribute.
- Specifying the percentage to be kept free on each data page (PCTFREE integer). This is considered only by LOAD and REORG utilities.
- Identifying the granularity of locks (LOCKSIZE ROW | TABLE). The default lock size is ROW when a table is created.
- Identifying the table is in append mode (APPEND ON | OFF). If append mode is ON, data is appended to the end of table data, and information about free space on pages will not be kept.

**UNICODE Enhancements** — The Unicode character encoding standard is a fixed-length, character encoding scheme that includes characters from almost all the living languages of the world. Unicode characters are usually shown as *U+xxxx* where *xxxx* is the hexadecimal code of the character. In a DB2 database defined as Unicode, the character columns are UTF-8 (code page 1208 for Multi-Byte Character Set) and the graphic columns are UCS-2 (code page 1200 for Double-Byte Character Set).

UNICODE is not part of the DB2 certifications.

Each character is 16 bits (2 bytes) wide, regardless of the language. While the resulting 65536 code elements are sufficient for encoding most of the characters of the major languages of the world, the Unicode standard also provides an extension mechanism that allows for encoding as many as a million more characters. This extension is known as surrogates and is used for encoding some 32-bit characters as two successive code elements.

Certification objective: 2D Ability to create and manipulate the various DB2 objects.



**Transition Statement** — Let's look at table partitioning.

## Table partitioning

- 
- Data organization scheme in which table data is divided across multiple storage objects called data partitions or ranges
    - Each data partition is stored separately
    - These storage objects can be in different table spaces, in the same table space, or a combination of both
  - Benefits
    - Easier roll-in and roll-out of table data
    - Supports very large tables
    - Use Hierarchical Storage Management (HSM) solutions more effectively
    - Easier administration
    - Flexible index placement
    - Better query processing

© Copyright IBM Corporation 2007

Figure 5-8. Table partitioning

CF238.3

### **Notes:**

Storage objects behave much like individual tables, making it easy to accomplish fast roll-in by incorporating an existing table into a partitioned table using the ALTER TABLE...ATTACH statement. Likewise, easy roll-out is accomplished with the ALTER TABLE...DETACH statement. Query processing can also take advantage of the separation of the data to avoid scanning irrelevant data, resulting in better query performance for many data warehouse style queries.

Table data is partitioned as specified in the PARTITION BY clause of the CREATE TABLE statement. The columns used in this definition are referred to as the table partitioning key columns.

This organization scheme can be used in isolation or in combination with other organization schemes. By combining the DISTRIBUTE BY and PARTITION BY clauses of the CREATE TABLE statement, data can be spread across database partitions spanning multiple table spaces. The DB2 organization schemes include:

- DISTRIBUTE BY HASH

- PARTITION BY RANGE
- ORGANIZE BY DIMENSIONS

Table partitioning functionality is available with DB2 Version 9.1 Enterprise Server Edition for Linux, UNIX, and Windows.

The following example creates a table **customer** where rows with `l_shipdate >= '01/01/2006'` and `l_shipdate <= '03/31/2006'` are stored in table space **ts1**, rows with `l_shipdate >= '04/01/2006'` and `l_shipdate <= '06/30/2006'` are in table space **ts2**, and so forth.

```
CREATE TABLE customer (l_shipdate, DATE, l_name CHAR(30))
  IN ts1, ts2, ts3, ts4, ts5
  PARTITION BY RANGE(l_shipdate) (STARTING FROM ('01/01/2006')
  ENDING AT ('12/31/2006') EVERY (3 MONTHS))
```

Large objects for a partitioned table reside, by default, in the same table space as the data. This default behavior can be overridden by using the `LONG IN` clause to specify one or more table spaces for the large objects. Create a table named **DOCUMENTS** whose large object data is to be stored (in a round-robin fashion for each data partition) in table spaces **TBSP1** and **TBSP2**.

```
CREATE TABLE documents
  id INTEGER,
  contents CLOB
  ) LONG IN tbsp1, tbsp2
  PARTITION BY RANGE (id) (STARTING 1 ENDING 1000 EVERY 100)
```

Alternatively, use the long form of the syntax to explicitly identify a large table space for each data partition. In this example, the CLOB data for the first data partition is placed in **TBSP3**, and the CLOB data for the remaining data partitions is spread across **TBSP1** and **TBSP2** in a round-robin fashion.

```
CREATE TABLE documents
  id INTEGER,
  contents CLOB
  ) LONG IN tbsp1, tbsp2
  PARTITION BY RANGE (id)
    (STARTING 1 ENDING 100 LONG IN TBSP3,
    STARTING 101 ENDING 1000 EVERY 100)
```

## ***Instructor notes:***

**Purpose** — This topic is an introduction to table partitioning, also known as range partitioning. This is not the same as DPF.

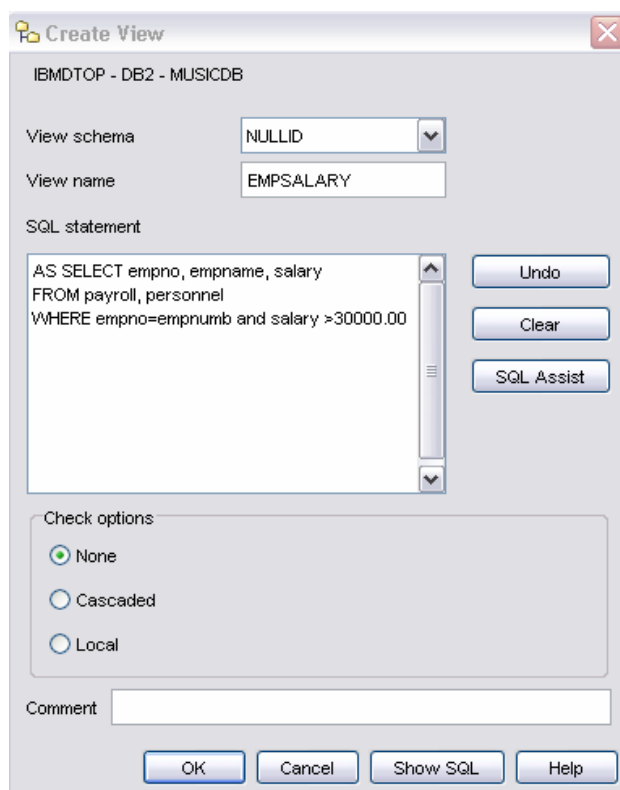
**Details** —

**Additional information** — Ensure the students know the difference between DPF and table partitioning.

Certification objective: 2G Knowledge of partitioning capabilities (for example, Table partitioning, Hash partitioning, MDC, Hybrid).

**Transition statement** — Let's look at creating a view.

# Create View



© Copyright IBM Corporation 2007

Figure 5-9. Create View

CF238.3

## Notes:

To access the Create View GUI from the Control Center:

- Select **Views** (right-click the DB node *Views*)
  - Select **Create**

The Create Views GUI allows you to perform the following during the creation of a view:

- Indicate the schema and name the view
- Formulate SQL statement, with optional assistance from SQLAssist
- Select check options
- Comment on the view

***Instructor notes:***

**Purpose** — Identify how to create a view using the GUI.

**Details** — The GUI screen doesn't help as much with the creation of a view as with other syntax, though you can use SQLAssist to help with creating the SELECT statement, it still requires composing the SQL SELECT statement.

**Additional Information** —

**Transition Statement** — Let's look at the statement interface for creating a view.

## Create View statement

```
CONNECT TO TESTDB;
CREATE VIEW EMPSALARY
  AS SELECT EMPNO, EMPNAME, SALARY
  FROM PAYROLL, PERSONNEL
  WHERE EMPNO=EMPNUMB AND SALARY > 30000.00;

SELECT * FROM EMPSALARY
```

| EMPNO | EMPNAME         | SALARY     |
|-------|-----------------|------------|
| 10    | John Smith      | 1000000.00 |
| 20    | Jane Johnson    | 300000.00  |
| 30    | Robert Appleton | 250000.00  |
| ...   |                 |            |

© Copyright IBM Corporation 2007

Figure 5-10. Create View statement

CF238.3

### Notes:

A view is an alternate representation of data from one or more tables. It can include some or all of the columns contained in the tables on which it is defined.

To create a view, you must be connected to a database — either implicitly or explicitly — and the base tables or views upon which the view is based must previously exist.

Views can be created using the SQL statement CREATE VIEW.

You must have SYSADM, DBADM, CONTROL, or SELECT privilege on each base table to create a view. **Privileges on the base tables granted to groups are not checked to determine authorization to create a view.** However, if the base table has SELECT privilege given to PUBLIC, a view could be created. In addition, you must have the IMPLICIT\_SCHEMA privilege or the CREATEIN privilege on the schema used.

Views may be used to exclude users from seeing certain data: rows or columns. The WHERE clause used in the CREATE VIEW statement determines which rows may be viewed by the user. The columns listed in the AS SELECT clause determine which columns may be viewed by the user.

Views can also be used to increase the access rights to data for a special user group.

Views may be used to improve performance. If a *difficult* SQL statement is to be used by users, it may be advantageous to create a view that is coded to utilize an index, or to ensure that a join is correctly coded.

Data for a view is not separately stored. The data is stored in the base tables.

When an object is dropped, views can become **inoperative** if they are dependent on that object. To recover an inoperative view, determine the SQL statement that was initially used to create the view. This information can be obtained from the SYSCAT.VIEWS.TEXT column. Recreate the view by using the CREATE VIEW statement with the same view name. Use the GRANT statement to regrant all privileges that were previously granted on the view. If you do not want to recover an inoperative view, you can explicitly drop it with the DROP VIEW statement.

An inoperative view only has entries in the SYSCAT.TABLES and SYSCAT.VIEWS catalog views. All entries in the SYSCAT.VIEWDEP, SYSCAT.TABAUTH, and SYSCAT.COLUMNS catalog views are removed.

```
CREATE VIEW view-name (column-name { ,column-name }) AS fullselect
    { WITH [ CASCADED | LOCAL ] CHECK OPTION }
```

WITH CHECK OPTION specifies the constraint that every row that is inserted or updated through the view must conform to the definition of the view. WITH CHECK OPTION must not be specified if the view is read-only. If WITH CHECK OPTION is specified for an updatable view that does not allow inserts, then the constraint applies to update only. If WITH CHECK OPTION is omitted, the definition of the view is not used in the checking of any insert or update operations that use the view. Some checking might still occur during insert or update operations if the view is directly or indirectly dependent on another view that includes WITH CHECK OPTION.

CASCADED causes the constraints of all dependent views to also be applied.

LOCAL causes the constraints of only this view to be applied.

A view may be defined on a view.

A read-only view cannot be the object of an INSERT, UPDATE, or DELETE statement.

For more information, refer to the *SQL Reference* manual.



***Instructor notes:***

**Purpose** — Identify how to create a view using a statement.

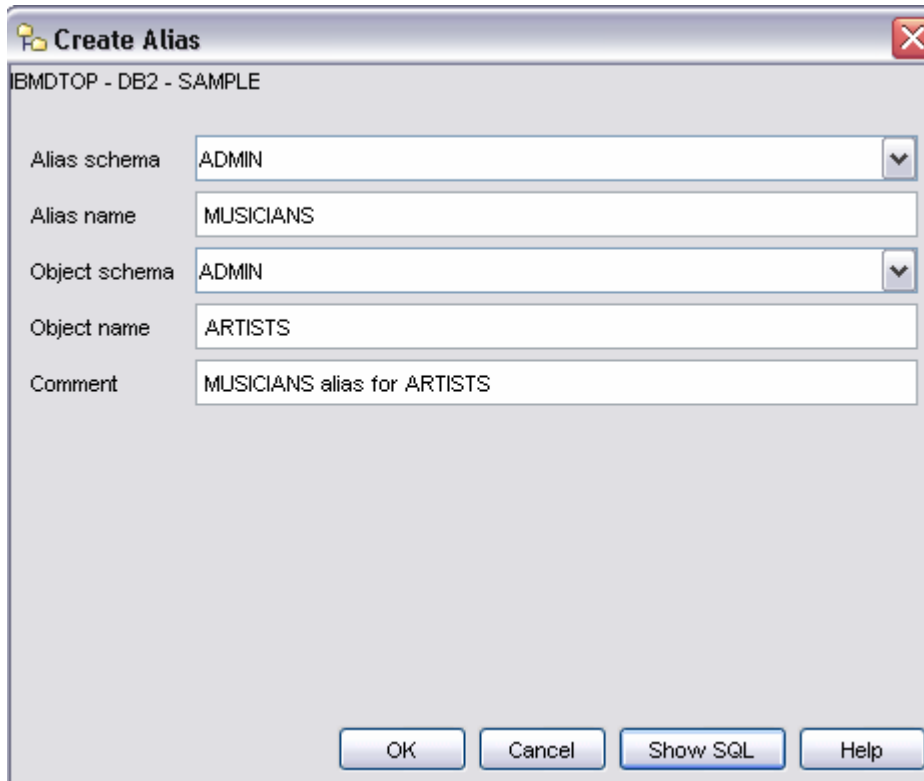
**Details** — The syntax of creating a view should be known by the students from prerequisite education. You should not need to spend much time on this page. You should probably delay the discussion of permissions required to create a view until the security unit.

**Additional Information** — In the student notes, there is a statement that says that a read-only view cannot be the subject of a delete, insert, or update statement. You can create an INSTEAD OF trigger for the base table, which forces the data to be deleted, inserted or updated and by-pass the read-only view; but technically, that makes it no longer a read-only view.

Certification objective: 2D Ability to create and manipulate the various DB2 objects, 3C Ability to create views on tables.

**Transition Statement** — Let's look at creating an alias.

# Create Alias

The image shows a 'Create Alias' dialog box from the IBM DB2 Admin Workshop. The title bar says 'Create Alias' with a close button. Below the title bar, it says 'IBMDTOP - DB2 - SAMPLE'. The dialog has five input fields: 'Alias schema' with a dropdown menu showing 'ADMIN', 'Alias name' with a text box containing 'MUSICIANS', 'Object schema' with a dropdown menu showing 'ADMIN', 'Object name' with a text box containing 'ARTISTS', and 'Comment' with a text box containing 'MUSICIANS alias for ARTISTS'. At the bottom, there are four buttons: 'OK', 'Cancel', 'Show SQL', and 'Help'.

© Copyright IBM Corporation 2007

Figure 5-11. Create Alias

CF238.3

## Notes:

To access the Create Alias GUI from the Control Center:

- Select **Aliases** (right-click the DB node *Aliases*)
  - Select **Create**

The Create Alias GUI allows you to perform the following during the creation of a alias:

- Select alias schema
- Provide alias name
- Identify schema and name of object (table, view, alias) on which the alias will be created
- Comment on the alias

***Instructor notes:***

**Purpose** — Identify how an alias can be created through the GUI.

**Details** — Creating an alias is very easy: just select the schema, indicate the name for the alias, select the schema for the object, and indicate the name of the object.

**Additional Information** —

**Transition Statement** — Let's look at the command line interface for creating the alias.

## Create Alias statement

- Cannot be the same as an existing table, view, or alias
- Can only refer to a table in the same database

```
CONNECT TO SAMPLE;  
CREATE ALIAS ADMIN.MUSICIANS FOR ADMIN.ARTISTS;  
COMMENT ON Alias ADMIN.MUSICIANS IS 'MUSICIANS  
  alias for ARTISTS';  
CONNECT RESET;
```

© Copyright IBM Corporation 2007

Figure 5-12. Create Alias statement

CF238.3

### Notes:

An alias is an indirect method of referencing a table or a view, so that an SQL statement can be independent of the qualified name of that table or view. Only the alias definition must be changed if the table or the view name changes.

An alias can be created on another alias.

An alias can be used in a view or trigger definition and in any SQL statements, except for constraint definitions, in which an existing table or view name can be referenced.

An alias can be defined for a table, view, or alias that does not exist. However, the object must exist when an SQL statement containing the alias is compiled.

When an alias, or the object to which an alias refers, is dropped, all packages dependent on the alias are marked invalid, and all views and triggers dependent on the alias are marked inoperative.

CREATE SYNONYM is an alternative for CREATE ALIAS. In DB2 for Linux, UNIX, and Windows, synonyms and aliases have exactly the same characteristics.

***Instructor notes:***

**Purpose** — Identify the characteristics of an alias.

**Details** — An alias gives you another way to refer to an object in the database. It is similar to creating a view on a table that selects all columns and all rows; it just gives another name for that set of data. One difference of an alias from a view, however, is that the object doesn't have to exist at the time the alias is created; only at the time the alias is used.

**Additional Information** — CREATE SYNONYM is an alternative for CREATE ALIAS.

**Transition Statement** — Let's look at declared temporary tables.

## Overview of Declared Temporary Tables



```
CREATE USER TEMPORARY TABLESPACE
  "USR_TEMP_TS"
  PAGESIZE 4 K  MANAGED BY AUTOMATIC STORAGE
  BUFFERPOOL  IBMDEFAULTBP ;

DECLARE GLOBAL TEMPORARY TABLE T1
  LIKE REAL_T1
  ON COMMIT DELETE ROWS
  NOT LOGGED
  IN USR_TEMP_TS;

INSERT INTO SESSION.T1
  SELECT * FROM REAL_T1 WHERE DEPTNO=:mydept;

/* do other work on T1 */
/* when connection ends, table is automatically dropped */
```

© Copyright IBM Corporation 2007

Figure 5-13. Overview of Declared Temporary Tables

CF238.3

### Notes:

A declared temporary table is a temporary table that is only accessible to SQL statements that are issued by the application which created the temporary table. A declared temporary table does not persist beyond the duration of the connection of the application to the database.

Use declared temporary tables to potentially improve the performance of your applications. When you create a declared temporary table, DB2 does not insert an entry into the system catalog tables, and, therefore, your server does not suffer from catalog contention issues. In comparison to regular tables, DB2 does not lock declared temporary tables or their rows. If your current application creates tables to process large amounts of data and drops those tables once the application has finished manipulating that data, consider using declared temporary tables instead of regular tables.

**To use a declared temporary table, perform the following steps:**

- Step 1: Ensure that a USER TEMPORARY TABLESPACE exists. If a USER TEMPORARY TABLESPACE does not exist, issue a CREATE USER TEMPORARY TABLESPACE statement. A USER TEMPORARY TABLESPACE stores declared

temporary tables. No user temporary table spaces exist when a database is created. At least one user temporary table space should be created with appropriate USE privileges to allow definition of declared temporary tables.

```
create user temporary tablespace usr_temp_ts managed by system
using ('c:\usrspace')
```

- Step 2: Issue a DECLARE GLOBAL TEMPORARY TABLE statement in your application.

The definition on the graphic creates a user temporary table called T1. The user temporary table is defined with columns that have exactly the same name and description as the columns of the REAL\_T1. The implicit definition only includes the column name, datatype, nullability characteristic, and column default value attributes. All other column attributes including unique constraints, foreign key constraints, triggers, and indexes are not defined. You can also specify column definitions as when creating a persistent table. See the *SQL Reference* for more information.

If you specify a schema for your temporary table, it must be SESSION.

## ***Instructor notes:***

**Purpose** — Introduce declared temporary tables.

**Details** — Prior to V7, if an application wanted a table to use just for its duration, a permanent table had to be created. This caused several issues:

- If an application was being invoked by several users at the same time, some way to ensure that they all kept their own separate tables needed to be determined.
- The creation of the permanent table impacted the catalog and could cause lock contention between the application working with the table and other applications wanting to select through the catalog.

Applications used mechanisms such as NOT LOGGED INITIALLY to reduce the amount of work required (avoiding the logging). However, the name contention and catalog contention were still issues.

In V7 and after, DB2 supports application temporary tables. Instead of being created with a CREATE TABLE statement, they are created with a DECLARE GLOBAL TEMPORARY TABLE statement. Declared temporary tables are not entered into the system catalog, DB2 keeps track of separate temporary tables created by separate invocations of the same application, and locking is avoided.

The definition of a declared temporary table requires a user temporary table space, and the USE privilege on that table space. When declaring a temporary table, it can be defined with columns or LIKE another table or AS SELECT... DEFINITION ONLY. The default action ON COMMIT is to DELETE ROWS. NOT LOGGED is optional syntax in V8 and after. In the EEE environment, a partitioning key may also be identified.

You can create indexes and run statistics on declared temporary tables on V8 and after.

**Additional Information** — Student question: Where does DB2 store the definition of the temporary table?

Table descriptor information is stored in app\_ctl\_heap\_sz. The optimization level used would be consistent with the bind time choice or the setting of CURRENT QUERY OPTIMIZATION.

Student question: Why are global temporary tables called global?

We think it means that a subroutine of the application can use the temporary table. Does it also mean that multiple threads of the same application can use the temporary table?

The understanding is correct. Any runnable entity (whether it be a thread of the application or a routine that the application calls) that can issue SQL under the connection established by the application can see the declared temporary table.

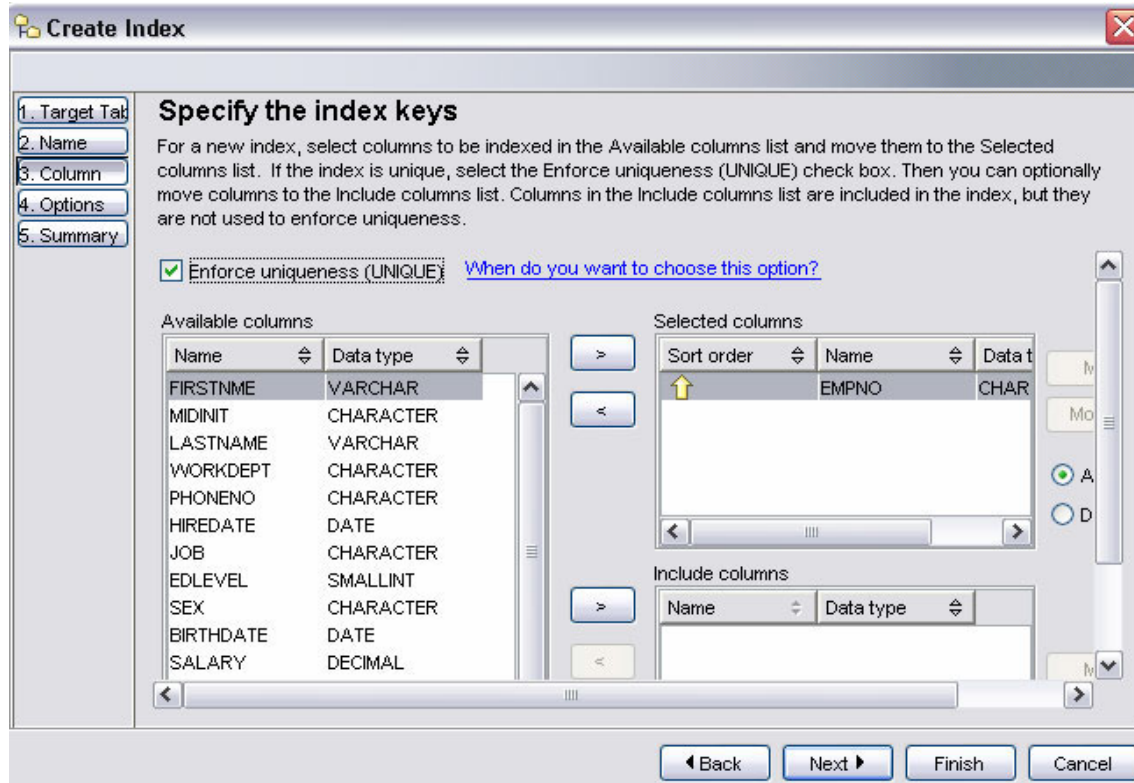
A local temporary table (not supported at this point by DB2) would only be accessible in the subconnection scope that created it. That is, if an application created a local temporary table and then called a procedure, that temporary table would not be accessible by the procedure (and vice versa).

Certification objective: 2D Ability to create and manipulate the various DB2 objects.



**Transition Statement** — Let's look at creating indexes.

# Create Index



© Copyright IBM Corporation 2007

Figure 5-14. Create Index

CF238.3

## Notes:

To access the Create Index GUI from the Control Center:

- Select **Indexes** (right-click the DB node *Indexes*)
  - Select **Create Index**

The Create Indexes GUI allows you to perform the following during creation of an index:

- Select index and table schemas
- Provide index name
- Identify table for which the index is to be created
- Select columns in the index; the sequence of selection determines the order of columns in the composite key
- Define ascending or descending order
- Define if the index is unique, clustered, or if it allows reverse scans
- Define the percentage of free space for index pages
- Define the minimum amount of used space prior to online index leaf page defragmentation
- Comment on the index

***Instructor notes:***

**Purpose** — Show the GUI screen for creating an index.

**Details** — The GUI screen prompts for the specification of the index schema (available from a pull-down list), index name (which needs to be typed in), and table schema and name that can be selected from a pull-down list. After the table is selected, its columns will show up in the Available columns list. Selecting one or more columns allows you to move them to the selected columns.

Other options will be addressed on the next page.

**Additional Information** —

**Transition Statement** — Let's look at creating indexes from the command line.

## Create Index statement



```
create unique index dba1.empno on
  dba1.employee (empno asc)
pctfree 10
minpctused 10
allow reverse scans
page split symmetric
collect sampled detailed statistics ;

create unique index itemno on albums (itemno)

create index item on stock (itemno) cluster

create unique index empidx on employee (empno)
  include (lastname, firstname)
```

© Copyright IBM Corporation 2007

Figure 5-15. Create Index statement

CF238.3

### Notes:

Indexes may allow more efficient access to rows in a table. Unique indexes ensure uniqueness of the index key. An index key is a column or an ordered collection of columns on which an index is defined.

For an index, column-name identifies a column that is to be part of the index key. For an index specification, column-name is the name by which the federated server references a column of a data source table. Each column-name must be an unqualified name that identifies a column of the table. Up to 64 columns can be specified. If table-name is a typed table, up to 63 columns can be specified. If table-name is a subtable, at least one column-name must be introduced in the subtable; that is, not inherited from a supertable (SQLSTATE 428DS). No column-name can be repeated (SQLSTATE 42711). The sum of the stored lengths of the specified columns must not be greater than the index key length limit for the page size. For key length limits, see *SQL limits*. Note that this length limit can be reduced even more by system overhead, which varies according to the data type of the column and whether or not the column is nullable.

To create an index, you must be connected to a database — either implicitly or explicitly — and the base table upon which the index is based must previously exist.

Indexes can be created using the SQL statement `CREATE INDEX`.

You can use the Design Advisor to recommend indexes by identifying the SQL statements to be executed and the frequency of their use (that is, workload). The Design Advisor helps to improve performance of your database by suggesting and creating an optimal set of indexes.

You must have `SYSADM`, `DBADM`, `CONTROL`, or `INDEX` privilege on the table to create an index. You must also have either the `IMPLICIT_SCHEMA` privilege or the `CREATEIN` privilege on the schema used.

Indexes may be used to improve performance. An index can be used to search a table for matching data versus doing a table scan to search for matching data.

Indexes may be bidirectional (`ALLOW REVERSE SCANS`) and can support forward and backward scans. This is the default condition when creating an index. These indexes will facilitate `MIN` and `MAX` functions, allow fetch of the previous key, eliminate the need for the optimizer to create a temporary table for a reverse scan, and eliminate the need for a redundant reverse order index.

Data for an index is stored separately. The index columns are stored in the index with a pointer to the base table's row.

An index may be based on one or more columns in the table. An index may be stored in `ASCending` or `DESCending` order. Ascending order is the default.

The `UNIQUE` option prevents the table from containing two or more rows with the same value of the index key. The constraint is enforced when rows of the table are updated or new rows are inserted. The constraint is also checked during the execution of the `CREATE INDEX` statement. If the table already contains rows with duplicate key values, the index is not created. When `UNIQUE` is used, null values are treated as any other values. The column may contain no more than one null value.

By creating a unique index with include columns, you can improve the performance of data retrieval by increasing the use of index-only accesses.

A clustering index indicates that data records should be clustered on pages based on the sequence of a the index, and maintains that clustering as much as possible over the course of insert activity. Clustering increases the efficiency of data retrieval when it involves accessing sequential value ranges for the clustering index.

If the named table already contains data, `CREATE INDEX` creates the index entries for it. If the table does not yet contain data, `CREATE INDEX` creates a description of the index; the index entries are created when data is inserted into the table.

```
CREATE [UNIQUE] INDEX index-name ON table-name
    (column-name [ASC|DESC] [, column-name])
    [ INCLUDE (column-name [, column-name]) ]
    [ CLUSTER ] [ PCTFREE integer ] [ MINPCTUSED integer ]
```

```
[ DISALLOW REVERSE SCANS | ALLOW REVERSE SCANS ]  
[ COLLECT [ [ SAMPLED ] DETAILED ] STATISTICS ]
```

The **PCTFREE** can be used to control the amount of free space left on an index page when an index is created. The default is 10% and the valid range of values is 0-90. By setting the value to a higher number, index page splits can be minimized. By setting the value to a lower number, fewer pages could be required to store the index for a read-only table.

**MINPCTUSED** indicates whether indexes are defragmented online and the threshold for the minimum percent of space used on an index leaf page. If, after a key is deleted from an index leaf page, the percentage of space used on the page is at or below the percentage specified for MINPCTUSED, an attempt is made to merge the remaining keys on this page with those of a neighboring page.

**PAGE SPLIT** specifies an index split behavior. The default is SYMMETRIC, which specifies that pages are to be split roughly in the middle.

**COLLECT STATISTICS** specifies that index statistics are to be collected during index creation.

For more information, refer to the *SQL Reference* manual.

**Instructor notes:**

**Purpose** — Identify considerations when creating indexes.

**Details** — The first statement shows the creation of a unique index on the albums table. DB2 will ensure the uniqueness of data in the itemno column of the ALBUMS table.

The second example shows that indexes can be created as clustered. This might be a little bit of a confusing term, as indexes themselves are *always* clustered. By defining a clustered index, you are asking DB2 to store the data in the table in sequence according to the clustered index. This is just a *request*, though; DB2 will not perform page splits to store the row on the desired page; so, as free space runs out in the table, the index will become less clustered.

The last example shows another unique index. In this case, we want to create a unique index on the column EMPNO of the table EMPLOYEE. But, by analyzing the queries that run against the EMPLOYEE table, we have found that frequently, when a look-up is being done by employee number, the data of interest is found in columns LASTNAME and FIRSTNAME (for example, those are the columns specified on the SELECT clause when the WHERE clause provides a value for EMPNO). With this being the case, we can use a capability of DB2: that of creating a unique index with include columns. The include columns are carried as data on the leaf pages of the index. You cannot do searches based on the values of the include columns, but by having the include columns in the index, you may be able to avoid data page access if all the data of interest is found in the index.

**Additional Information** — There are many considerations associated with definitional clustering; possibly the hardest to determine is the appropriate size for PCTFREE. This information is beyond the scope of this course. This information is included in the performance course, CF41.

Include columns can give you the ability to have a single index that both ensures uniqueness and provides index-only access to frequently accessed data.

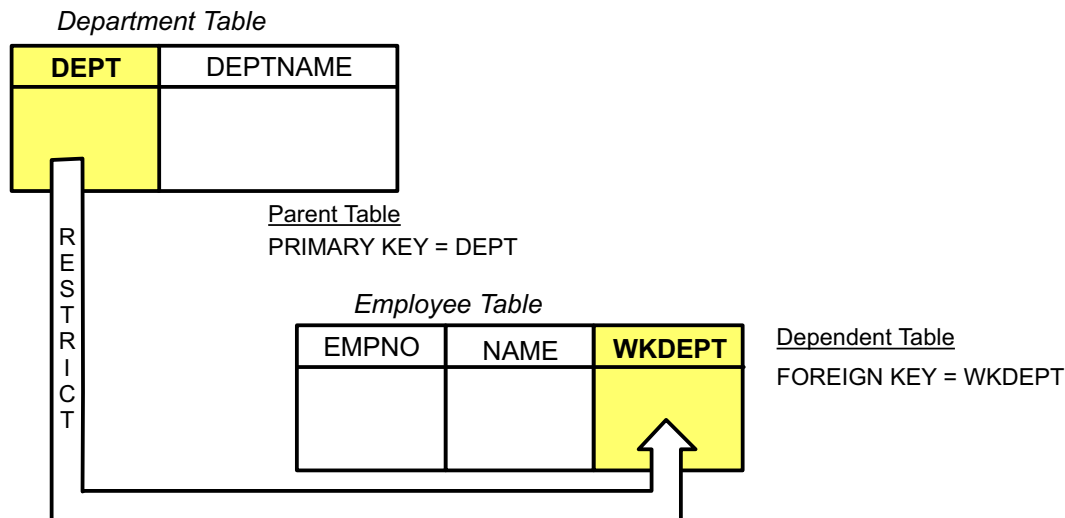
COLLECT STATISTICS is new syntax in V8. It does not appear on the graphic, but is in the student notes.

Certification objective: 2D Ability to create and manipulate the various DB2 objects, 3A Knowledge of the creation and management of indexes, 3B Ability to create constraints on tables (for example, RI, Informational, Unique, 3F Knowledge of how to enforce data uniqueness.

Certification objective: 2D Ability to create and manipulate the various DB2 objects.

**Transition statement** — Let's look at referential integrity.

# Overview of Referential Integrity



- Place constraints between tables
- Constraints specified with Create and Alter table statements
- Database services enforce constraints: inserts, updates, deletes
- Removes burden of constraint checking from application programs

© Copyright IBM Corporation 2007

Figure 5-16. Overview of Referential Integrity

CF238.3

## Notes:

Referential Integrity (RI) lets you define required relationships between and within tables. The database manager maintains these relationships, which are expressed as *referential constraints*, and requires that all values of a given attribute or column of a table also exist in some other table or column. Since DB2 enforces RI, benefits include greater application development productivity (no code is required in the application to enforce RI) and data integrity and consistency, since DB2 consistently enforces the RI requirements in all environments.

In the event of deletion of a row in the parent table, constraints can be specified that the dependent rows are either completely deleted along with the parent data, or that only the dependent data is removed from the rows leaving a null value in the otherwise intact rows, or that the parent row cannot be deleted since dependent rows exist.

**Primary Key** is a unique, non-null key. There can be none or one primary key per table. The primary key can comprise one to 16 columns, maximum length is 1024 bytes, and all of the columns in the primary key must be defined as NOT NULL. When a primary key is specified, Database Services automatically creates a **unique index** in ascending order on



that key if one does NOT exist. The unique index name, if DB2 automatically creates it for you, will be SYSIBM.SQLyyyyymmddhhmmssx (15-character timestamp). If a constraint name is specified, the index name is table\_schema.constraint\_name. The UNIQUERULE column in the SYSCAT.INDEXES catalog view will be **P**, representing a primary index.

A **Unique Constraint** is a rule that specifies that values of a key must be kept unique within the table. The columns specified in a unique constraint must be defined as NOT NULL. A unique index is used by DB2 to ensure uniqueness. A table can have any number of unique constraints, but only one on any given set of columns.

A **Parent Key** is a unique constraint or primary key that is referenced by a foreign key.

**Foreign Key** is a non-null value that matches the value of an associated parent key in the same table or a different table. The value is null if any nullable part of the key is null. A table can have none, one, or many foreign keys. Since more than one foreign key can be associated with a table, a **constraint name** may be defined by the user when the foreign key is defined. If no name is defined, Database Services assigns one using the format SQLnnnnn where nnnnn is a unique sequence (within the object table) of five numeric digits. The parent table with its parent key must exist before the foreign key is specified.

A **Parent Table** has a parent key that is referenced by one or more foreign keys.

A **Dependent Table** is a table with at least one foreign key.

A **Descendent Table** contains a foreign key that can be traced back to a parent key of a table. A table is a **descendent** if it is a dependent table or if it is a descendent of a dependent.

A **Dependent Row** is a row of a dependent table that has a non-null foreign key value that matches a parent key value of its parent table.

A **Parent Row** is a row of a parent table whose parent key value matches at least one foreign key value in a dependent table. When insert, update, and delete operations are performed on a parent table:

1. A row can only be inserted into a parent table if the value of the parent key is non-null and unique.
2. The parent key value of a parent row cannot be updated. The old row would need to be deleted and a new row inserted.
3. When a row is deleted from a parent table, a check is performed to determine if there are any dependent rows in the dependent table with matching foreign key values. If any dependent rows are found, the action taken is prescribed by the **Delete Rule** specified in the referential constraint.

### Delete Rules

- a. **Restrict or No Action** — A row of a parent table cannot be deleted if one or more rows exist in the dependent tables which have a foreign key value matching the parent key value.

- b. **Cascade** — If a row in a parent table is deleted, then all rows in the dependent tables with a foreign key matching the parent key value of this row will also be deleted.
- c. **Set Null** — If a row in a parent table is deleted, then all rows in the dependent tables with a foreign key matching the parent key value of this row will have their foreign key value changed to null. All columns of the foreign key which are nullable will be set to null. SET NULL can only be specified if at least one of the columns comprising the foreign key is nullable.

***Instructor notes:***

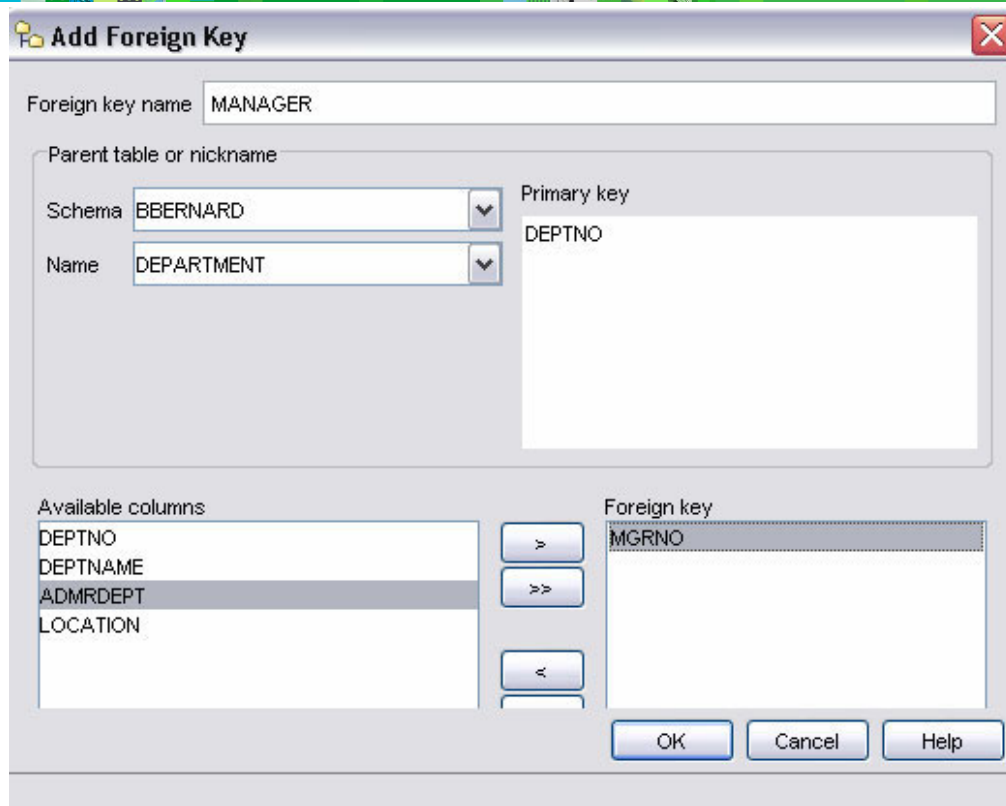
**Purpose** — Provide definitions of referential integrity concepts.

**Details** — Students in this class should already have been introduced to referential integrity by the prerequisite training, but you can use this opportunity to review the important points. One thing that they may not know from prerequisite training is that the parent key can be either a primary key or a unique constraint (but not a unique index).

**Additional Information** — Certification objective: 3B Ability to create constraints on tables (for example, RI, Informational, Unique).

**Transition Statement** — Continue by showing the GUI screen for defining a foreign key.

## RI — Add a foreign key



© Copyright IBM Corporation 2007

Figure 5-17. RI — Add a foreign key

CF238.3

### Notes:

To create a referential constraint during table creation:

- Right-click and select **Tables**
    - Select **Create**
      - Select Primary Keys where you can choose what to create (Primary, Unique, Foreign, or Partitioning key)
- Primary key is set on the parent table and foreign key is set on the dependent table.

A referential constraint can also be created between tables through the alter table GUI.

The other possibility is to highlight the table you want to change, right-click and select **ALTER**. There you will find a *KEYS* tab where you have the same choices as above.

***Instructor notes:***

**Purpose** — Identify the GUI interface for defining the foreign key.

**Details** — Once you fill in the parent table's schema and name, the GUI will show you the parent key options on that table. You then can select the foreign key definition: the column(s) that make up the foreign key and the delete rule.

**Additional Information** —

**Transition Statement** — Let's look at the command line interface.

## Referential Integrity — Create Table statement



```
CREATE TABLE DEPARTMENT
    (DEPT CHAR (3) NOT NULL,
    DEPTNAME CHAR(20) NOT NULL,
    CONSTRAINT DEPT_NAME
    UNIQUE (DEPTNAME),
    PRIMARY KEY (DEPT))

IN DMS99 ....
```

```
CREATE TABLE EMPLOYEE
    (EMPNO CHAR(6) NOT NULL,
    NAME CHAR(30),
    WKDEPT CHAR(3) NOT NULL,
    CONSTRAINT DEPT
    FOREIGN KEY
    (WKDEPT)
    REFERENCES DEPARTMENT
    ON DELETE RESTRICT)

IN SMS99 ...
```

© Copyright IBM Corporation 2007

Figure 5-18. Referential Integrity — Create Table statement

CF238.3

### Notes:

A unique constraint on any column or set of columns will not allow duplicate values in those columns. In the example above, a unique index called DEPT\_NAME will automatically be created in the same schema to enforce the unique constraint.

To create a table and include referential constraints in its definition, SYSADM, DBADM, or CREATETAB privilege on the database and REFERENCES or CONTROL privilege on the parent table are required. The CREATE TABLE statement allows specification of referential constraints at table creation time.

To create referential constraints on an existing table, at least SYSADM, DBADM, or CONTROL or ALTER privilege on the dependent table plus REFERENCES privilege on the parent table are required. The ALTER TABLE statement allows specification of referential constraints on existing tables. Data already in the table must meet the constraints, or the constraint cannot be created. The parent key must be defined on the parent table before a related foreign key may be defined.

A referential constraint is implicitly dropped:

- When the parent or dependent table of the relationship is dropped
- When the definition of the parent key of the parent table is dropped.

A second table could be defined that allows for look-up of department functions by department name, with a referential constraint on the DEPARTMENT.DEPTNAME as its parent key:

```
CREATE TABLE dept_function
  keyword    CHAR(25) NOT NULL,
  name       CHAR(20) NOT NULL,
  CONSTRAINT dn FOREIGN KEY (name)
  REFERENCES department (deptname)
  ON DELETE CASCADE
) IN sms01
```

## ***Instructor notes:***

**Purpose** — Identify how to create referential constraints using the command line interface.

**Details** — This graphic shows two tables. DEPARTMENT is defined with a primary key on the DEPT column. EMPLOYEE can then be defined with a foreign key that references the DEPARTMENT table. Since we don't specify the column of the DEPARTMENT table, it is assumed to be the primary key.

The department table has two parent keys defined: a primary key on DEPT and a unique constraint on DEPTNAME. It may not make business sense to have a unique constraint on DEPTNAME, but it is possible to make this definition and then have some tables use DEPT as the parent key, and other tables use DEPTNAME as the parent key. In the student notes, there is an example of a DEPT\_FUNCTION table that uses DEPTNAME as the parent key.

Point out the ability to define a referential constraint on either a primary key or a unique constraint on a table.

**Additional Information** — Certification objective: 3B Ability to create constraints on tables (for example, RI, Informational, Unique).

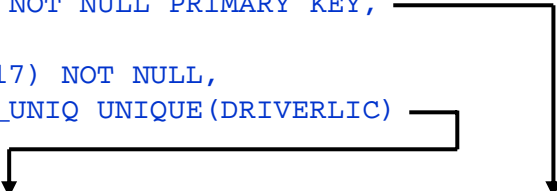
**Transition Statement** — A more common example of the need to have multiple parent keys on a table is on the next page.



## Unique Key considerations

- Multiple keys in one table can be foreign key targets

```
CREATE TABLE PAY.EMPTAB
  (EMPNO SMALLINT NOT NULL PRIMARY KEY,
   NAME CHAR(20),
   DRIVERLIC CHAR(17) NOT NULL,
   CONSTRAINT DRIV_UNIQ UNIQUE(DRIVERLIC)
  ) IN TBSP1
```



Unique indexes PAY.DRIV\_UNIQ and SYSIBM.yymmddhhmmssxxx created Columns must be NOT NULL

- Deferral of unique checking until end of statement processing
  - UPDATE EMPTAB SET EMPNO=EMPNO + 1

© Copyright IBM Corporation 2007

Figure 5-19. Unique Key considerations

CF238.3

### Notes:

A unique constraint is the rule that the values of a key are valid only if they are unique within the table. Unique constraints are optional and can be defined in the CREATE TABLE or ALTER TABLE statement using the PRIMARY KEY clause or the UNIQUE clause. The columns specified as a unique constraint must be defined as NOT NULL. A unique index is used by the database manager to enforce the uniqueness of the key during changes to the columns of the unique constraint.

A table can have an arbitrary number of unique constraints, with at most one unique constraint defined as a primary key. A table cannot have more than one unique constraint on the same set of columns.

A unique constraint that is referenced by the foreign key of a referential constraint is called a parent key. When a unique constraint is defined in a CREATE TABLE statement, a unique index is automatically created by the database manager and designated as a primary or unique system-required index. When a unique constraint is defined in an ALTER TABLE statement and an index exists on the same columns, that index is designated as unique and system-required. If such an index does not exist, the unique index is

automatically created by the database manager and designated as a primary or unique system-required index. CONSTRAINT constraint-name identifies the constraint. If this clause is omitted, an 18-character identifier, unique within the identifiers of the existing constraints defined on the table, is generated by the system. When used with a PRIMARY KEY or UNIQUE constraint, the constraint-name will be used as the name of an index that is created to support the constraint if a unique index does not exist. If the CONSTRAINT constraint-name clause is not used, the system-generated index will be named SYSIBM.SQlyyyymmddhhmmssx.

PRIMARY KEY provides a shorthand method of defining a primary key composed of a single column. Thus, if PRIMARY KEY is specified in the definition of column C, the effect is the same as if the PRIMARY KEY(C) clause is specified as a separate clause.

UNIQUE provides a shorthand method of defining a unique key composed of a single column. Thus, if UNIQUE is specified in the definition of column C, the effect is the same as if the UNIQUE(C) clause is specified as a separate clause.

The unique constraint clause defines a unique or primary key constraint.

CONSTRAINT constraint-name names the primary key or unique constraint.

UNIQUE(column-name, ...) defines a unique key composed of the identified columns. The identified columns must be defined as NOT NULL.

If a column or set of columns is defined as being unique, and an update statement is issued which updates a set of rows, DB2 will not check for uniqueness until all of the rows have been updated (within the single statement). This will allow a statement like

```
UPDATE EMPTAB SET EMPNO = EMPNO + 1
```

to execute successfully, even if there are consecutive values currently found in the EMPTAB table (which would cause duplicate values within the table for a short duration, during the processing of the UPDATE statement).

**Instructor notes:**

**Purpose** — Show another example of a table with both a primary key and a unique constraint.

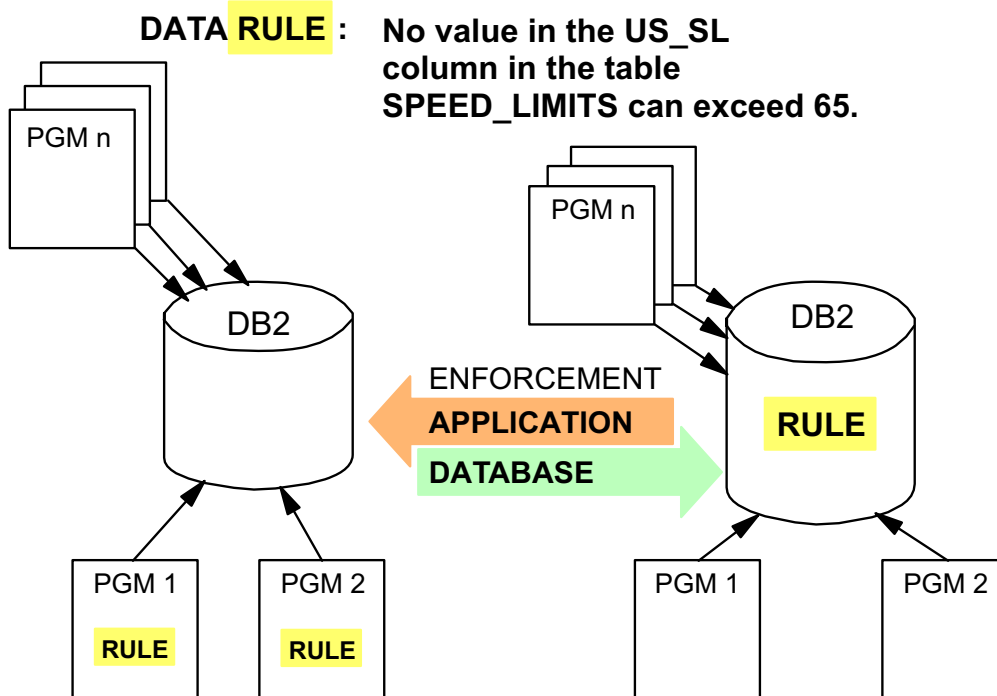
**Details** — This shows a common example of the need for a table to be able to have both a unique constraint and a primary key defined. In this case, for the employee data, you may have some tables that need to use EMPNO as the parent key, and other tables that need to use DRIVERLIC (or social security number, or social insurance number, or whatever your government-issued identification number is).

This graphic also indicates the behavior of DB2 checking for unique constraint violations, for example when an update is made to a set of rows, DB2 does not check for unique key violations until the update statement completes, not on intermediate results.

**Additional Information** — Certification objective: 3B Ability to create constraints on tables (for example, RI, Informational, Unique, 3F Knowledge of how to enforce data uniqueness.

**Transition Statement** — Let's look at another constraint that you can put onto a table.

## Overview of Check Constraints: The need



© Copyright IBM Corporation 2007

Figure 5-20. Overview of Check Constraints: The need

CF238.3

### Notes:

Referential Integrity is available to enforce relationships between one table and another through parent key and foreign key definitions. However, many data rules are specific to the column values in a single table. Check constraints can be used to define such rules to the DB2 database manager so that rules are enforced by the database manager.

Constraints will be checked during INSERT and UPDATE operations.

The source of the INSERT or UPDATE operation is not relevant for the defined constraint to be checked. Any application, whether installation-defined, vendor-provided, or ad hoc, will have INSERT and UPDATE statements validated such that the values in constrained columns meet the definition of the constraints.

Installations requiring such data checking should consider using the database manager support. The constraint is defined once, relieving the application programmer from coding such checks or invoking other code to do such checks.

***Instructor notes:***

**Purpose** — Introduce check constraints.

**Details** — Referential integrity is one way to ask DB2 to enforce business rules, where the allowed values can be found in another table. But, suppose that the allowed values can be defined by a rule, for example that the values in the US\_SL column of the table SPEED\_LIMITS cannot be over 65; in this case, you should define a check constraint.

The concept presented on the graphic is that we could have the check constraint enforced by all of the applications accessing the database, or we could code the constraint in the database, and then each of the applications would not be required to have the rule coded within the program. This is particularly helpful if ad hoc accesses (for example, through a command line processor) are allowed to update the table; then the rule would not be coded in the program.

**Additional Information** —

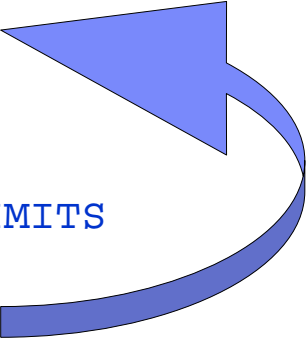
**Transition Statement** — Let's look at how a check constraint can be defined.

## Check Constraints — definition



```
CREATE TABLE SPEED_LIMITS
  (ROUTE_NUM  SMALLINT,
   CANADA_SL  INTEGER NOT NULL,
   US_SL      INTEGER NOT NULL
  CHECK (US_SL <=65) )
```

```
ALTER TABLE SPEED_LIMITS
ADD
CONSTRAINT SPEED65
CHECK (US_SL <=65)
```



© Copyright IBM Corporation 2007

Figure 5-21. Check Constraints — definition

CF238.3

### Notes:

Column constraints can be defined using the SQL statements CREATE TABLE or ALTER TABLE. The constraint name cannot be the same as any other constraint specified within that statement, and must be unique within the table.

If the ALTER TABLE statement is used, existing data is checked against the new constraint before the ALTER statement succeeds. If any rows exist that would violate the constraint, the ALTER TABLE statement fails.

To add constraints to a large table, it is more efficient to put the table into the check pending state, add the constraints, and then check the table for a consolidated list of violation rows. Use the SET INTEGRITY statement to explicitly set the check pending state. If the table is a parent table, check pending is implicitly set for all dependent and descendent tables.

When a table check constraint is added, packages that insert or update the table may be marked as invalid. More details on packages and implicit rebind will be presented in the Application Alternatives unit.

The definition of the constraint allows basic WHERE clause constructs to be used:

- Basic comparisons (>, <, =, >=, and so on)
- BETWEEN
- LIKE
- IN
- Deterministic UDFs

Values can only be inserted or updated in the column if the result of the constraint test resolves to True.

The definition of the constraint does not support:

- subqueries
- column functions
- functions that are not deterministic
- functions defined to have an external action
- user defined functions defined with either CONTAINS SQL or READS SQL DATA
- host variables or parameter markers
- special registers (such as CURRENT DATE)
- references to generated columns other than the identity column

The constraint can be explicitly named when it is defined. If it is not named, DB2 will create a name.

The ALTER TABLE statement can also be used to DROP constraints. For example:

```
ALTER TABLE SPEED_LIMITS DROP CONSTRAINT SPEED65
```

OR

```
ALTER TABLE SPEED_LIMITS DROP CHECK SPEED65
```

## ***Instructor notes:***

**Purpose** — Show examples of defining check constraints.

**Details** — The graphic shows two ways to put the same constraint onto a table. The first example shows adding the constraint as part of the CREATE TABLE statement. The second example shows adding the constraint as part of an alteration of the table.

The constraint indicates that only rows that have a value in the US\_SL column of less than or equal to 65 will be allowed to be inserted or updated in the SPEED\_LIMITS table.

The arrow on the left side is to indicate that you can name the constraint as part of the CREATE statement, as well as in the alter statement. It is a good idea to name the constraint, because if a name is not given, DB2 will give its own *very meaningful name* (SQL + the timestamp). If an user tries to insert a value that violates the constraint, DB2 will return an error that indicates that the insert failed due to constraint SQL20030405121567, supplying that *very meaningful name*. This will probably result in a call to the DBA to try to understand what the problem is; and the DBA will have to look it up in the catalog to understand what the problem is. If you name the constraint, that is, include the CONSTRAINT SPEED65 clause, then the error message will indicate that the insert failed due to constraint SPEED65; this may be a meaningful enough name for the user to figure out what their problem is.

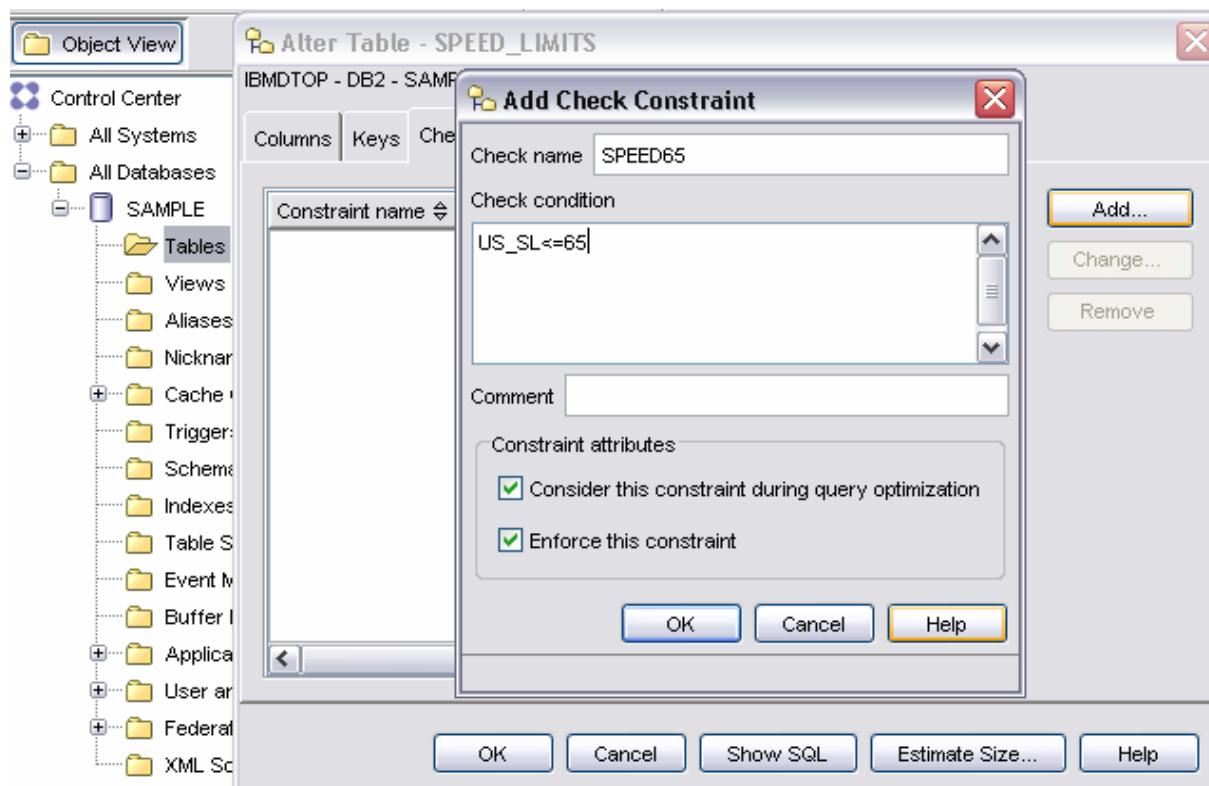
**Additional Information** — If data exists in the table that violates the constraint, the alter statement will not be successful. In the Moving Data lecture, we will teach the SET INTEGRITY statement: you can turn constraint checking off, alter the table, then turn constraint checking back on, indicating that the violating rows should be put in an exception table.

Certification objective: 2D Ability to create and manipulate the various DB2 objects.

**Transition Statement** — Let's look at adding check constraints through the GUI.



# Implementation of Check Constraints



© Copyright IBM Corporation 2007

Figure 5-22. Implementation of Check Constraints

CF238.3

## Notes:

To create a check constraint:

- Left-click and select **Tables**

Left-click and select the table you plan to work with from the *Contents* pane

- Select **Alter**

From the Alter Table GUI, you can add Check Constraints to the selected table.

Check constraints can also be specified when the table is initially being created.

***Instructor notes:***

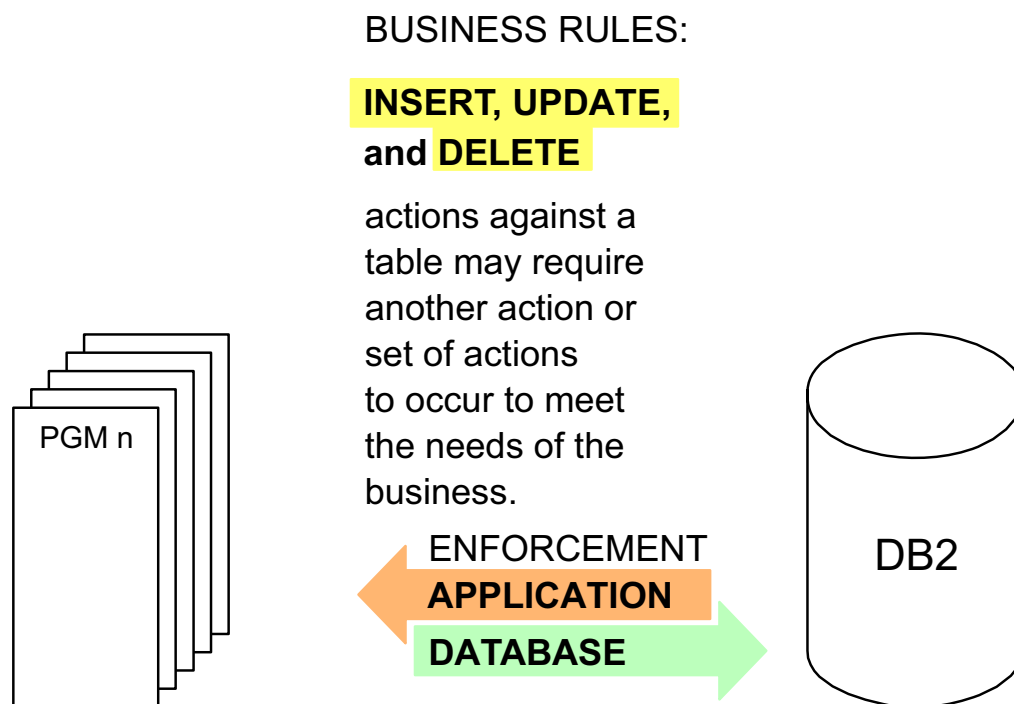
**Purpose** — Identify that check constraints can be added through the GUI as well.

**Details** — Check constraints can be added through the GUI, either by altering a table or when the table is created.

**Additional Information** —

**Transition Statement** — Let's look at triggers.

## Overview of triggers: The need



© Copyright IBM Corporation 2007

Figure 5-23. Overview of triggers: The need

CF238.3

### Notes:

Triggers can be defined to DB2 so that the database manager automatically enforces business rules that should follow an INSERT, UPDATE, or DELETE operation against the specified table. The benefits of DB2 enforcement of such rules include eliminating the need for redundant coding, global enforcement of the defined rules, and ease of maintenance if the business rules change.

Business rules that may be supported by triggers include:

- Validation of input
- Automatic generation of a value for an inserted row
- Reading from other tables for cross-referencing purposes
- Writing to other tables for audit-trail purposes
- Maintaining business rules
- Providing support for user alerts

## ***Instructor notes:***

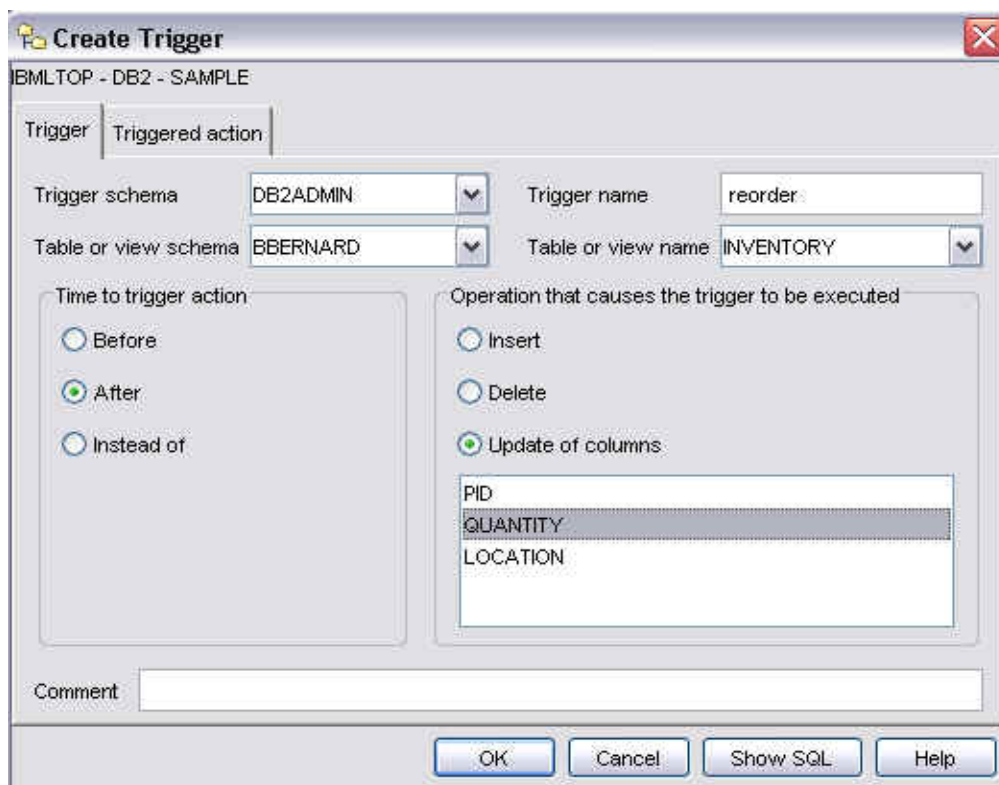
**Purpose** — Identify the need for triggers.

**Details** — Triggers again give you the capability of moving business rule enforcement from the application control to the control of DB2. This will ensure that the business rule is enforced regardless of which application makes the data change. Triggers allow you to define that insert, update, or delete actions against one table should cause another action to occur.

**Additional Information** —

**Transition Statement** — Let's look at the GUI screens for creating triggers.

## Implementation of triggers (1 of 2)



© Copyright IBM Corporation 2007

Figure 5-24. Implementation of triggers (1 of 2)

CF238.3

### Notes:

To access the Create Trigger GUI from the Control Center:

- Right-click and select **Triggers**
  - Select **Create**

The Create Triggers GUI has the following two tabs:

- Trigger
- Triggered action

The Create Trigger GUI allows you to create and define triggers on tables in the database.

## ***Instructor notes:***

**Purpose** — Identify the first GUI screen for defining a trigger.

**Details** — The first tab of the create trigger window has you specify the trigger schema and name, and the table it is to apply against. You also indicate if this is a before, after, or *instead of* trigger (*instead of* triggers are only valid against views). You also specify the action that causes this trigger to fire (insert, delete, or update; and with update, you can indicate which columns being updated cause the trigger to fire).

**Additional Information** —

**Transition Statement** — Let's look at the Triggered action tab.

## Implementation of triggers (2 of 2)

**Create Trigger**

IBMLTOP - DB2 - SAMPLE

Trigger Triggered action

Correlation name for the old rows

Correlation name for the new rows N

Temporary table for the old rows

Temporary table for the new rows

For each ☒ Row ☐ Statement

Triggered action

WHEN ( N.QUANTITY<=5 )  
INSERT INTO REORDER VALUES (N.PID, CURRENT TIMESTAMP)

Undo  
Clear  
SQL Assist

OK Cancel Show SQL Help

© Copyright IBM Corporation 2007

Figure 5-25. Implementation of triggers (2 of 2)

CF238.3

### Notes:

The Triggered action allows you to define what should occur when the trigger is invoked.

## ***Instructor notes:***

**Purpose** — Identify what can be specified on the Triggered action tab.

**Details** — On the triggered action tab, you can specify a correlation name for the old version of the data, or for the new version of the data (note: only an update has both old and new; inserts would only have new; deletes would only have old). With the correlation name, you can do comparisons and not have the trigger apply for all changes. You can also define a temporary table for the old rows, new rows, or both. The complete set of rows affected by the triggering SQL operation is available to the triggered action by using a temporary table name.

Next, you can indicate if this trigger is to be fired once per statement, or for each row.

Finally, you define the triggered action. In this example, we are checking to see if the new quantity is less than or equal to 5. If it is, we are indicating that an insert should be done to the REORDER table, including the item number and timestamp. Maybe on a nightly basis, we are planning to scan this table and place orders for items that have a stock quantity of less than 5.

**Additional Information** — Additional discussion that you may want to have (from Jiri):

Here I always ask the students if they think that this is a good trigger. The answer should be no: the triggered action is  $qty \leq 5$ . So when we have a quantity of 6 and reduce by 2, a trigger is fired. Now we have 4; we reduce by 2 again, a trigger is fired again, and so on.

I ask them how to do it better; the answer should be:

when  $n.qty \leq 5$  and  $o.qty > 5$ , “o” would have to be defined as the correlation name for the old row, so now we have a trigger that will be fired only once.

**Transition Statement** — [Let's look at the command line interface.](#)



## Create Trigger statement

```
create trigger reorder
  after update
  of qty on stock
  referencing new as n
  for each row mode db2sql
  when (n.qty <=5)
insert into reorder values (n.itemno, current
timestamp)
```

© Copyright IBM Corporation 2007

Figure 5-26. Create Trigger statement

CF238.3

### Notes:

A trigger defines a set of actions that are executed or *triggered* by an UPDATE, INSERT, or DELETE on a specified table. The triggered action is specified by an SQL procedure statement. The SQL procedure statement can contain a dynamic compound statement or any of the SQL control statements listed in “Compound SQL (Dynamic)” in the *SQL Reference* manual.

If the trigger is a BEFORE trigger, then an SQL procedure statement can also include a fullselect, or a SET variable statement. If the trigger is an AFTER trigger, then an SQL Procedure statement can also include one of the following:

- an INSERT statement (not using nicknames)
- a searched UPDATE statement (not using nicknames)
- a searched DELETE statement (not using nicknames)
- a SET variable statement
- a fullselect (a common table expression may precede a fullselect)

In the visual above, the CREATE TRIGGER statement has the following options:

**CREATE TRIGGER reorder** (trigger-name) specifies the name of the trigger. After creation you can also find this name in the system catalog tables, like SYSCAT.TRIGGERS and SYSCAT.TRIGDEP.

**AFTER** indicates that the triggered action happens after the changes are made to the table.

**NO CASCADE BEFORE** (not on visual) specifies that the triggered action is to be applied before any changes caused by the update are applied to the table. Also, this specifies that the action of the trigger should not cause other triggers to be activated.

**INSERT/UPDATE/DELETE** are the SQL operations which will cause the trigger to potentially fire.

**OF qty** (column-name) controls the granularity of the triggering, to specify a specific column rather than the whole table.

**ON stock** (table) specifies the base table to which the trigger applies.

**REFERENCING NEW AS n** (correlation name) is used to provide correlation names which are used as transaction variables. A specification of a correlation name for the variables prior to the triggering SQL operation can be given with **REFERENCING OLD AS o**. A temporary table name can be given for the set of affected rows prior to the triggering SQL operation by using **REFERENCING OLD\_TABLE AS identifier**; the set of affected rows, as modified by the triggering SQL operation, and by any SET statement in a BEFORE trigger that has already executed, can be identified by **REFERENCING NEW\_TABLE AS identifier**.

**FOR EACH ROW/STATEMENT** indicates that the trigger should be applied once for each row of the subject table that is affected by the triggering SQL operation (ROW), or that the triggered action is to be applied only once for the whole statement (statement).

**MODE DB2SQL** is a required parameter to specify the mode of the trigger. Only DB2SQL is currently supported.

**WHEN (n.qty <= 5)** (search-condition) specifies a condition that is true, false, or unknown. The search-condition provides the capability to determine whether or not a certain triggered action should be executed.

**INSERT INTO reorder VALUES (n.itemno, CURRENT TIMESTAMP)** (SQL-procedure-statement) is the SQL that should be executed if the search condition is true. The SQL procedure statement can include SQL control statements like FOR, IF, ITERATE, LEAVE, or WHILE to implement procedural logic, as well as statements such as GET DIAGNOSTICS, SIGNAL, and SET variable to interact with DB2. One of the column values here is the value of the current timestamp.

Triggers can also have multiple conditions which may be checked for via the CASE statement.

Adding a trigger to a table that already has rows in it will not cause any triggered actions to be activated. This may be of concern if the trigger is intended to enforce complex data integrity rules.

The creation of a trigger will cause packages to be invalidated if the package contains a statement that is now a potential triggering action.

Triggers can be removed with the DROP TRIGGER statement. Any packages that had a dependency on the trigger will be marked invalid.

Triggers are limited to a depth of 16 cascaded triggers. If a trigger at level 17 is activated, a negative SQLCODE (-724) will be returned and the triggering statement will be rolled back.

There is overhead for each additional trigger.

In order to create a trigger, you must have SYSADM or DBADM authority; or have ALTER privilege on the table; or have ALTERIN privilege on the schema and one of either IMPLICIT\_SCHEMA privilege on the database if the schema of the trigger does not exist, or CREATEIN privilege on the schema, if the schema name of the trigger refers to an existing schema.

### ***Instructor notes:***

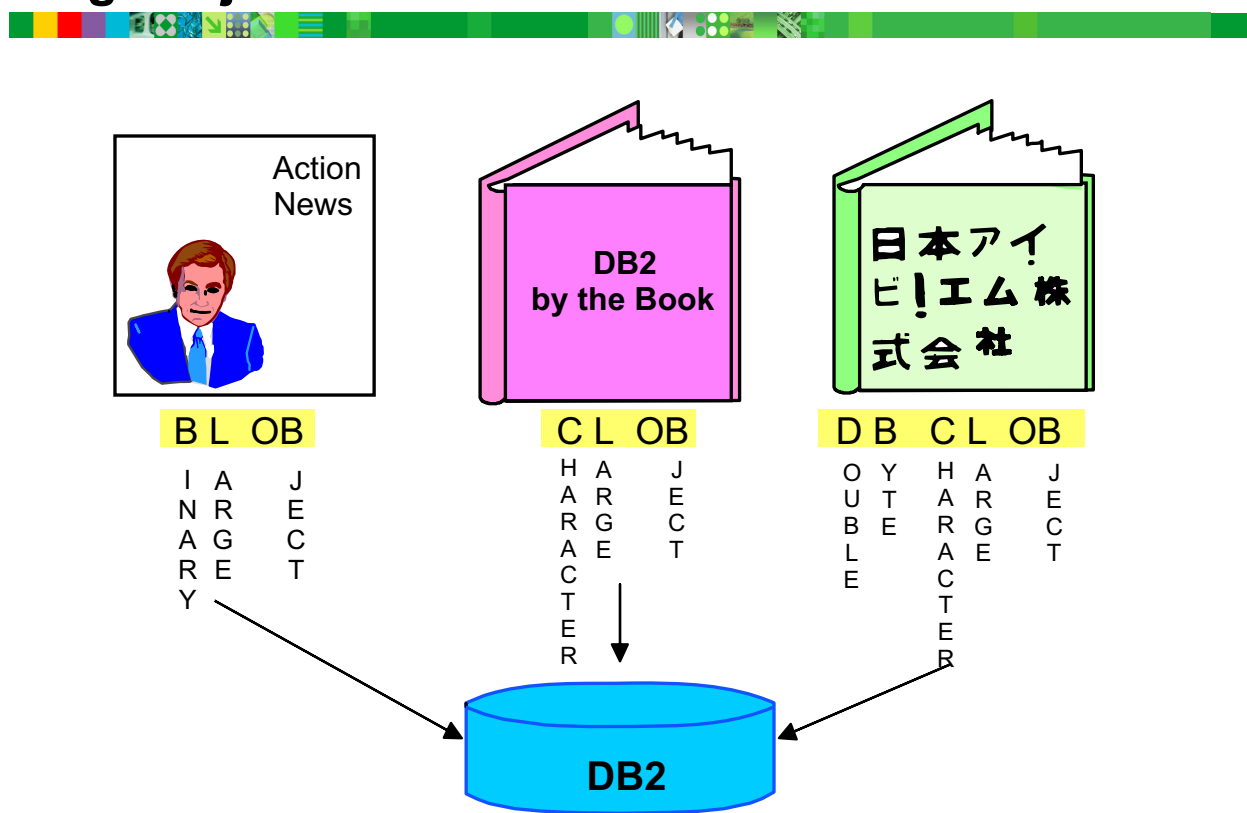
**Purpose** — Give example of creating a trigger through the command line interface.

**Details** — The example on the graphic is the same as that illustrated by the GUI. This trigger will be invoked after an update on the QTY column of the STOCK table. It will process each updated row, and check to see if the new quantity is less than or equal to 5. If it is, then the trigger will insert a row into the reorder table.

**Additional Information** — Certification objective: 2D Ability to create and manipulate the various DB2 objects.

**Transition Statement** — Let's look further at LOBs.

## Large objects: The need



© Copyright IBM Corporation 2007

Figure 5-27. Large objects: The need

CF238.3

### Notes:

As advancements in hardware allow greater volumes and types of data to be stored, installations require such data types to be handled as part of a relational system.

- LOB values can be very large, and the transfer of these values from database to client can be time consuming
- Because application programs typically process LOB values one piece at a time, rather than as a whole, applications can reference a LOB value by using a large object locator
- A large object locator, or LOB locator, is a host variable whose value represents a single LOB value on the database server
- An application program can select a LOB value into a LOB locator
- Then, using the LOB locator, the application program can request database operations on the LOB value by supplying the locator value as input
- The resulting output (data assigned to a client host variable) would typically be a small subset of the input LOB value

Large Objects, collectively known as **LOBs**, include binary large objects (**BLOBs**), character large objects (**CLOBs**), and double-byte character large objects (**DBCLOBs**).

- BLOBs can be used to store images and voice data. For example, an application could store the headline stories from a news broadcast that could then be selectively reviewed.
- CLOBs can be used any time large amounts of character (usually text) data are to be stored. For example, an application could store the text of popular books or online help text. (Can contain single- or double-byte characters).
- DBCLOBs are much like CLOBs, except that they store data in the double-byte character set. For example, the online help text mentioned for the CLOB data might need to be stored in Kanji. (Can contain double- or four-byte characters).

DB2 supports multiple LOB columns. The data for LOB columns is stored directly in the database.

Each LOB column can hold various sizes of data, with a maximum size of 2 GB.

On the CREATE TABLE, you can define whether or not changes to LOB data are to be LOGGED. Since LOB data by definition implies large data volumes, it may be advantageous not to log changes to LOB data. LOB data greater than 1 GB cannot be logged, and LOB data greater than 10 MB probably should not be logged.

Indicating COMPACT on the definition of a LOB specifies that the values in the LOB column should take up minimal disk space (any extra disk pages in the last group used by the LOB value should be freed), rather than leave any leftover space at the end of the LOB storage area that might facilitate subsequent append operations. Note that storing data in this way may cause a performance penalty in any append (length-increasing) operations on the column.

An example of DDL to define LOB columns is shown:

```
CREATE TABLE newsclip (  
    title VARCHAR(100) NOT NULL,  
    storydate DATE,  
    footage BLOB(4500000) NOT LOGGED,  
    audio BLOB(1000000) NOT LOGGED,  
    text CLOB(50000) COMPACT  
)
```

**BLOB** (*integer* [*K*|*M*|*G*]) specifies the maximum length in bytes for a binary large object. The length may be in the range of 1 byte to 2 GB.

If [*integer K*] is specified, the maximum length is 1024 (1 KB) times integer.

If [*integer M*] is specified, the maximum length is 1,048,576 (1 MB) times integer.

If [*integer G*] is specified, the maximum length is 1,073,741,824 bytes (1 GB) times integer.

**CLOB** (*integer* [*K*|*M*|*G*]) is for a character large object string. The meaning of *integer K*|*M*|*G* is the same as for BLOB.

**DBCLOB** (`integer[K|M|G]`) is for a double-byte-character large object string. The meaning of `integer K|M|G` is similar to that for a BLOB. The differences are that the value specifies the number of double-byte characters and that the maximum size is 1,073,741,823 (1 GB) double-byte characters.

## ***Instructor notes:***

**Purpose** — Identify why LOB data types may be needed.

**Details** — The graphic shows the three LOB types: BLOB, CLOB, and DBCLOB. BLOB columns could store, for instance, an audio or video clip. CLOB columns can store long strings of character data; CLOB columns should be used instead of LONG VARCHAR. DBCLOB columns store up to 1 GB of double-byte characters.

**Additional Information** — Additional pages about LOB data are in the appendix. If you have students with particular interests in this topic, you can use those pages.

**Transition Statement** — Let's look at a high level XML overview.



## 5.3 Creating tables with XML columns

### Instructor topic introduction

**What students will do** — Gain a high-level understanding of XML native stores in DB2.

**How students will do it** —

**What students will learn** —

**How this will help students on their job** —

## What is XML? (1 of 2)

- XML = eXtensible Markup Language
- XML is *self-describing data*
- Here is some data from an ordinary delimited flat file:

```
47; John Doe; 58; Peter Pan; Database systems; 29; SQL;
relational
```

- What does this data mean?

```
<book>
  <authors>
    <author id="47">John Doe</author>
    <author id="58">Peter Pan</author>
  </authors>
  <title>Database systems</title>
  <price>29</price>
  <keywords>
    <keyword>SQL</keyword>
    <keyword>relational</keyword>
  </keywords>
</book>
```

XML: Describes data  
HTML: Describes display

© Copyright IBM Corporation 2007

Figure 5-28. What is XML? (1 of 2)

CF238.3

### Notes:

The example of data above – taken from a flat file – shows a line of raw data. We see the numbers 47, 58, 29 and the words John Doe, Peter Pan – but we don't know what each of these pieces of data means; the context of the data is external to the data itself.

The XML markup shown above is in blue and the actual data is in red. By viewing both, we know what this data means.

XML is eXtensible Markup Language. The markup shows the tags used to describe the data. XML is a self-describing kind of data.

The key difference between HTML and XML markup languages:

- XML describes the relationship of the data.
- HTML describes how to display the data in a browser.
- XML is extensible – this means you can make up your own markup and tags to describe the data. You can create your own tags if they are not tied to a certain schema.
- HTML has a fixed vocabulary of tags and can only use those tags.

**Instructor notes:**

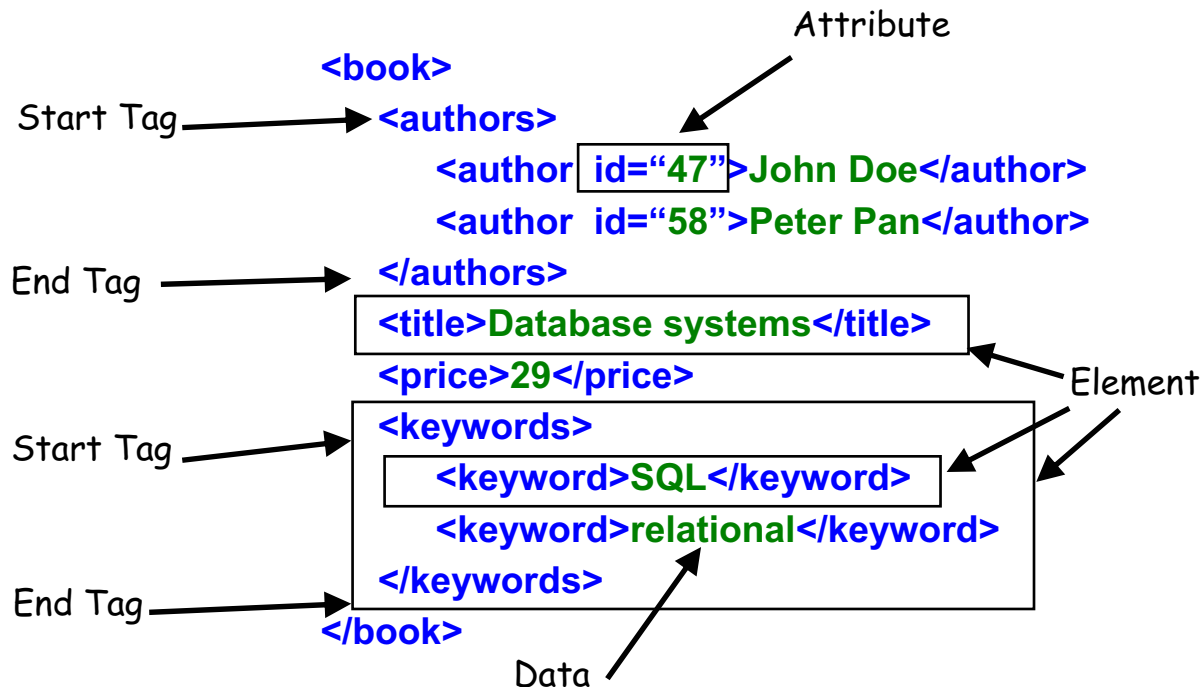
**Purpose** — Introduce some of the structure needed for XML, and clarify the XML/HTML difference.

**Details** — XML, or Extensible Markup Language, is a markup language that you can use to create your own tags. It was created by the World Wide Web Consortium (W3C) to overcome the limitations of HTML, the Hypertext Markup Language that is the basis for all Web pages. Like HTML, XML is based on SGML -- Standard Generalized Markup Language. Although SGML has been used in the publishing industry for decades, its perceived complexity intimidated many people that otherwise might have used it (SGML also stands for "Sounds great, maybe later"). XML was designed with the Web in mind.

**Additional information** —

**Transition statement** — More What is XML?

## What is XML? (2 of 2)



© Copyright IBM Corporation 2007

Figure 5-29. What is XML? (2 of 2)

CF238.3

### Notes:

Elements and attributes make up an XML document. These elements always have a start and stop tag. Tags are always in angle brackets `<>` and the end tag is denoted by the slash `/`. Elements can be nested.

The root element is the outer most element for the document – as shown above, the **book** element is the root element.

Attributes can be specified for an XML element. As shown above, the **author** element has an **id** attribute assigned to it. The attributes are always defined in the start tag. In the code shown above, the value of the **id** attribute for the first author is **47** and assigned to the **author** element in the first set of angle brackets.

Note that the **authors** element (denoted by the **authors** tag) contains two nested elements named **author**.

**Instructor notes:**

**Purpose** — Explain some of the keyword terminology, and how it is used in a simple XML document.

**Details** — Tag — used to define the start and end part of an element. Usually named to indicate the use of the element. Tags can be nested, but a start tag always must have an end tag to close the element. Tags cannot cross nested boundaries, that is,

<Tag1> xxx <Tag2> yyy </Tag2> </Tag1> is OK.

<Tag1> xxx <Tag2> yyy </Tag1> </Tag2> is wrong.

Content — the actual information value.

Element — definition of an information item... is made up of Tags and content.

**Additional information** —

**Transition statement** — Next, the XML document tree.

# The XML document tree

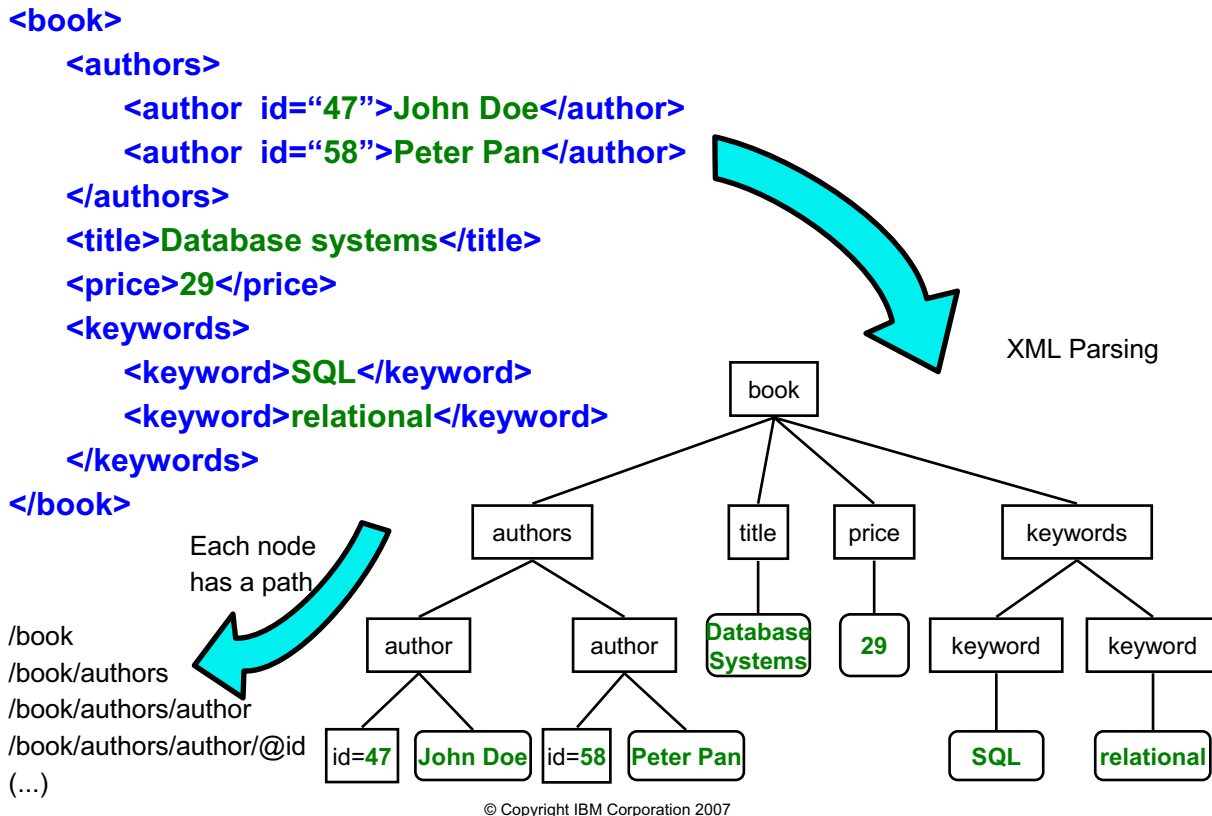


Figure 5-30. The XML document tree

CF238.3

## Notes:

Every XML document can be viewed as a tree – a hierarchy of nodes. Every XML document can be seen as a higher key of nodes. It may be intuitive that this represents a hierarchy of nodes.

In the example above, **book** is the root node of the tree. The data elements are always the leaf nodes in the tree (such as the **title** element in the example above).

XML parsing typically produces this tree form. This is how XML can be manipulated by software — **storing XML in trees in DB2 and not as text**, which is more efficient and faster to manipulate. This is done by parsing the tree.

Serialization is reversing the parsing operation, which takes a tree and returns it to XML document format.

**In the graphic shown above, each node would have the following paths:**

- root node (book element)

/book

- authors node (authors element)  
/book/authors
  - author node (author element)  
/book/authors/author (two members)  
/book/authors/author/@id (author attribute = id)
    - author text node (text element)  
/book/authors/author/text (values are *John Doe*, *Peter Pan*)
- title node (title element)  
/book/title
  - title text node (text element)  
/book/title/text (value is *Database System*)
- price node (price element)  
/book/price
  - price text node (text element)  
/book/price/text (value is *29*)
- keywords node (keywords element)  
/book/keywords
  - keyword node (keyword element)  
/book/keywords/keyword (two members)
    - keyword text node (text element)  
/book/keywords/keyword/text (values are *SQL*, *relational*)

## ***Instructor notes:***

**Purpose** — Describe that the XML code gets parsed into a tree structure.

**Details** — This tree structure is what is stored natively in DB2. When a change is needed to a document, that document is retrieved and serialized into the XML code form. The document change is made in the XML code form, then re-parsed back into the tree form and stored again.

**Additional information** —

**Transition statement** — Let's look at storing XML data as a PureXML datastore.



## PureXML datastore

- PureXML's seamless integration of XML with relational data:
  - Speeds application development
  - Improves search performance with highly optimized XML indexes
  - Is flexible because both SQL and XQuery can be used to query XML data
  - Enables well-formed XML documents to be stored in their hierarchical form
- Stored XML data can be accessed and managed by leveraging DB2 functionality
  - Enables efficient search and retrieval of XML
- Note current limitation:
  - XML data can only be stored in single-partition databases defined with the UTF-8 code set

© Copyright IBM Corporation 2007

Figure 5-31. PureXML datastore

CF238.3

### Notes:

The pure XML support in DB2 offers efficient and versatile capabilities for managing your XML data. DB2 stores and processes XML in its inherent hierarchical format, avoiding the performance and flexibility limitations that occur when XML is stored as text in CLOBs or mapped to relational tables. Unlike XML-only databases, DB2 9 also provides seamless integration of relational data and XML data within a single database, even within a single row of a table. This flexibility is reflected in the language support, which allows you to access relational data, XML data, or both at the same time. You can query your XML in any of the following four ways:

- Plain SQL (no XQuery involved)
- SQL/XML, that is, XQuery embedded in SQL
- XQuery as a stand-alone language (no SQL involved)
- XQuery with embedded SQL

The native XML data store enables well-formed XML documents to be stored in their hierarchical form within columns of a table. XML columns are defined with the XML data

type. By storing XML data in XML columns, the data is kept in its native hierarchical form, rather than stored as text or mapped to a different data model.

Because the native XML data store is fully integrated into the DB2 database system, the stored XML data can be accessed and managed by leveraging DB2 functionality.

The storage of XML data in its native hierarchical form enables efficient search and retrieval of XML. XQuery, SQL, or a combination of both can be used to query XML data. SQL functions that return XML data or take XML arguments (referred to as SQL/XML functions) also enable XML data to be constructed or published from values retrieved from the database.

XML data can only be stored in single-partition databases defined with the UTF-8 code set.

Note that using XML features prevents future use of the Database Partitioning Feature available with DB2 Enterprise Server Edition for Linux, UNIX, and Windows.

Use of the new XML data type and related native XML data store support is available as a separate feature of DB2 9. You must acquire the same licensing terms and conditions as the underlying DB2 data server.

**Instructor notes:****Purpose —**

**Details —** Note the UTF-8 code set limitation.

**Additional information —** XML documents complete with digital signatures can be stored in XML columns.

XPath can be used to navigate and query the documents.

XQuery can be used to query and transform the documents.

SQL/XML can be used to join the documents to relational data and create new XML documents.

The whole solution becomes much simpler, more robust and performs better.

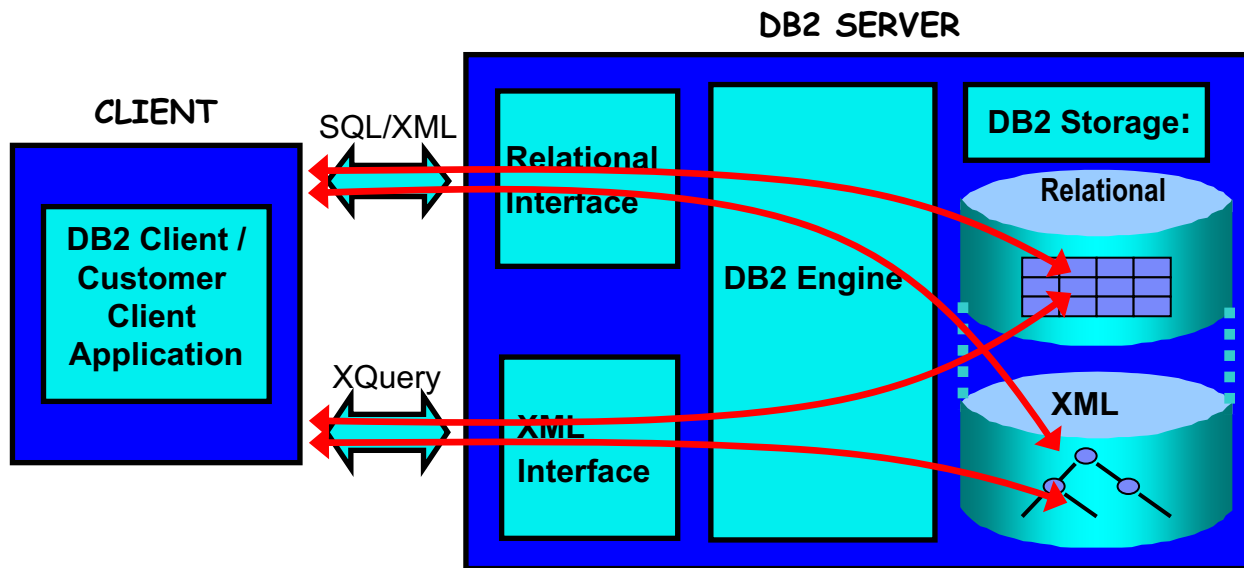
This means faster development, lower development and maintenance costs, improved flexibility and better integration!

As a result, you can deploy more XML applications at lower cost in order to drive competitive advantage.

**Transition statement —** Let's see how to query XML data.

## Integration of XML and relational capabilities

- Applications combine XML and relational data
- Native XML data type (server and client side)
- XML capabilities in all DB2 components



© Copyright IBM Corporation 2007

Figure 5-32. Integration of XML and relational capabilities

CF238.3

### Notes:

*The goal:*— integration of XML and relational capabilities so that applications can combine XML and relational data.

#### A new DB2 datatype: XML type

- The XML type is visible on the server and the client.
- There is deep integration as the figure above shows. This is not an add-on, extender, or wrapper.
- The DB2 infrastructure is reused wherever it made sense, with new system code where needed.
- DB2 supports a new query language: XQuery, with a new parser (side by side of SQL parser).
- XML data is natively storage: hierarchical, optimized for parent/child traversal.

XML permeates the engine. You can use SQL or XQuery to query a mix of relational and XML data stored in DB2.

The same compiler handles both XML and relational data, so SQL and XQuery queries can be mixed.

Because XML is not just another numeric type, but a significant change, special efforts are necessary in almost all system components, tools and utilities to handle XML.

**Xquery:**

Xquery is an emerging standard for access to XML data. It is designed, from the ground up, to handle semi-structured data.

## ***Instructor notes:***

**Purpose** — Goal = integration of XML and rel capabilities so that appls can combine XML and rel data.

**Details** — We did this by creating a new datatype in DB2: XML type. The XML type is visible on the server and the client.

Deep integration as the picture shows. This is not an add-on, extender, or wrapper. Reused the DB2 infrastructure wherever it made sense. We wrote new system code otherwise. New language: XQuery, new parser (side by side of SQL parser). Native storage: hierarchical, optimized for parent/child traversal.

XML permeates the engine. Can use SQL or XQuery to query a mix of relational and XML data in DB2. We leverage the DB2 engine capabilities. We have added a new storage for XML. Different parsers for SQL and XQuery but then same compiler handles the queries hence we can mix SQL and XQuery.

Because a new data type has been added, it requires support throughout the system. Moreover, XML is not just another numeric type, but a significant change, so special efforts are necessary in almost all system components, tools and utilities.

Different ways to manage XML documents exist. These include shredding into relational tables or just storing them as CLOBs. In DB2 Native XML, XML data is stored in a parsed, annotated tree form, similar to the DOM. The XML data is formatted to data pages which are buffered. The benefits of this format include faster navigation which results in faster query execution as well as simpler indexing of data.

The DB2 compiler has been extended to understand both SQL and XQuery, even combined in a single statement. Thus, both relational and XML data can be managed with the same system.

The red arrows in the visual indicate data flow. Data flow can be accomplished in several ways; the SQL/XML method uses the Relational Interface, while the XQuery method uses the XML Interface.

**Additional information** — XML and SQL will be handled on an equal footing. The XML extender will cease to exist. We don't have a *SQL extender* in DB2 today. DB2 *does* SQL natively. Similarly, we won't need an XML extender either. DB2 "will do" XML natively. Storage — We're extending our storage management expertise to support XML. XML requires new storage mechanisms.

Utilities — backups, dump and load, and so forth.

Certification objective: 2H Knowledge of XML structure (indexing for performance).

**Transition statement** — Next, What is XML?

## Two ways to query XML data

- Using XQuery as the primary language

```
XQUERY db2-fn:xmlcolumn ("CUSTOMER.INFO")
      Returns all Documents in column INFO
```

- Using SQL as the primary language

```
Select Pid from PRODUCT xmlp
WHERE XMLEExists('declare default element namespace
    "http://posample.org";$p/product/description[price
    < 100] '
    passing xmlp.description as "p");
```

© Copyright IBM Corporation 2007

Figure 5-33. Two ways to query XML data

CF238.3

### Notes:

You may use a new query language – XQuery – to retrieve XML documents (and parts of documents) from the database.

Example1– Xquery:

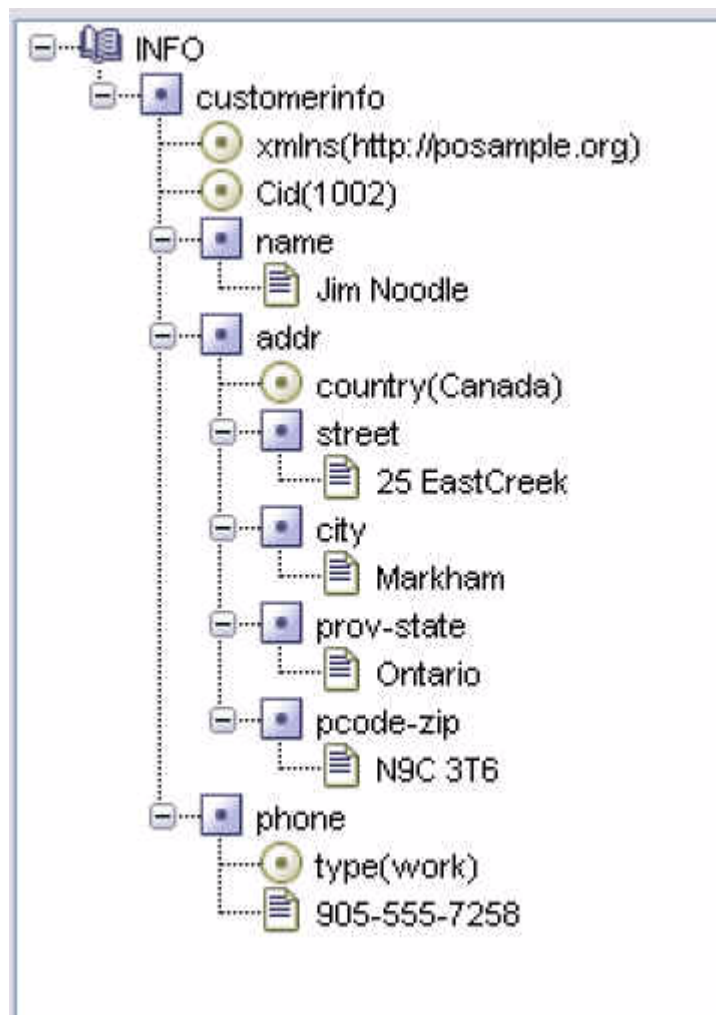
```
XQUERY db2-fn:xmlcolumn ("CUSTOMER.INFO")
```

Results - serial view:

```
...
<customerinfo xmlns="http://posample.org" Cid="1002">
  <name>
    Jim Noodle
  </name>
  <addr country="Canada">
    <street>
      25 EastCreek
    </street>
```

```
<city>
  Markham
</city>
<prov-state>
  Ontario
</prov-state>
<pcode-zip>
  N9C 3T6
</pcode-zip>
</addr>
<phone type="work">
  905-555-7258
</phone>
</customerinfo>
...
```

Tree view:



You may also use SQL to retrieve XML documents and relational database data, and combine those type of information in one query (SQL/XML).



## Example2 – SQL/XML:

```
Select Pid from PRODUCT xmlp
  WHERE XMLEExists('declare default element namespace
    "http://posample.org";$p/product/description[price < 100]' passing
    xmlp.description as "p");
```

## Results - serial view:

```
PID
-----
100-100-01
100-101-01
100-103-01
100-201-01
```

4 record(s) selected.

## ***Instructor notes:***

### **Purpose —**

**Details —** Querying and Transformation — Xquery.

Xquery is an emerging standard for access to XML data. It is being designed, from the ground up, to handle semi-structured data.

An important characteristic of our solution would be that irrespective of how the data is stored, in tables or in the semi-structured XML format, the user will be able to pick either language to query it. ... “No more impedance mismatch!” ... “No more mappings”.

### **Additional information —**

**Transition statement —** Where can I learn more?

## For more information



- To learn more about DB2 and XML, including the use of xQuery and SQL/XML:



- Course CG13 — 4 days  
*Query and Manage XML Data with DB2 9*
- Course CG10 — 2 days  
*Query XML data with DB2 9*
- Documentation:  
*DB2 XML Guide (SC10-4254-00)*  
*DB2 XQuery Reference (SC18-9796-00)*



© Copyright IBM Corporation 2007

Figure 5-34. For more information

CF238.3

### Notes:

#### *Query XML data with DB2 9 — CG10*

Duration: 2 days

#### **Course description:**

This course is designed to introduce the students to non-traditional ways of using DB2 to work with native XML data.

The students query XML data stored in DB2 tables and create XML documents from traditional DB2 data. They search XML documents stored in DB2 tables and access XML documents as though the documents were relational tables. This course can be taken by end users or other personnel who only need to access XML data stored in DB2 tables.

#### *Query and Manage XML Data with DB2 9 — CG13*

Duration: 4 days

#### **Course description:**

This course is designed to introduce the students to non-traditional ways of using DB2 to work with native XML data. It is also designed to enable database administrators to manage the DB2 — native XML environment.

The students query XML data stored in DB2 tables and create XML documents from traditional DB2 data. They search XML documents stored in DB2 tables and access XML documents as though the documents were relational tables. This content is addressed in the first two days of class and can be taken by users or other personnel who only need to access XML data stored in DB2 tables.

In the last two days of class, the students store native XML data in DB2 and create XML indexes. They analyze plan information for queries and take steps for improving query execution. The course also teaches how to manage the XML schemas and the techniques available for decomposing the XML documents.

### ***DB2 SQL Workshop — CF12***

Duration: 2 days

#### **Course description:**

This course provides an introduction to the SQL language and applies to the entire DB2 Family. This course is appropriate for customers working in all DB2 environments, that is, z/OS, VM/VSE, iSeries, Linux, UNIX, and Windows.

### ***RELATIONAL DATABASE DESIGN — CF18***

Duration: 4 days

#### **Course description:**

This course presents a methodology for modeling and designing relational databases.

### ***DB2 SQL Workshop for Experienced Users — CF13***

Duration: 2.5 days

#### **Course description:**

This course teaches you how to make use of advanced SQL techniques to access DB2 databases in different environments. This course is appropriate for customers working in all DB2 environments, that is z/OS, OS/390, VM/VSE, iSeries, Linux, UNIX, and Windows.

### ***Building the Data Warehouse — DW11***

Duration: 4.5 days

#### **Course description:**

This course teaches students practical methods and techniques for designing and constructing a data warehouse. Students will learn how to apply the techniques in the context of an incremental data warehouse development process which is suitable for constructing a data warehouse from a departmental perspective. In addition, students will also learn how this process can further be extended for the construction of consistent,

corporate-wide data warehouses. As such, the course is suitable for students who wish to learn how to construct a data mart as well as a corporate-wide, integrated data warehouse.

During the course, students will apply the techniques they learn doing several exercises, which are part of a case study which is consistently developed throughout the course. This approach greatly enhances the practical value of the course, by allowing students to apply the acquired knowledge in practice, under the guidance of the instructor.

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Let's look at the summary for this unit.

## Unit summary

Having completed this unit, you should be able to:

- List DB2 object hierarchy and physical directories and files
- Learn to create the following objects:
  - [Schema](#), [Table](#), [View](#), [Alias](#), [Index](#)
- Explore the use of table partitioning
- Review the use of Temporary Tables
- Explore the use and implementation of Check Constraints, Referential Integrity and Triggers
- Exploring Large Objects: the need and usage
- Study an overview of XML as a native store

© Copyright IBM Corporation 2007

Figure 5-35. Unit summary

CF238.3

### Notes:

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**



# Unit 6. Moving data

## Estimated time

01:00

## What this unit is about

This unit provides information on tools that can be used to manage table data, including the Export Utility, Import Utility, Load Utility, and the DB2MOVE utility.

The handling of set integrity pending conditions will also be addressed.

## What you should be able to do

After completing this unit, you should be able to:

- Discuss the INSERT statement and recognize its limitations
- Explain the differences between IMPORT and LOAD
- Explain the EXPORT, IMPORT, and LOAD syntax
- Create and use Exception Tables and Dump-Files
- Distinguish and resolve Table States:
  - Load Pending and Set Integrity Pending
- Use the SET INTEGRITY command
- Discuss the db2move and db2look commands

## How you will check your progress

Accountability:

- Machine exercises

## References

*Command Reference*

*Data Movement Utilities Guide and Reference*

*Replication Guide and Reference*

## Unit objectives

After completing this unit, you should be able to:

- Review the INSERT statement and recognize its limitations
- Review differences between IMPORT and LOAD
- Learn the EXPORT, IMPORT, and LOAD syntax
- Explore the use of Exception Tables and Dump-Files
- Distinguish the Table States:
  - Load pending, Set Integrity Pending
- Learn why and how to use the SET INTEGRITY command
- Explore use of the **db2move** and **db2look** commands

© Copyright IBM Corporation 2007

Figure 6-1. Unit objectives

CF238.3

### Notes:

***Instructor notes:***

**Purpose** — State the objectives of this unit.

**Details** —

**Additional information** — The **Set Integrity Pending** state was previously known as Check Pending.

**Transition statement** — First, let's review the INSERT statement.

## Review INSERT statement

- 
- The SQL INSERT statement can insert one or more rows of data into tables, nicknames, or views
    - SQL overhead is imposed, such as obeying RI, or Check Constraints, or Uniqueness, or executing triggers.
    - As INSERTs occur, the activity is also stored in logs
  - The SQL INSERT may not be the best or fastest method to load massive amounts of data into a database

© Copyright IBM Corporation 2007

Figure 6-2. Review INSERT statement

CF238.3

### Notes:

The INSERT statement inserts rows into a table, nickname, or view, or the underlying tables, nicknames, or views of the specified fullselect. Inserting a row into a nickname inserts the row into the data source object to which the nickname refers.

Inserting a row into a view also inserts the row into the table on which the view is based, if no INSTEAD OF trigger is defined for the insert operation on this view. If such a trigger is defined, the trigger will be executed instead.

***Instructor notes:***

**Purpose** — A quick review of the use of the INSERT statement to load data into the database. The student should be very familiar with this statement as it is one of the fundamental DML statements in SQL.

**Details** — The student may be less familiar with INSERT into a view. Thus you may wish to look at this in a little more detail.

In any case, with an INSERT, triggers can be fired. Again, this is a topic that the student may be less familiar with.

**Additional information —**

**Transition statement** — Let's take a look at another way to move data.



## 6.1 Export/Import utilities

### Instructor topic introduction

**What students will do** — Learn how to do simple imports and exports.

**How students will do it** — Listen to the lecture and practice in the lab.

**What students will learn** — How to use export and import utilities to do simple imports and exports.

**How this will help students on their job** — They will be able to export data from their tables and import data into their tables.

## Differences between IMPORT and LOAD



| IMPORT                                                                                                                                                                                                                                                                                                                   | LOAD                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>▪ Slow when moving large amounts of data</li> <li>▪ Creation of table/index supported with IXF format</li> <li>▪ Import into tables, views, aliases</li> <li>▪ Option to ALLOW WRITE ACCESS</li> <li>▪ All rows logged</li> <li>▪ Triggers fired, constraints enforced</li> </ul> | <ul style="list-style-type: none"> <li>▪ Faster for large amounts of data</li> <li>▪ Tables and indexes must exist</li> <li>▪ Load into tables only</li> <li>▪ Existing data can still be seen by read applications</li> <li>▪ Minimal logging; can make copy</li> <li>▪ Triggers not fired; unique constraints enforced, RI and check constraints via SET INTEGRITY</li> </ul> |

© Copyright IBM Corporation 2007

Figure 6-3. Differences between IMPORT and LOAD

CF238.3

### Notes:

The import utility performs SQL INSERTs, so its capabilities are similar to an application program performing inserts. The load utility formats the pages and writes them directly into the database.

A more detailed version of this chart can be found in the *Data Movement Guide and Reference*.



***Instructor notes:***

**Purpose** — Identify differences between the import and load utilities.

**Details** — Import and load can both be used to bring data into a table. This chart highlights some of the major differences between the two utilities.

**Additional Information** — A more detailed chart can be found in the appendix of the *Data Movement Utilities Guide and Reference*.

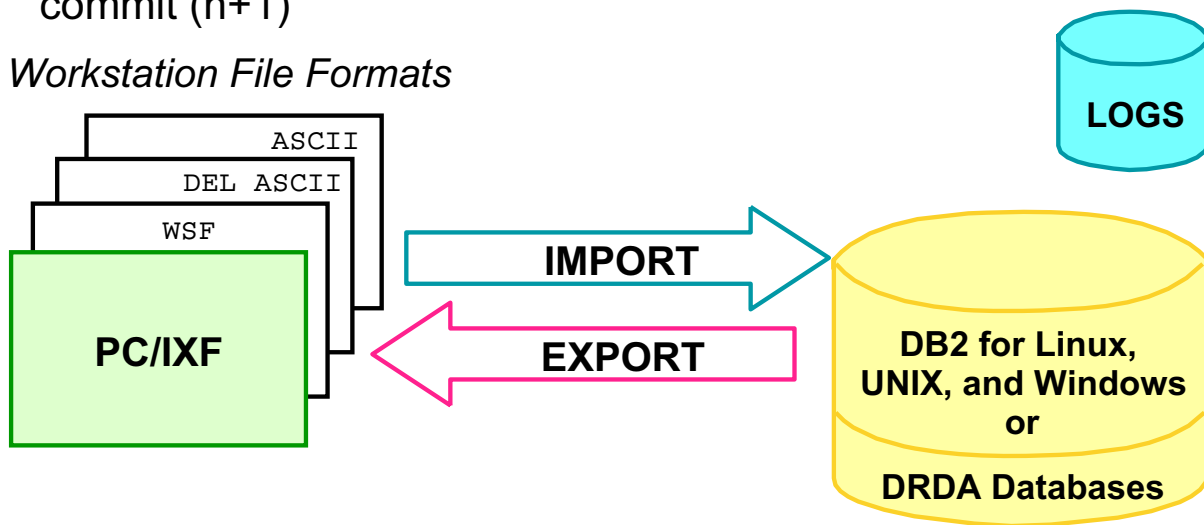
Certification objective: 5D Knowledge to identify when to use IMPORT versus LOAD.

**Transition statement** — Now let's look at an overview of the IMPORT/EXPORT methods of moving data.

## IMPORT/EXPORT overview

- Must be connected prior to call
- IMPORT COMMITCOUNT n (AUTOMATIC) keeps log sizes manageable
- IMPORT RESTARTCOUNT n allows to restart from last commit (n+1)

### Workstation File Formats



© Copyright IBM Corporation 2007

Figure 6-4. IMPORT/EXPORT overview

CF238.3

### Notes:

The IMPORT utility may be used to insert data from an input file into a table, with the input file containing data from another database or spreadsheet program.

The EXPORT utility may be used to copy data from a table to an output file for use by another database or spreadsheet program. This file can be used to load tables, providing a convenient method of migrating data from one database to another.

The IMPORT and EXPORT utilities may be used to move data between databases which exist on DB2 Database platforms. These utilities use the database engine, so, for example, you could create a table during the execution of the IMPORT utility.

The IMPORT and EXPORT utilities may be used to move data between DB2 and DRDA host databases if DB2 Connect is installed.

### Import and Export File Formats

- **DEL** — Delimited ASCII is commonly used for storing data that separates column values with a special delimiting character.

- **ASC** — Non-delimited ASCII may be used for importing data from other applications which create flat text files with aligned column data. **ASC may not be used with EXPORT.**
- **WSF** — Work sheet format is used for exchange with products such as Lotus 1-2-3 and Lotus Symphony.
- **IXF** — PC version of the Integrated Exchange Format. This is the preferred method for exchange within the database manager. Use PC/IXF to export data from a table so that it can be imported later into the same table or another table.

**For IXF data file formats, the table does not need to exist before beginning the import. The table can automatically be created when the data is imported.** For DEL, WSF, and ASC data file formats, the table including its column names and data types must be defined before the file can be imported.

For restrictions see: Using IMPORT to create an exported table.

During export, a UDT's base type will be stored in IXF files. If using the IXF file to create a new table during an import, the new table will have the base type for the column definitions instead of the UDT. The import utility supports importing values into a UDT column when the base data type supplied in the input file is castable to the type of the UDT.

For EXPORT and IMPORT, you can select LOB column types and have the data stored in separate files if the MODIFIED BY option LOBSINFILE is specified. The same path as the data file will be used.

**XML FROM xml-path** - Specifies one or more paths that contain the XML files.

**XMLPARSE** - Specifies how XML documents are parsed. If this option is not specified, the parsing behavior for XML documents will be determined by the value of the CURRENT XMLPARSE OPTION special register.

***Instructor notes:***

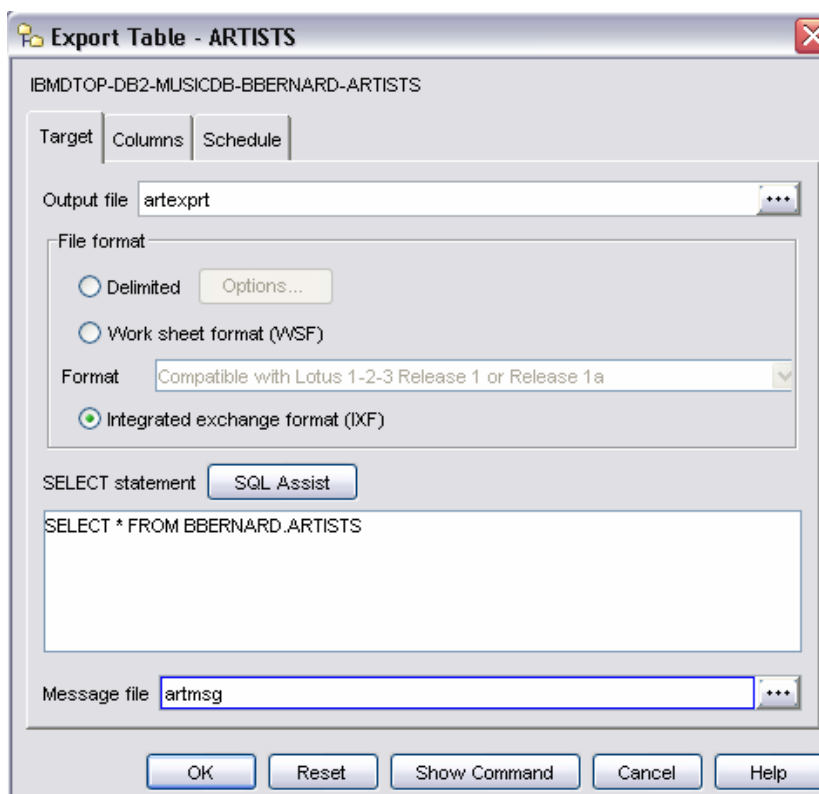
**Purpose** — Identify the commands for moving data to and from files.

**Details** — The graphic introduces the IMPORT and EXPORT commands. The EXPORT command can be used to copy data from the database into a file. The IMPORT command can take data from a file and put it into a table in the database.

**Additional Information** —

**Transition Statement** — Let's look more closely at EXPORT.

# EXPORT command



© Copyright IBM Corporation 2007

Figure 6-5. EXPORT command

CF238.3

## Notes:

To export data through GUI:

- Left-click and select **Tables**  
Right-click and select the table with which you plan to work from the *Contents* pane
  - Select **Export**  
From the Export GUI, you can export data from a table to a file.  
You will find the following tabs in the Export GUI.
    - Target
    - Columns
    - Schedule

The Export GUI allows you to export data from tables to files, place large objects in separate files, define which columns should be exported, define the type of export, and name the message file. You can run the export now, or schedule it to run once or on a repeating schedule.

## ***Instructor notes:***

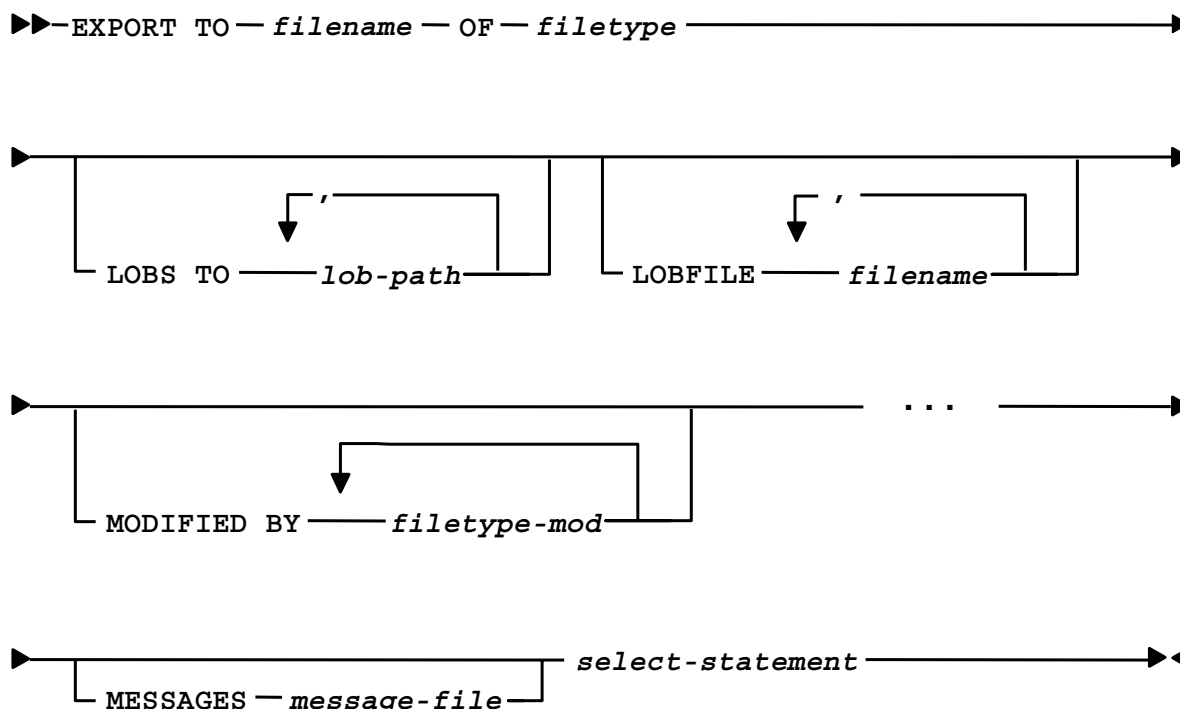
**Purpose** — Show the GUI for performing an export.

**Details** — The Export utility just needs to know the output location for the data, the file format, and where the messages should be written. Additional information, regarding the column names to be used in the output file, and where large objects should be written, can be specified on the *Columns* tab. On the *Schedule* tab, you can indicate that the job should run now, or you can schedule a task execution for a one-time run or a repeating run.

**Additional Information** —

**Transition Statement** — [Let's look at the command syntax.](#)

## EXPORT command syntax (basic)



© Copyright IBM Corporation 2007

Figure 6-6. EXPORT command syntax (basic)

CF238.3

### Notes:

The EXPORT command can be used to export data from a database to one of several external file formats. The user specifies the data to be exported by supplying an SQL SELECT statement.

File types that are supported include:

- **DEL (delimited ASCII format)** which is used by a variety of database manager and file manager programs.
- **WSF (work sheet format)** which is used by programs such as Lotus 1-2-3 and Lotus Symphony.
- **IXF (integrated exchange format, PC version)** in which most of the table attributes, as well as any existing indexes, are saved in the IXF file, except when columns are specified in the SELECT statement. With this format, the table can be recreated, while with the other file formats, the table must already exist before data can be imported into it.

There are many restrictions to the EXPORT command for table re-creation, such as you cannot export XML documents, and using SELECT \* FROM table only.

The MODIFIED BY options allow you to specify different items depending on the file type being created. For example, for delimited output data, you can specify the character string delimiter and the column delimiter. See the *Data Movement Utilities Guide and Reference* or *Command Reference* for details.

Some EXPORT command parameters:

### **LOBS TO lob-path**

Specifies one or more paths to directories in which the LOB files are to be stored. There will be at least one file per LOB path, and each file will contain at least one LOB. The maximum number of paths that can be specified is 999.

### **LOBFILE filename**

Specifies one or more base file names for the LOB files. When name space is exhausted for the first name, the second name is used, and so on. The maximum number of file names that can be specified is 999.

### **MODIFIED BY filetype-mod**

Specifies file type modifier options.

|                         |                                                                                                                                                                                                                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>lobsinfile</b>       | lob-path specifies the path to the files containing LOB data.                                                                                                                                                                                                                                        |
| <b>xmlinsefiles</b>     | Each XQuery Data Model (QDM) instance is written to a separate file. By default, multiple values are concatenated together in the same file.                                                                                                                                                         |
| <b>lobsinsefiles</b>    | Each LOB value is written to a separate file. By default, multiple values are concatenated together in the same file.                                                                                                                                                                                |
| <b>xmlnodeclaration</b> | QDM instances are written without an XML declaration tag. By default, QDM instances are exported with an XML declaration tag at the beginning that includes an encoding attribute.                                                                                                                   |
| <b>xmlchar</b>          | QDM instances are written in the character codepage. Note that the character codepage is the value specified by the CODEPAGE file type modifier, or the application codepage if it is not specified. By default, QDM instances are written out in Unicode.                                           |
| <b>xmlgraphic</b>       | QDM instances are written in the graphic codepage. Note that the graphic codepage is the graphic component of the value specified by the CODEPAGE file type modifier, or the graphic component of the application codepage if it is not specified. By default, QDM instances are written in Unicode. |



**MESSAGES message-file**

Specifies the destination for warning and error messages that occur during an export operation (the path must exist).

**Instructor notes:**

**Purpose** — Identify the syntax for the EXPORT command.

**Details** — The EXPORT command is not very complex. You indicate that the data should be exported to a file, you indicate the file type, and the SELECT statement to generate the data. Options on the command include:

- Indicating where messages should be written
- Different file type modifiers, for example, for delimited files, you can indicate what the column delimiter should be
- Where LOB/XML data should be written

**Additional Information** — Two additional things that you might want to do here to better illustrate these commands:

1. Start a list on the board: Write “Export” and in a column next to it, write DEL, WSF, and IXF. Explain that these are the file types that are supported by the EXPORT command (give a short description of each one).

You then can add to this list when you start talking about the IMPORT command: add Import using a different color marker, and put a check mark next to DEL, WSF, and IXF, indicating that those are valid for import as well. Then write ASC in at the bottom (write it more under IMPORT) as a new file type, valid for import, but not for export.

2. To explain about the LOB path and LOB filename, draw a “file” on the board (the top of a rectangle works for me), and put a “row” in it of delimited data:

42, “Beatles”, “P”, “very very long lob data column 1”, “lob data column 2”

You can indicate that it probably doesn’t make sense, if the LOBs can be up to 2 GB in length, to write them into the output file with the other data. Instead, the export command allows you to indicate that LOBs get written to files. Let’s say that LOBFILE was “lob1, lob2” and LOBS TO was “/dir1/, /dir2/”, and you specify MODIFIED BY LOBSINFILE. For example:

```
EXPORT TO checkit.txt OF DEL LOBS TO c:\dir1\, c:\dir2\ LOBFILE lob1, lob2  
MODIFIED BY LOBSINFILE SELECT * FROM admin.artists
```

Then, the first two rows would look something like this:

```
42, "Beatles", "P", "lob1.001.0.1409/", "lob1.002.0.243228/"  
68, "Queen", "P", "lob1.001.1409.1190/", "lob1.002.243228.206468/"
```

Note that the first LOB column uses the “lob1.001” name, and the second uses “lob1.002” name.

The *Data Movement Utilities Guide and Reference* says:

*A LOB Location Specifier (LLS) is used to store multiple LOBs in a single file when exporting LOB information. When exporting data using the lobsinfile modifier, the export utility selects the entire LOB file and places it in one of the LOB files. There may be multiple*

LOBs per LOB file and multiple LOB files in each LOB path. The data file will contain the LLS records.

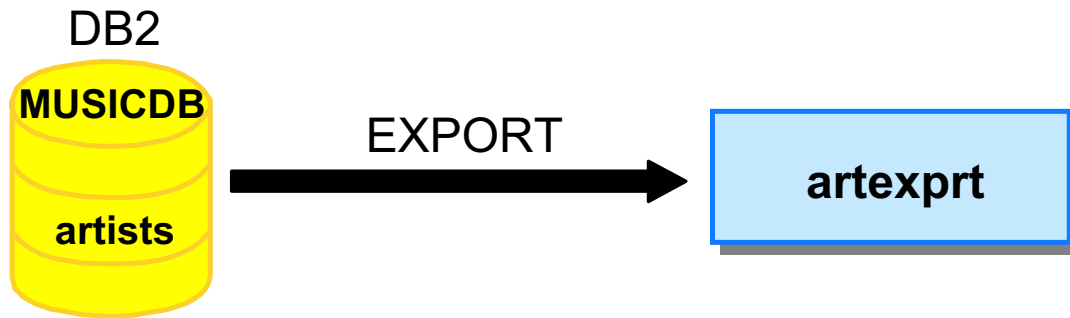
An LLS is a string indicating where LOB data can be found within a file. The format of the LLS is `filename.ext:nnn:mmm/`, where `filename.ext` is the name of the file that contains the LOB, `nnn` is the offset of the LOB within the file (measured in bytes), and `mmm` is the length of the LOB (in bytes). For example, an LLS of `db2exp.001.123.456/` indicates that the LOB is located in the file `db2exp.001`, begins at an offset of 123 bytes into the file, and is 456 bytes long. If the indicated size in the LLS is 0, the LOB is considered to have a length of 0. If the length is -1, the LOB is considered to be NULL and the offset and file name are ignored.

Certification objective: 5A Ability to use EXPORT utility to extract data from a table.

**Transition Statement** — Let's look at an example using the EXPORT command.

## EXPORT command example

- Exports data from database tables to file
- Check message for error or warning messages



```
db2 connect to musicdb
db2 export to artexprt
  of ixf messages artmsg
  select artno, name
  from artists
```

© Copyright IBM Corporation 2007

Figure 6-7. EXPORT command example

CF238.3

### Notes:

To export, you must have SYSADM, DBADM, or CONTROL or SELECT on each table or view that is referenced.

Any SELECT statement may be used to get information to be exported, but only SELECT \* FROM one table exported to IXF results in a recreatable IXF file.

***Instructor notes:***

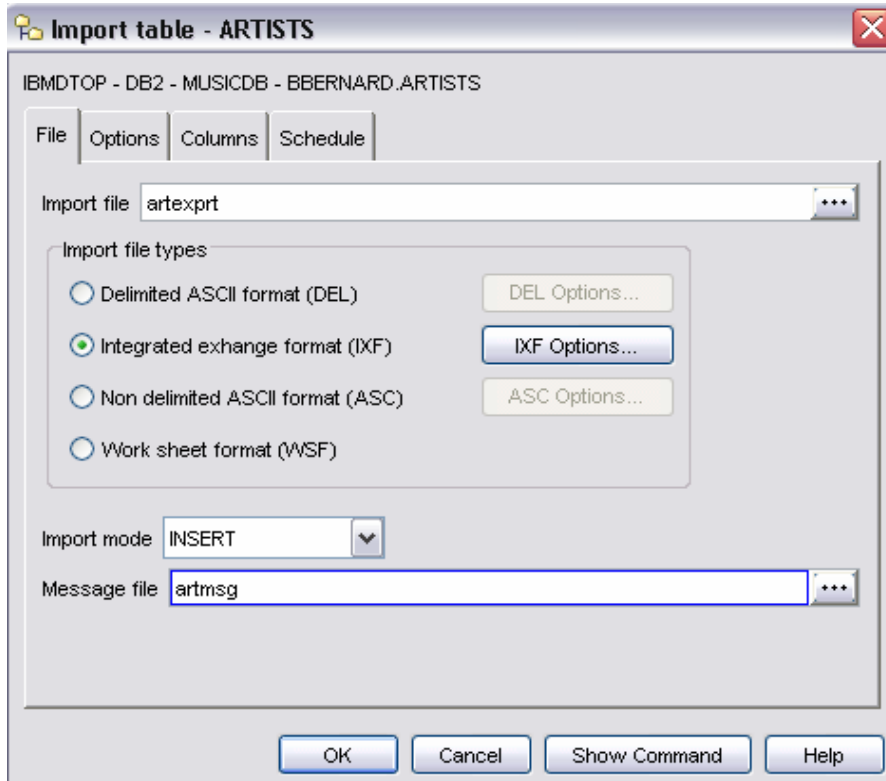
**Purpose** — Give an example of an EXPORT command.

**Details** — The example illustrates that you must be connected to do an export. The command on the graphic will create an output file named artexpri, in IXF format, that has the ARTNO and NAME column data for all of the rows in ARTISTS.

**Additional Information** —

**Transition Statement** — Let's now look at import.

# IMPORT command



© Copyright IBM Corporation 2007

Figure 6-8. IMPORT command

CF238.3

## Notes:

To Import data through GUI:

- Left-click and select **Tables**

Right-click and select the table with which you plan to work with from the *Contents* pane

- Select **Import**

From the Import GUI, you can import data from a file into a table.

You will find the following tabs in the Import GUI.

- File
- Columns
- Options
- Schedule

The Import GUI allows you to import data from files to tables, retrieve large objects from separate files, define which columns will be imported, define the method of import, and name the message file.

***Instructor notes:***

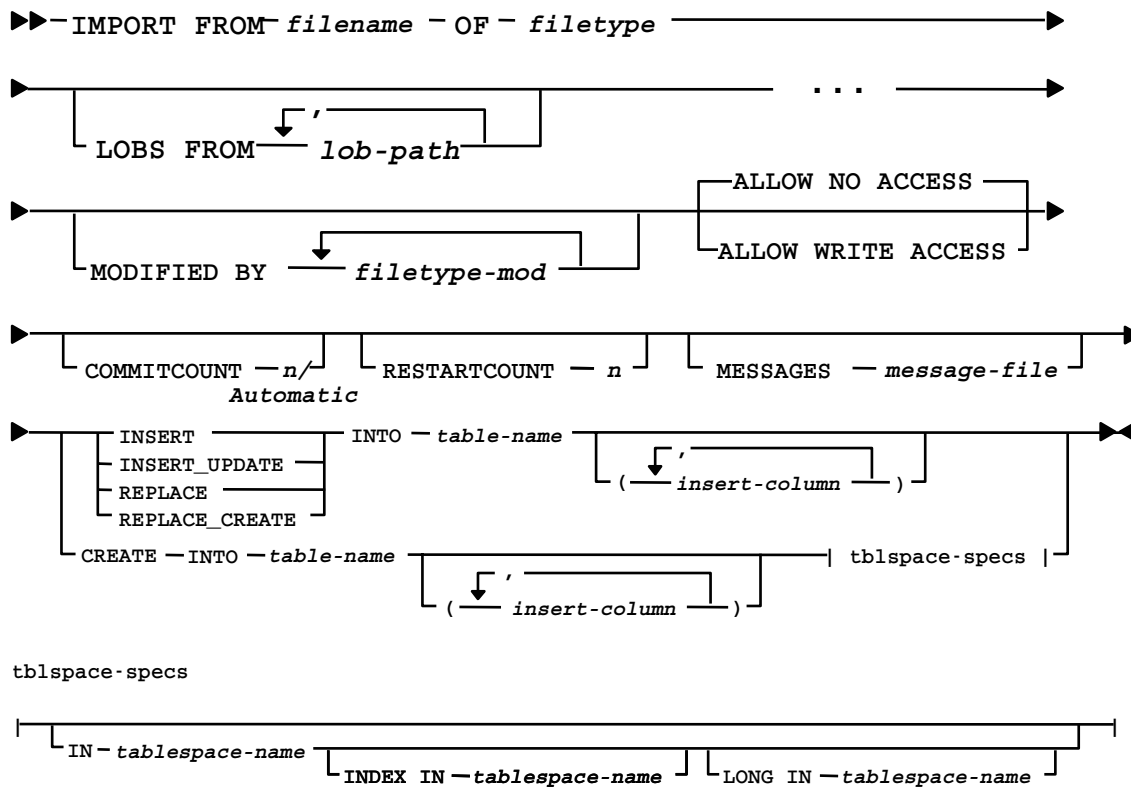
**Purpose** — Show the Import GUI.

**Details** — On the Import GUI, you must specify the file to import, the file type (note that there are four file types; ASC is added from the ones that were available for export), the import mode, and where messages should be written.

**Additional Information** —

**Transition Statement** — Let's look at the syntax of the command.

## IMPORT command syntax (basic)



© Copyright IBM Corporation 2007

Figure 6-9. IMPORT command syntax (basic)

CF238.3

### Notes:

The syntax of the IMPORT command is shown. More information on its options will be shown via examples on the following pages.

As default during the import an exclusive (X) lock is on the target table. This prevents concurrent applications from accessing table data. With the ALLOW WRITE ACCESS option the import runs in online mode. An intent exclusive (IX) is set on the target table. This allows concurrent readers and writers to access the table data. ALLOW WRITE ACCESS is not possible with the REPLACE, CREATE, or REPLACE\_CREATE options, or with buffered inserts. The import operation will periodically commit inserted data to prevent lock escalation and to avoid running out of active log space. These commits will be performed even if the COMMITCOUNT option was not used.

The COMMITCOUNT *n*/AUTOMATIC performs a commit after every *n* records. When AUTOMATIC is specified, the import internally determines when a commit needs to be performed. The import utility will commit for either one of two reasons:

- To avoid running out of active log space



- To avoid lock escalation

If ALLOW WRITE ACCESS option is specified and the COMMITCOUNT option is not specified, the import utility will perform commits as if COMMITCOUNT AUTOMATIC has been specified.

Some IMPORT command parameters:

### **LOBS FROM lob-path**

The names of the LOB data files are stored in the main data file (ASC, DEL, or IXF), in the column that will be loaded into the LOB column. The maximum number of paths that can be specified is 999. This option is ignored if the lobsinfile modifier is not specified.

### **MODIFIED BY filetype-mod**

Specifies file type modifier options.

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>compound=x</b>       | x is a number between 1 and 100 inclusive. Uses nonatomic compound SQL to insert the data, and x statements will be attempted each time.                                                                                                                                                                                                                                                                                                     |
| <b>generatedignore</b>  | This modifier informs the import utility that data for all generated columns is present in the data file but should be ignored. This results in all values for the generated columns being generated by the utility. This modifier cannot be used with the generatedmissing modifier.                                                                                                                                                        |
| <b>generatedmissing</b> | If this modifier is specified, the utility assumes that the input data file contains no data for the generated columns (not even NULLs), and will therefore generate a value for each row. This modifier cannot be used with the generatedignore modifier.                                                                                                                                                                                   |
| <b>identityignore</b>   | This modifier informs the import utility that data for the identity column is present in the data file but should be ignored. This results in all identity values being generated by the utility. The behavior will be the same for both GENERATED ALWAYS and GENERATED BY DEFAULT identity columns. This means that for GENERATED ALWAYS columns, no rows will be rejected. This modifier cannot be used with the identitymissing modifier. |
| <b>identitymissing</b>  | If this modifier is specified, the utility assumes that the input data file contains no data for the identity column (not even NULLs), and will therefore generate a value for each row. The behavior will be the same for both GENERATED ALWAYS and GENERATED BY DEFAULT identity columns. This modifier cannot be used with the identityignore modifier.                                                                                   |
| <b>lobsinfile</b>       | lob-path specifies the path to the files containing LOB data.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>no_type_id</b>       | Valid only when importing into a single sub-table. Typical usage is to export data from a regular table, and then to invoke an                                                                                                                                                                                                                                                                                                               |

|                      |                                                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | import operation (using this modifier) to convert the data into a single sub-table.                                                                           |
| <b>nodefaults</b>    | If a source column for a target table column is not explicitly specified, and the table column is not nullable, default values are not loaded.                |
| <b>norowwarnings</b> | Suppresses all warnings about rejected rows.                                                                                                                  |
| <b>seclabelchar</b>  | Indicates that security labels in the input source file are in the string format for security label values rather than in the default encoded numeric format. |
| <b>seclabelname</b>  | Indicates that security labels in the input source file are indicated by their name rather than the default encoded numeric format.                           |
| <b>usedefaults</b>   | If a source column for a target table column has been specified, but it contains no data for one or more row instances, default values are loaded.            |

## **ALLOW NO ACCESS**

Runs import in the offline mode. An exclusive (X) lock on the target table is acquired before any rows are inserted. This prevents concurrent applications from accessing table data. This is the default import behavior.

## **ALLOW WRITE ACCESS**

Runs import in the online mode. An intent exclusive (IX) lock on the target table is acquired when the first row is inserted. This allows concurrent readers and writers to access table data.

## **COMMITCOUNT n/AUTOMATIC**

Performs a COMMIT after every n records are imported. When a number n is specified, import performs a COMMIT after every n records are imported. When compound inserts are used, a user-specified commit frequency of n is rounded up to the first integer multiple of the compound count value. When AUTOMATIC is specified, import internally determines when a commit needs to be performed.

## **RESTARTCOUNT n**

Specifies that an import operation is to be started at record n + 1. The first n records are skipped. This option is functionally equivalent to SKIPCOUNT. RESTARTCOUNT and SKIPCOUNT are mutually exclusive.

## **MESSAGES message-file**

Specifies the destination for warning and error messages that occur during an import operation.

## **INSERT**

Adds the imported data to the table without changing the existing table data.

**INSERT\_UPDATE**

Adds rows of imported data to the target table, or updates existing rows (of the target table) with matching primary keys.

**REPLACE**

Deletes all existing data from the table by truncating the data object, and inserts the imported data. The table definition and the index definitions are not changed. This option can only be used if the table exists.

**REPLACE\_CREATE**

If the table exists, deletes all existing data from the table by truncating the data object, and inserts the imported data without changing the table definition or the index definitions.

If the table does not exist, creates the table and index definitions, as well as the row contents, in the code page of the database.

**INTO table-name**

Specifies the database table into which the data is to be imported. This table cannot be a system table, a declared temporary table or a summary table.

**CREATE**

Creates the table definition and row contents in the code page of the database. If the data was exported from a DB2 table, sub-table, or hierarchy, indexes are created.

**IN tablespace-name**

Identifies the table space in which the table will be created. The table space must exist, and must be a REGULAR table space. If no other table space is specified, all table parts are stored in this table space. If this clause is not specified, the table is created in a table space created by the authorization ID.

**INDEX IN tablespace-name**

Identifies the table space in which any indexes on the table will be created. This option is allowed only when the primary table space specified in the IN clause is a DMS table space. The specified table space must exist, and must be a REGULAR or LARGE DMS table space.

**LONG IN tablespace-name**

Identifies the table space in which the values of any long columns (LONG VARCHAR, LONG VARGRAPHIC, LOB data types, or distinct types with any of these as source types) will be stored.

## ***Instructor notes:***

**Purpose** — Identify the syntax of the IMPORT command.

**Details** — Since you have already talked about export, you can build on that discussion. The required items are: file name and file type must be specified. You must indicate what action is to be done (insert, insert\_update, replace, replace\_create, or create), and the table name. You can only use one of the “create” (replace\_create or create) options if the file type is IXF.

Optionally, if there are LOBs in the input file with the file name of where the LOB is located, then you specify LOBS FROM lob-path and MODIFIED BY LOBSINFILE. Other file type modifiers could be, for example, the delimiter character for delimited files.

Also take some time to discuss with the students the ALLOW NO ACCESS and ALLOW WRITE ACCESS options which have been introduced with V8.2.

The import utility is like an application doing inserts, so it logs its work. If you are importing a very large amount of data, you should specify COMMITCOUNT to prevent your logs from filling up. (**Note:** An import REPLACE will perform an auto-commit when it gets a log full message; an import INSERT will roll back.) If you had successfully imported a number of records, and then had a failure, the rows that had been committed would still be in the table. You could then restart the import and use RESTARTCOUNT; specify the last successful record number in RESTARTCOUNT, and the import would start by importing the record at  $n+1$ .

You can indicate where the messages should be written with the MESSAGES option.

**Additional Information** — If you started the list with Export, you can now add to it. Add Import using a different color marker, and put a check mark next to DEL, WSF, and IXF, indicating that these are valid for import as well. Then write ASC in at the bottom (write it more under Import) as a new file type, valid for import, but not for export.

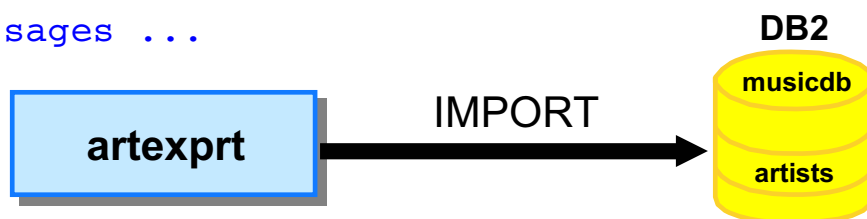
Certification objective: 5B Ability to use IMPORT utility to insert data into a table.

**Transition Statement** — Let's look at some examples.

## IMPORT command example

- Imports data from a file to a database table
- Check message for error or warning messages

```
db2 connect to musicdb
db2 import from artexprt of ixf
    messages artmsg 
        create
        insert
        insert_update
        replace
        replace_create
     into artists
.....
db2 import from artexprt of ixf modified by
compound 100
messages ...
```



© Copyright IBM Corporation 2007

Figure 6-10. IMPORT command example

CF238.3

### Notes:

There are several IMPORT options available to load a table.

- **CREATE** — If a table does not exist already, the table is created using PC/IXF file information before the data is inserted.
- **INSERT** — Data is inserted into an existing table without changing the existing table data.
- **INSERT\_UPDATE** — Add rows of imported data to the target table or updates existing rows (of the target table) with matching primary keys.
- **REPLACE** — Data in the existing table is deleted before the new data is inserted.
- **REPLACE\_CREATE** — Old data in an existing table is deleted before the new data is inserted. If the specified table does not already exist, then the table is created using PC/IXF file information before the data is inserted.

**MESSAGES** with no message file specified will result in messages written to standard output.

**COMMITCOUNT n/AUTOMATIC** performs a commit every **n** records. This option can help to keep log sizes manageable. When **AUTOMATIC** is specified, the import internally determines when a commit needs to be performed. The import utility will commit for either of two reasons:

- To avoid running out of active log space
- To avoid lock escalation

If the **ALLOW WRITE ACCESS** option is specified and the **COMMITCOUNT** option is not specified, the import utility will perform commits as if **COMMITCOUNT AUTOMATIC** has been specified.

**RESTARTCOUNT n** specifies that an import is to be started at **record n+1**. The first **n** records are skipped.

**METHOD L** specifies the start and end column numbers from which to import data. This option must be used for **ASC** files.

**METHOD N** specifies the names of the columns to be imported.

**METHOD P** specifies the order of column numbers to be imported.

To import using **INSERT**, you must have **SYSADM**, **DBADM**, or **CONTROL** on the table, or **INSERT** and **SELECT** on the table.

To import to an existing table using **INSERT\_UPDATE**, **REPLACE**, or **REPLACE\_CREATE**, you must have **SYSADM**, **DBADM**, or **CONTROL** on the table.

To import to a table that does not exist using **CREATE** or **REPLACE\_CREATE**, you must have **SYSADM**, **DBADM**, or **CREATETAB** authority and either **IMPLICIT\_SCHEMA** or **CREATEIN** privilege.

When using the **IMPORT** utility:

- **REPLACE** and **REPLACE\_CREATE** functions are not allowed if the object table has any dependents other than itself.
- Only a simple "SELECT \* FROM table" was used during **EXPORT**.
- The success of importing into a table with self-referencing constraints depends on the order in which the rows are inserted.

The **MODIFIED BY** option allows specification of additional information. Documentation of specific options is provided in the *Data Movement Utilities Guide and Reference* and the *Command Reference*. One such option is **COMPOUND n**, which allows **n** statements to be sent as a block using non-atomic compound SQL. The **COMPOUND** value ranges from 1 to 100. **COMPOUND** can greatly reduce the overhead associated with importing data.

***Instructor notes:***

**Purpose** — Give examples of using the Import utility.

**Details** — The first example is an import of an IXF file. Since it is IXF format, then any of the operations are valid. CREATE and REPLACE\_CREATE can only be issued with IXF format data.

The second example shows a MODIFIED BY option, MODIFIED BY COMPOUND 100. This option indicates that 100 records should be passed to the database manager for it to process, and let the database manager insert those 100 records, and then interface with the import utility again. Using the MODIFIED BY COMPOUND option, you can significantly improve the performance of the import, particularly if you are invoking it from a remote server. The largest number you can specify with the MODIFIED BY COMPOUND option is 100.

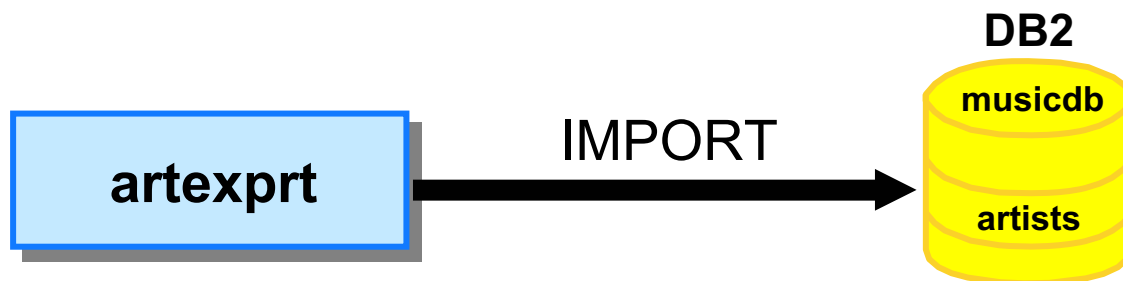
**Additional Information** —

**Transition Statement** — An additional example when using the CREATE operation.

## Import command ... CREATE ... IN TABLESPACE

- Imports data from a file to three DMS table spaces

```
db2 connect to musicdb
db2 import from artexprt of ixf
    messages artmsg create into    artists [(column_list)]
                                in      <tablespace>
                                index    in <indextablespace>
                                long     in <largetablespace>
```



© Copyright IBM Corporation 2007

Figure 6-11. Import command... CREATE... IN TABLESPACE

CF238.3

### Notes:

You can specify a target table space when issuing an import with the CREATE option. Note that to specify a table space for indexes or long data, you must specify a table space for the table. All three table spaces must be DMS if an index or long table space is indicated. You can indicate that the table should be created in an SMS table space, but all three components will be stored in the same table space.

It is not required to specify a separate table space for long data or indexes.



***Instructor notes:***

**Purpose** — Show another example when using the CREATE operation.

**Details** — When the CREATE option is used, you can specify where the table should be created (as with the CREATE TABLE statement). The regular table space, index data, and long data table spaces can all be indicated; if the index and long data table space are specified, they must all be DMS table spaces.

**Additional Information** —

**Transition Statement** — The Import utility is one way to get data from a flat file into database tables. Another way to do this is the LOAD utility.



## 6.2 LOAD utility

### Instructor topic introduction

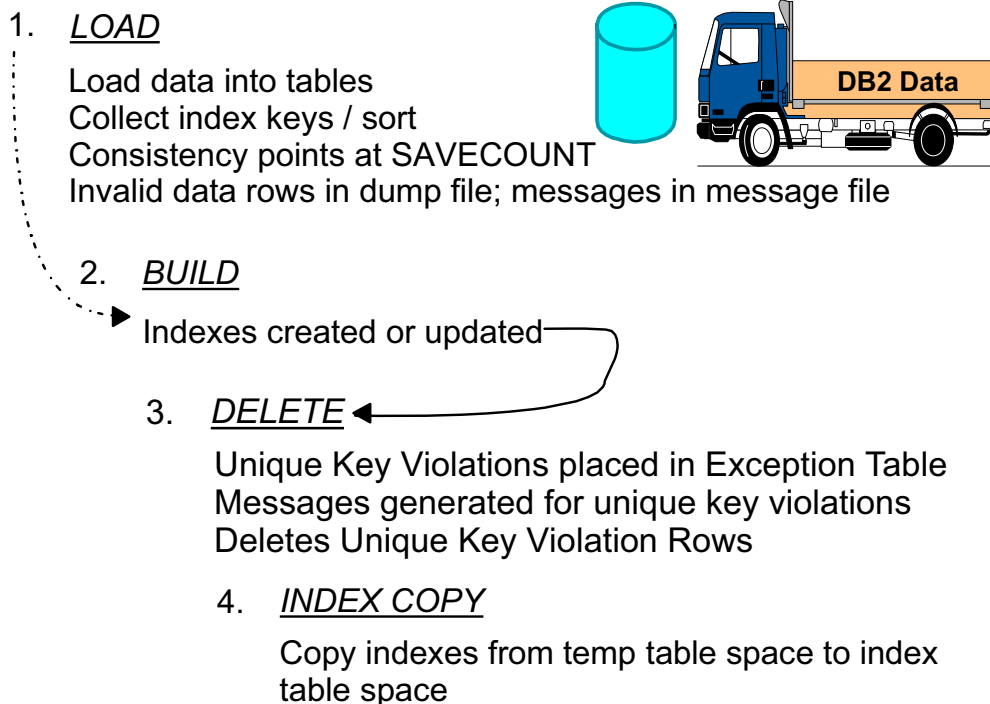
**What students will do** — Learn about the LOAD utility.

**How students will do it** — Listen to the lecture and practice in the lab.

**What students will learn** — High-level usage of the LOAD facility.

**How this will help students on their job** — They will be able to use the LOAD facility for simple loads.

## Four phases of Load



© Copyright IBM Corporation 2007

Figure 6-12. Four phases of Load

CF238.3

### Notes:

The LOAD utility is faster than the IMPORT utility, because it writes formatted pages directly into the database, whereas the import utility performs SQL INSERTs. The LOAD utility does not fire triggers, and does not perform referential or table constraint checking (other than validating the uniqueness of the indexes). The LOAD utility also performs only minimal logging.

The LOAD utility is intended for the initial load or an append of a table where large amounts of data are to be moved. All data types (except XML) can be loaded by the LOAD utility, including large objects (LOBs) and user-defined types (UDTs).

The following file formats are supported with LOAD:

- Non-delimited ASCII
- Delimited ASCII
- PC/IXF (International Exchange Format)

There are four phases of the LOAD process:

1. **Load** — when the data is inserted into the table.
2. **Build** — when the indexes are created or updated.
3. **Delete** — when the rows that caused a unique key violation are removed from the table.
4. **Index Copy** — when the index data is copied from a system temporary table space to the original table space.

The LOAD utility displays messages during the processing of each phase. If a failure occurs during the LOAD process, the messages will assist you in deciding how to recover from the failure. The LOAD utility can be invoked with a RESTART option, and the LOAD utility will determine where it needs to start processing.

The LOAD utility loads data from an input file, a device, or a pipe into a table. **This table must exist.** Indexes on the table may or may not exist; LOAD only builds indexes that are already defined on the table. If the table receiving the data already contains data, you can replace or append to the existing data.

## ***Instructor notes:***

**Purpose** — Identify the phases of the LOAD utility.

**Details** — There are four phases of the LOAD utility: Load, Build, Delete, and Index Copy.

During the Load phase, DB2 loads the data into extents on disk. If you characterize the import as doing *insert* type logic, Load does *smash* type logic: Load knows what the format of the data in extents should be, it gets the data from the input file, formats it into extents, and *smashes* it onto the disks. It doesn't go through the database manager to see if the row already exists, it doesn't fire triggers, it doesn't enforce constraints (though duplicate unique keys will be recognized, and constraints will be checked through a separate SET INTEGRITY processing). The Load phase collects the index keys, takes a consistency check at SAVECOUNT points (if requested), and puts invalid data rows into a dump file, and messages into a message file.

Once the Load phase is done, DB2 starts the Build phase. During the Build phase, DB2 creates or updates all of the indexes that are defined on the table at the time of the load.

Next, it enters the Delete phase. At this point, if there are any unique key violations, those rows are copied to an exception table. Messages are generated for these rows, and then the unique key violation rows are deleted from the base table.

Finally, the load will perform an index copy. It is only required to do this if the indexes are being rebuilt in a system temporary table space. If the invocation of the LOAD command requested this, then the index copy copies the index data to the index table space.

**Additional Information** — Table statistics are collected during the Load phase, and index statistics are collected during the build phase. The rows with unique key violations will be deleted in the Delete phase and the statistics are NOT changed. Check to ensure that the deletions are not a significant number compared to the total number of rows in the resultant table.

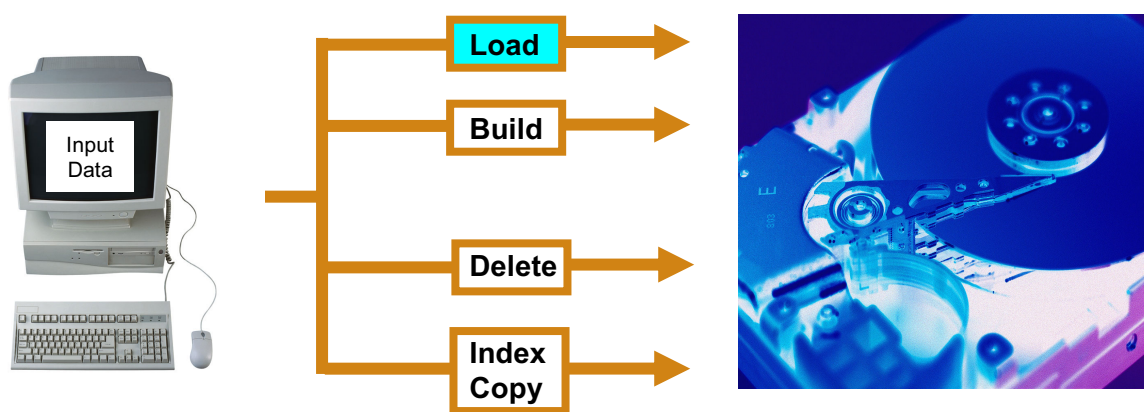
Index copy was added as a new phase in V8 to allow for read access to the table and its indexes during the load operation.

Certification objective: 5C Ability to use the LOAD utility to insert data into a table.

**Transition Statement** — Now let's look at each phase in detail, starting with the LOAD phase.

## LOAD: Load phase

- During the LOAD phase, data is stored in a table and index keys are collected
- Save points are established at intervals
- Messages indicate the number of input rows successfully loaded
- If a failure occurs in this phase, use the RESTART option for LOAD to restart from the last successful consistency point



© Copyright IBM Corporation 2007

Figure 6-13. LOAD: Load phase

CF238.3

### Notes:

During the **Load phase**, data is loaded into the table, and index keys and table statistics are collected, if necessary.

*Save points, or points of consistency*, are established at intervals specified by you in the **SAVECOUNT** parameter on the LOAD command. Messages are generated to let you know how many input rows have been successfully loaded at the time of the save point. If a failure occurs, you can restart the LOAD operation; the RESTART option automatically restarts the LOAD from the last successful consistency point. The TERMINATE option rolls back the failed load operation.

## ***Instructor notes:***

### **Purpose —**

**Details —** Following is a summary of changes to load behavior introduced in DB2 9.1:

- In DB2 UDB Version 8, if you do not specify the LOBS FROM option when using the LOAD command, the default search path for files containing LOB values is the current working directory. In DB2 Version 9.1, if you do not specify the LOBS FROM option, the default search path is the directory in which the input data files reside. If there are multiple input data files, the utility must verify whether the input data files come from the default path. If not, an SQL3052N message is returned and you are asked to specify the LOBS FROM option.
- In DB2 UDB Version 8, if a lob file is not found, the row is rejected if the column is not nullable, or NULL if the column is nullable. In DB2 Version 9.1, if a LOB file is not found, the row is rejected regardless of the nullability of the column.
- In DB2 Version 9.1, message SQL3040N is split into two distinct messages. SQL3040N is returned for lobfile errors and SQL3235N is returned for LOB path errors. The invalid file name or path name is indicated in the message.
- In DB2 UDB Version 8, if the LOB Location Specifier (LLS) contains a path, for example, the LLS is /home/try/newlob.001.12.345/ and the path is invalid, SQL3040N reason code 6 is returned and the utility exits immediately. In DB2 Version 9.1, the row is rejected and processing continues. In DB2 V9.1, the exported LLS never contains a path name.
- In DB2 UDB Version 8, a LOAD FROM CURSOR operation does not allow loading of mismatched codepage tables. An example of a mismatch includes a non-Unicode database with Unicode tables. In DB2 Version 9.1, codepage conversion is supported in the LOAD FROM CURSOR operation.
- In DB2 UDB Version 8, the FILE\_TRANSFER\_CMD handles the fetching of data. In DB2 Version 9.1, the fetching of data from the source-db is handled by the Load utility using a user exit process.

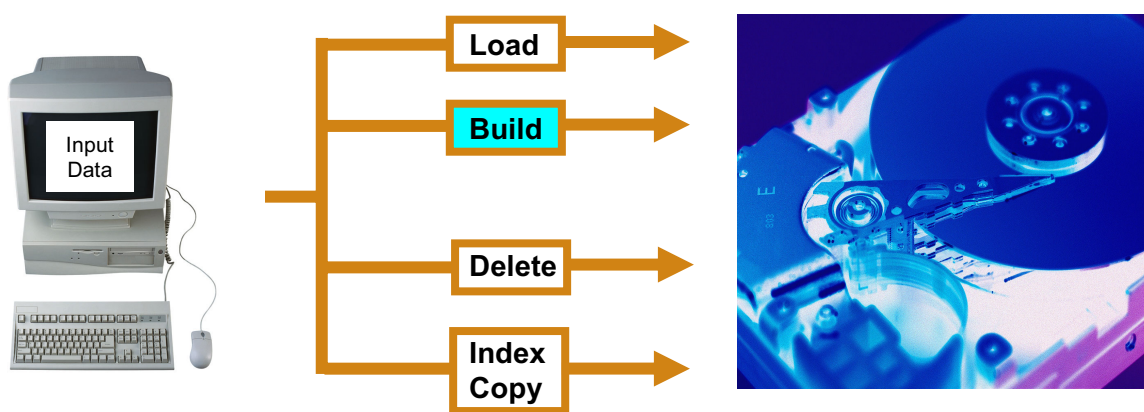
### **Additional information —**

**Transition statement —** Now we look at the BUILD phase.



## LOAD: Build phase

- During the BUILD phase, indexes are created based on the index keys collected in the load phase
- The index keys are sorted during the load phase
- If a failure occurs during this phase, LOAD restarts from the BUILD phase



© Copyright IBM Corporation 2007

Figure 6-14. LOAD: Build phase

CF238.3

### Notes:

During the **Build phase**, indexes are produced based on the index keys collected during the Load phase. The index keys are sorted during the load phase and index statistics are collected (if the STATISTICS YES with INDEXES option was specified). The statistics are similar to those collected through the RUNSTATS command. If a failure occurs during the Build phase, the RESTART option automatically restarts the load operation at the appropriate point.

***Instructor notes:***

**Purpose —**

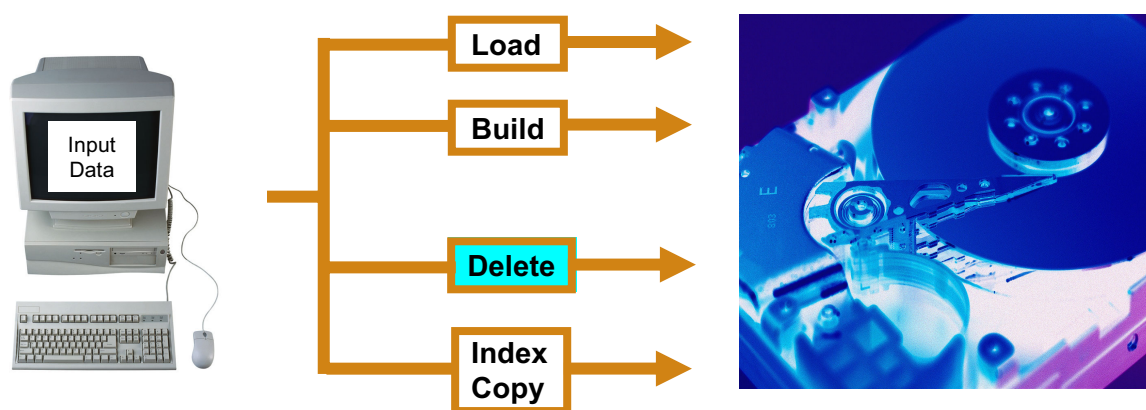
**Details —**

**Additional information —**

**Transition statement —** Now we look at the DELETE phase.

## LOAD: Delete phase

- During the DELETE phase, all rows that have violated a unique constraint are deleted
- If a failure occurs, LOAD restarts from the DELETE phase
- Once the database indexes are rebuilt, information about the rows containing the invalid keys is contained in an exception table, **WHICH SHOULD BE CREATED BEFORE LOAD**
- Finally, any duplicate keys found are deleted



© Copyright IBM Corporation 2007

Figure 6-15. LOAD: Delete phase

CF238.3

### Notes:

During the **Delete phase**, rows that caused a unique key violation are removed from the table.

Unique key violations are placed into the exception table, if one was specified, and messages about rejected rows are written to the message file. Following the completion of the load process, review these messages, resolve any problems, and insert corrected rows into the table.

Do not attempt to delete or to modify any temporary files created by the load utility. Some temporary files are critical to the Delete phase. If a failure occurs during the Delete phase, the RESTART option automatically restarts the load operation at the appropriate point.

Each deletion event is logged. If you have a large number of records that violate the uniqueness condition, the log could fill up during the Delete phase.

***Instructor notes:***

**Purpose —**

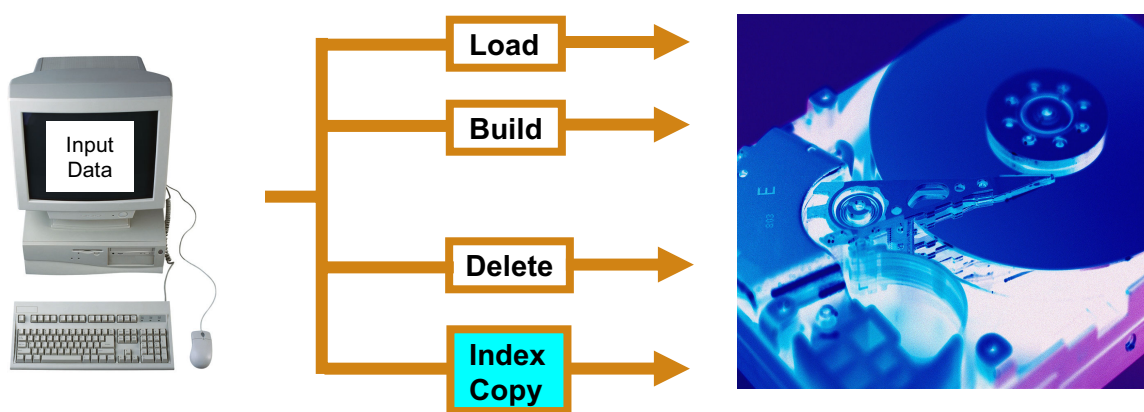
**Details —**

**Additional information —**

**Transition statement —** Now we look at the INDEX COPY phase.

## LOAD: Index Copy phase

- The index data is copied from a system temporary table space to the original table space.
- This will only occur if a system temporary table space was specified for index creation during a load operation with the READ ACCESS option specified.



© Copyright IBM Corporation 2007

Figure 6-16. LOAD: Index Copy phase

CF238.3

### Notes:

During the **Index copy** phase, the index data is copied from a system temporary table space to the original table space. This will only occur if a system temporary table space was specified for index creation during a load operation with the READ ACCESS option specified.

***Instructor notes:***

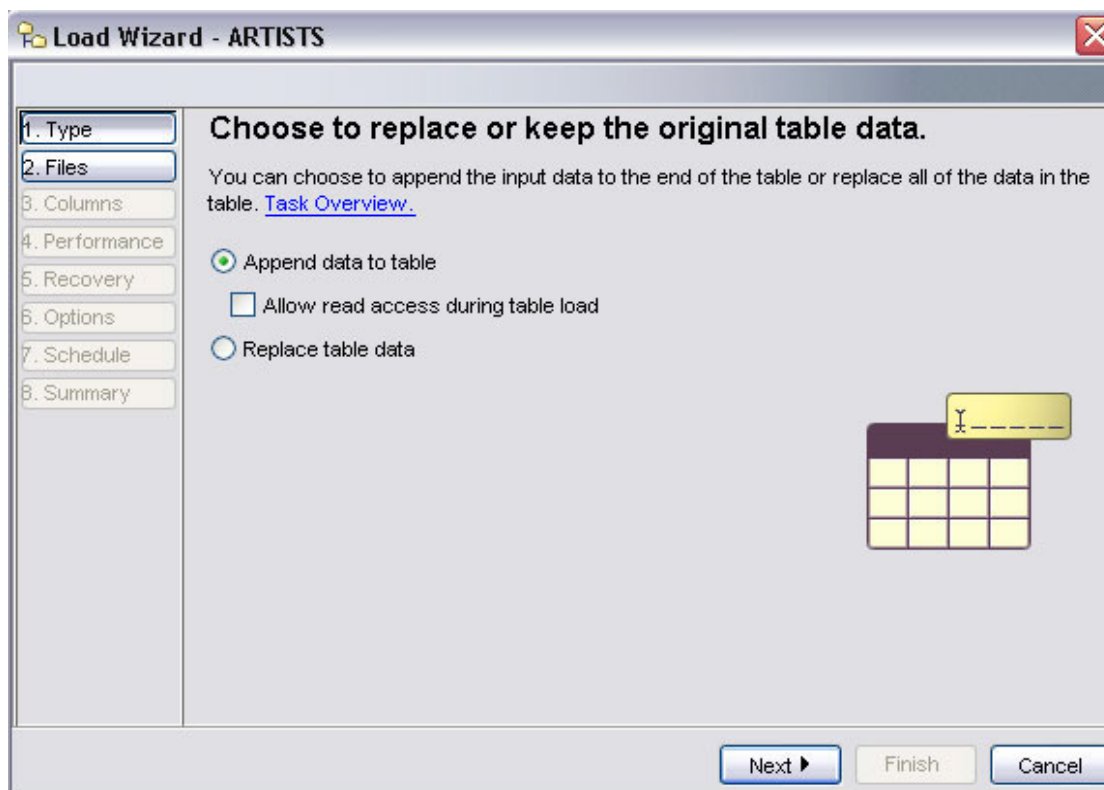
**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Let's look at the GUI to invoke the load.

# Load command



© Copyright IBM Corporation 2007

Figure 6-17. Load command

CF238.3

## Notes:

To load data through the Load wizard:

- Left click and select **Tables**

Right-click and select **the table** that you plan to work with from the *Contents* pane

- Select **Load** — during the load, data is loaded from a file into a table.

You will find the following pages in the Load wizard:

- Type
- Files
- Columns
- Performance
- Recovery
- Options
- Schedule
- Integrity
- Summary

The Load GUI allows the method (append or replace) for the load operation to be specified. It allows data to be loaded from files, pipes, or devices to tables, and it allows specification of the messages file. It allows selection of columns to be loaded.

On the **Performance** page, you specify options such as the level of checking of the input data, how indexes are updated, how the load interacts with other applications, the percentage of each data page left free, and if you want a storage management snapshot (if you have storage management enabled).

Recovery options include creating consistency points, generating row count messages, the order of the source data, what actions to take if the load fails, and how to handle forward recovery.

On the **Options** page, you can specify additional, more advanced options for the load task.

On the **Schedule** page, you specify when you want DB2 to perform the load task: you can run the task immediately, save the task to the Task Center, or schedule the task to be run at a later time.

If there are dependent tables on the table being loaded, the Integrity page will show a list of the tables that are dependent on the table being loaded; you can use this as an aid to choose the set integrity actions for the dependent tables after a successful load.

Finally, the **Summary** page will reflect the options you chose as you progressed through the wizard.



## ***Instructor notes:***

**Purpose** — Show the Load wizard GUI.

**Details** — Information about the Load wizard copied from the **help screen** (you should generally show the actual screen rather than the screen shot provided):

Use the Load wizard to load data into a selected table. The Load wizard guides you through load configuration and the selection of options. The Load wizard also lets you copy an existing load task and use the setting values of the existing load task for your new load task.

If you want to use exception tables with the load, you must create the exception tables before running the load task. (If you don't have exception tables, unique key violations will just be written to the messages file.)

### **Operation**

This page appears if your table resides in a partitioned database. On this page, you select the kind of load operation you require for the table. You can do a split operation only, a load operation only, or a split and load operation together. You can also select to perform an analysis of the input data and generate an optimized partition map.

### **Type**

On this page, you select the method (append or replace) for the load operation. If you select to append the data to the existing table, you can then specify if you want the users to have read access to the table during the load. If you choose to replace existing table data with the new data, the table is always taken offline during the replace.

### **File**

On this page, you specify information about the location of your input and output files, and the input file format. The input file formats that you can use are ASC, DEL, IXF, and CURSOR. The selected file names, path names, and format options are used in the generated LOAD command. The options presented on this page are dependent on where your input file resides, which combination of split and load you selected, and whether you are working with database partitions.

### **Partitions**

This page appears if your table resides in a partitioned database. On this page, you specify if you want to load data to all partitions, or only to selected partitions. You can also choose specific partitions to be used for data partitioning or let DB2 decide.

### **Columns**

On this page, you specify your column options and use a tabular mapper to specify values such as start position and end position. The options displayed on the Column page are determined by the file format of the input file. If your input file type is ASC positional text, you can also use a graphical mapper to map your columns.

The Column page does not appear if you selected Split only.

## Performance

On this page, you specify performance and statistics options for the load task. You specify options such as the level of checking of the input data, how indexes are updated, how the load interacts with other applications, the percentage of each data page left free, and if you want a storage management snapshot (if you have storage management enabled).

## Recovery

On this page, you specify the recovery options for the load task. Recovery options include creating consistency points, generating row count messages, the order of the source data, what actions to take if the load fails, and how to handle forward recovery.

## Options

On this page, you specify additional, more advanced options for the load task. You can change parameter values, restore the parameter values, or return to the default parameter values.

## Enabling Scheduling

This page only appears if you have not already created a scheduling database. On this page, you enable the scheduling function by creating a database for the DB2 tools catalog. The scheduling function allows you to run a task immediately, save the task to the Task Center, or schedule a task to be run at a later time. In order to use the scheduling function, you need to create a database for the DB2 tools catalog to store task and schedule information.

## Schedule

On this page, you specify when you want DB2 to perform the load task. You can run the task immediately, save the task to the Task Center, or schedule the task to be run at a later time. If you save the task to the Task Center, you can go to the Task Center later and schedule when you want the task to run.

## Integrity

On this page, you view a list of the tables that are dependent on the table being loaded. This is read-only information that you can use as an aid to choose the SET INTEGRITY actions for the dependent tables after a successful load.

## Summary

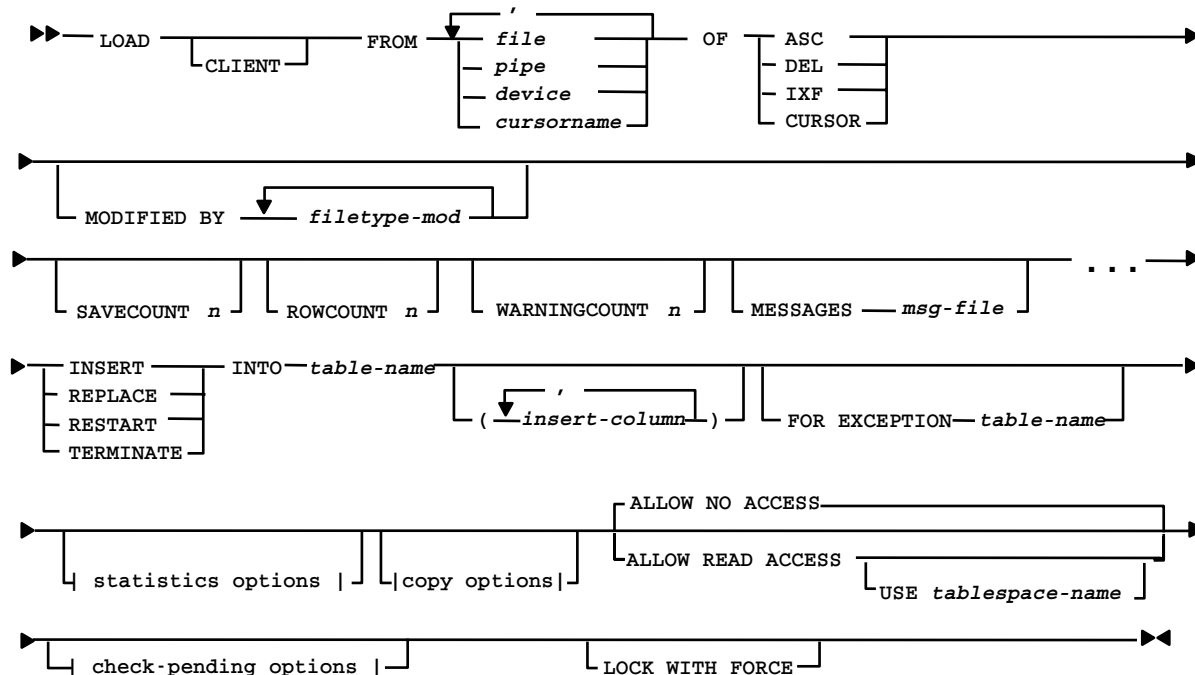
The Summary page that appears at the end of the wizard is dependent on your selections on the Schedule page. If you selected to run the task immediately, the Summary page shows the options that you selected for the task. You can review the task, go back in the wizard and make any necessary changes, and then click **Finish** to execute the commands.

If you selected to save the task to the Task Center and run the task later, the Summary page shows the task name, task script, and other information about the task. You can also select to launch the Task Center from this page.

**Additional Information —**

**Transition Statement** — Let's look at the LOAD command.

## LOAD command syntax (basic)



© Copyright IBM Corporation 2007

Figure 6-18. LOAD command syntax (basic)

CF238.3

### Notes:

The command syntax for the LOAD command is very extensive. Only some of the many parameters will be discussed in the following sections. Details regarding the LOAD command and all of its options can be found in the *Data Movement Utilities Guide and Reference*.

Some LOAD command parameters:

#### CLIENT

Specifies that the data to be loaded resides on a remotely connected client. This option is ignored if the load operation is not being invoked from a remote client. This option is ignored if specified in conjunction with the CURSOR filetype.

#### FROM filename/pipename/device/cursorname

Specifies the file, pipe, device, or cursor referring to an SQL statement that contains the data being loaded. If the input source is a file, pipe, or device, it must reside on the database partition where the database resides, unless the CLIENT option is specified.

**OF filetype**

Specifies the format of the data:

- **ASC** (non-delimited ASCII format)
- **DEL** (delimited ASCII format)
- **IXF** (integrated exchange format, PC version), exported from the same or from another DB2 table.
- **CURSOR** (a cursor declared against a SELECT or VALUES statement).

**MODIFIED BY filetype-mod**

Specifies file type modifier options. See File type modifiers for the Load utility.

**SAVECOUNT n**

Specifies that the load utility should set consistency points after every n rows. This value is converted to a page count, and rounded up to intervals of the extent size.

**ROWCOUNT n**

Specifies the number of n physical records in the file to be loaded. Allows a user to load only the first n rows in a file.

**WARNINGCOUNT n**

Stops the load operation after n warnings. Set this parameter if no warnings are expected, but verification that the correct file and table are being used is desired.

**MESSAGES message-file**

Specifies the destination for warning and error messages that occur during the load operation.

**INSERT**

One of four modes under which the load utility can execute. Adds the loaded data to the table without changing the existing table data.

**REPLACE**

One of four modes under which the load utility can execute. Deletes all existing data from the table, and inserts the loaded data. The table definition and index definitions are not changed.

**RESTART**

One of four modes under which the Load utility can execute. Restarts a previously interrupted load operation. The load operation will automatically continue from the last consistency point in the Load, Build, or Delete phase.

**TERMINATE**

One of four modes under which the load utility can execute. Terminates a previously interrupted load operation, and rolls back the operation to the point in time at which it started, even if consistency points were passed.

## **INTO table-name**

Specifies the database table into which the data is to be loaded. This table cannot be a system table or a declared temporary table. An alias, or the fully qualified or unqualified table name can be specified.

## **FOR EXCEPTION table-name**

Specifies the exception table into which rows in error will be copied. Any row that is in violation of a unique index or a primary key index is copied. The exception table must exist prior to LOAD.

## **ALLOW NO ACCESS**

Load will lock the target table for exclusive access during the load. The table state will be set to Load In Progress during the load. ALLOW NO ACCESS is the default behavior. It is the only valid option for LOAD REPLACE.

## **ALLOW READ ACCESS**

Load will lock the target table in a share mode. The table state will be set to both Load In Progress and Read Access. Readers can access the non-delta portion of the data while the table is being load.

## **USE tablespace-name**

If the indexes are being rebuilt, a shadow copy of the index is built in table space tablespace-name and copied over to the original table space at the end of the load during an INDEX COPY PHASE.

## **LOCK WITH FORCE**

The utility acquires various locks including table locks in the process of loading. Rather than wait, and possibly timeout, when acquiring a lock, this option allows load to force off other applications that hold conflicting locks on the target table. Applications holding conflicting locks on the system catalog tables will not be forced off by the load utility. Forced applications will roll back and release the locks the load utility needs.

**Instructor notes:**

**Purpose** — Identify the LOAD command syntax.

**Details** — The LOAD command requires that you specify where the data is coming from (file name, pipe name, device, or cursor name), the data type, the load mode (INSERT or REPLACE), and the table where the load is to occur. The other two options for load mode (RESTART and TERMINATE) are for error processing. We will address error processing and most of the other options as we continue in this unit. Many other MODIFIED BY OPTIONS are addressed in the CF46 class, *Advanced Administration for Experts*.

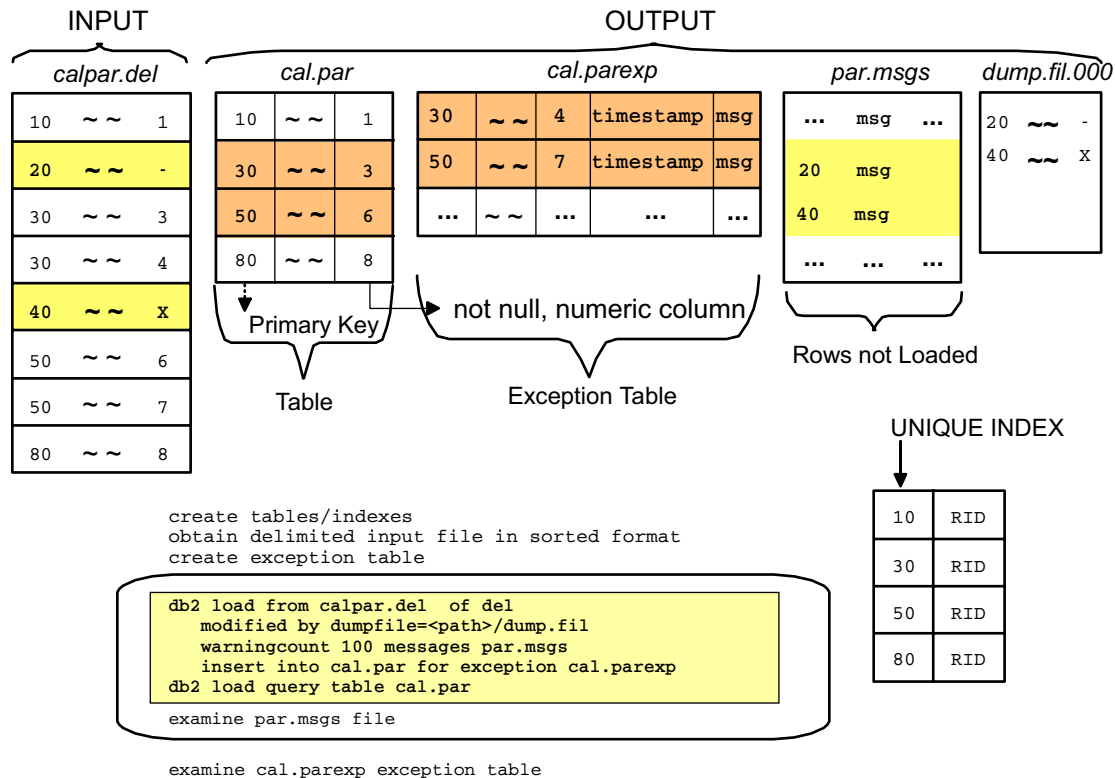
**Additional Information** — Use the *Data Movement Utilities Guide and Reference* for further research if needed.

Note that, if you created the table on the board for the file type options for export and import, then you can now indicate (maybe with another pen color), what file types are supported for load. (WSF is not supported, because no speed improvement was produced over import; cursorname is supported.)

Certification objective: 5C Ability to use the LOAD utility to insert data into a table.

**Transition Statement** — The easiest way to gain an understanding of most of the components of the LOAD is to look at a scenario.

# LOAD scenario



© Copyright IBM Corporation 2007

Figure 6-19. LOAD scenario

CF238.3

## Notes:

In our LOAD scenario, we have created tables and indexes, sorted the input data (or obtained it in sorted order), and created the exception table.

The LOAD exception table is a user-created table that mimics the definition of the table being loaded. It is specified by the FOR EXCEPTION option in the LOAD command. The table is used to store copies of rows that violate unique index rules.

**FOR EXCEPTION** indicates that any row that is in violation of a unique index rule will be stored in the table indicated. In our example, cal.parexp will contain those rows. If an exception table is not provided for the LOAD, and duplicate records are found, then the LOAD will continue. However, only a warning message is issued about the deletion of duplicate records, and the deleted duplicate records are not placed anywhere.

Other types of errors (for example, attempting to load a null into a column that is defined as NOT NULL) will cause a message to be written to the messages file. The second row in our example will cause this kind of warning in the message file. The DUMPFILE=qualified-filename option will write any rejected row to the named file. The



name must be a fully qualified name on the server. The file will be created with a partition number at the end; on a single partition database, this will be 000.

The exception tables used with LOAD are identical to the ones used in the SET INTEGRITY statement. They can be reused during checking with the SET INTEGRITY statements. There are a number of rules for creating exception tables; we will see them on the next visual.

In our example, cal is the owner/schema of the table.

To load data into a table, you must have one of the following:

- SYSADM or DBADM authority
- LOAD privilege on the database, and
  - INSERT privilege on the table when the load utility is invoked in INSERT mode
  - INSERT and DELETE privilege on the table when the load utility is invoked in REPLACE mode
  - INSERT privilege on the exception table, if used

Since you will typically be loading large amounts of data using the LOAD command, a LOAD QUERY command can be used to check the progress of the LOAD process. Options on the LOAD QUERY command allow you to indicate that you only want to see summary information (SUMMARYONLY), or just the new information since the last LOAD QUERY was issued (SHOWDELTA).

## Instructor notes:

**Purpose** — Show a load scenario.

**Details** — The best way to understand the load utility is to look at an example. In this example, we have these components:

- An input file, named *calpar.del*: this is a delimited input file sorted according to the values in the first column of the input data
- The table being loaded, named *cal.par*
- An exception table, named *cal.parexp*
- Two additional flat files: the messages file named *par.msgs*, and a file known as a dump file, named *dump.fil*

In the graphic, all of the objects are populated as if the commands have already taken place. For sake of this discussion, consider that the tables are empty when we start, and of course, *par.msgs* and *dump.fil* would be empty.

Look at the command area. For the load command to succeed, you need to have already created the tables that you want to load. Unlike the import utility, the load utility cannot create the table as part of its operation. If you want the load utility to create index data, you must also have already created the indexes for the table.

If you want the data, which is stored in the table, to be stored in a particular sequence, you should sort the input data prior to running the Load utility.

Finally, you must have already created the exception table. Let's look at the requirements for that. At this point, address the next two pages on the exception table, then come back to this graphic.

The tables and indexes have been created; the exception table has been created.

Let's look at the definition of the table we are loading, *cal.par*. The first column has been defined as a primary key. The last column is defined as numeric, and does not allow NULLs. When the exception table was created, the constraint of primary key was not applied to the first column, but the last column is defined as numeric and not allowing nulls.

Looking at the input data, we can see that there are a few records that may cause problems. Specifically, row 20 will be a problem because it has a null in the last column (that is what the dash in the last column is intended to represent). We also will have a problem with row 40 because it has an alpha character in the last column (which is defined in the table as numeric). We also have two row 30s and two row 50s. We'll see how those are handled as we go through the load process.

Let's look at the load command. It indicates *load from calpar.del*: the file we are loading is *calpar.del*. The *of del* indicates that the input data is delimited ascii. The next phrase indicates *modified by dumpfile=dump.fil*. We looked at the fact that the exception table is for unique key violations. The dump file is for records that violate the definitions of the table. Messages should be written to *par.msgs*. "warningcount 100" indicates that if more than 100 warnings are received, the load should stop. This may be a good setting if you think

that the data is clean; then if, for example, you specify an incorrect input file name, instead of having every record fail (which could take a very long time with a large input file), after 100 invalid records, the load will fail.

The load mode in this example is *insert*, or load append. If there was data already existing in the table, the existing data in the table will still be available while the load is appending data to the rest of the table.

The command also indicates that exception rows should be put into the cal.parexp exception table.

Let's look at what happens as the data is processed.

The first record, record 10, will go into the cal.par table with no problem.

The next record, record 20, fails the parsing check, so the data will be put out to the dumpfile, dump.fil. A message will also be written to the messages file, par.msgs.

During the Load phase, both record 30s will be inserted into the table. Even though they have duplicate keys, they won't be removed until the Build phase.

Record 40 fails the parsing check, so the data will be put out to the dumpfile, and a message will be written to the messages file.

Again, during the Load phase, both record 50s will be inserted into the table, and record 80 will be inserted in the table. At the end of the Load phase, all of the records in the calpar.del file except for the ones highlighted in yellow (20 and 40) will be in the cal.par table.

Then the Build phase starts. Since there is a primary key, the primary key index will be built.

During the Delete phase, the second record 30 and the second record 50 will be copied from cal.par to cal.parexp. The *second* records will be deleted from cal.par. Note that the first record is copied into the table with no problem. It is only the second and subsequent records that are considered to be duplicate keys.

If a separate table space had been specified for rebuilding the index, the Delete phase would be followed by the index copy phase: copying the rebuilt index from the temporary table space to the index table space.

The set of commands also shows the load query command. Call the students' attention to the information at the end of the student notes about options on the LOAD QUERY command: SUMMARYONLY and SHOWDETAIL. We will look at some of the information that can be gotten from the load query command on the next page (after the exception table information).

**Transition Statement** — You should already have addressed the next two pages on exception tables during the discussion of the scenario; so now let's look at the ALLOW NO | READ ACCESS options. If you did not address the next two pages during the discussion, then let's now discuss how to create the exception tables.

## Rules for creating Exception Table

- The first  $n$  columns are the same
- No constraints and no trigger definitions
- $n+1$  column      `TIMESTAMP`
- $n+2$  column      `CLOB (32 KB)`
- user                  `INSERT` privilege

© Copyright IBM Corporation 2007

Figure 6-20. Rules for creating Exception Table

CF238.3

### Notes:

The first  $n$  columns of the exception table reflect the definition of the table being loaded. All column attributes (type, length, and nullability attributes) should be identical. An exception table may not contain an identity column or any other type of generated column.

All the columns of the exception table should be free of any constraints and triggers. Constraints include referential integrity and check constraints, as well as unique index constraints that could cause errors on insert.

The  $n+1$  column of the exception table is an optional `TIMESTAMP` column. The timestamp column in the exception table may be used to distinguish newly-inserted rows from the old ones, if necessary.

The  $n+2$  column should be of type `CLOB (32 KB)` or larger. This column is optional but recommended, and will be used to give the names of the constraints that the data within the row violates.

No additional columns are allowed.

If the original table has generated columns (including the IDENTITY property), the corresponding columns in the exception table should not specify the generated property.

It should also be noted that a user invoking any facility (LOAD, SET INTEGRITY) that may cause rows to be inserted into the exception table must have INSERT privilege on the exception table.

For format of the Message Column Structure:

1. Number of constraint violations
2. Type of first constraint violation
3. Length of constraint/index name
4. Constraint name/index name
5. Separator
6. Type of next constraint violation
7. Length of constraint/index name
8. Constraint name/index name

Fields 5-8 repeat for each violation.

**Instructor notes:**

**Purpose** — Identify the characteristics of the LOAD exception table.

**Details** — The LOAD exception table is a user-created table that mimics the definition of the table being loaded. It is used to store copies of rows that violate unique index rules.

The first **n** columns of the exception table are the same as the columns of the table being checked. All column attributes should be identical. All of the columns of the exception table should be free of any constraints, such as referential integrity, check constraints, as well as unique index definitions and triggers.

The **n+1** column of the exception table is an optional **TIMESTAMP** column. The timestamp column in the exception table is used to distinguish newly inserted rows from the old ones, if necessary.

The **n+2** column should be of type **CLOB** (32 KB or larger). This column gives the names of the constraints that the data within the row violates. No additional columns are allowed.

Show the format of the Message column structure (call students' attention to the student notes information about the message column structure).

It should also be noted that, if you are invoking any facility that may cause rows to be inserted into the exception table, you must have **INSERT** privilege on the exception table.

**Additional Information** — For format of the Message Column Structure:

1. Number of constraint violations (5 characters, right justified padded with '0')
2. Type of first constraint violation (1 character; 'K' — Check Constraint violation; 'F' — Foreign Key violation; 'G' — Generated Column violation; 'I' — Unique Index violation; 'L' — DATALINK load violation; 'D' — Delete Cascade violation)
3. Length of constraint/index name (5 characters, right justified padded with '0')
4. Constraint name/index name (length from the previous field)
5. Separator (3 characters; <space><colon><space>)
6. Type of next constraint violation (1 character; 'K' — Check Constraint violation; 'F' — Foreign Key violation; 'G' — Generated Column violation; 'I' — Unique Index violation; 'L' — DATALINK load violation; 'D' — Delete Cascade violation)
7. Length of constraint/index name (5 characters, right justified padded with '0')
8. Constraint name/index name (length from the previous field)

Repeat field 5 through 8 for each violation.

**Transition Statement** — Let's look at how the exception table could be created.

## Suggested methods for creating Exception Tables

```
– CREATE TABLE T1EXC LIKE T1

ALTER TABLE T1EXC
  ADD COLUMN TS TIMESTAMP
  ADD COLUMN MSG CLOB(32K)

– CREATE TABLE T1EXC AS
  (SELECT T1.*,
    CURRENT_TIMESTAMP AS TS,
    CLOB(' ', 32767) AS MSG
  FROM T1)
  DEFINITION ONLY
```

© Copyright IBM Corporation 2007

Figure 6-21. Suggested methods for creating Exception Tables

CF238.3

### Notes:

There are several methods that can be used to create the exception tables. Two options are shown on the graphic that will create an exception table `t1exc` that could be used as an exception table for table `t1`. Two additional options would include:

- Generate DDL using the Control Center

The Control Center gives you the option of performing reverse engineering to create the DDL for a table. You can use this option to obtain the DDL for the original table, edit the statement generated to add the additional columns, and then run the file.

- Export using IXF

Since the IXF format of data carries the definition of the table as well as any data selected, you can use it to capture the definition of a table. The `IMPORT... CREATE` option would be used to create the new table, which would then be altered to add the additional columns.

## ***Instructor notes:***

**Purpose** — Show two methods for creating the exception table.

**Details** — There are several methods that can be used to create the exception tables. Two options are shown on the graphic that will create an exception table `t1exc` that could be used as an exception table for table `t1`.

The first option uses the LIKE syntax. This creates a second table (`t1exc`) with the same column definitions as the first table (`t1`). An Alter statement that adds the timestamp and message column definitions follows the table creation.

The second example shows the Create Table ... AS syntax. The definition indicates that the columns of T1 should be used, plus two other columns, a timestamp, and a CLOB message column.

Go back to the Load Scenario to finish talking about it.

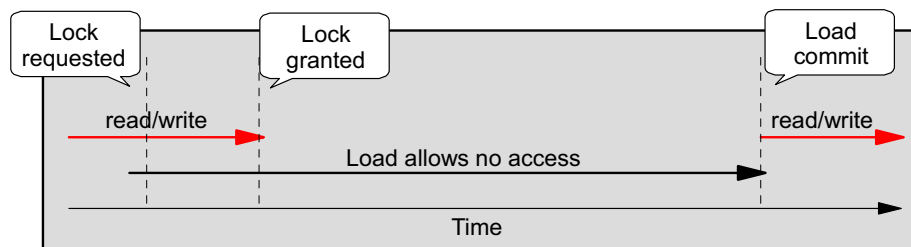
### **Additional Information** —

**Transition Statement** — Let's go back two pages to the Load Scenario to finish addressing it.

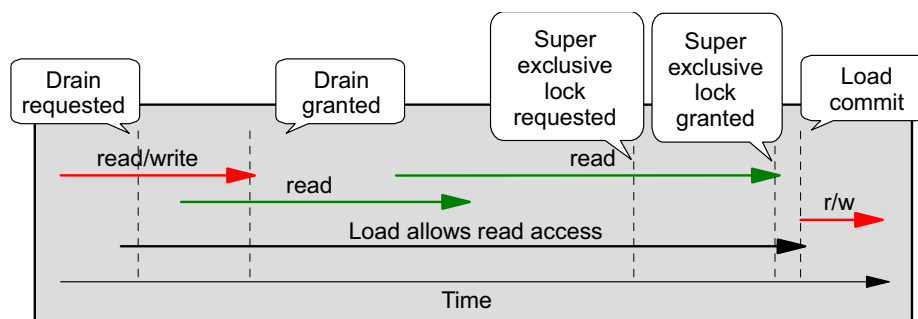


## Offline versus Online Load

### ■ ALLOW NO ACCESS



### ■ ALLOW READ ACCESS



© Copyright IBM Corporation 2007

Figure 6-22. Offline versus Online Load

CF238.3

### Notes:

In most cases, the LOAD utility uses table-level locking to restrict access to tables. The LOAD utility does not quiesce the table spaces involved in the load operation, and uses table space states only for load operations with the COPY NO option specified. The level of locking depends on whether the load operation allows read access. A load operation in ALLOW NO ACCESS mode will use an exclusive lock (Z-lock) on the table for the duration of the load. An load operation in ALLOW READ ACCESS mode acquires and maintains a share lock (S-lock) for the duration of the load operation, and upgrades the lock to an exclusive lock (Z-lock) when data is being committed.

Before a load operation in ALLOW READ ACCESS mode begins, the load utility will wait for all applications that began before the load operation to release locks on the target table. Since locks are not persistent, they are supplemented by table states that will remain even if a load operation is aborted. These states can be checked by using the LOAD QUERY command. By using the LOCK WITH FORCE option, the LOAD utility will force applications holding conflicting locks off the table into which it is trying to load.

ALLOW NO ACCESS is the default option on the LOAD utility.

## **Locking Behavior for Load Operations in ALLOW READ ACCESS Mode**

At the beginning of a load operation, the LOAD utility acquires a share lock (S-lock) on the table. It holds this lock until the data is being committed. The share lock allows applications with compatible locks to access the table during the load operation. For example, applications that use read-only queries will be able to access the table, while applications that try to insert data into the table will be denied. When the LOAD utility acquires the share lock on the table, it will wait for all applications that hold locks on the table prior to the start of the load operation to release them, even if they have compatible locks.

Since the LOAD utility upgrades the share lock to an exclusive lock (Z-lock) when the data is being committed, there may be some delay in commit time while the LOAD utility waits for applications with conflicting locks to finish.

**Note:** The load operation will not time out while it waits for the applications to release their locks on the table.

### **LOCK WITH FORCE Option**

The LOCK WITH FORCE option can be used to force off applications holding conflicting locks on a table so that the load operation can proceed. If an application is forced off the system by the load utility, it will lose its database connection and an error will be returned.

For a load operation in ALLOW NO ACCESS mode, all applications holding table locks that exist at the start of the load operation will be forced.

For a load operation in ALLOW READ ACCESS mode, applications holding the following locks will be forced:

- Table locks that conflict with a table share lock (for example, import or insert).
- All table locks that exist at the commit phase of the load operation.

When the COPY NO option is specified for a load operation on a recoverable database, all objects in the target table space will be locked in share mode before the table space is placed in backup pending state. This will occur regardless of the access mode. If the LOCK WITH FORCE option is specified, all applications holding locks on objects in the table space that conflict with a share lock will be forced off.

***Instructor notes:***

**Purpose** — Identify the coexistence of activity with the load.

**Details** — When the load starts, you indicate whether it should be in ALLOW NO ACCESS mode (the default), or in ALLOW READ ACCESS mode. If the load is started in ALLOW NO ACCESS, then all users of the table must finish their work, and then the load can start. You can expedite the starting of the load if you specify LOCK WITH FORCE, which allows the LOAD utility to force competing locks off.

If the load is started in ALLOW READ ACCESS, then any applications which are writing to the table must end before the load can start. Readers will also need to end if they started processing before the load was invoked. The load will allow reading applications to continue to use the table while the load is running as long as the reading application started after the load was invoked. At the end of the load utility, the load must upgrade its lock to commit the work. At that time, all applications must be off the table for the load to commit its work. Again, the LOCK WITH FORCE option can be used to allow the LOAD utility to force competing locks off.

**Additional Information** — We will look at this again in the Concurrency unit.

**Transition Statement** — Let's look at how to use the LOAD QUERY command.

## Table states

- 
- (Load pending, Set Integrity Pending)
  - LOAD QUERY TABLE <table-name>
  - Tablestate:
    - Normal
    - Set Integrity Pending
    - Load in Progress
    - Load Pending
    - Reorg Pending
    - Read Access Only
    - Unavailable
    - Not Load Restartable
    - Unknown
  - Table can be in several states at same time
    - Tablestate:
      - Set Integrity Pending
      - Load in Progress
      - Read Access Only

© Copyright IBM Corporation 2007

Figure 6-23. Table states

CF238.3

### Notes:

In addition to locks, the LOAD utility uses table states to control access to tables. A table state can be checked by using the LOAD QUERY command. The states returned by the LOAD QUERY command are as follows:

- **Normal** — No table states affect the table.
- **Set Integrity Pending** — The table has constraints which have not yet been verified. Use the SET INTEGRITY statement to take the table out of Check Pending state. The LOAD utility places a table in Set Integrity Pending state when it begins a load operation on a table with constraints.
- **Load in Progress** — There is a load operation in progress on this table.
- **Load Pending** — A load operation has been active on this table but has been aborted before the data could be committed. Issue a LOAD TERMINATE, LOAD RESTART, or LOAD REPLACE command to bring the table out of this state.

- **Read Access Only** — The table data is available for read access queries. Load operations using the ALLOW READ ACCESS option place the table in read access only state.
- **Reorg Pending**— There is a reorg operation in progress on this table.
- **Unavailable** — The table is unavailable. The table may only be dropped or restored from a backup. Rolling forward through a non-recoverable load operation will place a table in the unavailable state.
- **Not Load Restartable** — The table is in a partially loaded state that will not allow a load restart operation. The table will also be in load pending state. Issue a LOAD TERMINATE or a LOAD REPLACE command to bring the table out of the Not Load Restartable state. A table is placed in Not Load Restartable state when a roll forward operation is performed after a failed load operation that has not been successfully restarted or terminated, or when a restore operation is performed from an online backup that was taken while the table was in Load in Progress or Load Pending state. In either case, the information required for a load restart operation is unreliable, and the Not Load Restartable state prevents a load restart operation from taking place.
- **Unknown** — The LOAD QUERY command is unable determine the table state.

A table can be in several states at the same time. For example, if data is loaded into a table with constraints and the ALLOW READ ACCESS option is specified, the table state would be:

```
Tablestate:  
Set Integrity Pending  
Load in Progress  
Read Access Only
```

After the load operation but before issuing the SET INTEGRITY statement, the table state would be:

```
Tablestate:  
Set Integrity Pending  
Read Access Only
```

After the SET INTEGRITY statement has been issued, the table state would be:

```
Tablestate:  
Normal
```

## ***Instructor notes:***

**Purpose** — Identify how the table states can be queried and what they mean.

**Details** — The LOAD QUERY command can be used to query the table states. The graphic shows the table states. Use the student notes to define what each of the table states means.

Note also that a table can be in several states at the same time. Even though you have not yet addressed the SET INTEGRITY statement, you may want to show the information about how the table states can change as commands are run, as shown at the end of the student notes.

**Additional Information** — Certification objective: 2C Skill in discussing the various table space states.

**Transition Statement** — Let's look at recovering from load failure.

## Load Pending state: Recovering from LOAD failure



- Restart the Load
  - Check Messages files
  - Use Restart option
  - Load operation automatically continues from last consistency point in Load or Build phase
    - or Delete phase if ALLOW NO ACCESS
- Replace the whole table
  - LOAD ... REPLACE
- Terminate the Load
  - If LOAD ... INSERT, returns table to state preceding Load
  - If LOAD ... REPLACE, table will be truncated to an empty state
    - Create backup copy prior to Load to return to state preceding Load

**Do not tamper with Load temporary files**

© Copyright IBM Corporation 2007

Figure 6-24. Load Pending state: Recovering from LOAD failure

CF238.3

### Notes:

If the LOAD utility cannot start because of a user error, such as a nonexistent data file or invalid column names, it will terminate and leave the table in a normal state.

If a failure occurs while loading data, you can restart the load operation from the last consistency point (using the RESTART option), reload the entire table (using the REPLACE option), or terminate the load (using the TERMINATE option). Specify the same parameters as in the previous invocation so that the utility can find the necessary temporary files.

A load operation that specified the ALLOW READ ACCESS option can be restarted using either the ALLOW READ ACCESS option or the ALLOW NO ACCESS option. A load operation that specified the ALLOW NO ACCESS option can only be restarted using the ALLOW NO ACCESS option. If the index object is unavailable or marked invalid, a load restart or terminate in ALLOW READ ACCESS mode will not be permitted. If the original load operation was aborted in the Index Copy phase, a restart operation in the ALLOW READ ACCESS mode is not permitted because the index may be corrupted.

If a load operation in ALLOW READ ACCESS mode was aborted in the Load phase, it will restart in the Load phase. If it was aborted in any phase other than the Load phase, it will restart in the Build phase. If the original load operation was in ALLOW NO ACCESS mode, a restart operation may occur in the Delete phase if the original load operation reached that point and the index is valid. If the index is marked invalid, the Load utility will restart the load operation from the Build phase.

Load REPLACE deletes all existing data from the table and inserts the loaded data. Using load REPLACE and specifying an empty input file will truncate the table.

Terminating the load will roll back the operation to the point in time at which it started, even if consistency points were passed. The states of any tables involved in the operation return to normal, and all table objects are made consistent (index objects may be marked as invalid, in which case index rebuild will automatically take place at the next access). If the load operation being terminated is a load REPLACE, the table will be truncated to an empty table after the load TERMINATE operation. If the load operation being terminated is a load INSERT, the table will retain all of its original records after the load TERMINATE operation.



#### **Note**

The Load operation writes temporary files onto the server into a subdirectory of the database directory by default (the location can be changed using a `TEMPFILES PATH temp-pathname` parameter on the LOAD command). The temporary files written to this path are removed when the load operation completes without error. These temporary files must not be tampered with under any circumstances. Doing so will cause the load operation to malfunction, and will place your database in jeopardy.



**Instructor notes:**

**Purpose** — Identify ways to recover from problems during load.

**Details** — Suppose there is a problem when you run the load; what can you do?

For example, the Load utility, when running in insert mode, only uses previously empty extents in the table space; you may have deleted a large amount of data, but have not REORGed the table to reclaim the space, meaning that you have a lot of partially full extents. In insert mode, the load utility will use only *empty* extents, so you may end up with a load failure because the table space has filled up. If you rerun the load but as a load replace, it may succeed (but it will wipe out any data that you already had in the table, so that may not be an appropriate solution).

Another option would be to fix the problem (in this case, add more space to the table space) and then restart the load. The load utility will automatically restart from the appropriate place.

Suppose you just want to get back to the data that you had in the table before the load, and run with that data. If you were running a LOAD, then performing a LOAD will get the table data to the point it was before starting the load. If you were running a LOAD ... REPLACE, it will get you back to an empty table. If you want the option to return to the point right before the load, you should take a backup of the table spaces or of the database before starting the load to ensure recovery to that point.

In any case, you should not tamper with the temporary files written by load. They are required for load to be able to successfully terminate or restart.

**Additional Information** — You may want to examine the student notes regarding the table space states.

**Transition Statement** — Let's look at the impact of the COPY options on the Backup Pending status.

## Backup Pending state — COPY options

- When loading data and forward recovery enabled:
  - COPY NO (default)
    - During load, "Backup pending" and "Load in progress"
    - After load, "Backup Pending"
  - COPY YES
    - Load has made Copy ➡ not "Backup pending"
  - NONRECOVERABLE
    - No copy made and no backup required

### UNIX

Databasealias.Type.InstanceName.NodeName.CatNodeName.Timestamp.number

### Windows

Databasealias.Type.InstanceName.NodeName.CatNodeName.Timestamp.number

Type: 0=Full Backup  
 3=Table Space Backup  
 4 =Copy from Table Load

© Copyright IBM Corporation 2007

Figure 6-25. Backup Pending state — COPY options

CF238.3

### Notes:

If a load operation with the COPY NO option is executed in a recoverable database, the table spaces associated with the load operation are placed in the *Backup pending* table space state and the *Load in progress* table space state. This takes place at the beginning of the load operation. The load operation may be delayed at this point while locks are acquired on the tables within the table space.

When a table space is in Backup pending state, it is still available for read access. The table space can only be taken out of Backup pending state by taking a backup of the table space. Even if the load operation is aborted, the table space will remain in Backup pending state because the table space state is changed at the beginning of the load operation, and cannot be rolled back if it fails. The Load in Progress table space state prevents online backups of a load operation with the COPY NO option specified while data is being loaded. The Load in Progress state is removed when the load operation is completed or aborts.

Note that if the database is a recoverable database, then terminating the load will not eliminate the requirement to make a backup of the loaded table space.

When loading data, if forward recovery is enabled, there are three options to consider:

1. **COPY NO** (default) specifies that the table space in which the table resides will be placed in backup pending state if forward recovery is enabled (that is, logretain or userexit is on). COPY NO will also put the table space state into the Load in Progress table space state. This is a transient state that will disappear when the load completes or aborts. The data in any table in the table space cannot be updated or deleted until a table space backup or a full database backup is made. However, it is possible to access the data in any table by using the SELECT statement.
2. **COPY YES** on the LOAD specifies that a copy of the changes made will be saved. This copy is used during roll-forward recovery to recreate the changes to the database done by LOAD. This option is invalid if forward recovery is disabled. COPY YES slows the LOAD utility. If you are loading a lot of tables, you may wish to load all of the tables in the table space, then back it up.
3. **NONRECOVERABLE** specifies that the load transaction is to be marked as non-recoverable and that it will not be possible to recover it by a subsequent roll forward action. The roll forward utility will skip the transaction and will mark the table into which data was being loaded as *invalid*. The utility will also ignore any subsequent transactions against that table. After the roll forward operation is completed, such a table can only be dropped or restored from a backup (full or table space) taken after a commit point following the completion of the non-recoverable load operation.

With this option, table spaces are not put into backup pending state following the load operation, and a copy of the loaded data does not have to be made during the load operation. This can be used to enable loading several tables in a table space before performing a backup of the table space. It can also be used with tables that are always loaded with LOAD... REPLACE since they could be recovered by reloading.

If you do not create a copy, the LOAD will execute more quickly. You must also allow for the disk space that an additional backup copy would take.

The name of the backup file has entries for the **Type** field to indicate what it represents. There are three options:

1. 0 for full database backup
2. 3 for table space backup
3. 4 for Copy from Table Load

If load is used with the **nonrecoverable** option, there is no requirement to take a backup.

During a roll forward operation through a LOAD command with the COPY NO option specified, the associated table spaces are placed in *Restore pending* state. To remove the table spaces from Restore pending state, a restore operation must be performed. A roll forward operation will only place a table space in the Restore pending state if the load operation completed successfully.

## ***Instructor notes:***

**Purpose** — Identify recovery considerations of the Load utility.

**Details** — Since the Load utility does not log, if the database is defined for forward recovery (which means that you may want to execute a restore and roll forward through log files at some point in the future), then there are options for you to specify what you want to do regarding the recoverability of the load operation.

The default option is COPY NO. This means that no copy is made of the data during the load, but because the database is defined for forward recovery, the recoverability of the data is still important. In this case, when the load is issued, DB2 will place the table space into Load in Progress and Backup pending table space states. The Load in Progress state will just last for the duration of the load (to ensure that no one tries to issue an online backup of the table space while the load is running). The Backup pending status will continue to exist after the end of the load. Read processing will be allowed against the objects in the table space, but update/delete processing will be denied until the backup of the table space (or full database) completes.

Another option that can be specified on the load is COPY YES. With COPY YES, the load utility makes a copy of the data being loaded that is suitable for use during a roll forward operation. The making of the copy takes time and disk space, but it avoids the requirement of making a backup of the table space after the load.

A final option is to load the data with the option NONRECOVERABLE. This indicates to DB2 that, although it is in a recoverable database, the recoverability of this table is not important to you. You may choose this option if you are loading several tables in the same table space and just want to make one backup at the end. Or you may choose this option on a table on which you perform a LOAD REPLACE with data from another system; if the table on this database is lost, you can just load it from the last set of data. A lot more details on this and other options will be discussed during CF45, *Advanced Administration Workshop*.

If you do make a load copy, the information at the bottom of the graphic indicates what the file names will be that are created. The TYPE indicates a load copy if the TYPE has a value of “4”.

Only if the database is recoverable will the table space go into pending states during the load. During the load, the table space will be in *Load in Progress* and *Backup Pending*. After the load, the table space will just be in *Backup Pending*. The table space is still accessible while it is in Backup Pending, so an online backup can be made.

## **Additional Information —**

**Transition Statement** — A few pages ago we introduced the table pending states. Let's take a closer look at the Set Integrity Pending table state.

## Set Integrity Pending table state

- Load turns OFF constraint checking
  - Leaves table in Set Integrity Pending state
  - If parent table is in Set Integrity Pending, then dependents may also be in Set Integrity Pending
  - LOAD INSERT, ALLOW READ ACCESS
    - Loaded table in Set Integrity Pending with read access
  - LOAD INSERT, ALLOW NO ACCESS
    - Loaded table in Set Integrity Pending with no access
  - LOAD REPLACE, SET INTEGRITY PENDING CASCADE DEFERRED
    - Loaded table in Set Integrity Pending with no access
  - LOAD REPLACE, SET INTEGRITY PENDING CASCADE IMMEDIATE
    - Loaded table and descendant foreign key tables are in Set Integrity Pending with no access

© Copyright IBM Corporation 2007

Figure 6-26. Set Integrity Pending table state

CF238.3

### Notes:

Following a load operation, the loaded table may be in set integrity pending state in either READ or NO ACCESS mode if the table has table check constraints or referential integrity constraints defined on it. If the table has descendent foreign key tables, they may also be in set integrity pending state.

If the loaded table has descendent tables, the SET INTEGRITY PENDING CASCADE parameter can be specified to indicate whether the check pending state of the loaded table should be immediately cascaded to the descendent materialized query tables or descendent staging tables. SET INTEGRITY PENDING CASCADE does not apply to descendent foreign key tables. If the loaded table has constraints as well as descendent foreign key tables, and if all of the tables are in normal state prior to the load operation, the following will result based on the load parameters specified:

### INSERT, ALLOW READ ACCESS, and SET INTEGRITY PENDING CASCADE IMMEDIATE or DEFERRED

The loaded table will be placed in Set Integrity Pending state with read access. Descendent foreign key tables will remain in their original states.

## **INSERT, ALLOW NO ACCESS, and SET INTEGRITY PENDING CASCADE IMMEDIATE**

The loaded table will be placed in Set Integrity Pending state with no access. Descendent foreign key tables will remain in their original states.

## **INSERT or REPLACE, ALLOW NO ACCESS, and SET INTEGRITY PENDING CASCADE DEFERRED**

Only the loaded table will be placed in Set Integrity Pending state with no access. Descendent foreign key tables will remain in their original states.

## **REPLACE, ALLOW NO ACCESS, and SET INTEGRITY PENDING CASCADE IMMEDIATE**

The table and all its descendent foreign key tables will be placed in Set Integrity Pending state with no access.



### **Note**

Specifying the ALLOW READ ACCESS option in a load replace operation will result in an error.

**Instructor notes:**

**Purpose** — Identify that constraint checking must be done separately from the load.

**Details** — The LOAD command does not enforce constraints, except for unique index (including primary key and unique constraints) during the load operation. It leaves the table in Check Pending state. If a parent table is in Check Pending, then dependent tables may also be in Check Pending (if the load was load replace, and SET INTEGRITY PENDING CASCADE was issued).

**Note:** This is potentially confusing. Although the option is SET INTEGRITY PENDING CASCADE, it really only affects the Check Pending status of materialized query tables and staging tables. Since those kinds of tables are not addressed in CF2x, it can be confusing that the student notes state that INSERT, ALLOW NO ACCESS, CHECK PENDING CASCADE does not cascade the check pending condition to the foreign key dependents (it does cascade to descendent MQTs and staging tables). We elected to mention only MQTs and staging tables on the previous page. The main point to get across on this page is: the loaded table will be in check pending; the dependent foreign key tables will only be in check pending if you are doing a LOAD REPLACE, CHECK PENDING CASCADE IMMEDIATE (load replaces are always ALLOW NO ACCESS). Otherwise, the setting of CHECK PENDING CASCADE really has no impact on the Check Pending status of the foreign key table.

**Additional Information** — A change in behavior from V7 to V8: load will not automatically put foreign key tables into set integrity pending when the parent is loaded. If the load was LOAD INSERT, only the parent table will be in set integrity pending. The dependent tables will not go into set integrity pending until the set integrity statement is run on the parent table.

**Set Integrity Pending** was previously known as **Check Pending**.

**Transition Statement** — Let's look at an overview of the SET INTEGRITY statement.

## SET INTEGRITY command overview

- 
- SET INTEGRITY is used to:
    - Bring one or more tables out of Set Integrity Pending state (previously known as *Check Pending state*)
    - Bring one or more tables out of Set Integrity Pending state without performing required integrity processing on those tables
    - Place one or more tables in Set Integrity Pending state
    - Place one or more tables into full access state
    - Prune the contents of one or more staging tables

© Copyright IBM Corporation 2007

Figure 6-27. SET INTEGRITY command overview

CF238.3

### Notes:

When the statement is used to perform integrity processing for a table after it has been loaded or attached, the system can incrementally process the table by checking only the appended portion for constraints violations. If the subject table is a materialized query table or a staging table, and load, attach, or detach operations are performed on its underlying tables, the system can incrementally refresh the materialized query table or incrementally propagate to the staging table with only the delta portions of its underlying tables. However, there are some situations in which the system will not be able to perform such optimizations and will instead perform full integrity processing to ensure data integrity. Full integrity processing is done by checking the entire table for constraints violations, recomputing a materialized query table's definition, or marking a staging table as inconsistent. The latter implies that a full refresh of its associated materialized query table is required. There is also a situation in which you might want to explicitly request incremental processing by specifying the INCREMENTAL option.

The SET INTEGRITY statement is under transaction control.



***Instructor notes:***

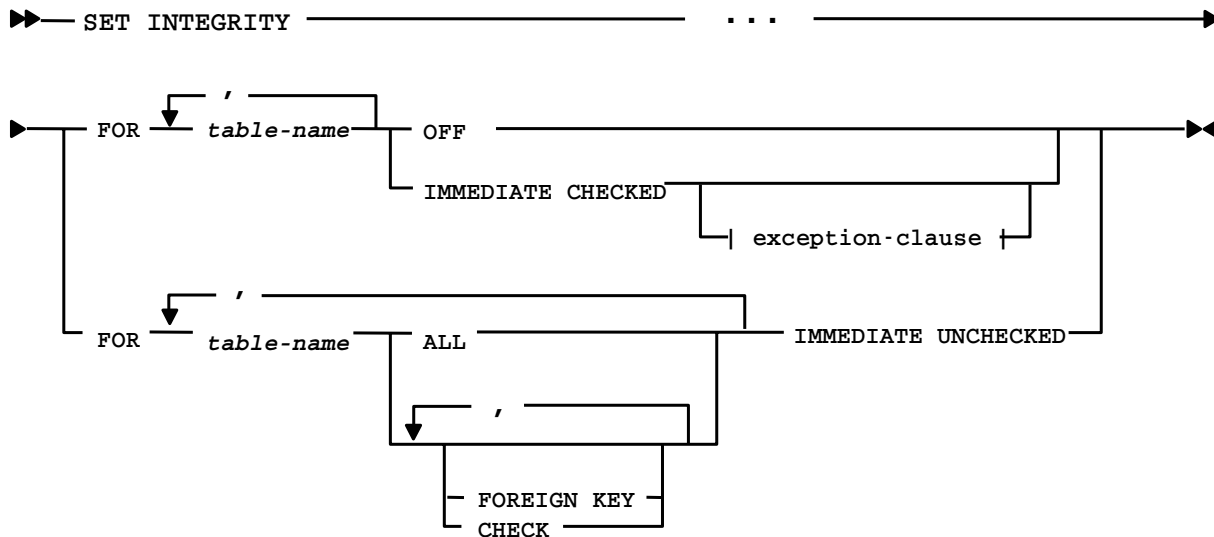
**Purpose** —

**Details** —

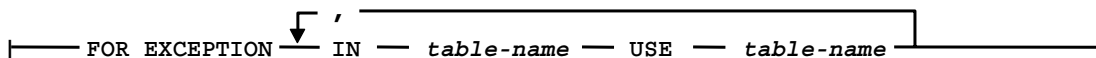
**Additional information** —

**Transition statement** — Let's look at the syntax of the SET INTEGRITY statement.

## SET INTEGRITY command syntax (basic)



### exception-clause



© Copyright IBM Corporation 2007

Figure 6-28. SET INTEGRITY command syntax ((basic))

CF238.3

### Notes:

The **OFF** option specifies that the tables (or table) are to have their foreign key constraints and check constraints turned off, and are therefore to be placed into the set integrity pending state, where only limited access by a restricted set of statements and commands is allowed. Primary key and unique constraints continue to be checked.

The **IMMEDIATE CHECKED** option specifies that the table is to have its integrity checking turned on and that the integrity checking that was deferred is to be carried out. This is done in accordance with the information set in the STATUS and CONST\_CHECKED columns of the SYSCAT.TABLES. The value in STATUS must be C (the table is in the set integrity pending state), and the value in CONST\_CHECKED indicates which integrity options are to be checked.

The **IMMEDIATE UNCHECKED** option specifies that the table is to have its integrity checking turned on (and thus is to be taken out of the set integrity pending state) without having the table checked for integrity violations. The implications with respect to data integrity should be considered before using this option.

Sample query to check the pending states on the tables:

```
select tabname,  
       substr(const_checked,1,1) as FK_CHECKED,  
       substr(const_checked,2,1) as CC_CHECKED  
from syscat.tables  
where status='C'
```

Byte 1 of CONST\_CHECKED column in SYSCAT.TABLES represents foreign key constraints. Byte 2 represents check constraints. Other bytes are reserved. Values are:

- **Y**=Checked by system
- **U**=Checked by user
- **N**=Not checked (pending)

***Instructor notes:***

**Purpose** — Identify the syntax of the SET INTEGRITY statement.

**Details** — Show the students the syntax of the SET INTEGRITY statement.

Identify that you can indicate OFF, IMMEDIATE CHECKED, or IMMEDIATE UNCHECKED. See the information in the student notes on the value in the CONST\_CHECKED column in SYSCAT.TABLES.

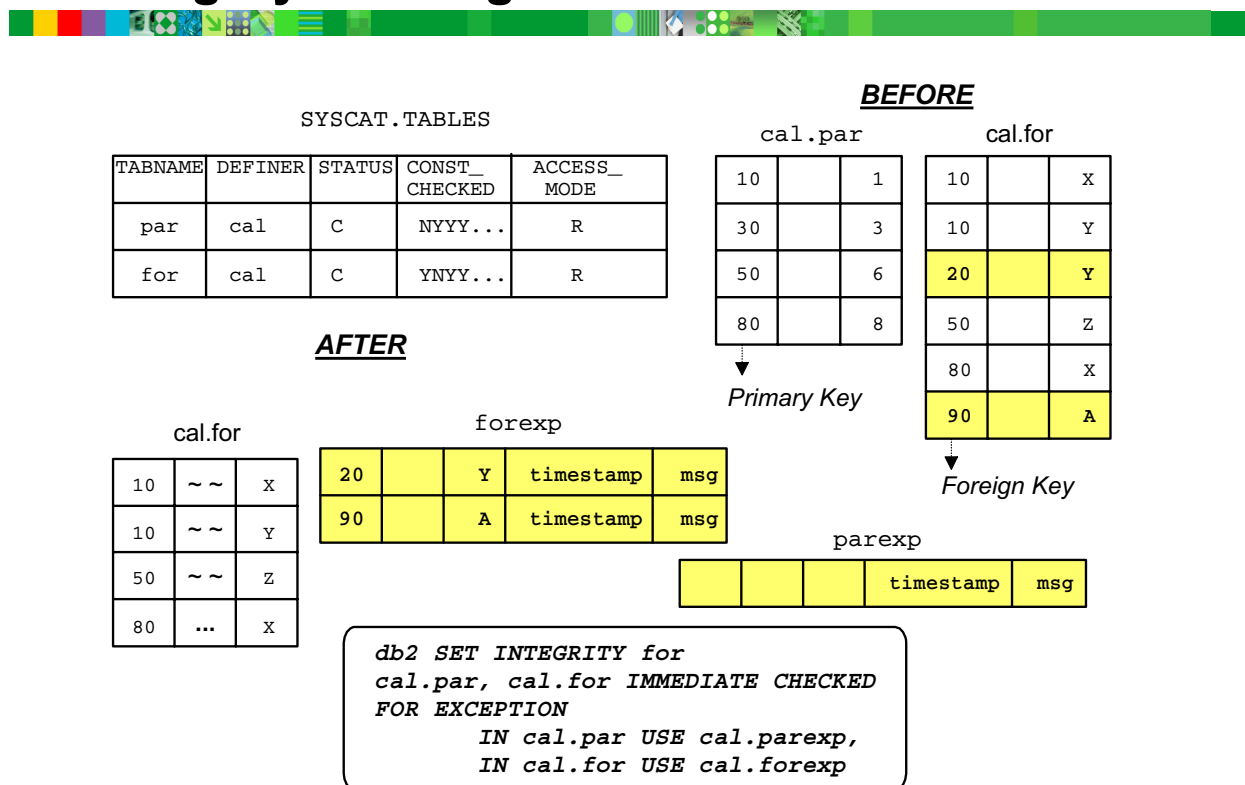
Sample query to check the pending states on the tables: Note that the flag for materialized query tables is in position 5, generated columns are in byte 6, and staging tables are in byte 7 (these are not addressed in this course).

Also mention the possibility of INCREMENTAL checking. By using INCREMENTAL, integrity checks will be performed on the appended portion of the table (if any). If the request could not be satisfied (for example, the system detects that the whole table needs to be checked) an error (SQLSTATE 55019) is returned. If the INCREMENTAL-options clause is not specified, the system will determine if incremental processing is possible; if not, the whole table will be checked.

```
select tabname,  
       substr(const_checked,1,1) as FK_CHECKED,  
       substr(const_checked,2,1) as CC_CHECKED,  
       substr(const_checked,5,1) as MaterializedQuery_CHECKED,  
       substr(const_checked,6,1) as Generated_CHECKED,  
       substr(const_checked,7,1) as Staging_CHECKED,  
from syscat.tables  
where status='C'
```

**Transition Statement** — Let's look at the statement you will use to get the table out of Set Integrity Pending.

## Set Integrity Pending state



© Copyright IBM Corporation 2007

Figure 6-29. Set Integrity Pending state

CF238.3

### Notes:

The STATUS flag of the SYSCAT.TABLES entry corresponding to the loaded table indicates the set integrity pending state of the table. For the loaded table to be fully usable, the STATUS must have a value of N and the ACCESS MODE must have a value of F, indicating that the table is in normal state and fully accessible.

In the example on the graphic, both CAL.PAR and CAL.FOR have been loaded. They both indicate that constraints need to be checked (STATUS = 'C'). CAL.PAR is a parent table, and CAL.FOR is its dependent. CAL.FOR also has a check constraint defined on one of its columns. In this case, both tables are in set integrity pending status because they have both been loaded.

The SET INTEGRITY statement is executed to check the constraints on both tables. CAL.FOR had two rows that did not have parent keys in CAL.PAR, so those rows were moved to the FOREXP exception table. The CAL.PAR table did not have any check constraint violations, so there are no rows added to the PAREXP table as a result of this SET INTEGRITY statement.

To remove the set integrity pending state, use the SET INTEGRITY statement. The SET INTEGRITY statement checks a table for constraint violations, and takes the table out of set integrity pending state. If all the load operations are performed in INSERT mode, the SET INTEGRITY statement can be used to incrementally process the constraints (that is, it will check only the appended portion of the table for constraint violations). For example:

```
db2 load from infile1.ixf of ixf insert into table1
db2 set integrity for table1 immediate checked
```

Only the appended portion of TABLE1 is checked for constraint violations. Checking only the appended portion for constraint violations is faster than checking the entire table, especially in the case of a large table with small amounts of appended data.

If a table is loaded with the SET INTEGRITY PENDING CASCADE DEFERRED option specified, and the SET INTEGRITY statement is used to check for integrity violations on the parent table, the descendent tables will be placed in set integrity pending state with no access when the SET INTEGRITY statement is issued. To take the descendent tables out of this state, you must issue an explicit SET INTEGRITY request on the descendent tables.

If the ALLOW READ ACCESS option is specified for a load operation, the table will remain in read access state until the SET INTEGRITY statement is used to check for constraint violations. Applications will be able to query the table for data that existed prior to the load operation once it has been committed, but will not be able to view the newly loaded data until the SET INTEGRITY statement has been issued.

Several load operations can take place on a table before checking for constraint violations. If all of the load operations are completed in ALLOW READ ACCESS mode, only the data that it existed in the table prior to the first load operation will be available for queries.

One or more tables can be checked in a single invocation of this statement. If a dependent table is to be checked on its own, the parent table cannot be in set integrity pending state. Otherwise, both the parent table and the dependent table must be checked at the same time. In the case of a referential integrity cycle, all the tables involved in the cycle must be included in a single invocation of the SET INTEGRITY statement. It may be convenient to check the parent table for constraint violations while a dependent table is being loaded. This can only occur if the two tables are not in the same table space.

Use the load exception table option to capture information about rows with constraint violations.

The SET INTEGRITY statement does not activate any DELETE triggers as a result of deleting rows that violate constraints, but once the table is removed from set integrity pending state, triggers are active. Thus, if we correct data and insert rows from the exception table into the loaded table, any INSERT triggers defined on the table will be activated. The implications of this should be considered. One option is to drop the INSERT trigger, insert rows from the exception table, and then recreate the INSERT trigger.

***Instructor notes:***

**Purpose** — Identify how the set integrity pending state of the table can be resolved.

**Details** — Columns in SYSCAT.TABLES will indicate whether any tables need to have constraints checked. A value of “C” in the status column indicates that check constraint processing should be done on the table. The CONST\_CHECKED column is a character 25 column where each byte refers to a constraint to check (only 7 are used up through V8). We will only address bytes 1 and 2 in this class; in the output, you can see an “N” in byte 1 for the CAL.PAR table, which indicates that referential constraints need to be checked. For CAL.FOR, there is an “N” in byte 2; this indicates that check constraints need to be checked. Actually, referential constraints also need to be checked on the CAL.FOR table, but that will not be determined until the SET INTEGRITY on CAL.PAR is done.

On the graphic, we see that there are rows in CAL.FOR that do not have parent rows in CAL.PAR. As a result of running the SET INTEGRITY statement, those rows are written to the FOREXP exception table, and delete from CAL.FOR. No additional rows are written to the CAL.PAREXP table as a result of the SET INTEGRITY statement.

**Additional Information** —

**Transition Statement** — Let’s summarize the steps for load.

## Meaningful steps for LOAD

- 
- Create tables and indexes
  - Create exception tables
  - Sort data
  - Back up TS/DB (if using REPLACE)
  - Consider freespace
  - Load .... for Exception ... Savecount ... Warningcount...
  - Examine xx.msg and dumpfile (after LOAD completes)
  - Examine exception tables (after LOAD completes)
  - Back up table space if log retain=recovery and COPY NO
  - Set Integrity for .... (only if table in Check Pending state)
  - Update statistics (if necessary)

© Copyright IBM Corporation 2007

Figure 6-30. Meaningful steps for LOAD

CF238.3

### Notes:

The following options may be used to create free space during load:

- **PCTFREE** can be set for indexes and tables; the default value is 10% free for indexes, 0% free for tables. The range of PCTFREE is 0 to 99. Allocating more index free space could reduce page splits which occur when more rows are inserted or loaded into the table. Allocating more table free space could ensure that space is available on the desired page when attempting to insert a row into a table with a clustered index.
- **PAGEFREESPACE=x** is a Load Utility parameter that specifies the percentage of each page that is to be left as free space. The value set by PAGEFREESPACE overrides the PCTFREE value specified for the table.

**STATISTICS YES** option may be used on the Load Utility to update statistics during execution of the load when the load is run in REPLACE mode.



***Instructor notes:***

**Purpose** — Conclude the topic with some additional thoughts.

**Details** — Most of this information can be used as a review of what has already been presented. New information is found in the student notes on free space.

**Additional information** —

**Transition statement** — Let's look at the db2move tool.

## Overview of the db2move tool

- 
- Moves data between different DB2 databases that may reside on different servers
  - Useful when a large number of tables need to be copied from one database to another
  - The utility can run in one of three modes:
    - Export — the EXPORT utility is used to export data from the table or tables specified into data files of type IXF
    - Import — the IMPORT utility used to import data files of type IXF into a given database
    - Load — the input files specified in the **db2move.lst** file are loaded into the tables using the LOAD utility

Tables with structured type columns (such as XML)  
are not moved when this tool is used.

© Copyright IBM Corporation 2007

Figure 6-31. Overview of the db2move tool

CF238.3

### Notes:

This tool, when used in the EXPORT/IMPORT/LOAD mode, facilitates the movement of large numbers of tables between DB2 databases located on workstations. The tool queries the system catalog tables for a particular database and compiles a list of all user tables. It then exports these tables in PC/IXF format. The PC/IXF files can be imported or loaded to another local DB2 database on the same system, or can be transferred to another workstation platform and imported or loaded to a DB2 database on that platform. Tables with structured type columns are not moved when this tool is used. When used in the COPY mode, this tool facilitates the duplication of a schema.

***Instructor notes:***

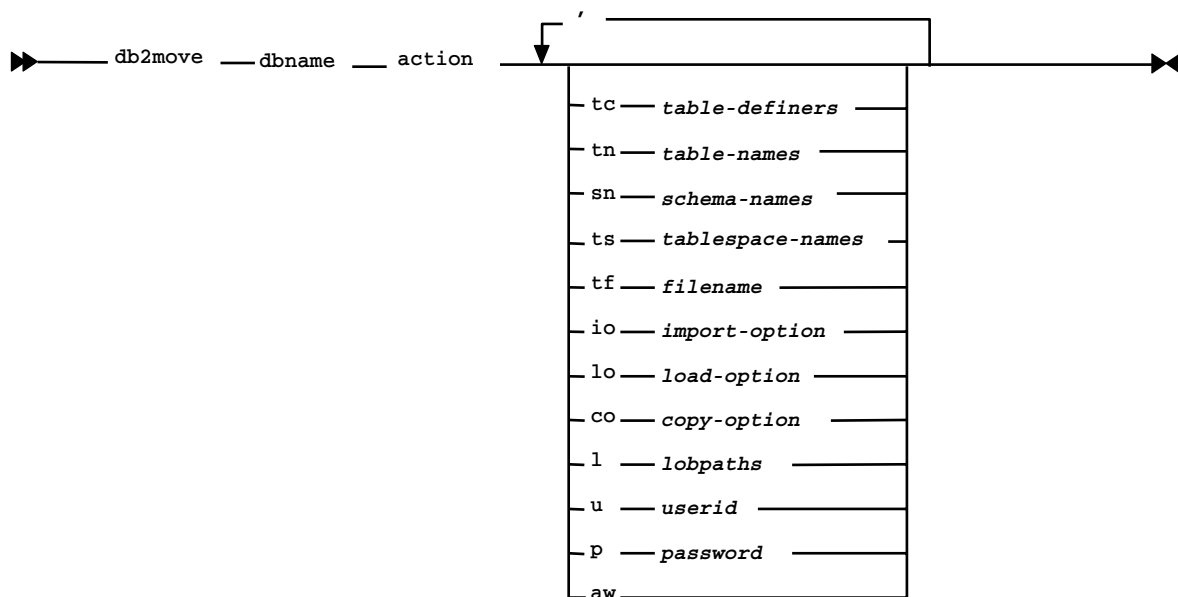
**Purpose** —

**Details** —

**Additional information** —

**Transition statement** — Next, the db2move tool syntax.

## db2move tool syntax



© Copyright IBM Corporation 2007

Figure 6-32. db2move tool syntax

CF238.3

### Notes:

#### db2move tool command parameters:

**dbname** Name of the database.

**action** Must be one of:

- EXPORT** Exports all tables that meet the filtering criteria in options. If no options are specified, exports all the tables. Internal staging information is stored in the **db2move.lst** file.
- IMPORT** Imports all tables listed in the internal staging file **db2move.lst**. Use the **-io** option for IMPORT specific actions.
- LOAD** Loads all tables listed in the internal staging file **db2move.lst**. Use the **-lo** option for LOAD specific actions.

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>COPY</b> | Duplicates a schema(s) into a target database. Use the -sn option to specify one or more schemas. See the -co option for COPY specific options. Use the -tn or -tf option to filter tables in LOAD_ONLY mode. See below for a list of files that are generated during each action.                                                                                                                                                                                                         |
| <b>-tc</b>  | table-definers. The default is all definers.<br>This is an EXPORT action only. If specified, only those tables created by the definers listed with this option are exported. If not specified, the default is to use all definers.                                                                                                                                                                                                                                                         |
| <b>-tn</b>  | table-names. The default is all user tables.<br>This is an EXPORT or COPY action only. If specified, only those tables whose names match exactly those in the specified string are exported or copied. If not specified, the default is to use all user tables.                                                                                                                                                                                                                            |
| <b>-ts</b>  | tablespace-names. The default is all table spaces.<br>This is an EXPORT action only. If this option is specified, only those tables that reside in the specified table space will be exported. If the asterisk wildcard character (*) is used in the table space name, it will be changed to a percent sign (%) and the table name (with percent sign) will be used in the LIKE predicate in the WHERE clause. If the -ts option is not specified, the default is to use all table spaces. |
| <b>-tf</b>  | filename<br>This is an EXPORT or COPY action only. If specified, only the tables listed in the given file will be exported or copied. The tables should be listed one per line, and each table should be fully qualified.                                                                                                                                                                                                                                                                  |
| <b>-io</b>  | import-option. The default is REPLACE_CREATE.<br>Valid options are: INSERT, INSERT_UPDATE, REPLACE, CREATE, and REPLACE_CREATE.                                                                                                                                                                                                                                                                                                                                                            |
| <b>-lo</b>  | load-option. The default is INSERT.<br>Valid options are: INSERT and REPLACE.                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>-co</b>  | When the db2move action is COPY, the following -co follow-on options will be available:<br><br><pre> "TARGET_DB &lt;db name&gt; [USER &lt;userid&gt; USING &lt;password&gt;]" "MODE" DDL_AND_LOAD LOAD_ONLY "SCHEMA_MAP" "NONRECOVERABLE" "OWNER" "TABLESPACE_MAP" </pre>                                                                                                                                                                                                                  |

- l** lobpaths.  
For IMPORT and EXPORT, if this option is specified, it will be also used for XML paths. The default is the current directory.
- u** userid.  
The default is the logged on user ID. Both user ID and password are optional.
- p** Password.  
The default is the logged on password. Both user ID and password are optional.
- aw** Allow Warnings.  
When '-aw' is not specified, tables that experience warnings during export are not included in the db2move.lst file (although that table's .ixf file and .msg file are still generated).

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —** Certification objective: 5F Ability to use DB2Move, DB2Look and DB2Batch.

**Transition statement —** Now let's look at some db2move tool examples.

## db2move tool examples

- 
- To export all tables in the SAMPLE database (using default values for all options), issue:  
`db2move sample export`
  - To export all tables created by userid1 or user IDs LIKE us%rid2, and with the name tname1 or table names LIKE %tname2, issue:  
`db2move sample export -tc userid1,us*rid2 -tn  
tname1,*tname2`
  - To import all tables in the SAMPLE database (LOB paths D:\LOBPATH1 and C:\LOBPATH2 are to be searched for LOB files; this example is applicable to Windows operating systems only), issue:  
`db2move sample import -l D:\LOBPATH1,C:\LOBPATH2`
  - To load all tables in the SAMPLE database (/home/userid/lobpath subdirectory and the tmp subdirectory are to be searched for LOB files; this example is applicable to Linux and UNIX-based systems only), issue:  
`db2move sample load -l /home/userid/lobpath,/tmp`

© Copyright IBM Corporation 2007

Figure 6-33. db2move tool examples

CF238.3

### Notes:

#### Usage note:

Loading data into tables containing XML columns is not supported. The workaround is to manually issue the IMPORT or EXPORT commands, or use the db2move -Export and db2move -Import behavior. If these tables also contain generated always identity columns, data cannot be imported into the tables.



***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Let's look at the db2look tool.

## Overview of the db2look tool

- 
- Generates a report on the statistics in the database
  - Extracts the DDL statements for the creation of tables, indexes, views, and so on, that are needed to generate a CLP script file used to recreate database objects
  - Extracts statistics that are stored in a script
    - Can be used to update the statistics
    - Model a development database to simulate a production database by updating the catalog statistics

© Copyright IBM Corporation 2007

Figure 6-34. Overview of the db2look tool

CF238.3

### Notes:

This tool extracts the required Data Definition Language (DDL) statements to reproduce the database objects of a production database on a test database. The db2look command generates the DDL statements by object type.

This tool can generate the required UPDATE statements used to replicate the statistics on the objects in a test database. It can also be used to generate the UPDATE DATABASE CONFIGURATION and UPDATE DATABASE MANAGER CONFIGURATION commands and the db2set commands so that query optimizer-related configuration parameters and registry variables on the test database match those of the production database.

It is often advantageous to have a test system contain a subset of the production system's data. However, access plans selected for such a test system are not necessarily the same as those that would be selected for the production system. Both the catalog statistics and the configuration parameters for the test system must be updated to match those of the production system. Using this tool makes it possible to create a test database where access plans are similar to those that would be used on the production system.

You should check the DDL statements generated by the db2look command since they might not exactly reproduce all characteristics of the original SQL objects. For table spaces on partitioned database environments, DDL might not be complete if some database partitions are not active. Make sure all database partitions are active using the ACTIVATE command.

***Instructor notes:***

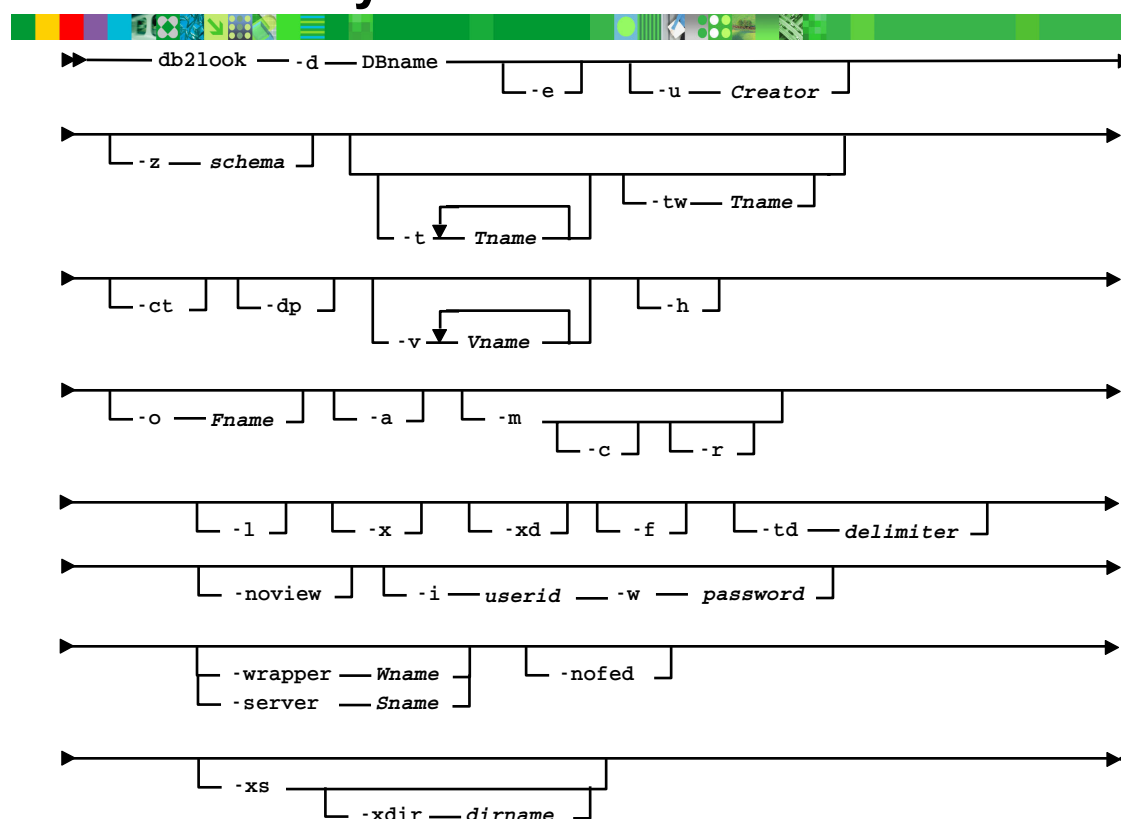
**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Now, the db2look tool syntax.

## db2look tool syntax



© Copyright IBM Corporation 2007

Figure 6-35. db2look tool syntax

CF238.3

### Notes:

db2look command parameters:

- d** DBname  
Alias name of the production database that is to be queried. DBname can be the name of a DB2 Database for Linux, UNIX, and Windows or DB2 Version 9.1 for z/OS (DB2 for z/OS) database.
- e** Extract DDL statements for database objects.
- u** Creator - Creator ID.  
Limits output to objects with this creator ID.
- z** schema -Schema name.  
Limits output to objects with this schema name.
- t** Tname1 Tname2... TnameN - Table name list.  
Limits the output to particular tables in the table list. The maximum number of tables is 30. Table names are separated by a blank space.

- tw** Tname  
Generates DDL for table names that match the pattern criteria specified by Tname. Also generates the DDL for all dependent objects of all returned tables.
- ct** Generate DDL by object creation time. Generating DDL by object creation time will not guarantee that all the object DDLs will be displayed in correct dependency order.
- dp** Generate DROP statement before CREATE statement. The DROP statement might not work if there is an object that depends on the dropped object.
- v** Vname1 Vname2... VnameN  
Generates DDL for the specified views. The maximum number of views is 30.
- h** Display help information.
- o** Fname  
If using LaTeX format, write the output to filename .tex. If using plain text format, write the output to filename.txt. Otherwise, write the output to filename.sql.
- a** When this option is specified the output is not limited to the objects created under a particular creator ID. All objects, including inoperative objects, created by all users are considered.
- m** Generates the required UPDATE statements to replicate the statistics on tables, statistical views, columns and indexes.
- c** When this option is specified in conjunction with the -m option, the db2look command does not generate COMMIT, CONNECT and CONNECT RESET statements. The default action is to generate these statements.
  - r** When this option is specified in conjunction with the -m option, the db2look command does not generate the RUNSTATS command. The default action is to generate the RUNSTATS command.
- l** If this option is specified, then the db2look command will generate DDL for user defined table spaces, database partition groups and buffer pools.
- x** If this option is specified, the db2look command will generate authorization DDL (GRANT statement, for example).
- xd** If this option is specified, the db2look command will generate all authorization DDL including authorization DDL for objects whose authorizations were granted by SYSIBM at object creation time.

---

|                 |                                                                                                                                                                        |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>-f</b>       | Use this option to extract the configuration parameters and registry variables that affect the query optimizer.                                                        |
| <b>-td</b>      | delimiter<br>Specifies the statement delimiter for SQL statements generated by the db2look command. If this option is not specified, the default is the semicolon (;). |
| <b>-noview</b>  | If this option is specified, CREATE VIEW DDL statements will not be extracted.                                                                                         |
| <b>-i</b>       | userid<br>Use this option when working with a remote database.                                                                                                         |
| <b>-w</b>       | password<br>Used with the -i option, this parameter allows the user to run the db2look command against a database that resides on a remote system.                     |
| <b>-wrapper</b> | Wname<br>Generates DDL statements for federated objects that apply to this wrapper.                                                                                    |
| <b>-server</b>  | Sname<br>Generates DDL statements for federated objects that apply to this server.                                                                                     |
| <b>-nofed</b>   | Specifies that no federated DDL statements will be generated.                                                                                                          |
| <b>-xs</b>      | Exports all files necessary to register XML schemas and DTDs at the target database, and generates appropriate commands for registering them.                          |
| <b>-xdir</b>    | dirname<br>Places exported XML-related files into the given path. If this option is not specified, all XML-related files will be exported into the current directory.  |

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —** Certification objective: 5F Ability to use DB2Move, DB2Look and DB2Batch.

**Transition statement —** Let's look at some db2look examples.



## db2look tool examples

- Generate the DDL statements for objects created by user *walid* in database DEPARTMENT. The **db2look** output is sent to file db2look.sql:  
`db2look -d department -u walid -e -o db2look.sql`
- Generate the DDL statements for objects that have schema name *ianhe*, created by user *walid*, in database DEPARTMENT. The **db2look** output is sent to file db2look.sql:  
`db2look -d department -u walid -z ianhe -e -o db2look.sql`
- Generate the UPDATE statements to replicate the statistics for the database objects created by user *walid* in database DEPARTMENT. The output is sent to file db2look.sql:  
`db2look -d department -u walid -m -o db2look.sql`
- Generate both the DDL statements for the objects created by user *walid* and the UPDATE statements to replicate the statistics on the database objects created by the same user. The **db2look** output is sent to file db2look.sql:  
`db2look -d department -u walid -e -m -o db2look.sql`

© Copyright IBM Corporation 2007

Figure 6-36. db2look tool examples

CF238.3

### Notes:

#### Usage note:

On Windows operating systems, the db2look command must be run from a DB2 command window.

More examples:

Generate the DDL statements for all objects (federated and non-federated) in the federated database FEDDEPART. For federated DDL statements, only those that apply to the specified wrapper, FEDWRAP, are generated. The db2look output is sent to standard output:

```
db2look -d feddepart -e -wrapper fedwrap
```

Generate a script file that includes only non-federated DDL statements. The following system command can be run against a federated database (FEDDEPART) and yet only produce output like that found when run against a database which is not federated. The db2look output is sent to a file out.sql:

```
db2look -d feddepart -e -nofed -o out
```

Generate the DDL statements for objects that have schema name walid in the database DEPARTMENT. The files required to register any included XML schemas and DTDs are exported to the current directory. The db2look output is sent to file db2look.sql:

```
db2look -d department -z walid -e -xs -o db2look.sql
```

Generate the DDL statements for objects created by all users in the database DEPARTMENT. The files required to register any included XML schemas and DTDs are exported to directory /home/ofcr/ofcr/. The db2look output is sent to standard output:

```
db2look -d department -a -e -xs -xdir /home/ofcr/ofcr/
```

Example output from db2look:

```
db2look -d sample -z -e db2user1 -t department -o sample.tex

-----
-- DDL Statements for table "DB2USER1"."DEPARTMENT"
-----

CREATE TABLE "DB2USER1"."DEPARTMENT"  (
    "DEPTNO" CHAR(3) NOT NULL ,
    "DEPTNAME" VARCHAR(36) NOT NULL ,
    "MGRNO" CHAR(6) ,
    "ADMRDEPT" CHAR(3) NOT NULL ,
    "LOCATION" CHAR(16) )
IN "USERSPACE1" ;

-- DDL Statements for indexes on Table "DB2USER1"."DEPARTMENT"

CREATE INDEX "DB2USER1"."XDEPT2" ON "DB2USER1"."DEPARTMENT"
("MGRNO" ASC)
ALLOW REVERSE SCANS;

-- DDL Statements for indexes on Table "DB2USER1"."DEPARTMENT"

CREATE INDEX "DB2USER1"."XDEPT3" ON "DB2USER1"."DEPARTMENT"
("ADMRDEPT" ASC)
ALLOW REVERSE SCANS;

-- DDL Statements for primary key on Table "DB2USER1"."DEPARTMENT"

ALTER TABLE "DB2USER1"."DEPARTMENT"
ADD CONSTRAINT "PK_DEPARTMENT" PRIMARY KEY
("DEPTNO");

-- DDL Statements for aliases based on Table "DB2USER1"."DEPARTMENT"

CREATE ALIAS "DB2USER1"."DEPT" FOR "DB2USER1"."DEPARTMENT";
```

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**

## Unit summary

Having completed this unit, you should be able to:

- Review the INSERT statement and recognize its limitations
- Review differences between IMPORT and LOAD
- Learn the EXPORT, IMPORT, and LOAD syntax
- Explore the use of Exception Tables and Dump-Files
- Distinguish the Table States:
  - Load pending, Set Integrity Pending
- Learn why and how to use the SET INTEGRITY command
- Explore use of the **db2move** and **db2look** commands

© Copyright IBM Corporation 2007

Figure 6-37. Unit summary

CF238.3

### Notes:

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —**

# Unit 7. Backup and recovery

## Estimated time

01:30

## What this unit is about

This unit provides you with the information necessary to develop a recovery strategy to support your installation's business requirements. You can use the information in this unit, as well as the product documentation, to plan, define, and implement a recovery strategy.

## What you should be able to do

After completing this unit, you should be able to:

- Describe the major principles and methods for backup and recovery
- State the three types of recovery used by DB2
- Explain the importance of logging for backup and recovery
- Describe how data logging takes place, including circular logging and archival logging
- Use the BACKUP, RESTORE, and ROLLFORWARD commands
- Perform a table space backup and recovery
- Restore a database to the end of logs or to a point-in-time
- Discuss the configuration parameters and the recovery history file and use these to handle various backup and recovery scenarios

## How you will check your progress

Accountability:

- Checkpoint
- Lab exercises on backup and recovery

## References

SC09-4831-00 *Data Recovery and High Availability Guide and Reference*

## Unit objectives

After completing this unit, you should be able to:

- Describe the major principles and methods for backup and recovery
- State the three types of recovery used by DB2
- Explain the importance of logging for backup and recovery
- Describe how data logging takes place, including circular logging and archival logging
- Use the BACKUP, RESTORE, and ROLLFORWARD commands
- Perform a table space backup and recovery
- Restore a database to the end of logs or to a point-in-time
- Discuss the configuration parameters and the recovery history file and use these to handle various backup and recovery scenarios

© Copyright IBM Corporation 2007

Figure 7-1. Unit objectives

CF238.3

### Notes:



***Instructor notes:***

**Purpose** — Review the unit objectives.

**Details** — Identify what is to be addressed in this unit.

**Transition Statement** — Now, let's look at DB2 logging, backup, and recovery. Note the emphasis that we will place here. This is Job #1.



## 7.1 Introduction to backup and recovery

### Instructor Topic Introduction

Note that the unit title and the topic title deal with both **backup** and **recovery**. Backup is one of the primary means of preparing for recovery. In former versions of this course, the unit was titled *recovery* and thus the retitling is deliberate to lay more emphasis on the preparatory phase of *backup* as the precursor to future recovery.

**What students will do** — Students will listen to a brief introduction regarding the need for recovery planning in DB2 installations.

**How students will do it** — Initially listen to the reasons for backup (Job #1) and recovery, but then engage in some class level discussion that will bring home the real need for backup and recovery.

**What students will learn** — Various scenarios requiring recovery and some degree of the level of recovery provided by DB2.

**How this will help students on their job** — Backup/recovery planning is an integral part of a Database Administrator's job. Without knowledge of the types of problems that may be encountered, the need for recovery planning may be underestimated.

## Backup and recovery



- Backup is *Job #1* for the system administrator — without it you cannot guarantee that your data will be available in the future
- *Backup is for Recovery*
- The system administrator is ultimately responsible for:
  - Establishing a backup policy
  - Enforcing the policy and verifying that the policy has been carried out correctly
  - Testing recovery using the products of backup
- Backup involves both the database and any support files that are not part of the database itself:
  - Database objects
    - Table spaces
    - Log files
  - Some non-database files

© Copyright IBM Corporation 2007

Figure 7-2. Backup and recovery

CF238.3

### Notes:

Backup is done for precisely one purpose: database recovery *when* it is needed.

You should note the use of the word *when* rather than *if*. We will see that recovery is needed for normal functioning of the system (*crash recovery*) and for abnormal functioning of the database system (*version recovery/restore* and *rollforward*).

You should also note that you as system administrator are the primary focus of the recovery process. Hence you cannot hide behind someone else not doing their job correctly. You are ultimately responsible for establishing, enforcing, and testing any recovery policy and process. You need to review the steps that will be undertaken by others to ensure that they are carried out as laid down in policy so that when the time comes you will have the files (generally tapes) needed to perform recovery.

A company that loses its data is seriously in danger of failing.

Some data that must be backed up is not backed up as part of the database backup. You need to become aware of the complete picture and take responsibility for all of this.

And, it *is* your Job #1.

## Instructor notes:

**Purpose** — Ensure that your students are aware of the importance of this role.

**Details** — Tell any stories from your own experience that bring out issues here. Personalizing it can be important as the student needs to personalize this too. As course developer and instructor, I generally describe situations such as the following:

1. **Backup tapes not kept offsite:** “Every night backup tapes were made and given to the mail room. The process was to take these tapes to a branch of the bank several miles away and leave them there for several months, before returning them to the computer room for reuse. I visited the bank branch vault to verify that the process was being carried out as documented. ‘Oh, they stopped doing that several years ago. There are no tapes here now.’ Apparently the mail delivery people had changed the offsite storage process. They were leaving the tapes in the mail room where they had some spare space. Of course, the mail room was next to the computer room. A good fire or an explosion would have taken out the computer system and all backup tapes.”
2. **Real reason for backup not understood:** “I was asked to sit through repairs after a disk failure. The replacement of the failed disk drive was uneventful, but the company did this under a ‘time & materials’ contract rather than a regular maintenance contract. When told that he could now use his backup tapes to restore his files, the DP manager cries. His tape drive had failed two years previously and since he really did not understand why he was doing backup, he had not repaired it — after all, he had never had to use tapes to restore his system before. Now he understood, but it was too late.”
3. **Almost everything was backed up:** Something was forgotten. I have been at sites where configuration files were lost, ... recovery site had a different type of tape drive (they had changed the types of tapes at the main site, but not at the recovery site, ... This is the reason for thinking about this continually for when the student as administrator *will have* a problem, sometime in the future.

Stories such as these — and you are better to use your own, where you are one of the participants — will get the main ideas across:

- Regular backup
- Offsite storage
- Verification that the *whole* process is carried as intended
- Thinking ahead to anticipate *all* that could go wrong

You might easily start a short classroom discussion here, and thereby hear some stories from your own students that you can use on future occasions.

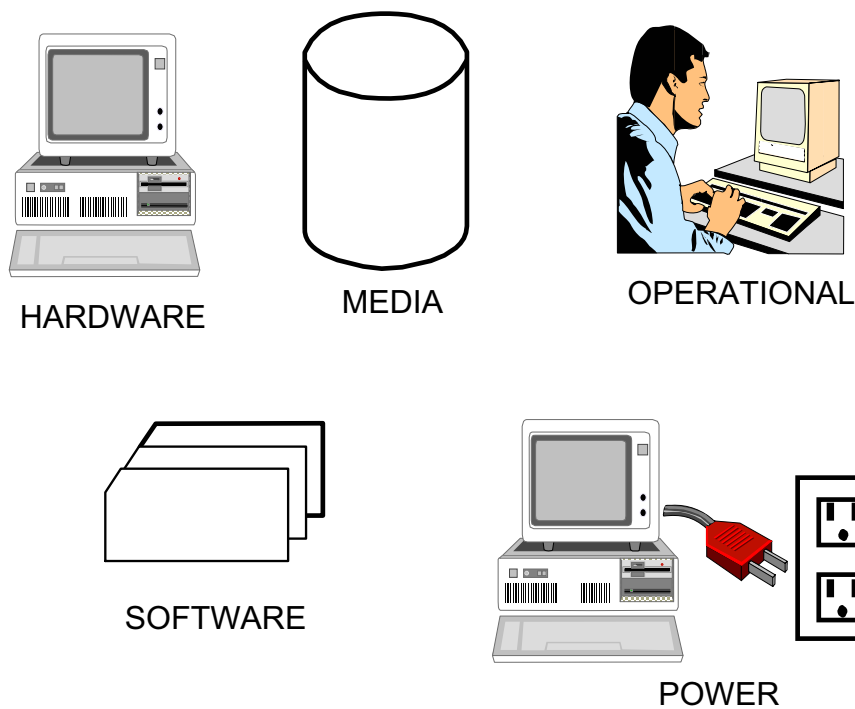
If any part of the process is not carried out, management will look to the system administrator to *fix* the problem — he/she is the *professional* who can do this, of course. Unlike the medical doctor, you are not allowed to have a percentage of failures. “You have to be successful on 100% of occasions that recovery is performed.”

## Additional information —

**Transition statement** — Let’s now look at the various types of failures that *can* occur.



## Types of failure



© Copyright IBM Corporation 2007

Figure 7-3. Types of failure

CF238.3

### Notes:

DB2 provides for recovering from a variety of data processing problems.

The degree of recovery depends on the type of recovery required.

For example:

- Recovery from a program logic flaw can only be made to a point before the erroneous program began.
- Recovery from a power failure can restore the database to a consistent state up to the last committed unit of work.

In order to support recovery from all the potential problems listed, the database administrator needs to take full advantage of all the recovery features of DB2. However:

- An installation must define its recovery needs.

Those needs may not require use of all the recovery features available.

## ***Instructor notes:***

**Purpose** — Identify the various types of problems that would require recovery of data.

**Details** — Use this visual to emphasize that the type of failure will influence the point to which a recovery can be made. For example, application logic errors cannot be isolated via DB2 recovery mechanisms to a given application. If application A makes changes to data while applications B through Z are also making changes to data, it is not possible to **undo** only the changes made by application A.

**Additional Information** — At times students will ask “Why isn't it possible to undo only a single application's logic errors?” The answer to this question is based on the underlying architecture of the DB2 database manager. Supporting such journaling would require a different, more costly logging structure. The architecture of the log is *optimistic*, that is, COMMIT is normally the appropriate action after changes are made. The need to ROLLBACK a given application's changes should not be introduced in a production environment. Appropriate levels of testing should be done before such applications are promoted to production. In other words, the cost of supporting application journaling was not considered worth the benefit for what should be an exception process.

There is an additional charge tool, DB2 Recovery Expert, that will allow you to selectively back out committed transactions.

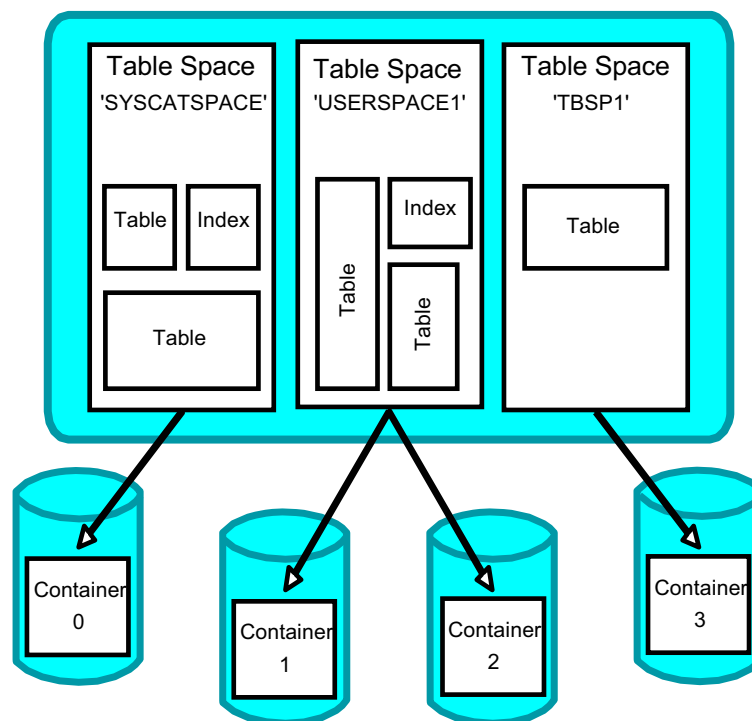
Redirected restore, point-in-time table space recovery, subset table space restore, and recovery of history file are addressed in the CF49 *Advanced Recovery Workshop* class. You should not allow yourself to be pulled into long discussions on these issues.

**Transition Statement** — In addition to the type of problem influencing the degree of recovery possible, the placement of data in table spaces will impact recovery strategies.



# Database objects

## Database



© Copyright IBM Corporation 2007

Figure 7-4. Database objects

CF238.3

### Notes:

The data in a database is placed into table spaces.

A table space is a storage model that provides a level of indirection between a database and the tables stored within that database.

Table spaces allow you to assign the location of database and table data directly onto containers. (A container can be a directory name, a device name, or a file name.)

A single table space can span several containers. For performance reasons, each container should use a different disk.

The architecture shown in the diagram has impacts on recovery processes. Two of these impacts are:

- The data for different tables might share the same table space. A restore and roll-forward recovery of the table space will restore and recover *all* tables in that table space.

- While it is a good practice to let related tables share a table space, you should consider storing unrelated tables separately in order to minimize recovery processing time when restoring data at the table space level.

The DB2 backup utility allows for backing up at the database and table space levels. Export or *db2move* could be used to back up individual tables.



## Note

### Dropped Table Recovery

**Q:** *How do I ensure that critical tables in my DB2 database are recoverable in case I inadvertently drop them?*

We will see later that DB2 allows you to recover a database's data using a database RESTORE operation. You can use the RESTORE operation in conjunction with the ROLLFORWARD operation to restore the database to a point before the table was dropped, but restoring the entire database can be very time consuming. More importantly, during this database-level operation, you cannot access your database, and furthermore there is always the finite chance that your backup media are defective.

There is a *dropped table recovery option* available for table spaces to solve this problem. This option is turned on by default when tables are created using the command line, but it is an option that must be specifically selected if you create a table space with the GUI.

To make sure a given name space has table recovery enabled, query the DROP\_RECOVERY column of the SYSCAT.TABLESPACES catalog table:

```
db2 => SELECT tbspace, drop_recovery FROM syscat.tablespace
```

To enable the DROPPED TABLE RECOVERY option after you have created a table space, use:

```
ALTER TABLESPACE mytablespace DROPPED TABLE RECOVERY on
```

If you drop a table, and don't remember its name, you can use:

```
db2 => LIST HISTORY DROPPED TABLE ALL FOR mydatabase
```

We do not have space here to describe the whole process for RECOVER and ROLLFORWARD needed to have restore this table. This type of recovery and rollforward is covered in *CF49 DB2 Advanced Recovery Workshop*.

Obviously you need to think ahead. It is too late to turn on DROPPED TABLE RECOVER after the table has already been dropped.

***Instructor notes:***

**Purpose** — Briefly introduce the concept regarding data placement and recovery.

**Details** — Additional considerations regarding table space recovery will be addressed later in the module. However, it is beneficial to highlight the two impacts of table space design on recovery that are in the student notes.

The comments on DROPPED TABLE RECOVERY are not intended to provide a solution to the problem of dropping a table, but rather are provided to provide an illustration of things that need to be known and taken into consideration when planning ahead. Do not attempt to go into greater detail here — it is just a *note* or *fyi* to open up the topic for *future* thinking and planning.

**Additional information —**

**Transition Statement** — There are several methods of recovery, each requiring different degrees of planning by the database administrator. The next topic addresses these methods.

## Forms of backup



- Table space backup
  - Full / incremental backup
  - Online / offline backup
- Log backup
  - Circular / archival logging
  - Single / mirror logging
- Alternate disaster and recovery approaches
  - High availability disaster recovery (HADR)
  - Data replication
- Other issues — towards a complete backup and recovery plan
  - Disk / tape / transmission to a remote site — offsite storage
  - Storage management systems (TSM, XBSA, ...)
  - Other critical files and hardware requirements
  - Backup and recovery sites
  - Network considerations

© Copyright IBM Corporation 2007

Figure 7-5. Forms of backup

CF238.3

### Notes:

Backup is not monolithic.

We need to learn about the types of table space backup and the types of log backup.

We need to understand the place and role of alternative and complimentary disaster and recovery approaches (but they are not discussed in this course).

And, finally, we need to understand the related technology, including the role of storage management systems such as Tivoli Storage Manager (TSM) or other Backup Services APIs (XBSA) and work towards a complete backup and recovery plan.

***Instructor notes:***

**Purpose** — Make sure that the student understands the *big picture* that goes beyond what we are learning in this unit.

**Details** —

**Additional information** —

**Transition statement** — There are several methods of recovery, each requiring different degrees of planning by the database administrator. The next topic addresses these methods.



## 7.2 Methods of recovery and logging

### Instructor Topic Introduction

**What students will do** — List the three major methods of recovering DB2 data.

**How students will do it** — Listen to lecture and complete lab assignments after completion of the unit.

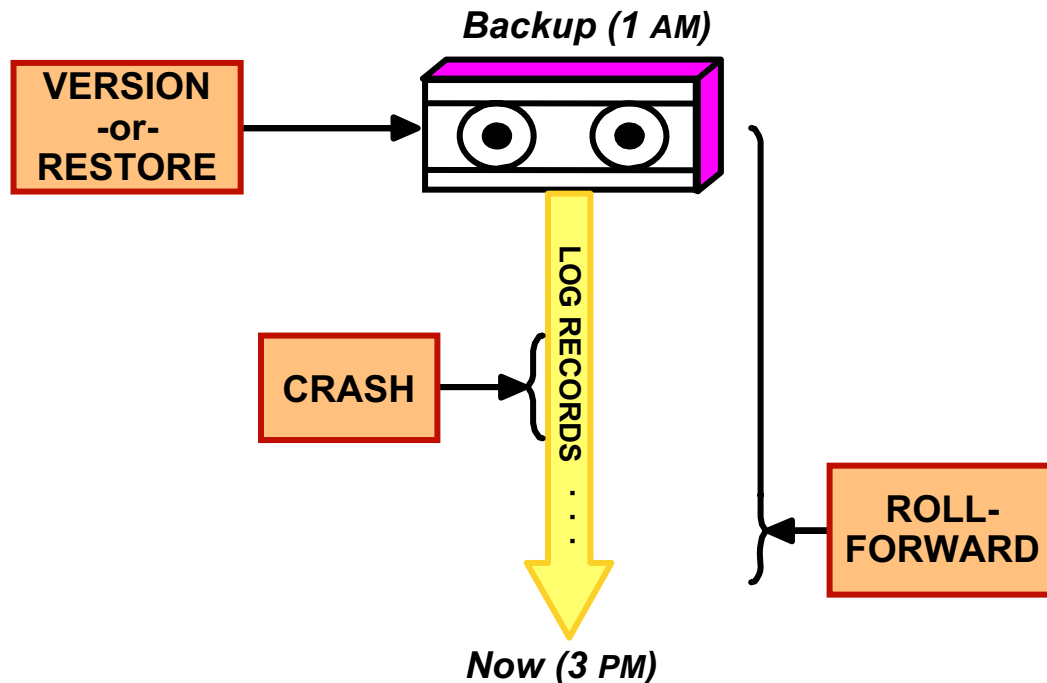
**What students will learn** — Recovery methods supported by DB2 and the planning considerations and implementation requirements of the database administrator.

**How this will help students on their job** — Each installation has different recovery needs. The discussion on the methods of recovery will enable a particular administrator to employ the recovery methods that meet the installation's requirements.

This topic discusses the different recovery methods that can be used in the event that there is a problem involving the database. Also discussed are considerations and decisions that will assist in determining the recovery method best suited to your business environment. For example, the logging method required to support a given recovery method.

Each recovery method is described along with the associated concepts and the commands provided to support these methods.

## Methods of recovery



© Copyright IBM Corporation 2007

Figure 7-6. Methods of recovery

CF238.3

### Notes:

You need to know the strategies available to you to help when there are problems with the database. Typically, you will deal with media and storage problems, power interruptions, and application failures. You need to know that you can back up your database, or individual table spaces, and then rebuild them should they be damaged or corrupted. The rebuilding of these objects is called recovery. There are three types or methods of recovery:

1. Crash
2. Version (or Restore)
3. Roll Forward

### Crash Recovery

Crash Recovery uses the logs to recover from power interrupts or application abends. This type of recovery is normally automated by using the default value for the configuration parameter **autorestart**. If autorestart is not used, manual intervention would be necessary to restart a database requiring crash recovery.



## Version or Restore Recovery

Version or Restore Recovery uses a backup image of the database to replace the current data in the database. This type provides recovery from media, hardware, operational, and software problems, but only to the point that the backup copy was made. Therefore, the more frequent the backups, the more current the recovered database will be.

## Roll Forward Recovery

Roll Forward Recovery uses a backup copy of a database or table spaces to replace the current data and then applies log records to recover changes made after the backup image was created. This type provides recovery from media, hardware, operational, and software problems, and the recovery can be to a point in time or to the last committed unit of work. Roll forward recovery using a recent backup will require fewer logs to be applied than a roll forward using an older backup, so the frequency of backup will still be a consideration for the database administrator.

## ***Instructor notes:***

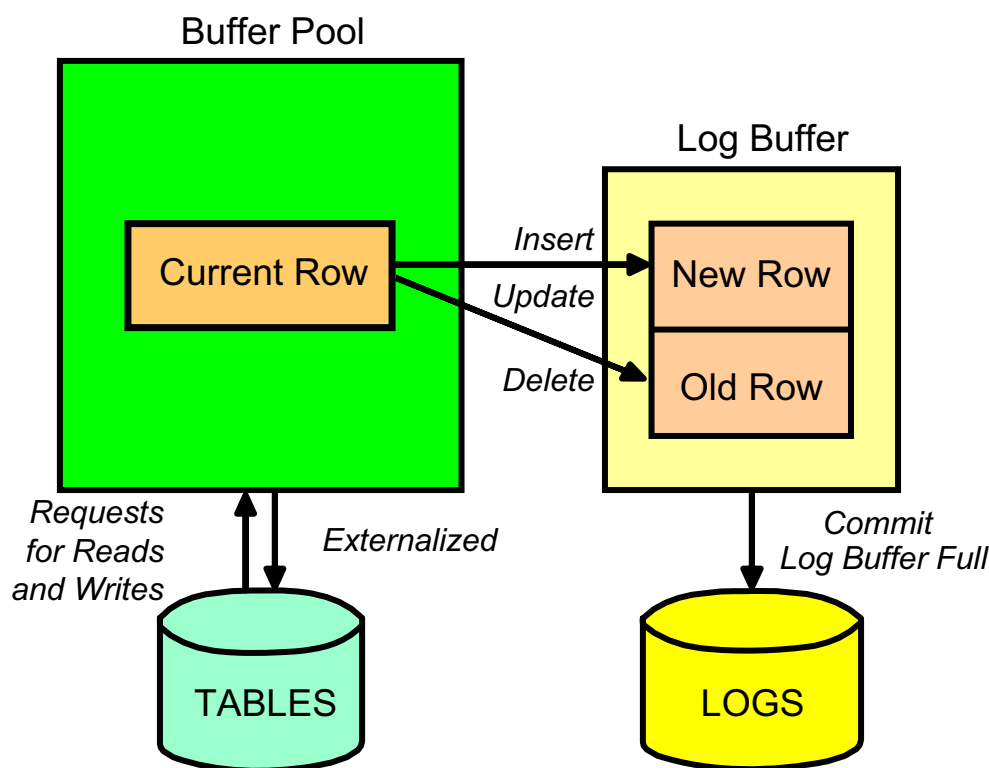
**Purpose** — Define the three methods of recovery.

**Details** — Emphasize that crash recovery is generally left to be automated. Also, highlight that transaction-based installations will generally favor supporting roll forward recovery, but that there are installation-specific requirements that will ultimately dictate the recovery method used.

**Additional information** — Certification objective: 6D Knowledge of crash recovery.

**Transition Statement** — Regardless of whether your installation will require RESTORE or ROLLFORWARD recovery, DB2 requires the use of logs to support CRASH recovery. Let's examine some basic concepts regarding logging.

# Introduction to logging



© Copyright IBM Corporation 2007

Figure 7-7. Introduction to logging

CF238.3

## Notes:

Transaction logging is the process that records each change to a database to permit recovery.

The database manager maintains a log of recent changes made to a database so that the recovery process can restore the database to a consistent state. The log records are written in a log buffer to reflect the activity occurring against data in the buffer pool. The data that is logged includes all column data on an insert or delete, but only from first-changed-column to last-changed-column on an update.

At commit, the log records **MUST** be written from the log buffer to the log files on disk in order to guarantee recoverability. The application issuing the commit will not receive confirmation that the commit has successfully completed until the log buffer has been written.

It is possible that log records will be written to disk before a commit, for example when the log buffer fills up. However, this does not impact the integrity of the system, because the execution of a commit itself is logged. A unit of work that has started and is externalized to

the log file is not considered complete until the commit associated with that unit of work is also externalized to the log file.

The pages in the buffer pool changed by a unit of work that has been committed do not need to be written to disk at commit. The log records contain the information required to recover such changes in the event of a recovery situation. It is desirable to leave pages in the buffer pool if the data they contain is accessed frequently.

If you are logging large object (LOB) data, you have to consider the impact to performance. If you turn logging on for LOB data, your application's performance will deteriorate and you may encounter problems related to the increased size of the log file. If you turn the logging off, your application's performance improves, however its recoverability is sacrificed.

For more information on the merits of logging LOB data, refer to the *Data Recovery and High Availability Guide and Reference*.

DB2 supports two basic types of logging: circular logging and archival logging (log retention logging). Details regarding these two basic types are the subject of subsequent pages.

***Instructor notes:***

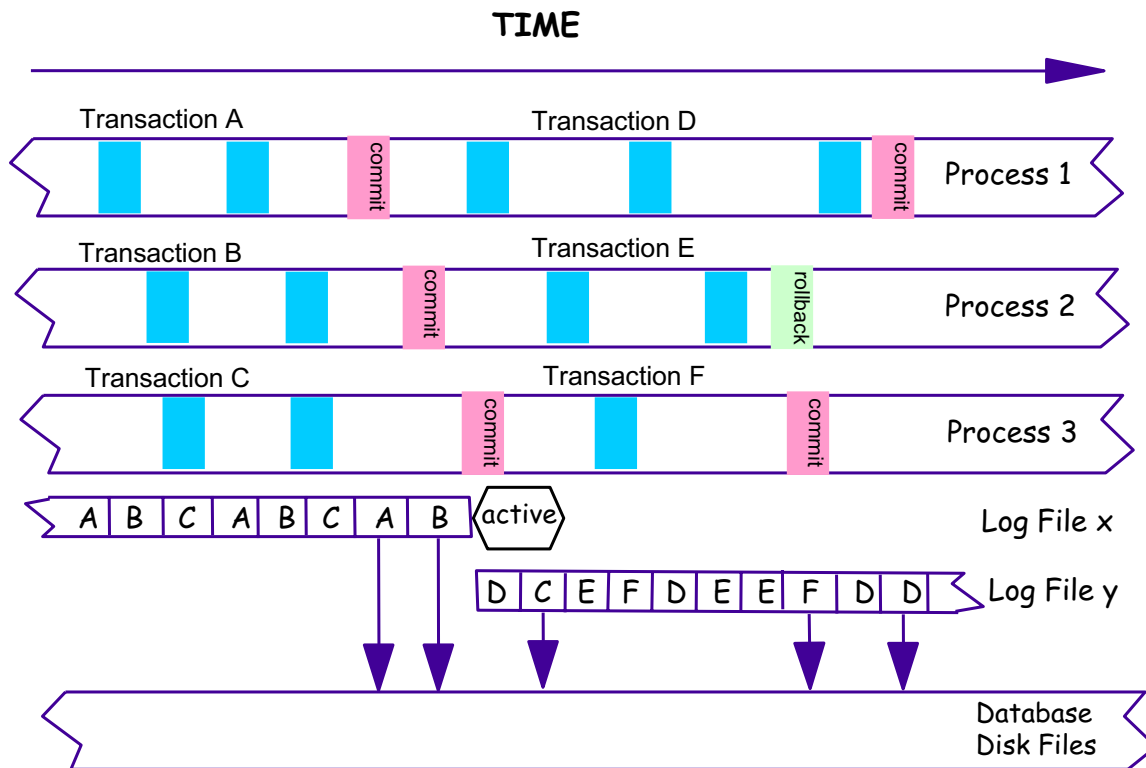
**Purpose** — Provide the students with the basic purpose of logging.

**Details** — Emphasize that commit requires log records associated with the unit of work to be successfully written to disk, while the data pages that have been changed may stay in a buffer. The key is that integrity is ensured by DB2.

**Additional information** — Certification objective: 6C Knowledge of database logging.

**Transition Statement** — The type of logging you use, Circular or Retention, depends on the type of recovery you intend to support. Let's examine these types of logging in greater detail.

## Log file use during concurrent transactions



© Copyright IBM Corporation 2007

Figure 7-8. Log file use during concurrent transactions

CF238.3

### Notes:

The visual shows three transactions initially taking place in parallel: A, B, and C. They happen to be alternating in the changes recorded to the log file — but the actions are independent and hence log file entries can be in any order and with any timing.

Commits take place. In one case there is a rollback.

While this is taking place, other transactions also start and complete. These are indicated with other transaction letters (D, E, F, ...) and could occur as part of the work of the same processes or in separate processes.

The log file itself switches from X to Y at an arbitrary point in time as the first file fills.

Also, at any arbitrary point in time (any vertical time slice across all this activity), the system can fail and all information in memory is lost. That which has been completely written to the disk log files is available for crash recovery. Some transactions are complete (commit or rollback) and some transactions are in midstream.

This is the problem that is solved by crash recovery.

**Instructor notes:**

This visual provides you with the opportunity to walk through some scenarios.

You can either use the visual which has the following coloring:

- Blue — change in one of the transactions
- Pink — commit (there are several)
- Green — rollback (there is just one, it is important to discuss this)

Or, if you have access to a white board and have different color marker pens, you may want to construct this visual in part before using the visual to complete an explanation. Indicate three processes involved in three transactions making changes to pages that are in the buffer pool, and that those changes are written to the log buffer. Suggestion: draw rectangles on the buffer pool area for pages that have been read in from disk, and make green, red, or blue Xs on them to represent changes being made. Every time you make a change to a page in the buffer pool, put the same color X into the log buffer. Then, say, the “blue” transaction commits. That writes a commit into the log buffer, and causes all of the green, red, and blue Xs to be written to a log file on disk. However, note that the green, red, and blue Xs are still in memory in the buffer pool; the commit does not require that the buffer pool externalizes.

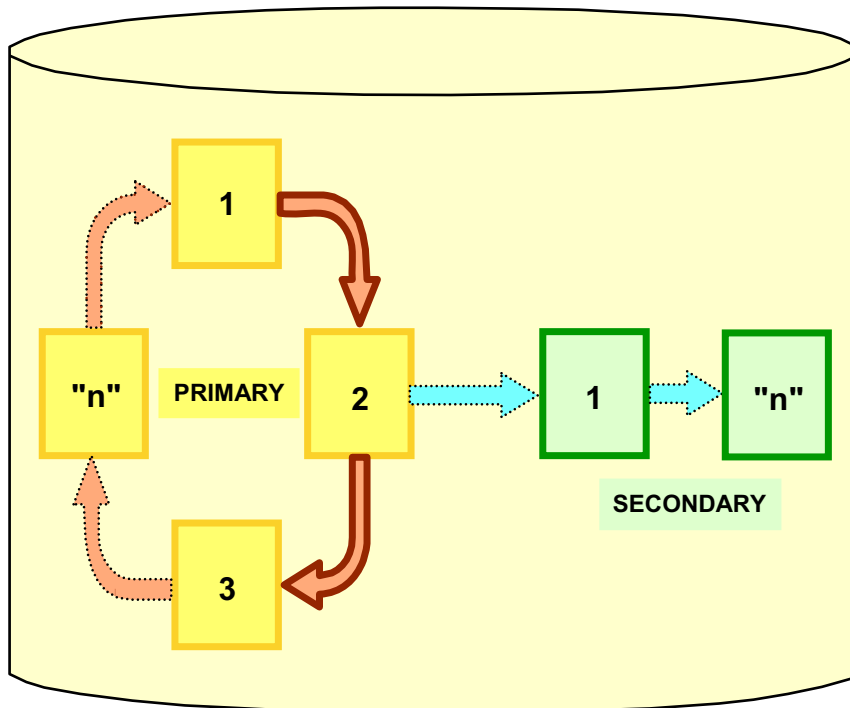
In either case, say, “What happens if a crash happens right now?” — drawing a vertical line across any part of your diagram (this could be repeated a second time at another time, if you wish). Obviously, anything in memory is lost, which means that the changes to the buffer pool pages will be lost (erase the pages in the buffer pool). But the information that has been written to the log files will continue to be available to us.

During crash recovery, DB2 will read the information that is in the log file on disk. DB2 will reapply the green, red, and blue changes (draw the buffer pool pages coming back into the buffer pool, and draw the green, red, and blue Xs onto the pages in the buffer pool). At the end of the log file, DB2 will see that the blue transaction has committed. But DB2 will also see that the green and red transactions have not committed. So, during crash recovery, DB2 will explicitly roll back any transactions that had not yet committed at the time of the crash and log that as a rollback (erase the green and red Xs in the buffer pool, and draw green and red Xs in the log file to represent the rollback). At the end of crash recovery, the committed transaction has been reapplied to the database, and the uncommitted transactions have been rolled back.

**Additional information** — Certification objective: 6C Knowledge of database logging.

**Transition Statement** — Although circular logging provides support for crash and version/restore recovery methods, many installations will require roll forward support. This necessitates retaining the log files.

## Circular logging



© Copyright IBM Corporation 2007

Figure 7-9. Circular logging

CF238.3

### Notes:

Circular logging will use the number of primary log files specified via a configuration parameter. The necessary log information is for in-process transactions. The log files are used in sequence. A log file can be reused when all units of work contained within it are committed or rolled back, and the committed changes are reflected on the disks supporting the database.

If the database manager requests the next log in the sequence and it is not available for reuse, a secondary log file will be allocated. After it is full, the next primary log file is checked for reuse again. If it is still not available, another secondary log file is allocated. This process continues until the primary log file becomes available for reuse or the number of secondary log files permitted for allocation is exceeded.

Primary log files are allocated when the database is created, while secondary log files are allocated as needed. Secondary log files are deallocated once the database manager determines that they are no longer needed. Therefore, the database administrator may elect to use the primary log files for typical processing, but permit the allocation of secondary log files to permit periodic applications that have large units of work. For



example, submission of an IMPORT utility with a large commit count may require the use of secondary log files. Supporting such applications via primary logs would be wasteful of space, since all primary logs, whether used or not, are allocated when the database is activated.

If DB2 cannot continue logging due to a log full condition, database activity will halt until a log file becomes available.

The logpath should be able to contain the sum of the primary and secondary logs. LOGPRIMARY + LOGSECOND must be less than or equal to 256. LOGFILSIZ has a maximum of 262,144 4 KB pages. The total active log file size limit is 256 GB.

Circular logging provides support for crash and version/restore recovery, but does NOT support roll forward recovery.

## ***Instructor notes:***

**Purpose** — Describe circular logging architecture.

**Details** — Emphasize that a log full condition will cause database activity to stop. This is because DB2 uses the logs to provide crash recovery and rollback capability.

Non-committed or non-externalized transactions must be supported by log records to provide these capabilities.

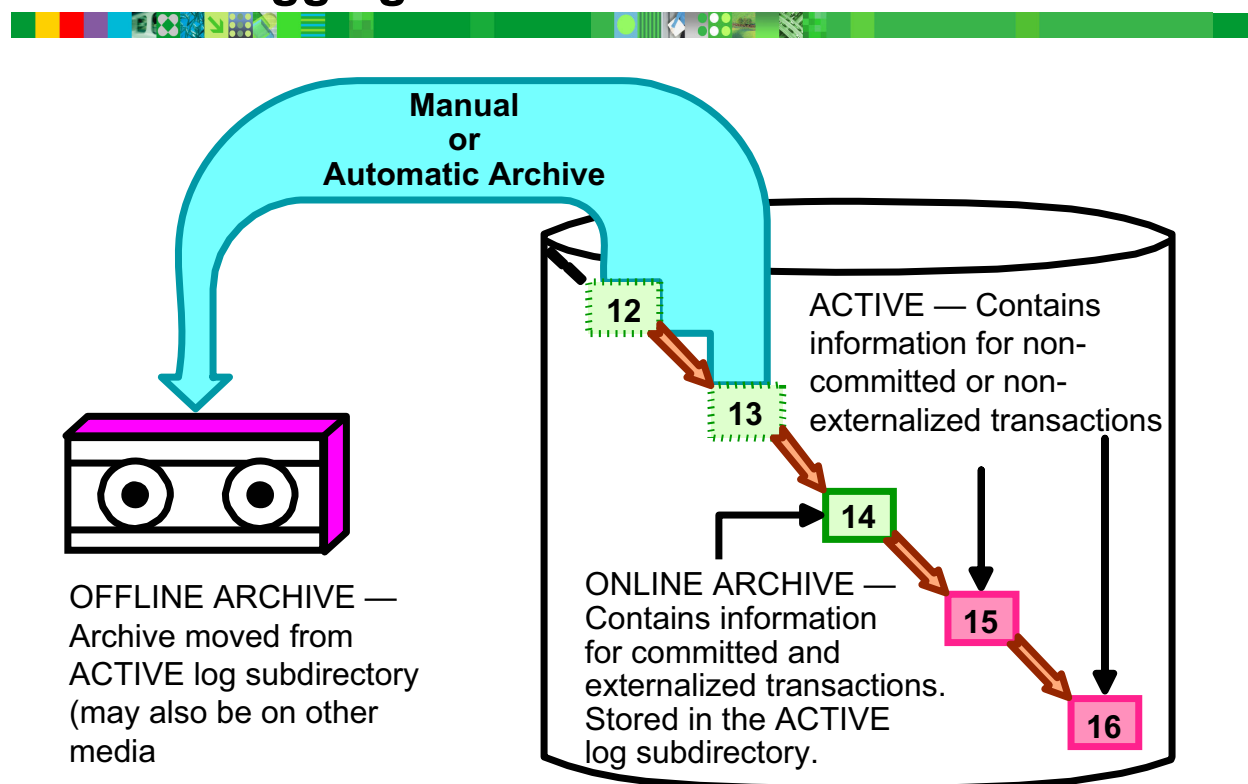
Also, emphasize that circular logging does not support rollforward recovery.

**Additional Information** — Raw devices can be used for DB2 logs. Only one raw device is allowed for the DB2 log. You can define the device over multiple disks at the operating system level. The logsecond parameter is not used. DB2 will not allocate secondary logs. The size of the active log space is the number of 4 KB pages that result from logprimary.

**Additional information** — Certification objective: 6A Ability to perform database-level and table space level BACKUP and RESTORE.

**Transition Statement** — Although circular logging provides support for crash and version/restore recovery methods, many installations will require roll forward support. This necessitates retaining the log files.

## Archival logging



© Copyright IBM Corporation 2007

Figure 7-10. Archival logging

CF238.3

### Notes:

The second type of logging supported by DB2 is archival logging (log retention logging), where log files are not reused.

When a log becomes full, another log file is allocated. Usually, the database administrator will configure several primary log files so that a log file being allocated is not immediately needed for logging. (Allocation is done ahead of the need for the file.) The number of log files allocated when the database is created is specified by the number of primary logs.

This type of logging is enabled through configuration parameters highlighted later in this unit.

If DB2 allocates the sum of primary and secondary logs as defined in the database configuration file and requires additional space for logging, a log full condition will result. This situation can be caused by an application that attempts to process too large a unit of work. It can also be caused by a configuration that allocates too few log files, or log files that are too small to handle the work load.

The logpath (or newlogpath) should be able to contain two times the sum of the primary and secondary logs plus 1. The maximum total log file size limit is 256 GB (that is, the number of log files (LOGPRIMARY + LOGSECOND) multiplied by the size of each logfile in bytes (LOGFILSIZ \* 4096) must be less than 256 GB). LOGPRIMARY + LOGSECOND must be less than or equal to 256. LOGFILSIZ has a maximum of 262,144 4 KB pages.

There are three types of log files associated with log retention logging:

1. Active — these files contain information related to transactions that have not yet committed (or rolled back) work. They also contain information for transactions that have been committed, but whose changes have not yet been written to the database files. (The changes could be in the buffer pool.)

The active log supports crash recovery.

2. Online Archive — these files contain information related to completed transactions that no longer require crash recovery protection. They are termed *online* because they reside in the same subdirectory as the active log files.
3. Offline archive — these files have been moved from the active log file subdirectory. The method of moving these files could be a manual process or a process invoked through a user exit.



#### Note

When using archival (log retention) logging, DB2 will truncate and close the last log file written to free up space when the last application disconnects from the database and the database deactivates. This is a positive feature when the database is to be inactive for some period of time. However, if an installation has a low level of activity and there are short periods where no application will be connected to the database, it will be costly to truncate the last active log and then reallocate primary log files when a new application connects.

The database administrator should consider using the ACTIVATE DATABASE command. This command will keep the database active and prevent log file truncation. However, the administrator must remain aware of the impact on recovery. If the database is truly not being used for an extended period of time, preventing log file truncation will also prevent the most recent log information from being archived. This will make the recovery point for the database to be less than the most recent unit of work if a failure on the log disk occurs. The DEACTIVATE DATABASE command can be specified to allow log file truncation to occur for databases on which the ACTIVATE command has been used.

An additional benefit of the `ACTIVATE DATABASE` command would be that memory is normally allocated at first connect and released when the database deactivates. This normally occurs when the last application disconnects are retained. This can enhance the performance of databases with periods of inactivity.

## ***Instructor Notes:***

**Purpose** — Describe log retention logging.

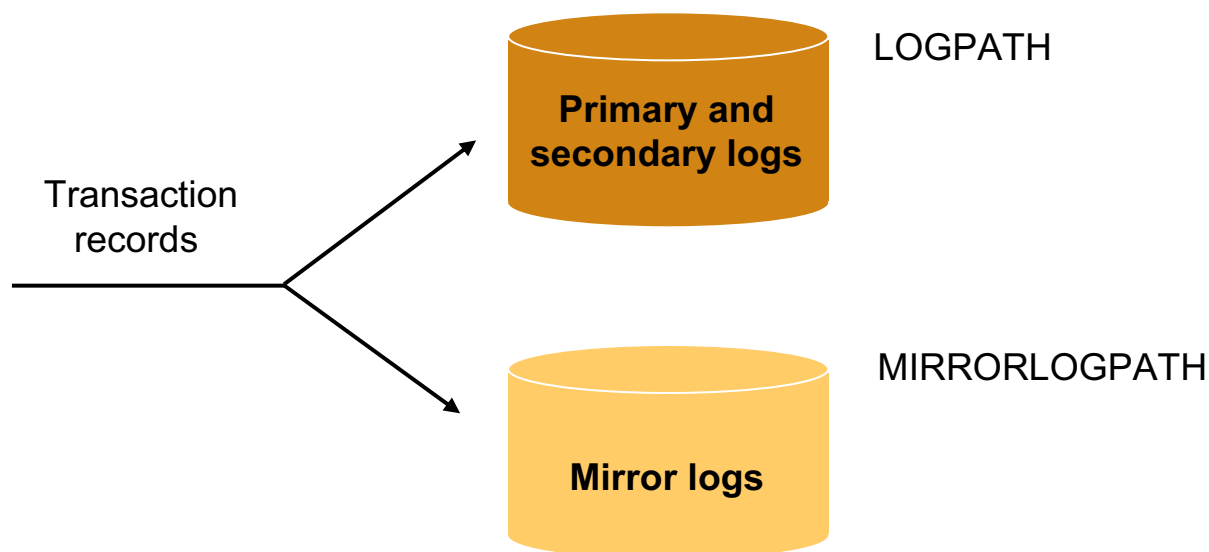
**Details** — Spend some time describing the definitions concerning active, online archive, and archive logging. A key consideration is that active logs are required to support crash recovery and will not be permitted to be removed from the active log path. Ensure that the students understand the concept of moving log files from the active log path to an archive. It is also important to emphasize the benefit and possible negative impact of using the `ACTIVATE DATABASE` command.

**Additional Information** — Infinite active log is a new capability in DB2 UDB V8, but is not addressed in this course (it is addressed in CF49). With infinite active logging, you will never get log full conditions (the active logs can be archived, which would require that the user exit bring the logs back online during a roll back or during crash recovery).

Certification objective: 6A Ability to perform database-level and table space level BACKUP and RESTORE.

**Transition statement** — If there is just one copy of the logs, this would be a single point of failure. With dual or mirrored logging, we avoid this *single* point of failure.

## Mirrored logging



© Copyright IBM Corporation 2007

Figure 7-11. Mirrored logging

CF238.3

### Notes:

Dual or mirrored logging provides a way to maintain mirror copies of both primary and secondary logs — this capability was added to DB2 UDB in v8.1. If either the primary or the secondary log becomes corrupt (but not both), or if the device where either logs is stored becomes unavailable, the database can still be accessed.

Mirrored logs should be kept on different disk drives and preferably on different disk controllers (and, certainly, different RAID controllers). Since log files are written serially, dedicated drives should be preferred as other disk action on the same drive can reduce performance.

Mirrored logging is enabled by setting the MIRRORLOGPATH database configuration parameter to a path where the mirror logs are to be located.

***Instructor notes:***

**Purpose** — Explain the value of mirrored logs.

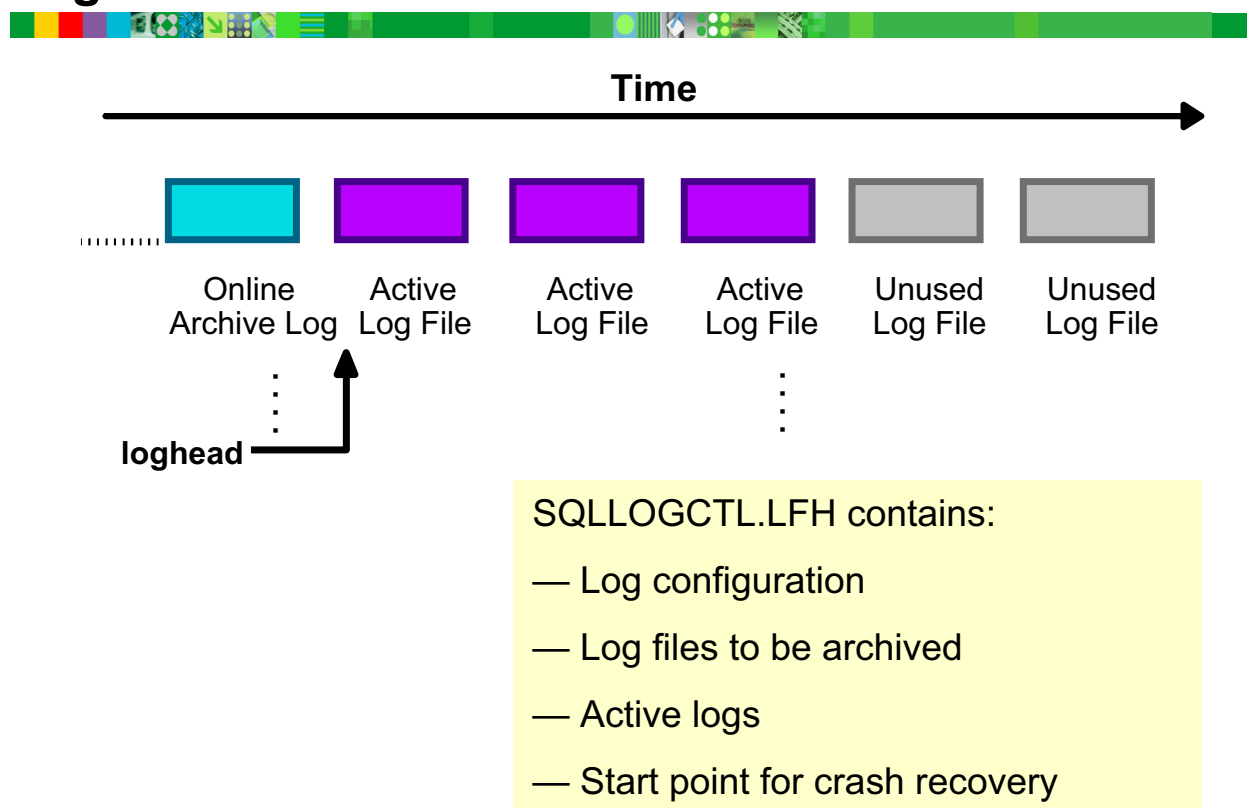
**Details** — Any single point of failure can mean loss of transactions. Loss of any or all of the only copy of log files means that recovery — if needed — will be incomplete.

**Additional information** —

**Transition statement** — Information concerning the logs used for a database can be obtained and analyzed.



## Log file information



© Copyright IBM Corporation 2007

Figure 7-12. Log file information

CF238.3

### Notes:

DB2 uses a control file to determine the status of log files. The control file identifies the active log file with the *lowest* name. This is called the loghead.

In archival logging, the values for loghead can be valuable to the database administrator. It can be obtained via a GET DATABASE CONFIGURATION command. The value of *First active log file* identifies the loghead.

Log files with names that are *less* than the loghead are archive files. They are not required for crash recovery and could be moved to a different media.

### ***Instructor notes:***

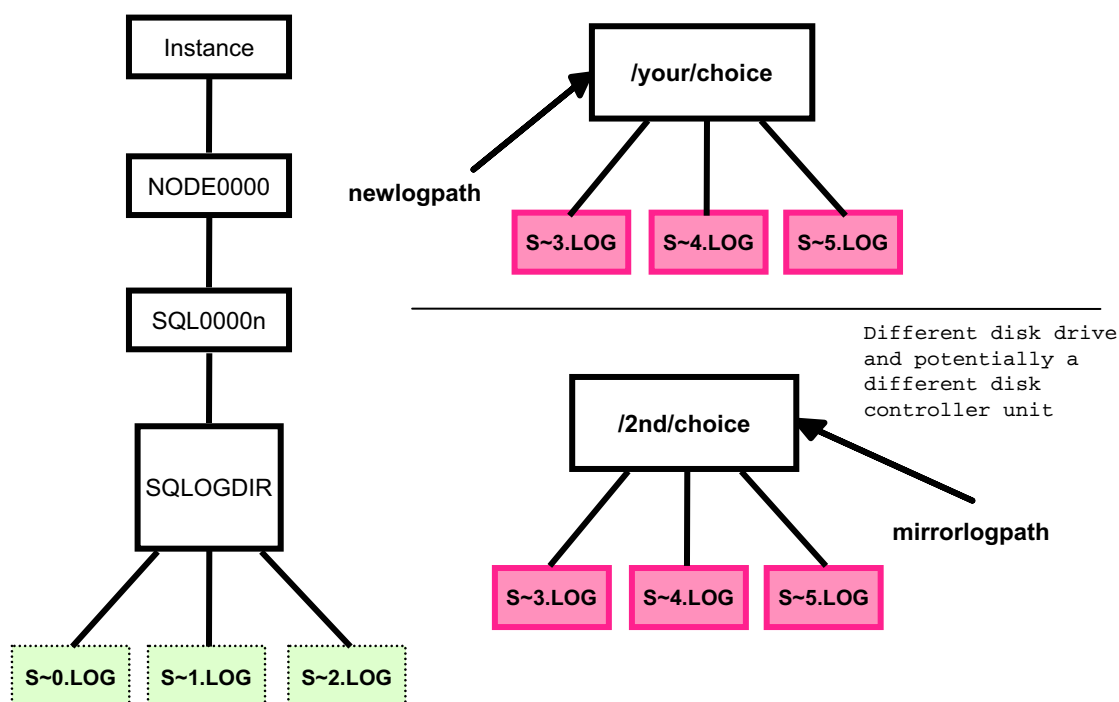
**Purpose** — Describe the availability of informational database configuration parameters that address log files.

**Details** — Emphasize that the header file should not be edited. The API to retrieve information should be used. Also, emphasize the use of loghead for monitoring and tuning log file allocation parameters, such as logprimary, logfilsiz, and logsecond.

**Additional information** —

**Transition Statement** — In addition to the size and number of log files, the database administrator can designate the location of the log files.

## Location of log files



© Copyright IBM Corporation 2007

Figure 7-13. Location of log files

CF238.3

### Notes:

By default, log files are located in SQLLOGDIR, which is a subdirectory of the database directory. It is not generally good practice to store log files on the same physical device as the database files for which they provide recovery support.

The location of log files currently in use is identified by the informational configuration parameter *Path to log files*.

The NEWLOGPATH database configuration parameter allows the administrator to redirect logging support to a specified path. The new path does not become active until the database becomes inactive and the database is in a consistent state. (A database may be in an inconsistent state due to an incomplete recovery process. This simply means that all units of work are not complete. The informational database configuration parameter *Database is consistent* contains this status.) When the database becomes active again, the value in NEWLOGPATH will be used to identify the new location of the log files.

The situation illustrated is not generally desirable. In the illustration, a database was created and used before the log path was changed. Therefore, log files exist in the default directory SQLLOGDIR. At some point in time, while **S0000002.LOG** was the active log file,

the database administrator updated the configuration file to indicate a NEWLOGPATH of /usr/your/choice. Assuming all applications completed units of work and disconnected from the database before allocation of **S0000003.LOG**, the logpath was changed to /usr/your/choice at the time a new application connected to the database. The file **S0000003.LOG** was allocated in the new location. This scenario can cause subsequent recovery processes to be more complex than necessary. If a log path change is desired, change the log path **BEFORE** using the database in order to direct the logs to a device that does not contain database files.

If a change to the log path is required after a database has been used, create a database backup after changing the log path.

A recovery strategy involving a change to log path during the recovery process is not recommended.

DB2 supports log mirroring at the database level. Mirroring log files helps protect a database from:

- Accidental deletion of an active log
- Data corruption caused by hardware failure

If you are concerned that your active logs may be damaged (as a result of a disk crash), you should consider using the DB2 configuration parameter, MIRRORLOGPATH, to specify a secondary path for the database to manage copies of the active log, mirroring the volumes on which the logs are stored.

The MIRRORLOGPATH configuration parameter allows the database to write an identical second copy of log files to a different path. It is recommended that you place the secondary log path on a physically separate disk (preferably one that is also on a different disk controller). That way, the disk controller cannot be a single point of failure.

When MIRRORLOGPATH is first enabled, it will not actually be used until the next database startup. This is similar to the NEWLOGPATH configuration parameter.

If there is an error writing to either the active log path or the mirror log path, the database will mark the failing path as *bad*, write a message to the administration notification log, and write subsequent log records to the remaining *good* log path only. DB2 will not attempt to use the *bad* path again until the current log file is completed. When DB2 needs to open the next log file, it will verify that this path is valid, and if so, will begin to use it. If not, DB2 will not attempt to use the path again until the next log file is accessed for the first time. There is no attempt to synchronize the log paths, but DB2 keeps information about access errors that occur, so that the correct paths are used when log files are archived. If a failure occurs while writing to the remaining *good* path, the database shuts down.

***Instructor notes:***

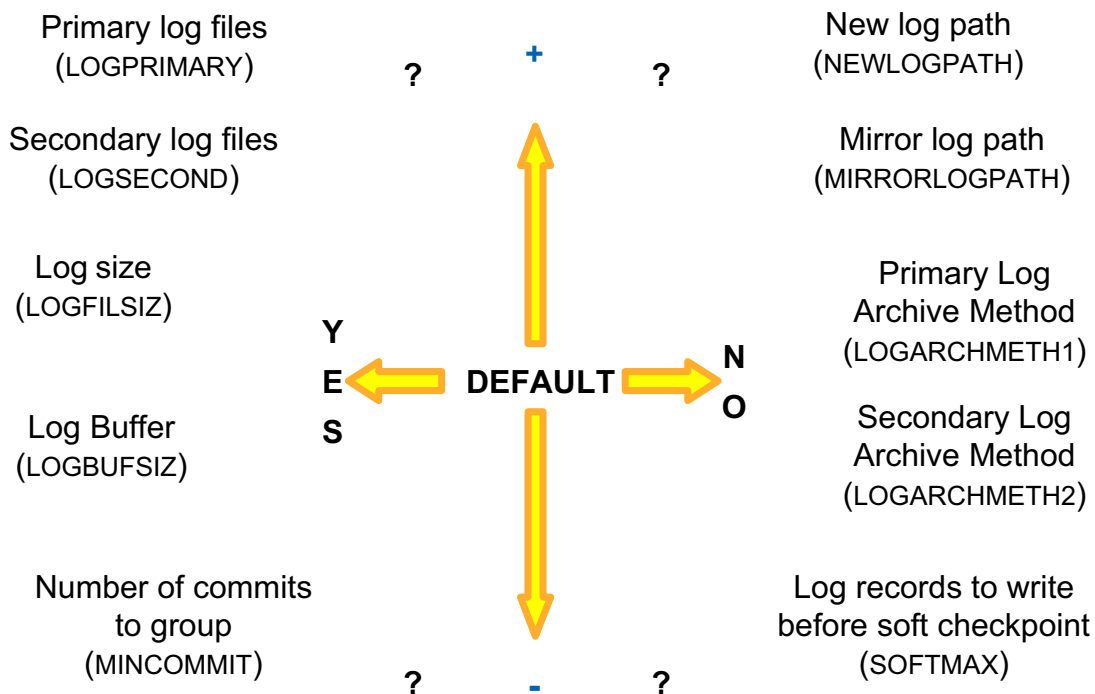
**Purpose** — Identify the capability to place log files where desired.

**Details** — It is critical that students realize that log files should generally not be on the same disk as the database supported, and that a single log path for recovery is desirable. In addition, if you change the value of the NEWLOGPATH, you should take a new backup of the database so that, if you get into a recovery scenario, you don't have DB2 expecting the log files in one location for part of the time, and in another location for another part of the time. You can also perform log file mirroring with DB2 with the MIRRORLOGPATH parameter.

**Additional information** —

**Transition Statement** — We have introduced several configuration parameters that impact logging. The next page shows how to update these parameters using the GUI.

## Configure database logs — parameters



© Copyright IBM Corporation 2007

Figure 7-14. Configure database logs — parameters

CF238.3

### Notes:

The configuration parameters listed above can impact logging for the database. The charts on the following pages summarize some considerations regarding these parameters. The proper values for these parameters are highly dependent on the installation requirements, so care must be taken regarding rules of thumb. Consult the *Data Recovery and High Availability Guide and Reference* for further details.

| Parameter (scope) | Description                                     | Default (range)                             | General recommendations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------|-------------------------------------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| logprimary (db)   | Number of primary log files                     | 3 (2-256)                                   | This value represents the number of primary log files that will be allocated to support database logging. The appropriate value is highly dependent on installation-specific requirements. For circular logging that is frequently using secondary logs, it may be necessary to increase this value. For log retention logging with highly active systems, it may be necessary to use a value greater than the default so that waits for log allocations are not experienced. In the opposite direction, a database that is not accessed frequently may be properly supported by just two log files, with a savings in disk space. |
| logsecond (db)    | Number of secondary log files                   | 2 (-1; 0-254)                               | This parameter specifies the number of secondary log files that are created and used for recovery log files (only as needed). When the primary log files become full, the secondary log files (of size logfilsiz) are allocated one at a time as needed, up to a maximum number as controlled by this parameter. Changes to this parameter are immediately effective.                                                                                                                                                                                                                                                              |
| logfilsiz (db)    | Size of primary and secondary log file in pages | UNIX:<br>1000<br>Intel: 250<br>(4 - 262144) | This parameter determines the number of pages to be allocated when a log file is requested. Combined with logprimary and logsecond, this value determines the disk space required to support logging. Since all primary logs are allocated, even if not used, the value of this parameter can have major impact on the system disk utilization. The value specified must be balanced between available disk space and the activity level of the database. A database that supports a high level of insert, update, and/or delete transactions favors a larger log file size, but at the expense of disk space.                     |

|                 |                                            |                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------|--------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| logbufsz (db)   | Number of pages used to buffer log records | 8 (32-bit: 4 - 4096, 64-bit: 4 - 65535) | This parameter determines the amount of database shared memory that is utilized for buffering log records before physical writes. A large value can improve logging I/O for active databases, but the cost is memory.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| mincommit (db)  | Number of commits to group                 | 1 (1 - 25)                              | This parameter indicates that grouping of commits issued by multiple applications is to be attempted, if the value is set greater than 1. If the number of applications connected to the database is greater than or equal to this parameter, commits will not cause immediate physical writes of the log. The writes will be delayed one second or until the number of commits requested by all applications equals this value. Grouping commits can enhance the performance of a database servicing multiple connects with high change activity. Setting this value greater than 1 can introduce one second commit waits for change applications that do not execute concurrently with other change applications. Changes to the value specified for this parameter take effect immediately. |
| newlogpath (db) | New log path for database logs             | Null (any valid path)                   | Database log files are, by default, written to the SQLOGDIR which is a subdirectory of the database directory. For recovery purposes, it will be beneficial for installations to place log files on a different physical disk than the database files. This parameter identifies the path for placement of log files. Set this parameter to direct log files to a disk that does not have high I/O requirements and does not contain the database itself.                                                                                                                                                                                                                                                                                                                                      |



|                     |                              |                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------|------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mirrorlog-path (db) | Log path for mirrored logs   | Null (any valid path)                        | This parameter allows you to specify a string of up to 242 bytes for the mirror log path. The string must point to a path name, and it must be a fully qualified path name, not a relative path name. If you configure the mirrorlogpath parameter, you must specify a path name for newlogpath, not a raw device. If mirrorlogpath is configured, DB2 will create active log files in both the log path and the mirror log path. All log data will be written to both paths. The mirror log path has a duplicated set of active log files, such that if there is a disk error or human error that destroys active log files on one of the paths, the database can still function. |
| logarch-meth1 (db)  | Primary log archive method   | Off (DISK, TSM, Vendor, userexit, logretain) | This parameter is the primary log archive method configuration parameter and is used to specifies the media type of the primary destination for archived logs. <ul style="list-style-type: none"> <li>- DISK:&lt;path&gt;</li> <li>- TSM:[management class name]</li> <li>- Vendor:&lt;vendor library&gt;</li> <li>- USEREXIT</li> <li>- LOGRETAIN</li> </ul>                                                                                                                                                                                                                                                                                                                      |
| logarchopt1 (db)    | Options for logarch-meth1    | Null                                         | This parameter specifies the options field for the primary destination for archived logs (if required). For example, <pre>UPDATE DB CFG FOR mydb USING LOGARCHOPT1 -SERVERNAME=mytmsserver2</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| logarch-meth2 (db)  | Secondary log archive method | Off (DISK, TSM, Vendor, userexit, logretain) | This parameter specifies the media type of the secondary destination for archived logs. If this path is specified, log files will be archived to both this destination and the destination specified by the <i>logarchmeth1</i> database configuration parameter.                                                                                                                                                                                                                                                                                                                                                                                                                  |
| logarchopt2 (db)    | Options for logarch-meth2    | Null                                         | This parameter specifies the options field for the secondary destination for archived logs (if required).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

|                |                                             |                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| logretain (db) | Log retention logging to be used            | No (No   Recovery)        | <p>Setting this parameter to Recovery indicates that log retention logging is to be used. The log files become archive log files when full instead of being reused. If an installation wishes to enable roll forward recovery, this parameter (or userexit) must be enabled.</p> <p>Now deprecated. Use instead logarchmeth1 and logarchopt1, and also logarchmeth2 and logarchopt2 if needed.</p>                                                                                                                                                                                                                                                                                                                      |
| userexit (db)  | User exit enable                            | No (Yes   No)             | <p>This parameter indicates whether a user exit should be called when processing archive or retrieval requests for log files. Setting this value to yes enables log retention logging and roll forward recovery, regardless of the setting for logretain. Installations that wish to automate the process of managing archive log files may benefit from utilizing this database exit.</p> <p>Now deprecated. Use instead logarchmeth1 and logarchopt1, and also logarchmeth2 and logarchopt2 if needed.</p>                                                                                                                                                                                                            |
| softmax (db)   | Log records to write before soft checkpoint | 100 (1 - 100* logprimary) | <p>This parameter will influence the number of logs that need to be recovered following a crash (such as a power failure). For example, if the default value is used, the database manager will try to keep the number of logs that need to be recovered to 1. If you specify 300 as the value of this parameter, the database manager will try to keep the number of log files that need to be recovered to 3. To influence the number of logs required for crash recovery, the database manager uses this parameter to trigger the page cleaners to ensure that pages older than the specified recovery window are already written to disk. This parameter will also determine the frequency of soft checkpoints.</p> |

Changes to most log configuration parameters are not immediately effective. For example, enabling log retention logging will not change circular logs allocated until all applications disconnect from the database and the database is deactivated. The next activation of the database will cause new allocation of log files to support the new strategy. Only LOGSECOND and MINCOMMIT will be effective as soon as the change is committed, though other parameters may become effective earlier in later releases of DB2.

There are some more logging configuration parameters. Consult the *Data Recovery and High Availability Guide and Reference* for further details.

## ***Instructor notes:***

**Purpose** — Provide insight regarding critical configuration parameters impacting logging.

**Details** — The primary information is contained in the notes for the student. The parameters logprimary, logsecond, logfilsiz, and newlogpath have been introduced previously, so detailed explanations for these are not required.

Highlight the fact that either logretain or userexit yes will cause log retention logging.

Some time may be required to adequately explain mincommit and softmax. The detailed implementation of the log control file and softmax is NOT necessary. The point to make is that the soft checkpoint will facilitate faster crash recovery because the log control file will contain more accurate information concerning which log records must be applied.

**Additional Information** — LOGPRIMARY establishes a fixed amount of storage allocated to the recovery log files. This parameter allows you to specify the number of primary log files to be preallocated.

LOGSECOND specifies the number of secondary log files that are created and used for recovery log files only as needed. When the primary log files become full, the secondary log files (of size logfilsiz) are allocated one at a time as needed, up to a maximum number as controlled by this parameter. Setting this parameter to -1 indicates that there is no limit on the size or number of in-flight transactions running on the database.

Log Size defines the size of each primary and secondary log file. The size of these log files limits the number of log records that can be written to them before they become full and a new log file is required.

Log Buffer size allows you to specify the amount of memory to use as a buffer for log records before writing these records to disk.

Number of Commits to Group allows you to delay the writing of log records to disk until a minimum number of commits have been performed. This delay can help reduce the database manager overhead associated with writing log records, and as a result improve performance when you have multiple applications running against a database and many commits are requested by the applications within a very short time frame.

New log path allows you to specify a string of up to 242 bytes to change the storage location of the log files. The string can point to either a path name, or to a raw device.

Mirror log path indicates that you want DB2 to maintain an identical set of logs on a secondary log path. This can help to prevent problems that would occur if the files on the primary log path were accidentally deleted, or if there was a disk failure that affected the primary log path.

Prior to DB2 9, one of the important configuration parameters was LOGRETAIN with the following values: No, to indicate that logs are not retained; and Recovery, to indicate that the logs are retained, and can be used for forward recovery. If logretain is set to Recovery, or userexit is set to Yes, the active log files are retained and become archive log files for use in roll forward recovery.

If USEREXIT is enabled, log retention logging is performed regardless of how the logretain parameter is set. This parameter also indicates that a user exit program should be used to archive and retrieve the log files.

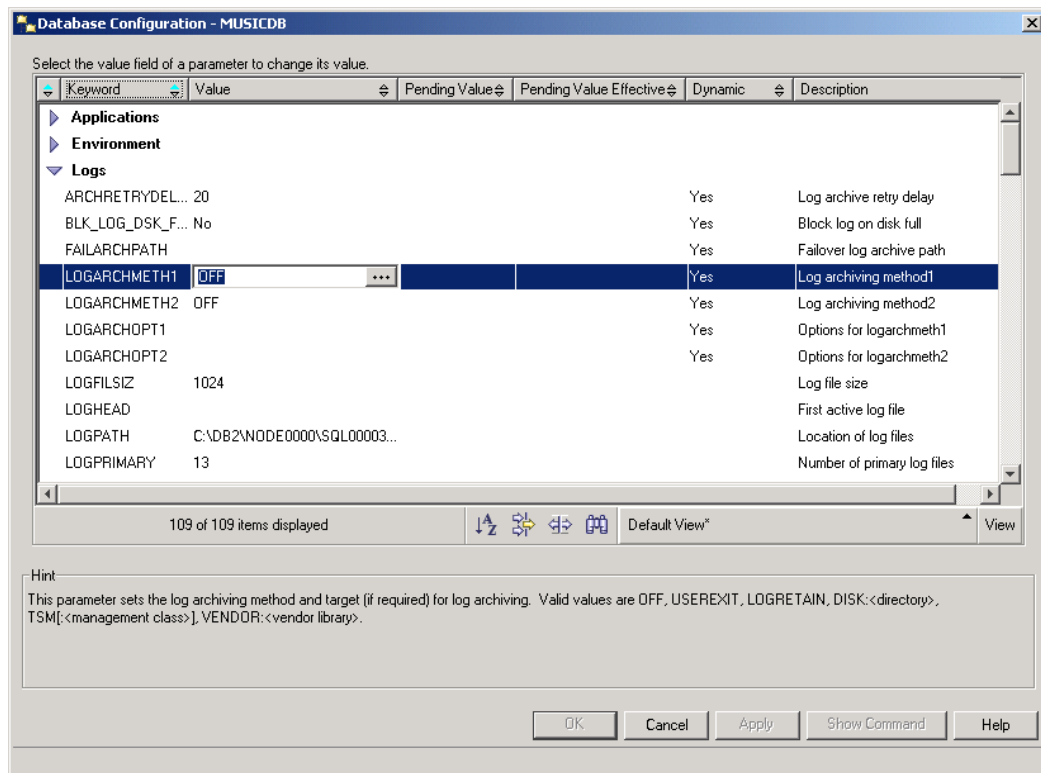
You should tell your students that LOGRETAIN and USEREXIT are being phased out (and hence are essentially deprecated). In their place LOGARCHMETH1/LOGARCHOPT1 and LOGARCHMETH2/LOGARCHOPT2 should be used instead (these were also available in V8).

Log Records to Write Before Soft Checkpoint is used to influence the number of logs that need to be recovered following a crash. It also determines the frequency of soft checkpoints.

**Additional information** — We have retained LOGRETAIN and USEREXIT explanations as the student may be familiar with them or need them for early releases of DB2.

**Transition Statement** — Now let us look at two different methods that we can modify the backup and restore configuration parameters using two different GUIs.

## Configure database logs — GUI (Control Center)



© Copyright IBM Corporation 2007

Figure 7-15. Configure database logs — GUI (Control Center)

CF238.3

### Notes:

To configure logs through GUI:

- Right-click and select the database
  - Left-click and select **Configure**
  - Scroll down to the parameters affecting logs

Log parameters such as logfilsiz, logprimary, logretain, and many others can be configured from this GUI.

Note that this GUI is used to change any configuration parameters — not just those for logging. The configuration parameters are arranged into about 9 groupings. The ones relevant to logging are gathered under the header logs (the triangle on the left can be flipped to show or hide the configuration parameters in each group).

***Instructor notes:***

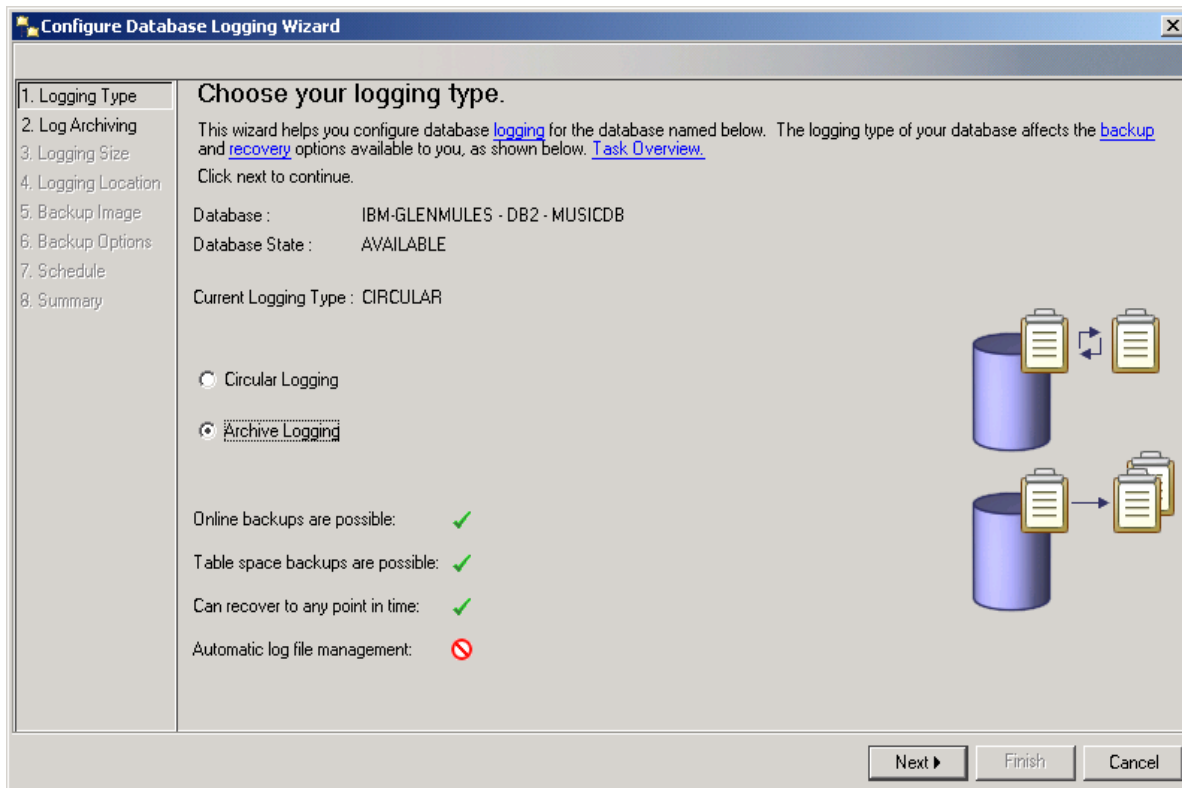
**Purpose** — Identify how the logging parameters can be changed using the GUI.

**Details** — All of the database configuration parameters can be updated through the GUI, including the logging-related parameters.

**Additional Information** — The database configuration parameters are all listed on one screen in the V8 GUI; there are not separate tabs for the different parameters as there were in Version 7 and before.

**Transition Statement** — Let's look at another possibility: to configure logging with the wizard available through the Control Center.

## Configure database logs — Wizard (CC)



© Copyright IBM Corporation 2007

Figure 7-16. Configure database logs — Wizard (CC)

CF238.3

### Notes:

To configure logs through the Logging Wizard GUI from the Control Center:

- Right-click and select the database
  - Left-click and select **Configure Database Logging**
  - Select logging type (Circular or Archive)
  - Log Archiving (Manual, User Exit, or DB2)
  - Logging Size
  - Logging Location
  - Run now or schedule for later execution

When you use the *Configure Database Logging* wizard, DB2 performs all the relevant steps for you, such as changing the relevant parameters as well as making a database backup.



***Instructor notes:***

**Purpose** — Explain to the students how the wizard can be used.

**Details** — Also state that, when the wizard is used, DB2 attempts automatically to make a backup of the database; so everything can be done from one GUI.

**Additional information** —

**Transition statement** — Before we go ahead with the examining the log files in more detail, let's revisit updating configuration parameters in command line mode and see how we can make things a little easier.

## Configure database logs — Command line

- Windows: `db2 get db cfg for musicdb | find /i "log"`
- Linux/UNIX: `db2 get db cfg for musicdb | grep -i log`
- Update: `db2 update db cfg for musicdb using logprimary 15`

```

Log retain for recovery status                = NO
User exit for logging status                  = NO
Catalog cache size (4KB)                     (CATALOGCACHE_SZ) = 260
Log buffer size (4KB)                        (LOGBUFSZ)      = 98
Log file size (4KB)                          (LOGFILSIZ)    = 1024
Number of primary log files                   (LOGPRIMARY)   = 13
Number of secondary log files                 (LOGSECOND)    = 4
Changed path to log files                     (NEWLOGPATH)   =
Path to log files                            = C:\DB2\NODE0000\SQL00003\SQLOGDIR\
Overflow log path                            (OVERFLOWLOGPATH) =
Mirror log path                              (MIRRORLOGPATH)  =
First active log file                        =
Block log on disk full                       (BLK_LOG_DSK_FUL) = NO
Percent max primary log space by transaction (MAX_LOG)    = 0
Num. of active log files for 1 active UOW (NUM_LOG_SPAN) = 0
Percent log file reclaimed before soft ckcpt (SOFTMAX)    = 520
Log retain for recovery enabled               (LOGRETAIN)    = OFF
User exit for logging enabled                 (USEREXIT)     = OFF
HADR log write synchronization mode           (HADR_SYNCMODE) = NEARSYNC
First log archive method                     (LOGARCHMETH1) = OFF
Options for logarchmeth1                     (LOGARCHOPT1)  =
Second log archive method                     (LOGARCHMETH2) = OFF
Options for logarchmeth2                     (LOGARCHOPT2)  =
Failover log archive path                     (FAILARCHPATH) =
Number of log archive retries on error        (NUMARCHRETRY) = 5
Log archive retry Delay (secs)                (ARCHRETRYDELAY) = 20
Log pages during index build                  (LOGINDEXBUILD) = OFF

```

© Copyright IBM Corporation 2007

Figure 7-17. Configure database logs — Command line

CF238.3

### Notes:

The most efficient way of updating configuration parameters is generally through the command line.

Since there are many configuration parameters, it is generally useful to filter the results of the GET CFG commands. This can be done with one of two operating system utilities depending on the platform where you are using the command:

- find — on Windows
- grep — on Linux and the various UNIX platforms

The search can be done case-insensitive (“/i” on Windows, “-i” on Linux/UNIX).

Of course, not all configuration parameters for logging will have *log* in the configuration list, and some irrelevant information may be listed.

***Instructor notes:***

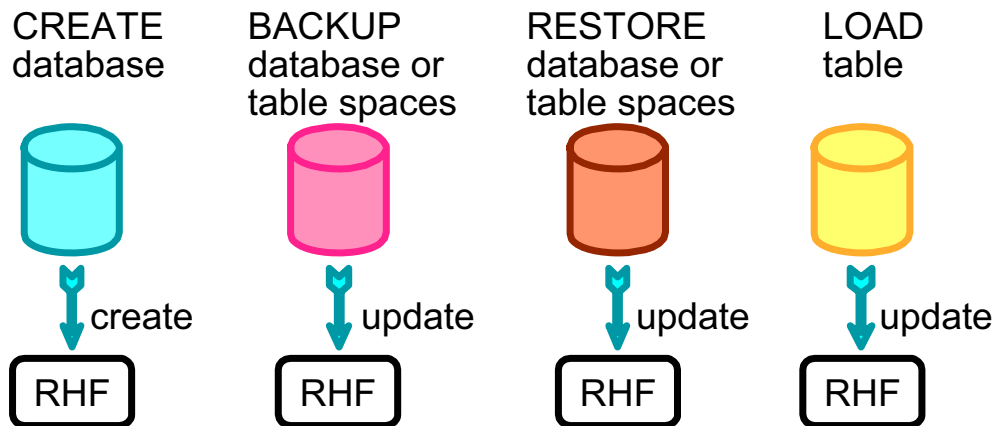
**Purpose** — Students need to understand listing and updating configuration parameters in both GUI and character mode.

**Details** —

**Additional information** — Note the differences between the Windows and Linux/UNIX operating system styles. Windows uses slash for the options to the find command, Linux/UNIX uses dash — and find requires that the search parameter be enclosed in quotes, but grep does not require quotes unless the search text has embedded whitespace.

**Transition statement** — Now that we have addressed logging and the methods of recovery, let's examine what information the database keeps on backup and restore operations.

## Recovery history file



- The part of the database that was copied and how
- The time the copy was made
- The location of the copy
- The last time a restore was done

→ `db2 LIST HISTORY ALL FOR MUSICDB`

© Copyright IBM Corporation 2007

Figure 7-18. Recovery history file

CF238.3

### Notes:

A recovery history file contains historical information for a database. The information provided in the chart above is not a complete listing of all the information provided in the history file. The file is maintained at the database level and resides in the same directory as the database configuration file. If the database is dropped, the history file is lost.

A recovery history file is created with each database and is automatically updated whenever:

- A database or table spaces are backed up
- A database or table spaces are restored
- A database or table spaces are rolled forward
- A table space is created
- A table space is altered
- A table space is quiesced
- A table space is renamed
- A table space is dropped
- A table is loaded

- A table is dropped
- A table is reorganized

You can use the summarized backup information in this file to recover all or part of a database to a given point in time. The information in the file includes:

- An identification (ID) field to uniquely identify each entry
- The part of the database that was copied and how
- The time the copy was made
- The location of the copy (stating both the device information and the logical way to access the copy)
- The last time a restore operation was done
- The time at which a table space was renamed, showing the previous and the current name of the table space
- The status of the backup operation: active, inactive, expired, or deleted
- The last log sequence number saved by the database backup or processed by a roll-forward recovery
- roll-forward recovery

The command shown in the visual will list all history for the *musicdb* database. If the word **backup** was used instead of history, only backups and restores would be shown.

An option on the RESTORE command permits only the recovery history file in a backup image to be restored. This is useful in recovery situations where the history file currently associated with the database is not accessible and information contained in the history file is needed to plan the recovery strategy.

The default retention period for the history file is 366 days, but this may be overridden by using the database configuration parameter REC\_HIS\_RETENTN. The maximum value is 30000. You can also manually remove old entries from the file by using the PRUNE command. The list history command syntax is:

```
LIST HISTORY [ backup | rollforward | reorg | create tablespace
              alter tablespace | dropped table | load | rename tablespace
              archive log ]
[ALL | SINCE timestamp |
  CONTAINING [ schema.objectname | objectname ] ]
FOR [DATABASE] database-alias
```

LIST BACKUP can be used as an alternative to LIST HISTORY BACKUP.

Examples of LIST HISTORY command:

```
db2 list history containing mel.table50 for musicdb
db2 list backup containing userspace1 for musicdb
db2 list history since 20040302 for musicdb
db2 list backup since 20041012 for musicdb
db2 list history all for musicdb
db2 list backup all for musicdb
```

The PRUNE HISTORY command may be used to delete entries from the recovery history file. A minimum of yyyy may be specified as the timestamp. All entries with timestamps equal to or less than the timestamp provided are deleted from the recovery history file. The WITH FORCE OPTION specifies that the entries will be pruned according to the timestamp specified, even if some entries from the most recent restore set are deleted from the file.

Examples are:

```
db2 prune history 20040228100922
db2 prune history 20040228 with force option
```

***Instructor notes:***

**Purpose** — Identify the recovery history file.

**Details** — This information can be used in a variety of situations concerning recovery planning and implementation. For example, assume that the log files required to support restore of an online backup have been lost. The backup taken just prior to the online backup in question would be needed. The history file could assist in finding this backup.

**Additional information** —

**Transition Statement** — Now that we have examined the recovery methods available and the logging necessary to support these methods, let's examine the actual utilities required to support a backup and recovery strategy.





## 7.3 Backup and restore utilities

### Instructor Topic Introduction

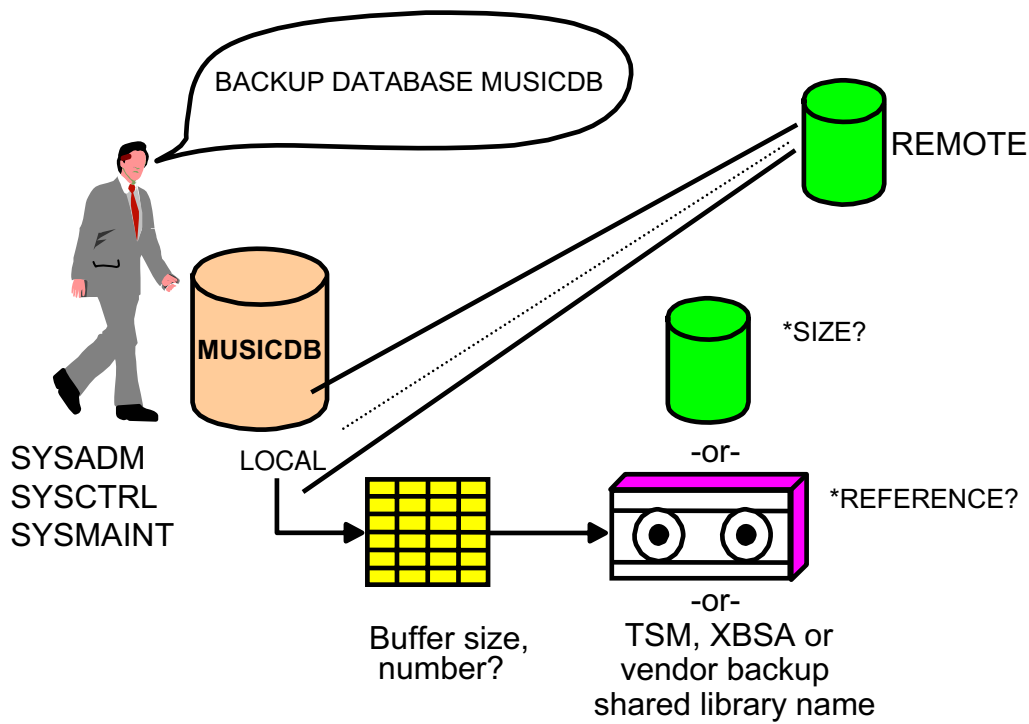
**What students will do** — Use the utilities to implement a backup and recovery strategy appropriate for the installation.

**How students will do it** — Lecture and lab. The lab exercises will be done at the end of the unit.

**What students will learn** — The syntax of backup, restore, and roll forward as well as the interplay of the log files with these utilities.

**How this will help students on their job** — A database that cannot be recovered will not support an installation.

# BACKUP command



© Copyright IBM Corporation 2007

Figure 7-19. BACKUP command

CF238.3

## Notes:

Your planning considerations should include:

- You must have SYSADM, SYSCTRL, or SYSMANT authority to use the BACKUP DATABASE command.
- The database may be local or remote. The backup image remains on the database server unless a storage management product such as Tivoli Storage Manager (TSM) is used.
- You can back up a database or table space to a directory, a tape, or a location managed by the TSM utility, to an XBSA interface, or to another backup vendor (if they provide the shared library).
- TSM allows you to specify that the backup is to use Tivoli Storage Manager (formerly ADSM) output. XBSA specifies that the XBSA interface is to be used.
- Backup Services APIs (XBSA) are an open application programming interface for applications or facilities needing data storage management for backup or archiving

purposes. Legato NetWorker is one of several storage manager that currently supports the XBSA interface.

- When backing up a database, you can indicate the amount of memory to be used for an internal buffer area on the backup command. If the command does not specify a BUFFER size, 1024 pages will be used.
- If you change the database configuration file to enable roll forward recovery (archive logging) by updating either logretain or userexit, you must take an offline backup of the database before it is usable.

If you are not able to make a backup copy of the database (because of time constraints) but need to use the database, you may change the database configuration file to disable roll forward recovery before the first backup. This means you are abandoning your attempt to implement log retention logging.

- A table space backup and a table space restore cannot be run at the same time, even if the backup and restore cover different table spaces.
- If you have tables that span more than one table space, you should back up (and restore) the set of table spaces together. This will eliminate some of the complexity of your recovery strategies.
- If your database is enabled for roll forward recovery and you are using a tape system that does not support the ability to uniquely reference a backup image, do not keep multiple backup copies of the same database on the same tape.
- Multiple files may be created to contain the backed up data from the database or table space.

## ***Instructor notes:***

**Purpose** — Introduce the backup command.

**Details** — A backup is made by issuing the BACKUP command. You must have SYSADM, SYSCTRL, or SYSMAINT authority level to back up the database. When the backup utility runs, it copies the backup data to buffers, and then to the output location. You can indicate the size and number of the buffers on the backup command. You can also indicate where the backup data should be written: to disk, to tape, to TSM, to use an XBSA interface, or to a shared library name containing the vendor backup and restore I/O functions to be used.

Remind the students about the LOGARCHMETH1 and related parameters.

**Additional Information** — XBSA is defined by the X/Open committee (the owners now of the name *UNIX*). Thus, the “X” in XBSA: X/Open Backup Services Application.

**Transition Statement** — Let’s look at the syntax for the command since it provides us with the details that we need to understand.

## Syntax of the BACKUP command

Creates a backup copy of a database or table spaces

```
BACKUP DATABASE database-alias [USER username [USING password]]
[TABLESPACE (tblspace-name [ {,tblspace-name} ... ])] [ONLINE]
[INCREMENTAL [DELTA]]
[USE {TSM | XBSA} [OPEN num-sess SESSIONS]
[OPTIONS {options-string | options-filename}]
| TO dir/dev
[ {,dir/dev} ... ] | LOAD lib-name [OPEN num-sess SESSIONS]

[OPTIONS {options-string | options-filename}]]
[WITH num-buff BUFFERS] [BUFFER buffer-size] [PARALLELISM n]
[COMPRESS [COMPRLIB lib-name [EXCLUDE]] [COMPROPTS options-string]]
[UTIL_IMPACT_PRIORITY [priority]] [{INCLUDE | EXCLUDE} LOGS] [WITHOUT
PROMPTING]
```

© Copyright IBM Corporation 2007

Figure 7-20. Syntax of the BACKUP command

CF238.3

### Notes:

Consult the *Command Reference* for more detailed syntax of the BACKUP command.

The following considerations are useful when running the BACKUP DATABASE command:

- You must start the database manager (DB2START) before running the BACKUP command or API. When using the Control Center, you do not need to explicitly start the database manager.
- When using the command, API, or task under the Control Center, you must specify a database alias name, not the database name itself.
- A table space backup can include a single table space or multiple table spaces.
- You may specify multiple target areas (directory or tape devices) if you wish to take advantage of parallel backup. Consult the *Data Recovery and High Availability Guide and Reference* for details.
- The LOAD option is used to interface with vendor backup and restore I/O functions. Consult the *Data Recovery and High Availability Guide and Reference* for details.

- During the backup procedures, an internal buffer is filled with data to be backed up. When this buffer becomes full, the data is copied to the backup medium. You can optionally select to use multiple buffers and I/O streams to improve the performance of the backup procedure. The number of buffers you allocate should be at least twice as many as the number of I/O devices you are using.

You may specify the number of pages to use for the backup buffers when you invoke the BACKUP DATABASE command. If no value is specified, a buffer size of 1024 pages will be used. If there is not enough memory available to allocate the buffer, an error will be returned.

The ideal backup buffer size should be a multiple of the extent size for the table spaces. If you have multiple table spaces with different extent sizes, specify a value that is a multiple of the largest extent size.

If any of the table spaces in a database is in an *abnormal* state, you cannot back up the database.

- If a system crash occurs during a critical stage of backing up a database, you cannot successfully connect to the database until you reissue the BACKUP DATABASE command.
- The BACKUP DATABASE command provides a concurrency control for multiple processes that are making backup copies of different databases. The control keeps a container open (on the backup target device) until the entire backup process has ended. If an error occurs during a backup process, the open container cannot be closed. With the container open, other backup processes to the same target drive may receive access errors. To correct any access errors, you must completely exit the backup process that caused the error and disconnect from the target device.
- If you are using the BACKUP DATABASE command for concurrent backup processes to tape, ensure that the processes do not target the same tape.
- The decision to select which table spaces to back up will also reduce the time during which the database is not available.
  - Long field and large object (LOB) data for a table must be placed in the same table space.
  - LONG FIELD and LOB data can be in a separate table space from the rest of the table data.
  - INDEX data could also be kept in a separate table space from the rest of the table data.

Backup strategies could be planned to take advantage of table space backup, thereby shortening the time for the backup to complete.

It is also possible to group several related tables together so that the table spaces they share can be considered a *unit* for backup and restore. This could provide the *granularity* required for backup and restore to complete in a shorter time frame, while keeping the management of backups reasonable.

Backup can be invoked locally or remotely. However, the backup image must be directed to local devices, unless TSM or another vendor product is used.

To reduce the amount of time required to complete a backup:

- Increase the backup buffer size
- Increase the number of buffers
- Use multiple target devices.

Reducing the amount of time naturally increases the CPU and I/O load on your system and will impact users on the system.



### Note

#### Throttling back CPU & I/O intensive commands and utilities

If you do not want to impact users on the system, you may want to throttle the backup process. This will lengthen the time taken to complete the backup, but perhaps notably so. When throttling is in effect — typically with a constraint of 10% — online users will generally not notice the effect of backup.

We will discuss throttling commands and utilities such as this in a later unit.

The following parameters reference the visual:

#### **database-alias**

Specifies the alias of the database to back up.

#### **USER *username***

Identifies the user name under which to back up the database.

#### **USING *password***

The password used to authenticate the user name. If the password is omitted, the user is prompted to enter it.

#### **TABLESPACE *tablespace-name***

A list of names used to specify the table spaces to be backed up.

#### **ONLINE**

Specifies online backup. The default is offline backup. Online backups are only available for databases configured with logretain or userexit enabled. During an online backup, DB2 obtains IN (Intent None) locks on all tables existing in SMS table spaces as they are processed and S (Share) locks on LOB data in SMS table spaces.

#### **USE TSM**

Specifies that the backup is to use Tivoli Storage Manager output.

## **USE XBSA**

Specifies that the XBSA interface is to be used. Backup Services APIs (XBSA) are an open application programming interface for applications or facilities needing data storage management for backup or archiving purposes.

## **OPEN *num-sessions* SESSIONS**

The number of I/O sessions to be created between DB2 and TSM or another backup vendor product. This parameter has no effect when backing up to tape, disk, or other local device.

## **TO *target-area***

A list of directory or tape device names.

## **LOAD *library-name***

The name of the shared library (DLL on Windows operating systems) containing the vendor backup and restore I/O functions to be used. It can contain the full path. If the full path is not given, it will default to the path on which the user exit program resides.

## **WITH *num-buffers* BUFFERS**

The number of buffers to be used. DB2 will automatically choose an optimal value for this parameter unless you explicitly enter a value. However, when creating a backup to multiple locations, a larger number of buffers can be used to improve performance.

## **BUFFER *buffer-size***

The size, in 4 KB pages, of the buffer used when building the backup image. DB2 will automatically choose an optimal value for this parameter unless you explicitly enter a value. The minimum value for this parameter is 8 pages.

## **WITHOUT PROMPTING**

Specifies that the backup will run unattended, and that any actions which normally require user intervention will return an error message.



**Instructor notes:**

**Purpose** — Highlight the actual syntax of the BACKUP command.

**Details** — To back up a database, you issue the BACKUP DB command, naming the database. You must indicate where the backup should be written: to TSM or XBSA, to a disk or tape device, or to a load library. Note that you can indicate multiple output locations, or multiple sessions to open with TSM, XBSA, or the load library; this gives you the ability to write to several output locations. The major constraining factor in backup is where you are writing; if you can add more output locations, you can generally speed up the backup.

Looking back at the second line, you can only make table space level or online backups if log retain is on (archive logging).

On the last line are some additional options; you should have twice as many buffers as your output locations. The buffer size should be related to your extent size (a multiple or equal), and if you are writing to tape, the block size of the tape should be considered as well.

If you are running the utility through a script and it will be unattended, you should specify WITHOUT PROMPTING. With this option, if a response is required, DB2 won't wait for the response; the utility will just fail.

**Additional Information** — API invocation of this function could automate the frequency of backups. This can be especially useful for taking online table space backups of critical table spaces. The online backup permits access to the objects in the table spaces, which is normally desirable when using automated invocation. (The job which issues the API could be invoked on a periodic basis, for example, nightly at 2:00 am.)

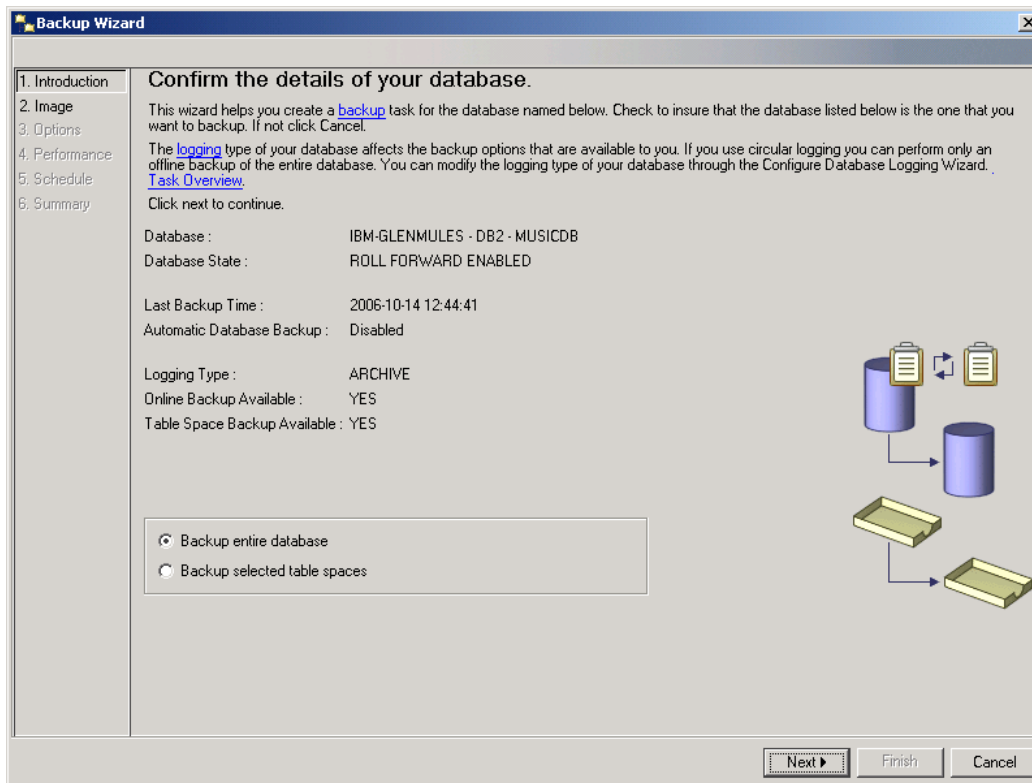
It is important to note that a backup can be of the database or one or more table spaces.

It is not the intention of the course to deliver detailed information on all components of the syntax. The USE TSM and LOAD options should not be a point of focus. Direct students to the *Data Recovery and High Availability Guide and Reference* for detailed discussion. Using multiple buffers should be mentioned because of the performance benefit.

Certification objective: 6A Ability to perform database-level and table space level BACKUP and RESTORE.

**Transition Statement** — In the beginning, or even later when you are a sophisticated administrator, you may prefer to use the GUI. We think, however, that you will come back to command line use since it can be scripted for daily use by system operators.

# BACKUP using the GUI interface



© Copyright IBM Corporation 2007

Figure 7-21. BACKUP using the GUI interface

CF238.3

## Notes:

To access the Backup Database GUI:

- Right-click and select the database
  - Left-click and select **Backup**

Use the Backup wizard to back up the objects in a database or database partition. First you specify if you want to back up the entire database or database partition, or if you want to back up only selected table spaces or table space partitions; then you make more advanced choices.

**Introduction** — This page lists the object that you have selected to back up. Verify that it is correct. If you are using archive logging, you can also choose whether you want to back up the entire database or back up only selected table spaces.

**Note:** Backing up all table spaces is not equivalent to backing up the entire database.

**Table Spaces** — This page is available only if you are in archive logging mode and you have chosen to back up selected table spaces. Use this page to specify the table spaces you want to back up. All non-temporary table spaces in your database are eligible to be

backed up, and are listed in the left-hand table. If you launched this wizard from one or more table space objects, they will be preselected in the right-hand table.

**Image** — Use this page to specify where you want to store your backup image. Indicate the media type you want to use and information about the particular media type.

**Options** — Use this page to specify whether you want to back up all data, back up data that has changed since the last full backup, or back up data that has changed since the last backup of any type. You can also specify whether you want the backup to occur while the database is online or offline. If the backup will be run offline, you can indicate that you would like to run a quiesce database command before the backup operation is run. You can also specify if you want (for an online backup) to include the relevant log files.

**Note:** If you are using circular logging, you can perform only an offline backup.

**Performance** — On this page, the wizard suggests recommended values for parallelism, the number of buffers, and the size of each buffer. You can modify these values. If you change the values on this page, you should click **Recommend** to have the wizard recalculate the recommended values.

**Schedule** — Use this page to indicate whether you want to run the backup operation immediately, or if you want to schedule the operation to run later. If you decide to run it later you have also the possibility to decide to run it once or repetitively, and to schedule it according your needs (automatic backup).

**Summary** — This page allows you to review the decisions that you made for your backup operation, and to view the command that will back up the objects in your database.

### ***Instructor notes:***

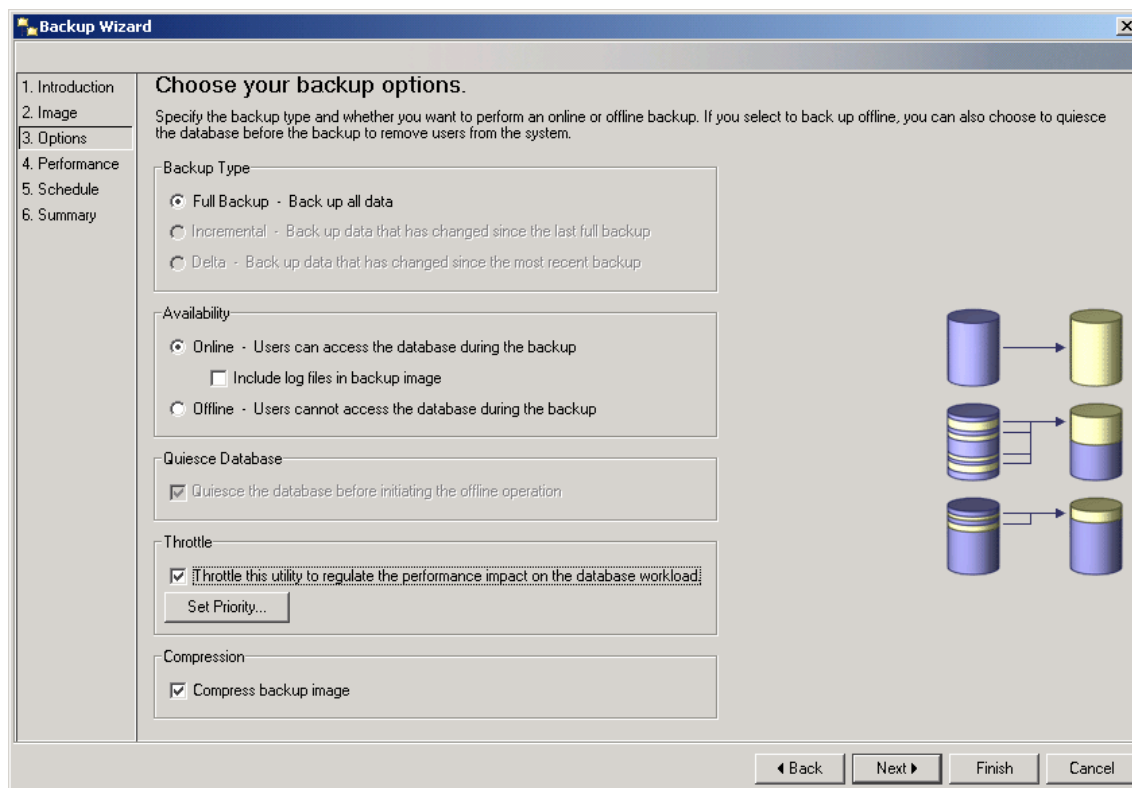
**Purpose** — Identify the functions available within the backup GUI.

**Details** — The backup GUI allows you to specify the options for the backup you want to perform.

**Additional Information** — Prior to V8, there was a simpler backup GUI, but now only the wizard is available. Notes are from the Task Overview online help.

**Transition Statement** — One of the options you can select is whether the backup should be offline or online.

# Online versus offline BACKUP



© Copyright IBM Corporation 2007

Figure 7-22. Online versus offline BACKUP

CF238.3

## Notes:

You can do a backup while the database is either offline or online.

- If the backup is performed offline, only the backup task can be connected to the database. The stored data is consistent, and can be used for a restore only recovery or a restore followed by a roll forward.

The implication of offline backup is that the rest of your organization cannot connect to the database while the backup task is running.

- An offline backup is the only backup supported when circular logging is being used.
- If the backup is performed online, other applications or processes can continue to connect to it while the backup task is running.

Online backups are supported only if archive logging is enabled. The backup will not be valid for recovery purposes if you do not retain the log files written while the backup is taken. At the end of an online backup, the current active log is truncated (by default) and made available for offline archival.

While the online backup operation is running, changes can be performed on the tables.

The data, when restored from the backup files, is NOT consistent until the logs are applied during roll forward recovery.

## ***Instructor notes:***

**Purpose** — Emphasize the difference between ONLINE and OFFLINE backups.

**Details** — The default is OFFLINE backups. The user must specifically designate ONLINE to override the need for exclusive use of the database. Note that OFFLINE versus ONLINE has a scope related to the DATABASE, not to the TABLE SPACES named with the TABLESPACE parameter. This means that, when taking a backup of even a single table space, no other users can CONNECT to the database unless the ONLINE parameter is also specified. There is no concept of taking a table space offline.

### **Additional information —**

**Transition statement** — Although the history file would typically be used to obtain information regarding backups, the names of the backup images themselves could be useful.

## The backup files

- File name for backup images *on disk* has:

- Database alias
- Type of backup (0=full, 3=table space, 4=copy from table load)
- Instance name
- Node number
- Catalog node number
- Timestamp of backup (YYYYMMDDHHMMSS)
- Sequence number (for multiple files)

**MUSICDB.0.DB2.NODE0000.CATN0000.20061013150616.001**

- Exact naming convention may differ slightly by platform
- Tape images are not named, but internally contain the same information in the backup header for verification purposes
- Backup history provides key information in easy-to-use format

© Copyright IBM Corporation 2007

Figure 7-23. The backup files

CF238.3

### Notes:

Backup images consist of:

|                       |                                                                                                                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Database alias</b> | A 1- to 8-character database alias name that was specified when the backup utility was invoked.                                                                                                                                                |
| <b>Type of backup</b> | Type of backup operation, where: 0 represents a full database-level backup, 3 represents a table space-level backup, and 4 represents a backup image generated by the LOAD...COPY TO command.                                                  |
| <b>Instance name</b>  | A 1- to 8-character name of the current instance that is taken from the DB2INSTANCE environment variable.                                                                                                                                      |
| <b>Node number</b>    | The database partition number. In single partition database environments, this is always NODE0000. In partitioned database environments, it is NODExxxx, where xxxx is the number assigned to the database partition in the db2nodes.cfg file. |

**Catalog node number** The database partition number of the catalog partition for the database. In single partition database environments, this is always CATN0000. In partitioned database environments, it is CATNxxxx, where xxxx is the number assigned to the database partition in the db2nodes.cfg file.

**Timestamp of backup** A 14-character representation of the date and time at which the backup operation was performed. The time stamp is in the form *yyyymmddhhnnss*, where:  
*yyyy* represents the year (1995 to 9999)  
*mm* represents the month (01 to 12)  
*dd* represents the day of the month (01 to 31)  
*hh* represents the hour (00 to 23)  
*nn* represents the minutes (00 to 59)  
*ss* represents the seconds (00 to 59)

**Sequence number** A 3-digit number used as a file extension (for multiple files).

The recovery history file is updated automatically with summary information whenever you carry out a backup or restore of a full database or table space. The history file is usually sufficient to support management of the backup images. However, there may be cases where knowledge of the naming convention used by DB2 regarding backup files could be useful. One such case would involve a damaged history file.

The `db2ckbkp` utility allows you to display information regarding the backup images. It allows you to:

- Test the integrity of a backup image and determine whether it can be restored or not.
- Display the metadata about the backup that is stored in the backup header.

You must have read permissions on the backup images that you specify when using this utility. Refer to the *Data Recovery and High Availability Guide and Reference* for additional information on this utility.

Assume that a database backup of the database **MUSICDB** in the instance **INST00** was taken on **Friday, October 13, 2006 at 15:06:16**.

**On Windows and Linux/UNIX, the naming convention would result in the following:**

| Alias                                                 | Instance | Cat Node | Month | Hour | Second          |
|-------------------------------------------------------|----------|----------|-------|------|-----------------|
|                                                       |          |          |       |      |                 |
|                                                       |          |          |       |      |                 |
| musicdb.0.inst00.NODE0000.CATN0000.20061013150616.001 |          |          |       |      |                 |
|                                                       |          |          |       |      |                 |
|                                                       |          |          |       |      |                 |
|                                                       | Type     | Node #   | Year  | Day  | Minute Sequence |

The type of backup taken is encoded:

- **0** is for full database
- **3** is for table space



- 4 is for copy from a table load



### Note

Since operating systems do not share identical conventions for naming files, DB2 does not necessarily make the backup file names identical across platforms. However, the names are very easy to decode. Prior to DB2 9, and on Intel only, the naming convention would have resulted in the following:

| Alias                                                  | Instance | Cat    | Node | Month | Hour | Second   |
|--------------------------------------------------------|----------|--------|------|-------|------|----------|
|                                                        |          |        |      |       |      |          |
|                                                        |          |        |      |       |      |          |
| MUSICDB.0\INST00\NODE0000\CATN0000\20040701\131259.001 |          |        |      |       |      |          |
|                                                        |          |        |      |       |      |          |
|                                                        |          |        |      |       |      |          |
|                                                        | Type     | Node # |      | Year  | Day  | Minute   |
|                                                        |          |        |      |       |      | Sequence |

In this approach, a series of folders were used as part of the naming convention. With DB2 9, the naming conventions used for Intel and Linux/UNIX are *now* the same.

## ***Instructor notes:***

**Purpose** — Provide the naming convention used when creating backup images.

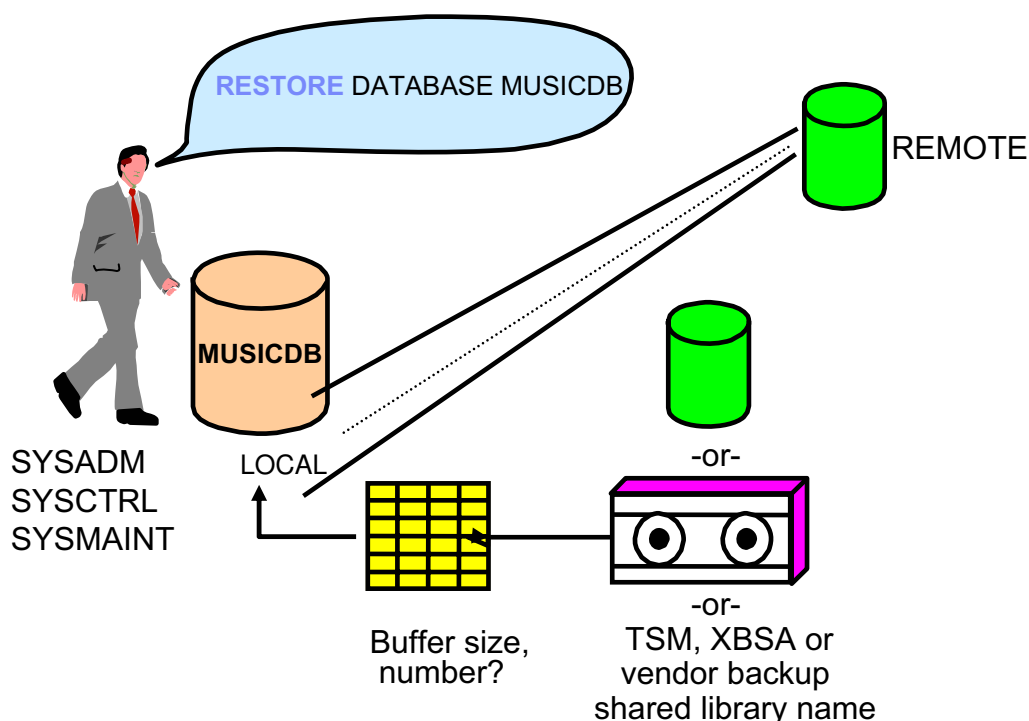
**Details** — As the student notes suggest, the history file contains information regarding backups. It could be used to obtain the information that is embedded in the backup image names. The reason this information is presented in the course is to handle the case when the history file itself has been damaged, or the case when someone prunes too much of the information.

Identify that the db2ckbkp command can be used to check the integrity of a backup and to ensure that it is restorable.

**Additional Information** — Although tape images are not named according to this convention, the information can generally be found in the header of the tape. An octal dump (piped to the more command) can be used to view the header of a tape.

**Transition Statement** — Of course, creating backups is completed to support the capability of doing a restore. Let's now look at the RESTORE command.

# RESTORE command



© Copyright IBM Corporation 2007

Figure 7-24. RESTORE command

CF238.3

## Notes:

When restoring a database, you should consider the following:

- You must have SYSADM, SYSCTRL, or SYSMAINT authority to restore to an existing database from a database or table space backup.
- \* **SYSMAINT cannot restore to a new database.** To accomplish this, you must have SYSADM or SYSCTRL authority.
- You can only use this command if the database or table space has been previously backed up with the BACKUP DATABASE command.
- You can choose to restore only the recovery history file, one or more table spaces, or everything in your backup image.**
- A table space restore is particularly helpful if an error occurs in the table space containing the system catalog tables.** An error in this table space will prevent users from connecting to the database, and having a backup containing only this table space could reduce your recovery time significantly.

- The RESTORE DATABASE command can use the Tivoli Storage Manager (TSM) utility, and any restrictions of that utility should also be considered.
- Another vendor product, or the XBSA interface, may also be used if that option was used to store the original backup image.
- A database restore requires an exclusive connection: that is, no applications can be running against the database when the task is started. Once it starts, it prevents other applications from accessing the database until the restore is completed.
- A table space restore can be online (share mode) or offline (exclusive mode). An online restore permits access to **OTHER** table spaces, but still requires exclusive use of the target table spaces of the restore.
- The size and number of the buffers used to support the restore can be specified as a command option. If the hardware configuration has multiple I/O controllers available, supporting I/O with multiple buffers can improve the performance of the restore. If the command does not specify a BUFFER size, a buffer size of 1024 pages will be used.
- The database may be local or remote.

***Instructor notes:***

**Purpose** — Introduce the concept of RESTORE.

**Details** — Cover the RESTORE command at a high level on this visual. The actual syntax diagram for the command is the subject of the next visual.

The restore is issued by a person with SYSADM, SYSCTRL, or SYSMAINT authority. The image is brought back in from wherever it was written by the backup, back through the buffers, and written into the database.

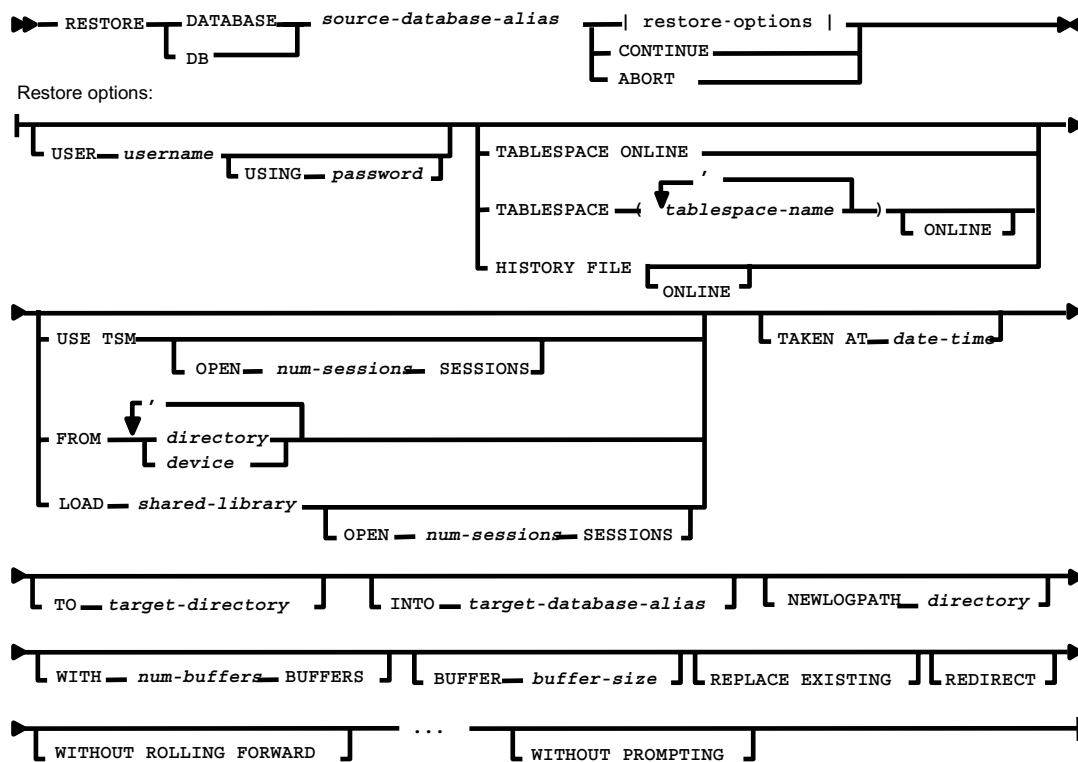
Ensure that students understand the authorization requirements, particularly that SYSMAINT cannot perform a restore that will create a new database. You may also wish to highlight that the content of the backup image dictates which table spaces can be restored.

**Additional Information** — You may want to have discussions with the students at this point on:

- a) Is it possible to restore a DB to another machine? **Yes**
- b) Is it possible to restore a DB to another machine with another operating system? **Yes, with limitations: you can restore a UNIX backup to another UNIX machine, except for Linux. Linux can only be restored to Linux; Windows can only be restored to Windows.**
- c) Is it possible to restore a DB from a 32-bit machine to a 64-bit machine? **Yes, but only on the same operating system. If you are doing a cross-platform restore (from one UNIX operating system to another), the instances must be the same “bit-ness”.**

**Transition statement** — Let's first look at the syntax for doing a restore. After that, we will look at the GUI.

# Syntax of the RESTORE command



© Copyright IBM Corporation 2007

Figure 7-25. Syntax of the RESTORE command

CF238.3

## Notes:

Consult the *Command Reference* for more detailed syntax of the RESTORE command.

The following considerations are useful when running the RESTORE command:

- The database manager must be started before restoring a database.
- The database to which you restore the data may be the same as the one from which the data was originally backed up, or it may be different. You may restore the data to a new or an existing database.

If you are restoring the database to a new database, but the containers for the table spaces do not yet exist, you will want to do a redirected restore (RESTORE ... REDIRECT). This will restore the definitions for the table space containers, and allow you to modify the definitions before continuing the restore (RESTORE ... CONTINUE). You may also want to use NEWLOGPATH to indicate where the log file directory should be placed.

If the database into which you are restoring already exists, you can avoid the warning message that requires a response before overwriting the database by indicating REPLACE EXISTING.

- During the restore process, you can optionally select to use multiple buffers to improve the performance of the restore utility. The multiple internal buffers may be filled with data from the backup media. The number of buffers you allocate should be at least twice the number of the I/O devices you are using.

You may specify the number of pages to use for each restore buffer when you invoke the RESTORE DATABASE command. If you do not specify the number of pages, each buffer will have a size of 1024 pages. If there is not enough memory available to allocate the buffer, an error will be returned.

- The backup copy of the database or table space to be used by the RESTORE DATABASE command can be located in a directory, a tape, or a location managed by the Tivoli Storage Manager (TSM) utility, to an XBSA interface, or to another vendor (if they provide the shared library).
- The LOAD option is used to interface with vendor backup and restore I/O functions. Consult the *Data Recovery and High Availability Guide and Reference* for details.
- Once the RESTORE DATABASE command starts for a database restore, the database is not usable until the command completes successfully. It is also possible that a roll forward will be required before access is permitted.
- Once the RESTORE DATABASE command starts for a table space restore, the table space is not usable until the RESTORE command followed by a roll forward recovery completes successfully.
- If a system failure occurs during any stage of restoring a database, you cannot connect to the database until you reuse the RESTORE DATABASE command and successfully complete the restore.
- Individual table spaces can be restored from either a database or a table space backup.
- A table space restore using the ONLINE parameter allows other table spaces to be used while the restore is running.
- If a system failure occurs during the restoring of a table space backup, only the table space being restored is unusable. The other table spaces in the database can still be used.
- TABLESPACE (tablespace-name) allows specification of a subset of the table spaces in the backup image to be restored.
- If the history file for the database has been deleted or corrupted, you can restore just the history file from the backup image (HISTORY FILE).

## ***Instructor notes:***

**Purpose** — Describe the syntax of the RESTORE command.

**Details** — Many of the parameters are counterparts to backup command parameters and do not need detailed explanations. For example, discussion regarding the buffer parameters could very well be redundant if this was discussed in relation to backup.

Many parameters will not be specified during a typical restore because the defaults are desirable. For example, a restore of an existing database will not require the specification of the TO and INTO parameters, since the defaults from the backup image will be generally the desired option.

In general, you indicate RESTORE DATABASE database-name, and where the backup image can be found. If more than one backup image exists in that location, you can use the TAKEN AT option to indicate which backup you want to restore. If you are restoring into a new database, you indicate INTO, and if you want to specify the path for this new database, you indicate TO target-directory and possibly the NEWLOGPATH. Just as on the backup command, you can specify a buffer size and a number of buffers.

If you are restoring into an existing database, you can use the REPLACE EXISTING option to avoid being warned that you are about to replace the database.

REDIRECT allows you to alter the table space container names before the restore operation continues. Details on redirected restore are addressed in CF49.

Highlight the TABLESPACE ONLINE parameter. This parameter is only relevant if the backup image contains one or more table space backups. Specification of this parameter allows access to other table spaces that are not being restored. If the backup image is actually a database backup, the restore will require exclusive use of the entire database. If you want to restore one or more table spaces from a backup image that has many table spaces (maybe even a full database backup), the TABLESPACE (tablespace-name) option can be used. ONLINE again means that the other table spaces in the database will remain online. Whatever you are restoring (database or table space) is always in exclusive use by the Restore utility.

Specification of HISTORY FILE indicates that only the history file is to be restored from the backup image. The rest of the database can be used while this file is restored if you specify the ONLINE option.

Highlight WITHOUT ROLLING FORWARD. This parameter can only be specified if the backup image restored is an OFFLINE backup at the database level.

WITHOUT PROMPTING allows the restore to continue without waiting for responses.

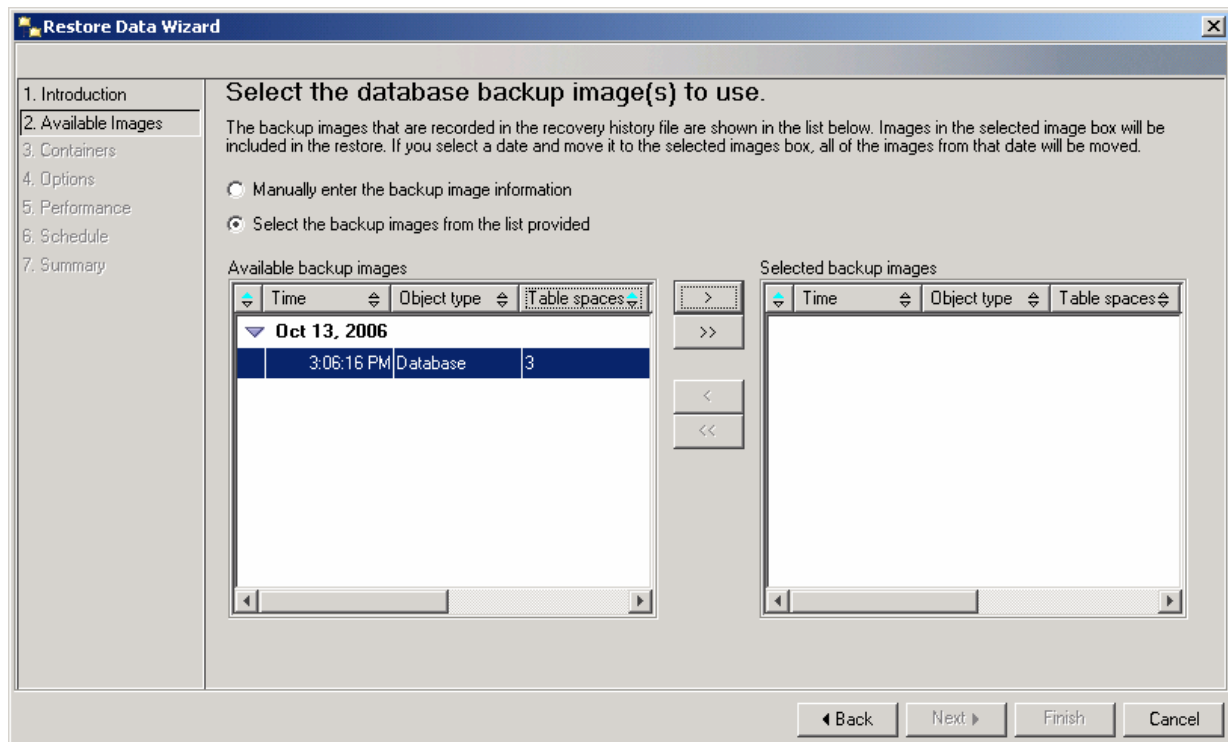
Highlight the TAKEN AT option. This option is not necessary if the source device only contains one backup image of the alias specified. However, if more than one image exists, the date and time must be specified in the form of *yyyymmddhhmmssnnnnnn*. This is the local time of the backup and does not include editing symbols. The reason this form is highlighted is that it will contrast with the roll forward to a point-in-time date, which is in terms of CUT time and contains editing symbols.



**Additional information** — Certification objective: 6A Ability to perform database-level and table space level BACKUP and RESTORE.

**Transition statement** — Let's look at the GUI for doing a restore.

# RESTORE using the GUI interface



© Copyright IBM Corporation 2007

Figure 7-26. RESTORE using the GUI interface

CF238.3

## Notes:

To access the Restore Database GUI:

- Right-click and select the database
  - Left-click and select **Restore**

Use the Restore wizard to perform any of the following tasks:

- Restore a database or database partition
- Roll forward a database or database partition
- Restore a database backup image to a new database
- Restore the history file for a database
- Restore a table space or table space partition
- Roll forward a table space or table space partition

First, you specify the objects you want to restore, and select the image that you want to use; then you make more advanced choices.

**Introduction** — This page lists the objects that you have selected to restore. Verify that they are correct. Also use this page to specify whether you are restoring to an existing database, restoring to a new database, or restoring the history file for the database.

Note that, if you have log archiving enabled, the summary will indicate that you can restore to the last transaction. If you are using circular archiving, certain restore options will not be available.

**Restore Objects** — This page is available only if you are restoring to an existing database and if you are in log archiving mode. If you only want to restore a subset of the database, use this page to specify the table spaces you want to restore.

**Restore to a New Database** — This page is available only if you have chosen on the Introduction page of this wizard to restore to a new database. Use this page to specify the name and location of the new database.

**Available Images** — Use this page to select the database image or images that you want to use to restore your database.

**Containers** — Use this page to modify the table space containers used for your database. You can add new containers, change existing containers, or remove containers.

**Roll Forward** — This page is available only if you are in log archiving mode. On this page, you can set roll forward parameters and specify where your log files are already stored so that they can be retrieved and used in the roll forward operation.

**Final State of Database** — Use this page to specify the state the database should be put in when the restore has completed. You can leave the database in roll forward pending state so that you can continue to make changes to the restore, or you can return the database to active state so that it is usable.

**Options** — This page allows you to set various restore options. If you are restoring an archival database, you can set quiesce options.

**History Options** — This page is available only if you have selected to restore only the history file for your database. Use this page to select how you want to have the restore operation run: online or offline.

**Performance** — On this page, the wizard suggests recommended values for parallelism, the number of buffers, and the size of buffers. You can modify these values.

**Schedule** — Use this page to schedule when the restore operation is to run.

**Summary** — This page allows you to review the decisions that you made for your restore operation, and to view the command that will restore the objects in your database.

***Instructor notes:***

**Purpose** — Identify the choices available with the Restore wizard.

**Details** — The Restore wizard steps you through the process of performing the restore.

**Additional Information** — Notes are from the Task Overview online help.

**Transition statement** — When performing a table space backup and restore, several factors need to be considered.

## Backup/restore table space considerations

- Roll-forward must be enabled
- Can choose to restore a subset of table spaces
- Generally best to put multiple spaces in one backup image
  - Makes table space recovery strategy easier
  - Provides access to related tables spaces and coherent management of these table spaces
- Handling of long/LOB/XML data requires a correlated strategy
- Point-in-time recovery is supported, but has requirements
- Faster recovery for Catalogs using Tablespace Level backup
- Critical business application tables should obviously be the *focus* of the backup/restore, but other tables are needed in support of these tables

© Copyright IBM Corporation 2007

Figure 7-27. Backup/restore table space considerations

CF238.3

### Notes:

In order to use table space level backup and restore, archive logging must be used.

A subset of table spaces can be restored from a database or table space backup image.

One of the key reasons to use table space level recovery is to reduce the time of backup and restore. This is accomplished by reducing the amount of data involved in these processes. However, the database administrator should not strive to simply minimize the amount of data in a backup. Although this could be accomplished by placing a single table in its own table space, this would be likely to lead to a management problem concerning backup and recovery.

If the table spaces associated with closely related tables are all contained in a single backup, only the applications targeting the closely related tables are affected. In many cases, such applications would be affected even if a single table in the group was unavailable. This is especially true in the case of referentially constrained tables. You should consider grouping the table spaces that support referential structures in a single backup image.

The key is to reduce the amount of data involved in the event recovery; this is necessary while controlling the impact on management of the backup/restore strategy.

LOB data and long field data can be placed in a table space that is separate from the regular data. This is often desirable from a recovery standpoint because the frequency of taking backups of the LOB data can be much less than that of the regular data. The nature of LOB data is that it tends not to be updated frequently and it is large. However, if a REORG of such a table and its LOB data is one of the actions recorded in the log files through which you are rolling forward, you must have all table spaces relating to the table in the backup, defeating one of your reasons to separate the data. The solution is to establish new backups of the table spaces associated with such tables AFTER a REORG that includes the LOB data.

**Point-in-time recovery during roll forward is supported. The LIST TABLESPACES command can be used to get a minimum roll forward time.**

If the catalog tables are involved in a recovery situation, access to the entire database is impacted. Therefore, it is good practice to maintain table space level backups of your catalog tables. If you need to recover the catalog, the duration of the process will be reduced if you can restore a table space backup instead of a database backup.

Application tables considered critical to the business should also be considered prime candidates for table space recovery. The major reason is the reduction in downtime provided through the table space backup/restore strategy.

***Instructor notes:***

**Purpose** — Emphasize the special considerations regarding table space recovery.

**Details** — Each Item on the list **should be discussed**. Identify key items in the student notes to detail the items in the list.

**Additional information** — Certification objective: 6A Ability to perform database-level and table space level BACKUP and RESTORE.

**Transition Statement** — After a successful restore, the database or one or more table spaces may be in a roll forward Pending state.

## Roll forward pending

- 
- *Roll forward pending* is set as a result of:
    - Restore of *offline* database backup omitting the command option **WITHOUT ROLLING FORWARD**
    - Restore of an *online* database backup
    - Restore of *any table space* level backup
    - DB2 detects media failure isolated at a table space
  - Scope of pending state managed by DB2
    - *Database* in pending state will not permit any activity
    - *Table spaces* in pending state will permit access to other table spaces

© Copyright IBM Corporation 2007

Figure 7-28. *Roll forward pending*

CF238.3

### Notes:

Roll forward pending is a state used by DB2 to protect the integrity of the database. This state indicates that a roll-forward process is necessary to ensure consistency of the data.

Roll forward pending can occur either at the database or at the table space level.

If the database is in a roll-forward pending state, no activity to the database is allowed. If a table space is in a roll forward pending state, only that table space is unusable.

A database will be put into a roll-forward pending state when:

- Restoring an OFFLINE DATABASE backup and omitting the option WITHOUT ROLLING FORWARD. This applies only to a database using archive logging.
- Restoring an ONLINE DATABASE backup.

A table space will be put into a roll forward pending state when:

- A table space is restored.

DB2 detects a media failure and isolates it at the table space level.



**Instructor notes:**

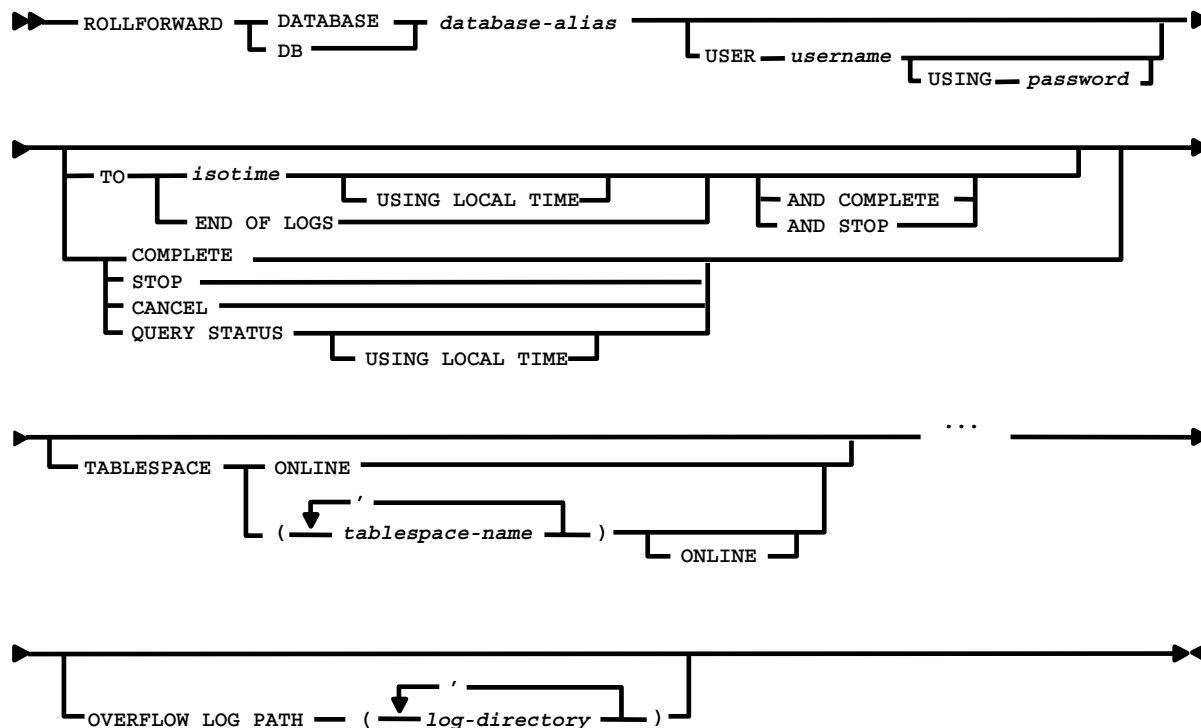
**Purpose** — How do you get into pending state?

**Details** — Protects data during *in flight* recovery process. (Restore has completed but you desire roll forward based on your configuration.)

**Additional Information** — Certification objective: 6E Knowledge of point in time recovery, 6F Knowledge of Roll Forward recovery.

**Transition Statement** — Let's look at the syntax of the ROLLFORWARD command.

## Syntax of the ROLLFORWARD command



© Copyright IBM Corporation 2007

Figure 7-29. Syntax of the ROLLFORWARD command

CF238.3

### Notes:

Authorization for this command requires SYSADM, SYSCTRL, or SYSMAINT.

ROLLFORWARD applies transactions recorded in the database log files.

The command needs to be invoked after a database or a table space backup has been restored, or if any table spaces have been taken offline by the database due to a media error.

Restore is the first phase of a complete roll forward recovery of a database or table space.

After a successful database restore, a database that was configured for roll forward recovery at the time the backup was taken enters a roll forward pending state, and is not usable until the ROLLFORWARD command has been run successfully. If the restore was for a table space, the table spaces restored are placed in a roll forward pending state.

When the ROLLFORWARD DATABASE command is issued, if the database is in a roll-forward pending state, the database is rolled forward. If the database is not in a roll forward pending state, all table spaces in the database in the roll forward pending state are processed.

Another database RESTORE is not allowed when the roll forward process is running.



If you restored from a full **OFFLINE** database backup image, you can bypass the roll-forward pending state during the recovery process. The RESTORE DATABASE command gives you the option to use the restored database immediately **WITHOUT ROLLING FORWARD** the database.

You **CANNOT** bypass the roll-forward phase when recovering at the table space level or if you restore from a backup image that was created using the **ONLINE option** of the BACKUP DATABASE command.

### Command Parameters

- DATABASE database-alias  
The alias of the database to roll forward.
- USER username  
Identifies the user name under which the database is to be rolled forward.
- USING password  
The password used to authenticate the user name. If the password is omitted, the user is prompted to enter it.
- TO isotime  
The point in time to which all committed transactions are to be rolled forward (including any transactions committed precisely at that time, as well as all transactions committed previously).  
  
This value is specified as a timestamp, a seven-part character string that identifies a combined date and time. The format is *yyyy-mm-dd-hh.mm.ss.nnnnnn* (year, month, day, hour, minutes, seconds, microseconds), which is specified in Coordinated Universal Time (CUT). An additional option, USING LOCAL TIME, can be specified to indicate the time in local time of the server.
- TO END OF LOGS  
Specifies that all committed transactions from all available active and archived log files should be applied to the database or tablespace that was restored.
- AND STOP / AND COMPLETE  
Completes the roll forward recovery process by rolling back any incomplete transactions and turning off the roll forward pending state of the database. This allows access to the database once again. Both STOP and COMPLETE perform the same function.

- STOP / COMPLETE

Does not roll forward any more log records, and completes the roll forward recovery process by rolling back any incomplete transactions and allowing access to the database once again. Both STOP and COMPLETE perform the same function.

- CANCEL

Cancels the roll forward process and leaves the database or table spaces on which forward recovery has been started in restore-pending state.

- QUERY STATUS

Lists the log files that the database manager has rolled forward, the next archive file required, and the time stamp (in CUT) of the last committed transaction since roll forward processing began. The information returned contains the following fields:

- Roll forward status — Status may be database or table space roll forward pending, roll forward in progress, roll forward processing STOP, or no roll forward pending.
- Next log file to be read — A string containing the name of the next required log file.
- Log files processed — A string containing the names of the processed log files that are no longer needed for recovery, and that can be removed from the directory.
- Last committed transaction — A string containing a time stamp in ISO format (yyyy-mm-dd-hh.mm.ss). This time stamp marks the last transaction committed after the completion of roll forward recovery.

**Note:** QUERY STATUS is the default if the TO, STOP, COMPLETE, and CANCEL clauses are omitted.

- TABLESPACE ONLINE

This keyword is specified to allow the table space level roll forward recovery to be done online. This means that other agents are allowed to connect and use other table spaces while roll forward recovery is in progress.

- TABLESPACE ONLINE tablespace-name

The table space name must be indicated for table space level roll forward to a point-in-time. This also allows a subset of table spaces to be specified for a roll forward to the end of logs.

- OVERFLOW LOG PATH log-directory

Specifies an alternate log path to be searched for archived logs during recovery.

If enabling an existing database for archive logging, change the number of primary log files to the sum of the number of primary log files and secondary log files +1. This recommendation assumes that the prior configuration was determined to meet the needs of the largest unit of work in the system. More information will be logged for LONG VARCHAR fields and LOB data in a database using archive logging.

Rolling databases and table spaces forward may involve prerequisites and restrictions that are beyond the scope of this course.

For more detailed information, refer to the *Data Recovery and High Availability Guide and Reference*.

## ***Instructor notes:***

**Purpose** — Discuss the syntax of the ROLLFORWARD command.

**Details** — Most of the parameters should be discussed to some degree, with perhaps the exception of USER and USING, since they have been shown on the backup and restore commands.

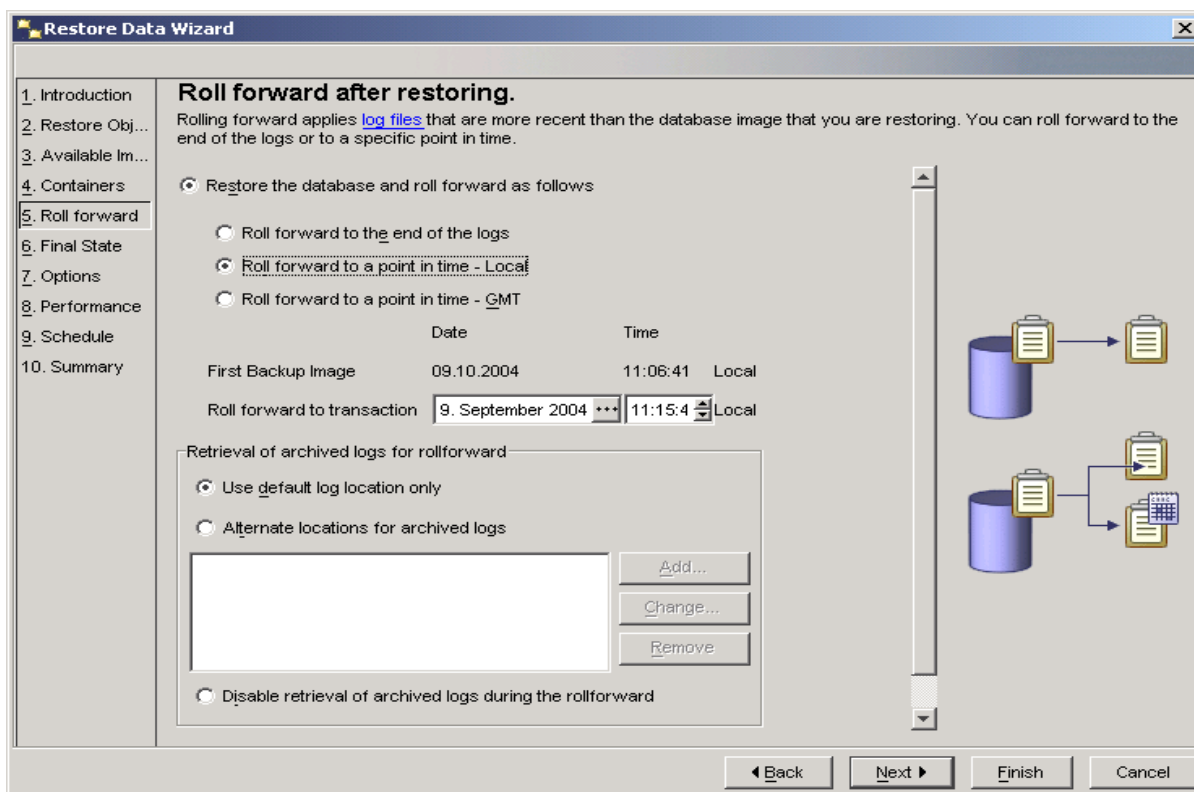
Emphasize the following points, which are not intuitively obvious.

- **isotime** is designated with editing symbols and is the coordinated universal time (CUT). This is different than the timestamp notation and value used in the RESTORE command. Universal time is used to avoid concerns regarding time zones that go on and off daylight savings time. This may concern some students, but remind them that point-in-time recovery is not typical, and the individual submitting roll forward commands should be well-versed in the mechanisms used.
- USING LOCAL TIME can also be specified, so the timestamp can be specified in local time on the server.
- END OF LOGS means to apply all the logs in the current logpath. If logs are contained in multiple paths, it is possible to apply the logs in one path, use OVERFLOW LOG PATH option to indicate the paths where log files can be found.
- AND STOP / AND COMPLETE or a subsequent ROLLFORWARD ... STOP needs to be specified in order to indicate to the database manager that the desired logs have been applied and that non-committed units of work need to be rolled back.
- TABLESPACE ONLINE is only applicable if specific table spaces are in roll forward pending. If the database is in the pending state, it is not available until the ROLLFORWARD process completes.
- ROLLFORWARD does not identify a table space or list of table spaces via the command. All table spaces (or the database) that are in the pending status will be rolled forward. If you only want to roll forward a subset of the table spaces in roll forward pending, you can provide a list of table spaces. But eventually, all of the table spaces will need to be rolled forward and taken out of roll forward pending so that they can be used.

**Additional information** — Certification objective: 6F Knowledge of Roll Forward recovery.

**Transition Statement** — Now let's look at the GUI for accomplishing the same RECOVERY.

## ROLLFORWARD — GUI interface



© Copyright IBM Corporation 2007

Figure 7-30. ROLLFORWARD — GUI interface

CF238.3

### Notes:

To access the Roll Forward Database GUI:

- Right-click and select the database
  - Left-click and select **Roll forward**

The roll forward GUI can also be accessed from the Restore Database Wizard. (The screen capture above is from the Restore Wizard.) The administrator can select roll forward to the end of logs or to a point in time. The point in time can be specified as local time on the client or on the server when using the GUI.

Use the Roll forward wizard to perform any of the following tasks:

- Roll forward a database or database partition
- Roll forward a table space or table space partition

**Roll Forward** — You will only be able to enter the Roll forward wizard if the object is in roll forward pending state. On this page, you can set roll forward parameters and specify where your log files are already stored so that they can be retrieved and used in the roll forward operation.

**Final State of Database** — Use this page to specify the state the database should be put in when the roll forward has completed. You can leave the database in roll forward pending state so that you can continue to apply logs to the restored image, or you can return the database to active state so that it is usable.

**Summary** — This page allows you to review the decisions that you made for your roll forward operation, and to view the command that will roll forward the objects in your database.



***Instructor notes:***

**Purpose** — Introduce the Roll forward wizard.

**Details** — The screen capture is of the roll forward in the restore wizard. The Roll forward wizard has only the Roll forward, Final state of the database, and Summary panels.

**Additional Information** — Notes taken from the task overview online help.

**Transition statement** — Let's examine the point of recovery in a little more detail.

## ROLLFORWARD — how far?



- End-of-logs
  - *End* means the end of the current log path
  - Other logs may need to be moved into the path
- Point-in-time (PIT)
  - Specified in Universal Time Coordinated (UTC) via command
  - Specified in local time *on server* with USING LOCAL TIME
  - Specified in local time on the client via GUI interface
  - Format: yyyy-mm-dd-hh.mm.ss.nnnnnn
- ONLINE backup requirements
  - Requires ROLLFORWARD past end of backup
  - Recovery history file (RHF) is useful
- Table space point-in-time considerations
  - Minimum roll forward time maintained for each table space — and the earliest possible point-in-time is the latest of the times in each of the relevant table spaces
  - Backup required after roll forward to establish a baseline for future backup/recovery

© Copyright IBM Corporation 2007

Figure 7-31. ROLLFORWARD — how far?

CF238.3

### Notes:

The database administrator can clear a ROLLFORWARD PENDING condition by issuing the ROLLFORWARD command. The point in time to which the roll forward stage proceeds is also controllable by the administrator. An administrator can roll forward to end of logs or to a specific point in time. Use the Coordinated Universal Time (CUT) or local time on the server when roll forward is specified in a command. Use the CUT, local time on the server, or local time on the client when roll forward is specified through the GUI.

The integrity of the database must be protected, therefore the earliest point in time at which the roll forward stage can end is the end of the online backup image.

Table space point-in-time recovery must protect the integrity of the relationships that exist between tables in the table spaces. These relationships include referentially constrained tables and single tables that have objects contained in multiple table spaces. A minimum roll forward time is kept at the table space level and can be displayed with the LIST TABLESPACES command. A backup is required following a point-in-time recovery of a table space.

The parameter **isotime** can be coded on the ROLLFORWARD command to identify a particular point-in-time up to which the logs should be applied. This time is specified as the ISO format of the coordinated universal time (CUT) or local time of the server.



### Important

The coordinated universal time is used in log records so that the database manager does not have a recovery dependency regarding daylight savings time or other local time anomalies. However, this introduces a degree of complexity for the database administrator. While backup images are identified via timestamps reflecting local time, rolling forward (since it applies to logs) designates times in CUT format. If specifying USING LOCAL TIME, care must be taken to indicate the time correctly, particularly if daylight savings time changes are close to the time of the roll forward.

In many cases, recovery will be desired to the most current point possible. Therefore, the END OF LOGS option will be commonly used.

The administrator can STOP / COMPLETE the roll forward process and allow access to the database.

AND STOP / AND COMPLETE is a necessary parameter to permit the database manager to roll back any transactions that are not complete after applying the log records to the indicated point. This is true even if END OF LOGS is utilized. Otherwise, the database will remain in ROLLFORWARD PENDING status.

## ***Instructor notes:***

**Purpose** — Provide emphasis regarding the point-in-time capabilities and limitations.

**Details** — Three key points to make:

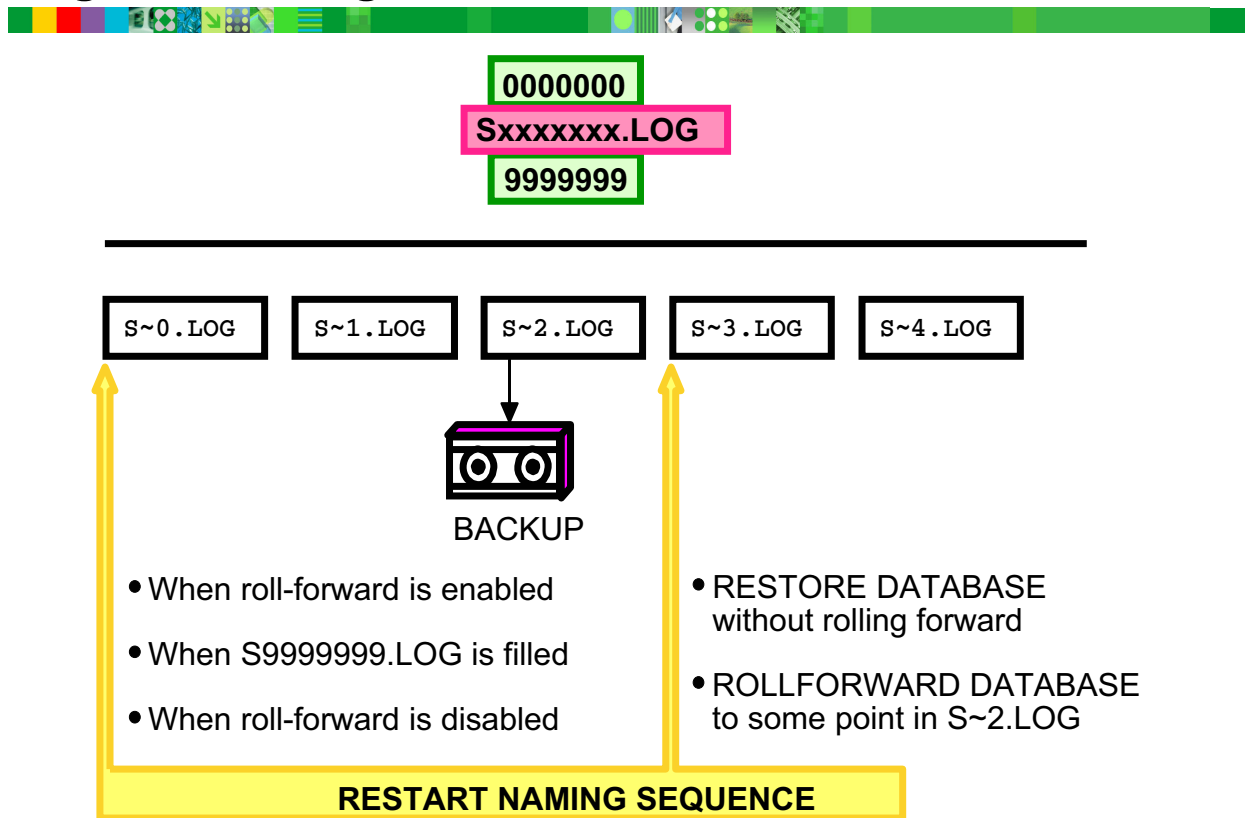
1. Table space point-in-time recovery requires a backup following the roll forward.
2. Point-in-time recovery is not typical.
3. If point-in-time recovery against an online backup is used, the database must be rolled forward to a point after the online backup was completed.

Note the different times that can be specified: they were indicated on the GUI screen capture of the roll forward, so could have been addressed there.

**Additional information** — Certification objective: 6E Knowledge of point in time recovery.

**Transition Statement** — Log files are an integral part of recovery support. Therefore, the location and naming of the log files is important to the DBA.

## Log file naming



© Copyright IBM Corporation 2007

Figure 7-32. Log file naming

CF238.3

### Notes:

The DB2 log file naming convention starts at **S0000000.LOG** and sequentially names logs until **S9999999.LOG** is reached. The higher numbers will only be used if archive logging is configured for the database. Otherwise, only the numbers corresponding to the actual number of primary and secondary logs for circular logging will be used.

The database manager will restart the naming sequence at a value different from **S0000000.LOG** after a successful **RESTORE** of a database configured for roll forward recovery. The actual name of the log file at which reuse starts depends on the recovery point. The log file that corresponds to the point of recovery is not reused. The next name will be reused. In the diagram above, a **RESTORE WITHOUT ROLLING FORWARD** to an **OFFLINE** backup created at the time the **S0000002.LOG** was active will cause **S0000003.LOG** to be the name at which reuse will start. The same name would also be reused if a **ROLLFORWARD** to a point in time within **S0000002.LOG** was completed.

Normally, the reuse of log names should not be a concern to the database administrator, since a restore with or without **ROLLFORWARD** establishes a new point of recovery. Therefore, log files associated with names greater than the point of recovery are not useful,

and reusing them is not a concern. An exception would be that the database administrator wants to retain the recoverability of multiple backups or the recoverability of a single backup to a later point in time. For example, assume that the backup in the above illustration is BACKUP1. Assume that a second backup, BACKUP2, was taken at some point that corresponds to the **S0000003.LOG**. If the database administrator restores to BACKUP1 and rolls forward to some point within **S0000002.LOG**, the name **S0000003.LOG** will be used as the next log after recovery. If the prior log of this name is not moved to a different directory, it will be overwritten during normal use of the database. Such reuse will make a future recovery to BACKUP2 (or BACKUP1 with a ROLLFORWARD to a point within the old **S0000003.LOG**) impossible. Again, this is not usually a concern because typical recovery strategies establish a new *starting* point after a restore is successful.

***Instructor notes:***

**Purpose** — Identify the log file naming scheme and the times when DB2 will reuse the names for archive logging.

**Details** — Cover the situation in the student notes in detail. It is important for the DBA to realize the mechanism for reuse of log file names. The concept is not that difficult, but the negative impact can be high if the DBA does not understand the naming convention and reuse.

**Additional information —**

**Transition Statement** — Now that we have addressed general backup and restore capabilities, let's ensure that you understand the implications for a disaster recovery.

## Disaster recovery considerations



*Should you use database recovery, table space recovery, or a combination?*

- Database recovery
  - Offline backup does not require a roll forward
  - Entire database can be restored on another system
  - Reduces backup management
- Table space recovery
  - Requires a roll forward
  - Backup afterwards may be required
  - Increases the number of backup images

© Copyright IBM Corporation 2007

Figure 7-33. Disaster recovery considerations

CF238.3

### Notes:

The term *disaster recovery* is used to describe the activities that need to be done to restore the database in the event of a fire, earthquake, vandalism, or other catastrophic events. A plan for disaster recovery can include one or more of the following:

- A site to be used in the event of an emergency
- A different machine on which to recover the database
- Offsite storage of database backups and archived logs

If your plan for disaster recovery is to recover the entire database on another machine, you require at least one full database backup and all the archived logs for the database. You may choose to keep a standby database up to date by applying the logs to it as they are archived. Or, you may choose to keep the database backup and log archives in the standby site, and perform restore and roll forward operations only after a disaster has occurred. (In this case, a recent database backup is clearly desirable.) With a disaster, however, it is generally not possible to recover all of the transactions up to the time of the disaster.

The usefulness of a table space backup for disaster recovery depends on the scope of the failure. Typically, disaster recovery requires that you restore the entire database; therefore,



a full database backup should be kept at a standby site. Even if you have a separate backup image of every table space, you cannot use them to recover the database. If the disaster is a damaged disk, a table space backup of each table space on that disk can be used to recover. If you have lost access to a container because of a disk failure (or for any other reason), you can restore the container to a different location.

Both table space backups and full database backups can have a role to play in any disaster recovery plan. The DB2 facilities available for backing up, restoring, and rolling data forward provide a foundation for a disaster recovery plan. You should ensure that you have tested recovery procedures in place to protect your business.

### ***Instructor notes:***

**Purpose** — Ensure that DBAs realize the need to use full database backups to support disaster recovery.

**Details** — When planning for disaster recovery, you need to ensure that you have a full database backup with which to start recovery on a remote machine. You cannot build a database from a set of table space backups. If the full database backup was an offline backup, you would not need to transport log files to the disaster recovery site.

**Additional Information** —

**Transition Statement** — Some additional thoughts.

## Questions to consider

- 
- Will the log files and the database files be on separate physical disks?
  - What medium can best handle the volume of log data to be generated?
  - Should copies of log files, backups, or both be made and managed?
  - Can the OS support mirroring? Should a mirror log path be defined?
  - How far back should recovery be enabled?
  - When will all log files and backups be discarded?
  - Can a user-exit sample be tailored or used?
  - Are paging space/swap files on the same device or drive as DB2 logs?

© Copyright IBM Corporation 2007

Figure 7-34. Questions to consider

CF238.3

### Notes:

No two installations will have identical recovery requirements, but the questions listed above are included to highlight some key areas that should be considered.

Particular consideration should be paid to the use of user-exit programs. Although these are supplied by IBM as samples, and are therefore not guaranteed regarding functionality, they can be good starting points. It is necessary to test the functionality at your installation. Consult the *Data Recovery and High Availability Guide and Reference* for additional details.

## ***Instructor notes:***

**Purpose** — Leave the students with questions that can only be addressed by considering a specific installation's hardware, applications, and recovery requirements.

**Details** — The first question: Should log files and database files be on separate physical disks? Hopefully, the answer to this question is *Yes*, both for recovery and performance reasons. Your log files tend to be the most active files on your system; thus you don't want them impacting the access to the data devices.

The next question also relates to the high activity of the log device.

Next, do you want to keep copies of log files or backups? Some shops keep backups at an offsite location, but also locally in case a disk drive failure occurs; they don't want to have to wait for a backup to be brought from a remote location.

Mirroring of log files is something you want to consider. In previous versions of DB2, you could only mirror through the operating system. Now, MIRRORLOGPATH gives another option. If you only have one copy of the log, and it is not available, your database is not available.

How far back to enable recovery? You may have backups from three years ago; but will you ever really use those? You may have audit requirements from your industry that requires a minimum retention period, but you should have a plan for when the log files and backups can be discarded.

If you have archival logging configured, you may want to look at using a user exit, tailoring the sample for the needs of your business.

Finally, look at where your log files are with regard to paging space or the swapper file; again, the log files are very active logs, so you don't want them on the same device as paging space, particularly if you are in a system that has paging occurring regularly.

## **Additional Information —**

**Transition Statement** — The next chart summarizes the key areas of recovery support.

## Logging/backup requirements summary



| <i>Consideration</i>         | <i>CRASH</i>        | <i>VERSION</i>      | <i>ROLL-FORWARD</i> |
|------------------------------|---------------------|---------------------|---------------------|
| Logging                      | Circular or Archive | Circular or Archive | Archive             |
| Backup                       | N/A                 | OFFLINE             | OFFLINE or ONLINE   |
| Resources                    | Low                 | Medium              | High                |
| Management                   | Low                 | Medium              | High                |
| Currency after Media Failure | Not Provided        | Latest Backup       | Last Committed Work |
| Table Space Recovery         | N/A                 | NO                  | YES                 |

© Copyright IBM Corporation 2007

Figure 7-35. Logging/backup requirements summary

CF238.3

### Notes:

The table above summarizes the key points regarding backup and logging. The descriptions of resources and management are relative. For example, some database administrators may consider management of an installation's roll forward recovery strategy quite simple, once the planning and implementation process has been completed. However, compared to supporting crash recovery, the cost is high.

***Instructor notes:***

**Purpose** — Summarize the key concepts presented.

**Details** — Use this chart to review what has been discussed up to this point.

**Additional Information** —

**Transition Statement** — We have just looked at the basic concepts of recovery, but there are many more things that can be done.

## Advanced recovery features

- On-demand log archiving
- Block transactions on log directory full
- Split mirror backup and suspended I/O
- Cloning a database
- Split mirror as standby database
- Split mirror as a backup image
- Incremental and delta backup
- Backup to named pipe
- Parallel rollforward recovery
- Relocating a database or a table space

© Copyright IBM Corporation 2007

Figure 7-36. Advanced recovery features

CF238.3

### Notes:

DB2 advanced recovery and logging features include:

- **On-demand log archiving**

You can close and archive the active log at any time. For recoverable databases, this feature lets you create a complete set of log files to at a specific time for such purposes as offsite storage. To do so, use the following command:

```
ARCHIVE LOG FOR DATABASE sample USER  
name USING password
```

If other applications have transactions in progress, you'll notice a slight performance degradation as the ARCHIVE LOG command flushes the log buffer to disk. If user exits are enabled, an archive request is issued after the logs are closed and truncated. Completion of the archive command does not guarantee that the logs have been moved to the archive directory, so you should check to make sure.

- **Block transactions on log directory full**

In earlier versions, DB2 would crash if the log directory was full and a new log file could not be created. In this version, DB2 will attempt to create the log every five minutes, while reporting the condition to the db2diag.log. This feature is controlled by the database configuration option BLK\_LOG\_DSK\_FUL.

This feature allows read-only applications to continue, provided that they are not dependent on rows locked by an update transaction. Applications that are updating data will not complete their operation until the log has been created.

- **Split mirror backup and suspended I/O**

To meet the need for increased availability, the following backup and recovery options may be implemented. Backup on very large databases requires substantial time and resources, and can interfere with continuous availability. Many customers cannot afford offline or online backups on a 1 TB live database.

Newer disk drive technology provides opportunities for high availability through fault tolerance and redundancy.

The online split mirror function allows a consistent mirror of a database to split off while update processing is temporarily suspended using the SET WRITE SUSPEND FOR DATABASE command. Recovery on the split copy is acceptable for consistency; backups and system copies can be done from a mirror image.

The db2inidb utility must be used on the split copy to indicate the intended use of the database. Use only logs from the primary database, not from the split image. You must run db2inidb on all partitions on a partitioned database environment before the split image can be accessed.

The command follows:

```
db2inidb <db_name> as snapshot
db2inidb <db_name> as standby
db2inidb <db_name> as mirror
```

Snapshot results in crash recovery, making the database consistent (a new log chain is started). You cannot roll forward through any of the logs from the original database. The database will be fully available at this time for any operation, including backup.

Standby places the database in a roll forward pending state; crash recovery is not performed, and the database remains in an inconsistent state with in-flight transactions still outstanding. This feature lets authorized users perform an offline backup of either the entire database or a subset of the table spaces.

- **Cloning a database**

A primary database can be cloned so that it can be used for read-only purposes such as running reports or for analysis purposes. The steps are as follows:

1. Suspend I/O on primary system: `db2 set write suspend for database.`



2. Split the mirror from the primary database using an operating system-level command.
3. Resume I/O on primary system: `db2 set write resume for database`. The database on the primary system should be back to a normal state.
4. Attach to the mirrored database from another machine.
5. Start database instance: `db2start`.
6. Start the DB2 crash recovery: `db2inidb <db_name> as snapshot`.

You can also clone a database for an offline backup, but you cannot use this backup to roll forward because the log chain will not match if restored on the primary system.

- **Split mirror as standby database**

For a standby database, the mirrored database is continually rolling forward through the logs. Even new logs created by the primary database are constantly fetched from the primary system. This results in higher availability and less recovery time. To execute, use the first four steps from the cloning process and add these steps:

5. Copy the logs: Set up a user exit program to retrieve log files from the primary system so that the latest logs will be available for this mirrored database.
6. Place the mirror in roll forward pending state, and roll forward the mirror: `db2inidb <db_name> as standby`. Remove the suspend write state and roll forward the database to the end of the logs.
7. Continue this process (copy and roll forward) until the primary database is down.

- **Split mirror as a backup image**

To use the mirrored system as a backup image to restore over the primary system:

1. Copy over the image: Use operating system commands to copy the mirrored data and logs on top of the primary system.
2. Start the database instance (`db2start`).
3. Place the mirror in roll forward pending state, and roll forward the mirror: `db2inidb <db_name> as mirror`.
4. Remove the suspend write state and roll forward the database to end of logs. The logs from the primary database must be used when rolling forward: `db2 rollforward database <db_name> to end of logs`.

- **Incremental and delta backup**

DB2 supports two types of smaller backups: incremental and delta. These backups provide the benefits of a smaller backup size. With a reduced amount of data read during the backup process and fewer log records read during recovery, you can run backups more frequently. This feature is available for backups at both the database and the table space level.

Cumulative backups contain all data changed since the last full backup. Delta backups are non-cumulative, and contain only the data changed since the last full or incremental backup.

- **Backup to named pipe**

Named pipes will provide additional backup targets and sources for DB2 restores such as compression. The named pipe for backup and restore must be created on the same machine in the local file system before backing up the database.

- **Parallel roll forward recovery**

Multiple agents will work on recovery on an SMP machine. Log records distributed to these agents can be concurrently reapplied, improving crash and database roll forward recovery. DMS table space containers can also be created or resized in parallel.

Although some components of recovery cannot be done in parallel, most can. Table space roll forward recovery will not use multiple agents. The log records will be parallelized at page level, meaning that the same agent will process the log records for the same data page. The most common log records (insert, delete, add key, update, and delete key) can run in parallel.

- **Relocate Database**

The command `db2relocatedb` renames a database, or relocates a database or part of a database (for example, container, log directory) as specified in the configuration file provided by the user. This tool makes the necessary changes to the DB2 instance and database support files.

To change the name of the database TESTDB to PRODDB in the instance DB2INST1 that resides on the path `/home/db2inst1`, create the following configuration file:

```
DB_NAME=TESTDB, PRODDB
DB_PATH=/home/db2inst1
INSTANCE=db2inst1
NODENUM=0
```

Save the configuration file as `relocate.cfg` and use the following command to make the changes to the database files:

```
db2relocatedb -f relocate.cfg
```

## Example 1

To move the database DATAB1 from the instance JSMITH on the path `/dbpath` to the instance PRODINST, do the following:

1. Move the files in the directory `/dbpath/jsmith` to `/dbpath/prodinst`.
2. Use the following configuration file with the `db2relocatedb` command to make the changes to the database files:

```
DB_NAME=DATAB1
DB_PATH=/dbpath
INSTANCE=jsmith,prodinst
NODENUM=0
```

## Example 2

The database PRODDB exists in the instance INST1 on the path `/databases/PRODDB`. The location of two table space containers needs to be changed as follows:

- SMS container `/data/SMS1` needs to be moved to `/DATA/NewSMS1`.
- DMS container `/data/DMS1` needs to be moved to `/DATA/DMS1`.

After the physical directories and files have been moved to the new locations, the following configuration file can be used with the `db2relocatedb` command to make changes to the database files so that they recognize the new locations:

```
DB_NAME=PRODDB
DB_PATH=/databases/PRODDB
INSTANCE=inst1
NODENUM=0
CONT_PATH=/data/SMS1,/DATA/NewSMS1
CONT_PATH=/data/DMS1,/DATA/DMS1
```

## Example 3

The database TESTDB exists in the instance DB2INST1 and was created on the path `/databases/TESTDB`. Table spaces were then created with the following containers:

```
TS1
TS2_Cont0
TS2_Cont1
/databases/TESTDB/TS3_Cont0
/databases/TESTDB/TS4/Cont0
/Data/TS5_Cont0
/dev/rTS5_Cont1
```

TESTDB is to be moved to a new system. The instance on the new system will be NEWINST and the location of the database will be `/DB2`. When moving the database, all of the files that exist in the `/databases/TESTDB/db2inst1` directory must be

moved to the `/DB2/newinst` directory. This means that the first five containers will be relocated as part of this move. (The first three are relative to the database directory, and the next two are relative to the database path.) Since these containers are located within the database directory or database path, they do not need to be listed in the configuration file. If the two remaining containers are to be moved to different locations on the new system, they must be listed in the configuration file. After the physical directories and files have been moved to their new locations, the following configuration file can be used with `db2relocatedb` to make changes to the database files so that they recognize the new locations:

```
DB_NAME=TESTDB
DB_PATH=/databases/TESTDB,/DB2
INSTANCE=db2inst1,newinst
NODENUM=0
CONT_PATH=/Data/TS5_Cont0,/DB2/TESTDB/TS5_Cont0
CONT_PATH=/dev/rTS5_Cont1,/dev/rTESTDB_TS5_Cont1
```

## Example 4

The database TESTDB has two partitions on nodes 10 and 20. The instance is SERVINST and the database path is `/home/servinst` on both nodes. The name of the database is being changed to SERVDB and the database path is being changed to `/databases` on both nodes. In addition, the log directory is being changed on node 20 from `/testdb_logdir` to `/servdb_logdir`. Since changes are being made to both nodes, a configuration file must be created for each node, and `db2relocatedb` must be run on each node with the corresponding configuration file. On node 10, the following configuration file will be used:

```
DB_NAME=TESTDB,SERVDB
DB_PATH=/home/servinst,/databases
INSTANCE=servinst
NODE_NUM=10
```

On node 20, the following configuration file will be used:

```
DB_NAME=TESTDB,SERVDB
DB_PATH=/home/servinst,/databases
INSTANCE=servinst
NODE_NUM=20
LOG_DIR=/testdb_logdir,/servdb_logdir
```

In a partitioned database environment, this tool must be run against every partition that requires changes. A separate configuration file must be supplied for each partition, including the `DBPARTITIONNUM` value of the partition being changed. For example, if the name of a database is being changed, every partition will be affected, and the `db2relocatedb` command must be run with a separate configuration file on each partition. If containers belonging to a single partition are being moved, the `db2relocatedb` command only needs to be run once on that partition.

***Instructor notes:***

**Purpose** — Identify some of the advanced recovery features.

**Details** — Do tell the students that they can learn details and gather hands-on experience about these and other advanced recovery topics by completing CF49, *DB2 Advanced Recovery and High Availability Workshop*.

**Additional Information** —

**Transition Statement** — If you need additional study in advanced recovery, there is a class for you. Let's look at the course description for CF49.

## DB2 Advanced Recovery and HA Workshop



### *CF49 — DB2 Advanced Recovery Workshop (4 days with Labs)*

- Recovery log management / User exit support
- Dropped table recovery
- Table space recovery
- Incremental backup and restore
- Database crash recovery
- Redirected restore
- Additional recovery facilities
- Recovery history file
- Disaster recovery of DB2 databases
- DB2 high availability and split mirror support
- DB2 partitioned database recovery considerations
- DB2 high availability disaster recovery (HADR)

© Copyright IBM Corporation 2007

Figure 7-37. DB2 Advanced Recovery and HA Workshop

CF238.3

### **Notes:**

You, or other people in your organization, may desire additional details. Please visit [www.ibm.com/services/learning](http://www.ibm.com/services/learning).

### **DB2 Advanced Recovery (CF49)**

#### Overview

Gain a deeper understanding of the advanced recovery features of DB2 for Linux, UNIX, and Windows environments with multiple partition databases. Get practical experience in the planning and utilization of a wide variety of DB2 recovery facilities, in a series of database recovery scenarios you complete during lab exercises using DB2 Enterprise Server Edition.

#### Skills taught

- Gain a better understanding of the unique recovery planning requirements for DB2 ESE.
- Explore the DB2 recovery facilities and database configuration options.
- Plan the implementation of a user exit for archival of database logs.

- Recover a DB2 table following a DROP TABLE command issued in error.
- Plan and execute the recovery of tablespaces to a selected point in time.
- Effectively utilize incremental backup and restore to reduce the size and duration of DB2 database backups.
- Gain a better understanding of DB2 crash recovery facilities.
- Utilize the redirected restore option to recover DB2 data to alternate disk configurations and invoke the db2relocatedb command to alter the configuration of a DB2 database.
- Execute recovery scenarios, including loss of DB2 log data using the DB2 log mirroring option.
- Utilize the information in the DB2 recovery history file to plan and execute various DB2 utilities.
- Understand the benefits and limitations of disaster recovery options for DB2 databases.
- Explore the options for operation of DB2 databases in high availability environments including the use of split mirrors of the database.
- HADR *High Availability and Disaster Recovery*

***Instructor notes:***

**Purpose** — Identify opportunity for further study.

**Details** —

**Additional information** —

**Transition Statement** — End of topic.



# Checkpoint

## Exercise — Unit Checkpoint

- \_\_\_ 1. State the three methods of recovery.

\_\_\_\_\_  
**Crash, version, roll forward recovery**

- \_\_\_ 2. What is required to avoid the need for a roll forward?

\_\_\_\_\_  
**Offline database backup  
Restore...without Rolling Forward**

- \_\_\_ 3. How might a database be placed in roll forward pending status?

\_\_\_\_\_  
**RESTORE online database backup  
RESTORE offline database backup omitting WITHOUT ROLLING FORWARD**

- \_\_\_ 4. How might a table space be placed in roll forward pending status?

\_\_\_\_\_  
**RESTORE a table space backup.  
DB2 detects a media failure isolated to a table space.**

- \_\_\_ 5. What is the minimum time to which a database must be rolled forward?

\_\_\_\_\_  
**To the end of the time of the online backup.  
a table space?**

\_\_\_\_\_  
**To the minimum roll forward time queried via LIST TABLESPACES.**

## Unit summary



Having completed this unit, you should be able to:

- Describe the major principles and methods for backup and recovery
- State the three types of recovery used by DB2
- Explain the importance of logging for backup and recovery
- Describe how data logging takes place, including circular logging and archival logging
- Use the BACKUP, RESTORE, and ROLLFORWARD commands
- Perform a table space backup and recovery
- Restore a database to the end-of-logs or to a point-in-time
- Discuss the configuration parameters and the recovery history file and use these to handle various backup and recovery scenarios

© Copyright IBM Corporation 2007

Figure 7-38. Unit summary

CF238.3

### Notes:

***Instructor Notes:***

**Purpose** — Review the unit objectives.

**Details** — Question the students on what they've learned.

**Transition Statement** — Now, go to the Backup and Recovery lab.



# Unit 8. Locking and concurrency

## Estimated time

00:40

## What this unit is about

This unit addresses the concepts of the DB2 database manager's implementation of locking to ensure data integrity while permitting multiple applications access to data. The database administrator needs a working knowledge of locking administration to successfully manage factors that influence the locking strategies used. Using this knowledge, the administrator can tailor the environment to meet the installation's concurrency requirements.

## What you should be able to do

After completing this unit, you should be able to:

- Explain why locking is needed
- List objects that can be locked
- Describe and discuss the various lock modes and their compatibility
- Explain four different levels of data protection
- Set isolation level and lock time out for current activity
- Explain lock conversion and escalation
- Describe the situation that causes deadlocks

## How you will check your progress

Accountability:

- Checkpoint
- Lab exercises

## References

*Administration Guide*

*System Monitor Guide and Reference*

## Unit objectives

After completing this unit, you should be able to:

- Explain why locking is needed
- List objects that can be locked
- Describe and discuss the various lock modes and their compatibility
- Explain four different levels of data protection
- Set isolation level and lock time out for current activity
- Explain lock conversion and escalation
- Describe the situation that causes deadlocks

© Copyright IBM Corporation 2007

Figure 8-1. Unit objectives

CF238.3

### **Notes:**

***Instructor notes:***

**Purpose** — Inform students of the importance of locking and management of parameters that impact locking.

**Details** — Each objective should be mentioned and related to the job of database administration.

**Additional information —**

**Transition Statement** — The first item of concern is “Why is locking necessary?”





## 8.1 Locking objects, strategies, and modes

**What students will do** — The students will identify objects locked by the database manager and define the locking strategies used. They will also be able to determine if two applications can acquire given combinations of locks on a single object at the same time.

**How students will do it** — The students will listen to the lecture and ask questions for clarification.

**What students will learn** — Basic concepts concerning locking that will aid in the understanding of other topics or reading of reference material.

**How this will help students on their job** — Locking concepts are required to understand many aspects of database administration. These concepts can be considered base knowledge from which to grow other skills.

## Why are locks needed?

- Ensure data integrity while permitting multiple applications to access data
- Prohibit applications from accessing uncommitted data written by other applications (*unless* Uncommitted Read isolation level is used)
- Control undesirable effects
  - Lost data due to concurrent updates
  - Unrepeatable reads
  - Phantom read phenomenon



© Copyright IBM Corporation 2007

Figure 8-2. Why are locks needed?

CF238.3

### Notes:

Because many users access and change data in a relational database, the database manager must be able both to allow users to make these changes and to ensure that data integrity is preserved. *Concurrency* refers to the sharing of resources by multiple interactive users or application programs at the same time.

The primary reasons why locks are needed are:

- **Ensure data integrity.** Stop one application from accessing or changing a record while another application has the record locked for its use.
- **Access to uncommitted data.** Application A might update a value in the database, and application B might read that value before it was committed. If the value of A is not later committed, but backed out, calculations performed by B are based on uncommitted (and presumably invalid) data. Of course, you may want to read even uncommitted data, for example to get a rough count of the number of records of a particular type without the guarantee of instantaneous precision. You can use an Uncommitted Read (UR) isolation level to do this — we will see more about this later.

The database manager controls this access to prevent undesirable effects, such as:

- **Lost updates.** Two applications, A and B, might both read the same row from the database and both calculate new values for one of its columns based on the data these applications read. If A updates the row with its new value and B then also updates the row, the update performed by A is lost.
- **Nonrepeatable reads.** Some applications involve the following sequence of events: application A reads a row from the database, then goes on to process other SQL requests. In the meantime, application B either modifies or deletes the row and commits the change. Later, if application A attempts to read the original row again, it receives the modified row or discovers that the original row has been deleted.
- **Phantom Read Phenomenon.** The phantom read phenomenon occurs when:
  1. Your application executes a query that reads a set of rows based on some search criterion.
  2. Another application inserts new data or updates existing data that would satisfy your application's query.
  3. Your application repeats the query from Step 1 (within the same unit of work).
  4. Some additional (phantom) rows are returned as part of the result set, but were not returned when the query was initially executed (Step 1).

## ***Instructor notes:***

**Purpose** — Emphasize that locking is implemented to support desirable goals.

**Details** — Allowing concurrent use of data is a requirement of most installations. However, the use of such data must be controlled when *competing* applications attempt to use the data in a manner that is not compatible. Note that for most of the items on the graphic (all except lost data), the choice of the isolation level will impact whether the negative effect is eliminated or allowed.

**Additional Information** — If a student has had past experience with locking in terms of problem resolution, they may feel that locking in general is a problem. However, the items on this visual are meant to emphasize that locking is necessary to ensure integrity. In addition, controlling access to uncommitted data and unrepeatable reads are *options* within the DB2 locking scheme, not requirements. Greater detail is the subject of discussing isolation levels.

Locks can be used by DB2 to:

- Prevent loss of data due to simultaneous update.
- Prevent access to data that is uncommitted if so desired.
- Allow data that has been read by an application to be protected from change if so desired.
- Prevent phantom reads.

The database administrator must understand DB2 locking so that the benefits of locking are realized without unnecessary impact on performance and concurrency.

**Transition Statement** — Let's look at how we can classify locks: object, mode, duration. We will first look at object and mode, and then, at the end of the unit, we will look at duration especially to obtain a lock and what happens if we have a *deadlock*.

## Classification of locks



Locks can be categorized based on the following attributes:

- **OBJECT** — what resource is locked?
- **MODE** — what type of lock is being used?
- **DURATION** — when will the lock be released?

and four different levels of protection to isolate data:

- **Uncommitted read (UR)**
- **Cursor stability (CS)**
- **Read stability (RS)**
- **Repeatable read (RR)**

Isolation levels will be discussed later in this unit

© Copyright IBM Corporation 2007

Figure 8-3. Classification of locks

CF238.3

### Notes:

Our discussion of locks is divided into a number of subtopics:

- **Object** — What resource is locked? — databases, table spaces, tables, rows
- **Mode** — What type of lock is being used? — we will look at the variety of table and row locks in detail, and how they interact
- **Duration** — When is the lock gained? and when will it be released?

Isolation levels are discussed in greater detail later in this unit as a separate topic.

## ***Instructor notes:***

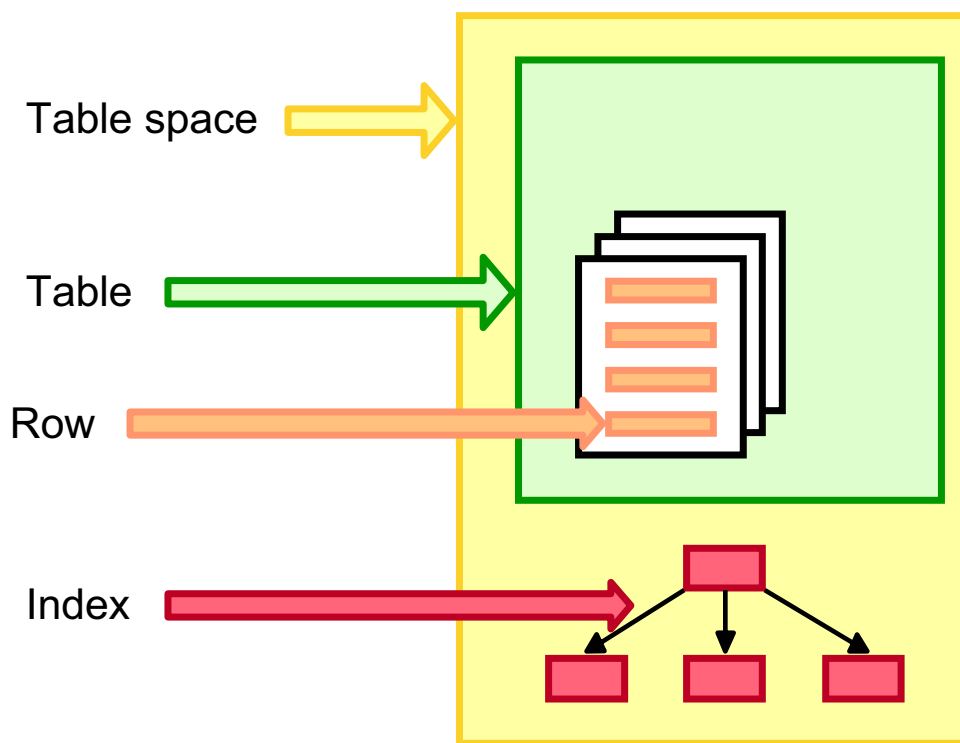
**Purpose** — Provide perspective on the broad issues related to locks.

**Details** — Locks are best seen from the three perspectives listed: object (what?), mode (how locked?), and duration (when? and how long?). We will elaborate on these perspectives over the next few visuals.

**Additional information** —

**Transition statement** — As our first step in understanding how DB2 prevents undesirable effects through locking, examine the objects that are subject to locks.

## Objects of locks



© Copyright IBM Corporation 2007

Figure 8-4. Objects of locks

CF238.3

### Notes:

DB2 utilizes locks on table spaces, tables, rows, and indexes to ensure data integrity while providing application concurrency.

Indexes and table space locks are shown on this visual in order to identify all objects that are subject to locking. However, the locks used on indexes and table spaces are not generally examined during problem determination for application processing. Objects that are the focus of such analysis include the table and row.

For example, a table space lock may be obtained to support a QUIESCE request. However, such a request is not considered typical application processing.

This topic will focus on table and row locking.

## ***Instructor notes:***

**Purpose** — Identify the objects that may be locked.

**Details** — Emphasize that table and row locks are of primary concern during lock analysis.

**Additional Information** — The visual identifies all the objects that may be locked by DB2 and also includes a *schematic* set of pages. The instructor should emphasize that page locking does **NOT** occur. Also, concentration on table and row locking is the focus of the unit. Table space locking is usually only seen during utility processing (for example, a restore requires exclusive use of the table space). Index locking is not explicit, and is used to support requirements internal to the product. Index integrity is protected by a locking algorithm at the row level during insert and delete activity. In effect, row locks are used to *cover* the integrity requirements for the indexes. Unlike some other database management systems, index locking is not a major concern during concurrency analysis.

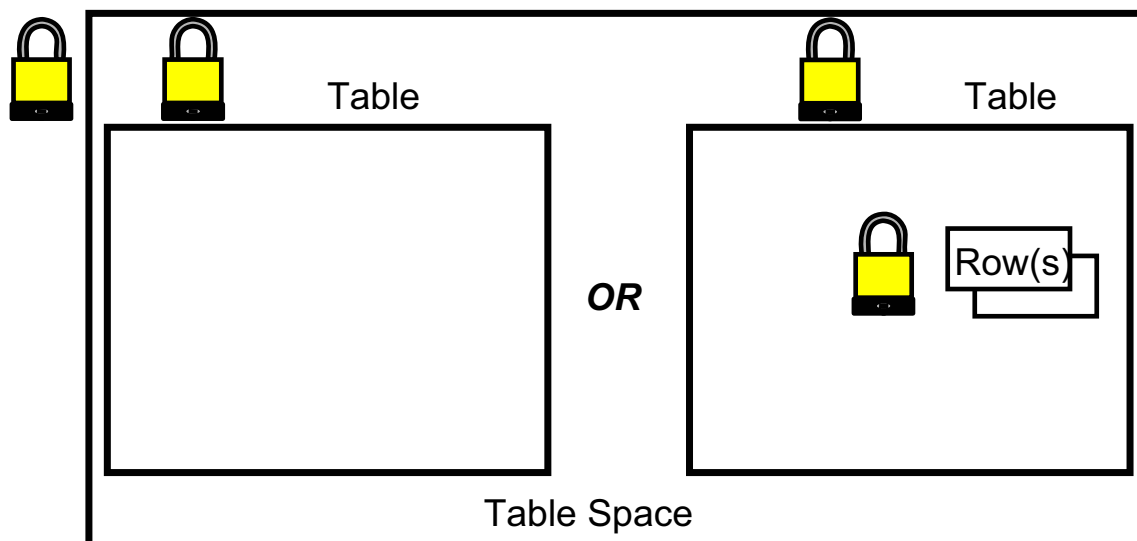
Table space locking with a greater degree of exclusivity (above IX and IS) may be an issue to support RESTORE activities, but such locking is not the subject of this unit.

The use of Type 2 indexes also reduces much of the need for next key locks. Type 2 indexes have been introduced with V8.

**Transition Statement** — Since table and row objects are the focus of application concurrency analysis, the next visual will examine the strategies involving these objects.



## Locking strategies



DB2 employs *either* strict table locking *OR* table locking in conjunction with row locking for typical application processing

© Copyright IBM Corporation 2007

Figure 8-5. Locking strategies

CF238.3

### Notes:

The above visual illustrates the locking strategies that DB2 utilizes for access to data.

The table space lock, if required, is acquired *first* in either strategy. It is shown on the visual for completeness, but will not generally be of concern when analyzing application concurrency due to the nature of the lock mode.

DB2 may lock only the table (sometimes called *strict table locking*), or may lock the table and a row or many rows.

- If the locking strategy is only on the table, then all rows in the table will be affected to the same degree.
- If the locking strategy is on the table and the underlying rows, then the entire table is locked with a lock that allows a great deal of concurrency at the table level, followed by locking of requested rows.



## Note

The mode of the lock (whether it will support READ or CHANGE) has been **intentionally** omitted from the discussion at this point. It will be addressed later in this unit.

***Instructor notes:***

**Purpose** — Detail the strategies used to support application locking.

**Details** — Point out:

- *Locks* have varying degrees of restrictiveness. However, leave the detail for other visuals.
- Table space locks for applications do not tend to conflict with one another. However, they are necessary to support the overall strategy of the database manager to support other processes, such as a QUIESCE on a table space.
- The text on the bottom of the visual is meant to emphasize that the focus of this unit is application processing, not DBA-initiated activity, such as a REORG (which can take a table space lock if the NOTRUNCATE option is omitted), RESTORE, or QUIESCE operation. (We will look at QUIESCE briefly, however, later, as an example of explicit locking.)

**Additional Information** — Basic knowledge of the hierarchy of locking (table space, to table, to row) will enable the student to comprehend other parts of this unit as well as product documentation.

**Transition Statement** — Some of you may be wondering “Why lock at the table level and then lock at the row?” Let’s look at lock modes to begin answering that question.

## Table lock modes



|     |                             |
|-----|-----------------------------|
| IN  | Intent None                 |
| IS  | Intent Share                |
| IX  | Intent eXclusive            |
| SIX | Share with Intent eXclusive |
| S   | Share                       |
| U   | Update                      |
| X   | eXclusive                   |
| Z   | superexclusive              |

Row Locking also used

Strict Table Locking

(See next page)

© Copyright IBM Corporation 2007

Figure 8-6. Table lock modes

CF238.3

### Notes:

The lock modes listed above are used by DB2 at the table level and are defined below:

- **IN — Intent None:** The lock owner can read any data in the table including uncommitted data, but cannot update any of it. Other concurrent applications can read or update the table. No row locks are acquired by the lock owner. Both table spaces and tables can be locked in this mode.
- **IS — Intent Share:** The lock owner can read any data in the locked table if an S lock can be obtained on the target rows. The lock owner cannot update the data in the table. Other applications can read or update the table, as long as they are not updating rows on which the lock owner has an S lock. Both table spaces and tables can be locked in this mode.
- **IX — Intent Exclusive:** The lock owner can read and update data provided that an X lock can be obtained on rows to be changed, and that a U or S lock can be obtained on rows to be read. Other concurrent applications can both read and update the table, as long as they are not reading or updating rows on which the lock owner has an X lock. Both table spaces and tables can be locked in this mode.

- **SIX — Share with Intent Exclusive:** The lock owner can read any data in the table and change rows in the table provided that it can obtain an X lock on the target rows for change. Row locks are not obtained for reading. Other concurrent applications can read the table. Only a table object can be locked in this mode. The SIX table lock is a special case. It is obtained if an application possesses an IX lock on a table and requests an S lock, or vice versa. The result of lock conversion in these cases is the SIX lock.
- **S — Share:** The lock owner and all concurrent applications can read but not update any data in the table and will not obtain row locks. Tables can be locked in this mode.
- **U — Update:** The lock owner can read any data in the table and may change data if an X lock on the table can be obtained. No row locks are obtained. This type of lock may be obtained if an application issues a SELECT... 'for update'. Other units of work can read the data in the locked object, but cannot attempt to update it. Tables can be locked in this mode.
- **X — Exclusive:** The lock owner can read or update any data in the table. Row locks are not obtained. Only uncommitted read applications can access the locked object. Tables can be locked in this mode.
- **Z — Super Exclusive:** This lock is acquired on a table in certain conditions, such as when the table is altered or dropped, or for some types of table reorganization. No other concurrent application can read or update the table. Tables and table spaces can be locked in this mode. No row locks are obtained.

The modes **IS**, **IX**, and **SIX** are used at the table level to **SUPPORT** row locks. They permit row-level locking while preventing more exclusive locks on the table by other applications.

The following examples are used to further clarify the lock modes of **IS**, **IX**, and **SIX**:

- An application obtains an IS lock on a table. That application may acquire a lock on a row for read only. Other applications can also **READ** the same row. In addition, other applications can **CHANGE** data on other rows in the table.
- An application obtains an IX lock on a table. That application may acquire a lock on a row for change. Other applications can **READ/CHANGE** data on other\* rows in the table.
- An application obtains an SIX lock on a table. That application may acquire a lock on a row for change. Other applications can **ONLY READ** other\* rows in the table.

The modes of **S**, **U**, **X**, and **Z** are used at the table level to enforce the strict table locking strategy. **No row-level locking** is used by applications that possess one of these modes.

The following examples are used to further clarify the lock modes of **S**, **U**, **X**, and **Z**:

- An application obtains an S lock on a table. That application can read any data in that table. It will allow other applications to obtain locks that support read-only requests for any data in the entire table. No application can **CHANGE** any data in the table until the S lock is released.
- An application obtains a U lock on a table. That application can read any data in that table, and may eventually change data in that table by obtaining an X lock. Other applications can only **READ** data in the table.

- An application obtains an X lock on a table. That application can read and change any or all of the data in the table. No other application can access data in the entire table for **READ\*** or **CHANGE**.
- An application obtains a Z lock on a table. That application can read and change any or all of the data in the table. No other application can access data in the entire table for **READ** or **CHANGE**.

The mode of **IN** is used at the table to permit the concept of Uncommitted Read. An application using this lock will **not** obtain row-level locks.

\* Denotes an exception to a given application scenario. Applications that use Uncommitted Read can read rows that have been changed. More details regarding Uncommitted Read are provided later in this unit.



### Note

Some of the lock modes discussed are also available at the table space level. For example, an IS lock at the table space level supports an IS or S lock at the table level. However, further details regarding table space locking are not the focus of this unit.

***Instructor notes:***

**Purpose** — Introduce the concept of lock modes.

**Details** — The intent locks support row locking while the strict table locks do not.

**Additional Information** — It is often helpful to display this visual in conjunction with the previous visual. Highlight the fact that the notes are fairly detailed. You should not attempt to review every note. You may wish to specifically highlight the note on Intent Exclusive and contrast it with the note on Exclusive. This will emphasize that intent locks imply that row locking is also being used, while strict table locking impacts all rows to the same degree.

You may wish to defer discussion of SIX until lock conversion is covered.

Do not get bogged down in too much discussion at this point, since the actual row locks have not yet been shown and the compatibility matrix is upcoming.

**Transition Statement** — At this point, it is not expected for all of you to have a clear picture of the locking strategies used by DB2. Let's uncover the next *level* of locking used IF the strategy involves row locks.

## Row lock modes

| Row Lock  |                         | Minimum* Supporting Table Lock |
|-----------|-------------------------|--------------------------------|
| <b>S</b>  | Share                   | <b>IS</b>                      |
| <b>U</b>  | Update                  | <b>IX</b>                      |
| <b>X</b>  | eXclusive               | <b>IX</b>                      |
| <b>W</b>  | Weak exclusive          | <b>IX</b>                      |
| <b>NS</b> | Next key Share          | <b>IS</b>                      |
| <b>NW</b> | Next key Weak exclusive | <b>IX</b>                      |

An application does not acquire  
if it is using Table Locks of

**Row locks**

**S, U, X, or Z**

© Copyright IBM Corporation 2007

Figure 8-7. Row lock modes

CF238.3

### Notes:

The above modes are for row locks. The definitions are similar to the definitions for corresponding table locks, except that the object of the lock is a row.

- **S — Share:** The row is being READ by one application and is available for READ ONLY by other applications.
- **U — Update:** The row is being READ by one application but is possibly to be changed by that application. The row is available for READ ONLY by other applications. The major difference between the U lock and the S lock is the INTENT TO UPDATE. The U lock will support cursors that are opened with the **FOR UPDATE OF** clause. Only one application can possess a U lock on a row.
- **X — Exclusive:** The row is being changed by one application and is not available for other applications, except those that permit Uncommitted Read.
- **W — Weak Exclusive:** This lock is acquired on the row when a row is inserted into a non-catalog table and a duplicate key for a unique index is encountered. The lock owner can change the locked row. This lock is similar to an X lock except that it is compatible with the NW lock.



- **NS — Next Key Share:** The lock owner and all concurrent applications can read, but not change, the locked row. Only individual rows can be locked in NS mode. This lock is acquired in place of a share (S) lock on data that is read with the RS or CS isolation levels.
- **NW — Next Key Weak Exclusive:** This lock is acquired on the next row when a row is inserted into the index of a non-catalog table. The lock owner can read, but not change, the locked row. This is similar to X and NX locks, except that it is compatible with the W and NS locks.

Row locks are only requested by applications that have supporting locks at the table level. These supporting locks are the INTENT locks: IS, IX, and SIX.

\* **Denotes** the least restrictive lock necessary. However, this does not imply that the table lock listed is the only table lock that supports the row lock listed. For example, an application that possesses an IX table lock could possess S, U, or X locks on rows. Likewise, an application that possesses a SIX table lock could possess X locks on rows.

## ***Instructor notes:***

**Purpose** — Define the row lock modes.

**Details** — Focus initially on the discussion of the three modes possible at the row level. Then, highlight the need for the supporting table lock. This is meant to *drive* home the point about table intent locks one more time.

**Additional Information** — Highlight the student note concerning the *minimum* supporting table lock. Also, you may wish to state, without going into detail as to how, that the U lock supporting updatable cursors can aid in deadlock prevention.

Next key locks are only obtained when actions will change index entries. The next key locks are obtained on the row **after** the affected row, in addition to a lock **on** the affected row.

**Transition Statement** — Now that we have defined the locking strategies and the lock modes used at the table and row levels, let's begin to bring the pieces of the puzzle together.

## Lock mode compatibility



| MODE OF LOCK A | MODE OF LOCK B |     |     |     |     |     |     |    |
|----------------|----------------|-----|-----|-----|-----|-----|-----|----|
|                | IN             | IS  | S   | IX  | SIX | U   | X   | Z  |
| IN             | YES            | YES | YES | YES | YES | YES | YES | NO |
| IS             | YES            | YES | YES | YES | YES | YES | NO  | NO |
| S              | YES            | YES | YES | NO  | NO  | YES | NO  | NO |
| IX             | YES            | YES | NO  | YES | NO  | NO  | NO  | NO |
| SIX            | YES            | YES | NO  | NO  | NO  | NO  | NO  | NO |
| U              | YES            | YES | YES | NO  | NO  | NO  | NO  | NO |
| X              | YES            | NO  | NO  | NO  | NO  | NO  | NO  | NO |
| Z              | NO             | NO  | NO  | NO  | NO  | NO  | NO  | NO |

**Table Locks**

**Row Locks**

| LOCK A MODE | MODE OF LOCK B |     |    |     |     |     |
|-------------|----------------|-----|----|-----|-----|-----|
|             | S              | U   | X  | W   | NS  | NW  |
| S           | YES            | YES | NO | NO  | YES | NO  |
| U           | YES            | NO  | NO | NO  | YES | NO  |
| X           | NO             | NO  | NO | NO  | NO  | NO  |
| W           | NO             | NO  | NO | NO  | NO  | YES |
| NS          | YES            | YES | NO | NO  | YES | YES |
| NW          | NO             | NO  | NO | YES | YES | NO  |

© Copyright IBM Corporation 2007

Figure 8-8. Lock mode compatibility

CF238.3

### Notes:

The symbols **A** and **B** in the above diagrams are used to represent two different applications. The chart regarding table locks can be used to determine if the two applications can run concurrently if they are requesting access to the same table with a given lock mode.

For example, if application A obtains an IS lock against a given table, application B could obtain an IN, IS, S, IX, SIX, or U lock against the same table at the same time. However, an X or Z lock would not be permitted at the same time.

This particular example illustrates the concept of the IS lock acting as a supporting lock for a lower level of locking. The only table locks that are not compatible are the X and Z locks, which would require exclusive use of the table. The presence of the IS lock indicates that a lower level of locking is required for this table, and the X or Z lock request is not given.

Study of the chart simply reinforces the definitions of table and row lock modes presented on the previous two pages. Review the row for IX under application A. Assume that application A obtains an IX lock on the table Y. This lock indicates that the application intends to obtain locks to support change at the row level. The application will allow other

rows to be read and updated, but will prevent access to the *target* rows (with the exception of Uncommitted Read applications.) Examine each of the possible *competing* table locks that application B might request:

- **IN** — No row lock intention. This lock is compatible. There will be no contention since application B is requesting Uncommitted Read. Even rows changed and not committed by application A are available. (The Z lock is the only mode that is not compatible with IN.)
- **IS** — Intent to lock for read only at the row level. This lock is compatible. There may be contention at the row level if application A is changing the same row that application B wants to read. The *Row Locks* table would need to be examined: if application A has acquired an X or a W lock on the row that application B is attempting to read, then application B will need to wait. Otherwise, the two applications can proceed with concurrency.
- **S** — Share lock at the table level. This lock is NOT compatible, since the S lock states that the entire table is available for READ ONLY by the application possessing the lock and all other applications. The IX lock states an intent to change data at the row level, which contradicts the requirement for READ ONLY. Therefore, application B could not obtain the S lock.
- **IX** — Intent to lock for change at the row level. This lock is compatible. There may be contention at the row level if application A is changing the same row that application B wants to change. The *Row Locks* table would need to be examined: if application A has acquired an X or a W lock on the row that application B is attempting to change, then application B will need to wait. Otherwise, the two applications can proceed with concurrency.
- **SIX** — The SIX lock states that a lock request for changing data may be required at the row level for the application possessing the lock. In addition, the rest of the table is available for READ ONLY applications. The IX lock implies change at the row level as well. Application B could not obtain the SIX lock on the table because of the S characteristic of the SIX lock, which is not compatible with the IX lock already assumed owned by application A.
- **U** — Read with intent to update. This table level lock states that the application possessing the lock may read any data, and may potentially exchange the U lock for an X lock. However, until this exchange is done, other applications can obtain locks supporting READ ONLY. Application B would NOT be able to obtain the U lock at the same time that application A possessed an IX lock on the same table.
- **X** — The application possessing this mode of lock on the table requires exclusive use of the table. No other access, with the exception of Uncommitted Read applications, is permitted. The IX lock possessed by application A would prevent application B from obtaining an X lock.
- **Z** — The application possessing this mode of lock excludes all other access to the table, including Uncommitted Read applications. Since application A has obtained an incompatible lock (IX), application B would not be able to obtain the Z lock at the same time.

The same type of statements could be logically derived for the other rows in the chart.

Many different applications could have compatible locks on the same object. For example, ten transactions may have IS locks on a table, and five different transactions may have IX locks on the same table. There is no concurrency problem at the table level in such a scenario. However, there may be lock contention at the row level:

- The basic concept of the row lock matrix is that rows being READ by an application can be READ by other applications, and that rows being changed by an application are not available to other applications that use row locking.
- Note that the U row lock is not compatible with another U row lock. Only one application can read a row with the INTENT TO UPDATE. This U lock reduces the number of deadlocks that occur when applications perform updates and deletes via cursors. When a row is FETCHED using a cursor declared **...FOR UPDATE OF...**, the U row lock is used.

## ***Instructor notes:***

**Purpose** — Summarize information presented so far and allow students to draw an analogy to something other than *terminology*. A primary purpose of this visual is to answer the question “Why do I have IX and IS locks?”

### **Details**

**Setup** — The house represents a table and the rooms represent rows within the table. You should be sensitive to stereotypes when you choose students to represent various roles to be defined. For the analogy to work, you need to make several points clear to the student:

1. Table space locking is not of concern. We are assuming no conflicts at that level. (If you wanted to illustrate the table space, you could draw a fence around the house. However, this is not necessary since the thrust of the unit is to consider application locking where the table space locks are not generally of concern.)
2. Tell students that you will only have an S lock while you are looking at a given room. (This emulates cursor stability; however, for flow reasons isolation level has not yet been addressed. Therefore, provide this as given information.)
3. Tell students that we will allow people to look at a room in the house provided that work is not being done in that room. (This may be the only *unrealistic* assumption for the analogy. You must provide this ground rule or some students will assume that change locking requires the whole table.)

For this page, it will be more interesting if you pick several students from the class and use their names in the example.

The *characters* for our scenario are:

- Owner (Due to some of the further activity that might be attributed to this student, if you are aware of someone in the class that you feel particularly comfortable teasing, they are the one you should pick for this position).
- Realtor
- Looker
- Plumber (or painter or workman)
- Inspector

Here's the premise: we are going to look at this house as if it is a table, and the rooms of the house as if they are rows. We want to examine what ***applications*** can coexist in the house, depending on the type of work they are doing.

Now, you might not know this, but *student 1* builds houses and sells them in addition to his/her work as a DBA in DB2. *Student 1* has built this house, and has now decided that it is about ready to sell. So, in order to sell the house, *student 1* decides to hire a realtor to come out to the house and show it to prospective buyers. *Student 1* calls his/her friend *student 2* the realtor to work as a realtor at the house.

Owner *student 1* tells realtor *student 2* the rules that he/she wants to be enforced at the house. Owner *student 1* wants prospective buyers to be able to look at the house, but the house is not quite finished. So, there may be workmen coming out to the house to finish up some of the details on the construction of the house. Owner *student 1* wants to ensure that any lookers will not be going into any room that a workman is working in. Realtor *student 2* agrees that he/she will enforce that rule, and goes out to the house.

Soon, *student 3* comes out to the house to look around. Realtor *student 2* needs to give a lock on the house (table) to enforce the rules that owner *student 1* wanted applied: that is, that lookers and workers could be in the house at the same time, but they couldn't be in the same room at the same time.

What kind of lock should Realtor *student 2* give to looker *student 3* to accomplish this?

Answer: IS lock on the table. (If students give other answers, write them down, and then go through each one with the reasoning for why they would not be appropriate.)

Write under the house on the board, *student 3's* name and IS.

Now, you might not know this but *student 3* is a gourmet cook: he/she subscribes to all the fancy cooking magazines and throws fabulous parties. So, he/she goes into the kitchen to see if all the appliances and facilities he/she needs to do his/her cooking are available.

Write on the kitchen an S for *student 3's* row lock on the kitchen.

Owner *student 1* knows that he/she has some additional work that needs to be done on the house: he/she knows that there is a leak under the sink in the master bath that needs to be fixed. So, he/she calls his/her plumber *student 4* to go out to the house to fix the leak under the sink in the master bath.

Plumber *student 4* shows up at the house and meets realtor *student 2* at the door.

Can the plumber *student 4* go into the house while *student 3* is looking around? What were the rules that owner *student 1* gave us for this?

The owner said that workers and lookers could both be in the house at the same time, as long as they were not in the same room at the same time.

So, plumber *student 4* should be able to work on the master bath while *student 1* is looking around, as long as they are not in the same room.

What kind of lock should realtor *student 2* give to plumber *student 4* for him/her to be able to fix the master bath?

Answer: IX lock on the table. (If students give other answers, write them down, and then go through each one with the reasoning for why they would not be appropriate.)

Write under the house on the board, *student 4's* name and IX, and then X on the master bath to indicate that *student 4* is updating the master bath.

While plumber *student 4* is fixing the master bath, could looker *student 3* walk over there to look at the master bath?

No: their lock modes at the row level would be incompatible.

Plumber *student 4* has finished his/her work on the master bath, and “commits” his/her work and leaves.

Erase the X lock on the master bath and the IX lock on the table.

Realtor *student 2* calls owner *student 1* and tells him/her that it looks like they have a really good prospect for someone to buy the house: this looker *student 3* has been looking for a long time, and may be a **hot prospect**.

Owner *student 1* knows that before he/she can sell the house (at least in Texas), it must be inspected. So, he/she calls his/her friend inspector *student 5* and asks *student 5* to go over to inspect the house. Now, *student 5* has dealt with *student 1* before, and knows that his/her houses are not always of the highest quality or completely finished when he/she goes over to inspect them. In order to save himself/herself time, he/she says to the owner, “Is there any work going on in the house right now?”

Owner *student 1* responds that there is no work going on in the house right now.

Inspector *student 5* says that he/she will not inspect the house if any workmen are in the house when he/she gets there.

So, inspector *student 5* drives out to the house and meets realtor *student 2* at the door. Inspector *student 5* says that he/she wants to be sure that there are no workers in the house while he/she is inspecting. Realtor *student 2* asks if it is ok if someone is looking at the house while the inspector is inspecting, and he/she says that is fine.

What kind of lock should realtor *student 2* give to inspector *student 5*?

Answer: S lock on the table. (If students give other answers, write them down, and then go through each one with the reasoning for why they would not be appropriate.) (Note that the inspector is not going to update the house in any way, just look at it.)

Write under the house on the board, *student 5*'s name and S on the table.

When inspector *student 2* gets into the family room, he/she finds a total mess. Wires are hanging from the ceiling, and the electricity is arcing between the wires, and the smell of ozone is in the air. He/she realizes that there is no way this house will ever pass inspection, and so leaves the house right then. He/she tells the realtor that there isn't much hope in the house ever selling.

Erase the S lock on the table for inspector *student 5*.

(Assure the looker *student 3* that the kitchen is really in very good shape, so that's why they haven't noticed any problems.)

So, inspector *student 5* calls owner *student 1* from his/her car phone and tells him/her that there is no way that this house will ever sell. “In fact, the best thing you could do is to just call our buddies over at...” (at this point, I draw a huge crane with a wrecking ball on it next to the house) “Acme Wrecking Company and have them come over and **drop** the house.”

Owner *student 1* tries to talk the inspector out of his/her assessment, but finds that they cannot be convinced, and so decides to **drop** the house.



What kind of lock would the Acme Wrecking company get to **drop** the house?

Answer: Z lock on the house.

Could they obtain that lock at this time?

No, because the looker is still in the house.

This illustrates why a table lock is obtained to indicate what kind of work is going on in the house (table). The realtor didn't have to look in all the rooms (rows) to determine if anyone was in the table, the IS lock on the table indicated that someone was in there. You may wish to expand this concept to a table with a million rows in it.

**Additional Information** — Other variations can include:

- Before the wrecking crew comes along, a neighbor kid comes over and doesn't go through the realtor, but is looking in the windows. The realtor sees him and indicates that with an IN lock on the table. You wouldn't want to wreck the house while the neighbor child is in the yard. The neighbor kid would have been able to look in the master bath while the plumber was doing their repairs.
- After the wrecking ball is there with a Z lock, suppose someone else came along who wanted to look at the house. What kind of lock would they need?

Answer: IS

Would they be allowed to look?

Well, it might look like they would because the IS is compatible with the IS that the looker already in there has. But, DB2 will make them wait behind the Z lock since it was there first.

**Transition Statement** — Let's look at isolation levels.



## 8.2 Isolation levels

### Instructor topic introduction

**What students will do** — The students will be able to describe the isolation levels available. They will also determine the impact of isolation on lock duration of row share locks.

**How students will do it** — The students will listen to the lecture and ask questions for clarification.

**What students will learn** — The definition and implication of the isolation levels that can be requested.

**How this will help students on their job** — The duration for holding share locks can impact the concurrency of a database system. By defining the function and locking requirements of the isolation levels available, the DBA can determine the proper isolation to recommend to application programmers or to use when binding production applications.

## Introduction to isolation levels

- 
- DB2 provides different levels of protection to isolate data:
    - Uncommitted read (UR)
    - Cursor stability (CS) - default
    - Read stability (RS)
    - Repeatable read (RR)
  - Cursor stability is the default isolation level
  - Isolation level can be specified for a session, a client connection, or an application before a database connection
    - For embedded SQL, the level is set at bind time
    - For dynamic SQL, the level is set at run time

---

© Copyright IBM Corporation 2007

Figure 8-9. Introduction to isolation levels

CF238.3

### **Notes:**

This visual sets the stage for our discussion on isolation levels.

The isolation level chosen for an application can impact both the lock strategy and the duration of row locks. The isolation level can be specified during program preparation or bind.

Cursor stability is the default.

***Instructor notes:***

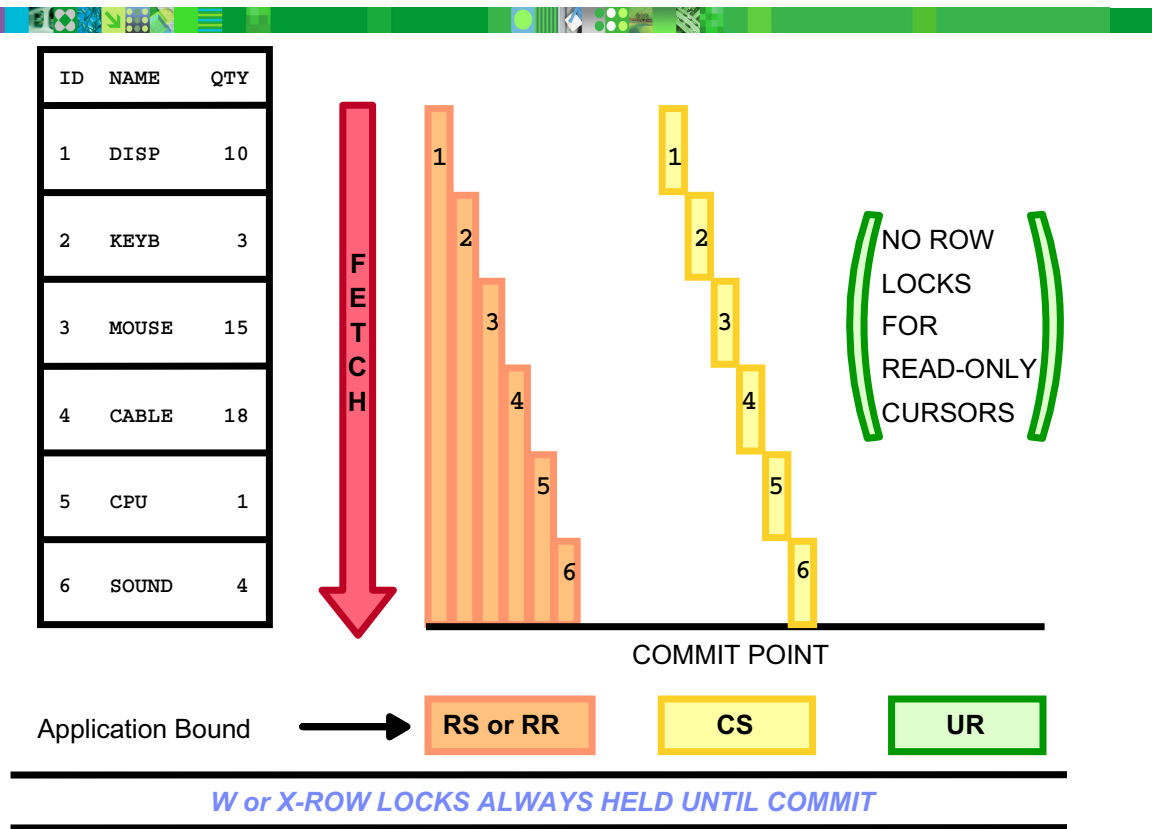
**Purpose** — Provide an introduction to the topic of isolation levels.

**Details** — Here we are naming the four levels. Details follow in the coming visuals and accompanying text.

**Additional information** —

**Transition statement** — Let's now look at these isolation levels in more detail.

## Isolation levels



© Copyright IBM Corporation 2007

Figure 8-10. Isolation levels

CF238.3

### Notes:

This visual details the impact of isolation level on row lock duration. For the following notes, assume that row locks are being obtained:

- **Read Stability (RS):** NS row locks are held until the next commit or rollback. If a row is read multiple times within a unit of work, the value for the row will not change. However, the result set can change if a cursor that has been processed is subsequently reopened with the same search criteria within an unit of work. This is because Read Stability does not prevent new rows from being *added* to the result set via insert or update activity.

This isolation level reduces concurrency at the row level, but may be useful for applications that require examination of multiple rows before a processing decision can be made.

- **Repeatable Read (RR):** S row locks are held until the next commit or rollback. If a row is read multiple times within a unit of work, the value for the row will not change. Also, the result set will not change if a cursor that has been processed is subsequently reopened using the same search criteria within a unit of work.

This isolation level reduces concurrency at the row level, but may be useful for applications that require examination of multiple rows before a processing decision can be made and also requires the result set to be static if the same criteria is used to open a cursor a second time within a unit of work.

- **Cursor Stability (CS):** NS row locks are held only while the cursor is positioned on the row. Rows that have been previously examined within a unit of work are not locked.

This isolation level increases concurrency at the row level, and it is normally preferred for most application requirements.

- **Uncommitted Read (UR):** S row locks are not obtained when single rows are read or when READ-ONLY cursors are OPENED and FETCHED. A cursor is READ-ONLY if:
  - The result table for the SELECT statement is read-only. For example, the SELECT statement contains an ORDER BY clause. (Other syntax constructs that cause a result table to be read-only are documented in the *SQL Reference*.)
  - The cursor is declared with FOR FETCH ONLY.

For non read-only cursors, the locking behavior is the same as cursor stability.

This isolation level may improve performance of applications since waits for S row locks are eliminated. However, the application programmer must be aware of any data integrity risks associated with this isolation level.

Isolation level applies only to rows that are read. Rows that are changed require an X or a W lock to be acquired by the application. **X or W locks are not released until rollback or commit, regardless of the application's isolation level.**

If a temporary table is needed to support an application using cursor stability, the S row locks are obtained and released during the OPEN of the cursor. FETCH from the temporary table may therefore present a row to the application that has since been changed or deleted in the underlying base table. Temporary tables may be used for read-only cursors that require sort activity. The use of temporary tables is externalized via the EXPLAIN function.



### Note

#### SET CURRENT ISOLATION

To change the isolation level for a specific connection session, you can update a special registry variable. This is covered in an upcoming visual.

## ***Instructor notes:***

**Purpose** — Stress the impact of isolation on the duration of S-row locks.

**Details** — Isolation is specified at precompile and can be overridden at bind. However, the specification can impact concurrency to a great extent. The database administrator must be aware of the possible negative concurrency impact of using RS or RR, and the possible integrity exposure of using UR.

**Additional Information** — Many students do not understand that isolation only impacts the duration of S locks on rows. Emphasize the text at the bottom of the visual. Also, the note concerning the impact of temporary tables should be pointed out, even though this concept is covered in detail in programming classes. This is necessary because this particular area is often not clear to the DBA and programmer alike.

**Transition Statement** — Although Read Stability and Repeatable Read appear to be identical in the visual, let's examine how they differ.



## Repeatable Read versus Read Stability



- Repeatable Read (RR)
  - Locks all the rows an application references within a unit of work — *every row referenced is locked*
  - If the application issues the same query (SELECT) more than once within a unit of work, the same result set is obtained each time
  - No other application can UPDATE, DELETE, or INSERT a row that would affect the result table until the unit of work complete
- Read Stability (RS)
  - Locks all the rows an application retrieves within a unit of work — *only rows that qualify are locked*
  - If the application issues the same query more than once within a unit of work, additional *phantom rows* may be returned

© Copyright IBM Corporation 2007

Figure 8-11. Repeatable Read versus Read Stability

CF238.3

### Notes:

RR and RS both lock all the rows an application retrieves within a unit of work.

Using repeatable read, a SELECT statement issued by an application more than once within a unit of work gives the same result each time. No other application can update, delete, or insert a row that would affect the result set until the unit of work completes. Every row that is referenced is locked, not just the rows that are retrieved, preventing *phantom rows* from occurring. Thus if you scan 10,000 rows and apply predicates to them, locks are held on all 10,000 rows, even though only 10 rows may qualify.

Using read stability, if your application issues the same query more than once, you may see additional *phantom rows*. A phantom row can occur in the following example situation:

1. Application 1 reads the set of n rows that satisfy a search condition.
2. Application 2 then inserts one or more rows that satisfy the search condition.
3. Application 1 reads the set of rows again with the same search condition, and obtains both the original rows and the rows inserted by application 2.

RS locks only the rows that qualify — perhaps just 10 out of the 10,000 rows in the table.

### ***Instructor notes:***

**Purpose** — Differentiate between the Repeatable Read and Read Stability isolation levels.

**Details** — See student notes; use an example of the number of rows locked if only 10 rows qualify. You may want to note that the difference is due to the access strategy that DB2 uses: if DB2 had an index to acquire the rows of interest, then only the rows returned would be locked, regardless of whether the isolation level was Repeatable Read or Read Stability.

### **Additional information —**

**Transition Statement** — Let's review the negative effects that locking should help us with, and how each isolation level impacts the control of the negative effect.

## Summary of isolation levels



| Isolation level              | Access to uncommitted data | Nonrepeatable reads | Phantom read phenomenon |
|------------------------------|----------------------------|---------------------|-------------------------|
| <b>Repeatable Read (RR)</b>  | Not possible               | Not possible        | Not possible            |
| <b>Read Stability (RS)</b>   | Not possible               | Not possible        | Possible                |
| <b>Cursor Stability (CS)</b> | Not possible               | Possible            | Possible                |
| <b>Uncommitted Read (UR)</b> | Possible                   | Possible            | Possible                |

© Copyright IBM Corporation 2007

Figure 8-12. Summary of isolation levels

CF238.3

### Notes:

The table summarizes the different isolation levels in terms of their impact on undesirable effects. Note that all of the isolation levels protect against lost data due to concurrent updates.

### ***Instructor notes:***

**Purpose** — Review the impact of the isolation levels on the undesirable effects.

**Details** — This chart should be used as a review, looking back at the first graphic of the positive effects of locking. This now summarizes how each of the isolation levels helps to support the desired effect.

**Additional Information** —

**Transition Statement** — We promised you (in a “Note”) that we would look at how to set isolation levels even in the middle of a unit of work / transaction. This is next.

## SET CURRENT ISOLATION

- Assigns a value to the CURRENT ISOLATION special register  

```
SET [CURRENT] ISOLATION [=] {UR | CS | RR | RS | RESET}
```
- CURRENT ISOLATION can be set anytime while connected to a database (special registers are attributes of the connection session)
- Applies to the current session only
- The register setting is used immediately for subsequent SQL statements *until value is set again*, thus allowing different isolation to be applied to different SELECT (and other) statements even within one UOW/transaction
- You can read the current register setting with:

**VALUES CURRENT ISOLATION**

© Copyright IBM Corporation 2007

Figure 8-13. SET CURRENT ISOLATION

CF238.3

### Notes:

The SET CURRENT ISOLATION statement assigns a value to a special registry. This statement can be embedded in an application program or issued through the use of dynamic SQL statements. The registry setting is used immediately for following SQL statements until the value is set again, thus allowing, for example, different isolation to be applied to different SELECT statements, even within one Unit of Work.

The SET CURRENT ISOLATION statement is not under transaction control. The register setting is retained across COMMIT boundaries.

This statement can be embedded in an application program or issued through the use of dynamic SQL statements. It is an executable statement that can be dynamically prepared. No particular authorization is required to execute this statement.

The syntax of the statement (as shown is the visual above) is:

```
SET [CURRENT] ISOLATION [=] {UR | CS | RR | RS | RESET }
```

The value of the CURRENT ISOLATION special register is replaced by the specified value or set to blanks if RESET is specified.

The following syntax is also supported:

- The word `CURRENT` is optional \*
- The equals sign is optional — but `TO` can be specified in place of the equal sign (`=`) \*
- `DIRTY READ` can be specified in place of `UR` \*
- `READ UNCOMMITTED` can be specified in place of `UR` \*
- `READ COMMITTED` is recognized and upgraded to `CS` \*
- `CURSOR STABILITY` can be specified in place of `CS` \*
- `REPEATABLE READ` can be specified in place of `RR` \*
- `SERIALIZABLE` can be specified in place of `RR`

The asterisk items (\*) are for compatibility with IBM Informix SQL syntax.

***Instructor notes:***

**Purpose** — Make the students aware of the SET CURRENT ISOLATION special registry which allows, for example, the use of different isolation levels for different iterations of SELECT/INSERT/UPDATE within one unit of work.

**Details** — The SET ISOLATION statement is a feature of IBM Informix Dynamic Server (IDS). The ability to set current isolation at any time, regardless of where you are in a unit of work / transaction, was added to DB2 UDB in V8.2 both for compatibility with Informix and because it was a good idea.

**Additional information —**

**Transition statement** — So far, we have seen implicit locking. We can also explicitly lock various objects. Let's see the differences between explicit and implicit locking.





## 8.3 Explicit versus implicit locking

### Instructor topic introduction

**What students will do** — The student will learn about explicit locking and be able to differentiate it from the implicit locking that they have learned about so far.

**How students will do it** — Lecture and discussion.

**What students will learn** — The student will learn that certain objects can be explicitly locked and how to apply these locks. The locks that have been investigated so far are implicit locks and are obtained by DB2 through other actions such as SELECT, INSERT, UPDATE, and DELETE.

We will also see the effects of lock conversion and escalation.

**How this will help students on their job** — Explicit locking is frequently needed to obtain exclusive access to the database itself or to particular objects within the database.

## Explicit versus implicit locking



- Locking is controlled by isolation level
- By default, DB2 uses row-level locking
- Database, table spaces, and tables can be explicitly locked
- Database, tables, and rows can be implicitly locked

© Copyright IBM Corporation 2007

Figure 8-14. Explicit versus implicit locking

CF238.3

### Notes:

The lock mode that is used by an application is determined by the isolation level. Locks are placed on the table or on individual rows, or on *pages* of rows in the table. DB2 for Linux, UNIX, and Windows does not support page level locking. The default used by DB2 is row locking.

Databases, table spaces, and tables can be **explicitly** locked. Here are some examples of commands that can be used to explicitly lock these database objects:

- Database lock

```
CONNECT TO database IN EXCLUSIVE MODE
```

- Table space lock

```
QUIESCE table-spaces FOR TABLE table-name INTENT FOR UPDATE
```

- Table lock

```
LOCK TABLE table-name IN EXCLUSIVE MODE
```

The LOCK TABLE statement in the subject of the next visual.

Database, tables, and rows can be *implicitly* locked. For example:

- Database is locked during full database restore
- Tables are locked during lock escalation
- Rows are locked through normal data modification (SELECT, INSERT, UPDATE, and DELETE)

## ***Instructor notes:***

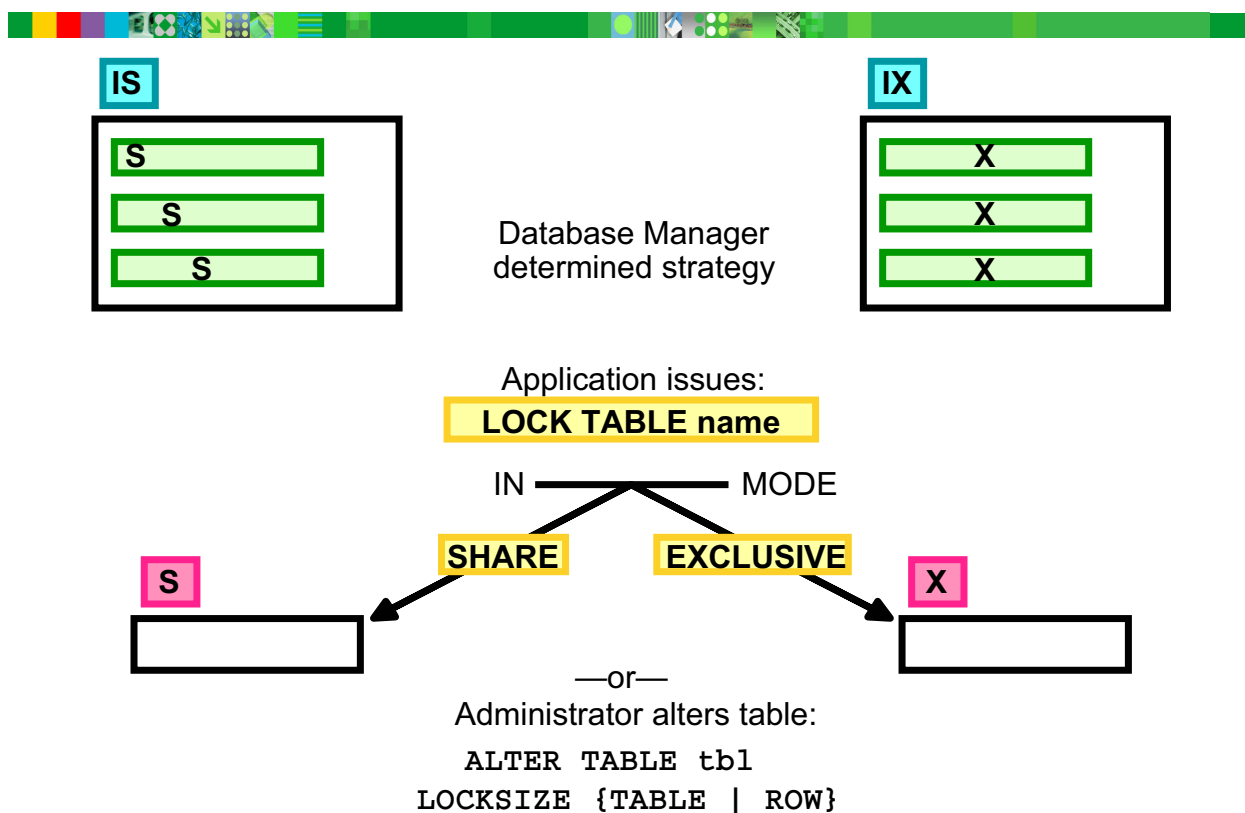
**Purpose** — Distinguish between explicit and implicit locking.

**Details** — Previous to this, the locking that we have investigated is implicit locking. Here we look at explicit locking and the SQL statements required to perform explicit locking.

**Additional information** —

**Transition statement** — Now let us look at the LOCK TABLE statement that provides us either shared or exclusive access to an individual table.

## LOCK TABLE statement



© Copyright IBM Corporation 2007

Figure 8-15. LOCK TABLE statement

CF238.3

### Notes:

Isolation level and access strategy are factors that affect the database manager when it determines the locking strategy to use when reading or manipulating data. **Generally**, intent locks at the table level, and row locking, are used to support transaction-oriented applications.

However, the use of intent locking may not be appropriate for a given application.

The LOCK TABLE statement provides the application programmer with the flexibility to lock a table at a more restrictive mode than requested by the database manager. Only applications with a need for a more restrictive mode of lock should issue the LOCK TABLE statement. Such applications could include report programs that must show *snapshots* of the data at a given point in time, or data modifying programs that normally don't make changes to significant portions of a table except during certain periods such as for month-end processing.

SHARE MODE allows other processes to SELECT data in the TABLE, but does not allow INSERT, UPDATE, or DELETE operations.

EXCLUSIVE MODE prevents any other processes from performing any operation on the table, with the exception of Uncommitted Read applications.

Locks obtained via the LOCK TABLE statement are acquired when the statement is executed. These locks are released by commit or rollback.



### Note

The visual is not intended to imply that an application can only request a more restrictive table lock of the same nature (IS to S / IX to X), although this would be the typical case.

The table can also be altered to indicate the size (granularity) of locks used when the table is accessed. If LOCKSIZE TABLE is indicated, then the appropriate share or exclusive lock is acquired on the table, and intent locks (except intent none) are not used. Use of this value may improve the performance of queries by limiting the number of locks that need to be acquired. However, concurrency is also reduced since all locks are held over the complete table.

Even though the intent lock strategy is common for typical transaction-oriented applications, there are situations when strict table locking will be selected by the database manager. For example, the isolation level of Repeatable Read combined with an access strategy of TABLE SCAN or INDEX SCAN with no WHERE clause will be supported with a strict table lock. If the strict table locking that results causes unacceptable concurrency problems, the applications using Repeatable Read should be examined to determine if a different access strategy can be used or if the isolation level can be changed. Repeatable Read can be logically simulated, although the application code required to do so may carry a high cost for development, maintenance, or both.

Strict table locking cannot be avoided when issuing DDL against a table or index. When possible, the database administrator should restrict submission of such statements to periods of low activity.

In any case, strict table locks determined during the optimization process are externalized by the Explain function.

***Instructor notes:***

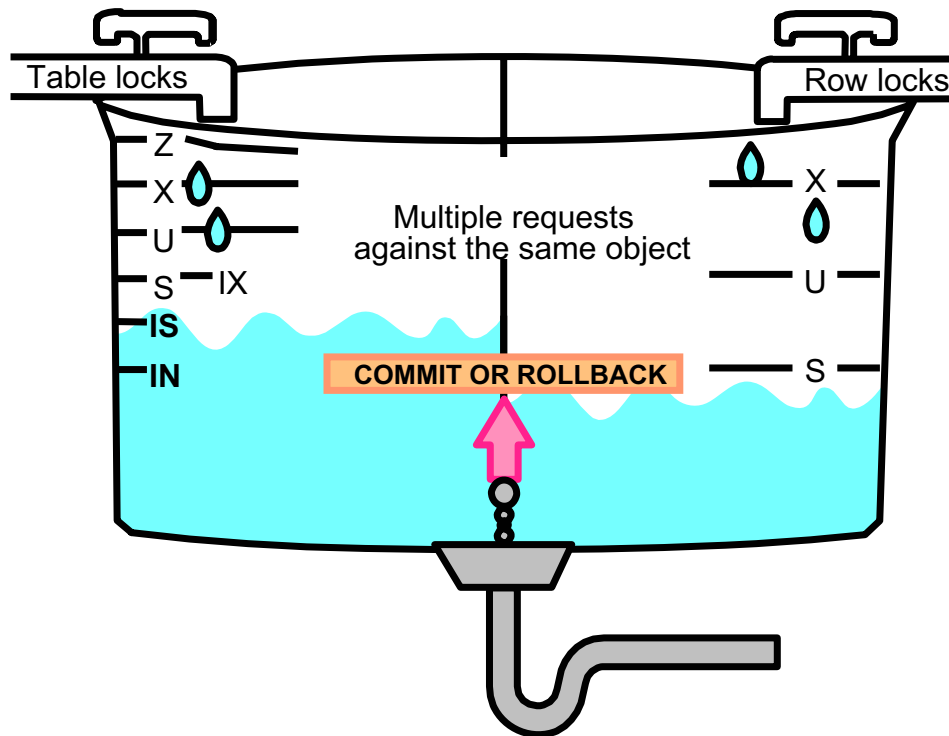
**Purpose** — Inform the students that the normal transaction locking strategy is intent locking with row locks. However, this is not always the desired strategy.

**Details** — The LOCK TABLE statement is the only EXPLICIT method to request a lock within application programs. The student notes suggest examples of when such a request may be appropriate. The administrator can also override the default row locking strategy for a given table with the ALTER TABLE LOCKSIZE statement.

**Additional Information** — The students must be aware that the use of the LOCK TABLE or ALTER TABLE LOCKSIZE statement is only acceptable after the impacts on concurrency have been carefully analyzed. Also, it is relevant to identify that there are cases when the database manager may determine that strict table locking is appropriate. The student notes identify a common example: Repeatable Read combined with a TABLE SCAN. A key thought to leave with the student is that you don't need to memorize every case where strict table locking is used. Exploit EXPLAIN to indicate when it has been chosen and then analyze the particular case.

**Transition Statement** — The table locking needed by each statement in an application is externalized via Explain. However, there may be multiple locking requirements against the same object within a unit of work. This may cause lock conversion.

## Lock conversion



© Copyright IBM Corporation 2007

Figure 8-16. Lock conversion

CF238.3

### Notes:

Although the Explain tool provides information regarding the strategy determined at bind, the information provided is on a statement-by-statement basis. The overall locking requirement of an application on a table must be derived by combining knowledge of the application with the output of the Explain tool.

Lock conversion is the process of obtaining a lock with a greater degree of restrictiveness as SQL statements are processed within a unit of work that require different lock modes on the same object.

Lock modes convert to higher modes only and remain at the higher mode until commit or rollback. (This is not to imply that locks acquired under cursor stability isolation level will be held longer than previously described.) After this point, table and row locks requested in the next unit of work will establish a new *high water mark*.

The SIX table lock is a special case. It is obtained if an application possesses an IX lock on a table and requests an S lock, or vice versa. The result of lock conversion in these cases is the SIX lock.



As an example of table lock conversion, consider the following application (assume RR for isolation and IXCOL = ? utilizes an INDEX SCAN):

| Statement                            | Table lock required for statement (externalized via Explain) | Conversion required? | Resultant lock   |
|--------------------------------------|--------------------------------------------------------------|----------------------|------------------|
| SELECT ... FROM A<br>WHERE IXCOL = ? | IS                                                           | N/A                  | IS               |
| SELECT ... FROM A                    | S                                                            | Yes                  | S                |
| SELECT ... FROM A<br>WHERE IXCOL = ? | IS                                                           | NO                   | S                |
| UPDATE A SET ...<br>WHERE IXCOL = ?  | IX                                                           | Yes                  | SIX              |
| COMMIT                               | N/A                                                          | No                   | (locks released) |
| SELECT ... FROM A<br>WHERE IXCOL = ? | IS                                                           | N/A                  | IS               |
| LOCK TABLE A IN<br>EXCLUSIVE MODE    | X                                                            | Yes                  | X                |
| UPDATE A SET ...<br>WHERE IXCOL = ?  | IX                                                           | No                   | X                |
| ROLLBACK                             | N/A                                                          | No                   | (locks released) |

All locks (row and table) are released at commit or rollback, with the exception of table locks associated with cursors declared WITH HOLD. These locks are held through commit.

## ***Instructor notes:***

**Purpose** — Define lock conversion.

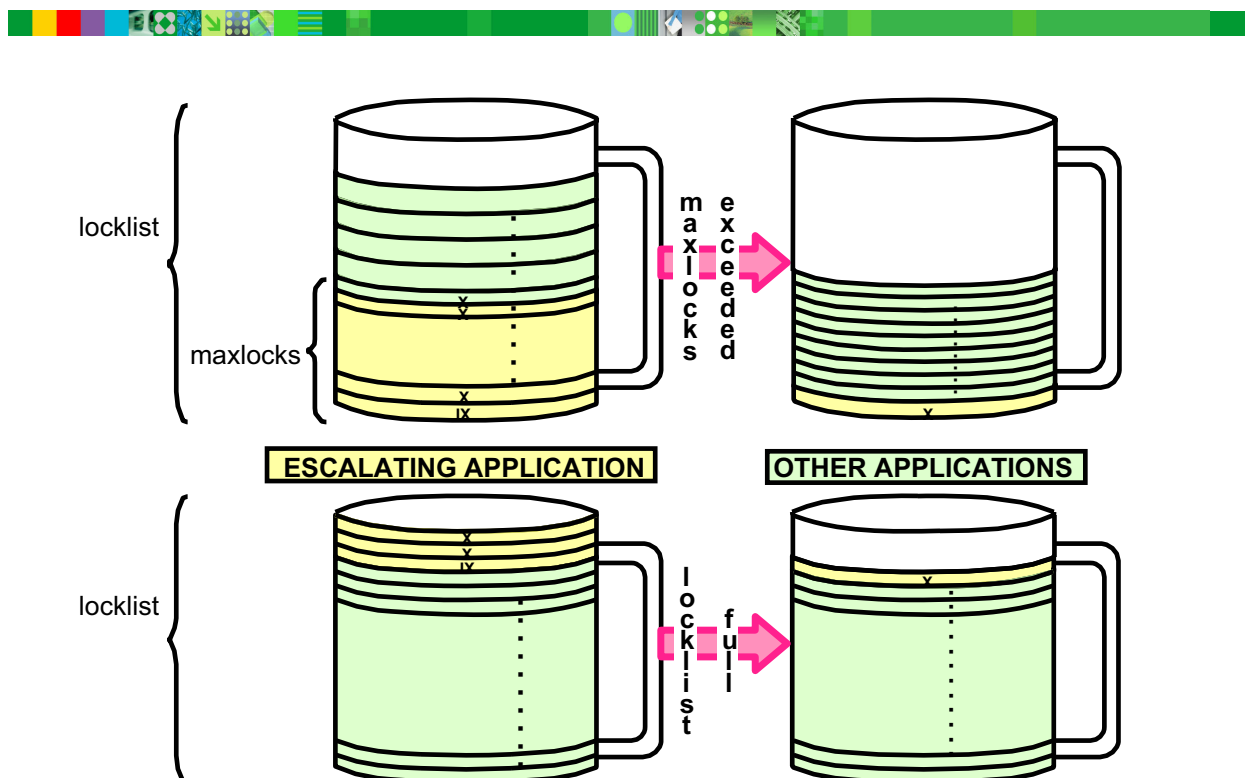
**Details** — Explain can only detail the locking requirements of a single statement. The database manager cannot assume the contents or flow of a unit of work since application logic may determine whether or not a particular SQL statement will execute.

**Additional Information** — Knowledge of lock conversion allows the DBA to determine the locking requirements of each unit of work, not just each statement. Since concurrency issues revolve around units of work, such knowledge is absolutely required to be proficient in lock analysis.

It is normally required to go through at least the first example in the notes section to solidify the meaning of the diagram. The second example (after the COMMIT), can be reviewed if necessary.

**Transition Statement** — Another area concerning application programming and locks involves lock escalation.

## Lock escalation



© Copyright IBM Corporation 2007

Figure 8-17. Lock escalation

CF238.3

### Notes:

In order to service as many applications as possible, the database manager provides the function of lock escalation. This process entails obtaining a table lock and releasing row locks. The desired effect of the process is to reduce the overall storage requirement for locks by the database manager. This will enable other applications to obtain locks requested.

Two database configuration parameters have a direct impact on the process of lock escalation:

- **locklist** — The number of 4 KB pages allocated in the database global memory for lock storage. This parameter is configurable online.

The default value for locklist and maxlocks in DB2 9 is now AUTOMATIC, so the self tuning memory management routines can adjust the size of the locklist and maxlocks to match the demands of the current workload.

- **maxlocks** — The percentage of the total locklist permitted by a single application. This parameter is configurable online.

**Lock escalation can occur in two different situations:**

1. A single application requests a lock that will cause the application to exceed the percentage of the total locklist as defined by maxlocks. The database manager will attempt to free memory space by obtaining a table lock and releasing row locks for the requesting application.
  2. An application triggers lock escalation because the total locklist is full. The database manager will attempt to free memory space by obtaining a table lock and releasing row locks for the requesting application. Note that the application being escalated may or may not have a significant number of locks. The total lock volume may be reaching a threshold because of high system activity where no individual application has reached the limit established by maxlocks.
- A lock escalation attempt can fail. If a failure occurs, the application for which escalation has been attempted will receive a -912 SQLCODE. Such a return code should be handled by the application.

## Instructor notes:

**Purpose** — Define the database configuration parameters that directly influence locking.

**Details** — Students should realize that lock management is a global database issue. Therefore, the storage required is shared.

The instructor should also lead a discussion regarding what should be done if escalations are occurring. Escalations should not be considered *normal*. To emphasize, it may be appropriate to illustrate a deadlock caused by escalation; the instructor should make a time judgement.

|            |   |    |                                           |
|------------|---|----|-------------------------------------------|
| IX         | X | IS |                                           |
| X          |   |    | 1. "Left" application gets IX and a bunch |
| X          | S |    | of X locks.                               |
| X          |   |    |                                           |
| X          |   |    | 2. "Right" application gets IS, but is    |
| .          |   |    | waiting for S lock on row "Left" has      |
| .          |   |    | changed.                                  |
| X (exceeds |   |    | 3. "Left" reaches maxlocks and wants to   |
| maxlocks)  |   |    | escalate IX to X at table level. Cannot   |
|            |   |    | since "Right" already has IS. DEADLOCK.   |

Ask the students the *solutions* if an unacceptable number of escalations are occurring.

1. Examine the *offending* application that is getting too many locks with focus on commit scope.
2. Examine tuning maxlocks, locklist, or both.
3. Get more memory so that parameters can be raised.
4. Throttle the number of concurrent applications via maxagents.

**Additional Information** — A lock requires 32 bytes for an object that already has a lock and 64 bytes for the first lock on an object.

**Transition Statement** — Another configuration parameter impacting locking determines the lock timeout interval.



## 8.4 Lock timeouts and deadlock

### Instructor topic introduction

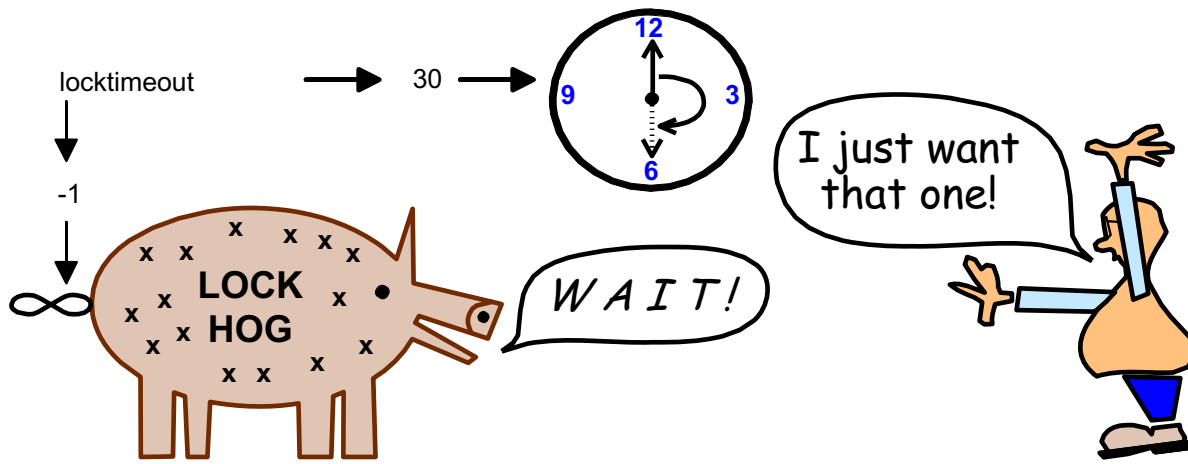
**What students will do** — The student will learn about lock timeouts and how to change from the default settings to explicit lock waits. In addition, the student will learn about the causes and effects of deadlocks.

**How students will do it** — Listen to lecture and participate in discussions on when/why one might want to change lock timeouts.

**What students will learn** — How to set lock timeouts and to avoid deadlocks.

**How this will help students on their job** — Setting lock timeouts can improve programming against a databases as the programmer can vary the wait time for locks from the previous monolithic value. Students need to understand the nature and causes of deadlock so that they can avoid (program around the possibility) them.

## Lock wait and timeout



If the application *hogging* the locks doesn't COMMIT or ROLLBACK other applications wait until lock is available or timeout exceeded

© Copyright IBM Corporation 2007

Figure 8-18. Lock wait and timeout

CF238.3

### Notes:

Applications that request a lock that is not compatible with the existing locks on the object, or with locks already requested but not given, will be queued for service. The database manager will not suspend the waiting application, unless a deadlock is the cause of the wait, or the timeout period for lock wait is exceeded.

If the timeout period is exceeded, the waiting application will receive a -911 SQLCODE with reason code 68. The unit of work will be automatically rolled back by the database manager.

The length of the timeout period is configurable using the database parameter **locktimeout**. The value is specified in seconds. For transaction environments, the recommended starting value for this parameter is 30 seconds. However, benchmarking and tuning may be necessary to determine the optimum value for a given installation.

**If the parameter is set to -1, which is the default, lock timeout is disabled and the waiting application will not receive a timeout.**



Although locktimeout values other than -1 will prevent an application from an endless wait, they do not remove the need for long-running applications to commit or roll back statements within reasonable amounts of time. Large applications that require massive amounts of data or require extensive time to complete a unit of work should be scheduled during non-prime hours. If this is not possible, the application should be analyzed to determine if the unit of work can be reduced.

## ***Instructor notes:***

**Purpose** — Define the locktimeout parameter.

**Details** — The locktimeout parameter allows the administrator to customize the duration of the timeout interval at the database level. Use the suggested starting point of 30 seconds and then tune as appropriate, unless prior experience or knowledge of installation requirements suggests otherwise. The default value of -1 means that lock waits are indefinite (thus the pig's tail is an infinity symbol.) Although this is not likely to be a desired value for production installations (or even test installations), it is the default to provide upward compatibility with an earlier release of the product when no such parameter existed and timeouts were infinite.

**Additional Information** — Information concerning the number of timeouts is available in the monitor. Analyzing this information can assist in tuning efforts for this parameter.

You may get into a discussion here on what a good value should be. 30 seconds is often used in OLTP environments; but that will mean that an application may hang for 30 seconds waiting on a lock before an error is returned to the application. You need to balance between declaring a lock timeout too quickly, and having a wait that is too long for the application user.

**Transition Statement** —

## SET CURRENT LOCK TIMEOUT

- Assigns a value to the CURRENT LOCK TIMEOUT special register for the current session

```
SET [CURRENT] LOCK TIMEOUT [=] { WAIT | NOT WAIT
    | NULL | WAIT [integer-constant] | host-variable }
```

- CURRENT LOCK TIMEOUT can be set anytime while connected to a database
- The register setting is used immediately for subsequent SQL statements until value is set again
- You can read the current register setting with:

```
VALUES CURRENT LOCK TIMEOUT
```

© Copyright IBM Corporation 2007

Figure 8-19. SET CURRENT LOCK TIMEOUT

CF238.3

### Notes:

Prior to DB2 UDB v8.2, the lock timeout *for all users and all sessions* was controlled by one configuration parameter:

Lock timeout (sec) (LOCKTIMEOUT) = -1

Now each session is able to control its own lock timeout and change the value as often as needed. If the session does not SET its own lock timeout, the default from the configuration file applies. If it does SET its own lock timeout, the setting applies until a new value is set. The lock timeout applies to *each* lock requested. If the timeout period expires, an error is returned to the program requesting the lock.

The new CURRENT LOCK TIMEOUT special register provides a more granular approach to lock timeout for applications that require varying lock wait times, depending on the business function.

If CURRENT LOCK TIMEOUT is not set, the default database configuration parameter is used instead. When it is set, the value overrides the default database configuration parameter until, and if, SET CURRENT LOCK TIMEOUT = NULL is used.

## Functionality

The SET CURRENT LOCK TIMEOUT statement changes the value of the CURRENT LOCK TIMEOUT special register. It is not under transaction control, and is persistent across COMMIT boundaries for the session.

The statement can be embedded in an application program or issued interactively. It is an executable statement that can be dynamically prepared.

The statement applies to the current session and no authorization is needed to execute this statement.

```
SET [CURRENT] LOCK TIMEOUT [=] { WAIT | NOT WAIT | NULL
    | WAIT [integer-constant] | host-variable }
```

The specified value must be an integer between -1 and 32767, inclusive, representing the number of seconds maximum that the application is prepared to wait for a lock — or the value NULL to reset the register to the default setting.

### WAIT

Specifies a CURRENT LOCK TIMEOUT value of -1, which means that the database manager is to wait until a lock is released, or a deadlock is detected.

### NOT WAIT

Specifies a CURRENT LOCK TIMEOUT value of 0, which means that the database manager is not to wait for locks that cannot be obtained, and an error will be returned.

### NULL

Specifies that the CURRENT LOCK TIMEOUT value is to be unset, and that the value of the locktimeout database configuration parameter is to be used when waiting for a lock. The value that is returned for the special register will change as the value of locktimeout changes.

### WAIT integer-constant

Specifies an integer value between -1 and 32767. A value of -1 is equivalent to specifying the WAIT keyword without an integer value. A value of 0 is equivalent to specifying the NOT WAIT clause. If the value is between 1 and 32767, the database manager will wait that number of seconds (if a lock cannot be obtained) before an error.

### host-variable

A variable of type INTEGER. The value must be between -1 and 32767. If host-variable has an associated indicator variable, and the value of that indicator variable specifies a null value, the CURRENT LOCK TIMEOUT value is unset. This is equivalent to specifying the NULL keyword.

For compatibility with IBM Informix database servers:

- MODE can be specified in place of TIMEOUT
- TO can be specified in place of the equals (=) operator

- SET LOCK MODE WAIT can be specified in place of SET CURRENT LOCK TIMEOUT WAIT
- SET LOCK MODE NO WAIT can be specified in place of SET CURRENT LOCK TIMEOUT NOT WAIT.

An updated value of the special register takes effect immediately upon successful execution of this statement. Because the special register value that is to be used during statement execution is fixed at the beginning of statement execution, an updated value of the CURRENT LOCK TIMEOUT special register will only apply to statements that start execution after the SET statement has completed successfully.

### Examples

*Example 1:* Set the lock timeout value to wait for 30 seconds before returning an error.

```
SET CURRENT LOCK TIMEOUT 30
```

*Example 2:* Unset the lock timeout value, so that the LOCKTIMEOUT database configuration parameter value will be used instead.

```
SET CURRENT LOCK TIMEOUT NULL
```

## ***Instructor notes:***

**Purpose** — Understand the SET CURRENT TIMEOUT statement.

**Details** — A single wait time for all locks is not an efficient approach. Different installations and different applications at an installation will need different wait times for locks.

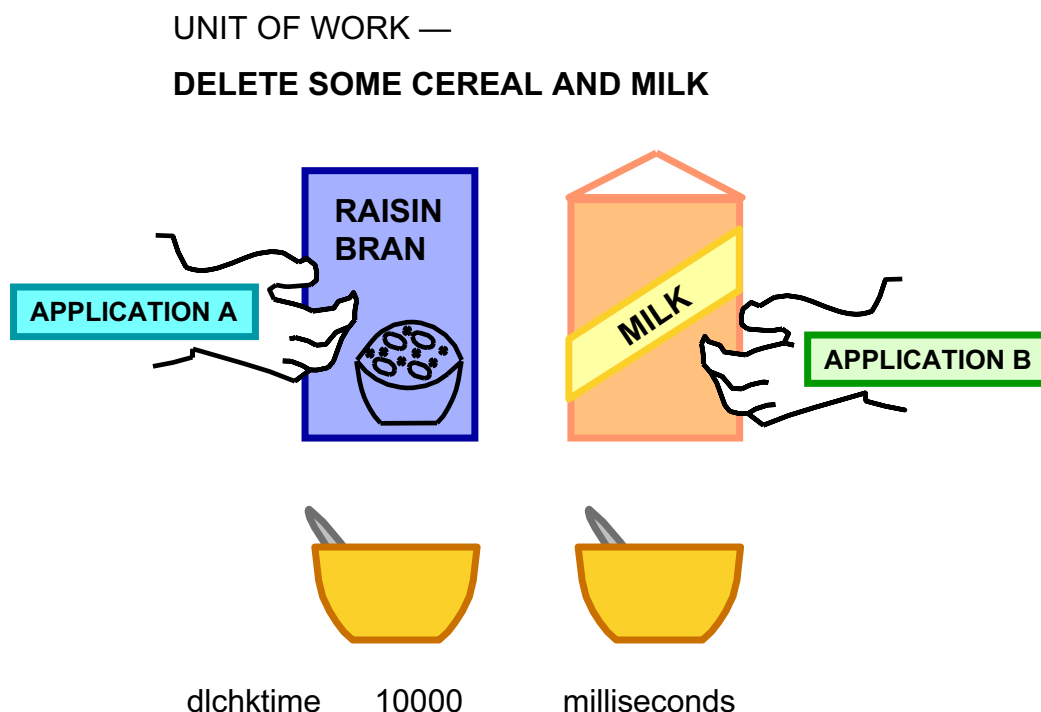
Emphasize that the programmer must determine how long to wait for a lock. And, if the lock is not obtained, a choice must be made: (a) wait again (presumably for the same time), or (b) abandon the attempt to lock the object.

Note that the timeout period is just that. If I decide to wait 5 seconds for a lock, and then lock is available after just one second, I will obtain it then (that is, after one second). The timeout period is the length of time that I am prepared to wait for the lock to become available if it is not currently available. All applications attempting the wait will be put into a wait list and the first entry in the list will be given the lock when it becomes free. Thus, in a busy system and a *popular* object, my own wait time may span the use of the lock by several processes as I might not be the first in the wait list.

**Additional information** —

**Transition statement** — Deadlocks can also impact the concurrency of applications.

## Deadlock causes and detection



© Copyright IBM Corporation 2007

Figure 8-20. Deadlock causes and detection

CF238.3

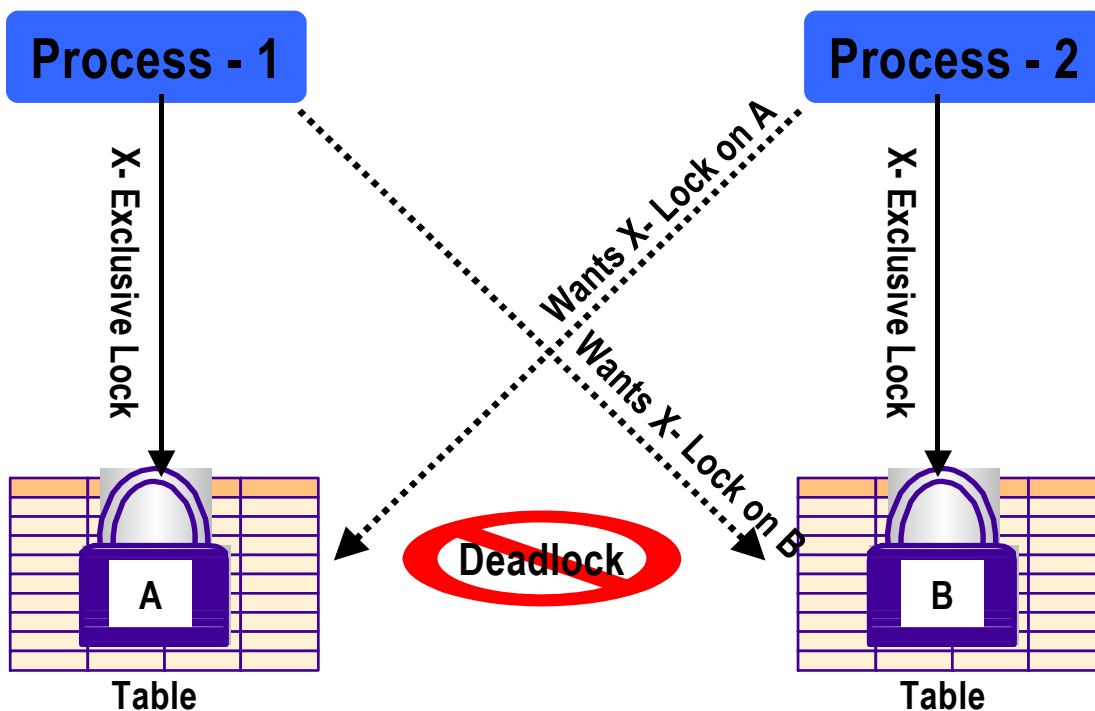
### Notes:

A deadlock occurs when applications cannot complete a unit of work due to conflicting lock requirements that cannot be resolved until the unit of work is completed.

The visual illustrates the concept of deadlocks. The unit of work that both application A and application B need to complete before committing is to get a bowl of cereal with milk. For the sake of simplicity, assume there is only enough milk and cereal left for a single bowl. (Another way for the scenario to work is to assume the milk represents a single row and the cereal represents a single row.)

1. Application A obtains an X lock on the cereal.
2. Application B obtains an X lock on the milk.
3. Application A wants an X lock on the milk but cannot obtain it until application B commits.
4. Application B wants an X lock on the cereal but cannot obtain it until application A commits.

Neither application can proceed to a commit point.



This type of deadlock, caused by accessing objects in reverse order, can be reduced by establishing rules of access at an installation for highly used objects. In this particular example, if both applications access the cereal first, followed by the milk, the first application to process would obtain an X lock on the cereal and prevent the other from continuing at that point. The application possessing the X lock on the cereal could proceed to get the X lock on the milk and complete the unit of work.

#### Other contributors to deadlocks:

- Repeatable Read and Read Stability (major)
- Lock Escalation (major)
- Lock Conversion (minor)
- Catalog Modification (moderate)
- Referential Constraint Enforcement (minor)

The ultimate cause of all deadlocks is poor programming. With proper design, deadlocks are impossible.

#### Deadlock Detector

Deadlocks are handled by a background process called the *deadlock detector*. If a deadlock is detected, a victim is selected, then the victim is automatically rolled back and returned a negative SQL code (-911) and reason code 2. Rolling back the victim releases locks and should allow other processes to continue.

The deadlock check interval (DLCHKTIME) defines the frequency at which the database manager *checks* for deadlocks among all the applications connected to a database.



- Time\_interval\_for\_checking\_deadlock = dlchktime
- Default [Range]: 10,000 (10 seconds) [1000–600,000]
- Unit of measure: milliseconds

**dlchktime** — Configuration parameter that sets the deadlock check interval. This value is designated in milliseconds and determines the interval for the asynchronous deadlock checker to *wake up* for the database. The valid range of values is 1000 to 600,000 milliseconds. Setting this value high will increase the time that applications will wait before a deadlock is discovered, but the cost of executing the deadlock checker is saved. If the value is set low, deadlocks are detected quickly, but a decrease in run-time performance could be experienced due to checking. The default value corresponds to 10 seconds. This parameter is configurable online.

## ***Instructor notes:***

**Purpose** — Define the classic cause of deadlock and introduce the dlchktme parameter.

**Details** — The student should be given the impression that deadlocks are not completely avoidable; they are a price that is paid to increase the concurrency of data access in database systems. However, deadlocks should be analyzed and controlled when possible.

**Additional Information** — Sometimes there will be discussion about **which** application will be rolled back. It seems that in DB2 for z/OS, there is a belief (which may or may not be true) that the application holding the fewer locks will be rolled back (Patti is writing this, and when she took a DB2 for OS/390 DBA class a few years ago, she tried to verify this by testing during class, and was unable to prove it; so the fact that this is taught in DB2 for OS/390 classes does not necessarily make it so!). In DB2 UDB for Linux, UNIX, and Windows, the documentation indicates: “The deadlock detector arbitrarily selects one of the applications in the deadlock and releases the locks currently held by that ‘volunteered’ application.” In other words, we don’t know for sure which application will be rolled back. In previous versions (before V7), it appeared to be always the application that caused the deadlock (that is, if appA requested something that appB held, we were in a lock wait, but if appB then requested something that appA held, we were in a deadlock, and thus appB would be rolled back). Starting in V7, it wasn’t this predictable, and I have heard that it rolls back the application with the *higher* transaction number, which, if transaction numbers were always increasing, may imply that it would roll back the transaction with the least amount of work. But, this isn’t documented, **and the best thing to say is, “DB2 chooses a victim, and the victim’s unit of work is rolled back”**.

Some students will no doubt realize that changing the order of access is not **ALWAYS** a valid solution to this classic example of deadlock. Accept such statements to emphasize that not all deadlocks can be avoided.

The student notes also include other contributors to deadlocks, but do not detail the causes. The intention here is that the performance courses will address the details. You may wish to address a deadlock situation caused by lock escalation on the next visual. However, if a student insists on further explanation of other contributors, you can handle questions at a break using the information below, unless you believe you have time to handle during the lecture.

Although some deadlocks will occur in systems that provide data integrity and the capability to change multiple items within a unit of work, there are certain conditions that are more prone to causing deadlocks.

### **Repeatable Read Isolation Level**

An application running in repeatable read continues to hold S locks on rows that are read until the unit of work is completed. This greater number of locks increases the chance for another application to require a resource held by the repeatable read application. If such an application holds a non-compatible lock that the repeatable read application requires, a deadlock occurs. If the application can be written to use cursor stability, the chances for deadlock will be lowered.

## Relational Scans Under Repeatable Read

A repeatable read application that must perform a relational scan will use strict table locking. Such a strict lock will be required on each table scanned. If more than one table is accessed via the relational scan during a unit of work, the conditions are prime for deadlock. For example, assume that application A obtains an S lock on table W to support a relational scan. Application B is presently updating data in table Y using an IX table lock and X row locks. At this point, application A attempts to acquire an S lock on table Y. It cannot proceed due to the IX lock posted by Application B. Application B attempts to update one row via a unique index in table W. It requires an IX lock on the table. Since application A holds an S lock on the table, the request for the IX lock causes a deadlock.

Strict table locking potentially conflicts with the locking requirements of more applications than row locking. If the application using strict table locking is also acquiring locks on other objects within the unit of work, the chance for deadlock exists.

If this type of deadlock is occurring to an unacceptable extent, examine the application to determine if the isolation level of cursor stability could be used. Also, determine if the SQL statements causing the relational scans could be written to utilize index access or if creating an index would eliminate the relational scan.

### Lock Escalation for an Application

If an application is using repeatable read, or is changing a large number of rows, it may run into the limit on the number of locks allowed for an application. Lock escalation causes the application to obtain a higher-level table lock, potentially on multiple tables, thus creating the same kind of problem as described above in “Relational Scans Under Repeatable Read”. Also, the escalation attempt itself may result in a deadlock.

| Application | Table A Lock                   | Row Locks                     |
|-------------|--------------------------------|-------------------------------|
| Lotsalocks  | IX                             | X.... close to threshold      |
| Nicepuppy   | IS                             | waiting for X lock to release |
| Lotsalocks  | IX... waiting to escalate to X | X.... still holding locks     |
| Nicepuppy   | IS                             | still waiting                 |

The two applications have deadlocked. *Lotsalocks* cannot proceed because of the IS lock held on the table by *Nicepuppy*. *Nicepuppy* cannot proceed because of the X row lock held by *Lotsalocks*.

Applications that cause escalation should be examined to determine if the escalations could be avoided or minimized. For example, if the cause of escalation is a large number of S row locks held due to repeatable read, determine if the application can be written using cursor stability.

If the application is escalating because of a large number of rows being changed in the same unit of work, the application should be evaluated to determine if the unit of work truly needs to be so large. For example, an application that is deleting all obsolete rows from a table may do so by executing a single DELETE statement with a date comparison on the WHERE clause. If no business problem is introduced, a cursor could be declared WITH HOLD, and the rows could be deleted using DELETE WHERE CURRENT OF CURSOR.

The application could commit periodically, thus breaking the unit of work down into smaller units of work that do not cause escalation. Using a cursor may have an undesired performance impact due to the increased SQL statements executed. Another alternative is for the deleting application to issue a LOCK TABLE statement before issuing the DELETE statement. This would acquire only the table lock and avoid the possibility of a deadlock between row and table level locks, since escalation would not be necessary.

If none of these other solutions is possible, the percentage of the lock list allowed per application can be increased in the database configuration, or the lock list size specified in the database configuration can be increased. Such an increase would need to be evaluated from a memory utilization standpoint.

### Lock Conversion

Assume that an application must retrieve rows, write the contents of the rows to a report, and then update the rows. The application can be executed by multiple users at the same time. Examine the following pseudocode and the execution and locking requirements of two invocations of the same application. Also, assume that the row to be updated is identified by both invocations and that the invocations are simultaneous:

| Statement                                                             | Invocation 1<br>Table Lock | Invocation 1<br>Row Lock                                | Invocation 2<br>Row Lock                                | Invocation 2<br>Table Lock |
|-----------------------------------------------------------------------|----------------------------|---------------------------------------------------------|---------------------------------------------------------|----------------------------|
| DECLARE DOG<br>CURSOR FOR<br>SELECT ...<br>WHERE IXCOL<br>BETWEEN ... | None                       | None                                                    | None                                                    | None                       |
| OPEN DOG                                                              | IS                         | None                                                    | None                                                    | IS                         |
| FETCH DOG                                                             | IS                         | S                                                       | S                                                       | IS                         |
| (Write row to<br>report)                                              | IS                         | S                                                       | S                                                       | IS                         |
| UPDATE ...<br>WHERE IXCOL<br>= hostvar                                | IX                         | S -> X (can't<br>because of invo-<br>cation 2's S lock) | S -> X (can't<br>because of invo-<br>cation 1's S lock) | IX                         |

If the application is written to exploit the FOR UPDATE clause in the definition of the CURSOR, this deadlock due to lock conversion is avoided. Only the first application to FETCH a given row will obtain the U lock:

| Statement                                                                                | Invocation 1<br>Table Lock | Invocation 1<br>Row Lock | Invocation 2<br>Row Lock | Invocation 2<br>Table Lock |
|------------------------------------------------------------------------------------------|----------------------------|--------------------------|--------------------------|----------------------------|
| DECLARE DOG CUR-<br>SOR FOR SELECT ...<br>WHERE IXCOL<br>BETWEEN ... FOR<br>UPDATE OF... | None                       | None                     | None                     | None                       |
| OPEN DOG                                                                                 | IX                         | None                     | None                     | IX                         |

|                                     |    |        |                     |    |
|-------------------------------------|----|--------|---------------------|----|
| FETCH DOG                           | IX | U      | wants U, must wait  | IX |
| (Write row to report, invocation 1) | IX | U      | still waiting for U | IX |
| UPDATE ... WHERE<br>IXCOL = hostvar | IX | U -> X | still waiting for U | IX |

The first invocation to obtain the U row lock will be able to proceed to commit. The second invocation will then be able to proceed. The U lock provides a mechanism to queue intent to update requests.

The same considerations apply to applications that wish to report row contents and then subsequently delete the row.

### Inconsistent Access

This cause of deadlock was highlighted on the previous visual. In general, these types of deadlocks can be minimized if a consistent protocol for modifying different objects is followed.

### Modifying System Catalog Tables

There are three ways in which catalog tables are modified:

- DDL (creating, dropping, or altering tables, indexes, views, or plans)
- Authorization (granting or revoking privileges)
- Binding access plans

These types of operations can create locking problems among themselves and with applications doing normal database processing, for the following reasons:

- All catalogs have indexes, and many have more than one.
- All of the different types of modifications to catalogs can involve deleting and inserting rows. Since the catalogs are indexed, index locking requirements come into play.
- All catalog access by the Database Manager must be done with repeatable read.
- Applications performing normal processing must access rows in SYSIBM.SYSPLAN and SYSIBM.SYSBAUTH, and may additionally access rows in SYSIBM.SYSTABLES for dynamic SQL, SYSIBM.SYSSECTIONS for authorization checking. Since these accesses are done with repeatable read, applications will hold S locks in these catalogs. This is true even if the application is running at the uncommitted read isolation level.

For example, if Application A is accessing a table with a name of U.TABLEB, it may not be possible to create a table with a name of U.TABLEA. Since all of these considerations are beyond the control of an application, the best solution is to avoid entirely or at least minimize these types of operations during normal database processing, perhaps postponing them until non-prime or off-shift time.

## Enforcement of Referential Integrity

When referential constraints are defined, deletes from a parent table can result in one of three types of accesses to a dependent table:

1. Reads, if the delete rule is RESTRICT
2. Deletes, if the delete rule is CASCADE
3. Updates, if the delete rule is SET NULL

All accesses to dependent tables are performed under the cursor stability isolation level, regardless of the isolation level of the package. This will result in a row locking strategy on the dependent table of:

- Lock Intent Share with S row locks for reads.
- Lock Intent Exclusive with X row locks for deletes or updates if there is an index on the foreign key.
- Lock Intent Exclusive with U row locks for deletes or updates that do not use an index.

Usually, deadlocks due to constraint enforcement can be minimized by following guidelines for consistent access order on the parent table. In extreme cases, the parent table, and possibly the dependent tables, could be locked via the LOCK table statement before the other SQL is attempted. However, since overall concurrency could be impacted drastically, this technique should be considered as a last resort.

**Transition Statement** — End of topic. Provide a transition statement if appropriate.

# Checkpoint

## Exercise — Unit Checkpoint

- \_\_\_ 1. You are evaluating the locking strategy of a program that reads and changes data. The program reads or changes a limited number of rows in the target tables and should run during the time when heavy use of the system is expected by other transactions. What lock modes best support this level of required concurrency at the table level?

---

**IS for reading and IX for changing.**

**Ref:** This question is meant to emphasize the most common table locking strategy expected for table spaces and tables in transaction processing.

- \_\_\_ 2. A programmer requests your advice. An application that will run during a batch cycle (no other concurrent requests against the target table) will be written to update a large portion of the table. Explain indicates an IX lock for the table space and table. Would you suggest any technique for the programmer to investigate, and if so, what is the technique?

---

**Include: LOCK TABLE name IN EXCLUSIVE MODE.**

**Ref:** Eliminating row locking may be beneficial in terms of performance for the application in question. However, the impact on concurrency needs to be carefully evaluated before using the LOCK TABLE statement.

- \_\_\_ 3. An airline requires an application that will determine available seats on a particular flight to support the reservation system. The application should list the seats available that meet a general grouping (for example, aisle seat toward the front of the plane) so that reservations can suggest a seat when booking a passenger. However, the application should not prevent update to the row concerning the seat while the passenger is considering possible options. (In other words, the seat is not held for the passenger while the passenger is thinking; in fact, the cursor is closed after producing the answer set.) Assume that the airline uses conversational transactions (that is, the transaction does not commit prior to displaying a screen). What isolation level would be most suitable?

---

**Cursor Stability, CS**

**Ref:** The isolation levels of Repeatable Read or Read Stability are not appropriate because they would maintain S locks on rows accessed. This is in conflict with the business requirement for other applications to be able to update the rows examined.

Uncommitted read is not appropriate, since rows in the process of being updated should not be considered *available*. (It is possible that some students may disagree with this interpretation of the term *available*. As long as a student demonstrates understanding of the difference between UR and CS, the interpretation of the term is not a concern.)

Cursor Stability permits obtaining a list of seats that are *available* at the time they were examined, but does not prevent changes to the status when the rows are presented to the reservation agent.

- \_\_\_ 4. Assume that the airline described in the previous question now wants to ensure that any seat being shown to the user will be available, should that user wish to reserve it. What isolation level would be most suitable?

---

### Read Stability

**Ref:** Some people may suggest repeatable read, but this is not appropriate as it will maintain S locks on all rows accessed, not just the ones being presented to the user. Cursor Stability will not lock all of the rows that have been read; in fact, since the cursor was previously closed, none of the rows will be held.

- \_\_\_ 5. You are the database administrator for a large financial institution. Most processing requirements of the business are accomplished by using an isolation level of Cursor Stability. However, assume that government regulations dictate that a status report of an account table must be generated at 12:00 p.m. on the last day of business each month. Furthermore, the regulations stipulate that the state of all rows is not permitted to change from the start of the report application until it is completed. How would you suggest meeting this requirement?

---

### Include: LOCK TABLE name IN SHARE MODE.

**Ref:** Many students will tend to suggest repeatable read as a solution to this requirement. However, repeatable read does not obtain and hold row locks until the row is accessed. It is therefore possible for other applications to change data that has not yet been read by the report application. The only way to guarantee the stability of an entire table is to lock the table before processing the rows.

- \_\_\_ 6. You receive several phone calls from users of your database complaining about a high number of deadlocks and timeouts. You begin investigating by taking a snapshot with the monitor and discover a high number of lock escalations. You know that you have configured the locklist and maxlocks parameters to handle your normal application requirements. What would be your next area of investigation?

---

**Look for programs not committing frequently.**



**Ref:** An inappropriate number of lock escalations can be caused by a lock list that is not large enough to handle the application load or a maxlocks parameter that does not support the largest unit of work generally required by some applications. Assuming that these areas have been eliminated as a possible source, the most likely cause is an application that exceeds the normal design case concerning lock requirements. Students may also respond with other valid arguments as well. Again, the absolute answer is not important. The assumptions and thought process to arrive at a conclusion are the key.

## Unit summary

Having completed this unit, you should be able to:

- Explain why locking is needed
- List objects that can be locked
- Describe and discuss the various lock modes and their compatibility
- Explain four different levels of data protection
- Set isolation level and lock time out for current activity
- Explain lock conversion and escalation
- Describe the situation that causes deadlocks

© Copyright IBM Corporation 2007

Figure 8-21. Unit summary

CF238.3

### **Notes:**

***Instructor notes:***

**Purpose** — Review the objectives for the unit.

**Transition Statement** — Now, it is time to complete the labs dealing with locks and concurrency.



# Unit 9. Problem determination

## Estimated time

01:00

## What this unit is about

This unit provides information about DB2 monitoring and problem determination tools.

## What you should be able to do

After completing this unit, you should be able to:

- Collect information for problem analysis and resolution
- Use error logs for basic problem analysis
- Describe four types of monitors:
  - Snapshot Monitor
  - Event Monitor
  - Activity Monitor
  - Health Monitor
- Describe the function of EXPLAIN and use this facility to assist basic analysis
- Use a series of basic commands to better work with connections and sessions
- Retrieve statistics and other information from a running DB2 instance

## How you will check your progress

Accountability:

- Checkpoint
- Lab exercises

## References

*Administration Guide*

*System Monitor Guide and Reference*

## Unit objectives



After completing this unit, you should be able to:

- Collect information for problem analysis and resolution
- Use error logs for basic problem analysis
- Describe four types of monitors:
  - Snapshot Monitor
  - Event Monitor
  - Activity Monitor
  - Health Monitor
- Describe the function of EXPLAIN and use this facility to assist basic analysis
- Use a series of basic commands to better work with connections and sessions
- Retrieve statistics and other information from a running DB2 instance

© Copyright IBM Corporation 2007

Figure 9-1. Unit objectives

CF238.3

### Notes:

***Instructor notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Identify what students should be able to do at the end of this unit.

**Additional information** —

**Transition Statement** — Let's look at some performance and problem determination tools within DB2 Universal Database.





## 9.1 Problem solving

### Instructor topic introduction

**What students will do** — Learn about a systematic approach to information gathering to embark on problem solving.

**How students will do it** — Lecture and discussion.

**What students will learn** — The students will learn about key elements that need to be captured on a timely basis for analysis and problem solving work after a problem has been observed.

**How this will help students on their job** — Having an approach and a checklist will help the student gather the right information when the time comes.

## How to solve a problem

- 
- Understand the nature of the problem
    - Common problem types
  - Determine the cause of the conditions
    - Is the problem reproducible?
    - Was it a one time occurrence?
  - Keep detailed notes — your problem description should include:
    - ALL error codes/error conditions
    - ALWAYS include the reason code, if applicable
    - The actions which preceded the error
    - A reproducible scenario, if possible

© Copyright IBM Corporation 2007

Figure 9-2. How to solve a problem

CF238.3

### Notes:

In order to solve the problem, you must understand the nature of the problem, and determine the cause of the conditions.

**Common problem types** are:

- Unexpected messages/SQL codes
- Abends (*abnormal endings*) of application or DBM
- Loops and hangs
- Traps
- Database/data corruption
- Data loss
- Incorrect or inconsistent documents
- Incorrect output
- Install failure
- Performance
- Usability

Important points to consider are:

- Is the problem reproducible? If so, how is it reproducible (by the clock, a certain application)?
- Was it a one time occurrence? What were the operating conditions at the time?

A comprehensive problem description should include:

- ALL error codes/error conditions
- ALWAYS include the reason code, if applicable
- The actions which preceded the error
- A reproducible scenario, if possible
- System and application log files, if available

***Instructor notes:***

**Purpose** — Provide a general background to problem solving for database issues.

**Details** — The methodology used for problem solving is important. This visual, plus the next, attempt to lay down the foundations for a generalized problem solving approach.

**Additional information** —

**Transition statement** — Let's see what type of checklist of items do we need to collect to provide the basis for solving a database problem.

## Diagnostic data and data checklist

- What information is generally required to analyze a DB2 problem and determine the solution?
  - ✓ **SQLCODE / SQLSTATE / reason codes / system error codes**
  - ✓ **A good problem description**
  - ✓ **System-time of the error (to pin-point the time in logs)**
  - ✓ **Include the SQL code and any associated reason code**
  - ✓ **A description of the actions preceding the error**
  - ✓ **GET DBM CFG / GET DB CFG listings**
  - ✓ **db2diag.log** entries (see next visuals)
  - ✓ Dump files listed in the db2diag.log / trap files in the DIAGPATH
  - ✓ **Operating system software level, and service patches**
  - ✓ **Hardware model and peripheral equipment**
  - ✓ **Database (DB2) version and fixpak level**
  - ✓ **Other available data**

© Copyright IBM Corporation 2007

Figure 9-3. Diagnostic data and data checklist

CF238.3

### Notes:

To troubleshoot the problem correctly, you need to collect the information required to analyze a DB2 problem and determine the solution.

Use the **db2diag.log** file to view captured diagnostic information.

- If reproducible, setting the DIAGLEVEL to 4 and recapturing the information is recommended.
- Any dump files mentioned in **db2diag.log (pid/tid.node)**.
- Any traceback/trap files in the DIAGPATH (**tpid/tid.node** or **\*.trp**). You must check for them manually—know the format!
- With workgroup edition (WE) or enterprise server edition (ESE), send all files in the DB2DUMP directory.
  - To reduce the number of viewed files, clean up this directory on a regular basis.
  - Also copy and truncate the **db2diag.log** file.

### ***Instructor notes:***

**Purpose** — Provide a checklist of items that need to be gathered to solve a database problem.

**Details** — The notes only provide a discussion of db2diag.log (perhaps the item least known by the students). You probably will want to open discussion up in the classroom to see if the students can improve the list and can see the reasons for the items already on the list.

**Additional information** —

**Transition statement** — [Let's look at db2diag.log in more detail.](#)

# First Failure Data Capture (FFDC)

## *DBM configuration parameters*

### NOTIFY LEVEL — (0-4, default 3)

- 0 — No administrative notification messages captured
- 1 — Fatal or unrecoverable errors
- 2 — Immediate action required
- 3 — Important information, no immediate action required
- 4 — Informational messages

### DIAGLEVEL — (0-4, default 3)

- 0 — No diagnostic data captured
- 1 — Severe errors only
- 2 — All errors
- 3 — All errors and warnings
- 4 — All errors, warnings, and informational messages



### DIAGPATH — valid directory path, may contain

- ✓ Notification file
- ✓ Error log
- ✓ Dump files
- ✓ Trap files

© Copyright IBM Corporation 2007

Figure 9-4. First Failure Data Capture (FFDC)

CF238.3

### **Notes:**

When significant events occur, DB2 writes information to the administration notification log. The information is intended for use by database and system administrators. Many notification messages provide additional information to supplement the SQLCODE that is provided. The type of event and the level of detail of the information gathered are determined by the NOTIFYLEVEL configuration parameter. However, detailed diagnostic information is not written to this log.

NOTIFYLEVEL specifies the type of administration notification messages that are written to the administration notification log. On UNIX platforms, the administration notification log is a text file called *instance.nfy*. On Windows, all administration notification messages are written to the Event Log. The errors can be written by DB2, the Health Monitor, the Capture and Apply programs, and user applications.

Valid values for this parameter are:

**0** — No administration notification messages captured. (This setting is not recommended.)

- 1** — Fatal or unrecoverable errors. Only fatal and unrecoverable errors are logged. To recover from some of these conditions, you may need assistance from DB2 service.
- 2** — Immediate action required. Conditions are logged that require immediate attention from the system administrator or the database administrator. If the condition is not resolved, it could lead to a fatal error. Notification of very significant, non-error activities (for example, recovery) may also be logged at this level. This level will capture Health Monitor alarms.
- 3** — Important information, no immediate action required. Conditions are logged that are non-threatening and do not require immediate action but may indicate a non-optimal system. This level will capture Health Monitor alarms, Health Monitor warnings, and Health Monitor attentions.
- 4** — Informational messages.

The administration notification log includes messages having values up to and including the value of *notifylevel*. For example, setting *notifylevel* to 3 will cause the administration notification log to include messages applicable to levels 1, 2, and 3.

For a user application to be able to write to the notification file or Windows Event Log, it must call the `db2AdminMsgWrite` API.

**Recommendation:** You may wish to increase the value of this parameter to gather additional problem determination data to help resolve a problem. Note that you must set *notifylevel* to a value of 2 or higher for the Health Monitor to send any notifications to the contacts defined in its configuration.

## Error log

Diagnostic information about errors is recorded in the `db2diag.log` text file. This information is used for problem determination and is intended for DB2 customer support.

The type of diagnostic errors recorded in the `db2diag.log` is determined by `DIAGLEVEL`. Valid values are:

- 0** — No diagnostic data captured
- 1** — Severe errors only
- 2** — All errors
- 3** — All errors and warnings
- 4** — All errors, warnings, and informational messages

It is the *diagpath* configuration parameter that is used to specify the directory that will contain the error file, event log file (on Windows NT only), alert log file, and any dump files that may be generated based on the value of the *diaglevel* parameter.

## Diagnostic file location

`DIAGPATH` allows you to specify the fully qualified path for DB2 diagnostic information. This directory could possibly contain dump files, trap files, an error log, a notification file, and an alert log file, depending on your platform.



If this parameter is null, the diagnostic information will be written to files in one of the following directories or folders:

- For supported Windows environments:
  - If the DB2INSTPROF environment variable or keyword is **not** set, information will be written to x:\IBM\SQLLIB \%DB2INSTANCE%, where x:\IBM\SQLLIB is the drive reference and directory specified in the DB2PATH registry variable or environment variable, and %DB2INSTANCE% is the name of the instance owner.
  - Note:** The directory does not have to be named SQLLIB. By default, DB2 UDB installs on Windows into \Program Files\IBM\SQLLIB.
  - If the %DB2INSTPROF% environment variable or keyword is set, information will be written to x:\%DB2INSTPROF% \%DB2INSTANCE%, where DB2INSTPROF is the name of the instance profile directory and %DB2INSTANCE% is the name of the instance owner.
- For UNIX-based environments: \$INSTHOME/sqllib/db2dump, where \$INSTHOME is the home directory of the instance owner.



### Note

You should clean out the dump directory periodically to keep it from becoming too large.

## DB2 Administration Server First Failure Data Capture

The DB2 Administration Server (DAS) also collects information for first failure data capture. First failure data capture (FFDC) is a general term applied to the set of diagnostic information the DB2 administration server captures automatically when errors occur. This information reduces the need to reproduce errors to get diagnostic information. The diagnostic information is contained in a single location.

The information captured by the DB2 administration server FFDC includes:

- **Administration notification logs** — When an event occurs, the DB2 administration server writes information to the DB2 administration server log file, db2dasdiag.log.
- **Dump files** — For some error conditions, extra information is logged in external binary dump files named after the failing process ID. These files are intended for use by DB2 Customer Support.
- **Trap files** — The DB2 administration server generates a trap file if it cannot continue processing because of a trap, segmentation violation, or exception. Trap files contain a function flow of the last steps that were run before a problem occurred.

By default, the DB2 administration server FFDC information is placed in the following locations:

- On Windows systems:

- If the DB2INSTPROF environment variable is not set:

`db2path\DB2DAS00\dump`

where db2path is the path referenced in the DB2PATH environment variable, and DB2DAS00 is the name of the DAS service. The DAS name can be obtained by typing the **db2admin** command without any arguments.

- If the DB2INSTPROF environment variable is set:

`x:\%DB2INSTPROF%\DB2DAS00\dump`

where x: is the drive referenced in the %DB2PATH% environment variable, db2instprof is the instance profile directory, and DB2DAS00 is the name of the DAS service.

- On UNIX-based systems:

`$DASHOME/das/dump`

where \$DASHOME is the home directory of the DAS user.



#### Note

You should clean out the dump directory periodically to keep it from becoming too large.

***Instructor notes:***

**Purpose** — Identify the configuration parameters that affect error and notification logging.

**Details** — DB2 will capture information when it detects problems in the system. You can configure what types of messages should be captured. By default, both NOTIFYLEVEL and DIAGLEVEL are set to capture severe problems, errors, and “warning” type messages. This is a good setting to keep, except perhaps when you are installing new software or upgrading a system, when you may want to collect informational messages as well.

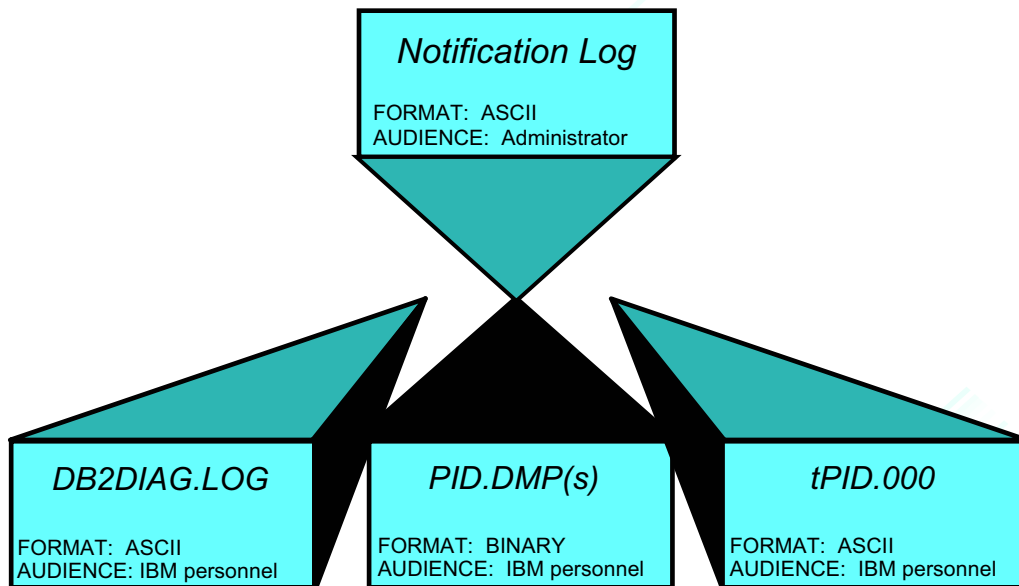
The DIAGPATH indicates where the files should be written. In the student notes, there is documentation of where the files will be written by default.

**Additional Information** — Notification log was introduced in V8. Prior to that version, the error and administration logging information was all contained in the db2diag.log.

Certification objective: 1I Skill in interpreting the Notification log.

**Transition Statement** — Let’s look at the files that may be written.

## FFDC structure



© Copyright IBM Corporation 2007

Figure 9-5. FFDC structure

CF238.3

### Notes:

First-failure data capture (FFDC) is a general term applied to the set of diagnostic information that DB2 captures automatically when errors occur. This information reduces the need to reproduce errors to get diagnostic information. The **DIAGPATH** variable, specified in the database manager configuration, gives the fully qualified path to the FFDC storage directory. The **DIAGLEVEL** and **NOTIFYLEVEL** configuration parameters control the detail of information you receive in the logs.

The information captured by FFDC includes the following:

- **Administration Notification Logs** — When significant events occur, DB2 writes information to the administration notification log. The information is intended for use by database and system administrators. Many notification messages provide additional information to supplement the **SQLCODE** that is provided. The type of event and the level of detail of the information gathered are determined by the **NOTIFYLEVEL** configuration parameter.
- **db2diag.log** — Diagnostic information about errors is recorded in this text log file. This information is used for problem determination and is intended for DB2 customer

support. The level of detail of the information is determined by the DIAGLEVEL configuration parameter.

- **Dump files** — For some error conditions, extra information is logged in external binary files named after the failing process ID. These files are intended for use by DB2 customer support.
- **Trap files** — The database manager generates a trap file if it cannot continue processing because of a trap, segmentation violation, or exception.
- **Core files** (UNIX only) — When DB2 terminates abnormally, the operating system generates a core file. The core file is a binary file that contains information similar to the DB2 trap files. Core files may also contain the entire memory image of the terminated process.

## ***Instructor notes:***

**Purpose** — Identify the files that are created and who may use them.

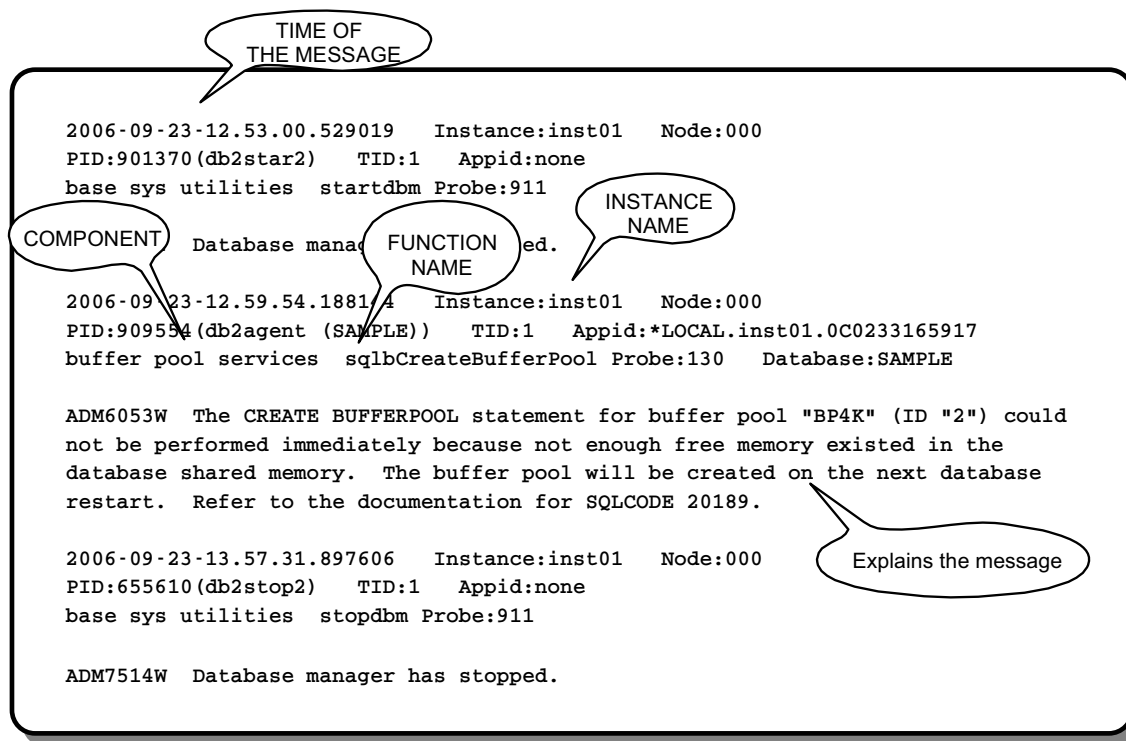
**Details** — The notification log is the first point of reference for the DBA. The location of the notification log was identified on the previous page. It is also accessible through the Journal.

The db2diag.log file may also have information of interest to the DBA, but in general, the goal is to place information for DBA use in the notification log.

**Additional information** —

**Transition Statement** — Let's look at an example of output from the notification log.

## Interpreting the Notification Log



© Copyright IBM Corporation 2007

Figure 9-6. Interpreting the Notification Log

CF238.3

### Notes:

The db2diag.log and administration notification log are files that contain text information logged by DB2. They are located in the directory specified by the DIAGPATH database manager configuration parameter. **On Windows NT, Windows 2000, and Windows XP systems, the DB2 administration notification log is found in the event log and can be reviewed through the Windows Event Viewer.**

The information that DB2 records in the administration logs is determined by the settings for DIAGLEVEL and NOTIFYLEVEL.

Use a text editor to view the file on the machine where you suspect a problem to have occurred. The most recent events recorded are the furthest down the file. Generally, each entry contains the following parts:

- A timestamp
- The location reporting the error. Application identifiers allow you to match up entries pertaining to an application on the logs of servers and clients.

A diagnostic message (usually beginning with DIA or ADM) explaining the error.

- Any available supporting data, such as SQLCA data structures and pointers to the location of any extra dump or trap files.

If the database is behaving normally, this type of information is not important and can be ignored.

- The Administration logs *grow continuously*. When they get too large, back them up and then erase the file. A new set of files is generated automatically the next time they are required by the system.

The figure shows the start of an instance, and a message generated because a buffer pool was created for which there was not enough free memory. Then the notification log indicates that the instance was stopped.

**Other types of messages that will be placed in the administration notification log:**

- Lock timeout information. This will be captured at notification level 4, including locked object, lock mode, and application holding the lock. Deadlock information will also be available if notification level 4 is set.
- Lock escalations
- Abnormal termination to Governor
- Full event monitor log file and the event monitor turning off
- Current set of active units of work exceed primary log files when infinite logging is turned on
- Disk or file system for active log path being full
- Primary or secondary log path being unavailable
- Beginning of database restart operation

## Some DIAGLEVEL 4 considerations

- DIAGLEVEL 4 logs more information than lower levels
- This causes DB2 to run a little slower, but only during the following times:
  - During an error condition
  - During **db2start** processing
  - During an initial connect to a database
- Balance out the extra data provided with the decreased response time to determine the best setting for the environment



### Warning

Be careful when using DIAGLEVEL 4. Do not set it at that level during *normal* daily activities; use it only as a troubleshooting aid. Remember, **db2diag.log** grows in size as it is used, so it is a good idea to copy it to a safe location and truncate the original file periodically.



**Instructor notes:**

**Purpose** — Give an example of output from the notification log.

**Details** — The graphic gives an example of output from the notification log. It indicates that the instance started; a request was made to create a buffer pool, but there was not enough free memory to do this immediately; and then later, the instance was stopped.

More information about the diagnostic log and problem determination is addressed in the CF45 *Advanced Administration Class*.

**Additional Information** — More about information that may be logged (this information was found by searching the manuals for *notification log*):

- To log more information about lock-request timeouts in the administration notification log, set the database manager configuration parameter *notifylevel* to four. The logged information includes the locked object, the lock mode, and the application holding the lock. The current dynamic SQL statement or static package name might also be logged. A dynamic SQL statement is logged only at *notifylevel* four.

To log more information about deadlocks, set the database manager configuration parameter *notifylevel* to four. The administration notification log stores information that includes the object, the lock mode, and the application holding the lock on the object. The current dynamic SQL statement or static package name might also be logged. The dynamic SQL statement is logged only if *notifylevel* is four.

- Follow these general steps to diagnose the cause of unacceptable lock escalations and apply a remedy:
  - a. Analyze in the administration notification log on all tables for which locks are escalated. This log file includes the following information:
    - The number of locks currently held
    - The number of locks needed before lock escalation is completed
    - The table identifier information and table name of each table being escalated
    - The number of non-table locks currently held
    - The new table level lock to be acquired as part of the escalation. Usually, an “S,” or Share lock, or an “X,” or eXclusive lock is acquired.
    - The internal return code of the result of the acquisition of the new table lock level.

Use the information in administration notification log to decide how to resolve the escalation problem. Consider the following possibilities:

- Increase the number of locks allowed globally by increasing the value of the *maxlocks* or the *locklist* parameters, or both, in the database configuration file. In a partitioned database, make this change on all partitions. You might choose this method if concurrent access to the table by other processes is most important. However, the overhead of obtaining record level locks can induce more delay to other processes than is saved by concurrent access to a table.

- Adjust the process or processes that caused the escalation. For these processes, you might issue LOCK TABLE statements explicitly.
- Change the degree of isolation. Note that this may lead to decreased concurrency, however.
- Increase the frequency of commits to reduce the number of locks held at a given time.
- Consider frequent COMMIT statements for transactions that require long VARCHAR or various kinds of long object (LOB) data. Although this kind of data is not retrieved from disk until the result set is materialized, the descriptor is locked when the data is first referenced. As a result, many more locks might be held than for rows that contain more ordinary kinds of data.
- When the governor encounters an error or stop signal, it does cleanup processing before it ends. Using a list of applications whose priorities have been set, the cleanup processing resets all application agent priorities. It then resets the priorities of any agents that are no longer working on an application. This ensures that agents do not remain running with nondefault priorities after the governor ends. If an error occurs, the governor writes a message to the administration notification log to indicate that it ended abnormally.
- If a file event monitor runs out of disk space, it shuts itself down after logging a system-error-level message in the administration notification log. By setting *logsecond* to -1, you will have no limit on the size of the unit of work or the number of concurrent units of work. However, rollback (both at the savepoint level and at the unit of work level) could be very slow due to the need to retrieve log files from the archive. Crash recovery could also be very slow for the same reason. DB2 will write a message to the administration notification log to warn you that the current set of active units of work has exceeded the primary log files. This is an indication that rollback or crash recovery could be extremely slow.
- Block on Log Disk Full configuration parameter — blk\_log\_dsk\_ful

This configuration parameter can be set to prevent disk full errors from being generated when DB2 cannot create a new log file in the active log path. Instead, DB2 will attempt to create the log file every five minutes until it succeeds. After each attempt, DB2 writes a message to the Administration Notification log. The only way that you can confirm that your application is hanging because of a log disk full condition is to monitor the Administration Notification log. Until the log file is successfully created, any user application that attempts to update table data will not be able to commit transactions. Read-only queries may not be directly affected; however, if a query needs to access data that is locked by an update request, or to access a data page that is fixed in the buffer pool by the updating application, read-only queries will also appear to hang.

- If there is an error writing to either the active log path or the mirror log path, the database will mark the failing path as **bad**, write a message to the administration notification log, and write subsequent log records to the remaining **good** log path only.

DB2 will not attempt to use the **bad** path again until the current log file is completed. When DB2 needs to open the next log file, it will verify that this path is valid, and if so, will begin to use it. If not, DB2 will not attempt to use the path again until the next log file is accessed for the first time. There is no attempt to synchronize the log paths, but DB2 keeps information about access errors that occur, so that the correct paths are used when log files are archived. If a failure occurs while writing to the remaining *good* path, the database shuts down.

- If you want the rollback of incomplete units of work to be done automatically by the database manager, enable the automatic restart (*autorestart*) database configuration parameter by setting it to ON. (This is the default value.) If you do not want automatic restart behavior, set the *autorestart* database configuration parameter to OFF. As a result, you will need to issue the RESTART DATABASE command when a database failure occurs. If the database I/O was suspended before the crash occurred, you must specify the WRITE RESUME option of the RESTART DATABASE command in order for the crash recovery to continue. The administration notification log records when the database restart operation begins.
- To ensure that an appropriate buffer pool is available in all circumstances, DB2 creates small buffer pools, one with each page size: 4 KB, 8 KB, 16 KB, and 32 KB. The size of each buffer pool is 16 pages. These buffer pools are hidden from the user. They are not present in the system catalogs or in the buffer pool system files. You cannot use or alter them directly, but DB2 uses these buffer pools in the following circumstances:
  - When a buffer pool of the required page size is inactive because not enough memory was available to create it after a CREATE BUFFERPOOL statement was executed with the IMMEDIATE keyword. A message is written to the administration notification log. If necessary, table spaces are remapped to a hidden buffer pool. Performance might be drastically reduced.
  - When the ordinary buffer pools cannot be brought up during a database connect. This problem is likely to have a serious cause, such as an out-of-memory condition. Although DB2 will be fully functional because of the hidden buffer pools, performance will degrade drastically. You should address this problem immediately. You receive a warning when this occurs and a message is written to the administration notification log.

**Transition Statement** — Because analysis of this log is an important part of being able to understand database problems, we will look into analysis of the log data in more detail.

## Diagnostic Log Analysis Tool: *db2diag*

- 
- Command line tool for filtering and formatting **db2diag.log** files:
    - Complex set of options, reflecting wide variety of filtering and analysis
    - Ability to use *grep*-like filtering to reduce the number of records returned
  - Profile registry variables (db2set) and DB or DBM configuration parameters are now logged

### Note:

*The UNIX “grep” command allows records to pass from input to output if they match a provided “pattern”*

*The equivalent DOS/Windows command is “find”*

© Copyright IBM Corporation 2007

Figure 9-7. Diagnostic Log Analysis Tool: *db2diag*

CF238.3

### Notes:

The **db2diag.log** format has been improved in a number of ways with DB2 UDB 8.2. This log file structure is now easier to read manually and easier to parse in software. The improvements include:

- Each entry has a complete set of header information
- Message and data portions of the logging record are clearly marked, making the logging record easier to use and to understand
- Timestamps (with time zone) are included in each record
- Each field has a clear field name (in uppercase)
- Header and message field line lengths are restricted to 80 characters
- New fields have been added, most notably a severity-level field to help you find the most important entries

Other changes have been made as well, such as changing the database field name to *DB*.

## Analysis Tool (db2diag)

**db2diag** is a new command line tool for filtering and formatting **db2diag.log** files. You can use this tool to filter diagnostic log files that use the new message format for V8.2.

This tool has a wide range of options that you can use to modify the output to suit your needs. Among other options, you can indicate which fields to display, use a “grep”-like filter to reduce the number of records, and have the empty fields omitted.

### Example output from db2diag

Lines of particular interest are shown in **bold**. The actual file naturally does not have highlighted text. This information was gathered by executing the following steps in sequence:

- Removing the old **db2diag.log** file (this can be done at any time, but would be generally done by renaming and archiving the file) — remember that you should do this as a normal administrative operational step from time to time as the diagnostic log file grows rapidly on a production system
- Stopping database manager (**db2stop**) and then starting the current database manager instance (**db2start**)
- Altering the DBM configuration file for the instance (DB2) by changing the DIAGLEVEL from “3” to “4” (db2 update dbm cfg using diaglevel 4) — to illustrate that these changes are now recorded in the diagnostic log:

```
2006-10-30-18.23.57.726000-300 I1H929          LEVEL: Event
PID      : 3648          TID   : 3884          PROC  : DB2STOP.EXE
INSTANCE: DB2          NODE   : 000
FUNCTION: DB2 UDB, RAS/PD component, pdLogInternal, probe:120
START    : New Diagnostic Log file
DATA #1 : Build Level, 128 bytes
Instance "DB2" uses "32" bits and DB2 code release "SQL09010"
with level identifier "02010107".
Informational tokens are "DB2 v9.1.0.356", "s060629", "NT32", Fix Pack "0".
DATA #2 : System Info, 1564 bytes
System: WIN32_NT IBM-GLENMULES Service Pack 2 5.1 x86 Family 6, model 9, stepping 5
CPU: total:1 online:1 Cores per socket:1 Threading degree per core:1
Physical Memory(MB): total:2047 free:134 available:134
Virtual Memory(MB): total:3942 free:1788
Swap Memory(MB): total:1895 free:1654
Information in this record is only valid at the time when this file was
created (see this record's time stamp)
```

```
2006-10-30-18.23.57.726000-300 I933H1527      LEVEL: Event
PID      : 3648          TID   : 3884          PROC  : DB2STOP.EXE
INSTANCE: DB2          NODE   : 000
FUNCTION: DB2 UDB, base sys utilities, sqleStartStopSingleNode, probe:1130
DATA #1 : String, 30 bytes
C:\IBM\SQLLIB\bin\DB2STOP2.EXE
DATA #2 : Hexdump, 256 bytes
0x0012AD24 : 433A 5C49 424D 5C53 514C 4C49 425C 6269      C:\IBM\SQLLIB\bi
0x0012AD34 : 6E5C 4442 3253 544F 5032 2E45 5845 004E      n\DB2STOP2.EXE.N
0x0012AD44 : 4F4D 5347 0053 4E00 0000 0000 0000 0000      OMSG.SN.....
```

```
0x0012AD54 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AD64 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AD74 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AD84 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AD94 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012ADA4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012ADB4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012ADC4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012ADD4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012ADE4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012ADF4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AE04 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AE14 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

```
2006-10-30-18.23.58.086000-300 I2462H390          LEVEL: Event
PID      : 3496                TID   : 3808        PROC  : db2syscs.exe
INSTANCE: DB2                 NODE   : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:210
DATA #1 : String, 29 bytes
Deactivate phase is bypassed.
DATA #2 : Hexdump, 4 bytes
0x04BCFEBC : 0100 0000          ....
```

```
2006-10-30-18.23.58.086000-300 I2854H410          LEVEL: Event
PID      : 3496                TID   : 3808        PROC  : db2syscs.exe
INSTANCE: DB2                 NODE   : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:230
DATA #1 : String, 49 bytes
Deactivate phase is completed, preparing to stop.
DATA #2 : Hexdump, 4 bytes
0x04BCFEBC : 0100 0000          ....
```

```
2006-10-30-18.23.58.086000-300 I3266H297          LEVEL: Event
PID      : 3496                TID   : 3808        PROC  : db2syscs.exe
INSTANCE: DB2                 NODE   : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:240
DATA #1 : String, 26 bytes
Stop phase is in progress.
```

```
2006-10-30-18.23.58.086000-300 I3565H312          LEVEL: Event
PID      : 3496                TID   : 3808        PROC  : db2syscs.exe
INSTANCE: DB2                 NODE   : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:250
DATA #1 : String, 41 bytes
Requesting system controller termination.
```

```
2006-10-30-18.23.58.106000-300 I3879H405          LEVEL: Warning
PID      : 3496                TID   : 4112        PROC  : db2syscs.exe
INSTANCE: DB2                 NODE   : 000
FUNCTION: DB2 UDB, routine_infrastructure, sqlerKillAllFmps, probe:5
MESSAGE : Bringing down all db2fmp processes as part of db2stop
DATA #1 : Hexdump, 4 bytes
0x03EEFA64 : 0000 0000          ....
```

```
2006-10-30-18.23.58.116000-300 I4286H314          LEVEL: Event
PID      : 3496                TID   : 3808        PROC  : db2syscs.exe
INSTANCE: DB2                 NODE   : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:260
DATA #1 : String, 43 bytes
System controller termination is completed.
```

```

2006-10-30-18.23.58.136000-300 I4602H385          LEVEL: Event
PID      : 3496                      TID   : 3808          PROC  : db2syscs.exe
INSTANCE: DB2                      NODE  : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:280
DATA #1 : String, 24 bytes
There is no active EDUs.
DATA #2 : Hexdump, 4 bytes
0x04BCFE64 : 0000 0000                      ....

2006-10-30-18.23.58.256000-300 E4989H311          LEVEL: Event
PID      : 3496                      TID   : 3808          PROC  : db2syscs.exe
INSTANCE: DB2                      NODE  : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StopMain, probe:911
MESSAGE  : ADM7514W Database manager has stopped.
STOP     : DB2 DBM

2006-10-30-18.24.06.238000-300 I5302H1527          LEVEL: Event
PID      : 4824                      TID   : 3604          PROC  : DB2STOP.EXE
INSTANCE: DB2                      NODE  : 000
FUNCTION: DB2 UDB, base sys utilities, sqleStartStopSingleNode, probe:1130
DATA #1 : String, 30 bytes
C:\IBM\SQLLIB\bin\DB2STOP2.EXE
DATA #2 : Hexdump, 256 bytes
0x0012AD24 : 433A 5C49 424D 5C53 514C 4C49 425C 6269      C:\IBM\SQLLIB\bi
0x0012AD34 : 6E5C 4442 3253 544F 5032 2E45 5845 004E      n\DB2STOP2.EXE.N
0x0012AD44 : 4F4D 5347 0053 4E00 0000 0000 0000 0000      OMSG.SN.....
0x0012AD54 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AD64 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AD74 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AD84 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AD94 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADA4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADB4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADC4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADD4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADE4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADF4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE04 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE14 : 0000 0000 0000 0000 0000 0000 0000 0000      .....

2006-10-30-18.24.14.460000-300 I6831H1528          LEVEL: Event
PID      : 5788                      TID   : 5540          PROC  : DB2START.EXE
INSTANCE: DB2                      NODE  : 000
FUNCTION: DB2 UDB, base sys utilities, sqleStartStopSingleNode, probe:1130
DATA #1 : String, 30 bytes
C:\IBM\SQLLIB\bin\DB2STAR2.EXE
DATA #2 : Hexdump, 256 bytes
0x0012ADB4 : 433A 5C49 424D 5C53 514C 4C49 425C 6269      C:\IBM\SQLLIB\bi
0x0012ADC4 : 6E5C 4442 3253 5441 5232 2E45 5845 004E      n\DB2STAR2.EXE.N
0x0012ADD4 : 4F4D 5347 0000 0000 0000 0000 0000 0000      OMSG.....
0x0012ADE4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012ADF4 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE04 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE14 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE24 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE34 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE44 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE54 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE64 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE74 : 0000 0000 0000 0000 0000 0000 0000 0000      .....
0x0012AE84 : 0000 0000 0000 0000 0000 0000 0000 0000      .....

```



```
0x0012AE94 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
0x0012AEA4 : 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

```
2006-10-30-18.15.59.667000-240 E2565H853          LEVEL: Warning
PID      : 1744                      TID   : 2944          PROC  : db2syscs.exe
INSTANCE: DB2                      NODE  : 000
FUNCTION: DB2 UDB, oper system services, licRequestAccess, probe:1
MESSAGE : ADM12007E  There are "63" day(s) left in the evaluation period for
          the product "DB2 Enterprise Server Edition". For evaluation license
          terms and conditions, refer to the IBM License Agreement in the
          try_LA file, located in the following directory:
          "C:\IBM\SQLLIB\license\en". If you have licensed this product, please
          ensure the license key is properly installed. You can install the
          license via using License Center or db2licm command line utility. The
          license file can be obtained from your licensed product CD.
```

```
2006-10-30-18.24.16.573000-300 E8361H853          LEVEL: Event
PID      : 5264                      TID   : 5192          PROC  : db2syscs.exe
INSTANCE: DB2                      NODE  : 000
FUNCTION: DB2 UDB, base sys utilities, DB2StartMain, probe:911
MESSAGE : ADM7513W  Database manager has started.
START    : DB2 DBM
DATA #1 : Build Level, 128 bytes
Instance "DB2" uses "32" bits and DB2 code release "SQL09010"
with level identifier "02010107".
Informational tokens are "DB2 v9.1.0.356", "s060629", "NT32", Fix Pack "0".
DATA #2 : System Info, 1564 bytes
System: WIN32_NT IBM-GLENMULES Service Pack 2 5.1 x86 Family 6, model 9, stepping 5
CPU: total:1 online:1 Cores per socket:1 Threading degree per core:1
Physical Memory(MB): total:2047 free:168 available:168
Virtual Memory(MB): total:3942 free:1929
Swap      Memory(MB): total:1895 free:1761
```

```
2006-10-30-18.24.41.689000-300 I9216H361          LEVEL: Event
PID      : 5264                      TID   : 5308          PROC  : db2syscs.exe
INSTANCE: DB2                      NODE  : 000
APPHDL   : 0-7                      APPID: *LOCAL.DB2.061030232441
FUNCTION: DB2 UDB, config/install, sqlfLogUpdateCfgParam, probe:30
CHANGE   : CFG DBM: "Diaglevel" From: "3" To: "4"
```

```
2006-10-30-18.24.41.689000-300 I9579H500          LEVEL: Info
PID      : 5264                      TID   : 5308          PROC  : db2syscs.exe
INSTANCE: DB2                      NODE  : 000
APPHDL   : 0-7                      APPID: *LOCAL.DB2.061030232441
FUNCTION: DB2 UDB, config/install, sqf_upd_krcb_dbm_cfg, probe:200
MESSAGE : Not all changes were successful. Now trying to re-apply the
          unsuccessful ones
DATA #1 : Hexdump, 4 bytes
0x04C25808 : 0000 0000          ....
```

```
db2diag invoked with the default set of filtering/formatting options
on file "C:\IBM\SQLLIB\DB2\db2diag.log".
Enter "db2diag -h" to get a short description of all available options.
```



## Command Options

|                 |                                                                    |
|-----------------|--------------------------------------------------------------------|
| <i>filename</i> | one or more space-separated path names of diagnostic logs          |
| -help, -h, ?    | help information. To get help on help, try " <b>db2diag -h h</b> " |
| -filter, -g     | case-sensitive search for a list of field-pattern pairs            |
| -gi             | case-insensitive search for a list of field-pattern pairs          |
| -gv             | case-sensitive invert matching                                     |
| -gvi, -giv      | case-insensitive invert matching                                   |
| -invert, -v     | invert the sense of matching for all filtering options             |
| -exist          | record field must exist in order to be processed                   |
| -pid            | find all records for a list of process IDs                         |
| -tid            | find all records for a list of thread IDs                          |
| -node, -n       | find all records for a list of nodes                               |
| -error, -e      | find all records for a list of errors                              |
| -level, -l      | find all records for a list of severity levels                     |
| -history, -H    | display the history of logged records for a time interval          |
| -time, -t       | display all the records within a particular time interval          |
| -count, -c      | display a count of matching records                                |
| -verbose, -V    | display all record fields whether they contain data or not         |
| -strict         | display records using one "field: value" pair per line             |
| -cbe            | display records in the Common Base Event (CBE) format              |
| -fmt            | format tool's output using a format string                         |
| -output, -o     | save output into a file                                            |
| -follow, -f     | continuously display appended records as the file grows            |
| -archive, -A    | archive a diagnostic log file                                      |
| -rc             | display descriptions of DB2 error return codes, ZRC or ECF         |
| -ecfid          | display function info extracted from the numeric ECF ID            |

The various options and the option parameters are too detailed to cover here. For further information, consult the *Command Reference* manual.

## Help Options

- **db2diag -help** — provides a short description of the options
- **db2diag -h option1[,...]** — provides help on one or more options
- **db2diag -h brief** — provides descriptions for all options without examples
- **db2diag -h notes** — provides usage notes and restrictions
- **db2diag -h examples** — provides a small set of examples to get started
- **db2diag -h tutorial** — provides examples for all available options
- **db2diag -h all** — provides the most complete list of options

## ***Instructor notes:***

**Purpose** — Provide greater opportunity to understand the log records.

**Details** — You should discuss some — but not necessary all — of the examples listed as working through these will make it easier for students to look at this file when needed.

**Additional information** — Note the comment in the text: “The various options and the option parameters are too detailed to cover here. For further information, consult the *Command Reference* manual.”

The students do NOT need to know this information in detail — and the information provided here is intended to “demystify” log analysis rather than impose a burden of something additional to be learned in detail. The log examples here are merely examples of common things that will be seen and are shown to take the mystery away from this log.

**Transition statement** — Let's look at monitoring in the database.

## 9.2 Database monitoring

### Instructor topic introduction

**What students will do** — Learn about database monitoring.

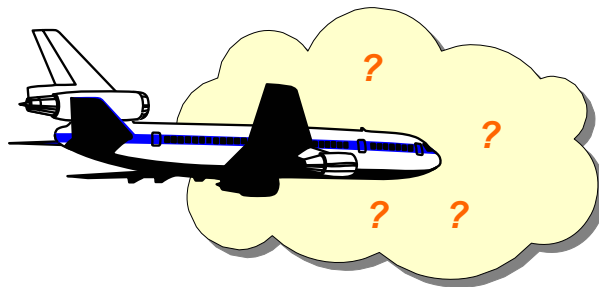
**How students will do it** — Listen to the lecture, practice in the lab.

**What students will learn** — The capabilities of database monitoring in DB2.

**How this will help students on their job** — They will be aware of database monitoring capabilities.

## Why use database monitors?

*No need to fly  
in or into the dark!!*



### DB2 system monitor usage:

- ✓ Improve database and application performance
- ✓ Tune configuration parameters
- ✓ Problem determination
- ✓ Better understand user and application activity

© Copyright IBM Corporation 2007

Figure 9-8. Why use database monitors?

CF238.3

### Notes:

To tune a database server effectively, a database administrator needs large amounts of information about the database and the applications accessing the database. The database administrator requires a tool like a database monitor to obtain the required information to improve the database environment (for example, tune configuration parameters).

A database monitoring plan should be developed to help the database administrator decide on the critical success factors of the database application environment. For example, if response time is the most crucial aspect of the database application, then the administrator needs to monitor the number of lock escalations and effective buffer pool usage. Another example could involve monitoring any changes to a particular table.

**Keep in mind that there is overhead involved when using database monitors.**

**Instructor notes:**

**Purpose** — Identify some common reasons why monitors might be required.

**Details** — If you have some personal examples of problems you have solved via a monitor, this is a good place to tell the story about how monitoring helped you. It is particularly good to use an example if the example can be used to reinforce points further in the lecture.

**Additional Information** — An example monitoring story (problem determination):

Patti was going back to her office one day when one of the developers of the system that IT Education Services uses for its enrollment and administrative functions stopped her in the hallway. “I’ve got a locking problem in the database on the updates for instructor schedules”, he said. “It is taking about 5 minutes for each update on the schedule. Can you help me figure out where the problem is?”

Now, Patti, being an instructor, saw this not only as an opportunity to put her database skills to good use, but also knew that this was the application that was proposed for instructors to use to update their schedules in the new system. If it took 5 minutes to update a single day of instructor activity, this was going to be an extreme annoyance to all instructors, most especially Patti! So, she stopped to help.

Since monitoring requires DB2 SYSADM authority, Patti and the programmer found the DBA, and signed on to the system. They decided to try issuing snapshots to see if, indeed, the program was experiencing lock waits. The good news about an application that runs for 5 minutes is that it gives you lots of time to issue snapshots (you have to look for positive aspects!).

The snapshots, however, did not show that the program was experiencing any lock waits. In fact, each time they did a snapshot, they saw that the status of the program was “SQL Executing”. The application was really executing SQL for nearly all of the 5 minutes.

Well, if you were the DBA at this point, what would your next step be?

Patti, the DBA, and the programmer’s next step was to look at the SQL and see what the strategy was that was being used. Since the program was written in static embedded SQL, there was a package stored in the database, so they could use the db2expln tool to do a quick-and-dirty text output of the strategy. In looking at the strategy, they found that the statement was looking at columns of “START\_DATE > a value”, “START\_TIME > a value”, “END\_DATE < a value”, and “END\_TIME < a value”. The strategy externalized by db2expln showed that DB2 was using an index, a composite key on START\_DATE, START\_TIME, END\_DATE, and END\_TIME.

Now, you may already know this, but since the predicates are all inequalities, really the only column being used in the index for locating the rows was the one on START\_DATE. If you don’t already understand that, hopefully by the end of this course you will.

Now, this is where knowledge of the database came into play. It turns out, in InfoMgmt Education Services, that our database has information about past classes going back over a long period, but we have very little future schedule. We only schedule about six months to a year into the future. (We are not scheduling offerings of CF21 V8 in year 2010 — we don’t

exactly know whether folks will want to know about Administration on DB2 V8 by then!) So, based on the predicate we saw used in the SQL statement, we decided that a data model change might be appropriate. We created a new index, on END\_DATE, END\_TIME, START\_DATE, START\_TIME, and rebound the program. We looked at db2expln to see if it was using the new index, and it was.

The programmer ran the program again, and this time it ran in a little less than 5 seconds. From 5 minutes down to 5 seconds on a design change.

PS. This is a true story, for the most part. The tool never actually went into production (it was to allow instructors to update their schedules in TA). But, you don't have to share that information with the students; and, you can make it "your" story (not "Patti's") if you want.

**Transition Statement** — Let's look at ways to monitor the database.

## Ways to monitor the database

- Snapshot Monitor
  - Query operational database status for an INSTANT in time
- Event Monitor
  - Query operational status OVER time for a specific activity
- Activity Monitor
  - GUI for improving efficiency
- Health Monitor
  - Uses health indicators to gauge the healthfulness of the database manager or database performance
  - Can be accessed from Health Center, Web Health Center, CLP, APIs

© Copyright IBM Corporation 2007

Figure 9-9. Ways to monitor the database

CF238.3

### Notes:

With the increasing complexity of DB2 database systems, there exists a strong need for database and database server information to debug problems and monitor performance.

Snapshot monitoring is useful to determine the current state of a database and its applications.

The Activity Monitor tool assists DBAs in improving the efficiency of database performance monitoring, problem determination, and resolution. It helps to diagnose application locking situations and to tune queries for optimal utilization of database resources. It provides easy access to relevant and well-organized monitor data through a set of predefined reports, such as on top CPU-time consuming applications, and on SQL statements with the largest total sort time. For each predefined report, appropriate actions may be recommended to help resolve resource utilization problems, to optimize performance, or to invoke another tool for further investigation. Lock monitor data is also supplied to illustrate the details of lock waiting situations. Application lock chains can be displayed to show lock waiting dependencies. The Activity Monitor is accessible through a GUI interface, CLP, and in the form of stored procedures and UDFs.

Event monitoring is used to gather information about state changes regarding many different event types: databases, buffer pools, connections, tables, statements, transactions, deadlocks, and table spaces.

The amount of memory that can be used for database monitoring (both Snapshot and Event type monitors) is configurable in the DBM configuration file using the *mon\_heap\_sz* parameter. The memory is allocated at DB2START and the memory is used when monitoring is taking place. If this parameter is set to zero, no monitoring can take place. If this resource is exceeded, the application may receive an SQLCODE, or an error will be logged to the administration notification log, or both.

**Monitoring can be invoked through APIs:**

- db2MonitorSwitches — turn on or off switches for monitor groups
- db2GetSnapshot — collects monitor data into a user buffer
- db2GetSnapshotSize — estimates buffer size needed
- db2ResetMonitor — resets monitor data to zero

or through CLP:

- GET MONITOR SWITCHES
- GET SNAPSHOT FOR ....
- RESET MONITOR ....

The Health Monitor and the Health Center tools provide a *management by exception* capability to DB2 Universal Database by alerting you to potential system health issues. This enables you to address health issues before they become real problems that affect your system's performance.



## Instructor notes:

**Purpose** — Identify the different methods to monitoring within DB2.

**Details** — There are different ways in DB2 to monitor what is going on in the database.

If you know that a current problem exists on the database, then a snapshot through the snapshot monitor may be the most appropriate way to address the problem.

For problems that may come up at any time, the event monitor may be a better option. It will look for certain events occurring and provide output at the time that the event occurs. This is valuable for something like deadlocks that may occur at any time, but which would be very hard to capture using the snapshot monitor.

The health monitor uses health indicators to gauge the healthiness of your system. You can set thresholds for different health indicators, and DB2 can let you know when the database or database manager is outside those thresholds.

**Additional Information** — Melanie Stopfer explains the difference between Event and Snapshot Monitoring as follows:

You and your friend have just arrived at the horse race track. You have brought a video camera to record the race. Your friend has brought a Polaroid camera to take pictures during the race. When the race starts, you turn on your video camera and record the entire race just like an Event Monitor. You cannot view the monitoring until the race has finished. When the race has finished, you can review all information that took place. Your friend takes eight Polaroid pictures during the race. Your friend can look at the pictures while the race is taking place. When the race is finished your friend has only the snapshots (pictures) that he/she has taken to review the event. Discuss the advantages and disadvantages with the class and use this visual simulation as a comparison between a snapshot (Polaroid camera) and event (video camera) with the students.

### Choosing snapshot or event monitoring:

- Snapshot monitor queries the status at a particular time. Use to analyze interim information on long-running transactions before they complete.
- Event monitor records the status of a database over a period of time. Use to collect summary information about transient events that are difficult to monitor through snapshots, such as deadlocks and transaction completions.
- Snapshot monitor for performing sampling
- Event monitor can be used for historical comparisons, for example to compare the current monitor values for transactions against previous iterations.
- Use event monitor to analyze long-running applications.
- Event monitor's counters cannot be reset without stopping the monitor.

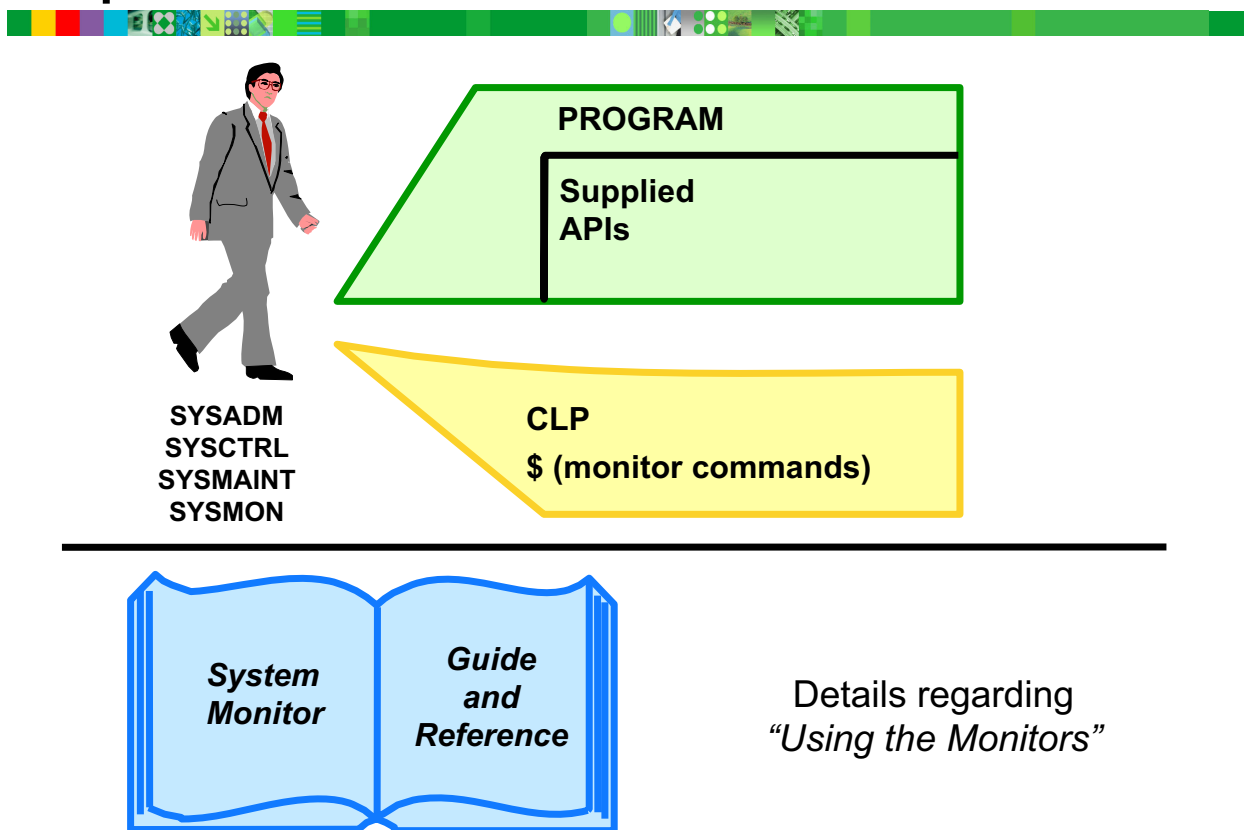
**Note:** No exact match to DB2 MVS trace classes.

Certification objectives: 4B Ability to capture snapshots using Get Snapshots or SQL functions.

**Transition Statement** — Let's look at the snapshot monitor in a little more detail.



## Snapshot Monitor — who and how?



© Copyright IBM Corporation 2007

Figure 9-10. Snapshot Monitor — who and how?

CF238.3

### Notes:

SYSADM, SYSCTRL, SYSMAINT, or SYSMON authorization is required to use the Snapshot Monitor.

An individual possessing the proper authority can use supplied APIs within an application or issue commands using the Command Line Processor.

The Monitor function of DB2 provides the database administrator with a tool that can assist with analysis of many areas.

In this unit, the basic use of the Snapshot Monitor will be discussed. The System Monitor has several other components and provides a great deal of function.

Details regarding the use of the Monitor are provided in the product documentation noted, as well as education related to performance.

### ***Instructor notes:***

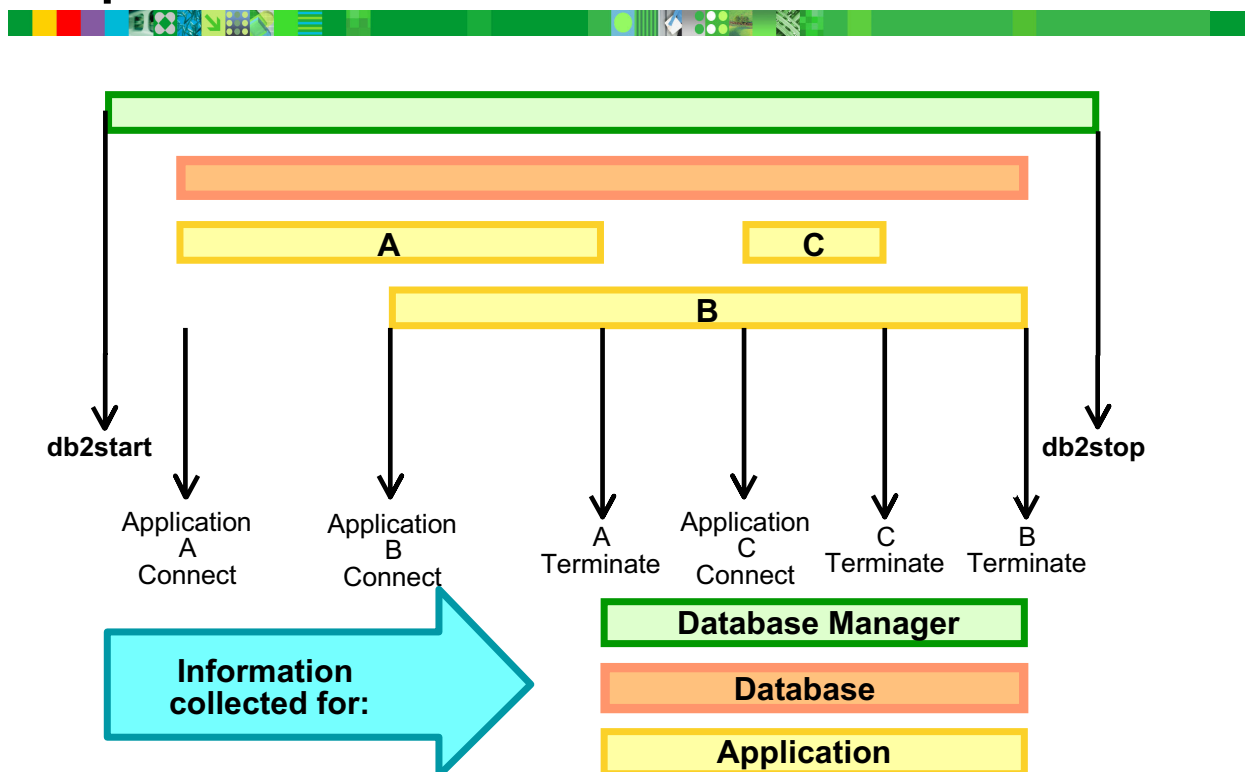
**Purpose** — Introduce the concept of the monitor and the required authorization.

**Details** — This topic is not meant to make a student an expert regarding the use of the monitor. It is only meant to provide details sufficient to complete lab exercises to highlight locking.

**Additional Information** — Consult documentation or attend other education.

**Transition Statement** — Monitoring is available for several levels of statistics. Let's examine when items are monitored.

## Snapshot Monitor — when?



© Copyright IBM Corporation 2007

Figure 9-11. Snapshot Monitor — when?

CF238.3

### Notes:

The Monitor will collect information at different levels within the system.

**Database manager instance** level statistics are collected from the time the instance is started until the instance is stopped.

**Database** level statistics are collected from the time the first application connects to the database until the last connection is terminated. The visual depicts this monitoring period as a single, continuous process. However, application connections may not overlap as shown in the visual. It may be necessary to invoke an application that establishes a connection and does not disconnect from the database until the monitoring period is over.

**Application** level statistics are collected from the time the application connects to the database until the application disconnects from the database.

The level of information used depends on the type of analysis to be completed. You should start analysis at the database manager level and proceed to lower levels if necessary. This method enables the database administrator to resolve system tuning problems first, which may in turn solve database and application problems.

***Instructor notes:***

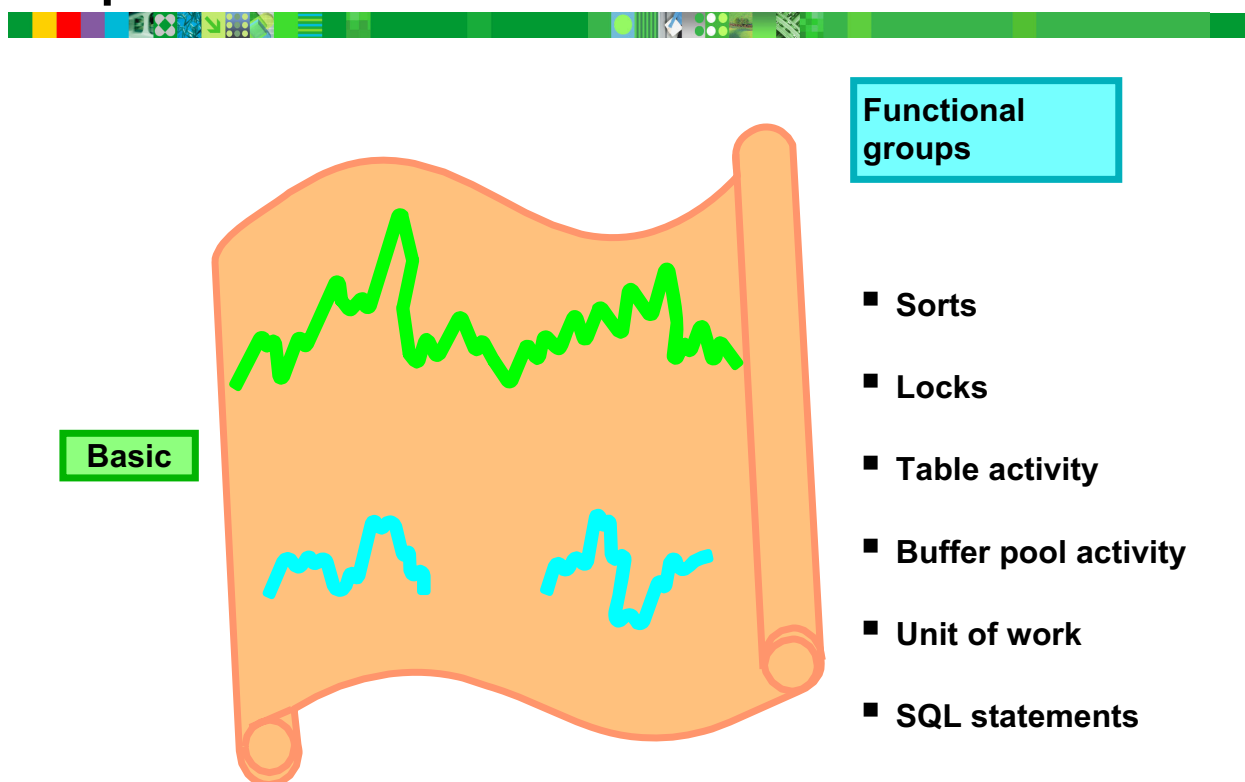
**Purpose** — Define levels of snapshot monitoring elements.

**Details** — Make students aware of the periodic nature of the snapshot monitor.

**Additional Information** — Consult product documentation.

**Transition Statement** — What type of information is collected?

## Snapshot Monitor — what?



© Copyright IBM Corporation 2007

Figure 9-12. Snapshot Monitor — what?

CF238.3

### Notes:

Certain items are always monitored and available for analysis. These items are classified as **BASIC** monitor elements. Some items are only monitored and available if specifically requested. These items are classified within a **FUNCTIONAL GROUP**.

It is possible to monitor only BASIC information or to monitor BASIC information plus one or more FUNCTIONAL GROUPS. You should not monitor FUNCTIONAL GROUPS on a permanent basis due to system overhead of monitoring the elements within the group. The monitor architecture allows FUNCTIONAL GROUP details to be requested when necessary to perform certain analysis on the system.

For example, assume that an installation is experiencing long lock waits. The LOCKS functional group could be monitored during the time that the lock waits occur in an effort to resolve the problem. This functional group will provide, in addition to lock information considered basic, the agent holding the lock waited on, the object name, the object type, the table FID of the locked object, and the application holding the lock. Applications that are holding locks for long periods of time, and therefore contributors to lock wait problems, could be identified through these monitor elements.

### ***Instructor notes:***

**Purpose** — Introduce the concept of basic information always available, and detailed information available within functional groups upon request.

**Details** — Make sure students realize that basic information includes some elements within each functional group, and that the functional groups provide ***additional*** information.

**Additional Information** — Consult product documentation.

**Transition Statement** — The next visual provides the syntax necessary to use the monitor switches.



## Snapshot Monitor — CLP commands

```
▶▶ GET MONITOR SWITCHES ▶▶
```

```
▶▶ UPDATE MONITOR SWITCHES USING switch-name ON
                                     OFF ▶▶
```

```
▶▶ RESET MONITOR ALL
                  FOR DATABASE database-alias ▶▶
```

© Copyright IBM Corporation 2007

Figure 9-13. Snapshot Monitor — CLP commands

CF238.3

### Notes:

The commands listed above provide the capability to use the Database System Monitor via a Command Line Processor session.

**GET MONITOR SWITCHES** will return the current setting of the monitor functional groups illustrated previously. If a switch is ON, the date and time the group became active will also be displayed.

**UPDATE MONITOR SWITCHES** allows one or more functional groups to be turned on or off. The switch names that can be specified are as follows:

- BUFFERPOOL
- LOCK
- SORT
- STATEMENT
- TABLE
- TIMESTAMP
- UOW

Turning a functional group off and then on will cause all resettable monitor elements within the group to be reset to zero. These elements are counter-oriented. This reset impacts all databases defined to the instance for the application which issues the RESET statement.

**RESET MONITOR** will reset monitor elements to zero. The elements reset are counter-oriented. Note that this command can be directed for a specific database or for all databases defined to the instance. This command is NOT specific to a given monitor functional group.

***Instructor notes:***

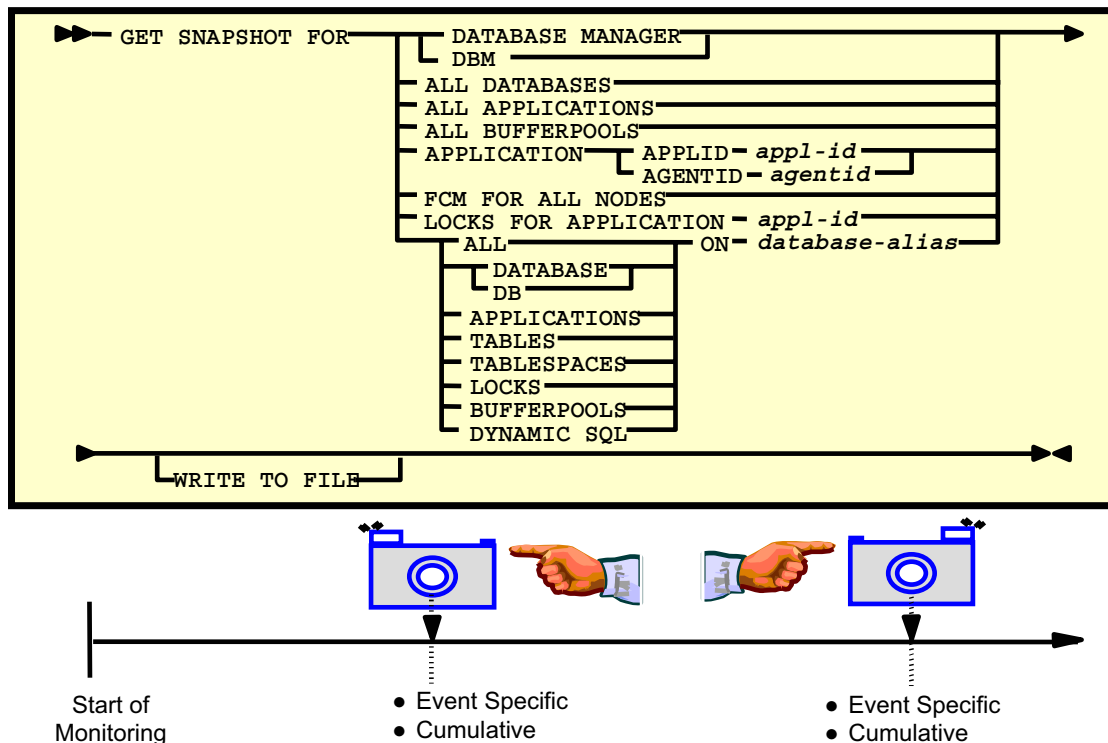
**Purpose** — Introduce monitor switch commands.

**Details** — Quickly present the syntax and state the impact on cumulative counters.

**Additional information** — Certification objective: 4B Ability to capture snapshots using Get Snapshots or SQL functions.

**Transition Statement** — Snapshots provide point-in-time monitoring, but can be useful in basic problem determination. Let's see how to request a snapshot.

## Snapshot Monitor — taking snapshots



© Copyright IBM Corporation 2007

Figure 9-14. Snapshot Monitor — taking snapshots

CF238.3

### Notes:

**GET SNAPSHOT** can be issued with a wide degree of scope concerning the level of information and the type of monitoring.

A snapshot, as the name implies, is a point-in-time view of the monitoring elements being analyzed. Some of these elements are cumulative in nature. They will contain information relative to the start of monitoring (or reset of counters) to the point of the snapshot. Other elements are related to specific events occurring within the system. These events may not appear on two consecutive snapshots, or they may contain totally different and unrelated information.

**For example**, assume that the following series of commands were issued during a CLP session.

- RESET MONITOR ALL
- GET SNAPSHOT FOR DATABASE ON musicdb
- (analyze information)
- GET SNAPSHOT FOR DATABASE ON musicdb
- (analyze information)

An element that exemplifies a cumulative counter that would be available in the snapshots taken would be **deadlocks detected**. This element would contain the number of deadlocks detected since the RESET MONITOR command. On the first snapshot, this element may be 0, while the second snapshot may indicate a higher number.

An element that exemplifies an event-specific piece of information that would be available in the snapshots taken would be **lock list memory in use**. This element would contain the number of bytes currently in use for the lock list. In an active system, it is unlikely that this element will have the exact same value from snapshot to snapshot. The number will fluctuate as applications obtain and release locks.

An alternative to the GET SNAPSHOT command is to issue a SELECT statement against table functions provided with DB2. Using the SQL SELECT interface allows you to select individual monitor elements from the returned data, as well as to use the filtering and aggregation capabilities of selects.

## ***Instructor notes:***

**Purpose** — Introduce the GET SNAPSHOT command.

**Details** — Emphasize the difference between event-specific and cumulative information by highlighting the examples in the student notes. Also, point out that the syntax provides a great deal of flexibility. Using the snapshot correctly means *knowing* what you are looking for.

The ability to get a snapshot by selecting from a provided DB2 table function is mentioned in the student notes. Details on this capability are addressed in the performance classes.

**Additional information** — Certification objective: 4B Ability to capture snapshots using Get Snapshots or SQL functions.

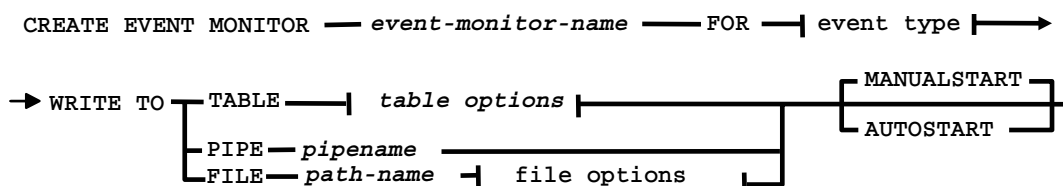
**Transition Statement** — Let's look at event monitoring.

# Event Monitor



Output of events:

- Table
- File
- Named pipe

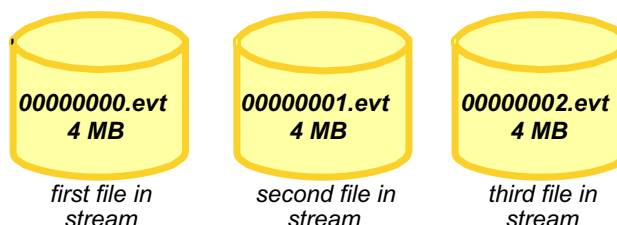


Example file structure

```

'/evmon1' directory
MAXFILESIZE 1024 (4 KB)
MAXFILES 3
  
```

Event Monitor turned OFF  
after 3 files generated



© Copyright IBM Corporation 2007

Figure 9-15. Event Monitor

CF238.3

## Notes:

A table event monitor streams event records to SQL tables, presenting a simple alternative to file and pipe event monitors in enabling you to easily capture, parse, and manage event monitoring data. For every event type an event monitor collects, target tables are created for each of the associated logical data groups.

If the event monitor output is directed to a file, the data is written into a number of files up to MAXFILES, where each file is of size MAXFILESIZE. Once the limit has been reached, the event monitor turns off, and logs that it did so in the administrative notification log file.

In the example above, the event monitor would log 12 MB of event data before turning itself off.

When an event monitor is created to write its output data into a file, there are a number of file options available. The creator must specify a directory location for the monitor files. The directory does not have to exist at CREATE EVENT MONITOR time. However, a check is made for the existence of the target path when the event monitor is activated. At that time, if the target path does not exist, an error is raised.

The options include:

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MAXFILES</b>    | Specifies that there is a limit on the number of event monitor files that will exist for a particular event monitor at any time. The default is that there is no maximum (MAXFILES NONE).                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>MAXFILESIZE</b> | Specifies that there is a limit to the size of each event monitor file.<br>Default: (Windows) 200 4 KB pages; (UNIX) 1000 4 KB pages                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>BUFFERSIZE</b>  | Specifies the size of the event monitor buffers (4 KB pages).<br>When the event monitor is started, <b>two buffers</b> of the specified size are allocated; these permit asynchronous I/O.<br>The larger the buffers, the less I/O will be performed.<br>The minimum and default size of each buffer is 4 pages (two buffers, each 16 KB in size). The maximum size of the buffers is limited by the size of the monitor heap (MON_HEAP_SZ), since the buffers are allocated from the heap. If you are using many event monitors at the same time, increase the size of the MON_HEAP_SZ DBM configuration parameter. |
| <b>BLOCKED</b>     | Specifies that each agent that generates an event should wait for an event buffer to be written out of disk if the agent determines that both event buffers are full. BLOCKED should be selected to guarantee no event data loss. This is the DEFAULT.                                                                                                                                                                                                                                                                                                                                                               |
| <b>NONBLOCKED</b>  | Specifies that each agent that generates an event should not wait for the event buffer to be written out to disk if the agent determines that both event buffers are full. NONBLOCKED event monitors do not slow down database operations to the extent of BLOCKED event monitors.<br>However, NONBLOCKED event monitors are subject to data loss on highly active systems.                                                                                                                                                                                                                                          |
| <b>APPEND</b>      | Specifies that if event data files already exist when the event monitor is turned on, then the event monitor will append the new event data to the existing stream of data files. When the event monitor is reactivated, it will resume writing to the event files as if it had never been turned off.<br>APPEND is the default option.                                                                                                                                                                                                                                                                              |
| <b>REPLACE</b>     | Specifies that if event data files already exist when the event monitor is turned on, then the event monitor will erase all of the event files and start writing data to file 00000000.evt.                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>MANUALSTART</b> | Specifies that the event monitor should not be started automatically each time the database is started. Event monitors with the MANUALSTART option must be activated manually using the SET EVENT MONITOR STATE statement. This is the default.                                                                                                                                                                                                                                                                                                                                                                      |
| <b>AUTOSTART</b>   | Specifies that the event monitor be started automatically each time the database is started.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

It is recommended that you send monitor output to a different volume or disk than the database or log files.



***Instructor Notes:***

**Purpose** — Identify where event monitor data can be written.

**Details** — Event monitor data can be written to tables, pipes, or files. We don't address the options for writing to tables in this class.

When writing to files, you specify a directory (path) name, and potentially options regarding how much event data you want to write. In this particular case, we specified that the event data should be written to the /evmon1 subdirectory. We also indicated a maximum file size of 1024 4 KB pages, and three files. This means that after 12 MB of event data has been written, the event monitor will be turned off. This may be appropriate, for example if you are collecting SQL text to examine through Explain. Once you have gathered 12 MB of event data regarding statements, that is probably enough to keep you busy with explains for a little while. You may just be gathering a sample to start your evaluation.

**Additional Information** — Certification objective: 4C Ability to create and activate event monitors.

**Transition Statement** — Let's look at filters that you can use.

## Filtering *Event Monitor* output



- Creator can specify filters using WHERE clause

- Example:

```
CREATE EVENT MONITOR georgepay FOR STATEMENTS
WHERE APPL_NAME = 'PAYROLL' AND AUTH_ID = 'GEORGE'
WRITE TO FILE '/monitors/georgepay'
MAXFILES 25
MAXFILESIZE 1024
NONBLOCKED
APPEND
```

- Catch whenever GEORGE invokes payroll application
- WHERE conditions can be specified for:
  - Connection
  - Statements
  - Transactions

© Copyright IBM Corporation 2007

Figure 9-16. Filtering *Event Monitor* output

CF238.3

### **Notes:**

Specific event information for connections, statements, or transactions may be captured based on APPL\_ID, AUTH\_ID, or APPL\_NAME.

Remember that any number of event monitors can be defined (created), but only 32 can be active at a time.

To start using the Event Monitor, it must be started:

```
SET EVENT MONITOR georgepay STATE = 1
```

To stop using the Event Monitor:

```
SET EVENT MONITOR georgepay STATE = 0
```

***Instructor notes:***

**Purpose** — Certain event types can be filtered.

**Details** — If you are collecting event data for connections, transactions, or statements, a lot of event data could be generated. The filtering options allow you to indicate that you only want to collect the data for certain application IDs, authorization IDs, or application names. In the example on the graphic, we are only interested in gathering statements issued by *GEORGE* through the *PAYROLL* application. Maybe George is the user who complains most about the performance of his SQL, so we specifically want to examine his statements. Or maybe George is a very representative user for our application.

**Additional Information** — Certification objective: 4C Ability to create and activate event monitors.

**Transition Statement** — Once the data is collected, how do you examine it?

## Examining *Event Monitor* output



*How do I examine Event Monitor output?*

- Write an application to read the file or to receive pipe output
- Use *db2evmon* productivity tool (can write to a pipe or file only)
- Use the GUI event analyzer tool
- Use SQL against event monitor tables

© Copyright IBM Corporation 2007

Figure 9-17. Examining *Event Monitor* output

CF238.3

### **Notes:**

Event monitor output can be viewed using the tool DB2EVMON, or an application can be written using the DB2 APIs to collect and display the monitor information. Data written to tables can be examined using SQL.

The DB2EVMON command formats event monitor file and named pipe output, and writes it to standard output. The command syntax is as follows:

```
db2evmon database-alias event-monitor-name
```

If the event writes the data to a FILE, the db2evmon tool formats the files and writes the output to standard out.

If the event writes the data to a PIPE, the tool formats the output to standard out as the event occurs.

***Instructor notes:***

**Purpose** — Once event data has been created, how can it be viewed?

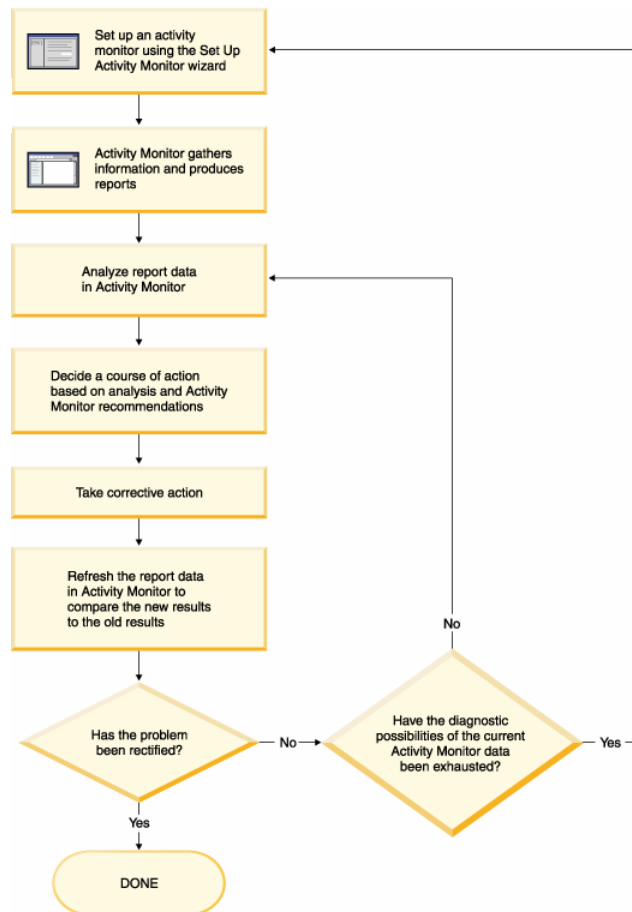
**Details** — The creation of the event data is not very helpful if you don't look at it. The event monitor data can be viewed by using the db2evmon program, or using the GUI event viewer. If the event monitor data was written to a table, the table data can be queried using SQL. You can also write an application program to process the file or pipe output.

**Additional Information** —

**Transition Statement** — Let's look at the Activity Monitor.

# Activity Monitor

- Monitors a database or database partition for
  - Application performance and concurrency
  - Resource consumption
  - SQL statement usage
- Provides predefined reports based on a specific subset of monitor data
- Gives recommendations to
  - Assist in diagnosing the cause of database performance problems
  - Tune queries so that use of database resources is optimized



© Copyright IBM Corporation 2007

Figure 9-18. Activity Monitor

CF238.3

## Notes:

To access the Activity Monitor GUI from the Command Center:

- Expand a particular database from the explorer window (for example, SAMPLE or MUSICDB)
- Right-click and select **Activity Monitors --> Create**
- Choose what partition is to be monitored
- Then select or create a monitoring task (well explained in the GUI)

## Tasks that you can perform from the Activity Monitor

| <i>Tasks from the Activity Monitor</i> | <i>Aspects of tasks</i>                                                                                   | <i>Invocation</i>                                                                                                                                                                                                                                                      |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transactions                           | View transactions running on a selected application                                                       | Select one or more applications in the <i>Report data</i> pane. Right-click and select <b>Show Latest Transactions</b> . The <i>Application Transactions</i> window opens.                                                                                             |
| Statements                             | View SQL statements running on a selected application.                                                    | View SQL statements running on a selected application. Select one or more applications in the <i>Report data</i> pane. Right-click and select <b>Show Latest Statements</b> . The <i>Application Statements</i> window opens.                                          |
|                                        | View the text of SQL statements running on a selected application.                                        | Select one or more applications in the <i>Report data</i> pane. Right-click and select <b>Show Latest Statements</b> . The <i>Application Statements</i> window opens.                                                                                                 |
| Application Lock Chains                | View locks and lock-waiting situations that currently affect a selected application.                      | Select an application in the <i>Report data</i> pane. Right-click and select <b>Show Lock Chains</b> . The <i>Application Lock Chains</i> window opens.                                                                                                                |
|                                        | View information about a selected application for which you are viewing lock information.                 | From the <i>Application Lock Chains</i> window, right-click an application, and select <b>About</b> .                                                                                                                                                                  |
|                                        | View information about the locks held and the locks waited on by a selected application in your database. | From the <i>Application Lock Chains</i> window, right-click an application, and select <b>Show Lock Details</b> .                                                                                                                                                      |
| View report data and recommendations   | View information to help you interpret report data.                                                       | From an <i>Activity Monitor</i> window, an <i>Application Statements</i> window, or an <i>Application Transactions</i> window, use the <i>Report</i> arrow to select the report and click the <b>Report Details</b> push button. View the <i>Details</i> page.         |
|                                        | View recommendations provided by Activity Monitor.                                                        | From an <i>Activity Monitor</i> window, an <i>Application Statements</i> window, or an <i>Application Transactions</i> window, use the <i>Report</i> arrow to select the report and click the <b>Report Details</b> push button. View the <i>Recommendations</i> page. |

## ***Instructor notes:***

**Purpose** — Learn about the Activity Monitor.

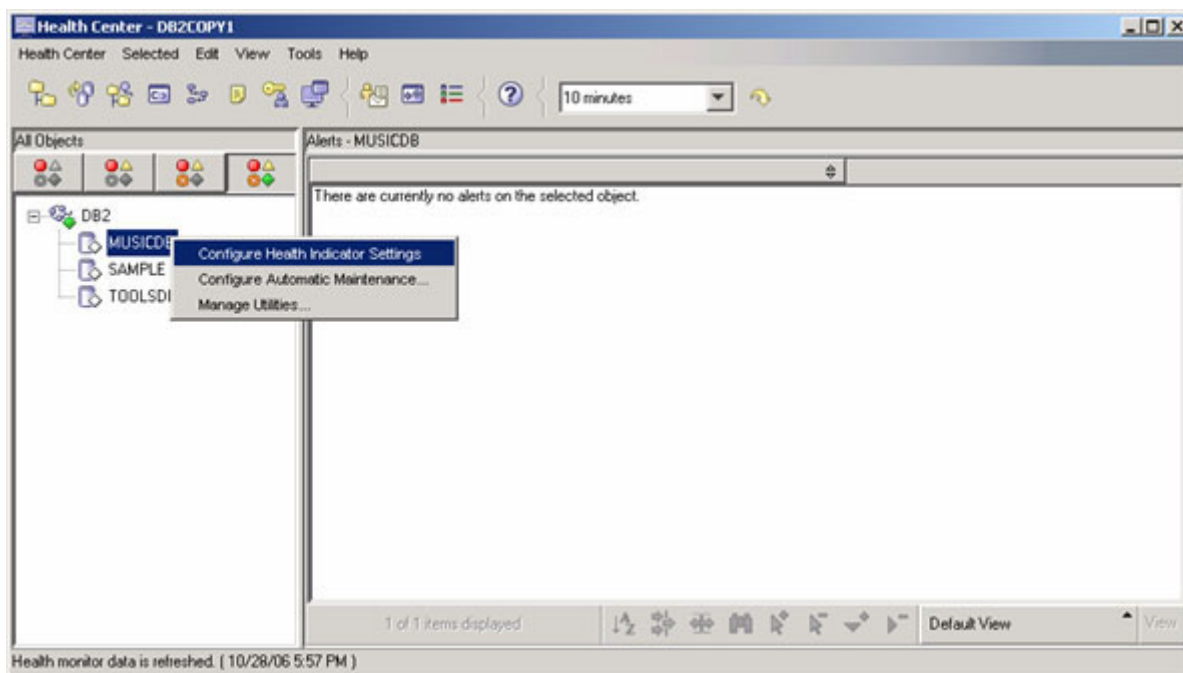
**Details** — The discussion we have here of the Activity Monitor is merely introductory, giving just how to access it and then (in the table above) what can be learned. Like all monitoring activity, this monitor is best understood by using it.

**Additional information** —

**Transition statement** — Let's now move to the Health Monitor. We will use the next few slides to go into detail about this monitor.



# Health Monitor



`db2 get alert cfg for database on musicdb`

© Copyright IBM Corporation 2007

Figure 9-19. Health Monitor

CF238.3

## Notes:

The Health Monitor is a server-side tool that constantly monitors the health of the instance, even without user interaction. If the Health Monitor finds that a defined threshold has been exceeded (for example, the available log space is not sufficient), or if it detects an abnormal state for an object (for example, an instance is down), the Health Monitor will raise an alert.

A *health indicator* is a system characteristic that the Health Monitor checks. The Health Monitor comes with a set of predefined thresholds for these health indicators. The Health Monitor checks the state of your system against these health-indicator thresholds when determining whether to issue an alert. Using the Health Center, commands, or APIs, you can customize the threshold settings of the health indicators, and define who should be notified and what script or task should be run if an alert is issued.

The Health Center provides the graphical interface to the Health Monitor. You use it to configure the Health Monitor, and to configure Automatic Maintenance, as well as to manage utilities and to see the rolled-up alert state of your instances and database objects.

The Health Center and Control Center are integrated through Health Beacons. Health Beacons in the Control Center provide notifications about new alerts in the Health Center. Beacons are implemented on all Control Center windows and notebooks; simply click on a Health Beacon to access the Health Center.

The Health Monitor gathers information about the health of the system using interfaces that do not impose a performance penalty. It does not turn on any snapshot monitor switches to collect information. The Health Monitor is enabled by default when a instance is created; you can deactivate it using the database manager configuration parameter *health\_mon*.

DB2 UDB also provides a Web Health Center that can be used to access the Health Monitor information from a Web browser or PDA.

You can also use DB2 commands and APIs to retrieve health information from the Health Monitor, allowing you to integrate DB2 Health Monitoring with existing system-wide monitoring solutions. For example, to obtain the current configuration settings for an instance, database, table space, or container, you can use the GET ALERT CFG command. For example:

```
db2 get alert cfg for database on musicdb using db.spilled_sorts
```

Alert Configuration

|                             |                                           |
|-----------------------------|-------------------------------------------|
| Indicator Name              | = db.spilled_sorts                        |
| Type                        | = Threshold-based                         |
| Warning                     | = 30                                      |
| Alarm                       | = 50                                      |
| Sensitivity                 | = 0                                       |
| Formula                     | = (db.sort_overflows/db.total_sorts)*100; |
| Actions                     | = Disabled                                |
| Threshold or State checking | = Enabled                                 |

```
SQL22004N  Cannot find the requested configuration for the given object.  
Returning default configuration for "databases".
```

Note that for this example, the database SAMPLE did not have specific settings; it was using the default settings for all databases.

**Instructor notes:**

**Purpose** — Introduce the interfaces to the health monitor.

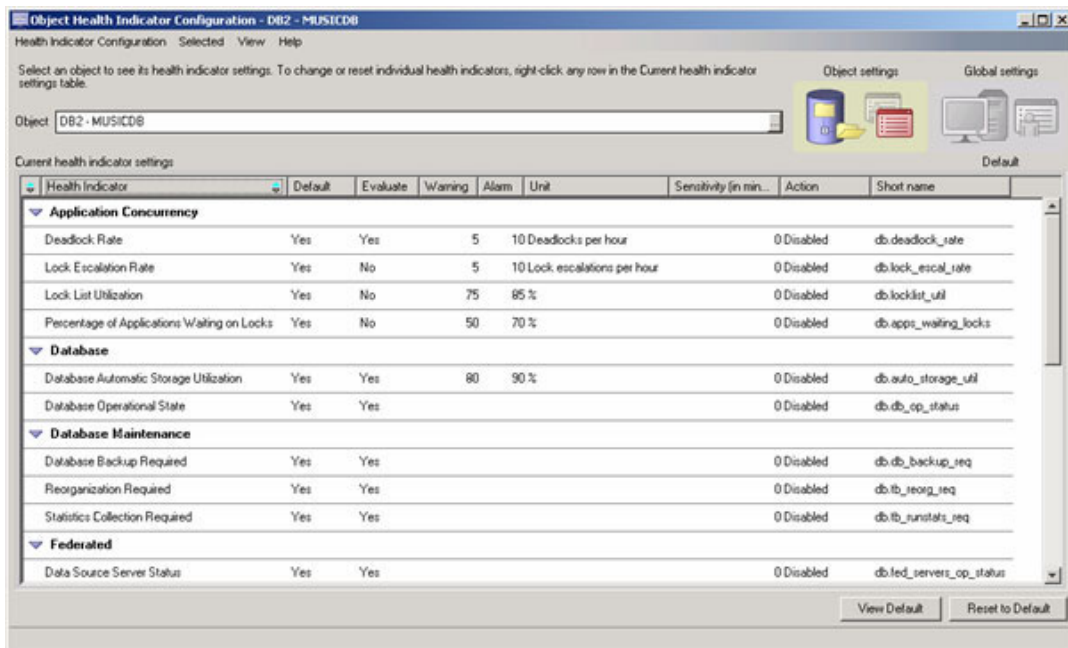
**Details** — The health monitor can be accessed through a GUI Health Center or through commands. On the Health Center, the screen capture is showing that we are going in to configure the health indicator settings for the SAMPLE database. We could find the current settings for all indicators, or for specified ones, using the command line processor. An example of the output looking at the setting for one indicator, db.spilled\_sorts, is shown in the student notes.

**Additional Information** — More information about the Health Monitor is found in Appendix A of the *System Monitor Guide and Reference*.

Certification objective: 4D Ability to configure Health Monitor using the Health Center.

**Transition Statement** — Let's look at the Health Indicator Settings window and what you can do with health indicator settings.

# Health indicator settings



```
db2 get description for health indicator db.spilled_sorts
db2 update alert cfg for database on musicdb using
db.spilled_sorts set alarm 50, warning 30
```

© Copyright IBM Corporation 2007

Figure 9-20. Health indicator settings

CF238.3

## Notes:

Health indicators are used by the Health Monitor to evaluate specific aspects of database manager or database performance. A health indicator is a specific measurement that gauges the healthiness of some aspect of a particular class of database objects, for example table spaces. Health indicators measure either a finite set of distinct states or a continuous range of values to determine whether the state is healthy or unhealthy. The state defines whether or not the database object or resource is operating normally. If the change in the state is determined to be unhealthy, the Health Monitor issues an alert through the specified reporting channels.

An alert is generated in response to either a change to a non-normal state or the value of the health indicator falling into a warning or alarm zone based on defined threshold boundaries. For health indicators measuring distinct states, an attention type alert is issued if a non-normal state is registered. For health indicators measuring a continuous range of values, threshold values define boundaries or zones for normal, warning, and alarm states. If, for example, the value enters the threshold range of values that defines an alarm zone, an alarm type alert is issued to indicate that the problem needs immediate attention. There are three types of alerts: alarm, warning, and attention.

Health Monitor information can be accessed through the Health Center, Web Health Center, the CLP, or APIs. Health indicator configuration is available through these same tools.

The following are the categories of health indicators:

- Table space storage
- Sorting
- Database management system
- Database
- Logging
- Application concurrency
- Package and catalog caches, and workspaces
- Memory

## Command line examples

```
db2 get alert cfg for database on musicdb using db.spilled_sorts
```

### Alert Configuration

|                             |                                           |
|-----------------------------|-------------------------------------------|
| Indicator Name              | = db.spilled_sorts                        |
| Type                        | = Threshold-based                         |
| Warning                     | = 30                                      |
| Alarm                       | = 50                                      |
| Sensitivity                 | = 0                                       |
| Formula                     | = (db.sort_overflows/db.total_sorts)*100; |
| Actions                     | = Disabled                                |
| Threshold or State checking | = Enabled                                 |

SQL22004N Cannot find the requested configuration for the given object.  
Returning default configuration for "databases".

```
db2 get description for health indicator db.spilled_sorts
```

### DESCRIPTION FOR db.spilled\_sorts

Sorting is considered healthy if there is sufficient heap space in which to perform sorting and sorts do not overflow unnecessarily. Sorts that overflow to disk can cause significant performance degradation. If this occurs, an alert may be generated. The indicator is calculated using the formula:  $(\text{db.sort\_overflows} / \text{db.total\_sorts}) * 100$ . The system monitor data element db.sort\_overflows is the total number of sorts that ran out of sort heap and may have required disk space for temporary storage. The data element db.total\_sorts is the total number of sorts that have been executed.

```
db2 update alert cfg for database on musicdb using db.spilled_sorts set alarm 60, warning 40
DB20000I The UPDATE ALERT CONFIGURATION command completed successfully.
```

```
db2 get alert cfg for database on musicdb using db.spilled_sorts
```

### Alert Configuration

|                             |                                           |
|-----------------------------|-------------------------------------------|
| Indicator Name              | = db.spilled_sorts                        |
| Type                        | = Threshold-based                         |
| Warning                     | = 40                                      |
| Alarm                       | = 60                                      |
| Sensitivity                 | = 0                                       |
| Formula                     | = (db.sort_overflows/db.total_sorts)*100; |
| Actions                     | = Disabled                                |
| Threshold or State checking | = Enabled                                 |

Notice that, compared with example #1, there is now a specific setting for the MUSICDB database, so the SQL22004 message is no longer returned.

## Instructor notes:

**Purpose** — Identify what the health indicator settings are used for.

**Details** — The health indicators allow you to indicate whether or not an indicator is to be evaluated, and if it is to be evaluated, what the warning and alarm threshold levels should be. The command line interface also provides access to the descriptions of the health indicators, and the ability to update them. See the student notes for an example of getting the current alert configuration, getting a description for a health indicator, updating the health indicator, and then getting the current alert configuration again (verifying the change).

**Additional Information** — More information can be found about the elements in the *System Monitor Guide and Reference*. Here is the documentation from that manual on db.spilled\_sorts:

### Percentage of sorts that overflowed — db.spilled\_sorts

#### Description:

- Sorting is considered healthy if there is sufficient heap space in which to perform sorting and sorts do not overflow unnecessarily.
- Sorts that overflow to disk can cause significant performance degradation. If this occurs, an alert may be generated.
- The indicator is calculated using the formula:  $(\text{db.sort\_overflows} / \text{db.total\_sorts}) * 100$ . The system monitor element db.sort\_overflows is the total number of sorts that ran out of sort heap and may have required disk space for temporary storage. The element db.total\_sorts is the total number of sorts that have been executed.

#### Additional Information:

A display string containing the values of db2.max\_priv\_sort\_mem and sortheap.

#### Resolution:

- Increase sort heap

Increase the database configuration parameter sortheap so that spilling won't occur unnecessarily. Update the sortheap value to 100% of the current value of the snapshot element db2.max\_priv\_sort\_mem. The sortheap memory utilization frequency statistic, included in the additional information for this health indicator, provides an indication of behavior on this attribute.

From the Control Center:

1. Expand the object tree until you find the Databases folder, and expand the folder until you find the database that you want.
2. Right-click the database, and click **Configure...** from the pop-up menu. The Configure Database dialog opens.
3. On the **Performance** tab, update the sort heap parameter as recommended above, and click **OK** to apply the update.

From the Command Line Processor, type, as shown in the following example:

```
UPDATE DATABASE CONFIGURATION FOR DATABASE name USING SORTHEAP  
size
```

- Tune workload

You can run the Design Advisor to tune the database performance for your workload by adding indexes and materialized query tables. This can help reduce the need for sorting. You will need to provide your query workload and database name. The Design Advisor will evaluate the existing indexes and materialized query tables in terms of the workload and recommend any new objects required.

From the Control Center:

1. Expand the object tree until you find the Databases folder, and expand the folder until you find the database that you want.
2. Right-click the database, and click **Optimize for Workload Performance...** from the pop-up menu. The Design Advisor opens.
3. Follow the steps of the Design Advisor to optimize query performance and apply the recommendations of the Design Advisor.

This function is also available from the command line by typing:

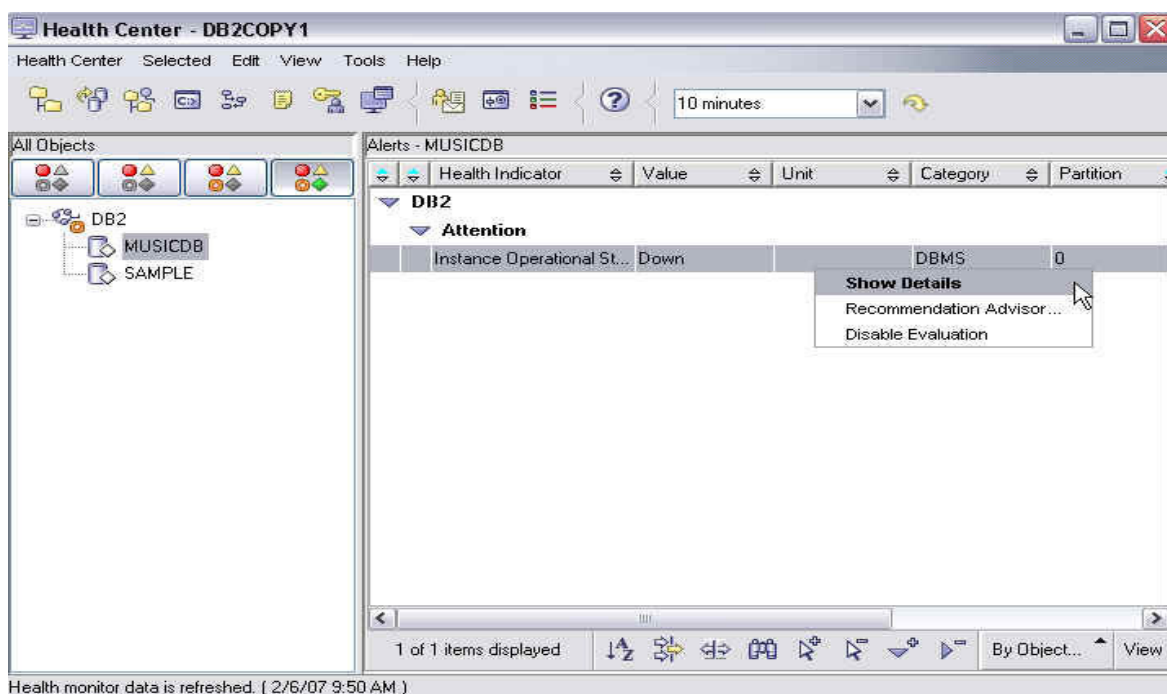
```
db2adviz
```

Certification objective: 4D Ability to configure Health Monitor using the Health Center.

**Transition Statement** — Let's look at what happens when the health monitor detects that a threshold has been breached (the value has gone into the warning or alarm zone).



## Health Monitor — alerts view



`db2 get health snapshot for database manager`

© Copyright IBM Corporation 2007

Figure 9-21. Health Monitor — alerts view

CF238.3

### Notes:

When an alert is raised, two things can occur:

- Alert notifications can be sent by e-mail or to a pager address, allowing you to contact whoever is responsible for a system.
- Preconfigured actions can be taken. For example, a script or a task (implemented from the new Task Center) can be run.

Using the Health Monitor's drill-down capability, you can access details about current alerts and obtain a list of recommended actions that describe how to resolve the alert.

## db2 get health snapshot for database manager

### Database Manager Health Snapshot

Node name =  
Node type = Enterprise Server Edition with local and remote clients  
Instance name = DB2  
Snapshot timestamp = 10/30/2006 17:59:32.826677

Number of database partitions in DB2 instance = 1  
Start Database Manager timestamp = 10/23/2006 11:59:14.567027  
Instance highest severity alert state = Alarm

#### Health Indicators:

Indicator Name = db2.db2\_op\_status  
Value = 0  
Evaluation timestamp = 10/30/2006 17:55:30.090000  
Alert state = Normal

Indicator Name = db2.mon\_heap\_util  
Value = 100  
Unit = %  
Evaluation timestamp = 10/30/2006 17:55:30.090000  
Alert state = Alarm

***Instructor notes:***

**Purpose** — Identify how you can find out about alarm, warning, or attention conditions.

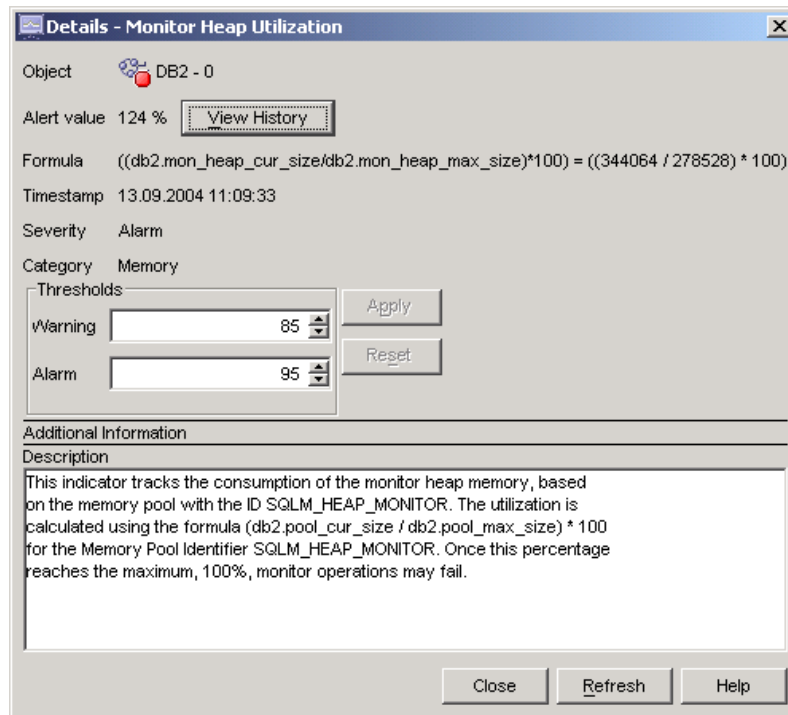
**Details** — In the Health Center, when an object goes into alarm, warning, or attention condition, you will see a change in the object (for example, the red stop sign indicates an alarm condition). The information in the right pane indicates that the sort overflows are causing the alarm.

From the command line interface, you can also get a health snapshot to determine what indicators have placed the monitored object into a non-normal status.

**Additional Information** — Certification objective: 4D Ability to configure Health Monitor using the Health Center.

**Transition Statement** — Let's look at how you can proceed when an alert is found.

## Alarm details



`db2 get recommendations for health indicator db2.mon_heap_util`

© Copyright IBM Corporation 2007

Figure 9-22. Alarm details

CF238.3

### Notes:

You can follow one of the recommended actions to address the alert. If the recommended action is to make a database or database manager configuration change, a new value will be recommended, and you can implement the recommendation by clicking a button. In other cases, the recommendation will be to investigate the problem further by launching a tool, such as the CLP or the Memory Visualizer.

`db2 get recommendations for health indicator db2.mon_heap_util`

Recommendations:

Recommendation: Investigate memory usage of monitor heap.

This parameter determines the amount of the memory, in pages, to allocate for database system monitor data. Memory is allocated from the monitor heap for database monitoring activities such as taking a snapshot, turning on a monitor switch, resetting a monitor, or activating an event monitor.

For more information on the monitor heap, refer to the DB2 Information Center.

Investigate the amount of memory that was used for the monitor heap over time to determine the most appropriate value for the monitor heap configuration parameter. The database system monitor tracks the highest amount of memory that was used for the monitor heap.

Take one of the following actions:

Launch DB2 tool: Memory Visualizer

The Memory Visualizer is used to monitor memory allocation within a DB2 instance. It can be used to monitor overall memory usage, and to update configuration parameters for individual memory components.

To open the Memory Visualizer:

1. From the Control Center, expand the object tree until you find the instances folder.
2. Click the instances folder. Any existing instances are displayed in the contents pane on the right side of the window.
3. Right-click the instance that you want in the Contents pane, and click View Memory Usage in the pop-up menu. The Memory Visualizer opens.

To start the Memory Visualizer from the command line issue the `db2memvis` command.

The Memory Visualizer displays a hierarchical list of memory pools for the database manager. Monitor Heap is listed under the Database Manager Memory group for each database. On Windows, it is listed under the Database Manager Shared Memory group.

Click the check box on the Show Plot column for the Monitor Heap row to add the element to the plot.

Recommendation: Increase the monitor heap size.

Increase the database manager configuration parameter `mon_heap_sz`

sufficiently to move utilization to normal operating levels. Set the new value of `mon_heap_sz` to be equal to  $(\text{pool\_cur\_size} / (4096 * U))$  where  $U$  is the desired utilization rate. For example, if the desired utilization rate is 60% of the warning threshold level, which has been set at 75%, then  $U = 0.6 * 0.75 = 0.45$  (or 45%).

Take one of the following actions:

Launch DB2 tool: DBM Configuration Window

The DBM Configuration window can be used to view and update database manager configuration parameters.

To open the DBM Configuration window:

1. From the Control Center, expand the object tree until you find the instances folder.
2. Click the instances folder. Any existing instances are displayed in the contents pane on the right side of the window.
3. Right-click the instance that you want in the Contents pane, and click Configure Parameters in the pop-up menu. The DBM Configuration window opens.

Expand the Performance category, then update the monitor heap size parameter as suggested and click OK to apply the update.

From the Command Line Processor, issue the commands shown in the following example:

```
UPDATE DATABASE MANAGER CONFIGURATION USING MON_HEAP_SZ size
```

## ***Instructor notes:***

**Purpose** — Identify how to proceed with processing an alert.

**Details** — From the GUI, you can get details and recommendations of the next action to take. You get a description of the actions from the command line interface.

**Additional Information** — Certification objective: 4D Ability to configure Health Monitor using the Health Center.

**Transition Statement** — Let's look at how to use DB2 trace.

## Independent trace facility — *db2trc*

```
db2trc on
```

```
=====
```

```
db2trc dump trace.bin
```

```
db2trc off
```

```
db2trc fmt trace.bin trace.fmt
```

```
edit trace_fmt (Windows)
```

```
vi trace.fmt (UNIX)
```

### trace.fmt

```
DB2 cei_retcode base_sys_utilities sqlgsem (1.23.5.97)
pid 17027; fid 1; cpid 5; time 0; trace_point 254
return_code = 000000 = 0

DB2 cei_entry oper_system_services sqlcinca (1.20.151.121)
pid 17027; fid 1; cpid 5; time 0; trace_point 0
```

© Copyright IBM Corporation 2007

Figure 9-23. Independent trace facility — *db2trc*

CF238.3

### Notes:

DB2 provides the `db2trc` command to trace DB2 events. The `db2trc` command collects event information, dumps the trace to a file, and formats the trace dump file.

*db2trc* should be performed with the assistance of your IBM Support Center. SYSADM, SYSCTRL, or SYSMANT authority is needed to run *db2trc*.

### Running the `db2trc` command:

- Start the trace.

```
db2trc on
```

Perform the action you wish to trace.

- Dump the trace to a file. If no directory path is specified, then the trace file will be written to the current directory.

```
db2trc dump trace.bin
```

- Turn the trace off.

```
db2trc off
```

- Format the dump trace file. The `fmt` option lists each event chronologically. The `flw` option sorts each event by process or thread. If the output is not redirected, then the output will go to the terminal.

```
db2trc fmt trace.bin trace.format
```

```
db2trc flw trace.bin trace.flow
```

- Send the formatted trace to the IBM Support Center.

`db2trc on -l 10000000` sets aside a 10 MB buffer for the trace. Here `-l` is the letter *e* — not the digit *one*.

The following are trace options:

- f filename** Specifies that trace information should be continuously written to the specified file, until `db2trc` is turned off.
- p pid.tid** Only enables the trace facility for the specified process IDs (`pid`) and thread IDs (`tid`). A period must be included if a `tid` is specified.
- l buffer\_size** Specifies retention of the last trace records (the first records are overwritten when buffer is full)
- i buffer\_size** Specifies retention of the initial trace records (no more records are written to the buffer once it is full)

Refer to the *Command Reference* for further information.



***Instructor notes:***

**Purpose** — Identify how trace may be used.

**Details** — The db2 trace facility is only used in conjunction with the IBM Support Center. Frequently, when you open a problem report, if the problem is reproducible, you will be asked to turn trace on and reproduce the problem. The options to use when starting the tool will be given to you by service. You will then format the file, usually in both formatted and flow formats, and send it to the IBM Support Center.

**Additional Information** — More information about the trace facility is addressed in CF45.

**Transition Statement** — Let's look at EXPLAIN.



## 9.3 EXPLAIN

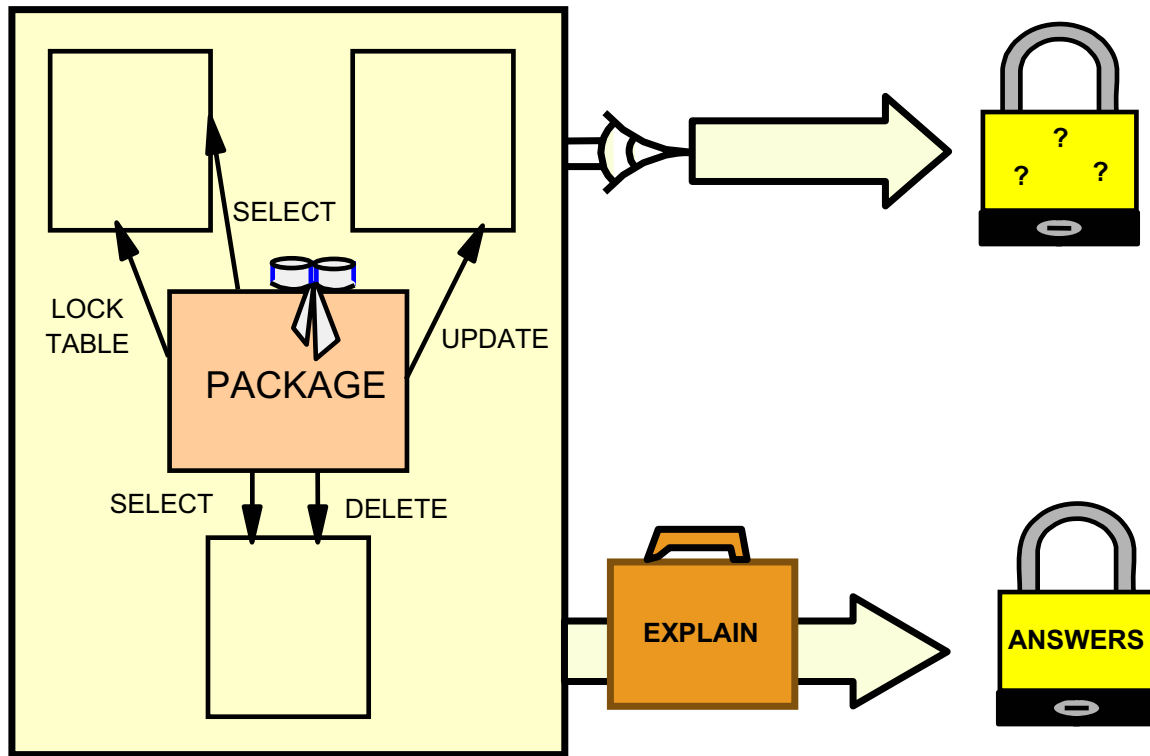
### Instructor topic introduction

**What students will do** — Get information about how EXPLAIN can help them understand and/or solve performance issues.

**How students will do it** — Listen to the lecture.

**What students will learn** — The function and value of EXPLAIN.

## Explain tool



© Copyright IBM Corporation 2007

Figure 9-24. Explain tool

CF238.3

### Notes:

Analyzing an application by viewing a listing may provide insight regarding the application's function and flow. However, the manner in which SQL will be executed is externalized via EXPLAIN.

Questions that could be answered through the use of EXPLAIN include:

- What indexes are used for this statement?
- What is the order of table access?
- What are the locking requirements for this program?
- Is any sorting required, and is it planned to be piped?

Consult the *Administration Guide* for additional details.

***Instructor notes:***

**Purpose** — Identify the basic purpose of EXPLAIN.

**Details** — SQL statements, whether issued through dynamic SQL or through a package, access data in the database, but the techniques that DB2 uses to access that data remain hidden unless you use the EXPLAIN facility.

**Additional Information** —

**Transition Statement** — There are several EXPLAIN tools available in DB2.

## EXPLAIN facility tools



| Requirements                                          | Visual Explain | Explain Tables | db2exfmt | db2expln |
|-------------------------------------------------------|----------------|----------------|----------|----------|
| GUI interface                                         | ✓              |                |          |          |
| Text output                                           |                |                | ✓        | ✓        |
| "Quick and dirty" static SQL analysis                 |                |                |          | ✓        |
| Static SQL supported                                  | ✓              | ✓              | ✓        | ✓        |
| Dynamic SQL supported                                 | ✓              | ✓              | ✓        | ✓*       |
| CLI applications supported                            | ✓              | ✓              | ✓        |          |
| Available to DRDA application requesters              |                | ✓              |          |          |
| Detailed optimizer information                        | ✓              | ✓              | ✓        |          |
| Suited for analysis of multiple statements            |                | ✓              | ✓        | ✓        |
| Information available from within application         |                | ✓              |          |          |
| <b>Note:</b> * Creates static package and explains it |                |                |          |          |

© Copyright IBM Corporation 2007

Figure 9-25. EXPLAIN facility tools

CF238.3

### Notes:

The table summarizes the different tools available within the DB2 EXPLAIN facility and their individual characteristics. Use this table to select the tool most suitable for your environment and needs.

- Visual Explain allows for the analysis of access plan and optimizer information from the EXPLAIN tables through a graphical interface.
- The EXPLAIN Tables are accessible on all supported platforms and can contain information for both static and dynamic SQL statements. You can access the EXPLAIN tables using SQL statements, which allow for easy manipulation of the output and comparisons of the same query over time.
- db2exfmt allows you to obtain reports from the explain tables in a predefined format.
- db2expln will allow you to see the access plan information that is stored in the system catalog as part of a static package. It provides a text-based report on the strategy DB2 will use to execute the statements.

More details on the above EXPLAIN tools can be found in your *Administration Guide*.

***Instructor notes:***

**Purpose** — Identify the ways to access EXPLAIN.

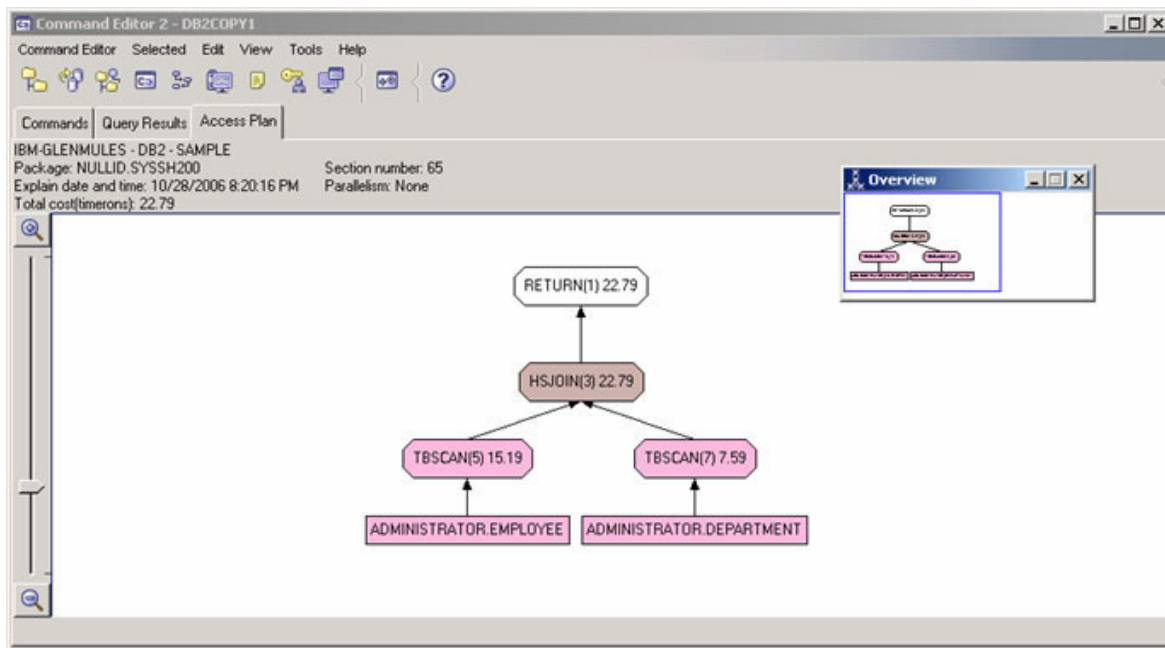
**Details** — EXPLAIN data can be accessed in a variety of ways. There is a GUI tool called Visual Explain. It formats data that has been collected in EXPLAIN tables. The EXPLAIN tables themselves can be queried directly. The EXPLAIN table data can be formatted into a text-based output using db2exfmt. Or else, a text-based tool, db2expln, can give a simple view of the EXPLAIN data that is stored in a package.

**Additional Information** — The function that used to be in dynexpln is now available in db2expln directly (V8).

Certification objective: 4A Ability to capture and analyze EXPLAIN/VISUAL EXPLAIN information.

**Transition Statement** — Let's look at an example of Visual Explain output.

# Visual explain access plan graph



```
SELECT *
FROM administrator.employee e, administrator.dept d
WHERE e.workdept = d.deptno
```

© Copyright IBM Corporation 2007

Figure 9-26. Visual explain access plan graph

CF238.3

## Notes:

The statement being explained on the graphic is:

```
SELECT *
FROM administrator.employee e, administrator.dept d
WHERE e.workdept = d.deptno
```

To access the Explain SQL Statement (Visual Explain) GUI:

- Right-click and select **Database**
  - Left-click and select **Explain SQL**

From the Explain SQL Statement GUI, an administrator can insert SQL text, save text, set a query tag, and set the optimization level.

Visual Explain allows for the analysis of access plan and optimizer information from the explain tables through a graphical interface. Both static and dynamic SQL statements can be analyzed using the tool. The analyzed statement can be either a previously bound static package or a previously explained dynamic SQL statement. Visual Explain is typically invoked from within the Command Editor.



There are two methods to gain access to Visual Explain from the Control Center:

- Explain statement history --> Open as details
- Packages --> (select the desired package) --> Open as Explain statement history

Visual Explain can also be accessed in the Command Center by identifying in the Command Center options that you want to automatically generate access plans, and then executing an explainable statement in the interactive page of the Command Center.

Visual Explain allows you to graph snapshots captured on another platform. For example, a Windows client can graph snapshots generated on a DB2 for Solaris server.

## ***Instructor notes:***

**Purpose** — Give an example of Visual Explain output.

**Details** — In this graph, reading from the bottom up, we can see that DB2 is accessing the STAFF table through an index (given the strange name, probably a primary key with a system generated name) and grouping the information. The SQL statement is a SELECT COUNT(\*).

**Additional Information** — Certification objective: 4A Ability to capture and analyze EXPLAIN/VISUAL EXPLAIN information.

**Transition Statement** — Let's look at additional considerations.

## 9.4 Additional commands

### Instructor topic introduction

**What students will do** — Additional commands that might be of interest.

**How students will do it** — Lecture, and later labs.

**What students will learn** — How they use these commands.

**How this will help students on their job** — These commands are commonly used, and can provide the opportunity to help diagnose problems or rectify situations.

## Additional commands

- 
- **db2 ? *command***
    - Request help text associated with command
  - **db2 ? *sqlnnnnN***
    - Requests help for a message specified by a valid SQLCODE *nnnn*
    - *N* = negative (most SQLCODES are negative), *W* = warning
  - **db2 DESCRIBE TABLE *tablename***
    - Displays columns of a table or a view
    - Optional SHOW DETAIL clause
  - **db2 DESCRIBE INDEXES FOR TABLE *tablename***
    - Displays indexes for a table
    - Optional SHOW DETAIL clause
  - Other uses for the DESCRIBE command

© Copyright IBM Corporation 2007

Figure 9-27. Additional commands

CF238.3

### Notes:

**db2 ? *command*** requests the help text associated with the command given. Note that this only provides help for commands, not for SQL statements.

**db2 ? *SQLnnnnN*** provides help for the message specified. At least a 4-digit number must be specified. The final letter (often “N” or “W” is optional), as the meaning is determined by the number.

**db2 describe table *table-name*** displays the columns of a table or a view.

**db2 describe table *table-name* show detail** gives you additional information.

**db2 describe indexes for table *table-name*** displays the indexes for a table.

You can apply **db2 ? *command*** to the DESCRIBE command itself. This will give you some additional ideas on what can be described:

```
DESCRIBE {[OUTPUT] {select-statement | call-statement | XQUERY xquery-statement} |  
         {TABLE | {INDEXES | DATA PARTITIONS} FOR TABLE} table-name [SHOW DETAIL]}
```

***Instructor notes:***

**Purpose** — Identify some additional commands that may be of interest.

**Details** — Just identify each command and how it can be used.

**Additional Information** —

**Transition Statement** — Let's look at another command.

## LIST ACTIVE DATABASES command



### db2 LIST ACTIVE DATABASES

```
Database name                = MUSICDB
Applications connected currently = 4
Database path                = /home/inst00/inst00/NODE0000/SQL00001/
```

© Copyright IBM Corporation 2007

Figure 9-28. LIST ACTIVE DATABASES command

CF238.3

### Notes:

The LIST ACTIVE DATABASES command can be used to display the active databases within an instance. The current number of active connections is also displayed for each database listed.

One of the following authorities is needed to run the LIST APPLICATIONS command:

- SYSADM
- SYSCTRL
- SYSMANT
- SYSMON

***Instructor notes:***

**Purpose** — Describe the LIST ACTIVE DATABASES command.

**Details** — This command shows all databases that are currently active. If the value for “Applications connected currently” is zero, this indicates that the database has been activated (via an ACTIVATE DATABASE command).

**Additional Information** —

**Transition Statement** — Let's look at LIST APPLICATIONS.

## LIST APPLICATIONS command

```
db2 LIST APPLICATIONS FOR DATABASE db_alias SHOW DETAIL
```

| Auth Id | Application Name | Appl.<br>Handle | Application Id             |
|---------|------------------|-----------------|----------------------------|
| -----   | -----            | -----           | -----                      |
| INST00  | db2bp_32         | 78              | *LOCAL.inst00.000327220543 |

| Seq#  | Number of<br>Agents | Coordinating<br>Node Number | Coordinating<br>pid/thread | Status      | Status<br>Change Time |
|-------|---------------------|-----------------------------|----------------------------|-------------|-----------------------|
| ----- | -----               | -----                       | -----                      | -----       | -----                 |
| 0001  | 1                   | 0                           | 230                        | UOW Waiting | Not Collected         |

| Node  | DBName  | DB Path                                |
|-------|---------|----------------------------------------|
| ----- | -----   | -----                                  |
| 0     | MUSICDB | /home/inst00/inst00/NODE0000/SQL00001/ |

© Copyright IBM Corporation 2007

Figure 9-29. LIST APPLICATIONS command

CF238.3

### Notes:

The LIST APPLICATIONS command displays an entry for each application connected to a specific database or to all databases within the instance (the default).

The output shows the application program name, authorization ID (user name), application handle, application ID, and the database name for each database application.

If the SHOW DETAIL parameter is used, the output will also display the application's sequence number, status, status change time, node, and database path.

One of the following authorities is needed to run the LIST APPLICATIONS command:

- SYSADM
- SYSCTRL
- SYSMANT
- SYSMON



***Instructor notes:***

**Purpose** — Identify the output from the LIST APPLICATIONS command.

**Details** — LIST APPLICATIONS returns the columns that are highlighted on the graphic. LIST APPLICATIONS SHOW DETAIL returns all the columns shown.

**Additional Information** —

**Transition Statement** — Suppose that, by looking at this information, you find the id of the application that is holding resources and needs to be forced off. Let's look at the FORCE APPLICATION command.

## FORCE APPLICATION command



```
db2 FORCE APPLICATION { ALL | (application-handle) }
```

- User application handle from LIST APPLICATIONS
- Rolls back uncommitted transactions

```
db2stop force
```

© Copyright IBM Corporation 2007

Figure 9-30. FORCE APPLICATION command

CF238.3

### **Notes:**

The FORCE APPLICATION command can be used to break the database connection between an application and its agent (db2agent). The FORCE APPLICATION command can be used to break the connections for ALL applications or specific applications.

When specifying specific applications, use the application handle number from the LIST APPLICATIONS command.

If an application is forced, an uncommitted Unit of Work will be rolled back. The force takes effect immediately. However, the command runs asynchronously and the user may regain control before all applications have been forced.

The FORCE command breaks the connection at the database server and does not terminate the database application. Do NOT use operating system commands to stop or kill a database agent. SYSADM, SYSCTRL, or SYSMON authority level is required to issue the FORCE APPLICATION command.

**Note**

Applications may also be forced while stopping the instance using the FORCE option on the db2stop command.

## ***Instructor notes:***

**Purpose** — Identify the FORCE APPLICATION command.

**Details** — The `force application` command can be used to force one or more applications off DB2. The ID to use is the application handle from the `list applications` command. If the application has an uncommitted unit of work, its work will be rolled back by the use of this command.

**Additional Information** — One of the issues that sometimes arises is that students want to force applications and then do an offline backup, but since the force executes asynchronously, applications may reestablish their connections before all of the other applications are forced off. In addition, applications may connect on other partitions while you are backing up a subset.

One suggestion would be to create two aliases in the system database directory for the database, for example `dss` and `dss1`. The users are directed to connect to database `dss`. When you are ready to do your offline backups, uncatalog the `dss` alias, and force the users. Now, they will be unable to reestablish their connection. You can do your backup, and then recatalog the `dss` database alias.

Another suggestion is to QUIESCE the database.

Another option for cleaning up all processes associated with an instance is `db2stop -kill`. This command should be used only as a last resort, as it kills processes and IPCs. However, it may be your only option to successfully clean up processes if DB2 crashes in an abnormal way.

Certification objective: 1D Knowledge of the DB2 force command.

**Transition Statement** — An opportunity for further study.

## The *db2set* command

- Command line tool for DB2 Profile Registry administration
- Displays, sets, resets, or removes profile variables

```
db2set variable=value
-g
-i instance [db-partition-number]
-n DAS node [[-u user id] [-p password]]
-u
-ul
-ur
-p
-r
-l
-lr
-v
-? (or -h)
-all
```

© Copyright IBM Corporation 2007

Figure 9-31. The *db2set* command

CF238.3

### Notes:

The *db2set* command allows us to display, set (or reset), and remove various profile variables.

Some of these settings are important to diagnostic analysis and hence we are reviewing this command here.

## ***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** We have one last command to look at. This is a relative new command and is expected to grow in value in future releases. It allows us to look at statistics from a running database instance.

## Retrieve stats from a running instance: *db2pd*



*db2pd* (DB2 performance data)

- Allows you to retrieve statistics from a running DB2 instance or database
- Provides information useful for troubleshooting and problem determination, performance improvement, and application development design:
  - Locks / bufferpools / table spaces / containers / dynamic SQL statements / agents / applications / memory pools and sets / transactions / logs / ...
- Collects data without acquiring any latches and without using any engine resources — thus the data is expected to be constantly changing (and hence not completely accurate)

© Copyright IBM Corporation 2007

Figure 9-32. Retrieve stats from a running instance: *db2pd*

CF238.3

### Notes:

**db2pd** is a new utility that can be used to retrieve statistics from a running DB2 instance or database.

The tool can provide a wide range of information useful for troubleshooting and problem determination, performance improvements, and application development design, including:

- locks
- buffer pools
- table spaces
- containers
- dynamic SQL statements
- agents
- applications
- memory pools and sets
- transactions
- logs

The tool collects this data without acquiring any latches and without using any engine resources. It is therefore possible (and expected) to retrieve data that is constantly changing while **db2pd** is collecting it — thus this data may not be completely accurate or unchanging (static) at the particular instance in time when the report is complete.

The primary benefit is the ability to collect and analyze data from a running instance or database without acquiring latches. Accordingly, we have fast retrieval and no competition for engine resources.

The command is modelled in concept after the **onstat** utility used with IBM Informix instances.

In a number of the options, `file=<filename>` is used to specify an output file name for the results of the command. A similar effect can be achieved by using output file redirection (stdout or >). Note that no spaces are generally allowed within `file=<filename>`.

### **Syntax (using db2pd -h)**

#### **Usage:**

```
-h | -help [file=<filename>]
    Help
-v | -version [file=<filename>]
    Version
-osinfo [disk] [file=<filename>]
    Operating System Information
-dbpartitionnum <num>[,<num>]
    Database Partition Number(s)
-alldbpartitionnums
    All partition numbers
-database | -db <database>[,<database>]
    Database(s)
-alldatabases | -alldbs
    All Active Databases
-inst
    Instance scope output
-file <filename>
    All Output to Filename
-command <filename>
    Read in predefined options
-interactive
    Interactive
-full
    Expand output to full length
-repeat [num sec] [count]
    Repeat every num seconds (default 5) count times
-everything
    All options on all database partitions
```

#### **Instance scope options:**

```
agents [db=<database>] [ [agent=<agentid>] | [application=<appid>] ]
    [file=<filename>]
    Agents
-fcm [file=<filename>]
    FCM Information
```



```

-mempools [file=<filename>]
    Memory Pools
-memsets [file=<filename>]
    Memory Sets
-dbmcfg [file=<filename>]
    DBM Config
-sysplex [db=<database>] [file=<filename>]
    Sysplex List
-utilities [file=<filename>]
    Utilities

```

### Database scope options:

```

-applications [ [application=<appid>] | [agent=<agentid>] ] [file=<filename>]
    Applications
-transactions [tran=<tranhdl>] [app=<apphdl>] [file=<filename>]
    Transactions
-bufferpools [file=<filename>]
    Buffer Pools
-logs [file=<filename>]
    Transaction Logs
-locks [tran=<tranhdl>] [file=<filename>] [showlocks]
    Locks
-tablespaces [file=<filename>] [group] [tablespace=<tablespace id>]
    Tablespaces/Containers
-dynamic [file=<filename>]
    Dynamic Cache
-static [file=<filename>]
    Static Cache
-mempools [file=<filename>]
    Memory Pools
-memsets [file=<filename>]
    Memory Sets
-dbcfg [file=<filename>]
    Database Config
-catalogcache [file=<filename>]
    Catalog Cache
-tcbstats [all|index] [tbspaceid=<tbspaceid>] [tableid=<tableid>]]
    [file=<filename>]
    Table Control Block Stats
-reorg [file=<filename>]
    Table Reorg Stats
-recovery [file=<filename>]
    Recovery Status
-reopt [file=<filename>]
    Reoptimized SQL Statements

```

### Examples:

```

db2pd -dbpartitionnum 0,1 -db sample -locks
db2pd -dbp 0,1 -database sample -locks app=<appid>
db2pd -alldbp -alldbs

```

## Some Sample Reports

### **Operating System Information (db2pd -osinfo):**

```
OSName:      WIN32_NT
NodeName:    IBM-VC5QJ67OIIYZ
Version:     5.0
Release:     Service Pack 4
Machine:     x86 Family 6, model 8, stepping 6

CPU Information:
TotalCPU      OnlineCPU    ConfigCPU    Speed(MHz)   HMTDegree
1             1             1            796          1

Physical Memory and Swap (Megabytes):
TotalMem      FreeMem        AvailMem     TotalSwap    FreeSwap
256           9             9            2177         1340

Virtual Memory (Megabytes):
Total         Reserved      Available    Free
2433          n/a           n/a          1349
```

### **Dynamic SQL statements held in server cache (db2pd -dynamic):**

```
F:\IBM\DB2\SQLLIB\BIN>db2 connect to sample
```

#### Database Connection Information

```
Database server      = DB2/NT 9.1.0
SQL authorization ID = ADMINIST...
Local database alias = SAMPLE
```

```
F:\IBM\DB2\SQLLIB\BIN>db2 "SELECT * FROM employee"
```

| EMPNO     | FIRSTNME  | MIDINIT | LASTNAME | WORKDEPT | PHONENO | HIREDATE   | JOB     | EDLEVEL |
|-----------|-----------|---------|----------|----------|---------|------------|---------|---------|
| 000010    | CHRISTINE | I       | HAAS     | A00      | 3978    | 01/01/1965 | PRES    | 18      |
| F 08/24/1 | 4220.00   |         |          |          |         |            |         |         |
| 000020    | MICHAEL   | L       | THOMPSON | B01      | 3476    | 10/10/1973 | MANAGER | 18      |
| M 02/02/1 | 3300.00   |         |          |          |         |            |         |         |
| 000030    | SALLY     | A       | KWAN     | C01      | 4738    | 04/05/1975 | MANAGER | 20      |
| F 05/11/1 | 3060.00   |         |          |          |         |            |         |         |
| ...       |           |         |          |          |         |            |         |         |

```
F:\IBM\DB2\SQLLIB\BIN>db2pd -dynamic -db sample
```

```
Database Partition 0 -- Database SAMPLE -- Active -- Up 0 days 00:02:59
```

```
Dynamic Cache:
Current Memory Used      217553
Total Heap Size          1271398
Cache Overflow Flag      0
Number of References      2
Number of Statement Inserts 3
Number of Statement Deletes 0
Number of Variation Inserts 2
```

Number of Statements                      3

Dynamic SQL Statements:

| Address                                               | AnchID | StmtID | NumEnv | NumVar | NumRef | NumExe | Text                        |
|-------------------------------------------------------|--------|--------|--------|--------|--------|--------|-----------------------------|
| 0x034B05D0                                            | 101    | 1      | 0      | 0      | 0      | 0      | SET CURRENT LOCALE LC_CTYPE |
| 'en_US'                                               |        |        |        |        |        |        |                             |
| 0x034B0830                                            | 144    | 1      | 1      | 1      | 1      | 1      | SELECT * FROM employee      |
| 0x034B6A20                                            | 170    | 1      | 1      | 1      | 1      | 1      | SELECT COUNT(*) FROM        |
| SYSCAT.PROCEDURES WHERE PROCNAME='SYSINSTALLROUTINES' |        |        |        |        |        |        |                             |

Dynamic SQL Environments:

| Address    | AnchID | StmtID | EnvID | Iso | QOpt | Blk |
|------------|--------|--------|-------|-----|------|-----|
| 0x034B08F0 | 144    | 1      | 1     | CS  | 5    | B   |
| 0x034B6B10 | 170    | 1      | 1     | CS  | 5    | B   |

Dynamic SQL Variations:

| Address    | AnchID | StmtID | EnvID | VarID | NumRef | Typ | Lockname                   |
|------------|--------|--------|-------|-------|--------|-----|----------------------------|
| 0x034B1230 | 144    | 1      | 1     | 1     | 1      | 6   | 01000000010000000100900056 |
| 0x034B7450 | 170    | 1      | 1     | 1     | 1      | 6   | 01000000010000000100aa0056 |

## ***Instructor notes:***

**Purpose** — Introduce the student to the **db2pd** command.

**Details** — The db2pd command has its conceptual origin in the IBM Informix onstat command (“Online dynamic server STATistics”). This command has many, many features and thus you should have the student run:

```
db2pd -h
```

to list all the current possibilities (and these will probably be added to in future releases of DB2). Do some demonstrations of simple commands, such as:

```
db2pd -osinfo
```

**Additional information** — Certification objective: 4E Ability to identify the functions of Problem Determination Tools (for example, db2pd, db2mtrk).

**Transition statement** — Obviously we cannot cover all matters in problem determination or performance tuning (they are related at the edge). You will want to go the following course — DB2 Performance Tuning and Monitoring Workshop (CF41), for single partition databases — when you have had some experience and are ready to go to the next level of understanding.

# Performance tuning and monitoring workshop



## *DB2 Performance Tuning and Monitoring Workshop (CF41)*

- Audience: database designers, database administrators, application developers
- Objectives:
  - Define the impact of database design on database performance
  - Describe database application programming considerations and how they affect performance
  - Identify and describe the parameters that affect performance
  - Tune these parameters to achieve optimal performance
  - Identify and use the tools that assist in monitoring and tuning of a database

© Copyright IBM Corporation 2007

Figure 9-33. Performance tuning and monitoring workshop

CF238.3

### **Notes:**

**Overview:** Learn how to tune for optimum performance the IBM DB2 UDB for Linux, UNIX, and Windows relational database management system and associated applications written for this environment. Learn about DB2 UDB for Linux, UNIX, and Windows on a serial processor, in a nonpartitioned database environment. Explore performance issues affecting the design of the database and applications using the database, the major database performance parameters, and the different tools that assist in performance monitoring and tuning.

### **Objectives:**

- Define the impact of database design (tables, indexes, and data placement) on database performance
- Describe database application programming considerations and how they affect performance
- Identify and describe the parameters (database, non-database) that affect performance
- Tune parameters to achieve optimum performance
- Identify and use the tools that assist in monitoring and tuning of a database

## ***Instructor notes:***

**Purpose** — Identify a follow-on course for the students.

**Details** — CF41 is the Performance Monitoring and Tuning course for the single partition environment.

**Additional Information** —

**Transition Statement** — First, we have a checkpoint. And, then, let's look at what we have addressed in this unit.

# Checkpoint

## Exercise — Unit Checkpoint

- \_\_\_ 1. What is the database manager configuration parameter that defines the level of error logging done to the administration notification log?

---

**NOTIFYLEVEL**

- \_\_\_ 2. What tools can be used to monitor DB2 activity?

---

**Snapshot Monitor, Event Monitor, Health Monitor**

- \_\_\_ 3. What type of tool is used to determine the access strategy DB2 will use to execute an SQL statement?

---

**Explain**

## Unit summary



Having completed this unit, you should be able to:

- Collect information for problem analysis and resolution
- Use error logs for basic problem analysis
- Describe four types of monitors:
  - Snapshot Monitor
  - Event Monitor
  - Activity Monitor
  - Health Monitor
- Describe the function of EXPLAIN and use this facility to assist basic analysis
- Use a series of basic commands to better work with connections and sessions
- Retrieve statistics and other information from a running DB2 instance

© Copyright IBM Corporation 2007

Figure 9-34. Unit summary

CF238.3

### Notes:



***Instructor notes:***

**Purpose** — Review the unit objectives.

**Details** — Question the students on what they've learned.

**Additional information** —

**Transition statement** — Now turn your attention to the Problem Determination lab.



# Unit 10. Application issues and performance

## Estimated time

01:00

## What this unit is about

DB2 provides many options for application development. Such options include stored procedures, static and dynamic embedded SQL, call level interface (CLI), and application programming interface (API) calls.

The developer must choose the correct interface to meet the application and installation requirements. No single interface is necessarily the correct choice for all requirements that an installation may have.

Application development projects that meet installation requirements from performance, security, and business process perspectives must require application programming and administration expertise.

This unit describes the interfaces available and provides some insight regarding the features and functions of each interface. The intent of the instruction is not to provide the detailed coding requirements of each interface, but to define the interfaces at a conceptual level.

This unit also provides the database administrator with the information necessary to properly manage the database with respect to application programs.

## What you should be able to do

After completing this unit, you should be able to:

- Prepare applications that access DB2 data for execution
- Use PRECOMPILE and BIND options that are appropriate for specific application requirements
- Describe the application alternatives available to access DB2 data or request other DB2 functions
- List reasons for REBIND of application programs and execute this command
- Define the concept of single dimensional clustering, and determine the proper cluster sequence, if any, for DB2 tables

- Use RUNSTATS, REORGCHK, and REORG to enhance application performance
- Work with the Explain facility

## How you will check your progress

Accountability:

- Checkpoint
- Machine exercises

## References

*Administration Guide*

*Command Reference*

*SQL Reference*

## Unit objectives

After completing this unit, you should be able to:

- Prepare applications that access DB2 data for execution
- Use PRECOMPILE and BIND options that are appropriate for specific application requirements
- Describe the application alternatives available to access DB2 data or request other DB2 functions
- List reasons for REBIND of application programs and execute this command
- Define the concept of single dimensional clustering, and determine the proper cluster sequence, if any, for DB2 tables
- Use RUNSTATS, REORGCHK, and REORG to enhance application performance

© Copyright IBM Corporation 2007

Figure 10-1. Unit objectives

CF238.3

### Notes:

## ***Instructor notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Identify what will be addressed in this unit.

Although this unit has detailed information about static embedded SQL applications, you should judge based on your audience how much of this needs to be addressed. For example, if your audience will all be doing dynamic SQL, then address static embedded SQL very lightly, but use it to explain what a package is, and the fact that characteristics of a program are carried in the package.

**This unit is quite long, so for time considerations, do not dwell on the subjects. Instead, concentrate on the lab. Try to move this unit along by not getting into lengthy discussions of the content. Students can read the notes if they need deeper detail.**

**Transition Statement** — Let's begin by listing the interfaces that are the subject of this unit.

## 10.1 Application alternatives

### Instructor topic introduction

**What students will do** — Checkpoint exercises to determine the correct application alternative to use.

**How students will do it** — Listen to lecture.

**What students will learn** — What alternatives there are in coding applications.

**How this will help students on their job** — A database administrator is frequently called upon to advise application programmers concerning solutions to various business problems. Although the DBA may not need to implement a given solution, the DBA should be aware of the strengths and weaknesses of each alternative.

## Application alternatives

- 
- Database Engine Access
    - Static SQL
    - Dynamic SQL
    - Application Programming Interfaces (APIs)
  - Programming Interfaces
    - Embedded SQL
    - Call Level Interface (CLI)
      - Open Database Connectivity (ODBC)
    - Java
      - Java Database Connect (JDBC)

© Copyright IBM Corporation 2007

Figure 10-2. Application alternatives

CF238.3

### Notes:

As a database administrator, you will generally be involved in assisting application programmers with decisions regarding the interfaces to DB2 data. This topic does not provide details on the coding intricacies of each alternative, but will provide insight concerning the strengths and weaknesses of each alternative.

Access to the database engine from an application program is through one of three methods.

1. Static or Dynamic SQL
2. Application Programming Interfaces (APIs)

Application programs interface to the Client Application Enabler component using one of the following interfaces:

- Embedded SQL
- Call Level Interface (CLI)
  - Open Database Connectivity (ODBC)
- Java

There are also several DB2 performance features that can be implemented, such as Stored Procedures.



***Instructor Notes:***

**Purpose** — Introduce concepts of the ways in which applications can access DB2 data.

**Details** — This course is not intended to make the student an expert on any of the options listed, but will provide the DBA with insight concerning the options so that advice can be provided when determining application architecture.

**Transition Statement** — First, let's look at the different options available to access the database engine.



## 10.2 Database engine access

### Instructor topic introduction

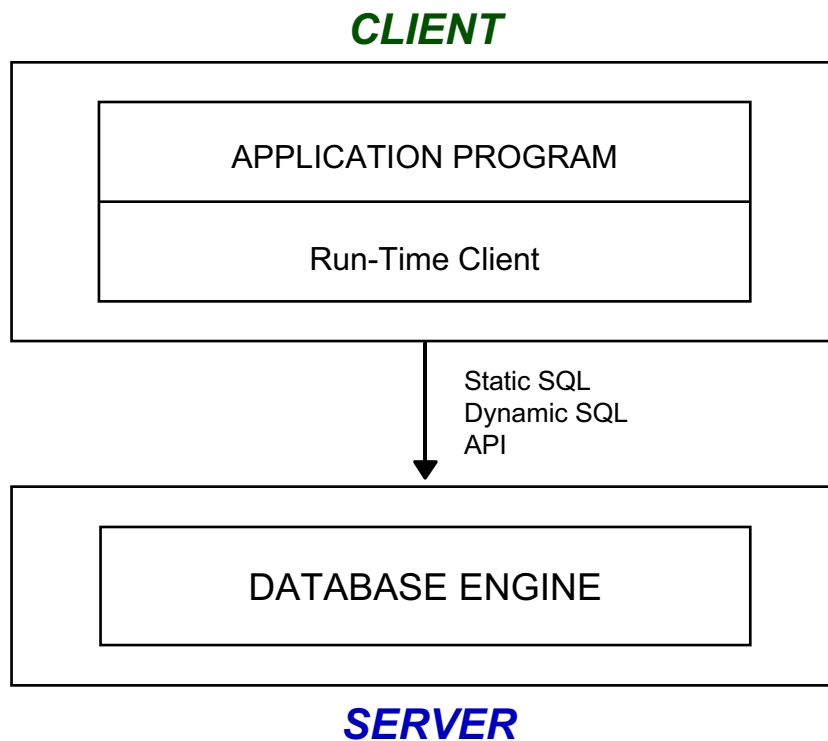
**What students will do** — Checkpoint exercises to determine the correct application alternative to use.

**How students will do it** — Listen to lecture.

**What students will learn** — What alternatives there are in coding applications.

**How this will help students on their job** — A database administrator is frequently called upon to advise application programmers concerning solutions to various business problems. Although the DBA may not need to implement a given solution, the DBA should be aware of the strengths and weaknesses of each alternative.

## Database engine access



© Copyright IBM Corporation 2007

Figure 10-3. Database engine access

CF238.3

### Notes:

An application program gets access to data through the Run-Time Client (RTC) code. An application will either:

- Request a static SQL statement to be executed on the server
- Pass a dynamic SQL statement to the server to be optimized and executed on the server
- Use a program API to request some database function to be performed on the server (such as backup, restore, load, and so forth)

The other possibility is the DB2 Run-Time Client Lite (DB2 RTCL) which is an installable component that makes it much easier to provide access to DB2 servers from Windows-based applications. The DB2 RTCL is designed to be redistributable by independent software vendors and to be used for application distribution in mass deployment scenarios. Similar to the DB2 Run-Time Client component, the DB2 RTCL provides application interfaces and the network libraries that are required for applications to run.

The main distinguishing features of the DB2 RTCL are:

- Significantly smaller disk footprint
- Shipped as a single executable, making it easy to redistribute
- Windows Installer Merge Module (.msm file) is available which simplifies the integration within a larger application

## ***Instructor Notes:***

**Purpose** — To describe the methods for accessing the database engine.

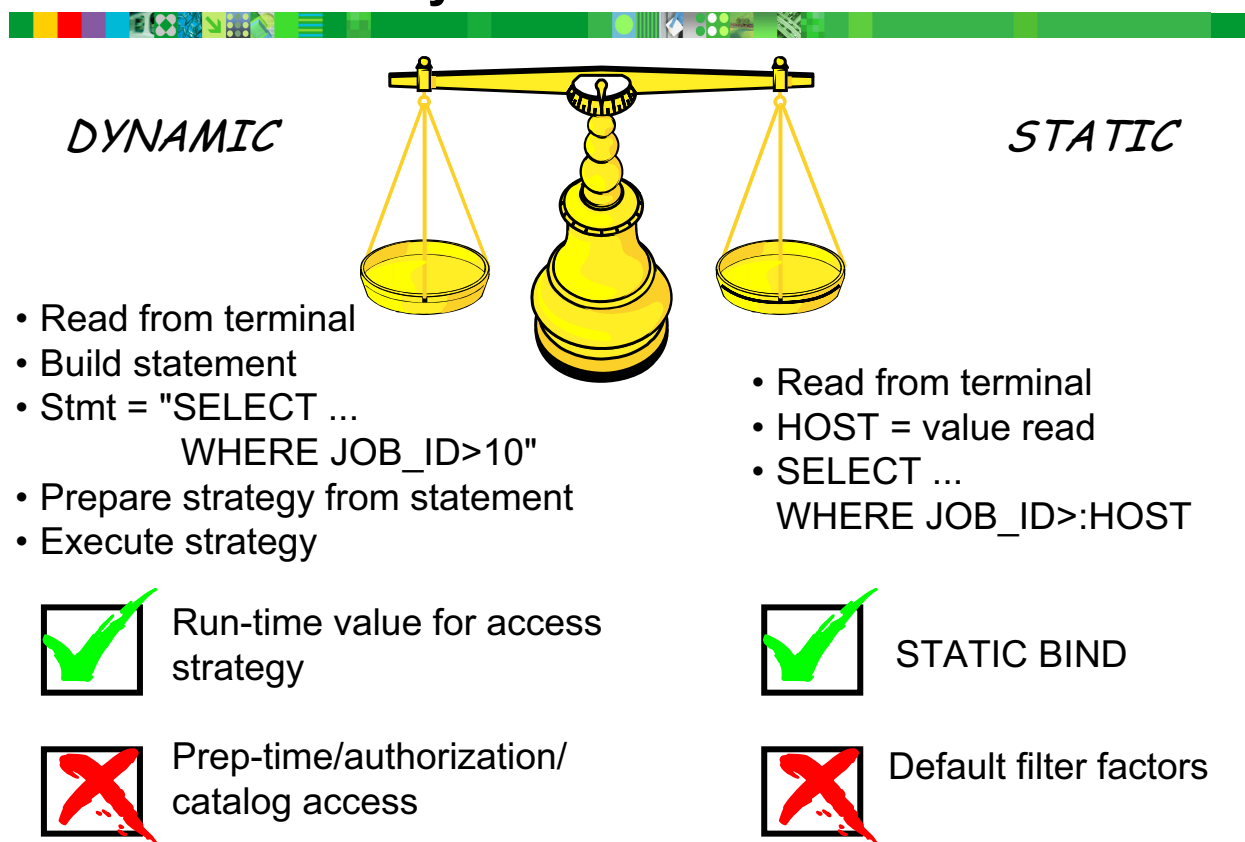
**Details** — This course is not intended to make the student an expert on any of the options listed, but will provide the DBA with insight concerning the options so that advice can be provided when determining application architecture. The DB2 Run-Time Client Lite makes it easier to build and redistribute applications. See student notes. DB2 RTCL provides application interfaces (CLI, ODBC, OLE DB, .NET Data Provider, and JDBC) and the network libraries that are required for applications to run. The main distinguishing features of the DB2 RTCL are:

- A significantly smaller disk footprint (potentially as low as 12 MB)
- Shipped as a single executable, making it easy to redistribute and deploy
- A Windows Installer Merge Module (.msm file) is available which simplifies the integration of the DB2 RTCL code within a larger application

For further information on the DB2 Run-Time Client Lite, refer to *Quick Beginnings for DB2 Clients*.

**Transition Statement** — First, let's look at a comparison between dynamic and static SQL.

## Performance — dynamic versus static SQL



© Copyright IBM Corporation 2007

Figure 10-4. Performance — dynamic versus static SQL

CF238.3

### Notes:

- Static SQL statements are statements that are fully defined at program creation time.
- SQL statements are placed in an application and are not specific to the host language.
- Variables can be used to receive data from the database manager or to provide data to the database manager. For example, assume that the fields in the sample screen are filled out by a travel agent. The application could use the Charge Card number provided to check the credit of the customer by specifying a variable on the WHERE clause of a SELECT statement. The returned credit limit could be stored in another program variable. If the credit check is passed, the application could then use the remaining information on the input screen to INSERT a row in a booking table.
- Information about the variables is present when the SQL is prepared, but the actual value of the variable is specified at run time.
- There are two major advantages of static SQL. First, preparation of the SQL is completed before execution and is not repeated every time the application runs. Second, the authorization of the package binder is used for the underlying SQL

statements, enabling security to be based on execute authority for the program, and not on authorization to underlying objects.

- A static statement that references host variables is optimized without knowledge of the actual value of the host variable. Default filter factors are used. At times, the default filter may be too optimistic. Range predicates are sensitive to this problem. For example, consider an application with the predicate `LASTNAME BETWEEN :host1 AND :host2`. Assume that an index on `LASTNAME` is chosen to support this predicate, based on default filtering. If `:host1` is set to *Abell* and `:host2` is set to *Brown* during execution, using the index would be appropriate. However, if `:host1` is set to *Abell* and `:host2` is set to *Williams* during execution, using the index would be less efficient than a table scan.
- Dynamic SQL is particularly useful in certain situations.
- Dynamic SQL is bound and optimized during execution, and all values are considered during the selection of access strategy. More accurate filter factors can be used. An access strategy that is specific to the current execution of the statement can be selected.
- Dynamic SQL, however, implies that the access strategy must be developed during execution. In many cases, even with the use of default filter factors, the total execution time of the static statements will be faster than the total execution time of the dynamic statements.



***Instructor Notes:***

**Purpose** — Identify the performance implications of static and dynamic solutions.

**Details** — Although the example illustrates *PREPARE* from the embedded dynamic perspective, the performance implications apply to other solutions that result in dynamic statements: CLI.

**Additional Information** — A key for the students is that one technique is likely to be appropriate for one situation, while the other is proper for a slightly different situation. Also, emphasize the authorization consideration.

**Transition Statement** — Now that we have examined the performance aspects of static and dynamic SQL, let's examine APIs.

## Database Manager APIs

- 
- Application Programming Interfaces (APIs)
    - Command functions that can be included in applications
    - Enable programmatic control of various database operations
    - Used for management and maintenance of the database
    - NOT an alternative for SQL operations
  - Examples
    - Start/Stop Database Manager
    - Create Database
    - Backup Database
    - Export/Import
    - Run Statistics
    - Bind

© Copyright IBM Corporation 2007

Figure 10-5. Database Manager APIs

CF238.3

### Notes:

- Database manager APIs provide access to database manager functions from compiled and interpreted programming languages. These APIs are subdivided as follows:
  - Database Manager Control - Quiescing databases and instances, starting and stopping instances
  - Database Control - Creating and dropping, activating and restarting databases
  - Database Manager and Database Configuration - Getting and setting parameters
  - Database Directory Management - Cataloging and uncataloging database directory
  - Client/Server Directory Management - Cataloging and uncataloging node directory
  - Network Support - LDAP support
  - Recovery - Backup, restore, and roll forward database, manipulate history file
  - Operational Utilities - Force application, reorganization, runstats
  - Database Monitoring - Snapshot monitoring
  - Health Monitoring - Contact, notification list, contact group, alert configuration
  - Data Utilities - Export, import, load, load query
  - General Application Programming - Autoconfigure, get error or SQLSTATE messages, signal handler

- Application Preparation - Precompile, bind, rebind programs
- Remote Server Utilities - Attach, change password, detach
- Table Space Management - Table space and container query
- Partition Management - Add and drop partitions
- Satellite - Query and update satellite synchronization
- Database Partition Group Management - Redistribute database partition group
- Additional APIs - Get authorizations, instance, query and set client, read log, row and table partitioning information, archive log, inspect database

### Sample program to create database with user-defined collating sequence:

```

/*****creddb.sqc*****/
/*
/* Sample C program for "DB2 WORKSTATION INTERMEDIATE PROGRAMMING"
/*
/*
/*
/*****
/* This program will create a database.
/*****

/*****
/* Include C library definitions
/*****
#include <memory.h>      /* for memcpy */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sql.h>
#include <sqlutil.h>
#include <sqlenv.h>

/*****????????????????????????????????????????????????????????????*****/
/* Look up what the following include file contains.
/*****
#include <sqle850b.h>

struct sqlca sqlca;

struct sqldbdesc dbdesc;
struct sqldbcountryinfo dbcinfo;

void sql_err();

/*****
/* Main routine
/*****
int main()
{
int rc;
char database[9];

    setbuf(stdout,NULL); /* disable output buffering */

```

10-18 DB2 9 DB Admin Workshop for Windows © Copyright IBM Corp. 1999, 2007

Course materials may not be reproduced in whole or in part  
without the prior written permission of IBM.

```
    return;  
} /* end of sql_err */
```

## ***Instructor Notes:***

**Purpose** — Introduce concepts of the ways in which applications can interface to the Run-Time Client.

**Details** — This course is not intended to make the student an expert on any of the options listed, but will provide the DBA with insight concerning the options so that advice can be provided when determining application architecture.

Note that this graphic is building on the one from a few pages ago; now discussing the programming interfaces.

**Transition Statement** — First, let's look at an application using embedded SQL.

## 10.3 Embedded SQL program preparation

### Instructor Topic Introduction

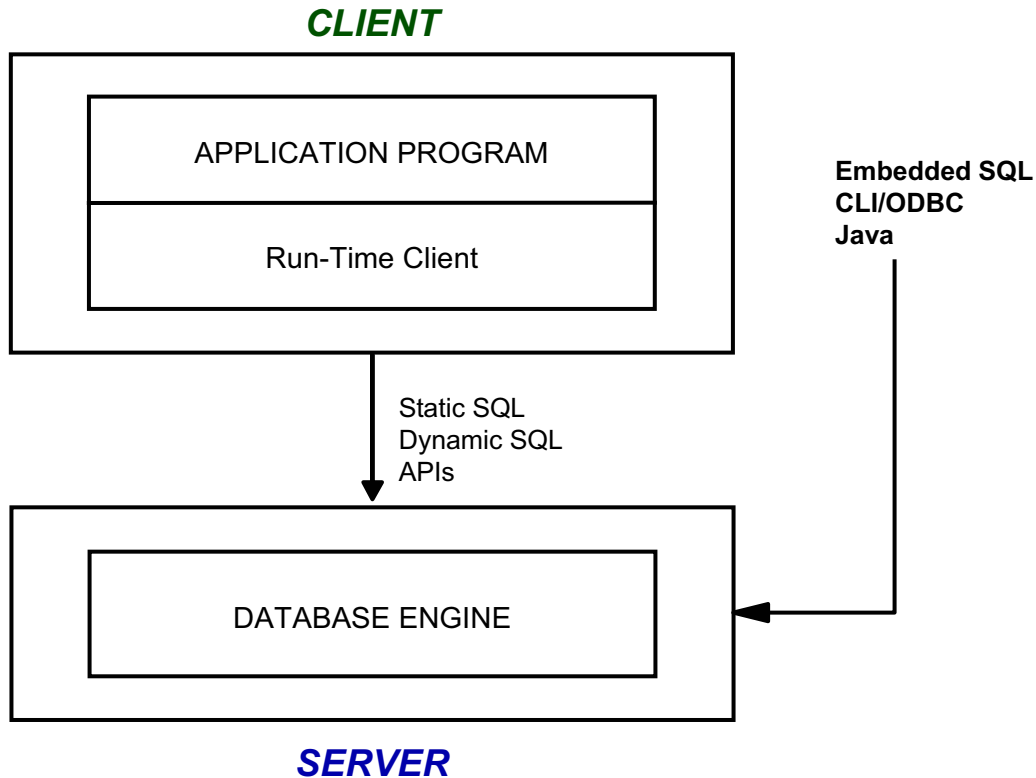
**What students will do** — Checkpoint exercises to determine the correct application alternative to use.

**How students will do it** — Listen to lecture.

**What students will learn** — What alternatives there are in coding applications.

**How this will help students on their job** — A database administrator is frequently called upon to advise application programmers concerning solutions to various business problems. Although the DBA may not need to implement a given solution, the DBA should be aware of the strengths and weaknesses of each alternative.

# Programming interfaces



© Copyright IBM Corporation 2007

Figure 10-6. Programming interfaces

CF238.3

## Notes:

Application programs can issue SQL via the Run-Time Client component using one of several possible programming interfaces.

- Embedded SQL
- CLI
  - ODBC
- Java
  - JDBC
  - SQLJ



***Instructor Notes:***

**Purpose** — Introduce concepts of the ways in which applications can interface to the Run-Time Client.

**Details** — This course is not intended to make the student an expert on any of the options listed, but will provide the DBA with insight concerning the options so that advice can be provided when determining application architecture.

Note that this graphic is building on the one from a few pages ago; now discussing the programming interfaces.

**Transition Statement** — First, let's look at an application using embedded SQL.

## Embedded SQL applications

- Embedded SQL Applications
  - SQL statements embedded in the programming language
  - Can embed both Static and Dynamic SQL
  - Application must be precompiled
  - Need to bind the SQL and create a package at the database

© Copyright IBM Corporation 2007

Figure 10-7. Embedded SQL applications

CF238.3

### Notes:

Structured Query Language (SQL) is the database interface language used to access and manipulate data in DB2 databases. You can embed SQL statements in your applications, enabling them to perform any task supported by SQL, such as retrieving or storing data. Using DB2, you can code your embedded SQL applications in the C/C++, COBOL, FORTRAN, Java (SQLJ), and REXX programming languages.



#### Note

The REXX and Fortran programming languages have not been enhanced since Version 5 of DB2.

An application in which you embed SQL statements is called a host program. The programming language you use to create a host program is called a host language. The program and language are defined this way because they host or accommodate SQL statements. For static SQL statements, you know before compile time the SQL statement

type and the table and column names. The only unknowns are specific data values the statement is searching for or updating. You can represent those values in host language variables. You precompile, bind, and then compile static SQL statements before you run your application. Static SQL is best run on databases whose statistics do not change a great deal. Otherwise, the statements will soon get out of date.

In contrast, dynamic SQL statements are those that your application builds and executes at run time. An interactive application that prompts the user for key parts of an SQL statement, such as the names of the tables and columns to be searched, is a good example of dynamic SQL. The application builds the SQL statement while it is running, and then submits the statement for processing.

You can write applications that have static SQL statements, dynamic SQL statements, or a mix of both.

Generally, static SQL statements are well-suited for high-performance applications with predefined transactions. A reservation system is a good example of such an application.

Generally, dynamic SQL statements are well-suited for applications that run against a rapidly changing database where transactions need to be specified at run time. An interactive query interface is a good example of such an application.

## ***Instructor Notes:***

**Purpose** — Describe embedded SQL applications.

**Details** — If you have students from other relational database backgrounds (other than DB2 on OS/390), they will probably never have heard of embedded SQL. In other databases, the only way to have a stored strategy is through the use of stored procedures. Therefore, a lot of DBAs who are coming to DB2 from a competitive database product will be looking to convert a lot of stored procedures. Otherwise, if they are implementing new databases, they may be planning to code most of their business logic in stored procedures. This is not a bad idea, but is not required in DB2 to get a predetermined, stored strategy.

**Additional Information** — Student notes copied from “Programming Client Applications”.

**Transition Statement** — Let's look at using embedded SQL.

## Write the program

### ■ myapp.sqc

```
EXEC SQL DECLARE c1 CURSOR FOR
  SELECT empno,lastname FROM xyzco.templ
  WHERE wrkdept = :dept;
strcpy(dept, 'D21');
EXEC SQL CONNECT TO musicdb;
EXEC SQL OPEN c1;
EXEC SQL FETCH c1 INTO :hempno,:hlastname;
while (sqlca.sqlcode==0){

  EXEC SQL FETCH c1 INTO :hempno,:hlastname;
} /*end while */

EXEC SQL CLOSE c1;
EXEC SQL CONNECT RESET;
```

© Copyright IBM Corporation 2007

Figure 10-8. Write the program

CF238.3

### Notes:

SQL statements placed in an application are not specific to the host language. The database manager provides a way to convert the SQL syntax for processing by the host language:

- For the C, C++, COBOL, or FORTRAN languages, this conversion is handled by the DB2 precompiler. The DB2 precompiler is invoked using the PREP command. The precompiler converts embedded SQL statements directly into DB2 run-time services API calls.
- For the Java language, the SQLJ translator converts SQLJ clauses into JDBC statements. The SQLJ translator is invoked with the sqlj command.

When the precompiler processes a source file, it specifically looks for SQL statements and avoids the non-SQL host language. It can find SQL statements because they are surrounded by special delimiters.

The extension of the source module identifies the source language in the embedded SQL program. Supported extensions include:

- **.sqc** for C files on all supported platforms
- **.sqC** for C++ files on UNIX platforms
- **.sqx** for C++ files on Windows operating systems
- **.sqb** for COBOL files on all supported platforms, though the TARGET precompile option allows for any file extension
- **.sqf** for FORTRAN files on all supported platforms, though the TARGET precompile option allows for any file extension
- **.sqlj** for SQLJ files on all supported platforms

The program on the graphic is provided for illustrative purposes. A brief description of its function:

- First, the program DECLARES the cursor, stating what SELECT statement it will be used with, and whether updates will be performed using it. Then, the program OPENS the cursor. This allows the application to gain access to the data that meets or will meet the definition of the cursor. (If host variables were in the definition of the cursor, the value of the host variables at OPEN would be used in the statement.)
- The program uses FETCH statements to move the cursor forward and to look at each row in the result set. When the program has finished processing the results of the query, it CLOSES the cursor, which destroys the result set.

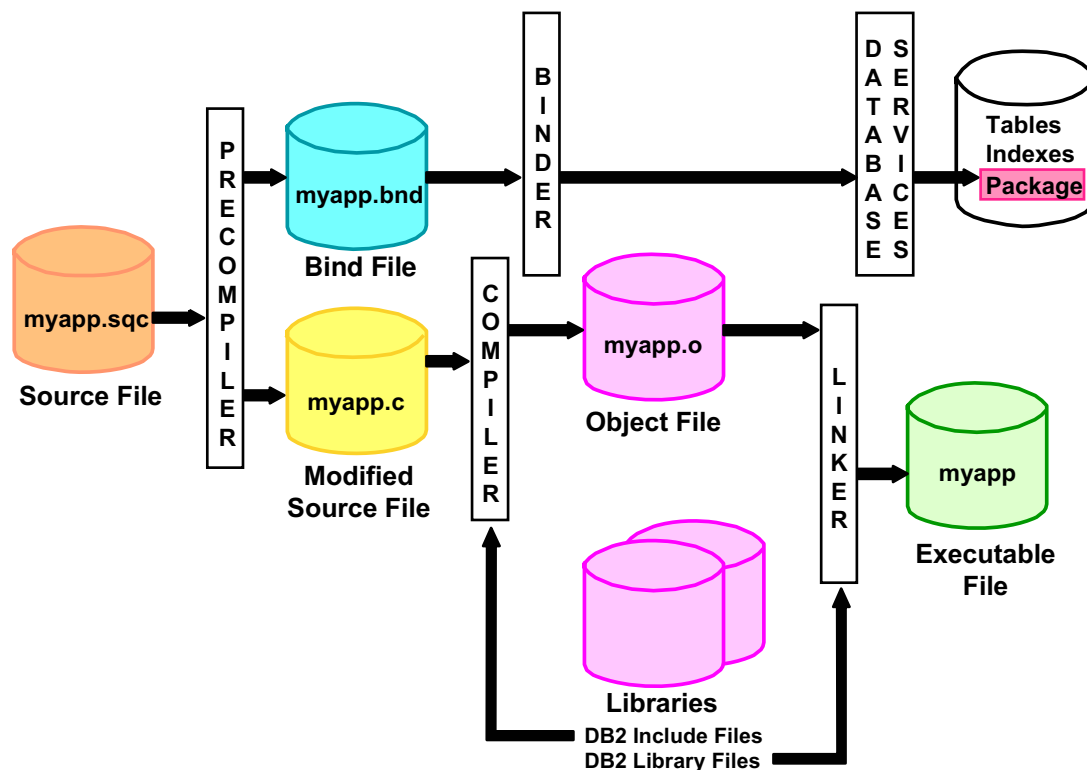
***Instructor Notes:***

**Purpose** — Provide a simple program to be used as an example in the unit.

**Details** — Introduce basic programming concepts regarding the cursor to support the lab.

**Transition Statement** — Once the program contains SQL, it must be prepared using interfaces particular to DB2, as well as more *standard* interfaces such as compile and link.

## Program preparation steps



© Copyright IBM Corporation 2007

Figure 10-9. Program preparation steps

CF238.3

### Notes:

When you embed SQL statements in your application, you must precompile and bind your application to a database with the following steps:

1. Create source files that contain programs with embedded SQL statements.
2. Connect to a database, then precompile each source file.

The precompiler converts the SQL statements in each source file into DB2 run-time API calls to the database manager. The precompiler also produces an access package in the database and, optionally, a bind file, if you specify that you want one created. The access package contains access plans selected by the DB2 optimizer for the static SQL statements in your application. The access plans contain the information required by the database manager to execute the static SQL statements in the most efficient manner as determined by the optimizer. For dynamic SQL statements, the optimizer creates access plans when you run your application.

The bind file contains the SQL statements and other data required to create an access package. You can use the bind file to rebind your application later without having to precompile it first. The rebinding creates access plans that are optimized for current database conditions.



You need to rebind your application if it will access a different database from the one against which it was precompiled. You should rebind your application if the database statistics have changed since the last binding.

3. Compile the modified source files (and other files without SQL statements) using the host language compiler.
4. Link the object files with the DB2 and host language libraries to produce an executable program.
5. Bind the bind file to create the access package if this was not already done at precompile time, or if a different database is going to be accessed.
6. Run the application. The application accesses the database using the access plan in the package.

The above visual represents the program preparation process. The following notes summarize the activities that occur during each step.

### 1. PRECOMPILE

- The correct precompiler is determined based on the extension of the source code.
- SQL statements are syntactically checked. Then, the SQL statements are commented out in the source code and replaced with host language calls. The SQL in the source file is found by the precompiler through the use of language specific delimiters. The resultant output is called the **modified source file**.
- Either a **bind file** or a **package** is also created. (It is also possible to create both.) The default is to create a package. However, there are several reasons to override the default and create a bind file. For discussion purposes, assume that the example shown uses a precompiler option that creates a bind file.

### 2. COMPILE

- The modified source file can then be processed using the compiler appropriate for the chosen language. There is no SQL in the modified file so that it can be processed.
- Compile will resolve INCLUDEs commonly used by application programmers. In addition to those libraries *normally* used by a particular installation, a DB2 library containing INCLUDEs relevant to the DB2 programming interface needs to be identified. This identification can be done through compiler command options if symbolic links (AIX), updates to system environment variables (Windows NT), or some other site-specific technique has not been used to place the location of the INCLUDE files into the path used by default to find INCLUDE files. Consult the product documentation for further details.

### 3. LINK

- Assuming that there are no errors discovered during compile, the resultant object module can be linked.
- The linker resolves references to run-time services DB2 APIs that were placed in the modified source file by the precompiler, in addition to all other module references needed by the *non-SQL* code. The library needed for the DB2 calls can be explicitly included as a command option on LINK if it has not been identified through techniques similar to those listed for the compile step.

#### 4. BIND

- The *SQL component* of the application that is present in the bind file is analyzed.
- Syntax and authorization are checked against the target database.
- Assuming that there are no syntax or authorization problems, an access strategy is determined for each SQL statement. This strategy is stored in a DB2 object called a **package**. More details concerning the generation of access strategies, called **optimization**, is presented in the Application Performance unit.
- DYNAMICRULES - There is a requirement to be able to lend the DML privileges of a package binder to the package executor during the execution of dynamic SQL. Some businesses have, or are heading towards having, applications with large amounts of dynamic SQL, coupled with the need to allow a large and potentially unknown user base access to the information the SQL is allowed to access. Web-initiated query tools are a good example. A user is given the opportunity to receive information based on a query generated from criteria they have supplied. SQL gets dynamically generated and the information is retrieved. When the application is bound with DYNAMICRULES BIND, access to the tables that contain the query information only have to be given to the package binder and not to every user who might need access to the information; the authorization ID of the package owner is used for authorization checking, not that of the user. An application bound with DYNAMICRULES RUN, the default, requires that the user has access to the tables that contain the query information. References to unqualified objects will be the CURRENT SCHEMA of the user.

Reasons to separate the BIND step from the PRECOMPILE step include:

1. The compile and link steps can be successfully completed before using database resources. Many programming errors can be detected and corrected before using database manager resources.
2. The package must be bound against more than one database.
3. Authorization of the ID used to do the BIND is different from that of the ID used to do the precompile.

During execution, the executable will pass the full package name, statement section number, and consistency token to the database manager. This allows DB2 to locate and execute the correct access strategy for the executable. This is also the architecture that supports executables that contain more than one object module, since each executable SQL statement results in an independent call to DB2 with specific information concerning the package and statement.

In order to execute a section of a package, the section must be loaded into memory. The size of the memory area allocated at the database level to support package sections is designated via the configuration parameter **pckcachesz**. If sections of SQL that may be executed on a repetitive basis can be kept in the cache, a performance benefit is realized because the database manager does not need to reload the section. This could be of significant benefit to transactional systems.

## ***Instructor Notes:***

**Purpose** — Describe the overall steps of program preparation.

**Details** — The depth of the presentation is key. The instructor must cover the steps shown on this visual to significant depth so that the DBA understands the process. However, some details are best left for later parts of the unit or for the programming classes. For example, it is not the goal of this unit to describe all the techniques used to identify the INCLUDE files and program libraries. However, it is a goal of this unit to identify the need to be concerned with these components and to emphasize that solutions for compile and link vary by environment and are documented.

It is appropriate to *recommend* the process of separating the BIND and PRECOMPILE steps. Ask: “How many of you write perfect C code, first time, every time?” Indicate that separation of the PRECOMPILE and BIND steps will give an option to create the package only after we know that the C code is good. However, you may wish to do some board work to illustrate the precompiler *default* to bypass creating a permanent bind file and create a package.

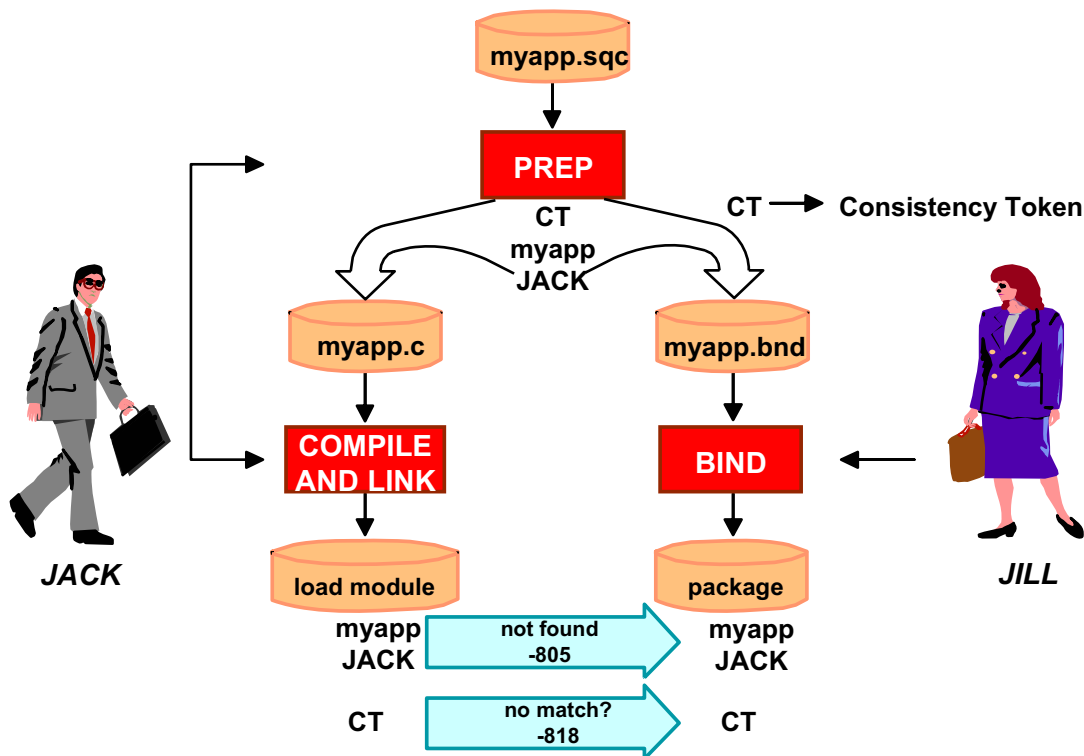
Emphasize that the package and the modified source file contain timestamps, and that the resultant executable must be from the same precompile step that produced the package.

### **Additional Information**

- More details on the concept of consistency tokens (timestamps) are presented later in the unit.
- Syntax and major options for PRECOMPILE and BIND are highlighted later in the topic.
- The package cache parameter is frequently acceptable when it is defaulted. However, installations that support large applications that repeat SQL sections within applications may benefit from increasing the memory allocated to the cache.

**Transition Statement** — The authorization IDs of the connection that does the BIND and PRECOMPILE have impacts concerning the code and the name of the package.

## Associating load modules and packages



© Copyright IBM Corporation 2007

Figure 10-10. Associating load modules and packages

CF238.3

### Notes:

During PRECOMPILE, a consistency token is placed in both the modified source and the bind file. This token is necessary to ensure that the resultant load module and package were derived from the same original source. (If a bind file is not created, the token is placed directly in the resultant package.)

Also at PRECOMPILE time, the name of the package that the executable load module will request is stored in the modified source module. By default, this will be the same as the name of the source module (the name excluding `.sqc`), truncated to eight characters.

A consistency token contains a timestamp of when the precompile was done. It is often referred to as a *timestamp*, but we will use the term *consistency token* both to avoid ambiguity and for consistency with other platforms.

When a call to the database manager is made by an application, the package creator, package name, and consistency token are passed to the database manager.

If a package that matches the creator and name cannot be found, the application receives a -805 SQLCODE.

If a package is found, the consistency token passed by the load module is compared to the token in the package. If the values match, this indicates that the load module and package were created from the same original source and execution proceeds. If the tokens do not match, the application receives a -818 SQLCODE which indicates that the load module and package were created from sources created by different precompiles.

Any changes to original source code require both SQL and non-SQL program preparation steps to be completed.

Why might you want to separate the precompile and bind activities and have them done by different individuals?

Answer: If you request a bind file at precompile time, certain object existence and authentication SQLCODES are treated as warnings instead of errors. This enables you to precompile a program and create a bind file without requiring that the referenced objects be present, or having to be authenticated. So, Jack would not need to have the authority to access the objects referenced in the SQL, as long as Jill did when she did the bind.

## ***Instructor Notes:***

**Purpose** — Identify what kind of errors may be experienced at execution time.

**Details** — Note that if the package cannot be found, specifically with JACK as the creator and the specific package name, a -805 will be returned. If the package is found, but the timestamp (consistency token) does not match between the executable and the package, a -818 will be returned.

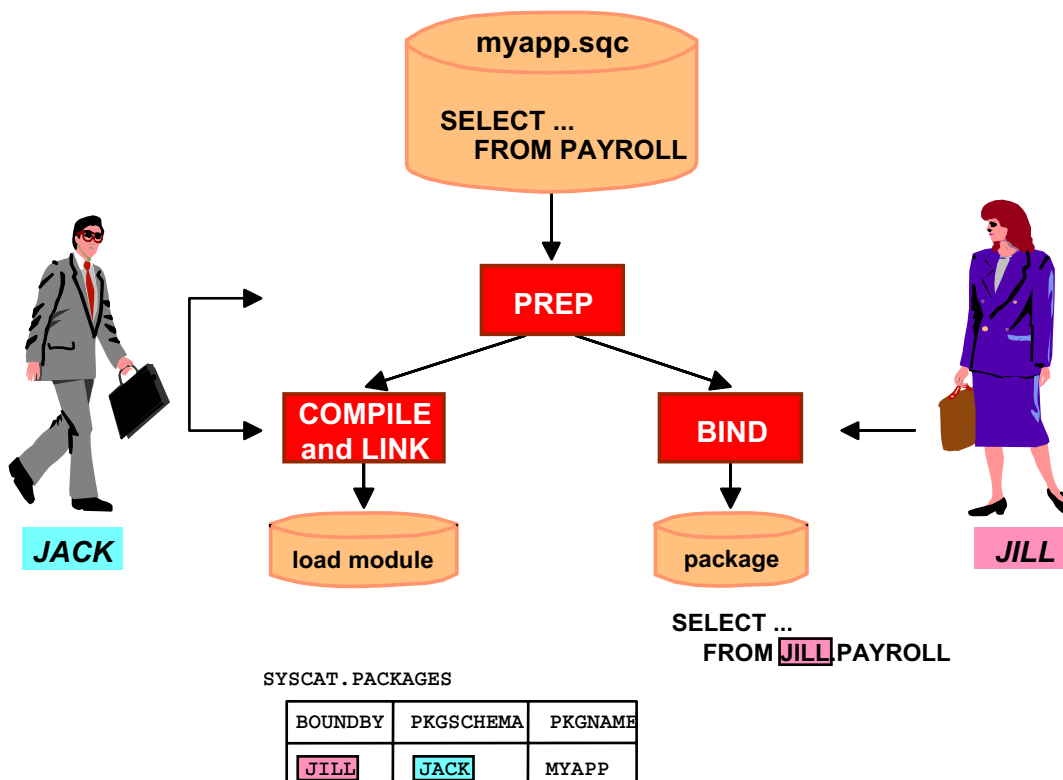
The consistency token is stored in the outputs from the precompile step. This is also when the creator ID will be set, and the name of the package. This is the last time that the two ultimate outputs (the executable and the package) have a common path. If any errors are found in the SQL, C code, or logic, the changes must be made to the .sqc file and the changes brought forward to the executable and the package. The executable and the package must be eventual outputs from the same precompile step.

**Additional Information** — Jill will be noted as the binder; see next page.

Note the question at the end of the student notes on why you might want to do this. More on this in the authorization area later in this topic.

**Transition Statement** — What will be the impact of Jill having done the bind?

## Precompiling versus binding



© Copyright IBM Corporation 2007

Figure 10-11. Precompiling versus binding

CF238.3

### Notes:

Unqualified references in applications are qualified with the ID of the owner in the package definition.

The value in the BOUNDBY column in the SYSCAT.PACKAGES table specifies the “owner” of the package being bound. That will be the binder’s ID only if an “owner” parameter is not specified on the bind.

It is possible that JACK has access to a test table named JACK.PAYROLL. When JACK precompiles the code, syntax and authorization will be done against his test table. When JILL does the production BIND, the target table will be JILL.PAYROLL.

If the application contained fully-qualified references, for example:

```
SELECT ....
FROM PROD3.PAYROLL
```

then the binder would need authorization against the PROD3.PAYROLL object, but the ID of the binder would not be used in qualification, since the reference is already fully defined.

## ***Instructor Notes:***

**Purpose** — Identify the implications of Jill doing the bind.

**Details** — Any unqualified references will be qualified with the binder's ID. Also, the catalog table will indicate Jill in the BOUNDBY column as owner of the package.

PKGSHEMA will reflect the ID that *created* the package.

**Transition Statement** — In addition to qualification of simple references, the DBA should be concerned with several options that can be specified on the PRECOMPILE command.



## Adjusting qualifier and owner

Jill issues:

```
db2 bind myapp.bnd qualifier u1 owner u2  
dynamicrules bind
```

- Unqualified SQL uses u1 schema
- u2 owns package
  - Can drop, rebind, grant privileges
  - u2's privilege checked for dynamic SQL

© Copyright IBM Corporation 2007

Figure 10-12. Adjusting qualifier and owner

CF238.3

### Notes:

QUALIFIER provides an 8-character implicit qualifier for unqualified objects contained in the package. The default is the owner's authorization ID, whether or not OWNER is explicitly stated.

OWNER designates an authorization identifier for the package owner. The owner must have the privileges required to execute the SQL statements in the package. The default is the primary authorization ID for the precompile/bind process if this option has not been explicitly specified.

The DYNAMICRULES option and its values can be supplied at BIND time, and define which rules apply to dynamic SQL at run time for the authorization id, and for the initial setting of the value used for implicit qualification of unqualified object references. Valid DYNAMICRULES options are RUN and BIND; RUN is the default.

## ***Instructor Notes:***

**Purpose** — Identify options on the bind for specifying the qualifier for SQL and the owner ID.

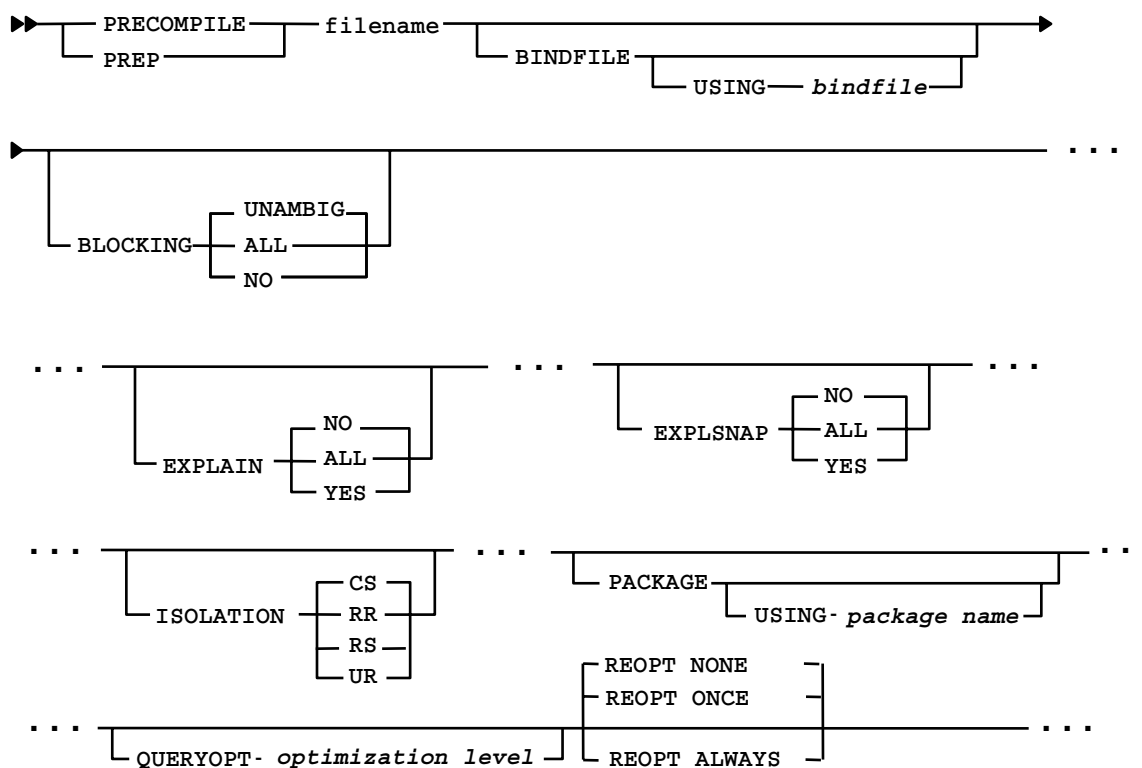
**Details** — In this case, any unqualified SQL will be qualified with the U2 schema. This is caused by the QUALIFIER keyword.

U2 owns the package, specified by the OWNER keyword. This means that U2 can drop, rebind, and grant privileges on the package, including the ability to grant privileges with grant privilege.

Finally, DYNAMICRULES BIND indicates that the OWNER's privileges should be checked when dynamic SQL is run through this program.

**Transition Statement** — In addition to qualification of simple references, the DBA should be concerned with several options that can be specified on the PRECOMPILE command.

## Precompile command syntax (basic)



© Copyright IBM Corporation 2007

Figure 10-13. Precompile command syntax (basic)

CF238.3

### Notes:

PRECOMPILE or PREP can be used interchangeably.

A database connection must exist before this command is executed. The syntax allows many options to be specified, but not all would be required in every case. As a simple example, consider the following.

```
db2 connect to musicdb
```

```
db2 prep myapp.sqc bindfile
```

```
db2 connect reset
```

This would run the precompiler and create the bind file. It would not create a package.

A package will be created if you:

1. Use both the BINDFILE option and the PACKAGE option
2. Use the BINDFILE option and then the DB2 bind command
3. Omit the BINDFILE option

The capability to explicitly name the bind file, package, or both is provided through the USING option. However, it is not recommended, since the default names can easily be related to the original source. For example, assume that you are preparing myapp.sqc; the default bind file would be myapp.bnd and the default package would be myapp.

BLOCKING allows the data transfer from server to client for READ-ONLY cursors to be in multiple row blocks instead of on a row-by-row basis. Greater detail regarding the meaning of the above settings for BLOCKING will be provided later in this topic.

EXPLAIN stores information in the EXPLAIN tables about the access plans chosen for each SQL statement in the package, if the YES option is chosen on the EXPLAIN parameter. The ALL value will collect information on each eligible static SQL statement. Information will be gathered for eligible dynamic SQL statements at run time, even if the CURRENT EXPLAIN SNAPSHOT register is set to NO. DRDA does not support the ALL value for this option. The NO option on the EXPLAIN parameter means that EXPLAIN information will not be collected. With the REOPT option, EXPLAIN information for each reoptimizable incremental bind SQL statement will be placed in the EXPLAIN tables at run time. In addition, Explain Snapshot information will be gathered for reoptimizable dynamic SQL statements at run time.

EXPLSNAP determines if an EXPLAIN snapshot should be stored in the Explain Tables. ALL specifies that an EXPLAIN snapshot will be created for static and dynamic SQL statements when the statement is optimized. YES specifies that only static statements should be EXPLAINed. NO disables creating an EXPLAIN snapshot. With the REOPT option, EXPLAIN information for each reoptimizable incremental bind SQL statement will be placed in the EXPLAIN tables at run time. In addition, EXPLAIN snapshot information will be gathered for reoptimizable dynamic SQL statements at run time.



### Note

EXPLAIN captures data for use with the db2exfmt tool, while EXPLSNAP captures data for use with the Visual Explain tool.

ISOLATION determines the degree to which this application will be isolated from the changes made by other applications. This option impacts the acquisition and duration of row share locks. The options are abbreviations for Cursor Stability (CS), Repeatable Read (RR), Read Stability (RS), and Uncommitted Read (UR).

QUERYOPT determines the level of optimization that should be performed. The higher the value (up to a maximum of 9), the greater the number of access paths considered.

REOPT specifies whether to have DB2 optimize an access path using values for host variables, parameter markers, and special registers. Valid values are:

- **NONE:** The access path for a given SQL statement will not be optimized using real values for the variables. The default estimates for these variables will be used instead, and this plan is cached and used subsequently. This is the default.

- **ONCE:** The access path for a given SQL statement will be optimized using the real values of the host variables when the query is first executed. This plan is cached and used subsequently.
- **ALWAYS:** The access path for a given SQL statement will always be compiled and reoptimized using the values of the host variables known at each execution time.

The PRECOMPILE command in a DRDA environment has options specific to that environment. Also, other options for the PRECOMPILE command exist. Consult the product documentation if desired.

## References

*Command Reference*

## ***Instructor Notes:***

**Purpose** — Show the general syntax of the PRECOMPILE (PREP) command and highlight those options that are of special interest to the DBA.

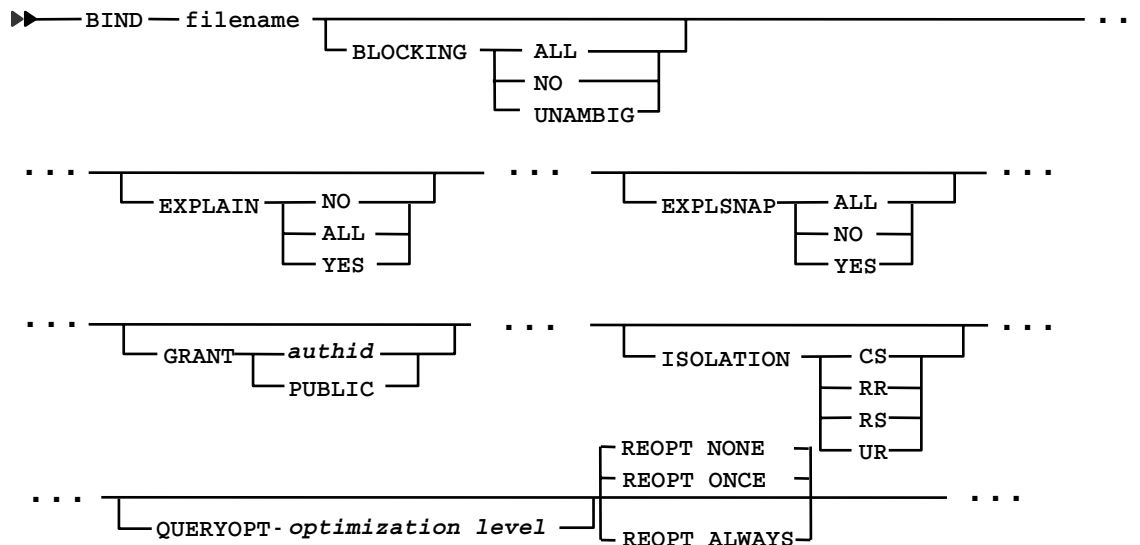
**Details** — Much of the information on this visual relates to topics previously presented or to topics to be presented at a later point in the database administration course. Therefore, the discussion at this point should be *high-level*. However, you may have to provide greater details if this topic is being delivered in a customized class without the other units that are related.

Since isolation is covered in detail in the concurrency topic, it is not addressed in any detail in this topic. If you are teaching this topic without the Concurrency unit, you may wish to embellish this area with board work. Otherwise, assure the students that further details on isolation are forthcoming, and mention that the option is included on the visual simply to indicate where the specification is done.

The details of CONNECT and SYNCPOINT are covered in the RUW and DUW topic.

**Transition Statement** — If you choose to use the bindfile option, you will need to bind the package at some later point using the BIND command.

## Bind command syntax (basic)



© Copyright IBM Corporation 2007

Figure 10-14. Bind command syntax (basic)

CF238.3

### Notes:

A database connection must exist before this command is executed. The syntax allows many options to be specified, but not all would be required in every case. As a simple example, consider the following.

```
db2 connect to musicdb
db2 bind myapp.bnd blocking all
db2 connect reset
```

This would BIND the statements in the specified bind file and use the option to block all cursors that are not explicitly updatable.

The filename specified on this command must have an extension of bnd.

It is possible to bind multiple packages by specifying a filename that is actually a list of bind files. The syntax for specifying a list is to preface the name of the file with the **at sign** (@). The bind files in the list file are designated according to the following syntax:

```
first.bnd+second.bnd+  
third.bnd+ (etc)  n.bnd+  
last.bnd
```

The first position in any line cannot contain the + sign, and the last line should not contain the + sign.

Options specified on the BIND command will take precedence over any options specified or defaulted on the PRECOMPILE command. Options not specified will use the value specified or defaulted at precompile.

The GRANT option of the BIND command is one that is not available on the PRECOMPILE command. If it is specified, BIND and EXECUTE authority are granted to PUBLIC or the authid named.

The BIND command in a DRDA environment has options specific to that environment. Also, other options for the BIND command exist. Consult the product documentation if desired.

With the BIND command, the several REOPT options can certainly also be used.

There is another option available with bind: **ACTION**.

ACTION indicates whether the package can be added or replaced.

- ADD indicates that the package does not exist and that a new package is to be created. If the package exists, the execution will be stopped and a error message is returned.
- REPLACE indicates that the existing package is to be replaced by a new one with the same package name and creator. This is the default for the ACTION option.
  - REPLVER *version-id* could be specified as an additional option for REPLACE. If this option is not specified and a package already exists that matches the name, creator, and version of the package being bound, then that package will be replaced; if not, then a new package will be added.

## References

*Command Reference*



***Instructor Notes:***

**Purpose** — Introduce the syntax of the BIND command.

**Details** — Stress that options specified at BIND override those specified at PRECOMPILE.

For your information, it might be confusing to you if you notice that EXPLAIN and EXPLSNAP have a different ordering for the option choices. The default, as stated above, is determined by the choice made on the PREP command, so there is no implication of a default by the order of the options on this command.

**Transition Statement** — Catalog views for packages.

## Package catalog views

- SYSCAT.PACKAGES
  - One row for each package on the system
  - Information about parameters, schema, binder, and so forth
- SYSCAT.PACKAGEAUTH
  - One row for every privilege held on packages
  - Information about grantor, grantee, privilege set
- SYSCAT.PACKAGEDEP
  - One row for each dependency of the package
  - Information about indexes, tables, views, triggers, functions, and aliases needed by the package

© Copyright IBM Corporation 2007

Figure 10-15. Package catalog views

CF238.3

### Notes:

The catalog views described above can be useful when managing the applications for an installation.

The following items are some examples of how the catalog views could be used.

- PACKAGES
  - You need to determine why a READ-ONLY application is running slowly. You suspect that the blocking option was improperly chosen. The column BLOCKING provides you with the information.
  - A package that accesses multiple servers is periodically returning -818 SQLCODES. You know that a change to the code was done on Friday night. The column EXPLICIT\_BIND\_TIME provides you with the time that the last explicit bind was done, so you could determine the servers that have an older version of the package (older than Friday's date.)

- PACKAGEAUTH

- You are asked by an auditor to list all IDs that can execute a particular sensitive application. The columns GRANTEE and GRANTEETYPE could be queried for the package for rows where the EXECUTEAUTH column contains a Y.
- You are about to go on vacation, and your backup needs the authority to BIND your applications while you are away. The BINDAUTH column indicates whether or not such an authority is possessed.

- PACKAGEDEP

- You are considering dropping an index that you believe to be rarely used. The columns PKGSCHEMA and PKGNAME could be queried to list those packages that are currently using the index in an access strategy.
- You wish to list all the tables and views that an application accesses. The columns BSCHEMA and BNAME could be queried for the given package for the BTYPE values of V (View) and T (table).

All the columns defined by the views are described in the *SQL Reference*.

## References

*SQL Reference*

## ***Instructor Notes:***

**Purpose** — Describe the three catalog views related to packages that a DBA might want to access.

**Details** — Although the underlying information defined by these views are the DB2 catalog tables, you should use the views as a standard. This is because DB2 has promised to keep the views unchanged, but the SYSIBM tables could change. Also, the examples in the student notes are just a few valid reasons to access the catalog views concerning packages.

**Additional Information** — The *SQL Reference* defines the columns and meanings for the catalog views.

Sometimes, students may ask about the SYSIBM.SYSSTMT and SYSIBM.SYSSECTION tables. These catalog tables contain information about the SQL within a package, but this information is not intended to be analyzed on a day-by-day basis by the DBA, and is not included in the course for that reason. This is especially true of SYSSECTION. There is a SYSCAT.STATEMENTS view defined on SYSSTMT.

**Transition Statement** — Let's summarize the basic information concerning the preparation process to this point.

## Application process



### TASK

### STATEMENT

|                       |                                 |
|-----------------------|---------------------------------|
| Connect to musicdb    | db2 connect to musicdb          |
| Prepare C Source Code | db2 prep myapp.sqc bindfile     |
| Compile and Link      | icc myapp.c db2api.lib          |
| Bind at Local         | db2 bind myapp.bnd blocking all |
| Reset Connection      | db2 connect reset               |
| Execute Object Code   | myapp > myapp.out               |
| Edit Output           | more < myapp.out                |

© Copyright IBM Corporation 2007

Figure 10-16. Application process

CF238.3

### Notes:

The above example uses a source code program named myapp, which is written in C language with embedded SQL statements that access a DB2 database called *musicdb*.

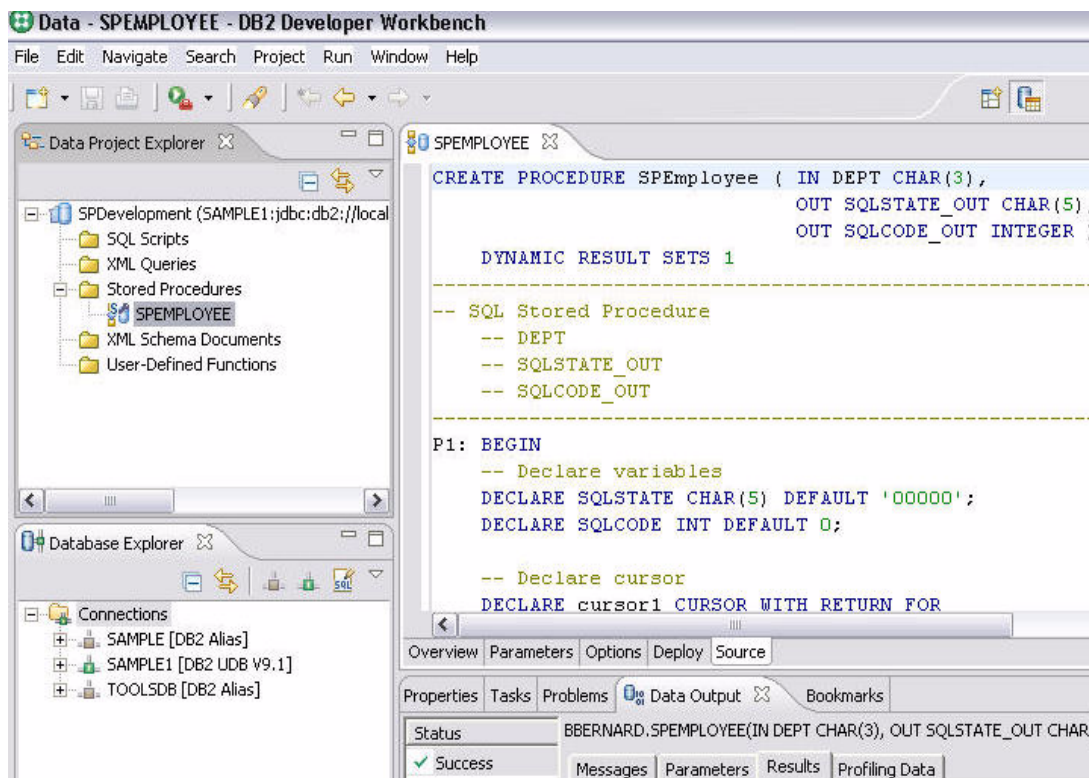
SQLLIB/samples directory has sample programs for a wide variety of languages and interfaces, including .NET, C/C++, CLP, COBOL, java, perl, php, SQL, Stored Procedures, Visual Basic, Visual C, and XML.

## ***Instructor Notes:***

**Purpose** — Summarize the basic process.

**Details** — Walk through the commands shown on the chart. The student notes show the COMPILE command for some different environments, but programming documentation may need to be consulted. Also, you may wish to ask the students what would need to be done if the package was needed on multiple servers.

# Developer Workbench and SQL PL



© Copyright IBM Corporation 2007

Figure 10-17. Developer Workbench and SQL PL

CF238.3

## Notes:

DB2 Developer Workbench is an Eclipse-based tool that replaces the Development Center from DB2 UDB for Linux, UNIX, and Windows, Version 8. Developer Workbench is a comprehensive development environment for creating, editing, debugging, deploying, and testing DB2 stored procedures and user-defined functions. You can also use Developer Workbench to develop SQLJ applications, and to create, edit, and run SQL statements and XML queries.

The Developer Workbench includes functionality that is comparable to Development Center, including these items:

- Database code designers for SQL statements, stored procedures, and user-defined functions
- Support for running statements in the database
- Debugging support for DB2 stored procedures

In addition to existing Development Center functionality, there are some additional new features:

**Migrate existing Development Center projects** - You can use a wizard to migrate existing Development Center projects into Developer Workbench.

**Compare routines** - You can use the compare editor to compare and make changes between two routines that are contained within a data development project in Developer Workbench. You can also compare routine attributes for routines that are stored on a server.

**Deploy routines to unlike servers** - You can deploy routines that were created for one DB2 database to a DB2 database on a different platform. For example, you can create a routine for a DB2 Database for Linux, UNIX, and Windows and then deploy it to a DB2 UDB for z/OS database. Not all server combinations are supported.

**Deploy binaries** - For SQL or Java stored procedures targeting DB2 UDB for z/OS, Version 8 or higher, you can deploy without going through a full rebuild. The binaries for a SQL procedure or the JAR file for a Java procedure are copied from the source to the target system.

**Launch Visual Explain** - You can launch Visual Explain for DB2 UDB for z/OS or DB2 for Linux, UNIX, and Windows SQL statements from either the routine editor for SQL routines or from the wizard for routine creation.

**Develop SQLJ applications** - You can develop SQLJ applications by using the following features:

**Generate an SQLJ template file by using a wizard** - Translate and compile SQLJ files automatically.

**Customize and bind SQLJ profiles by using a wizard** - Print SQLJ profile files.

**Edit applications by using code assist and templates** - Debug SQLJ files.

**Share project resources** - You can share your Developer Workbench data development project by using either Concurrent versions System (CVS) or ClearCase. After you share your project, you can manage all changes and update history, and you can synchronize your files with the repository.

**Edit table data** - You can use an editor to edit the data that is contained in a table. You can edit existing values, delete an existing row, or insert a new row.

**Extract and load data** - You can extract the data from a table or view into a file on the local file system. You can use this file to load the data into a table.

**Debug stored procedures** - Developer Workbench includes enhanced integrated stored procedure debugging capabilities. You can debug SQL or Java stored procedures for supported DB2 servers, or Java stored procedures for supported Derby servers.

**XML support** - Developer Workbench contains support for XML functions, the XML data type, and XML schema registration. You can also create XQueries with the XQuery builder.

**Learn about Developer Workbench through an information center and tutorials** - Developer Workbench help and tutorials are available in an information center that is installed with Developer Workbench. This information is for Developer Workbench only,



and it is not installed with the DB2 information center CD. To access the Developer Workbench information center, click **Help > Help Contents** from the main menu in the product. You can also link directly to important getting started information from the Welcome page in Developer Workbench by clicking **Help > Welcome**.

## ***Instructor Notes:***

**Purpose** — Summarize the functions of the Developer Workbench.

**Details** — The Developer Workbench in DB2 9 replaces Development Center in V8, which replaced the Stored Procedure Builder that was available in previous versions. It provides an interactive development environment for the development of stored procedures and user-defined functions.

**Transition Statement** — End of topic. Provide transition statement if appropriate.

## 10.4 Application performance

### Instructor topic introduction

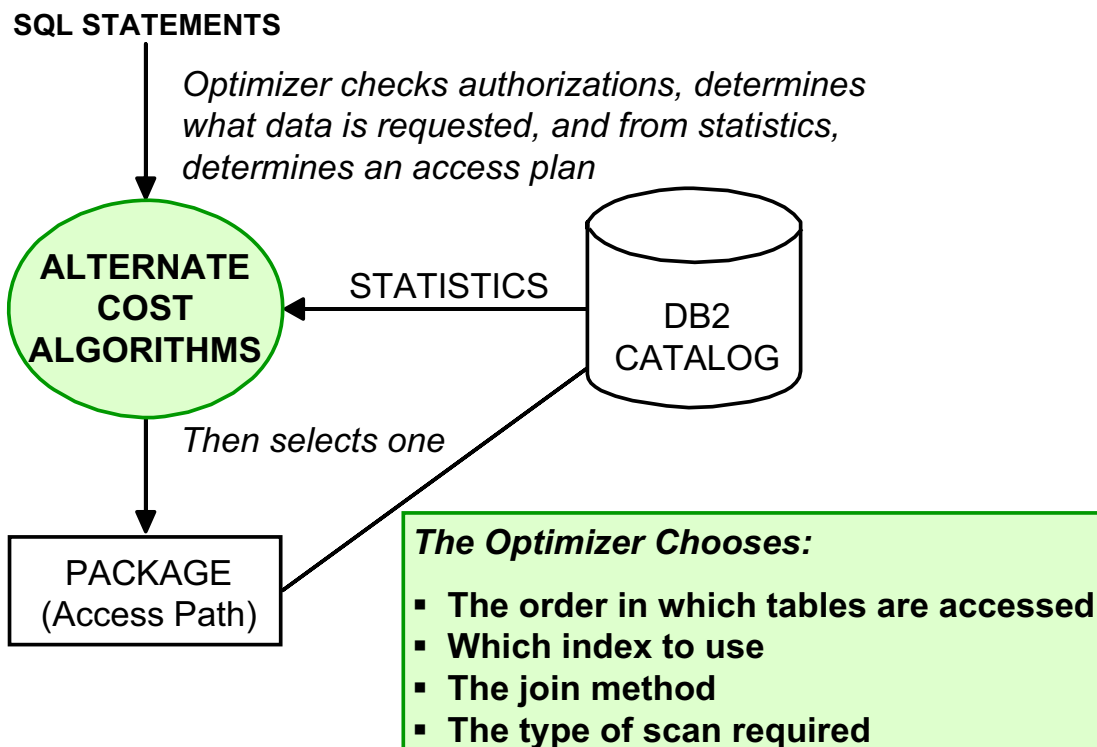
**What students will do** — Develop strategies to support applications at their installations.

**How students will do it** — Listen to lecture, ask questions, and complete paper and machine exercises.

**What students will learn** — The impact of optimization, blocking, REBIND, sorting, and configuration parameters.

**How this will help students on their job** — A database administrator is responsible for performance of applications from at least an advisory viewpoint. The topic provides the DBA with useful information to assist with this task.

## DB2 Optimizer



© Copyright IBM Corporation 2007

Figure 10-18. DB2 Optimizer

CF238.3

### Notes:

During the bind process, the optimizer will consider several factors to determine the access strategy.

The syntax constructs of the SQL statement itself will be analyzed. For example, consider the unique index on ARTNO in the ARTISTS table. A predicate of the form `ARTNO <> value` cannot usually take advantage of the index via a matching index scan, while a predicate of the form `ARTNO = value` could.

The size of tables, the number of unique values in columns and indexes, and the second highest and second lowest values in columns are just some examples of another key input for access strategy determination: catalog statistics. RUNSTATS is the utility that gathers these statistics.

The indexes available for consideration are available to the optimizer via the catalog. As a reminder, an index may be used to access particular data values or to avoid a sort.

The sizes of the buffer pools are another key input for the bind process. The size of the buffer pool determines the number of index and data pages that can be cached in it. Large

buffer pools can reduce the number of I/O operations necessary to satisfy SQL statements. Therefore, the size of the buffer pool can influence the optimizer because I/O costs are a major part of the overall cost formulas for access strategies.

The estimate of the average number of applications, indicated by the configuration parameter **avg\_appls**, can affect optimization. This parameter can be set between 1 and maxappls. If it is set to 1, the optimizer assumes that it has the entire buffer pool to use to support the statement. If it is set to higher values, the optimizer may consider strategies that burden the buffer pool to a lesser degree. In an environment that supports a large number of complex adhoc query requests, specifying a higher value may be appropriate to reflect the demand that will exist on the buffer pool.

The CPU speed of the machine, which can be indicated via the database configuration parameter **cpuspeed**, is also used by the optimizer as part of its cost evaluation. Normally, allowing DB2 to determine this value would be sufficient. Consult the reference material for further details.

Rather than optimizing a query once, when it is compiled, the Learning Optimizer (LEO) watches production queries as they run and then fine-tunes them as it learns about data relationships and user needs. This is helpful in large and complex databases.

LEO automatically self-validates the query optimizer's cardinality model by instrumenting the execution module to collect actual cardinalities at each step of the query execution plan. After the query completes, LEO compares these actuals to the query optimizer estimates to produce adjustment factors to be exploited by future optimization of queries that have similar predicates.

The approach is very general and can correct the result of any sequence of operations, not just the access of a single table or even just the application of predicates.

## ***Instructor Notes:***

**Purpose** — Identify some of the optimizer's function.

**Details** — Highlight some of the concepts addressed in the student notes as well as the LEO (Learning Optimizer). It may also be appropriate to relate the concept of optimization to a non-data processing task. An analogy that may be helpful is the following:

Assume you have a warehouse full of clothes that are organized by size (analogy to a clustered index.) Your task is to find all the size 6 outfits that are yellow. You could get all the yellow clothes together and then look at the size, but it would probably be easier to get to the size 6 section, and then pick out the yellow clothes. Of course, this analogy represents a basic decision and the optimizer has many more complex algorithms to consider.

Be certain to address the configuration parameters listed that affect optimization. You may also want to mention sortheap, but this is covered at a later point.

**Transition Statement** — The optimizer for DB2 is an advanced technology. It can analyze so many strategies that you may wish control the degree of analysis performed because it itself can be a costly application.

## Optimization classes

- QUERYOPT optimization level
  - 0 Use a minimal amount of optimization
  - 1 Use a degree of optimization roughly comparable to DB2/6000 Version 1, plus some additional low-cost features not found in Version 1
  - 2 Use features of opt level 5, but simplified join algorithm
  - 3 Perform a moderate amount of optimization; similar to the query optimization characteristics of DB2 for z/OS
  - 5 Use a significant amount of optimization; with Heuristic Rules (default)
  - 7 Use a significant amount of optimization; without Heuristic Rules
  - 9 Use all available optimization techniques

© Copyright IBM Corporation 2007

Figure 10-19. Optimization classes

CF238.3

### Notes:

When an SQL query is compiled, a number of optimization techniques can be used to determine the most efficient access plan for that query. Using more optimization techniques increases query compilation time as well as system resource usage. For this reason, you may wish to limit the number of techniques applied to optimizing your query by setting the optimization level when creating the package.

For example, if you know that the statements in an application are simple, perhaps with predicates that reference indexes with a high degree of cardinality, you may not want to expend the resources to optimize at a high degree.

A greater level of optimization may be justified for complex queries, since the cost spent at optimization may pay substantial benefit from the resulting access strategy. In other words, saving on optimization may result in an access strategy that is inefficient.

For dynamic SQL applications, the value of the **CURRENT QUERY OPTIMIZATION** special register will be used to determine the class to be used. This register can be updated using the **SET CURRENT QUERY OPTIMIZATION=n** statement, where n is a value of 0, 1, 2, 3, 5, 7, or 9, or a host variable containing one of these values.

More detail regarding optimization is provided in the *Administration Guide*.

## ***Instructor Notes:***

**Purpose** — Discuss the concept of influencing the resources expended when optimizing SQL statements.

**Details** — Describe at a high level what is done at the different optimization levels. Detailed discussion of these items are included in the *Performance Monitoring and Tuning* course, CF41.

The main points to highlight are the definitions on the visual itself and the general concept. This is also an area where the proper choice may not be intuitively obvious for a given application, but where some degree of *on the job* expertise development could be expected.

One approach to consider is to optimize at a high class if you have the cycles and don't see a negative impact during bind. Then, you could move to a lower class for applications that don't benefit greatly from the expensive optimization process.

**Additional Information** — Additional information about optimization, copied from the *Administration Guide: Performance*:

You can specify one of the following optimizer classes when you compile an SQL query:

**0** - This class directs the optimizer to use minimal optimization to generate an access plan. This optimization class has the following characteristics:

- Non-uniform distribution statistics are not considered by the optimizer.
- Only basic query rewrite rules are applied.
- Greedy join enumeration occurs.
- Only nested loop join and index scan access methods are enabled.
- List prefetch and index ANDing are not used in generated access methods.
- The star-join strategy is not considered.

This class should only be used in circumstances that require the lowest possible query compilation overhead. Query optimization class 0 is appropriate for an application that consists entirely of very simple dynamic SQL statements that access well-indexed tables.

**1** - This optimization class has the following characteristics:

- Non-uniform distribution statistics are not considered by the optimizer.
- Only a subset of the query rewrite rules are applied.
- Greedy join enumeration occurs.
- List prefetch and index ANDing are not used in generated access methods, although index ANDing is still used when working with the semijoins used in star joins.

Optimization class 1 is similar to class 0 except that Merge Scan joins and table scans are also available.

**2** - This class directs the optimizer to use a degree of optimization significantly higher than class 1, while keeping the compilation cost significantly lower than classes 3 and above for complex queries. This optimization class has the following characteristics:



- All available statistics, including both frequency and quantile non-uniform distribution statistics, are used.
- All query rewrite rules are applied, including routing queries to materialized query tables, except computationally intensive rules that are applicable only in very rare cases.
- Greedy join enumeration is used.
- A wide range of access methods are considered, including list prefetch and materialized query table routing.
- The star-join strategy is considered, if applicable.

Optimization class 2 is similar to class 5 except that it uses Greedy join enumeration instead of Dynamic Programming. This class has the most optimization of all classes that use the Greedy join enumeration algorithm, which considers fewer alternatives for complex queries, and therefore consumes less compilation time than classes 3 and above. Class 2 is recommended for very complex queries in a decision support or online analytic processing (OLAP) environment. In such environments, specific queries are rarely repeated exactly, so that a query access plan is unlikely to remain in the cache until the next occurrence of the query.

**3** - This class requests a moderate amount of optimization. This class comes closest to matching the query optimization characteristics of DB2 for MVS/ESA, OS/390, or z/OS. This optimization class has the following characteristics:

- Non-uniform distribution statistics, which track frequently occurring values, are used if available.
- Most query rewrite rules are applied, including subquery-to-join transformations.
- Dynamic programming join enumeration, as follows:
  - Limited use of composite inner tables
  - Limited use of Cartesian products for star schemas involving look-up tables
- A wide range of access methods are considered, including list prefetch, index ANDing, and star joins.

This class is suitable for a broad range of applications. This class improves access plans for queries with four or more joins. However, the optimizer might fail to consider a better plan that might be chosen with the default optimization class.

**5** - This class directs the optimizer to use a significant amount of optimization to generate an access plan. This optimization class has the following characteristics:

- All available statistics are used, including both frequency and quantile distribution statistics.
- All of the query rewrite rules are applied, including the routing of queries to materialized query tables, except for those computationally intensive rules which are applicable only in very rare cases.
- Dynamic programming join enumeration, as follows:
  - Limited use of composite inner tables
  - Limited use of Cartesian products for star schemas involving look-up tables

- A wide range of access methods are considered, including list prefetch, index ANDing, and materialized query table routing.

When the optimizer detects that the additional resources and processing time are not warranted for complex dynamic SQL queries, optimization is reduced. The extent or size of the reduction depends on the machine size and the number of predicates.

When the query optimizer reduces the amount of query optimization, it continues to apply all the query rewrite rules that would normally be applied. However, it does use the Greedy join enumeration method and reduces the number of access plan combinations that are considered.

Query optimization class 5 is an excellent choice for a mixed environment consisting of both transactions and complex queries. This optimization class is designed to apply the most valuable query transformations and other query optimization techniques in an efficient manner.

**7** - This class directs the optimizer to use a significant amount of optimization to generate an access plan. It is the same as query optimization class 5 except that it does not reduce the amount of query optimization for complex dynamic SQL queries.

**9** - This class directs the optimizer to use all available optimization techniques. These include:

- All available statistics
- All query rewrite rules
- All possibilities for join enumerations, including Cartesian products and unlimited composite inners
- All access methods

This class can greatly expand the number of possible access plans that are considered by the optimizer. You might use this class to find out whether more comprehensive optimization would generate a better access plan for very complex and very long-running queries that use large tables. Use Explain and performance measurements to verify that a better plan has actually been found.

**Transition Statement** — There are several methods for setting the optimization class.

## Adjusting the QUERY OPTIMIZATION class

- Database Configuration Parameter

```
db2 UPDATE DB CFG FOR musicdb USING DFT_QUERYOPT n
```

- Command

```
db2 SET CURRENT QUERY OPTIMIZATION=n
```

- Prep or Bind option

```
db2 PREP pgm1.sqc QUERYOPT n
```

- db2cli.ini file for CLI clients

```
DB2OPTIMIZATION=n
```

© Copyright IBM Corporation 2007

Figure 10-20. Adjusting the QUERY OPTIMIZATION class

CF238.3

### Notes:

DFT\_QUERYOPT defines the default optimization class used by the optimizer. This value can be set using the command:

```
db2 UPDATE DB CFG FOR <database> USING DFT_QUERYOPT n
```

where n is one of the optimization classes.

The optimization level used on dynamic SQL statements submitted through the CLP can be set using the command:

```
db2 SET CURRENT QUERY OPTIMIZATION=n
```

where n is one of the optimization classes.

- For an embedded SQL program, the optimization level can be set at PREP or BIND using the QUERYOPT n parameter.
- For dynamic SQL applications, the value of the **CURRENT QUERY OPTIMIZATION** special register can be used to determine the optimization level to use on subsequent dynamic SQL statements.
- For JDBC, ODBC or CLI applications, the value of **DB2OPTIMIZATION** set in the db2cli.ini file will set the optimization level.

## ***Instructor Notes:***

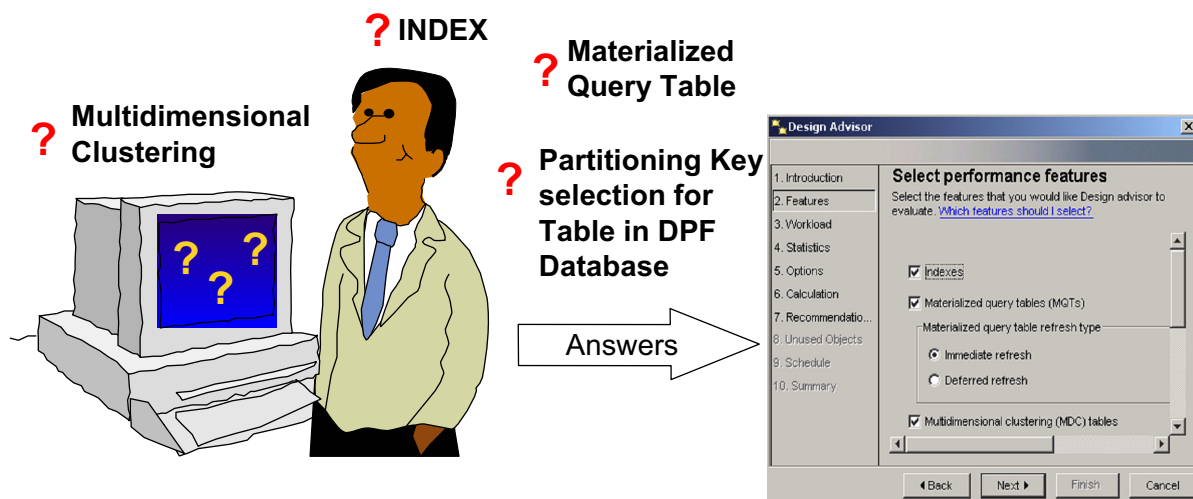
**Purpose** — To show the various methods for setting the optimization level.

**Details** — The optimization level can be set in different ways depending on the interface being used.

**Transition Statement** — You can get assistance in the design of your indexes through use of the Design Advisor.

## Design Advisor

- Assists in finding the right indexes, MQTs or MDC
  - Based on workload
  - Virtual indexes
- Reduce complexity of performance analysis and tuning



© Copyright IBM Corporation 2007

Figure 10-21. Design Advisor

CF238.3

### Notes:

Appropriate indexes have great impact on the access plans selected and the performance of SQLs.

The Design Advisor, which can be invoked by right-clicking a database and selecting Design Advisor, is a tool that can help you significantly improve your workload performance. The task of selecting which indexes, MQTs, clustering dimensions, or partitions to create for a complex workload can be daunting. The Design Advisor identifies all of the objects needed to improve the performance a particular workload (can include SELECT, INSERT, UPDATE, and/or DELETE statements). Given a set of SQL statements, it will generate recommendations for:

- New indexes
- New materialized query tables (MQTs)
- Conversion to multidimensional clustering tables (MDC)
- Repartitioning of tables
- Deletion of indexes and MQTs unused by the specified workload

You can decide to implement some or all of the recommendations immediately or schedule them for a later time.

The Design Advisor can also help you to migrate from a single-partition database to a multi-partitioned-environment.

For example, over a one month period of time your database manager may have to process 1,000 INSERTs, 10,000 UPDATEs, 10,000 SELECTs, and 1,000 DELETEs. The information in the workload is concerned with the type and frequency of the SQL statements over a given period of time. The advising engine uses this workload information in conjunction with the database information to recommend indexes. The goal of the advising engine is to minimize the total workload cost. This information is written to the ADVISE\_WORKLOAD table. With sufficient information/constraints, the Design Advisor is able to suggest the appropriate actions to take for your tables.

You can certainly run the Design Advisor from the command line, then the output is printed to stdout by default and saved in the ADVISE\_TABLE and ADVISE\_INDEX tables. Partitioning strategies can be found in the ADVISE\_PARTITION table. The RUN\_ID value in all these tables corresponds to the START\_TIME value in the ADVISE\_INSTANCE table for each execution of the Design Advisor.

To create the ADVISE\_WORKLOAD and ADVISE\_INDEX tables, run the EXPLAIN.DDL script found in the misc subdirectory of the sqllib subdirectory. If not already created, the Design Advisor can also create the tables.

***Instructor Notes:***

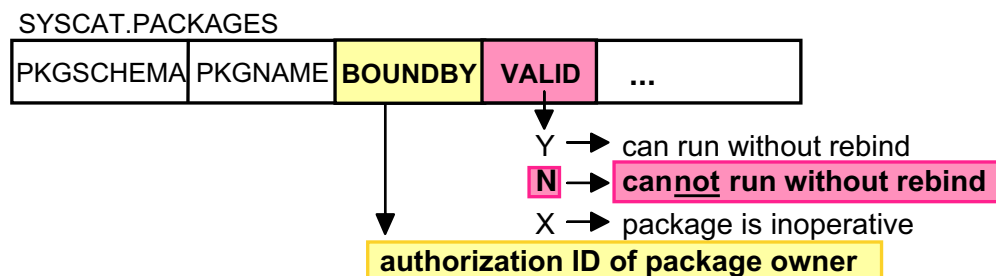
**Purpose** — Introduce the Design Advisor.

**Details** — If you need assistance in figuring out which indexes, MQTs, or MDCs are required, given workload frequency, DB2 Design Advisor will recommend.

**Transition Statement** — There are situations when the original access strategy determined by the bind process becomes invalid.

## Implicit rebind

- **Implicit rebinding** is automatic at next execution and triggered if:
  - Drop object referenced in package
  - Drop primary key on table referenced in package
  - Drop/add referential constraint on parent or child table referenced in package
  - Revoke **privilege required by owner** to execute static SQL statement embedded in package
  - Requested using the REOPT option



© Copyright IBM Corporation 2007

Figure 10-22. Implicit rebind

CF238.3

### Notes:

**Implicit rebinding** is available on DB2. It is totally automatic at execution time and is triggered under the above circumstances. When SYSCAT.PACKAGES.VALID equals **N**, then an implicit rebind is triggered at execution time. When an implicit rebind is performed, the access paths for all the SQL statements are rebound. The BOUNDBY authorization ID, the ID of the original package owner, is checked for the appropriate authority and privileges to execute the embedded static SQL statements. During an implicit bind, if an object does not exist or a privilege has been revoked but not regranted, the execution of the package will fail. SYSCAT.PACKAGES.VALID is marked **N** when an index is dropped. The package will be implicitly rebound and VALID will equal **Y** even if the index has not been re-created. SYSCAT.PACKAGES.VALID is marked **N** when a table or view is dropped. The package will be implicitly rebound and VALID will equal **Y** only if the table or view has been re-created before the implicit rebind.

During BIND you can also specify the REOPT parameter. NONE is the default. You can select ALWAYS or ONCE, and each for the first time that a statement containing host variables, parameter markers, or special registers is executed, the values for these variables are used to optimize the access path for the statement.



SYSCAT.PACKAGE.VALID will be marked **X** if the package is inoperative because some function instance that it depends on has been dropped. This would occur if the package were dependent on a user-defined function and that user-defined function were dropped. These packages will **NOT** be implicitly rebound.

**Note**

User-defined functions are discussed in the class, *DB2 for Linux, UNIX, and Windows Database Advanced Programming*.

Dropping or creating a trigger on a column or table referenced by an application will cause the package to become invalid.

## ***Instructor Notes:***

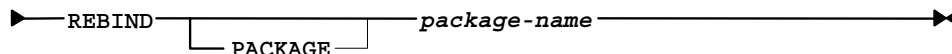
**Purpose** — Introduce the concept of implicit rebinding.

**Details** — DB2 may implicitly rebind your package if the package has been marked as invalid (**N** in the VALID column of SYSCAT.PACKAGES). (Chart gives examples of things that would cause the package to be marked invalid.)

**Additional Information** — The package could also be marked as inoperative, indicating that a user-defined function that it is dependent upon has been dropped. Such packages must be explicitly rebound. The automatic function is not applicable to packages marked inoperative.

**Transition Statement** — You may also elect or be required to explicitly rebind a package.

## Explicit rebind



- New indexes available
- Statistics in catalog have changed
- Control when invalid packages are bound
- Must explicitly bind inoperative packages

© Copyright IBM Corporation 2007

Figure 10-23. Explicit rebind

CF238.3

### Notes:

Rebinding is the process of re-creating a package for an application program that was previously bound.

Invalid packages are automatically (or implicitly) rebound by the database manager when they are executed.

Inoperative packages must be explicitly bound again.

You may choose to rebind a package to take advantage of a newly created index or make use of updated statistics after executing the RUNSTATS command.

Another reason to explicitly rebind is to choose the time when the binding operation occurs, rather than allowing it to occur when a user requests the application that has an invalid package.

The authorization checking for a REBIND is based on the ID contained in the BOUNDBY column of SYSCAT.PACKAGES.

Parameters cannot be changed through a REBIND operation.

With the db2rbind command you are able to rebind all packages of a given database.

### References — Command Reference

## ***Instructor Notes:***

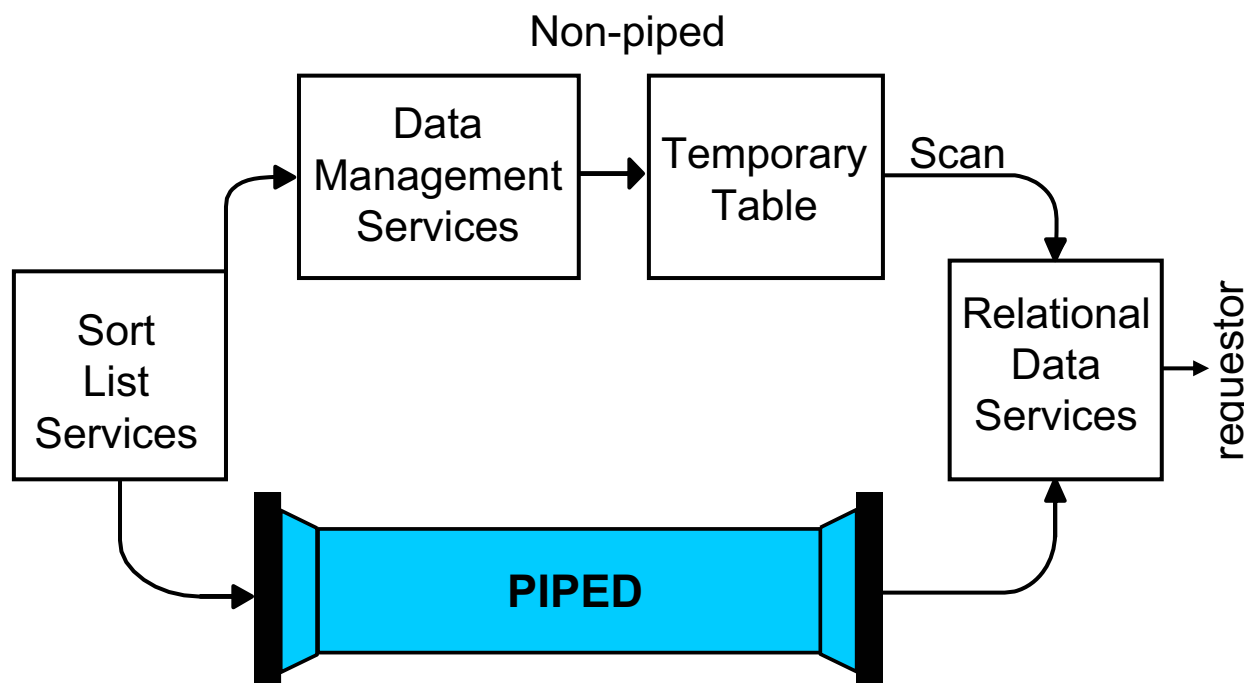
**Purpose** — Introduce explicit rebinds.

**Details** — You may choose to do an explicit rebind. See the chart for some reasons why.

**Additional Information** — Certification objective: 5E Ability to use the REORG, REORGCHK, REBIND and RUNSTATS utilities.

**Transition Statement** — Sorts are frequently a performance concern.

## Sorts — piped versus non-piped



© Copyright IBM Corporation 2007

Figure 10-24. Sorts — piped versus non-piped

CF238.3

### Notes:

A sort is piped if the rows can be returned directly from the sort buffers to Relational Data Services. Non-piped sorts require calls to Data Management Services in order to build a temporary table. This temporary table will then be scanned by Relational Data Services. Relational Data Services provides the results to the requesting application.

Piped sorts can provide performance benefits by reducing I/O operations. The number of piped sorts requested and the number accepted can be monitored. If many requested piped sorts are being rejected, tuning the database configuration parameter **sortheap**, or the database manager configuration parameter **sheapthres**, or both, may be necessary. **sortheap** determines the amount of memory allocated for an agent when a sort is requested. In-memory sorts are better performers than those that require sort passes. (See the next visual.)

**sheapthres** can be used to control the memory resources used to support sorting across all databases. When the value specified is reached, subsequent sort heap allocation against any database will be attempted, but will be constrained. Such *post threshold sorts* will not be piped. This parameter should be modified if benchmarking indicates poor balancing between sort performance and memory usage.

## ***Instructor Notes:***

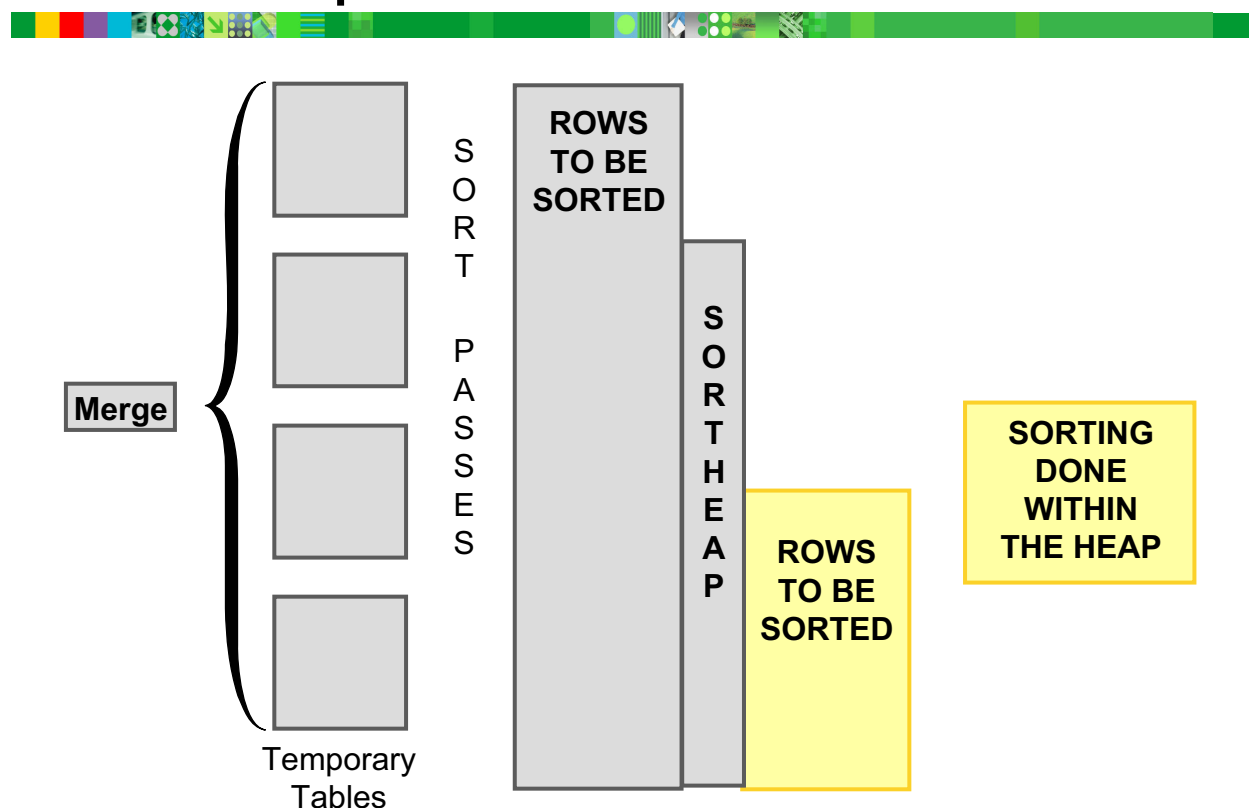
**Purpose** — Define piped and non-piped sorts.

**Details** — Piped sorts require less overhead than non-piped sorts. During the bind process, the optimizer will examine the access path, sortheap, and sheapthres to determine if piping should be requested. It is not useful in every case, for example, when an intermediate table is needed to support certain joins.

At execution time, piping will or will not occur depending on the optimizer's decision and the resources available at the time of the sort request.

**Transition Statement** — Another consideration concerning the performance of sorts is whether or not merge passes are required.

## Sorts — Sort passes



© Copyright IBM Corporation 2007

Figure 10-25. Sorts — Sort passes

CF238.3

### Notes:

Regardless of whether a sort is piped, a key consideration concerning the performance of sorts is the number of rows sorted.

If the number of rows sorted exceeds the size of the sort heap, Sort List Services will use temporary tables to hold subsets of the data to be sorted. If these tables cannot fit in memory, they will be written to disk. This is the *worst* case scenario illustrated in the visual.

The temporary files will contain the subset of rows sorted in a sort pass. Therefore, after all sort passes are complete, the temporary disk files will need to be read and merged. This technique increases the I/O and CPU cost of the sort.

If the space available in the sort heap can accommodate the entire set of rows to be sorted, a single pass sort can be completed and the use of temporary disk files is avoided.

## ***Instructor Notes:***

**Purpose** — Highlight the impact of sortheap on performance.

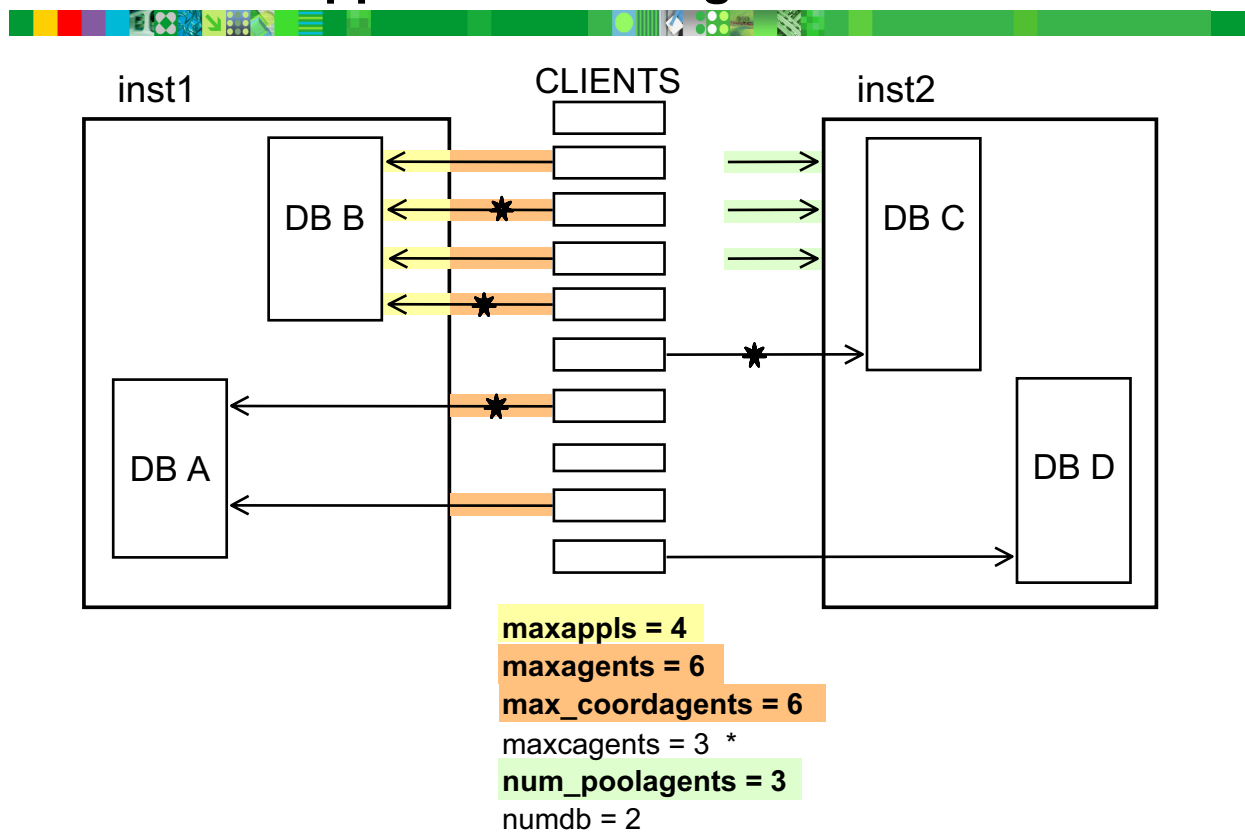
**Details** — The *temporary disk files* shown on the visual illustrate the worst case: when the temporary tables cannot fit in the buffer. It is possible that such would not be the case. However, it is still more expensive to use merge passes than not.

Allocating memory to sortheap is a trade-off of favoring sort performance over other uses of such memory. The value for sortheap is applicable to all sorts requested within the database. Therefore, providing a value that supports the largest sort requests against the database will likely waste memory for most sorts. The students should be aware that performance monitoring may be justified if fine tuning of this parameter is desired.

**Transition Statement** — Several parameters can be used to control the number of applications you allow to execute simultaneously, possibly enabling more resources for the applications getting serviced.



## Concurrent application tuning introduction



© Copyright IBM Corporation 2007

Figure 10-26. Concurrent application tuning introduction

CF238.3

### Notes:

The values used in the above example are in no way intended to mean that such values are good starting points. Most likely, higher values will be used at your installation. The values used were chosen to illustrate the purpose of these parameters.

**max\_coordagents** determines the maximum number of coordinating agents that can exist at one time on a server. **maxagents** controls the number of agents that can access any database within an instance. In a non-partitioned and non-SMP environment, **max\_coordagents** and **maxagents** should be set to the same value. In the above example, inst1 could not support another connection to the database DBA because **maxagents** has been reached. However, inst2 could support additional agents.

**maxappls** serves a similar purpose as **maxagents**, but at the database level. It determines the maximum number of connections that can currently exist for the database. In the example, DB B could not support another connection because **maxappls** has been reached. However, DB C and DB D could support additional connections because **maxagents** and **maxappls** has not been reached.

**maxcagents** determines the number of agents that can simultaneously execute a transaction. Therefore, the value of this parameter is less than or equal to the value of maxagents. This parameter can be used to limit the amount of peak system resources necessary to support connected agents. While it does not prevent connections across the instance, (that is the function of maxagents), it does control simultaneous resource demands of the database. In the example, only three agents can be simultaneously executing a transaction. Therefore, inst1 would not allow any other currently connected application to execute an SQL statement. However, inst2 has only one agent currently executing SQL, so two additional agents could run transactions.

**num\_poolagents** determines the number of agents that should be reserved in *standby* after they have been released by an application. In the example, inst2 has three idle agents waiting to become attached to a client when additional requests occur. Note that the idle agents, up to the number specified for the parameter, will exist either because they have previously served some application, or because they were created in the agent pool due to the setting of **num\_initagents**.

**num\_initagents** determines the initial number of idle agents that are created in the agent pool at DB2START time.

**numdb** determines the number of active databases (those that can have one or more connections) within the instance. In the example, inst2 could not support a connection to a database other than DB C or DB D, even though the agent-oriented thresholds have not been reached.

***Instructor Notes:***

**Purpose** — Highlight configuration parameters that impact the number of concurrent database and connections.

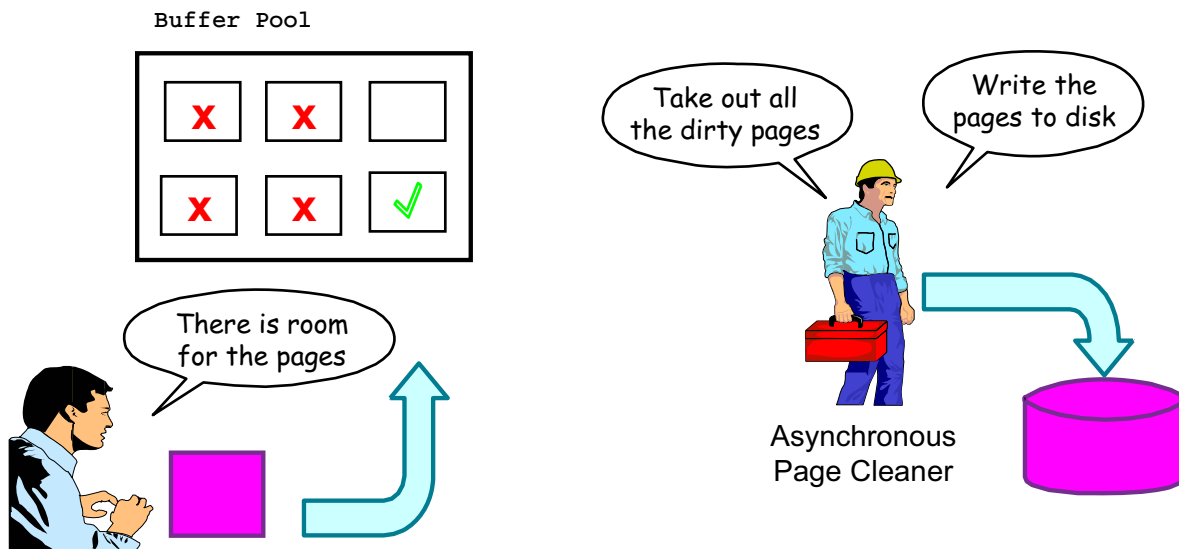
**Details** — The number of concurrent agents impacts the resources that must be available. Many configuration parameters, such as sortheap, are applied on a per agent basis. Therefore, the number of agents allowed can impact performance if resources are constrained. In other words, it is better to allow 50 applications to execute with adequate performance than attempt to let 100 applications try to execute with none generally succeeding.

**Additional Information** — This may be a good time to advertise the performance classes.

**Transition Statement** — Let's look at page cleaning.

# Asynchronous Page Cleaner

CHNGPGS\_THRESH=  
NUM\_IOCLEANERS=  
BUFFPAGE=



© Copyright IBM Corporation 2007

Figure 10-27. Asynchronous Page Cleaner

CF238.3

## Notes:

Asynchronous page cleaners will write changed pages from the buffer pool to disk before the space in the buffer pool is required by a database agent. This means that the agents will not have to wait for a changed page to be written out before being able to read a page, and your application's transactions should run faster.

Page cleaning is affected by the size of the buffer pool. The default size for buffer pools is set by the **BUFFPAGE** parameter, but the sizes of the independent buffer pools that are defined for the database is the primary setting that needs to be considered.

You may use the **CHNGPGS\_THRESH** database configuration parameter to specify the level (percentage) of changed pages at which the asynchronous page cleaners will be started, if they are not currently active. When the page cleaners are started, they will build a list of the pages to write to disk. Once they have completed writing those pages to disk, they will become inactive again and wait for the next trigger to start.

In a read-only environment, these page cleaners are not used.

For databases with a heavy update transaction workload, ensure that there are enough clean pages in the buffer pool by setting the CHNGPGS\_THRESH to be equal to or less than the default value (60). A percentage larger than the default can help performance if your database has a small number of very large tables.

The **NUM\_IOCLEANERS** database configuration parameter allows you to specify the number of asynchronous page cleaners for a database. These page cleaners write changed pages from the buffer pool to disk before the space in the buffer pool is required by a database agent. This means that the agents will not wait for changed pages to be written out before being able to read a page. As a result, your application's transactions should run faster.

If you set the parameter to zero, no page cleaners are started and as a result, the database agents will perform all of the page writes from the buffer pool to disk. This parameter can have a significant performance impact on a database stored across many physical storage devices, since in this case there is a greater chance that one of the devices will be idle. If no page cleaners are configured, your applications may encounter periodic log full conditions.

If the applications for a database primarily consist of transactions that update data, an increase in the number of cleaners will speed up performance. Increasing the page cleaners will also decrease recovery time from soft failures, such as power outages, because the contents of the database on disk will be more up-to-date at any given time.

When setting the value for NUM\_IOCLEANERS, consider the following recommendations:

- Application type
  - If it is a query-only database that will not have updates, set this parameter to zero.
  - If transactions are run against the database, set this parameter to be between one and the number of physical storage devices used for the database.
- Workload
  - Environments with high update transaction rates may require more page cleaners to be configured.
- Buffer pool sizes
  - Environments with large buffer pools may also require more page cleaners to be configured.

The database system monitor may be used to tune NUM\_IOCLEANERS using information from the event monitor about write activity from a buffer pool.

## ***Instructor Notes:***

**Purpose** — Identify considerations with page cleaning.

**Details** — Address the concept of page cleaning, and why it is good in a transaction processing environment. Define the purpose of the configuration parameters which are listed on the graphic.

**Transition Statement** — Communication costs between server and client could be a performance concern as well.

## 10.5 Minimizing communication overhead

### **Instructor topic introduction**

**What students will do —**

**How students will do it —** Listen to lecture.

**What students will learn —** What alternatives there are in minimizing communication overhead.

**How this will help students on their job —**

## Minimizing communication overhead



*"TALK ISN'T CHEAP"*



- Techniques to minimize overhead:
  - Blocking
  - Stored procedures

© Copyright IBM Corporation 2007

Figure 10-28. Minimizing communication overhead

CF238.3

### **Notes:**

In a client/server environment, communications can be a considerable cost element of applications.

By reducing the amount of communication between the client and server, performance gains can be realized.

Techniques that may possibly be exploited are:

- Blocking
- Stored Procedures

These techniques are the subject of several subsequent pages.



***Instructor Notes:***

**Purpose** — Identify techniques to minimize communication overhead.

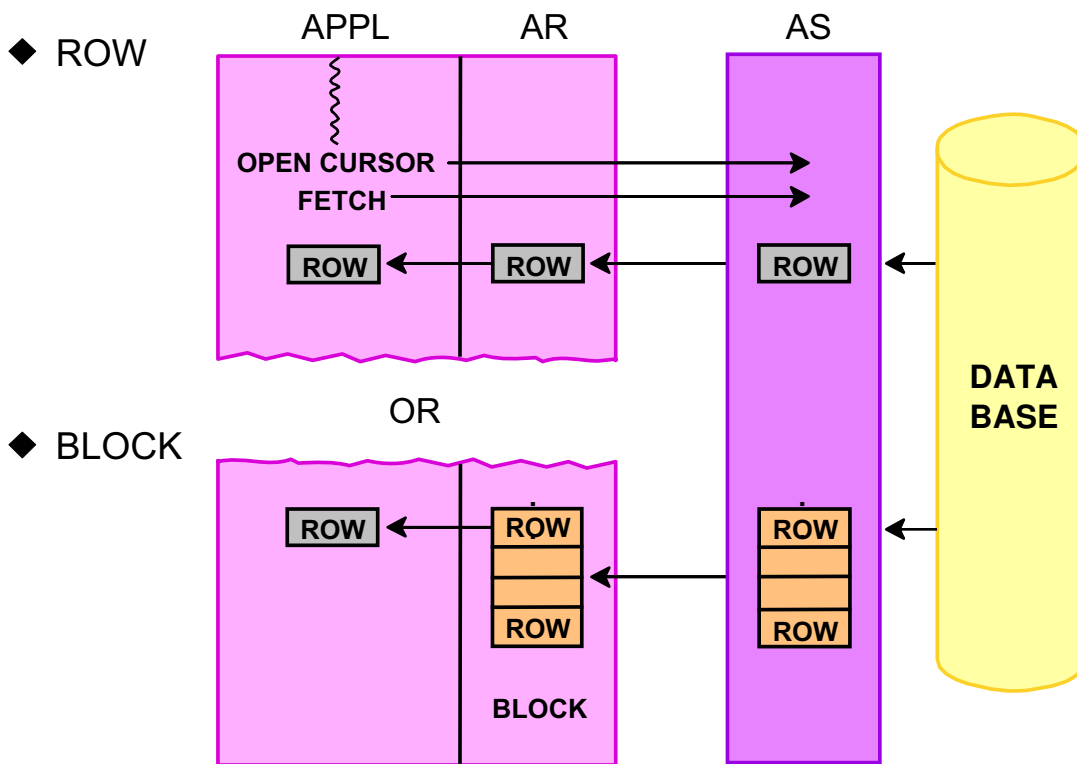
**Details** — Passing large amounts of information from server to client could result in poor performance. If transmissions can be minimized, a performance advantage could be realized.

The techniques listed on the graphic, blocking and stored procedures, are addressed on the following pages.

**Additional Information** —

**Transition Statement** — Let's look at blocking.

## Levels of data transmission



© Copyright IBM Corporation 2007

Figure 10-29. Levels of data transmission

CF238.3

### Notes:

A non-blocking operation entails a high communication expense. Each **FETCH** request is sent from the client to the server, and one row is returned for each request. This technique is high in CPU and data transmission costs.

A blocking operation reduces the communication and CPU expense. When record blocking is utilized, a single **FETCH** request from the client will result in a multiple row transfer between the server and the client. The results are fetched one at a time from a buffer allocated at the client.

After the client buffer is emptied, the next **FETCH** will cause an additional block of rows to be returned from the server.

The application program logic is not changed to enable or utilize blocking, with the possible exception of cursor declarations. However, other factors will determine when blocking is utilized.

Database Manager configuration parameters that determine the size of the block are **rqrioblk** and **aslheapsz**. These parameters also determine the size of a communication

buffer between an agent and an application. For local applications, the parameter **aslheapsz** is used to allocate the cache for row blocking. For remote applications, the parameter **rqrioblk** on the client workstation is used to allocate the cache for row blocking. The cache is allocated on the client. Generally, for adequate performance, these parameters must be greater than or equal to the following:

```
(sizeof(input SQLDA)+sizeof(output SQLDA)+sizeof(input SQLVARs)+250)
```

-----  
4096 (bytes/page)

Otherwise, the request to the database manager or the reply will not fit in a single send and receive pair. In simpler terms, the size of this buffer needs to accommodate the largest amount of data for a single request, or the request will be handled with more communication send/receive pairs.

If the system is not constrained on memory, setting these parameters to even larger values will cause larger blocks of memory to be allocated to support blocking cursors.

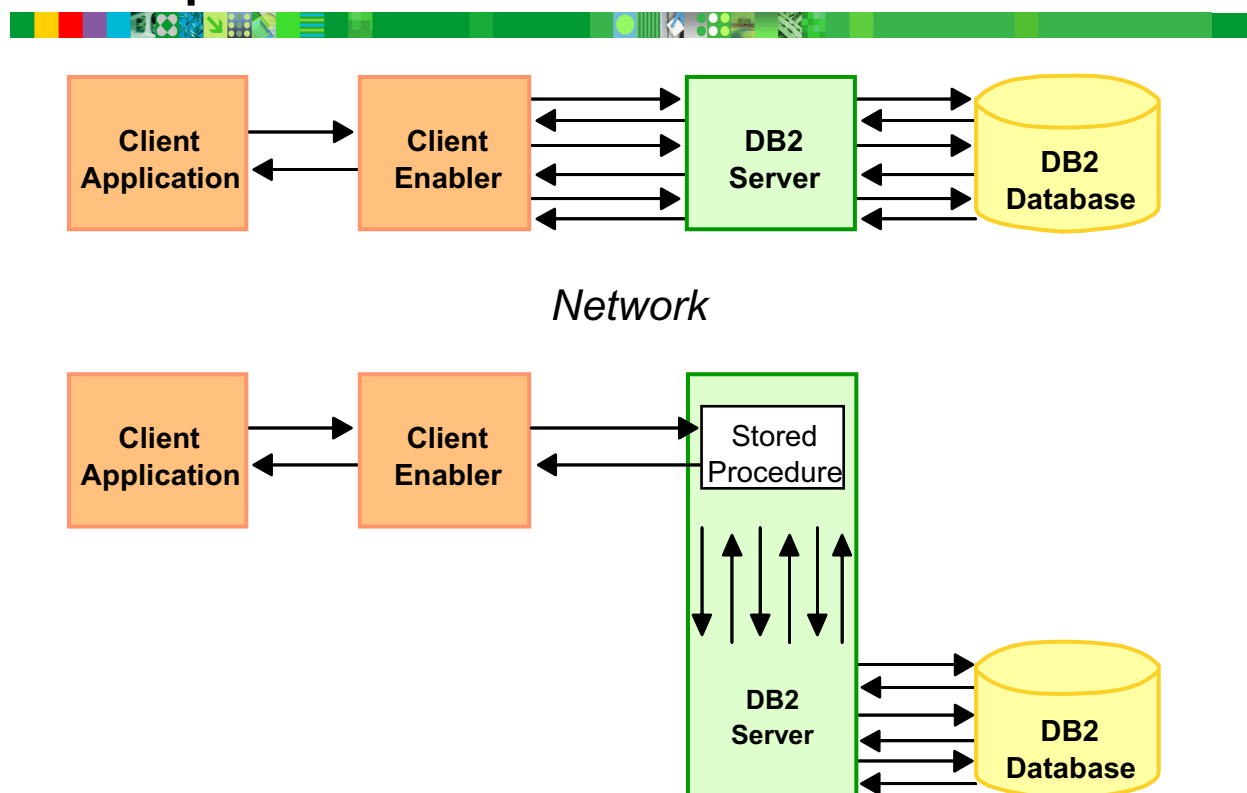
### ***Instructor Notes:***

**Purpose** — Emphasize the performance benefit of block protocol transmission for read-only data results.

**Details** — The major benefits of blocking are reduced CPU and communication overhead costs.

**Transition Statement** — Now let's have a look at stored procedures and how they can help to improve performance in an client/server environment.

## Stored procedures



© Copyright IBM Corporation 2007

Figure 10-30. Stored procedures

CF238.3

### Notes:

Stored procedures allow an application running on a client to call a procedure stored on a database server. This *stored procedure* executes and accesses the database locally and returns information to the client application.

To use this technique, an application must be written in two separate procedures. The calling procedure is contained in a client application and executes on the client. The *stored procedure* executes at the location of the database on the database server.

Applications that use stored procedures have the following advantages:

- Reduced network traffic
- Improved performance of server-intensive work
- Access to features that exist only on the database server.
- Minimized maintenance if logic in the server code requires modification.

Stored procedures can be *fenced* or *not fenced*. Fenced procedures run in processes separate from the database agent process or thread. Not fenced procedures run as part of the agent process or thread. The advantage of not fenced procedures is performance,

since the stored procedure and database agent do not need to communicate through a router. However, the integrity of the database agent is exposed. Not fenced stored procedures must be carefully tested.

Database manager configuration parameters that impact fenced procedures are listed below:

- **fenced\_pool** - For threaded db2fmp processes (processes serving threadsafe stored procedures and UDFs), this parameter represents the number of threads cached in each db2fmp process. For nonthreaded db2fmp processes, this parameter represents the number of processes cached.
- **keepfenced** - This parameter indicates whether a fenced mode process is kept after a fenced mode routine call is complete. Fenced mode processes are created as separate system entities in order to isolate user-written fenced mode code from the database manager agent process. This parameter is only applicable on database servers.

If *keepfenced* is set to *no*, and the routine being executed is not threadsafe, a new fenced mode process is created and destroyed for each fenced mode invocation. If *keepfenced* is set to *no*, and the routine being executed is threadsafe, the fenced mode process persists, but the thread created for the call is terminated. If *keepfenced* is set to *yes*, a fenced mode process or thread is reused for subsequent fenced mode calls. When the database manager is stopped, all outstanding fenced mode processes and threads will be terminated.

Setting this parameter to *yes* will result in additional system resources being consumed by the database manager for each fenced mode process that is activated, up to the value contained in the *fenced\_pool* parameter. A new process is only created when no existing fenced mode process is available to process a subsequent fenced routine invocation. This parameter is ignored if *fenced\_pool* is set to 0.

- **num\_initfenced** - This parameter indicates the initial number of nonthreaded, idle db2fmp processes that are created in the db2fmp pool at DB2START time. Setting this parameter will reduce the initial startup time for running non-threadsafe C and Cobol routines. This parameter is ignored if *keepfenced* is not specified.

It is much more important to set *fenced\_pool* to an appropriate size for your system than to start up a number of db2fmp processes at DB2START time.

## References

*Application Programming Guide*

*Administration Guide*

***Instructor Notes:***

**Purpose** — Introduce stored procedures.

**Details** — Stored procedures can provide a performance advantage when performing server-intensive work.

**Additional Information** —

**Transition Statement** — Let's look at the operational utilities.





## 10.6 Utilities and tools

### Instructor topic introduction

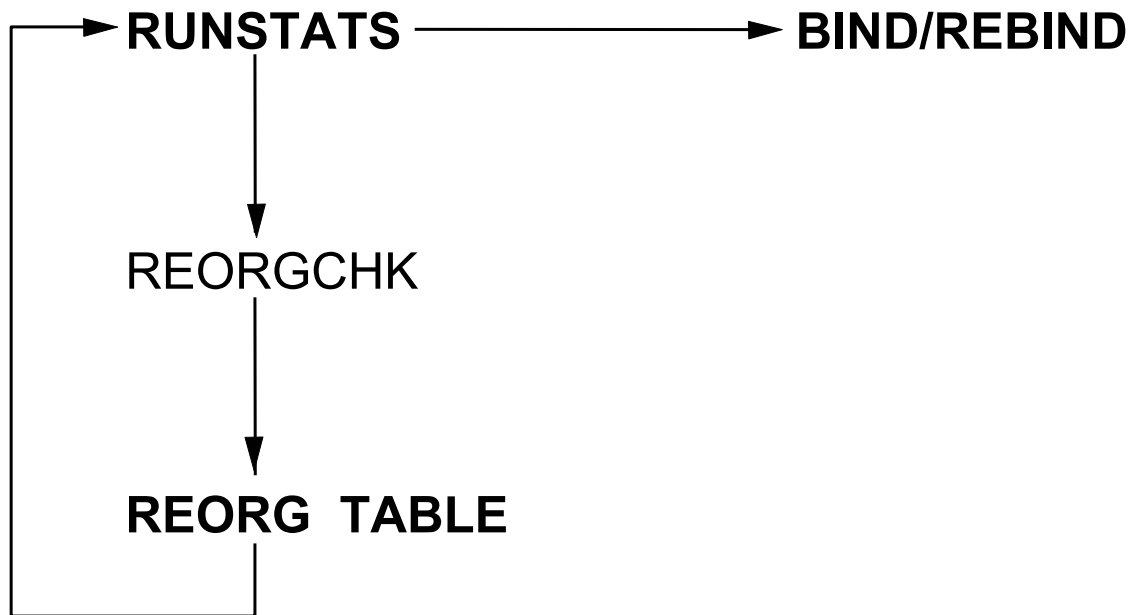
**What students will do** — Examine the utilities and tools related to application performance.

**How students will do it** — Listen in class and then do paper and machine exercises.

**What students will learn** — The proper use of utilities such as RUNSTATS, REORG, and REORGCHK.

**How this will help students on their job** — A database administrator is usually responsible for running the highlighted utilities as well as providing consulting support for problem determination. This topic provides the DBA with the functions of the targeted utilities as well as related information.

## Operational utilities



© Copyright IBM Corporation 2007

Figure 10-31. Operational utilities

CF238.3

### Notes:

**RUNSTATS** updates the statistics about tables and indexes. The statistics are stored in the system tables. **RUNSTATS** should be used after massive changes to the data and possibly after running **REORG**.

**REORGCHK** examines the data in the system tables and applies formulas to determine whether reorganization of a table, its indexes, or both is potentially needed. **REORGCHK** can also invoke **RUNSTATS** before examining the statistics. **REORGCHK** may be run periodically, or when users notice a degradation in performance.

**REORG** eliminates fragmentation in tables and indexes and may optionally order the rows of a table according to the order of an index. **REORG** should be used when **REORGCHK** indicates that **REORG** is needed and when performance degrades over time: when data inserts, updates, and deletes cause the clustering or space utilization to degrade.

**BIND** of new applications should occur after statistics for the accessed tables and their supporting indexes have been gathered.

After using RUNSTATS, it MAY be advantageous to REBIND applications so that the DB2 optimizer can take advantage of the newly updated statistics when choosing the access paths for the programs. The degree to which you choose to REBIND depends on your knowledge of the changes that have occurred to the state of the tables accessed since the last bind activity. Some installations will REBIND critical applications after updating statistics without much analysis simply because the cost of doing so is considered minimal. Other installations will carefully monitor changes in statistics and only REBIND applications when some percentage of change to the statistics reaches a installation defined limit.

The `db2rbind` command can be used to rebind a number of packages in a database with one command. By default, it will only rebind invalid packages, but you can specify that rebinding of all valid and invalid packages is to be done. Documentation on `db2rbind` can be found in the *Command Reference*.

## Instructor Notes:

**Purpose** — Introduce the utilities that will be discussed in greater detail in this unit.

**Details** — The instructor should clearly indicate that determining the frequency of the activities depicted is a combination of knowing the data and applications, monitoring and evaluating the performance of applications, and applying experience gained from the last reorganization cycle. For example, the following case illustrates when a package need not be REBOUND to restore adequate performance.

1. Table is clustered by a particular index.
2. RUNSTATS gathers statistics for table and index.
3. Package is bound and uses index because of clustering.
4. Clustering of data degrades due to INSERT, UPDATE, DELETE activity.
5. Package performance degrades due to loss of clustering.
6. Data is REORGANIZED, compacting space and reestablishing original clustering.
7. RUNSTATS is executed.

The package will benefit from the REORG even though it may not be rebound. This assumes that the INSERT, UPDATE, and DELETE activity primarily impacted the clustering of the data but did not drastically impact the number of rows, the cardinality of indexes, and so forth. The original strategy developed when the clustering was high simply *fits* the state of the data again.

Certainly, the approach of “We have the time and resources to REBIND x number of our most critical packages after RUNSTATS, so we'll do it” is appropriate in some cases.

**Note:** The above assumes that the DBA has a great deal of knowledge both about their database statistics and their applications. The best strategy in most cases would be to always run RUNSTATS after a REORG, and rebind all applications using those tables.

## Additional Information

The syntax of the db2rbind command is:

```
>>--DB2RBIND database /l logfile --|-----|---->
                                   |-all--|

                                   |-/r conservative--|
>-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
        |-/u userid--/p password--| |-/r any-----|
```

Command parameters:

database

specifies an alias name for the database whose packages are  
to be revalidated

/l

specifies the (optional) path and the (mandatory) file name to be used for recording errors that result from the package revalidation procedure

all

specifies that rebinding of all valid and invalid packages is to be done. If this option is not specified, all packages in the database are to be examined, but only those packages that are marked as invalid are rebound, so that they are not rebound implicitly during application execution.

/u

User ID. This parameter must be specified if a password is specified.

/p

Password. This parameter must be specified if a user ID is specified.

/r conservative

Use RESOLVE CONSERVATIVE option during rebind. This is the default. Only functions and types in the SQL path that were defined before the last explicit bind time stamp (for each package) are considered for function and type resolution. Conservative binding semantics are used.

/r any

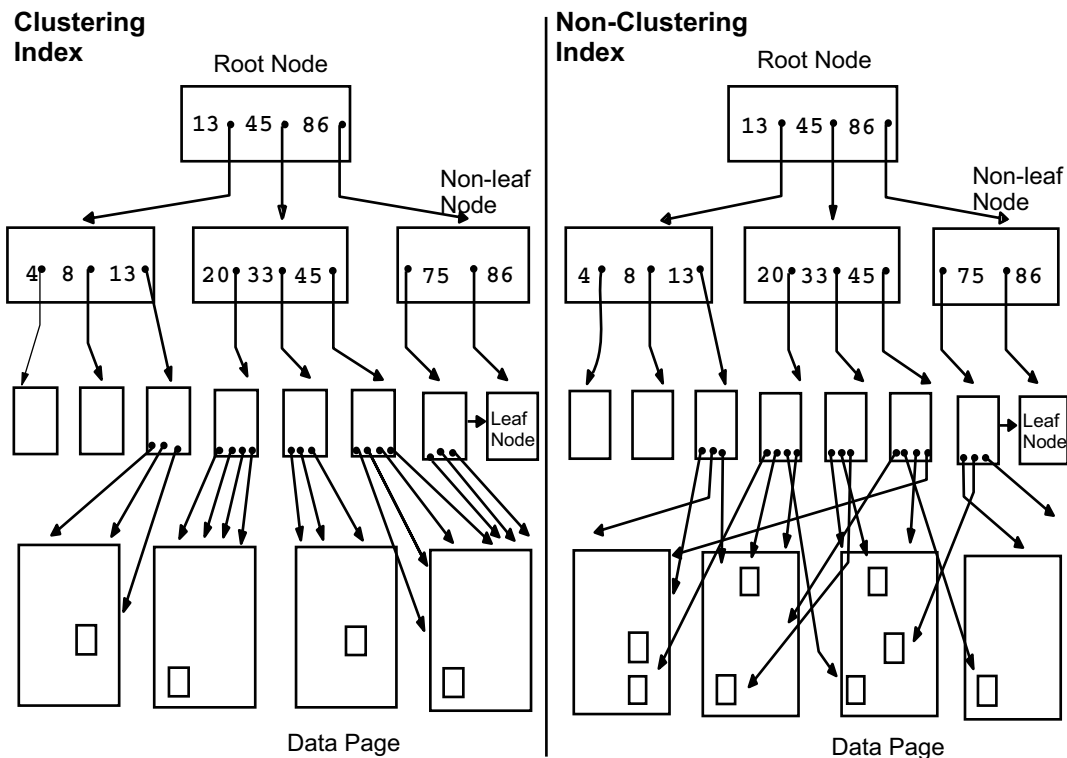
Use RESOLVE ANY option during rebind. Any of the functions and types in the SQL path are considered for function and type resolution. Conservative binding semantics are not used.

#### Usage notes

This command uses the CLP REBIND command to attempt the revalidation of all packages in the database.

**Transition Statement** — Let's examine the structure of indexes and why clustering may be a critical performance factor for some applications.

# Clustered and non-clustered indexes



© Copyright IBM Corporation 2007

Figure 10-32. Clustered and non-clustered indexes

CF238.3

## Notes:

An index is structured as a B+ tree. This structure enables the database manager to find values in the index rapidly.

The root node is at the top of this structure. In the illustration, the root node would contain an entry for each non-leaf, or intermediate node. The entry in the root node consists of the high value contained on the intermediate node and a pointer to this node.

The intermediate nodes are similar in structure to the root node, except that the range of values addressed is more specific. The intermediate node contains an entry for each of the leaf nodes addressed. The entry consists of the high value contained on the leaf node and a pointer to this leaf node.

The leaf nodes contain the value/rid pairs themselves. The leaf nodes collectively address the entire table.

The visual above is only one example of index structure. It is possible that an index contains more intermediate levels. Such an index would be required if the number of leaf nodes could not be addressed by one level of intermediate nodes.

Some insight regarding the structure of an actual index can be obtained by examining SYSCAT.INDEXES. The column NLEVELS indicates the number of root, intermediate, and leaf node levels. The column NLEAF indicates the number of leaf pages.

The degree to which the data rows correspond in order to the values on the leaf nodes is regarded as the clustering of the data. Therefore, the concept of clustering is only applicable when discussing a table and a given index.

The index on the left side of the visual illustrates a table clustered by an index, because the rows in the data pages are physically ordered according to the index sequence. The table on the right is not clustered by an index. The rows in the data pages do not seem to correspond to the order of the index entries.

During REORG, an index can be identified by which to cluster the data. Clustering benefits range type searches, such as BETWEEN or EQUAL for a non-unique key. This is due to a reduction in I/O costs. If the name of an index is not specified on the REORG command, and a clustering index exists, then the data will be ordered according to the clustering index.

INSERT, UPDATE, and DELETE activity can change the degree of clustering. For a clustering index, new rows are inserted physically close to rows where the key values of clustering index are in the same range. The CLUSTERRATIO or CLUSTERFACTOR columns of SYSCAT.INDEXES will indicate the degree of clustering for each index defined on the table.

## ***Instructor Notes:***

**Purpose** — Define the concept of clustering data and the general structure of an index.

**Details** — Start by presenting just the index tree structure component for the clustered index. Proceed through a traversal of the index tree starting at the root, going to the non-leaf node and getting to the leaf node. A *good* value to illustrate the concept is 74. Based on the high key orientation of the index, the root would indicate that the non-leaf page to access next would be the *last* page in the example, since the target value of 74 is greater than the high key pointer value of 45, but less than the value of 86. The pointer associated with the value of 75 is then traversed, since it is higher than the target value of 74. Therefore, the leaf page that contains any values of 74, and pointers to the data pages, would be found on the page identified by the non-leaf pointer labelled 75. Note that the number of entries found could be zero, one, or many: depending on whether there were qualifying rows.

You can test the students' understanding of the index tree structure concept by asking them what the value associated with the leftmost pointer in the root page should be. (It is not shown on the visual.) The correct answer should be 13, since this is the high key value on the non-leaf page addressed by this pointer.

You may wish to define the concepts of NLEAF and NLEVELS. In the index structure shown, NLEAF would be 8 and NLEVELS would be 3. (Of course, the NLEAF value in a real example would likely be much higher for a 3-level index.)

After you are satisfied that the students understand the basic concept of the index structure, you can address the concept of clustering. Emphasize, by comparing and contrasting the two index structures illustrated, the benefit of clustering for range processing.

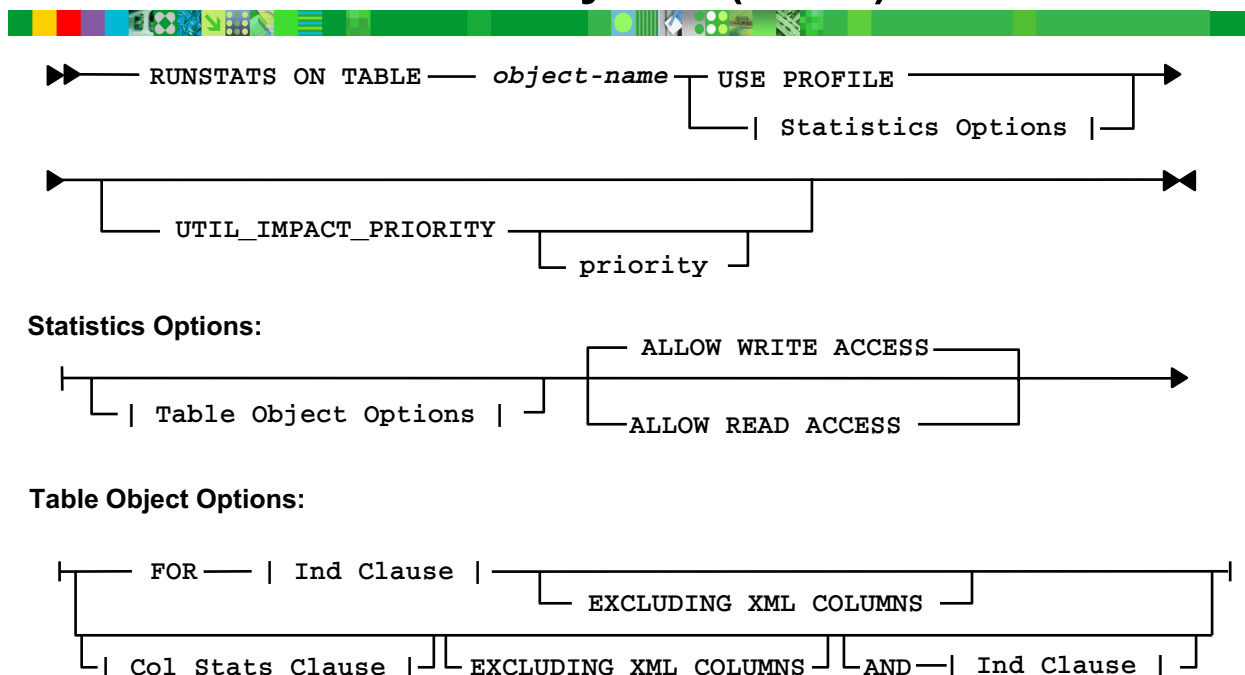
**Additional Information** — There are forward pointers between the index pages so that once the starting point for a range predicate is found, the leaf pages can be scanned without traversing the tree. There are no backward pointers.

There is an additional concept of Multi Dimensional Clustered tables, introduced in V8. These tables are addressed in CF45, *DB2 Advanced Administration*.

**Transition Statement** — In order to determine the degree to which data is clustered, as well as many other facts needed by the optimizer, RUNSTATS needs to be executed.



## RUNSTATS command syntax (basic)



See [Command Reference](#) for more detailed RUNSTATS information.

© Copyright IBM Corporation 2007

Figure 10-33. RUNSTATS command syntax (basic)

CF238.3

### Notes:

You must have SYSADM, SYSCTRL, SYSMANT, DBADM, or CONTROL privilege on the table to invoke RUNSTATS.

**If no options are specified, only basic table statistics will be updated, and other users will have access to the table while the statistics are being gathered.**

The table name can be an alias or a fully qualified reference of the form ***schema.tablename***.

*FOR Index Clause* indicates to update statistics for indexes only. If no table statistics have been previously collected on the table, basic table statistics are also collected. *AND Index Clause* indicates to update statistics for the indexes as well as the table.

Index Clause allows you to identify one, multiple, or ALL indexes on the table. The fully qualified name in the form of schema.index-name must be used.

DETAILED can be specified to calculate extended index statistics. DETAILED requests that CLUSTERFACTOR be determined as well as PAGE\_FETCH\_PAIRS. CLUSTERRATIO will not be determined. The CLUSTERFACTOR and PAGE\_FETCH\_PAIRS statistics in

combination reflect the clustering of the table to the index with a greater degree of accuracy than CLUSTERRATIO.

Using options with the Column Stats Clause, you can specify that statistics collection should be done on some columns but not on others. Columns can be specified either for basic statistics collection (on-cols-clause) or in conjunction with the WITH DISTRIBUTION clause (on-dist-cols-clause). The number of values collected is determined by the **numfreqval** database configuration parameter. The number of quantiles collected is determined by the **numquantiles** database configuration parameter. These statistics can be viewed through SYSCAT.COLDIST.

Distribution statistics are useful dynamic SQL or static SQL applications that do not use host variables, and where the SQL is accessing tables that lack relative uniformity in the data. Static SQL using host variables, or dynamic SQL using parameter markers, do not use distribution statistics.

ALLOW WRITE ACCESS specifies that other users can have read- and write-only access to the table while the statistics are being gathered. This is the default.

ALLOW READ ACCESS specifies that other users can have read-only access and write to the table while the statistics are being gathered.

If a table has been modified many times (lots of inserts, deletes, or updates) or has been reorganized with a REORG TABLE, it is a candidate for RUNSTATS.

If an application is performing poorly, it may be advantageous to REBIND the application **after** invoking RUNSTATS.

The syntax below gathers table statistics with distribution and detailed index statistics. (This is an example of gathering the greatest possible amount of statistics.)

```
RUNSTATS ON TABLE schema.tablename  
WITH DISTRIBUTION  
AND DETAILED INDEXES ALL
```

RUNSTATS can be invoked from the Control Center:

- Right-click and select table-name from the *Contents* pane.
- Select **Run Statistics...** from the pop-up menu.
- Choose the options for table, indexes and share level desired from the Run Statistics Window.
- Click **OK** to start the command.

If you want to see or save the command, or both:

- Select **Show Command** after choosing the options.

There is also another possibility to let DB2 do the runstats for you. In the Database Configuration file is a parameter called AUTO\_RUNSTATS which is OFF by default. Setting this parameter to ON enables the automatic table runstats. A runstats policy (a defined set of rules or guidelines) can be used to specify the automatic behavior. To be able to use this option, you must turn the AUTO\_MAINT parameter to ON.

All automatic maintenance features can also be configured from the Control Center. There you select (highlight) the database, right-click and select **Configure Automatic Maintenance**.

## ***Instructor Notes:***

**Purpose** — Show syntax and options for RUNSTATS.

**Details** — RUNSTATS should be executed on a periodic basis since the statistics updated by this facility are crucial to the optimizer's success in developing access strategies. Distribution and detailed index statistics may be justified on critical application tables, but are not likely required for every table in the system. However, some installations may use the most detailed form of RUNSTATS (shown in the example) unless they experience a performance or resource problem in doing so. Therefore, the level of statistics gathered could certainly be considered a tuning issue. Explain also the automatic capabilities in DB2 and the relevant DB CFG parameters, as well as the use of the appropriate GUIs.

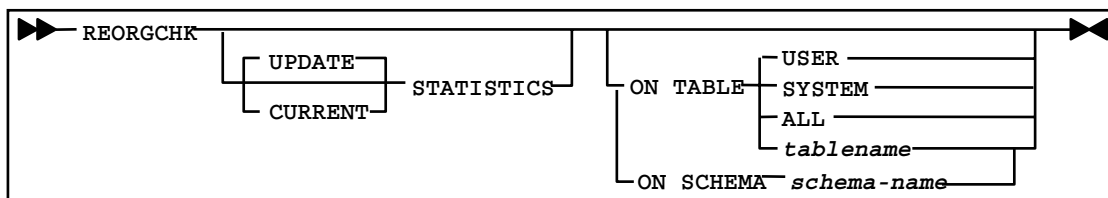
**Additional Information** — *Administration Guide*.

Certification objective: 5E Ability to use the REORG, REORGCHK, REBIND and RUNSTATS utilities.

**Transition Statement** — After gathering statistics about the current state of the database tables and indexes, it may be appropriate to determine if reorganization is desirable.

## Determine if table reorganization is required

- Default is to call RUNSTATS first



### Table Statistics

| SCHEMA.NAME       | CARD | OV | NP | FP | ACTBLK | TSIZE | F1 | F2 | F3  | REORG |
|-------------------|------|----|----|----|--------|-------|----|----|-----|-------|
| Table: INST01.ACT | 18   | 0  | 1  | 1  | -      | 648   | 0  | -  | 100 | ---   |

### INDEX Statistics

| SCHEMA.NAME          | INDCARD | LEAF | ELEAF | LVLS | NDEL | KEYS... | F4  | F5 | F6 | F7 | F8 | REORG |
|----------------------|---------|------|-------|------|------|---------|-----|----|----|----|----|-------|
| Table: INST01.ACT    |         |      |       |      |      |         |     |    |    |    |    |       |
| Index: INST01.PK_ACT | 18      | 1    | 0     | 1    | 0    | 18...   | 100 | -  | -  | 0  | 0  | ---   |
| Index: INST01.XACT2  | 18      | 1    | 0     | 1    | 0    | 18...   | 100 | -  | -  | 0  | 0  | ---   |

© Copyright IBM Corporation 2007

Figure 10-34. Determine if table reorganization is required

CF238.3

### Notes:

To invoke REORGCHK, the user must have SYSADM, SYSCTRL, SYSMANT, DBADM, or CONTROL privilege on the table.

**UPDATE STATISTICS** specifies to call the RUNSTATS routine to update table statistics and then use the updated statistics in the formula to determine if table reorganization is recommended. This is the default. **Running statistics on many tables can take time, especially if the tables are large.**

**CURRENT STATISTICS** specifies to use the current table statistics to determine if table reorganization is required.

**USER** specifies to check the tables that are under your current authorization ID. This is the default.

**ALL** specifies to check all user and system tables.

**SYSTEM** specifies to check the system tables only.

tablename can be explicitly provided in the form of an alias or a fully qualified reference of the form schema.tablename.

**SCHEMA** specifies a schema name to check all tables with that schema name.

**If a table or its indexes exceeds the bounds of the formulas, then the table is a candidate for reorganization (REORG).**

The Table Statistics Report headings mean:

- **CARD** - Number of rows in base table. In the example, the cardinality of the table is 1 million rows.
- **OV** - Number of OVERFLOW rows.
- **NP** - (NPAGES) number of pages that contain data.
- **FP** - (FPAGES) total number of pages.
- **TSIZE** - Table size in bytes. Calculated as the product of the number of rows in the table (CARD) and the average row length. The average row length is computed as the sum of the average column lengths (AVGCOLLEN in SYSCAT.COLUMNS) plus 10 bytes of row overhead. For long fields and LOBs, only the approximate length of the descriptor is used. The actual long field or LOB data is not counted in TSIZE. For example, on the graphic, the table size is indicated as "1.10e+008" or  $1.10 \times 10^8$ , or 110,000,000 bytes (about 110M).
- **F1, F2, F3** - Results of formulas 1, 2, and 3.
- **REORG** - Each hyphen (-) displayed in this column indicates that the calculated results were within the set bounds of the corresponding formula, and each asterisk (\*) indicates that the calculated results exceeded the set bounds of its corresponding formula. "-" or "\*" on the left side corresponds to F1 (Formula 1); "-" or "\*" in the middle of the column corresponds to F2 (Formula 2); "-" or "\*" on the right side of the column corresponds to F3 (Formula 3). Table reorganization is suggested when the results of the calculations exceed the bounds set by the formula.

The Index Statistics Report headings mean:

- **CARD** - Number of rows in base table.
- **LEAF** - Total number of index leaf pages.
- **ELEAF** - Number of pseudo empty index leaf pages. A pseudo empty index leaf page is a page on which all the RIDs are marked as deleted, but have not been physically removed.
- **LVLS** - (LEVELS) number of index levels.
- **ISIZE** - Index size, calculated from average column length for all columns participating in the index.
- **NDEL** - Number of pseudo deleted RIDs. A pseudo deleted RID is a RID that is marked deleted. This statistic reports pseudo deleted RIDs on leaf pages that are not pseudo empty. It does not include RIDs marked as deleted on leaf pages where all the RIDs are marked deleted.
- **KEYS** - (FULLKEYCARD) number of unique index entries. For example, on the graphic, the number of keys of ACCT\_GRP is indicated as 1000; the number of keys of ACCTINDX is indicated as "1e+006", or 1 million entries.
- **F4, F5, F6, F7, F8** - Results of formulas 4, 5, 6, 7, and 8.

- **REORG** - Each hyphen (-) displayed in this column indicates that the calculated results were within the set bounds of the corresponding formula, and each asterisk (\*) indicates that the calculated results exceeded the set bounds of its corresponding formula. "-" or "\*" on the left column corresponds to F4 (Formula 4); "-" or "\*" in the second from left column corresponds to F5 (Formula 5); "-" or "\*" in the middle column corresponds to F6 (Formula 6); "-" or "\*" in the second column from the right corresponds to F7 (Formula 7); "-" or "\*" on the right column corresponds to F8 (Formula 8). Index reorganization advice is as follows:
  - If the results of the calculations for Formula 1, 2, and 3 do not exceed the bounds set by the formula, and the results of calculations for Formula 4, 5, or 6 **do** exceed the bounds set, then index reorganization is recommended.
  - If only the results of the calculation Formula 7 exceed the bounds set, but the results of formulas 1, 2, 3, 4, 5, and 6 are within the set bounds, then cleanup of the indexes using the CLEANUP ONLY option of REORG INDEXES is recommended.
  - If the only calculation result to exceed the set bounds is that of Formula 8, then a cleanup of the pseudo empty pages of the indexes using the CLEANUP ONLY PAGES option of REORG INDEXES is recommended.

## ***Instructor Notes:***

**Purpose** — Show syntax of REORGCHK and format of output.

**Details** — REORGCHK is provided as an aid for evaluating statistics updated by RUNSTATS in order to determine if performance may benefit from REORG. The recommendations are simply guidelines. A given installation may have more stringent or lax guidelines. Guidelines may vary on a table by table basis.

**Additional Information** — *Command Reference*.

Certification objective: 5E Ability to use the REORG, REORGCHK, REBIND and RUNSTATS utilities.

**Transition Statement** — Let's examine the basic meaning of the formulas applied.



## REORGCHK statistics

### Table Statistics:

```
F1: 100 * Overflow / Card < 5%:
F2: 100 * Table Size / ((FPages - 1) *
    (TABLEPAGESIZE - 68)) > 70%
F3: 100 * NPages / FPages > 80%
```

### Index Statistics:

```
F4: Cluster Ratio or normalized Cluster Factor > 80%
F5: 100 * ( KEYS*(LEAF_RECSize+LEAF_RECSize_OVERHEAD)+
    (INDCARD-KEYS)*DUPKEYSIZE ) /
    ( (NLEAF NUM_EMPTY_LEAFS-1)*
    (INDEXPAGESIZE-LEAF_PAGE_OVERHEAD) ) >
    MIN(50, (100 - PCTFREE))
F6: ( 100-PCTFREE ) * ( (FLOOR((100 - LEVEL2PCTFREE) / 100 *
    (INDEXPAGESIZE - NLEAF_PAGE_OVERHEAD)/(NLEAF_RECSize +
    NLEAF_RECSize_OVERHEAD))) * (FLOOR((100 - MIN(10,
    LEVEL2PCTFREE)) / 100 * (INDEXPAGESIZE
    NLEAF_PAGE_OVERHEAD)/(NLEAF_RECSize +
    NLEAF_RECSize_OVERHEAD)) ** (NLEVELS - 3)) *
    (INDEXPAGESIZE
    LEAF_PAGE_OVERHEAD)) / (KEYS*(LEAF_RECSize+LEAF_RECSize_OVERHEAD) +
    (INDCARD - KEYS) * DUPKEYSIZE ) ) < 100
F7: 100 * (NUMRIDS_DELETED /
    (NUMRIDS_DELETED + CARD)) < 20
F8: 100 * (NUM_EMPTY_LEAFS/NLEAF) < 20
```

© Copyright IBM Corporation 2007

Figure 10-35. REORGCHK statistics

CF238.3

### Notes:

REORGCHK performs calculations on the statistics in order to determine if the tables and their corresponding indexes can benefit from being reorganized.

The following formulas are used **to analyze the physical location of rows and the size of the table**:

- **F1** looks at the number of overflow rows in the table. The number of overflow rows in the table should be less than 5 percent of the total number of rows. Overflow rows can be created when rows are updated and the new rows contain more bytes than the old ones (for example, when updating VARCHAR fields and making them longer), or when columns are added to existing tables.
- **F2** looks at the table size in bytes. The table size should be more than 68 percent of the total space allocated for the table. There should be less than 32 percent free space. The total space allocated for the table depends upon the page size of the table space in which the table resides (minus an overhead of 76 bytes). Because the last page allocated is usually not filled, 1 is subtracted from FPAGES.

- **For MDC tables:**

$$100 * TSIZE / ((ACTBLK - FULLKEYCARD) * EXTENTSIZE * (TABLEPAGESIZE - 68)) > 70$$

FULLKEYCARD represents the cardinality of the composite dimension index for the MDC table. Extent size is the number of pages per block. The formula checks if the table size in bytes is more than the 70 percent of the remaining blocks for a table after subtracting the minimum required number of blocks.

- **F3** looks at empty pages. The number of pages that contain no rows at all should be less than 20 percent of the total number of pages. Pages can become empty after rows are deleted.

- For MDC tables, the formula is:

$$100 * \text{activeblocks} / ((\text{fpages} / \text{ExtentSize}) - 1)$$

The following formulas are used **to analyze the relationship of the indexes to the table data**:

- **F4** looks at the cluster ratio of an index. The cluster ratio for the index by which you want the table data clustered should be greater than 80 percent. When multiple indexes are defined on a table, some of these indexes probably have a low cluster ratio. When looking at the output from REORGCHK, you want to check the value for F4 for the clustering index, or the index by which you want the data clustered. If the data is not well clustered according to the other indexes, this would not be a reason to perform a REORG.
- **F5** examines the empty space in an index. Less than 50 percent of the space reserved for index entries should be empty. This value is only checked when the number of leaf pages is greater than 1.
- **F6** looks at whether the index entries would fit into an index tree with one fewer level. This formula checks the ratio between the amount of space in an index tree that has one less level than the current tree, and the amount of space needed. If a tree with one less level could be created and still leave PCTFREE available, then a reorganization is recommended. The actual number of index entries should be more than 90% (or 100 - PCTFREE) of the number of entries an NLEVELS-1 index tree can handle (only checked if NLEVELS > 1).
- **F7** looks at how many pseudo deleted record identifiers there are on pages that also have valid record identifiers. The number of pseudo deleted RIDs on non-pseudo empty pages should be less than 20 percent. Entries are pseudo deleted in the index to minimize the requirement for next key locking to protect index entries.
- **F8** looks at the number of leaf pages that are empty due to pseudo deletes. The number of pseudo empty leaf pages should be less than 20 percent of the total number of leaf pages.

**Instructor Notes:**

**Purpose** — Highlight the basic meaning of the formulas and limits used by REORGCHK.

**Details** — Try to avoid getting into a *mathematical discussion* about the formulas presented. They are provided for students that may wish to apply more or less stringent formulas, perhaps even writing their own version of REORGCHK. The key is to state the general meaning of each formula.

Stress that results for very small tables will not be valid. For example, the results of the second formula may show poor space utilization if a table needed just over one page to support its data.

Certification objective: 5E Ability to use the REORG, REORGCHK, REBIND and RUNSTATS utilities.

**Transition Statement** — Assume that you ran REORGCHK and decided to REORG a table.

## Reorganize table / index command syntax

```

▶▶ REORG { TABLE table-name / Table Clause | INDEXES ALL FOR TABLE table-name / Index Clause | } ... ▶▶

```

### Table Clause:

```

/ INDEX index-name {
  ALLOW READ ACCESS
  ALLOW NO ACCESS
  INPLACE {
    ALLOW WRITE ACCESS
    ALLOW READ ACCESS
    STOP
    PAUSE
  }
  USE tblspace
  START
  RESUME
}
/

```

### Index Clause:

```

/ {
  ALLOW READ ACCESS
  ALLOW NO ACCESS
  WRITE
}
/

```

Consult the *Command Reference* for more detailed syntax for the REORG statement.

© Copyright IBM Corporation 2007

Figure 10-36. Reorganize table / index command syntax

CF238.3

### Notes:

You must have SYSADM, SYSCTRL, SYSMANT, DBADM, or CONTROL on table to REORG TABLE.

The REORG utility reorganizes a table or its indexes.

The table option reorganizes a table by reconstructing the rows to eliminate fragmented data, and by compacting information.

The index option reorganizes all indexes defined on a table by rebuilding the index data into unfragmented, physically contiguous pages.

The table to be reorganized can be specified through an alias or a fully qualified reference of the form *schema.tablename*. If you omit the schema name, the default schema is assumed.

Options on the command include the following:

- **INDEX *index-name*** specifies the index to use when reorganizing the table. If you do not specify the fully qualified name in the form: *schema.index-name*, the default schema is assumed. The schema is the user name under which the index was created. The

database manager uses the index to physically reorder the records in the table it is reorganizing.

For an inplace table reorg, if a clustering index is defined on the table and an index is specified, it must be a clustering index. If the inplace option is not specified, any index specified will be used. If you do not specify the name of an index, the records are reorganized without regard to order. If the table has a clustering index defined, however, and no index is specified, then the clustering index is used to cluster the table.

All of the indexes of the table are reorganized if the table is reorganized without the INPLACE option. None of the indexes of the table are reorganized if the INPLACE option is specified.

- **{ ALLOW READ ACCESS | ALLOW NO ACCESS }**

ALLOW READ ACCESS indicates that you want to allow only read access to the table during reorganization. This is the default behavior for reorganizations that do not specify INPLACE.

ALLOW NO ACCESS indicates that no one will be allowed access to the table during reorganization.

- **USE *tblspace*** specifies the name of a system temporary table space in which to store a temporary copy of the table being reorganized. If you do not provide a table space name, the database manager stores a working copy of the table in the table spaces that contain the table being reorganized.
- **INPLACE** reorganizes the table while permitting user access. The table is reorganized within its existing table space, using only a minimum amount of additional space to accomplish the reorganization. No index reorganization is accomplished during an INPLACE table reorg.

- **{ ALLOW WRITE ACCESS | ALLOW READ ACCESS }**

ALLOW WRITE ACCESS indicates that you want to allow write access to the table during reorganization. This is the default behavior for an INPLACE reorganization.

ALLOW READ ACCESS indicates that you want to allow only read access to the table during reorganization.

- **{ START | RESUME }**

START starts the inplace REORG processing. Because this is the default, this keyword is optional.

RESTART continues or resumes a previously paused inplace table reorganization.

- **{ STOP | PAUSE }**

STOP stops the inplace REORG processing at its current point.

PAUSE suspends or pauses the inplace REORG for the time being.

When the inplace reorganization is paused or stopped, the part of the table that has been processed to that point will have the benefits of a reorganized table (elimination of

fragmentation and compaction of data. A paused inplace reorganization can be resumed. A stopped inplace reorganization would need to be started again from the beginning to reorganize the entire table. You can pause an inplace reorganization for several reasons. For example, you may want to allow unimpeded use of the table by your online jobs (removing the overhead of the reorg from the system); you can then resume the utility when activity on the system is less intense. You may also pause an inplace reorganization as you prepare to stop the instance; the utility then can be resumed after the instance is restarted.

Use REORGCHK to determine if a table requires REORG.

After REORG, use RUNSTATS to update the table statistics.

If you have turned the Database Configuration Parameter AUTO\_AMIN to ON you can also use the automatic REORG parameter AUTO\_REORG and set it to on. This enables automatic Table and Index reorganization for the database. A reorganization policy (a defined set of rules or guidelines) can be used to specify the automatic behavior.

The automatic REORG function includes policies and includes things like:

- A state-based collection health indicator in the health monitor to surface any reorganization action that cannot be handled by the policy
- Notification of the identification of tables requiring manual reorganization
- Knowledge in DB2 to automatically identify the need for reorganization of a table
- Knowledge in DB2 to determine the type of reorganization required and when to run it
- Scheduling of reorganization action based on specifications

All this can also be configured using the Control Center GUI (Configure Automatic Maintenance) when selecting the database and use the right mouse button.

## References

*Command Reference*

**Instructor Notes:**

**Purpose** — Introduce the syntax and need for REORG.

**Details** — The REORG command has the ability to be run in place; this accomplishes a *trickle* reorganization of the table, but does not touch the indexes. You must separately run a REORG INDEXES command to reorganize the indexes. Also here make the students to be aware of the possibility of automatic maintenance.

**Additional Information** — Certification objective: 5E Ability to use the REORG, REORGCHK, REBIND and RUNSTATS utilities.

**Transition Statement** — If you have a need to prototype a non-existing database condition, you can use the statistical catalog views.

## Statistical Catalog views



- **SYSSTAT.**
  - COLDIST
  - COLUMNS
  - COLGROUPS
  - COLGROUPEDIST
  - COLGROUPEDISTCOUNTS
  - INDEXES
  - ROUTINES (formerly FUNCTIONS)
  - TABLES

© Copyright IBM Corporation 2007

Figure 10-37. Statistical Catalog views

CF238.3

### Notes:

The catalog views are defined so that statistics used by the optimizer can be updated through the view. An object will only meet the view definition for a given user if the user created the object, has control of the object, or holds the explicit DBADM authority.

With the exception of the ROUTINES view statistics, the normal technique to update statistics is to execute RUNSTATS. However, the updatable catalog views may be helpful in prototyping or problem determination.

COLDIST contains distribution statistics for the **n** most frequently occurring values within a column.

COLGROUPS contains information regarding the cardinality of groups of columns.

COLUMNS contains information concerning cardinality of the column as well as the range of values in the column.

INDEXES contains information regarding the number of levels and number of leaf pages in the index, the clustering of the index, distinct values within the index, and scan estimates for the index with varying size buffer pools.



ROUTINES contains information regarding cost estimates of User-Defined Functions, Stored Procedures, and Methods. I/O costs and CPU instructions are estimated in various classifications. These statistics are NOT updated by RUNSTATS and must be provided by the user if desired.

TABLES contains information regarding the number of rows in the table, the page utilization of the table, and the number of rows that have been relocated due to update activity.

You can run db2look or Generate DDL... from the Control Center to get a report on a table's statistics. From the Control Center:

- Right-click and select table-name from the *Contents* pane.
- Select **Generate DDL...** from the pop-up menu.
- Choose **Database statistics** and deselect other choices.
- Select **Generate to run the command**.
- Check the output in the Journal.

Consult the *Administration Guide* for details.

## References

*Administration Guide*

## ***Instructor Notes:***

**Purpose** — Identify the existence of updatable catalog views.

**Details** — It is not desirable to list all the columns within each of these view. It is desirable to identify, at a high level, the type of information that can be changed to influence optimization.

**Additional Information** — Advise students about performance courses which go into greater detail.

**Transition Statement** — Let's see how you would use the db2look tool in mimic mode, to create similar databases.

## The db2look mimic mode

- Using db2look to mimic the tables in a database
  - To extract the DDL for the tables in the database, use the **-e** option.
  - For example, create a copy of the SAMPLE database called SAMPLE2 such that all of the objects in the first database are created in the new database:

```
C:\>db2 create database sample2 DB20000I
The CREATE DATABASE command completed successfully.
C:\>db2look -d sample -e > sample.ddl
-- USER is:
-- Creating DDL for table(s)
-- Binding package automatically ...
-- Bind is successful
-- Binding package automatically ...
-- Bind is successful
```

- Edit the `sample.ddl` file
- Once you have changed the connect statement, execute the statements, as follows:

```
C:\>db2 -tvf sample.ddl > sample2.out
```

© Copyright IBM Corporation 2007

Figure 10-38. The db2look mimic mode

CF238.3

### Notes:

There are many times when it is advantageous to be able to create a database that is similar in structure to another database. For example, rather than testing out new applications or recovery plans on a production system, it makes more sense to create a test system that is similar in structure and data, and to then do the tests against it instead. This way, the production system will not be affected by the adverse performance impact of the tests or by the accidental destruction of data by an errant application. Also, when you are investigating a problem (such as invalid results, performance issues, and so on), it may be easier to debug the problem on a test system that is identical to the production system.

You can use the db2look tool to extract the required DDL statements needed to reproduce the database objects of one database in another database. The tool can also generate the required SQL statements needed to replicate the statistics from the one database to the other, as well as the statements needed to replicate the database configuration, database manager configuration, and registry variables. This is important because the new database may not contain the exact same set of data as the original database but you may still want the same access plans chosen for the two systems.

The db2look tool is described in detail in the *DB2 Command Reference*, but you can view the list of options by executing the tool without any parameters. A more detailed usage can be displayed using the -h option.



#### Note

If you want the DDL for the user-defined spaces, database partition groups and buffer pools to be produced as well, add the "-l" flag after "-e" in the command above. The default database partition groups, buffer pools, and table spaces will not be extracted. This is because they already exist in every database by default. If you wish to mimic these, you must alter them yourself manually.

### Mimicking statistics for tables

If the intent of the test database is to do performance testing or to debug a performance problem, it is essential that access plans generated for both databases are identical. The optimizer generates access plans based on statistics, configuration parameters, registry variables, and environment variables. If these things are identical between the two systems then it is very likely that the access plans will be the same.

### Extracting configuration parameters and environment variables

The optimizer chooses plans based on statistics, configuration parameters, registry variables, and environment variables. As with the statistics, db2look can be used to generate the necessary configuration update and set statements. This is done using the -f option.



#### Note

Only those parameters and variables that affect DB2 compiler will be included. If a registry variable that affects the compiler is set to its default value, it will not show up under "Environment Variables settings".

***Instructor notes:***

**Purpose** — Expound on the reasons you may want to *clone* a database, either in the same instance or another instance.

**Details** —

**Additional information** — Certification objective: 5F Ability to use DB2Move, DB2Look and DB2Batch.

**Transition statement** — If you need additional study in advanced administration topics.



## 10.7 Additional learning opportunities

### Instructor topic introduction

**What students will do** — Checkpoint exercises to determine the correct application alternative to use.

**How students will do it** — Listen to lecture.

**What students will learn** — What alternatives there are in coding applications.

**How this will help students on their job** — A database administrator is frequently called upon to advise application programmers concerning solutions to various business problems. Although the DBA may not need to implement a given solution, the DBA should be aware of the strengths and weaknesses of each alternative.

## Need more information? (1 of 3)

- CF46 — *DB2 for Linux, UNIX, and Windows Advanced Database Administration for Experts* (five days with labs)

- CF41 — *DB2 for Linux, UNIX, and Windows Performance Tuning and Monitoring Workshop* (four days with labs)



© Copyright IBM Corporation 2007

Figure 10-39. Need more information? (1 of 3)

CF238.3

### Notes:

#### ***DB2 for Linux, UNIX, and Windows Advanced Database Administration for Experts - CF46***

Perform advanced database administration tasks using DB2. These tasks include advanced load and utility issues, parallelism and Symmetric Multiprocessor (SMP) exploitation, DB2 Governor, job scheduling, job log, scripting, distributed data management, remote administration, advanced monitoring using the snapshot table functions, and federated database exploitation.

#### ***DB2 for Linux, UNIX, and Windows Performance Tuning and Monitoring Workshop - CF41***

Learn how to tune for optimum performance the IBM DB2 for Linux, UNIX, and Windows relational database management system and associated applications written for this environment. Learn about DB2 for Linux, UNIX, and Windows on a serial processor, in a nonpartitioned database environment. Explore performance issues affecting the design of the database and applications using the database, the major database performance parameters, and the different tools that assist in performance monitoring and tuning.



***Instructor Notes:***

**Purpose** — Suggest follow-on education.

**Transition Statement** — Let's look at some programming education available to you.

## More information (2 of 3)

- 
- Programming education
    - CF10 — *DB2 for Linux, UNIX, and Windows Database Programming Workshop*
      - Two days with labs
      - Static SQL
    - CF11 — *DB2 for Linux, UNIX, and Windows Database Advanced Programming*
      - Three days with labs
      - Dynamic SQL, Stored Procedures, APIs, UDTs, UDFs
    - CG11 — *DB2 for Linux, UNIX and Windows Programming Using Java*
      - Four days with labs
      - SQLJ, Stored Procedures, JDBC, UDTs, UDFs
  - Product documentation

© Copyright IBM Corporation 2007

Figure 10-40. More information (2 of 3)

CF238.3

### Notes:

You, or other people in your organization, may desire programming details regarding the use of the application alternatives described in this unit.

### ***DB2 for Linux, UNIX, and Windows Database Programming Workshop - CF10***

Learn to produce application programs that manipulate DB2 databases in a Linux, UNIX, or Windows environment. Emphasis is on embedding SQL statements and preparing programs for execution.

### Objectives:

- Connect to local and remote DB2 databases
- Incorporate static SQL statements in an application program
- Prepare the program for execution
- Discuss relational concepts
- Describe database locking from an application programming perspective
- Discuss program and DB2 options relative to performance of static SQL

***DB2 for Linux, UNIX, and Windows Database Advanced Programming - CF11***

Learn how to take advantage of advanced programming techniques to access DB2 databases in the workstation environment.

**Objectives:**

- Code programs to manipulate Large Objects (LOB)
- Code dynamic SQL applications
- Code stored procedures and call such procedures
- Provide programming support for user defined functions
- Use product defined Application Programming Interfaces (API)
- Code simple Call Level Interface (CLI) programs
- Identify factors that influence application performance of alternatives to static SQL

***DB2 for Linux, UNIX and Windows Programming Using Java - CG11***

Develop the skills you need to produce application programs in Java that manipulate DB2 databases. Learn how to use Java Database Connectivity (JDBC) to access data via SQL; perform updates, inserts, and deletes; and process data returned via SELECTs with result-set processing. Also, learn to use SQL for Java (SQLJ) to embed SQL statements into Java programs and prepare SQLJ programs for execution. Learn how to take advantage of advanced programming techniques to access DB2. Use DB2 for Windows in the labs, but the concepts presented apply to programming in Java for any DB2 database.

**Objectives:**

- Incorporate JDBC statements into an application program
- Connect to local and remote DB2 databases
- Discuss relational concepts
- Describe database locking from an application programming perspective
- Use object-relational capabilities of DB2, such as Large Object (LOB) manipulation, user-defined functions, and user-defined distinct types
- Code stored procedures and call such procedures
- Incorporate static SQL statements into an application program
- Prepare a program for execution
- Discuss program and DB2 options relative to performance of programs accessing DB2
- Identify tools that exist for monitoring the performance of the DB2 database and applications

## ***Instructor Notes:***

**Purpose** — Make students aware of formal education available to obtain detailed knowledge of programming interfaces.

**Details** — Provide insight concerning the value of the programming education.

**Transition Statement** — Let's look at some more opportunities in this area.

## More information (3 of 3)

- CF12 — *DB2 SQL Workshop*
  - Two days with labs
  - Basic SQL
- CF13 — *DB2 SQL for Experienced Users*
  - Two days with labs
  - DDL, complex selects
- CF71 — *DB2 Stored Procedures Programming Workshop*
  - Two days with labs
  - Stored Procedure Programming

© Copyright IBM Corporation 2007

Figure 10-41. More information (3 of 3)

CF238.3

### Notes:

#### **DB2 SQL Workshop - CF12**

Learn about the Structured Query Language (SQL) and how it applies to the entire DB2 Family. This course is appropriate for individuals working in an OS/390, VM/VSE, AIX, Windows, or OS/2 environment. Complete extensive hands-on exercises to develop your skills to use the SQL you learn. Perform these exercises either by using the DB2 for UNIX, Windows, and OS/2 product in a Windows environment, or by using the DB2 for z/OS and OS/390, Version 7, product through a Time Sharing Option (TSO) interface to a z/OS system.

#### **Objectives:**

- Use (write) SELECT, INSERT, UPDATE, and DELETE statements
- Use simple CREATE TABLE and CREATE VIEW statements

#### **DB2 SQL for Experienced Users - CF13**

Learn how to code simple Data Definition Language (DDL) statements and complex SELECT statements. Learn to use outer joins, complex subqueries, recursive SQL, and

table expressions. Also, learn about features, such as triggers, user-defined types, and user-defined functions.

**Objectives:**

- Use advanced SQL constructs, such as recursive SQL, case expressions, check constraints, and triggers
- Discuss basic relational database concepts, such as referential integrity, tables, and indexes
- Create tables and indexes
- Use outer joins
- Use complex subqueries
- Use the major scalar functions
- Use views

***DB2 Stored Procedures Programming Workshop - CF71***

Learn how to recognize when an IBM DB2 stored procedure is the correct solution for an application, how to use the Stored Procedure Builder, how to CALL a stored procedure, how to create a stored procedure using Data Definition Language (DDL), and how to troubleshoot a stored procedure. This course includes lectures and machine lab exercises. The exercises run on a Windows NT workstation, but are applicable to all IBM DB2 platforms.

**Objectives:**

- Describe a stored procedure and justify its use in an application
- Understand the Stored Procedure Builder and its capabilities
- Describe the DB2 SQL Procedure Language (SQL PL) statements and how to use them in an application
- Describe the basic structure of a Java application using stored procedures
- Create a DB2 stored procedure using the SQL DDL statement CREATE PROCEDURE
- Describe troubleshooting approaches for stored procedures

***Instructor Notes:***

**Purpose** — Make students aware of formal education available to obtain detailed knowledge of programming interfaces.

**Details** — Provide insight concerning the value of the programming education.

**Transition Statement** — End of unit.





# Checkpoint

## Exercise — Unit Checkpoint

- \_\_\_ 1. You are the only DBA permitted to create new packages on the production system. You are called at 2:00 a.m. by a member of the operations staff, who informs you that an application is ending with an SQLCODE of -805. What is your response?

---

**Analyze the function of the new application and BIND the new package if justified.**

**Ref:** Since it is given that the student is the only DBA with the authority to add new packages to the system, it follows that some degree of security is implied. The -805 code indicates that the package does not exist. Therefore, a new application is being put into production or the package for an existing application has been dropped. In either case, examining the function of the application is warranted.

- \_\_\_ 2. Assume that there is another DBA with the authority to BIND any existing application. Three nights after your previous call from the operations staff, you receive another call. This time, the SQLCODE is a -818. What would you advise the DBA on duty to do?

---

**Precompile, Bind, Compile, and Link the application.**

**Ref:** The -818 indicates a consistency token (timestamp) mismatch between the executable and the package. In this particular case, several responses are valid. If the DBA has the capability to *trace* the preparation steps that have been completed, it may be possible to determine the specific steps required to resolve the problem. For example, perhaps all that is required is to bind the package from a bind file. The answer provided assumes that the *trace* capability does not exist and also favors the *easy* solution.

- \_\_\_ 3. Chris is a programmer who will automate the creation of backups for your DB2 system. In a meeting to determine the proper interface, she asks for your advice on how to proceed. What do you tell her?

---

**Use an API.**

- \_\_\_ 4. Mike is a programmer who will write an application to support part of a point-of-sales application. His program will need to check on the credit history of customers attempting to use a credit card. The input to the program is the credit card number. The credit history is in a 40-million row DB2 table, which has an index by credit card number. What interface would you suggest?
-

### Embedded Static SQL.

**Ref:** Also, it is assumed that this transaction will be requested on a repetitive basis and that dynamic bind would not be desirable.

- \_\_\_ 5. Your consulting services are requested at a planning meeting of a major software vendor. The software product being discussed must be able to perform ad hoc queries against a number of different database products. In order to prevent having to provide specific code for each database product supported, you contribute a suggestion. What is it?
- 

### Use CLI.

**Ref:** Also, a student may also suggest ODBC. This is a valid answer as well, but is targeted by the next question.

- \_\_\_ 6. The software application described in the prior question is going to be implemented using Microsoft Windows. What architecture might you recommend?
- 

### ODBC

**Ref:** Also, ODBC is not limited only to the Windows environment, but historically has been a solution popular with Windows developers. Visigenic Software Inc. provides an ODBC Software Development Kit for non-Microsoft platforms. See the *Call Level Interface Guide* for details.

- \_\_\_ 7. As the database administrator, you have been constantly requested to supply samples of production data for a test machine. The test machine has enough security so that the content of the data can be a sampling of actual production data; however, only 5% of the data needs to be periodically sampled for the test environment. In the past, you have manually retrieved the sample and would like to eliminate this task from your busy schedule. You would like staff leaders from the programming group to be able to request a random 5% sampling of any table for which they have SELECT authority. The application supporting this function would sample the data and optionally transmit the data to a target machine. How would you suggest implementing this application?
- 

### Use a stored procedure and dynamic SQL.

**Ref:** Since sampling the data would require server-intensive activity, the stored procedure approach is desired.

- \_\_\_ 8. A programmer is writing an application against a DB2 database. Some of the columns desired and the predicate is not totally known until execution time. What would you advise the programmer to use?
-

---

## Dynamic SQL

- \_\_\_ 9. You are attempting to determine the best cluster sequence for a large customer table. During your initial analysis, you have determined that online access to the table is generally via a unique CUSTNUMB. You also have many *batch* applications that require data to be sorted on LASTNAME. There are indexes on both of these columns. Given these brief requirements, what would you choose as the index to specify when performing a REORG, if either?

---

### The index on LASTNAME.

**Ref:** Students may debate the correct answer to this question based on assumptions they may make concerning the access to this table. The response given reflects most accesses to data via unique keys. Clustering the data by such a key does not improve performance because access tends to be via equal predicates. There doesn't tend to be an inherent relationship between rows with *adjacent* values for the unique key. (Of course, there are exceptions to this general rule.)

The LASTNAME column has been indicated as one necessary to support ordering requirements, which in turn indicates that more than one data row will tend to be accessed. If the data is in cluster sequence according to this column, DB2 may choose to use the index to avoid actual sorts and also limit the I/O required to retrieve the data.

- \_\_\_ 10. The table analyzed for the prior question is examined more closely. Many online applications examine the data according to a range of ENTRY\_DATE values. There is an index on ENTRY\_DATE. Would you change your choice of cluster?

---

### Yes. Change the clustering according to the index on ENTRY\_DATE.

**Ref:** Supporting this answer is the assumption that MANY online applications access the data by a range of values against the ENTRY\_DATE column. By changing the clustering of the table, the online access for these applications could be improved by reducing the I/O requirements of access. Of course, the batch applications mentioned in the prior question that require sorting on LASTNAME may be negatively impacted. However, the DBA should generally favor online performance over batch applications. Certainly, there are exceptions to this general rule.

- \_\_\_ 11. An application programmer asks for your advice concerning a program that produces a simple business report from the table previously described. The program accesses the server from a client workstation and appears to run for longer than necessary. It simply FETCHES about 20% of the data and writes the rows to a report. EXPLAIN indicates that the index on ENTRY\_DATE is being used to support a range predicate in the query. The programmer assures you that the same

statement runs fine in CLP, but takes much longer inside the static program. What would be a PRECOMPILE / BIND parameter that you should examine as one, if not the first, possible cause of the problem?

---

**Blocking. Determine if it is enabled.**

**Ref:** Certainly, students may respond with other plausible causes of the problem. However, the symptoms described indicate a strong possibility that blocking (or lack of it) could be the problem.

- \_\_\_ 12. A programmer presents you with an application that has a very complex SQL statement that accesses many tables. Assume that the statement is logically correct and that your database design is proper. However, the programmer implies that the strategy externalized by EXPLAIN accesses the *wrong* table first. Assume that you have plenty of time and CPU for program preparation. What PRECOMPILE / BIND parameter might you experiment with to influence DB2?
- 

**QUERYOPT**

**Ref:** A basic assumption for this answer is that the installation does not always use a QUERYOPT level that would cause DB2 to consider all possible access strategies.

- \_\_\_ 13. You have just finished analyzing the SQL in a package that has been identified as a performance problem. You determined that an index to support the application was required, so you created the index. Assume that the new index was also based on the columns of the table by which the data should be clustered. What are your next three steps required for the package to take advantage of the new index?
- 
- 
- 

**REORG  
RUNSTATS  
REBIND**

**Ref:** The REORG puts the table in the correct sequence according to the key specified, which is assumed to be the new index. The RUNSTATS command would gather the statistics for the new index and physical structure of the data, which is required for the REBIND to function properly.

## Unit summary

Having completed this unit, you should be able to:

- Prepare applications that access DB2 data for execution
- Use PRECOMPILE and BIND options that are appropriate for specific application requirements
- Describe the application alternatives available to access DB2 data or request other DB2 functions
- List reasons for REBIND of application programs and execute this command
- Define the concept of single dimensional clustering, and determine the proper cluster sequence, if any, for DB2 tables
- Use RUNSTATS, REORGCHK, and REORG to enhance application performance

© Copyright IBM Corporation 2007

Figure 10-42. Unit summary

CF238.3

### Notes:

***Instructor notes:***

**Purpose** — Review the unit objectives.

**Details** — Question the students on what they've learned.

**Transition Statement** — Now, let's look at the lab on Performance and Applications.

# Unit 11. Security

## Estimated time

00:45

## What this unit is about

This unit provides information about controlling access to DB2 objects through privileges using either explicit or implicit grants to group IDs and user IDs.

## What you should be able to do

After completing this unit, you should be able to:

- Use DB2 access control mechanisms to implement security within the database
- Use group IDs to create a control hierarchy
- Describe Label Based Access Control (LBAC)
- Describe privileges within a database
- Describe privileges required for binding and executing a package
- Describe the difference between explicit privileges and implicit privileges
- Describe the different DB2 authorization levels

## How you will check your progress

Accountability:

- Checkpoint
- Machine exercises

## References

*Command Reference*

*Administration Guide*

## Unit objectives

After completing this unit, you should be able to:

- Differentiate between authentication and authorization
- Use DB2 access control mechanisms to implement security within the database
- Use group IDs to create a control hierarchy
- Describe Label Based Access Control (LBAC)
- Explore privileges in the database
- Describe privileges required for binding and executing a package
- Describe the difference between explicit privileges and implicit privileges
- Describe the different DB2 authorization levels

© Copyright IBM Corporation 2007

Figure 11-1. Unit objectives

CF238.3

### Notes:



***Instructor notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Level set.

**Transition Statement** — DB2 security is implemented through authentication and privileges. Let's look at these two ways of keeping your data secure.



## 11.1 Authentication versus authorization

### Instructor topic introduction

**What students will do** — Learn the differences between authentication and authorization.

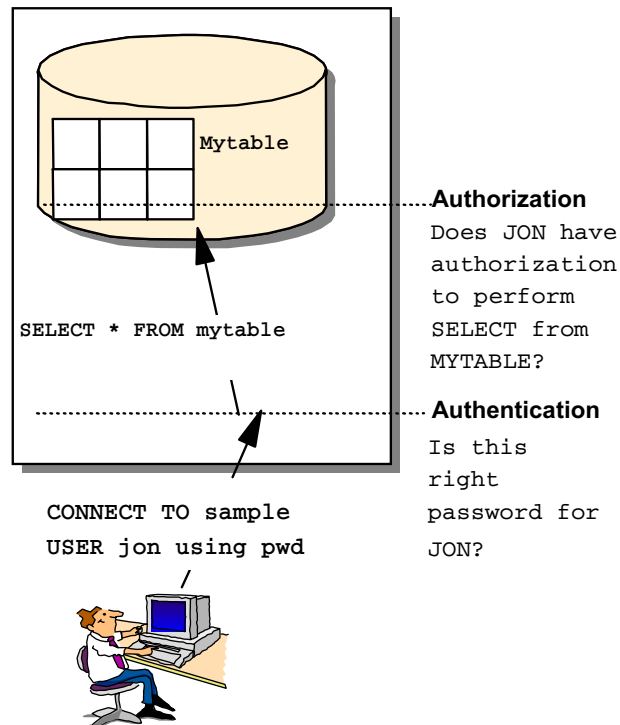
**How students will do it** — Listen to the lecture.

**What students will learn** — The differences between authentication and authorization.

**How this will help students on their job** —

# Authentication versus authorization

- DB2 uses a combination of:
  - External security service
  - Internal access control information
- Authentication
  - Identify the user
    - Check entered username and password
- Done by security facility outside of DB2
- Authorization
  - Check if authenticated user may perform requested operation
  - Done by DB2 facilities
    - Information stored in DB2 catalog, DBM configuration file



© Copyright IBM Corporation 2007

Figure 11-2. Authentication versus authorization

CF238.3

## Notes:

Access to an instance or a database first requires that the user be *authenticated*. The *authentication type* for each instance determines how and where a user will be verified. The authentication type is stored in the database manager configuration file at the server. It is initially set when the instance is created. There is one authentication type per instance, which covers access to that database server and all the databases under its control.

The following authentication types are provided:

**SERVER** — Specifies that authentication occurs on the server using local operating system security. If a user ID and password are specified during the connection or attachment attempt, they are compared to the valid user ID and password combinations at the server to determine if the user is permitted to access the instance. This is the default security mechanism.

**Note**

The server code detects whether a connection is local or remote. For local connections, when authentication is SERVER, a user ID and password are not required for authentication to be successful.

**SERVER\_ENCRYPT** — Specifies that the server accepts encrypted SERVER authentication schemes. If the client authentication is not specified, the client is authenticated using the method selected at the server.

If the client authentication is SERVER, the client is authenticated by passing the user ID and password to the server. If the client authentication is SERVER\_ENCRYPT, the client is authenticated by passing an encrypted user ID and encrypted password.

If SERVER\_ENCRYPT is specified at the client and SERVER is specified at the server, an error is returned because of the mismatch in the authentication levels.

**CLIENT** — Specifies that authentication occurs on the database partition where the application is invoked using operating system security. The user ID and password specified during a connection or attachment attempt are compared with the valid user ID and password combinations on the client node to determine if the user ID is permitted access to the instance. No further authentication will take place on the database server. This is sometimes called single signon. If the user performs a local or client login, the user is known only to that local client workstation.

**KERBEROS** — Used when both the DB2 client and server are on operating systems that support the Kerberos security protocol. The Kerberos security protocol performs authentication as a third-party authentication service by using conventional cryptography to create a shared secret key. This key becomes a user's credential and is used to verify the identity of users during all occasions when local or network services are requested. The key eliminates the need to pass the user name and password across the network as clear text. Using the Kerberos security protocol enables the use of a single sign-on to a remote DB2 server.

Kerberos authentication works on Windows as follows:

1. A user logging on to the client machine using a domain account authenticates to the Kerberos key distribution center (KDC) at the domain controller. The key distribution center issues a ticket-granting ticket (TGT) to the client.
2. During the first phase of the connection, the server sends the target principal name, which is the service account name for the DB2 server service, to the client. Using the server's target principal name and the target-granting ticket, the client requests a service ticket from the ticket-granting service (TGS) which also resides at the domain controller. If both the client's ticket-granting ticket and the server's target principal name are valid, the TGS issues a service ticket to the client.

3. The client sends this service ticket to the server via the communication channel (which may be, as an example, TCP/IP).
4. The server validates the client's server ticket. If the client's service ticket is valid, then the authentication is completed.

It is possible to catalog the databases on the client machine and explicitly specify the Kerberos authentication type with the server's target principal name. In this way, the first phase of the connection can be bypassed.

If a user ID and a password are specified, the client will request the ticket-granting ticket for that user account and use it for authentication.

**KRB\_SERVER\_ENCRYPT** — Specifies that the server accepts KERBEROS authentication or encrypted SERVER authentication schemes. If the client authentication is KERBEROS, the client is authenticated using the Kerberos security system. If the client authentication is not KERBEROS, or the Kerberos authentication service is not available, then the system authentication type is equivalent to SERVER\_ENCRYPT.



#### Note

The Kerberos authentication types are only supported on clients and servers running Windows 2000, Windows XP, and Windows.NET operating systems. Also, both client and server machines must either belong to the same Windows domain or belong to trusted domains. This authentication type should be used when the server supports Kerberos and some, but not all, of the client machines support Kerberos authentication.

### Control login restrictions of connecting user on AIX server updated in V8.2

By default, when a user is authenticated on an AIX server, DB2 checks the connecting user's local login restrictions before allowing the connection to proceed. The DB2LOGINRESTRICTIONS registry variable permits DB2 to enforce alternative modes of login restrictions.

If DB2LOGINRESTRICTIONS is not set, the default value is LOCAL.

The variable may be set to the following values:

- REMOTE: DB2 will only enforce remote login restrictions
- SU: DB2 will only enforce su restrictions (su in AIX is changing the ID during a session)
- NONE: DB2 will not enforce any particular mode of login restrictions
- LOCAL: DB2 will only enforce local login restrictions

In all cases, DB2 still checks for the following error conditions:

- Expired account
- Locked account
- Invalid user

## ***Instructor notes:***

**Purpose** — Identify the difference between authentication and authorization.

**Details** — In DB2, authentication takes place outside the database manager; authorization is performed within the database manager.

An analogy that you can use: pick a student, say his name is “Jiri”.

“Back on Monday, you introduced yourselves, and Jiri said that his name was ‘Jiri’ - but... did anyone check?” (you can give Jiri a suspicious look; better pick on someone who can take kidding). “So, Jiri could be a spy? If we wanted to be sure that Jiri was who he said he was, what would we need to do?” Answers might be to check his picture ID.

“In a computer system, how do we make sure that someone is who they say they are?” Answers should be “check a password”.

“Checking to see that someone is who they say they are is called ‘authenticating’ the user.”

“In DB2, authentication is done outside DB2. DB2 passes the authentication check to a facility outside DB2. By default, the operating system where the server is performs the authentication check: by requiring all DB2 users to be defined on the database server machine. You have other options; the list is in the student notes. For example, you could trust the client machine to authenticate the user (CLIENT authentication), which may not be very secure if you have Windows clients and you don’t have a very strict control over who the administrators of your Windows machines can be. Other options are SERVER\_ENCRYPT (which is like SERVER but requires the password to be encrypted when flowing over the network), KERBEROS, and KRB\_SERVER\_ENCRYPT.” (You can use the student notes to help you in discussing these options. Also mention that the *Universal Connectivity Guide*, a redbook, may help them with more information in this area.)

“Once we have made sure that Jiri is who he says he is, then we can look inside DB2 and find out what Jiri is allowed to do: his authorization on DB2.” (this will move you on to the next page).

**Additional Information** — Student notes are taken from the *Administration Guide*: Implementation. Additional information from this manual:

If the remote instance has CLIENT authentication, two other parameters determine the final authentication type: *trust\_allclnts* and *trust\_clntauth*.

### **CLIENT Level Security for TRUSTED Clients Only:**

Trusted clients are clients that have a reliable, local security system. Specifically, all clients are trusted clients except for Windows 9x operating systems.

When the authentication type of CLIENT has been selected, an additional option may be selected to protect against clients whose operating environment has no inherent security. To protect against unsecured clients, the administrator can select Trusted Client Authentication by setting the *trust\_allclnts* parameter to NO. This implies that all trusted platforms can authenticate the user on behalf of the server. Untrusted clients are

authenticated on the server and must provide a user ID and password. You use the *trust\_allclnts* configuration parameter to indicate whether you are trusting clients. The default for this parameter is YES.

**Note:** It is possible to trust all clients (*trust\_allclnts* is YES), yet have some of those clients as those who do not have a native safe security system for authentication.

You may also want to complete authentication at the server even for trusted clients. To indicate where to validate trusted clients, you use the *trust\_clntauth* configuration parameter. The default for this parameter is CLIENT.

**Note:** For trusted clients only, if no user ID or password is explicitly provided when attempting to CONNECT or ATTACH, then validation of the user takes place at the client. The *trust\_clntauth* parameter is only used to determine where to validate the information provided on the USER or USING clauses.

To protect against all clients except DRDA clients from DB2 for OS/390 and z/OS, DB2 for VM and VSE, and DB2 for iSeries, set the *trust\_allclnts* parameter to DRDAONLY. Only these clients can be trusted to perform client-side authentication. All other clients must provide a user ID and password to be authenticated by the server.

The *trust\_clntauth* parameter is used to determine where the above clients are authenticated: if *trust\_clntauth* is client, authentication takes place at the client. If *trust\_clntauth* is server, authentication takes place at the client when no password is provided, and at the server when a password is provided.

Security issues are important to the DB2 administrator from the moment the product is installed.

The steps to completing the installation of DB2 require a user name, a group name, and a password. During the installation, the administrator has default values for each of these requirements. Once the defaults have been used during the installation of DB2, the administrator is strongly recommended to create new user names, group names, and passwords before creating the instances where the databases will reside. Using new user names, group names, and passwords will minimize the risk of a user other than the administrator learning of the defaults and using them in an improper fashion within instances and databases.

Passwords are very important when authenticating users. If no authentication requirements are set at the operating system level and the database is using the operating system to authenticate users, then users will be allowed to connect. For example on a UNIX operating system, undefined passwords are treated as NULL. And any user without a defined password will be treated as if they have a NULL password. From the operating system's perspective, this is a match and the user is validated and able to connect to the database. You should require passwords at the operating system level if you want the operating system to do the authentication of users for your database.

Another security recommendation following the installation of DB2 is the changing of the default privileges granted to users. During the installation process, System Administration



(SYSADM) privileges are granted by default to the following users on each operating system:

- **Windows 9x** Any Windows 98, or Windows ME user.
- **Windows NT** On Windows NT, Windows 2000, Windows XP, or Windows .NET, a valid DB2 username which belongs to the Administrators group.
- **UNIX** A valid DB2 username which belongs to the primary group of the instance owner's user ID.

SYSADM privileges are the most powerful set of privileges available within DB2. (Privileges are discussed later in this chapter.) As a result, you may not want all of these users to have SYSADM privileges by default. DB2 provides the administrator with the ability to grant and revoke privileges to groups and individual user IDs.

By updating the database manager configuration parameter *sysadm\_group*, the administrator can control which group is defined as the System Administrative group with System Administrator privileges. You must follow the guidelines below to complete the security requirements for both DB2 installation and the subsequent instance and database creation.

Any group defined as the System Administration group (by updating *sysadm\_group*) must exist. The name of this group should allow for easy identification as the group created for instance owners. User IDs and groups that belong to this group have system administrator authority for their respective instances.

You should consider creating an instance owner user ID that is easily recognized as being associated with a particular instance. This user ID should have, as one of its groups, the name of the SYSADM group created above.

Another recommendation is to only use this instance owner user ID as a member of the instance owner group and not to use it in any other group. This should control the proliferation of user IDs and groups that could modify the instance environment.

The created user ID should always be associated with a password to allow for authentication before entry into the data and databases within the instance. The recommendation when creating a password is to follow your organization's password naming guidelines.

Each operating system provides ways to manage security. Some of the security issues associated with the operating systems are discussed in this section.

### **Windows NT Platform Security Considerations for Users**

System Administration (SYSADM) authority is granted to any valid DB2 user account which belongs to the local Administrators group on the machine where the account is defined.

By default in a Windows domain environment, only domain users that belong to the Administrators group at the Domain Controller have SYSADM authority on an instance. Since DB2 always performs authorization at the machine where the account is defined, adding a domain user to the local Administrators group on the server does not grant the domain user SYSADM authority to the group.

**Note:** In a domain environment such as is found in Windows NT, DB2 only authenticates the first 64 groups that meet the requirements and restrictions, and to which a user ID belongs. You may have more than 64 groups.

To avoid adding a domain user to the Administrators group at the PDC (Primary Domain Controller), you should create a global group and add the users (both domain and local) to whom you want to grant SYSADM authority. To do this, enter the following commands:

```
DB2STOP
DB2 UPDATE DBM CFG USING SYSADM_GROUP global_group
DB2START
```

## UNIX Platform Security Considerations for Users

DB2 does not support root acting directly as a database administrator. You should use **su - <instance owner>** as the database administrator.

On UNIX-based platforms, a group for fenced User-Defined Functions (UDFs) and stored procedures must be created, and any user IDs that use fenced UDFs or stored procedures must be a member of this group. As with the SYSADM group, the name of the fenced UDFs or stored procedures group should allow for easy identification. User IDs that belong to the fenced UDFs or stored procedures group have whatever authority and privileges that are associated with the group as their default.

For security reasons, we recommend you do not use the instance name as the Fenced ID. However, if you are not planning to use fenced UDFs or stored procedures, you can set the Fenced ID to the instance name instead of creating another user ID.

The recommendation is to create a user ID that will be recognized as being associated with this group. The user for fenced UDFs and stored procedures is specified as a parameter of the instance creation script (`db2icrt ... -u <FencedID>`). This is not required if you install the DB2 Clients or the DB2 Software Developer's Kit.

## General Rules for Naming Objects and Users

There are rules for the naming of all objects and users. Some of these rules are specific to the platform you are working on. For example, there is a rule regarding the use of upper and lower case letters in a name.

- On UNIX platforms, names must be in lower case.
- On Windows platforms, names can be in upper, lower, and mixed-case.

On UNIX, the **db2icrt** command creates the main SQL library (sqllib) directory under the home directory of the instance owner.

On Windows operating systems, the instance directory is located in the /sqllib subdirectory, in the directory where DB2 was installed.

## Notes:

1. The type of authentication you choose is important only if you have remote database clients accessing the database or when you are using federated database functionality.

Most users accessing the database through local clients are always authenticated on the same machine as the database. An exception can exist when Kerberos is used.

2. Do not inadvertently lock yourself out of your instance when you are changing the authentication information, since access to the configuration file itself is protected by information in the configuration file. The following database manager configuration file parameters control access to the instance:

- AUTHENTICATION \*
- SYSADM\_GROUP \*
- TRUST\_ALLCLNTS
- TRUST\_CLNTAUTH
- SYSCTRL\_GROUP
- SYSMANT\_GROUP
- CLNT\_KRB\_PLUGIN
- CLNT\_PW\_PLUGIN
- GROUP\_PLUGIN
- LOCAL\_GSSPLUGIN
- SRV\_PLUGIN\_MODE
- SRV\_CON\_AUTH
- SRV\_GSSPLUGIN\_LIST
- SRVCON\_PW\_PLUGIN

\* Indicates the two most important parameters, and those most likely to cause a problem.

There are some things that can be done to ensure this does not happen: if you do accidentally lock yourself out of the DB2 system, you have a fail-safe option available on all platforms that will allow you to override the usual DB2 security checks to update the database manager configuration file using a highly privileged local operating system security user. This user *always* has the privilege to update the database manager configuration file and thereby correct the problem. However, this security bypass is restricted to a local update of the database manager configuration file. You cannot use a fail-safe user remotely or for any other DB2 command. This special user is identified as follows:

- UNIX platforms: the instance owner
- NT platform: someone belonging to the local administrators group
- Other platforms: there is no local security on the other platforms, so all users pass local security checks anyway

### Authentication Considerations for Remote Clients

When cataloging a database for remote access, the authentication type may be specified in the database directory entry.

For databases accessed using DB2 Connect: If a value is not specified, SERVER authentication is assumed.

The authentication type is not required. If it is not specified, the client will default to SERVER\_ENCRYPT. However, if the server does not support SERVER\_ENCRYPT, the client attempts to retry using a value supported by the server. If the server supports multiple authentication types, the client will not choose amongst them but instead return an error. This is done to ensure that the correct authentication type is used. In this case, the client must catalog the database using a supported authentication type. If an authentication type is specified, then authentication can begin immediately provided that the value specified matches that at the server. If a mismatch is detected, then DB2 attempts to recover. Recovery may result in more flows to reconcile the difference or in an error if DB2 cannot recover. In the case of a mismatch, the value at the server is assumed to be correct.

### **Control login restrictions of connecting user on AIX server updated in V8.2**

By default, when a user is authenticated on an AIX server, DB2 checks the connecting user's local login restrictions before allowing the connection to proceed. The DB2LOGINRESTRICTIONS registry variable permits DB2 to enforce alternative modes of login restrictions.

If DB2LOGINRESTRICTIONS is not set, the default value is LOCAL.

The variable may be set to the following values:

**REMOTE:** DB2 will only enforce remote login restrictions

**SU:** DB2 will only enforce su restrictions

**NONE:** DB2 will not enforce any particular mode of login restrictions

**LOCAL:** DB2 will only enforce local login restrictions

In all cases, DB2 still checks for the following error conditions:

- Expired account
- Locked account
- Invalid user

Certification objective: 7A Knowledge of DB2 authentication, 7B Knowledge of DB2 authorizations.

**Transition Statement** — Now that you have made sure that the user is who he says he is, what can he do within DB2?

## 11.2 Access control — privileges

### Instructor topic introduction

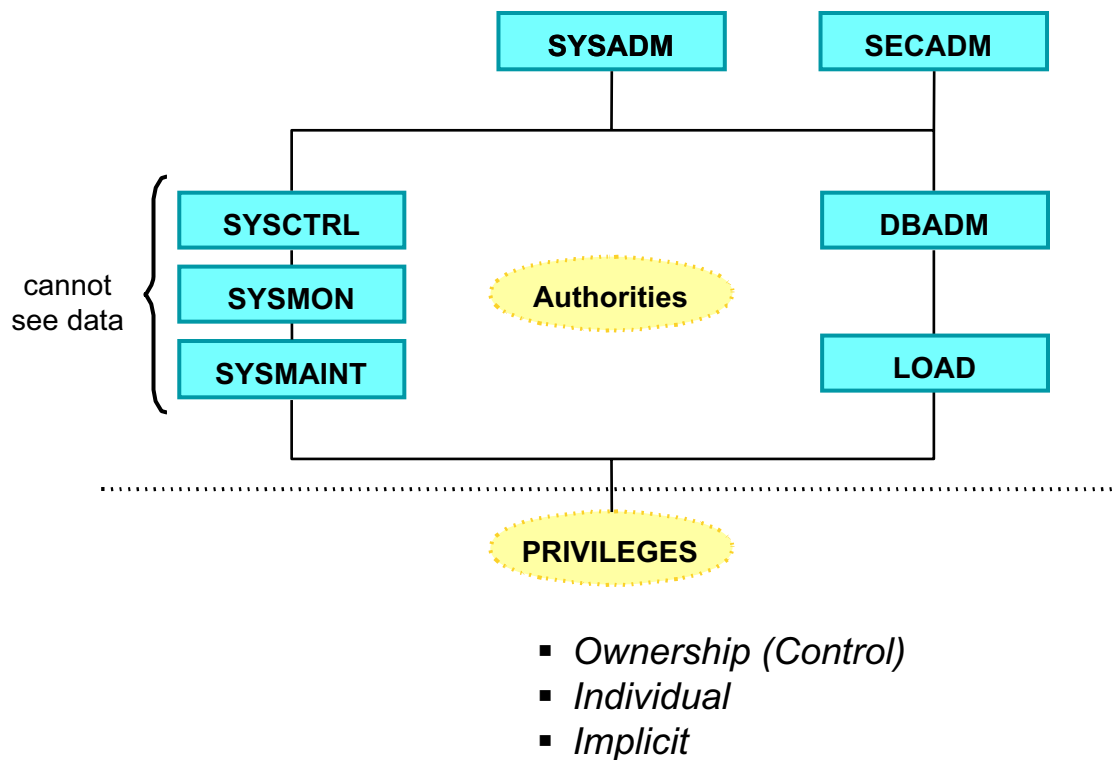
**What students will do** — List database privileges.

**How students will do it** — Lecture and lab.

**What students will learn** — Database privileges.

**How this will help students on their job** — Control access to the database through database privileges.

## DB2 access control hierarchy



© Copyright IBM Corporation 2007

Figure 11-3. DB2 access control hierarchy

CF238.3

### Notes:

This section describes the possible classes of users and their related authority levels in the DB2 environment.

When a privilege is associated with an object, that object must already exist.

Different levels of authority and several privileges can be assigned to users of DB2. Access to sysadm, sysctrl, sysmaint, and dbadm privileges should be restricted to avoid the risk of compromising system integrity. These authority levels are not required for general users.

**Authority** is a set of privileges covering a set of database objects. A user or group can have one or more of the following levels of authorization:

- **SYSADM** - Administrative authority - System Administrators are given full privileges over the entire DB2 instance. SYSADM can not be granted with a SQL statement.
- **SECADM** - The security administrator (SECADM) authority collects several security-related privileges under one authority. The abilities given by SECADM are not given by any other authority, not even SYSADM.

SECADM authority allows you to perform these actions:

- Create, drop, grant permission to access, or revoke the various objects that are part of label-based access control (LBAC)
- Use the TRANSFER OWNERSHIP statement on objects that you do not own
- Grant or revoke the SETSESSIONUSER privilege
- **SYSCTRL** - System Authority - System Controllers are given full privileges for managing the system, but are not allowed access to the data. SYSCTRL can not be granted with a SQL statement.
- **SYSMAINT** - System Authority - System Maintainers are given a subset of privileges for managing the system, but are not allowed access to the data. SYSMAINT cannot be granted with a SQL statement.
- **SYSMON** - Gives the ability to take database system monitor snapshots of a database manager instance or its databases. This includes the following commands: GET DBM MONITOR SWITCHES, GET MONITOR SWITCHES, GET SNAPSHOT, LIST ACTIVE DATABASES, LIST APPLICATIONS, LIST DCS APPLICATIONS, RESET MONITOR, UPDATE MONITOR SWITCHES.
- **DBADM** - Administrative authority - Database Administrators have control over an individual database. DBADM can be granted with a SQL statement. SYSCAT.DBAUTH has an entry for each database administrator.
- **LOAD** - The LOAD authority (LOAD) gives LOAD utility or AutoLoader utility privileges to load data into tables.

To create and set up an instance of the DB2 product, you must have operating system administration authority.

**Privilege** is the right of a particular user or group to create or access a resource.

1. **Ownership (CONTROL) privileges** - For most objects, the creator of the object has full access to that object. CONTROL privilege is automatically granted to the creator of an object (although some views are exceptions to this rule). CONTROL privileges are equivalent in all respects to ownership of the object, including the right to access an object in any way and to grant privileges on the object. Privileges are controlled by users with ownership or administrative authority. These users can grant or revoke privileges to or from other users.
2. **Individual privileges** - Individual privileges allow a specific function, sometimes on a specific object.
3. **Implicit privileges** - For example, users can be implicitly allowed to exercise privileges if they have the privilege to execute a package that requires those privileges.

On a UNIX platform, DB2 should be the only process running under the user ID of the instance owner. This safeguards the integrity of the operating system resources DB2 creates. If other processes were to run under this ID, they would have access to these resources, and might accidentally remove or corrupt them.

## ***Instructor Notes:***

**Purpose** — Introduce the concept of authorization levels.

**Details** — In DB2, there are certain authorities that provide the ability to perform some underlying tasks.

The highest level of authority is SYSADM. SYSADM is like *DB2 king*: SYSADM can do anything on the instance and the databases under it. The definition of who is SYSADM is specified by the group named in SYSADM\_GROUP in the DBM CFG. The members of the group are defined by the server operating system.

There are two lower levels of administrative authority on the instance: SYSCTRL (like a *DB2 prince*) and SYSMANT. SYSCTRL can do a subset of the functions that SYSADM can do, and SYSMANT (like a *DB2 knight*) can do a further subset of functions of SYSCTRL. Neither of these authority levels permits the user holding them to see data. Users who have SYSCTRL authority are determined by them being in the SYSCTRL\_GROUP defined in the DBM CFG. SYSMANT is defined by the members of the SYSMANT\_GROUP as defined in the DBM CFG. Note again that the members of the group are defined by the operating system definition of the group membership.

DBADM is another level of administrative authority. DBADM is a granted authority; you issue a GRANT DBADM TO username to grant this authority level (there isn't a group defined in the DBM CFG for DBADM). A user with DBADM authority has authority over a database.

LOAD authority carries with it some underlying functions besides just the ability to load tables.

SYSMON gives the authority for monitoring purposes of the instance or database.

**Additional Information** —

**Transition Statement** — Let's look at the underlying functions.



## Database Authority summary

| <i>Function</i>                     | <i>SYSADM</i> | <i>SYSCTRL</i> | <i>SYSMAINT</i> | <i>DBADM</i> | <i>LOAD</i> |
|-------------------------------------|---------------|----------------|-----------------|--------------|-------------|
| MIGRATE DATABASE                    | YES           |                |                 |              |             |
| UPDATE DBM CFG                      | YES           |                |                 |              |             |
| GRANT/REVOKE DBADM                  | YES           |                |                 |              |             |
| UPDATE db/node/dcs directories      | YES           | YES            |                 |              |             |
| FORCE USERS OFF SYSTEM              | YES           | YES            |                 |              |             |
| CREATE/DROP DATABASE                | YES           | YES            |                 |              |             |
| CREATE/DROP/ALTER TABLE SPACE       | YES           | YES            |                 |              |             |
| RESTORE TO NEW DATABASE             | YES           | YES            |                 |              |             |
| UPDATE DB CFG                       | YES           | YES            | YES             |              |             |
| BACKUP DATABASE or TABLE SPACE      | YES           | YES            | YES             |              |             |
| RESTORE TO EXISTING DATABASE        | YES           | YES            | YES             |              |             |
| PERFORM ROLLFORWARD RECOVERY        | YES           | YES            | YES             |              |             |
| START/STOP DATABASE INSTANCE        | YES           | YES            | YES             |              |             |
| RESTORE TABLE SPACE                 | YES           | YES            | YES             |              |             |
| RUN TRACE                           | YES           | YES            | YES             |              |             |
| TAKE DBM or DB SNAPSHOTS            | YES           | YES            | YES             |              |             |
| QUERY TABLE SPACE STATE             | YES           | YES            | YES             | YES          | YES         |
| UPDATE LOG HISTORY FILES            | YES           | YES            | YES             | YES          |             |
| QUIESCE TABLE SPACE                 | YES           | YES            | YES             | YES          | YES         |
| REORG TABLE                         | YES           | YES            | YES             | YES          |             |
| RUN RUNSTATS UTILITY                | YES           | YES            | YES             | YES          | YES         |
| READ LOG FILES                      | YES           |                |                 | YES          |             |
| CREATE/ACTIVATE/DROP EVENT MONITORS | YES           |                |                 | YES          |             |

© Copyright IBM Corporation 2007

Figure 11-4. Database Authority summary

CF238.3

### Notes:

**System Administrator Authority** - SYSADM authority is the highest level of administrative authority. Users with SYSADM authority can run utilities, issue database and database manager commands, and access the data in any table in any database within the database manager instance. It provides the ability to control all database objects in the instance, including databases, tables, views, indexes, and packages.

**System Control Authority** - SYSCTRL authority is the highest level of system control authority. This authority provides the ability to perform maintenance and utility operations against the database manager instance and its databases. These operations can affect system resources, but they do not allow direct access to data in the databases. System control authority is designed for users administering a database manager instance containing sensitive data. A user with SYSCTRL authority has the implicit privilege to connect to a database.

**System Maintenance Authority** - SYSMAINT authority is the second level of system control authority. This authority provides the ability to perform maintenance and utility operations against the database manager instance and its databases. These operations

can affect system resources, but they do not allow direct access to data in the databases. System maintenance authority is designed for users maintaining databases within a database manager instance that contains sensitive data. A user with SYSMAINT authority has the implicit privilege to connect to a database.

**System Monitor Authority** - SYSMON gives the ability to take database system monitor snapshots of a database manager instance or its databases. This includes the following commands: GET DBM MONITOR SWITCHES, GET MONITOR SWITCHES, GET SNAPSHOT, LIST ACTIVE DATABASES, LIST APPLICATIONS, LIST DCS APPLICATIONS, RESET MONITOR, UPDATE MONITOR SWITCHES.

**Security administration authority** - SECADM authority can only be granted by the SYSADM and can be granted to a user but not to a group. It gives these and only these abilities: Create and drop security label components, Create and drop security policies, Create and drop security labels, Grant and revoke security labels, Grant and revoke LBAC rule exemptions, Grant and revoke setsessionuser privileges, Execute the SQL statement TRANSFER OWNERSHIP on objects that you do not own.

No other authority gives these abilities, not even SYSADM.

**Database Administration Authority** - DBADM authority is the second highest level of administrative authority. It applies only to a specific database, and allows the user to run utilities, issue database commands, and access the data in any table in the database. When DBADM authority is granted, BINDADD, CONNECT, CREATETAB, and CREATE NOT FENCED privileges are granted as well. Only a user with SYSADM can grant/revoke DBADM. Users with DBADM can grant privileges on the database to others and can revoke any privilege from any user regardless of who granted it.

A user may hold DBADM authority on more than one database, but a DBADM can only perform functions on the databases for which DBADM authority is held.

**Load Authority** - Users having LOAD authority at the database level, as well as INSERT privilege on a table, can use the LOAD command to load data into a table. Users having LOAD authority at the database level, as well as INSERT privilege on a table, can LOAD RESTART or LOAD TERMINATE if the previous operation was a load to insert data. If the previous load operation was a load replace, the DELETE privilege must also have been granted to that user before the user can LOAD RESTART or LOAD TERMINATE. If the exception tables are used as part of a load operation, the user must have INSERT privilege on the exception tables. The user with this authority can perform QUIESCE TABLESPACES FOR TABLE, RUNSTATS, and LIST TABLESPACES commands.

The database manager configuration parameter CATALOG\_NOAUTH specifies whether users are able to catalog and uncatalog databases and nodes, or DCS and ODBC directories, without SYSADM authority. Default value is NO which means that SYSADM or SYSCTL authority is needed. If set to YES, the DB2 system administrator can allow users to maintain their own cataloging.

**Instructor Notes:**

**Purpose** — Identify what the different authority levels imply.

**Details** — The functions down the left side of this chart are NOT grantable privileges. They are only available to users by being in the *right group* - having the appropriate authority level.

To have the students look at this chart more closely, here are some questions to ask:

Looking at the graphic, what authority level is necessary to create a database?

From the chart, you can see that either SYSADM or SYSCTRL authority is required to create a database.

What authority is required to back up a database or a table space?

SYSADM, SYSCTRL, or SYSMAINT authority is required to backup a database or a table space.

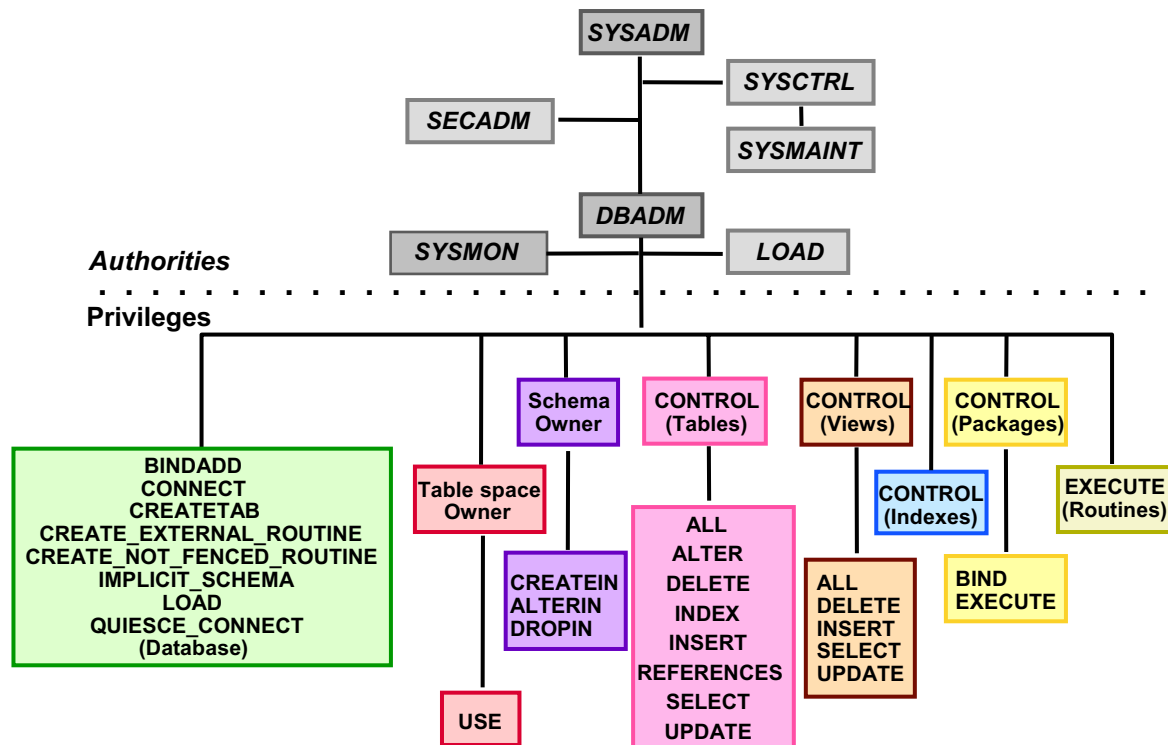
You may have been surprised to discover that an individual with DBADM authority cannot back up a database or table space. This is considered to be a system control type of function. If you had an individual who needed both DBADM authority and the ability to back up the database, how could you accomplish that?

You would need to ensure that the individual who is a DBADM is also in the SYSMAINT, or SYSCTRL, or SYSADM group. The minimum additional authority to give them the ability to perform backups would be SYSMAINT.

**Additional Information** —

**Transition Statement** — Let's look at privileges.

# Authorities and privileges



© Copyright IBM Corporation 2007

Figure 11-5. Authorities and privileges

CF238.3

## Notes:

Both DB2 authorities and privileges on objects are hierarchical in nature.

A **privilege** is the right of a particular user or group to create or access a resource.

Resources can be protected from unauthorized access by:

- Database privileges controlling both access and creation of databases.
- Table privileges controlling both access and creation of tables.
- View privileges controlling access and creation of views. Views can be used to control access to a table. By granting access to a view instead of the underlying tables, access is restricted to the rows and columns selected by the view definition.
- Index privileges controlling the creation of indexes.
- Package privileges controlling the creation, modification, and execution of packages.

Users with SYSADM authority automatically possess the privileges associated with DBADM authority for each database in the instance.

## DATABASE PRIVILEGES

- The creator of a database automatically receives DBADM authority.
- DBADM authority allows any SQL access to the database.
- DBADM authority is required to GRANT the following privileges:
  - BINDADD - Create new packages in the database.
  - CONNECT - Connect to the database.
  - CREATETAB - Create new tables in the database.
  - CREATE\_EXTERNAL\_ROUTINE - Allows a user to create a procedure for use by applications and other users of the database.
  - CREATE\_NOT\_FENCED\_ROUTINE - Create a user-defined function or procedure that is *not fenced*. UDFs or procedures that are *not fenced* must be extremely well tested because the database manager does not protect its storage or control blocks from these UDFs. An *unfenced* UDF runs on the same side of the *firewall* as the database engine code.
  - IMPLICIT\_SCHEMA - Create a schema implicitly by creating an object using a CREATE statement with a schema name that does not already exist. SYSIBM becomes the owner of the implicitly created schema, and PUBLIC is given the privilege to create objects in this schema.
  - LOAD - Load data into a table. LOAD is considered an authority since it implies other functions (quiesce table spaces, query table spaces, runstats), but it also gives the LOAD privilege at the database level.
  - QUIESCE\_CONNECT - Allows a user to access the database while it is quiesced.
- Only users with SYSADM or DBADM authority can grant and revoke these privileges to and from other users.

## TABLE SPACE PRIVILEGES

- The owner of the table space, typically the creator who has SYSADM or SYSCTRL authority, has USE privilege and the ability to grant this privilege to others.
- USE privilege allows the user to create tables within the table space.
- By default, at database creation time the USE privilege for table space USERSPACE1 is granted to PUBLIC, though this privilege can be revoked.
- The USE privilege cannot be used with SYSCATSPACE or any system temporary table spaces.

## SCHEMA PRIVILEGES

- The owner of the schema has all of the schema privileges and the ability to grant them to others.
- Being owner of a schema allows GRANTing of:
  - ALTERIN - Alter objects in the schema.
  - CREATEIN - Create objects within the schema.
  - DROPIN - Drop objects within the schema.
- Objects that are manipulated within the schema object include: tables, views, indexes, packages, data types, functions, triggers, procedures, and aliases.

## TABLE PRIVILEGES

- The creator of a table automatically receives **CONTROL** privilege. To grant CONTROL privilege on a table or view, the user must have SYSADM or DBADM authority.
- CONTROL privilege allows GRANTing of:
  - ALTER - Add columns to a table, add or change comments on a table and its columns, add a primary key or unique constraint, and create or drop a table check constraint. The user can also create triggers on the table, though additional authority on all objects referenced in the trigger, including SELECT on the table if the trigger references any columns of the table) is required. A user with ALTER on all the descendent table can drop a primary key; a user with ALTER privilege on the table and REFERENCES privilege on the parent table, or REFERENCES privilege on the appropriate columns, can create or drop a foreign key.
  - DELETE - Delete rows from the table/view.
  - INDEX - Create indexes on the table. The creator of the index has CONTROL privilege on index and can drop the index.
  - INSERT - Insert rows into the table. Run IMPORT utility against the table.
  - REFERENCES - Create referential constraints on a table, specifying the table as the parent in a relationship. The user might have this privilege only on specific columns.
  - SELECT - Retrieve rows from a table or view. Create a view on the table. Run the EXPORT utility against the table or view.
  - UPDATE - Change entries in the table or view or for one or more specific columns in a table or view. The user may have this privilege only on specific columns.
- CONTROL privilege is required to DROP the table, REORGANIZE the table, or refresh its statistics with RUNSTATS utility, and grant or revoke individual table privileges. When a user is granted CONTROL privilege on a table, all other privileges on that table are automatically granted WITH GRANT OPTION.
- The privilege to grant these privileges to others may also be granted using the WITH GRANT OPTION on the GRANT statement.

## VIEW PRIVILEGES

- To create a view, SELECT or CONTROL privilege on every table or view referenced is required.
- CONTROL privilege is ONLY granted if the creator has CONTROL privilege on all tables and views referenced, or if they have SYSADM or DBADM authority.
- CONTROL privilege is required to grant DELETE, INSERT, SELECT, and UPDATE privileges; SELECT is the only option for **read-only** views.
- View privileges allow an authorized user to perform actions against base tables that they are not allowed to perform directly.

## INDEX PRIVILEGES

- The creator of an index automatically receives CONTROL privileges.
- CONTROL privilege allows users to drop the index.
- No privilege is required to use an index.
- The table level INDEX privilege allows a user to create an index on that table.

## PACKAGE PRIVILEGES

- The creator of a package automatically receives CONTROL privilege.
- CONTROL privilege allows granting of:
  - BIND - Bind or rebind the package and to add new package versions of the same package name and creator.
  - EXECUTE - Execute or run the package.
- CONTROL privilege implies that you can drop a package.
- The binder must have privileges or authority required to execute every embedded static SQL statement in the package granted to their userid or to PUBLIC.
- Any user with EXECUTE privilege may execute the statements within the scope of executing the package. They may not execute the statements in a dynamic environment (such as CLP) unless they have been granted authority to do so.
- Package privileges provide a powerful way to extend partial privileges to users through executing a package that contains embedded SQL.
- Database privilege BINDADD allows users to create new packages or rebind an existing package in the database.

## ROUTINE PRIVILEGES

- Execute privileges involve actions on all types of routines such as functions, procedures, and methods within a database. Once having execute privilege, a user can then invoke that routine, create a function that is sourced from that routine (applies to functions only), and to reference the routine in any DDL statement such as CREATE VIEW, CREATE TRIGGER; or, when defining a constraint.
- The one who defines the externally stored procedure, function, or method receives EXECUTE WITH GRANT privilege.

SYSADM has implicit connect on all databases. DBADM has implicit connect on a specific database. When a database is created, the following privileges are automatically granted to PUBLIC. To remove these privileges, a DBADM or SYSADM must explicitly revoke the database privileges from PUBLIC.

- CREATETAB
- BINDADD
- CONNECT
- SELECT privilege on system catalog views.



## ***Instructor Notes:***

**Purpose** — Introduce students to privileges.

**Details** — There is a lot of information on this page. Suggest going through the student notes:

First, look at the notes on database privileges. The first note says that the creator of the database receives DBADM authority. But, wait a minute, a couple of pages ago, we said that to create a database, you needed SYSADM or SYSCTRL. If you have SYSADM, why do you need DBADM?

Well, the answer is, you don't. But, how did you get to be SYSADM? You were in the *right* group. Could you stop being SYSADM? Yes, if your ID was removed from the group that is defined as SYSADM\_GROUP for the instance. If that happened, you would still be DBADM for any databases you had created.

If you are DBADM, you can access any object in the database via SQL. You can also grant the database level privileges (the Administration manual calls these *authorities*):

BINDADD means that the user can create new packages. CONNECT means that the user can connect to the database. CREATETAB means that the user can create new tables. CREATE\_EXTERNAL\_ROUTINE means that the user can define UDFs and stored procedures that have external code. CREATE\_NOT\_FENCED\_ROUTINE means that the user can create UDFs and stored procedures that run within the db2agent.

IMPLICIT\_SCHEMA means that the user can implicitly create a schema by creating an object with a schema name that does not yet exist. LOAD means that the user can perform a load (and, as we saw from the previous page, it also implies the ability to do other functions. QUIESCE\_CONNECT means that the user can connect to the database when it is quiesced.

To be able to create objects in a table space, you must have USE privilege on that table space. To create a table in a table space, you need CREATETAB on the database, and USE privilege on the table spaces you create the table in. The table space owner, or a user with DBADM or SYSADM authority can grant USE privilege.

We will address schema privileges in more detail on the next page. But, you may want to mention here that to be able to create a table, you would also need to have CREATEIN on the schema you are creating in. You could also give someone the ability to alter objects that have a particular schema name, or drop objects with a particular schema name.

Let's look at table privileges. You can see the underlying table privileges, SELECT, INSERT, UPDATE, DELETE, ALTER. INDEX means that you can create an index on the table. REFERENCES means that you can define referential integrity. ALL means that you have all of the underlying privileges. See student notes for CONTROL - CONTROL means that you have all of the underlying privileges, with GRANT OPTION, and that you can drop, reorganize, or run runstats on the table.

Let's say that "John" has SELECT privilege on my table, and "Mary" has CONTROL privilege on my table. ("John" and "Mary" are two students in your class.) What is the



difference between what John can do and what Mary can do? You should get answers that John can select from the table, Mary can drop the table, do all actions (insert, update, delete), and grant privileges to others.

Let's move on to views. Views are interesting: for most objects, you get CONTROL when you create the object. For views, you only get control if you have CONTROL on all the underlying objects. Let's talk about John and Mary again: remember that John has SELECT on my table, Mary has CONTROL on my table. Who can create a view on my table?

The answer is: they both can.

What will John's subsequent authority on that view be? John will have SELECT privilege on the view: he can use the view, but he cannot give privileges to anyone else to use my table. That is appropriate: he was not able to give access to my table directly, and he cannot give access through his view.

What about Mary's authority on the view? She will have CONTROL privilege on the view: she can use the view, and she can give other users the ability to access my table through her view. This is appropriate because she could have given access to my table directly; she can also give access to my table through her view.

To wrap these privileges up: the only privilege on indexes is CONTROL. The person who creates the index gets CONTROL privilege. Note that you don't need any privilege to USE an index. If the optimizer determines that the quickest way to access the data in the table is through an index, all you have to have is access to the table; you can use any of its indexes to speed up that access. (Wouldn't it be horrible if there was a USE privilege on an index, so the optimizer would say — wow — this would give you really fast performance if you could use index IX1, but, whoops, you don't have the privilege to use that index, so you get slow performance!)

On packages, BIND lets you bind or rebind existing packages; EXECUTE lets you run an existing package. Show the students the note "The binder must have privileges or authority required to execute every embedded static SQL statement in the package granted to their userid or to PUBLIC". DB2 does not check group privileges when determining if a user can bind a package; more on this later.

Finally, routines have execute privilege as well.

### **Additional Information** — Sequence privileges:

The creator of a sequence automatically receives the USAGE privilege. The USAGE privilege is needed to use NEXTVAL and PREVVAL expressions for the sequence. To allow other users to use the NEXTVAL and PREVVAL expressions, sequence privileges must be granted to PUBLIC. This allows all users to use the expressions with the specified sequence.

Certification objective: 7B Knowledge of DB2 authorizations.

**Transition Statement** — Let's look at schemas.

## Controlling use of schemas

- Named collection of objects
- Forms high-order part of objects with a two part name, for example, MEL.T1
- User with DBADM authority creates schema PAY for user MEL

```
CREATE SCHEMA PAY AUTHORIZATION MEL
```
- Mel can create objects in schema pay

```
CREATE TABLE PAY.T1 (COL1 INT)
```
- Mel can grant privileges to other users:
  - `GRANT CREATEIN ON SCHEMA PAY TO USER CAL`
  - `GRANT ALTERIN, CREATEIN, DROPIN ON SCHEMA PAY TO GROUP G1 WITH GRANT OPTION`
- Achieving greater schema control:
  - `REVOKE IMPLICIT_SCHEMA ON DATABASE FROM PUBLIC`
  - `GRANT IMPLICIT_SCHEMA ON DATABASE TO USER JON`

© Copyright IBM Corporation 2007

Figure 11-6. Controlling use of schemas

CF238.3

### Notes:

A schema can be defined whenever it is desirable to group a set of objects. This grouping may be based on objects for such things as an application, a group of people, or an individual user.

The schema is defined explicitly using the CREATE SCHEMA statement. An option (AUTHORIZATION) on the CREATE SCHEMA statement allows owner specification at the time the schema is created. A schema is defined to have an owner. The owner of the schema is given the privilege to create objects using the schema name as the object qualifier and to drop any objects that are defined in the schema (regardless of definer). The schema owner also has the privilege to grant the privilege to create objects in the schema or the privilege to drop objects from the schema to other users. This means that the schema owner has the ability to manage objects in the schema and may grant similar ability to manage those objects to others.

When a new database is created, PUBLIC is given IMPLICIT\_SCHEMA database authority. With this authority, any user can create a schema by creating an object and specifying a schema name that does not already exist. SYSIBM becomes the owner of the

implicitly created schema and PUBLIC is given the privilege to create objects in this schema.

If control of who can implicitly create schema objects is required for the database, IMPLICIT\_SCHEMA database authority should be revoked from PUBLIC. Once this is done, there are only three ways that a schema object is created:

- Any user can create a schema using their own authorization name on a CREATE SCHEMA statement.
- Any user with DBADM authority can explicitly create any schema which does not already exist, and can optionally specify another user as the owner of the schema.
- Any user with DBADM authority has IMPLICIT\_SCHEMA database authority (independent of PUBLIC) so that they can implicitly create a schema with any name at the time they are creating other database objects.

SYSIBM becomes the owner of the implicitly created schema and PUBLIC has the privilege to create objects in the schema.

A user always has the ability to explicitly create their own schema using their own authorization name.

## ***Instructor Notes:***

**Purpose** — Identify privileges associated with schemas.

**Details** — First, schemas are the high level qualifier of objects with a two-part name. A user with DBADM or SYSADM authority can explicitly create a schema with an ID other than their own, and give authorization to someone else, for example, MEL. With the ownership of the PAY schema, Mel can create objects in that schema; she can grant privileges to other users, and even grant privileges with GRANT OPTION.

Note that when the database is created, IMPLICIT\_SCHEMA is given to PUBLIC by default. That says that anyone who can connect to the database (also PUBLIC by default) can cause a schema to be created implicitly just by creating an object that has a schema name that does not yet exist. So, *JoeBlow* could create a view *FINANCE.MYVIEW* and, with that statement, implicitly create the FINANCE schema.

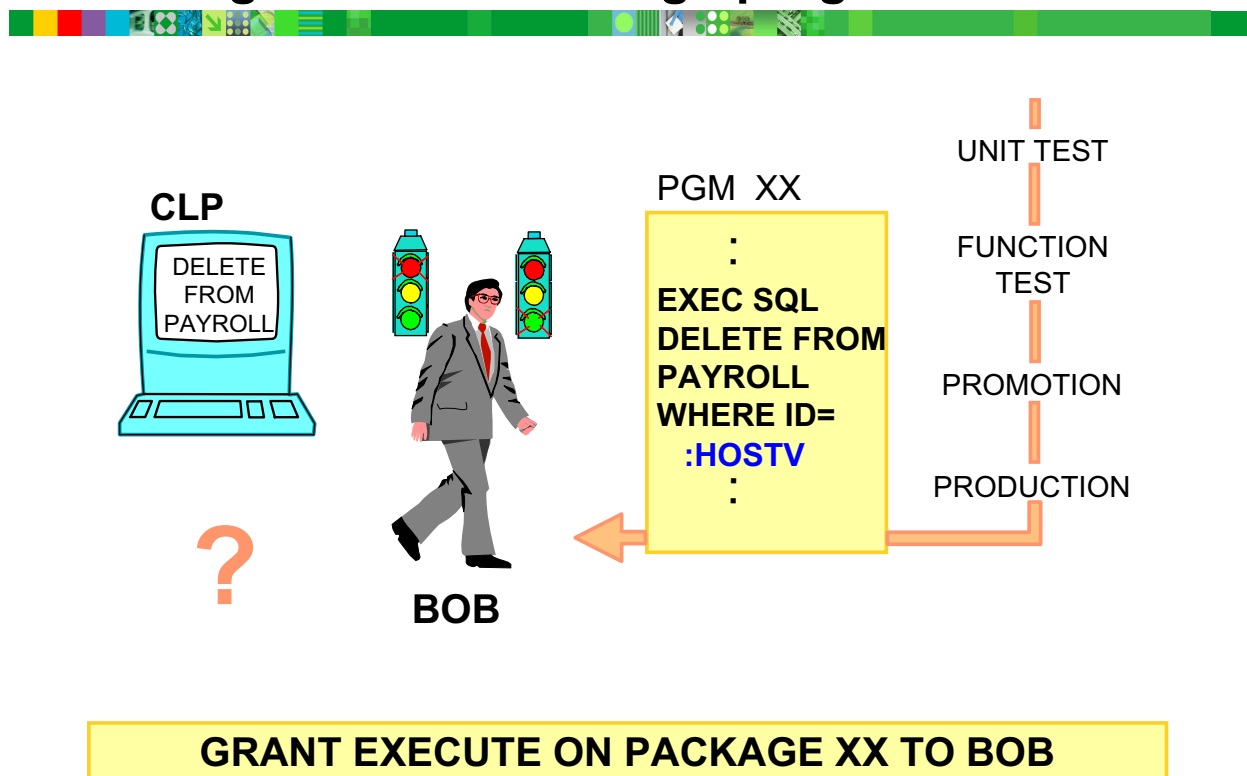
You may want tighter control over schema names than this allows; in that case, you should revoke IMPLICIT\_SCHEMA from PUBLIC. Then you can explicitly grant IMPLICIT\_SCHEMA to users who should have it.

One more item: if you revoke IMPLICIT\_SCHEMA from PUBLIC, then if a user's schema name does not exist, they will get an error when they try to create an object that is implicitly or explicitly qualified with their userid. So, if JoeBlow tries to create *MYVIEW*, he will get an error because the JOEBLOW schema doesn't exist. JoeBlow could explicitly create the schema with a CREATE SCHEMA JOEBLOW statement; you cannot take away the authority of a user to create a schema with an ID the same as their authorization ID.

**Additional Information** — Not all objects have two part names; for example, table spaces do not.

**Transition Statement** — Let's look at how programs can be used to protect resources.

## Protecting resources through programs



© Copyright IBM Corporation 2007

Figure 11-7. Protecting resources through programs

CF238.3

### Notes:

An individual can EXECUTE a package without having the authority for the underlying SQL statements.

Programs can be coded, tested, and installed to perform database tasks on behalf of the program executor.

The authority to EXECUTE a program which performs some SQL statement does not imply authority to perform the same SQL in an adhoc environment, such as CLP or Lotus Approach.

## ***Instructor notes:***

**Purpose** — Identify privileges through programs.

**Details** — One of the reasons that static embedded programs were so popular in the mainframe world was that they could be used to control what a user could do. For example, on the graphic, there is a program that deletes rows from the PAYROLL table, within the bounds of the program, allowing this to happen where ID equals a host variable. The program has been well tested and placed into production. BOB is given execute privilege on the package. This means that he can delete from the PAYROLL table, but only within the bounds of the program. He could not use a dynamic SQL statement, say through CLP, to delete rows from the PAYROLL program.

If you provide access to your database through dynamic SQL (JDBC, ODBC, CLI, and so on), the users who are executing the SQL will need direct access to the database objects. Theoretically, they could execute any delete statement against the table - not subject to bounds such as **ID = :HOSTV**.

**Additional Information** —

**Transition Statement** — Let's look at how we give privileges.

## 11.3 Explicit privileges

### Instructor topic introduction

**What students will do** — Examine explicit database privileges.

**How students will do it** — Lecture and lab.

**What students will learn** — Explicit privileges.

**How this will help students on their job** — Control access to the database through explicit database privileges.

## Explicit authorization



|                            |                       |                                 |                     |                        |                                      |
|----------------------------|-----------------------|---------------------------------|---------------------|------------------------|--------------------------------------|
| <b>* GRANT/<br/>REVOKE</b> | Database privileges   | ON DATABASE                     | <b>TO/<br/>FROM</b> | <b>USER/<br/>GROUP</b> | <b>userid<br/>groupid<br/>PUBLIC</b> |
|                            | Package privileges    | ON PACKAGE<br>package_name      |                     |                        |                                      |
|                            | Table/view privileges | ON TABLE<br>table/view_name     |                     |                        |                                      |
|                            | CONTROL               | ON INDEX<br>index_name          |                     |                        |                                      |
|                            | Schema privileges     | ON SCHEMA<br>schema_name        |                     |                        |                                      |
|                            | USE                   | OF TABLESPACE<br>tablespacename |                     |                        |                                      |

*\* must be SYSADM, DBADM, or have CONTROL on object*

© Copyright IBM Corporation 2007

Figure 11-8. Explicit authorization

CF238.3

### Notes:

The privileges and authorities can be obtained either implicitly or explicitly. You should be aware of all the ways users can obtain access to objects in a database.

Privileges and DBADM authority may be explicitly GRANTED to users or PUBLIC by someone who holds a higher authority or privilege (SYSADM, DBADM, or CONTROL). When granting database privileges, the name of the database is not specified, because you must be CONNECTed (using) a specific database in order to execute SQL statements. Therefore, DB2 knows to which database you are connected.



**Instructor notes:**

**Purpose** — Identify how privileges are granted.

**Details** — Privileges are granted through the GRANT statement; they are removed through the REVOKE statement. Most of the statements are very straightforward, for example GRANT SELECT ON TABLE mytab TO USER keith. There are two that warrant looking at more closely:

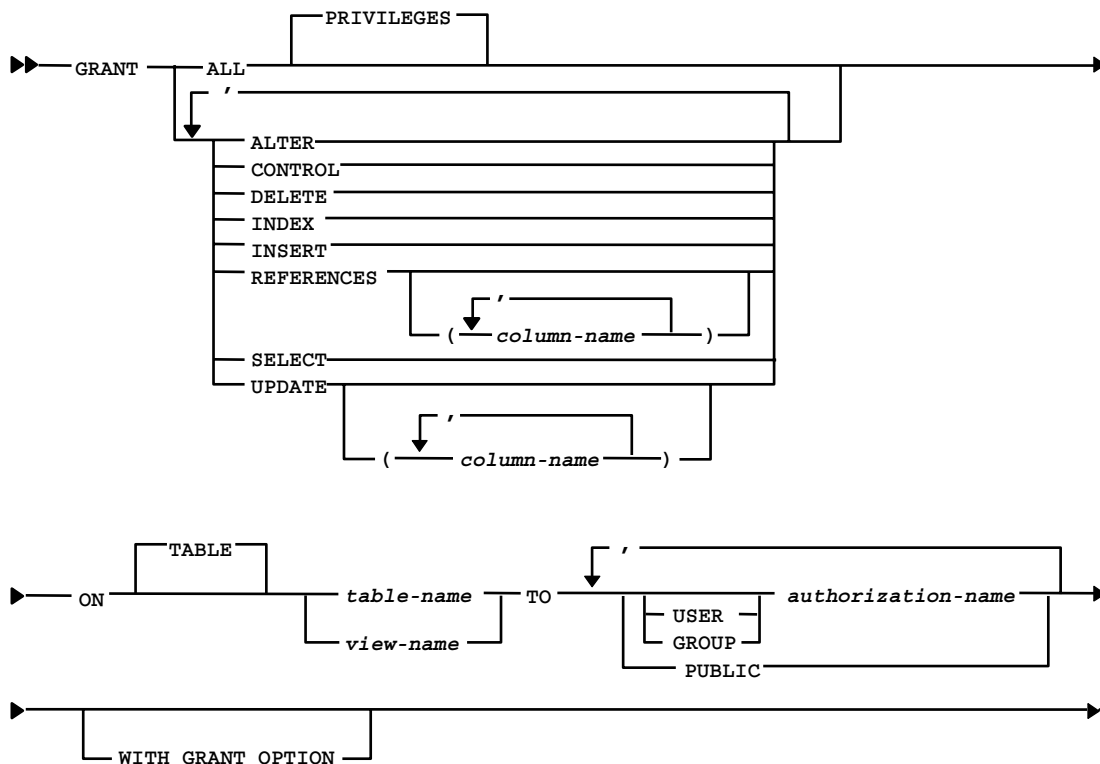
First, database privileges. You would just say, GRANT CONNECT ON DATABASE TO USER keith. No need to name the database. How does DB2 know what database you are granting CONNECT privilege to? The one you are connected to. If you indicate GRANT CONNECT ON DATABASE musicdb TO USER keith, DB2 will return an error, indicating that it doesn't need *musicdb*.

The second one that is weird is the table space privileges. Instead of granting USE \*ON\* a table space, you grant USE \*OF\* a table space. If you specify ON instead of OF, you will get an error message returned.

**Additional Information** —

**Transition Statement** — Let's look at the syntax for one GRANT statement.

## Grant statement — table / view privileges syntax



© Copyright IBM Corporation 2007

Figure 11-9. Grant statement — table / view privileges syntax

CF238.3

### Notes:

The **WITH GRANT OPTION** is available on the **GRANT** statement to indicate that the privilege can also be granted by the grantee to other users.

The ability to grant privileges at the column level is available on the **GRANT** statement for **UPDATE** (to indicate that the grantee can only update certain columns) or for **REFERENCES** (to indicate that the grantee can only define a referential constraint for certain columns).

The following example shows the contents of the table authorizations after using the **GRANT** option in various formats.

```
USER PATTI IS SYSADM.
```

```
grant all on pay.tb1 to cal
```

```
select * from syscat.tabauth where grantee='CAL'
```

The table below shows that CAL does NOT have CONTROL on TB1. CAL can NOT grant privileges to other users.

| Grantor | Grantee | Schema | Tabname | Control | Alter | Delete | Index | Insert | Select | Ref | Update |
|---------|---------|--------|---------|---------|-------|--------|-------|--------|--------|-----|--------|
| PATTI   | CAL     | PAY    | TB1     | N       | Y     | Y      | Y     | Y      | Y      | Y   | Y      |

```
grant select,insert on pay.tb2 to JON with grant option
select * from syscat.tabauth where grantee='JON'
```

| Grantor | Grantee | Schema | Tabname | Control | Alter | Delete | Index | Insert | Select | Ref | Update |
|---------|---------|--------|---------|---------|-------|--------|-------|--------|--------|-----|--------|
| PATTI   | JON     | PAY    | TB2     | N       | N     | N      | N     | G      | G      | N   | N      |

```
grant control on pay.tb3 to mel
select * from syscat.tabauth where grantee='MEL'
```

| Grantor | Grantee | Schema | Tabname | Control | Alter | Delete | Index | Insert | Select | Ref | Update |
|---------|---------|--------|---------|---------|-------|--------|-------|--------|--------|-----|--------|
| PATTI   | ME:     | PAY    | TB3     | Y       | G     | G      | G     | G      | G      | G   | G      |

## ***Instructor Notes:***

**Purpose** — Look at the GRANT statement for tables.

**Details** — The GRANT syntax for table and view privileges is shown on the graphic. A few things to note:

- If you grant ALL, you get everything but CONTROL
- If you grant CONTROL, you get all privileges, WITH GRANT OPTION
- You can grant REFERENCES and UPDATE privileges just to certain columns of the table
- You can specify WITH GRANT OPTION on any grant

Look at the examples in the student notes. Take the students through each step and the results as seen in the SYSCAT.TABAUTH output.

**Additional Information** — It might be interesting to use this example in class, but using your students' names.

A real “Swiss” example (which happened to a company here in Zurich).

In a company there is an employee, Jiri, who is responsible for all reports for the employee department. At one time, he has to go on vacation, and he asks his deputy, Bernhard, to make the reports for him. Bernhard says: “No problem, but I may be on a business trip during your vacation, so how do we want to solve this?”

“No problem, I grant you the select on the table with grant option, so you can grant it to your deputy”, and Jiri grants:

```
grant select on table employee to bernhard with grant option
```

As soon as Jiri leaves for vacation, Bernhard makes a select on the employee table to see the salaries of all the employees and immediately he calls his friend Rob: “Hey Rob, I see you have a bigger salary than your boss, do you know that?”

Rob says: “No, how should I?”

Bernhard: “No problem, I give you the select on the employee table”, and he writes:

```
grant select on table jiri.employee to Rob with grant option
```

(He adds “with grant option” just because he has no idea if this is a must or not, and he saw this command when Jiri wrote it.)

Rob immediately makes a select and one thing he see is that his girlfriend working for the same company earns more money then she was talking about. He gives her a call: “Hey, Julie, you have a higher salary than you told me.”

Julie: “How do you know that?”

Rob: “Well I have a select on the employee table; do you want to have a look?”

Julie: “Yeeees”

Rob: `grant select on table jiri.employee to julie with grant option`

Julie looks at the salaries and decides to make this transparent for the whole company. As she was in an SQL workshop she knows what to do:

`grant select on table jiri.employee to public`

You can imagine what happens: a big disaster!

When Jiri comes back from vacation, the executives ask him to revoke the select privileges from everybody. What does Jiri have to do?

He has to figure out who has the select privilege, and revoke them one by one. It's not enough to revoke Bernhard's privilege, because DB2 doesn't cascade revokes. So there is a lot of work for him to do.

Well, the example mentioned really happens (even with other persons).

Otherwise we can mention:

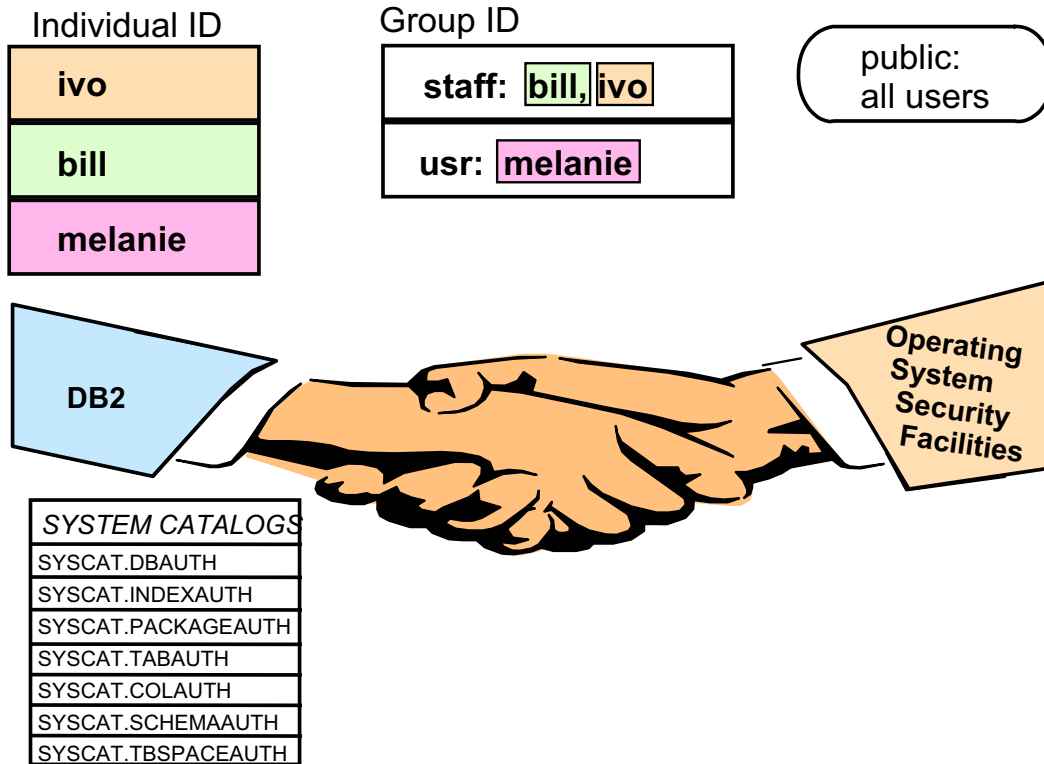
- A1 grants a select to the table xy to A2 with grant option
- A2 grants this select to A3 with grant option
- A3 grants this select to A4 with grant option

What happens if A1 revokes the select privilege from A2?

Answer: A2 no longer has the right to select data from table xy, but A3 and A4 are still allowed to do so. DB2 doesn't cascade revokes! So if you want revoke the select from A3 and A4 you have to do so explicitly. In practice, this might be a lot of work, because you should first find out who all the persons are that have now the select, based on the grant from A2.

**Transition Statement** — Let's look at how user IDs and group IDs are defined.

## Individual IDs, group IDs, and public



© Copyright IBM Corporation 2007

Figure 11-10. Individual IDs, group IDs, and public

CF238.3

### Notes:

Privileges may be explicitly granted to individual IDs, group IDs, or PUBLIC.

Individual IDs are created through the operating system's security facilities.

A group ID consists of one or more individual IDs. If an individual ID is a member of a group, then that individual ID automatically has privileges granted to the group ID. Groups are created through the operating system's security facilities.

**DB2 has a special group ID: PUBLIC.** It is not created with the operating system's security facilities. All users accessing a database are automatically members of the group PUBLIC. Granting a privilege to PUBLIC grants that privilege to all users of a database.

**Instructor Notes:**

**Purpose** — Identify the relationship between the operating system groups and DB2, and define PUBLIC.

**Details** — You can grant privileges in DB2 to groups. The membership of the group is defined outside DB2 in the operating system security facilities. There is a *handshake* between DB2 and the Operating System Security Facilities; DB2 goes to the operating system to find out who the group members are.

PUBLIC is a special group to DB2. DB2 does not go to the operating system to find out who is a member of PUBLIC; all users on the system are considered to be members of the PUBLIC group.

Definitions of privileges granted are stored in the system catalog tables. The views are listed on the graphic, and they will be identified again at the end of the unit.

**Additional Information** — Certification objective: 7C Ability to set user and/or group privileges.

**Transition Statement** — Let's look at group and user ID support.

## Using group IDs

| SQL | Permitted on |                          | Does the System Know About? |             |
|-----|--------------|--------------------------|-----------------------------|-------------|
|     | UNIX         | OS/2<br>or<br>Windows NT | User - cal                  | Group - cal |
| 1   | ↗            | ↗                        | ↗                           |             |
| 2   | ↗            | ↗                        |                             | ↗           |
| 3   | ↗            | N/A                      | ↗                           | ↗           |



Group - cal



User - cal

1 GRANT SELECT ON TABLE EMPLOYEE TO CAL  
- or -  
GRANT SELECT ON TABLE EMPLOYEE TO USER CAL

2 GRANT SELECT ON TABLE EMPLOYEE TO CAL  
- or -  
GRANT SELECT ON TABLE EMPLOYEE TO GROUP CAL

3 ✗ GRANT SELECT ON TABLE EMPLOYEE TO CAL → SQLCODE -569

© Copyright IBM Corporation 2007

Figure 11-11. Using group IDs

CF238.3

### Notes:

Some operating systems allow the same name to be created for a userid and a group ID. This can cause problems for authorization and privilege checking within DB2. The GRANT and REVOKE SQL statements provide an optional parameter which is used to indicate if the privilege is intended for a userid or a group ID.

Consider the above scenarios:

1. A user called cal exists on the operating system. The GRANT command shown can explicitly use the TO USER clause in the statement. If the TO USER clause is not specified, userid cal will get the SELECT privilege.
2. Only a group called cal exists on the operating system. The GRANT command can explicitly use the TO GROUP clause. If not, because there is no userid cal, the group will receive the privilege.
3. UNIX allows a user and group to have the same name. In UNIX, the following would occur:



- If the GRANT does not specify either TO USER or TO GROUP, the grant will fail with a -569 SQLCODE.
- If the GRANT specifies TO USER or TO GROUP, the exact receiver of the privilege will be specified.

If a privilege has been granted to both a userid and a group ID with the same name, you must specify the GROUP or USER keyword when revoking the privilege. Revoking a privilege from a group may not revoke it from all members of the group. If an individual ID has been directly granted a privilege, it will keep it until that privilege is directly revoked.

## ***Instructor Notes:***

**Purpose** — Identify assumptions that DB2 will and won't make.

**Details** — When you grant a privilege, if only a user or only a group exists with the name you grant TO, then the USER or GROUP keyword is optional.

For example, in the first case, only a user named CAL exists. Either of the grant statements illustrated would succeed in granting the user CAL the SELECT privilege.

In the second example, only a group named CAL exists. Either of the grant statements illustrated would succeed in granting the group CAL the SELECT privilege.

The last example will only work on UNIX operating systems. In the example, both a user CAL and a group CAL exist. If we try to issue the statement shown at the bottom of the graphic, a negative SQLCODE, -569, would be returned, in which case you must indicate specifically if you are giving the privilege to the user CAL or to the group CAL. This is to prevent DB2 making incorrect assumptions about what you might want.

### **Additional Information —**

**Transition Statement** — Let's look at some special considerations for PUBLIC and other group privileges with packages and views.

# Public privilege support for static SQL and views



## Mel is a member of group1

```
db2 grant select on table T1 to group group1
db2 grant update on table T1 to user mel
db2 grant insert on table T1 to public
db2 grant select on table T2 to public
```



### Mel attempts bind of pgm.sqc

```
exec sql insert into T1...
exec sql update T1 set...
exec sql select... from T1...
```

OK  
OK  
SQL bind fails

### Mel attempts to create a view

```
create view V2 as select C3 from T2
create view V1 as
select C1, C8 from T1
```

OK  
SQL create view fails

© Copyright IBM Corporation 2007

Figure 11-12. Public privilege support for static SQL and views

CF238.3

## Notes:

Privileges granted to a group will **not** be considered when checking static SQL statements and creating views. However, privileges granted to PUBLIC **will** be considered when checking static SQL statements and creating views.

The graphic illustrates the following points. Mel is a member of group1. When Mel attempts the bind of pgm.sqc:

- The insert to T1 is valid because insert on T1 has been granted to PUBLIC.
- The update of T1 is valid because update on T1 has been granted to user MEL.
- The select statement from T1 is NOT valid because select on T1 has not been granted to user MEL or to PUBLIC.

When Mel attempts the create of view V2:

- The select of column C3 from T2 is valid because select on T2 has been granted to PUBLIC.

When Mel attempts the create of view V1:

- The select of columns C1 and C8 from T1 is NOT valid because select on T1 has not been granted to user MEL or to PUBLIC.

## ***Instructor Notes:***

**Purpose** — Identify special considerations with packages and views and group privileges.

**Details** — Packages and views are unique objects in DB2 in that privileges need to exist when the object is created, but the privilege also needs to continue to exist for the object to continue to be functional in the database. For example, you must have `CREATETAB` on the database to create a table, but you don't have to continue to have `CREATETAB` for the table to continue to exist and be usable. For packages and views, the privileges that are required on the underlying objects must continue to exist.

For packages, you must have the ability to execute all of the underlying SQL statements for a static embedded SQL program; those privileges must be granted to your userid or to public. For views, you must have the ability to select from the underlying tables; again, the privilege must have been given to your userid or to public for the `CREATE VIEW` to succeed.

Let's look at the example. Mel is a member of `GROUP1`. We have granted `select` on the table `T1` to `GROUP1`, `update` on the table `T1` to `MEL`, and `insert` on the table `T1` to `PUBLIC`. We have also granted `SELECT` on the table `T2` to `PUBLIC`.

Let's look at the package: we have an embedded SQL program that contains an `insert` to `T1`, an `update` of `T1`, and a `select` from `T1`. If Mel attempts to bind this program, it will fail, because she does not have `select` on `T1` given to her userid or to public.

In a similar way, Mel would be able to create a view that is based on selecting from `T2`, but not selecting from `T1`.

Now, you may ask, "Why?". Here is my answer: These are long-lived objects that need to have the underlying privilege continue to exist for the object to continue to exist. Let's say that `V2` was successfully created, and we later revoked the `select` privilege on `T2` from public. The reason that view got created was due to that privilege, so the view would be marked as inoperative, and would no longer be usable because the underlying privilege no longer existed. In a similar way, if the package had only had the first two statements, it would have successfully been bound. If we had subsequently revoked `update` on `T1` from Mel, then the package would have been marked invalid, and would be no longer executable. The privilege needs to continue to exist.

Now, what would happen if DB2 looked at Mel's group membership in `GROUP1` and let the package get bound? How could Mel later lose the `select` privilege on `T1`?

Answer: By being removed from the group.

How could she be removed from the group? Is that a DB2 function?

No; it is done through the operating system.

So, when the system administrator performs maintenance on the membership of `GROUP1`, does the operating system knock on DB2's door (I usually knock on the board, which gets the students' attention!) and say, "Hey, DB2, here is some information you might be interested in!" The answer is of course, no, the operating system does NOT communicate

that to DB2. So, if DB2 allowed group privileges to be checked, then potentially, the underlying privilege could be lost without DB2's knowledge, and the object would be able to continue to exist when it should have been marked invalid or inoperative.

**Additional Information —**

**Transition Statement** — Let's look at privileges required for programming.

## Privileges required for programming



### *Action*

### *Privileges Required*

|                               |                                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------------|
| Precompile to bindfile        | CONNECT on database                                                                                |
| Create a new package          | CONNECT on database<br>BINDADD on database<br>Privileges need to execute each static SQL statement |
| Modify an existing package    | CONNECT on database<br>BIND on package<br>Privileges need to execute each static SQL statement     |
| Re-create an existing package | CONNECT on database<br>BIND on package                                                             |
| Execute a package             | CONNECT on database<br>EXECUTE on package                                                          |
| Drop a package                | CONNECT on database<br>CONTROL on package or creator of package                                    |

© Copyright IBM Corporation 2007

Figure 11-13. Privileges required for programming

CF238.3

### **Notes:**

This chart summarizes the authorities and privileges needed to prepare, bind, modify, execute, or drop a package.

Simple references are qualified with the ID of the binder.

At prepare time, simple references are qualified with the ID of the preparer for the purpose of determining authorization for doing the prepare.

The binder **must** have authorization, granted to their userid or to PUBLIC, to perform all underlying SQL statements at bind time. The preparer is not required to have the underlying SQL authorizations to create a bind file; warnings will be generated if the user ID doing the prepare does not have the authorizations to perform all underlying SQL statements. Also, all objects must exist in the database against which the prepare and the bind are being executed.

***Instructor Notes:***

**Purpose** — Identify the privileges required for programming.

**Details** — Go through each line and indicate the privileges required. Note that **Modify an existing package** means to run a bind command to create a package that overlays one that is already there. **Recreate an existing package** means to run a rebind command.

**Transition Statement** — Let's look at some information about security and dynamic SQL.

## Dynamic rules and dynamic SQL

- Jill issues:

```
db2 bind myapp.bnd qualifier u1 owner u2
dynamicrules run
```

- Unqualified SQL uses u1 schema
- u2 owns package
- Privileges of user executing package checked for
- Dynamic SQL
 

```
db2 bind myapp.bnd qualifier u1 owner u2
dynamicrules bind
```
- Privileges of u2 checked for dynamic SQL

© Copyright IBM Corporation 2007

Figure 11-14. Dynamic rules and dynamic SQL

CF238.3

### Notes:

With the default settings, the owner of a package is the auth ID used to bind the package (in the example, Jill), and that user ID is also used to qualify unqualified references, whether those unqualified references are found in dynamic or static SQL.

Use of the QUALIFIER option allows you to specifically indicate the qualifier to be used for both dynamic and static SQL.

Use of the OWNER option allows you to specifically indicate the user ID to be used for the package owner. Also, the authorization for the static SQL statements in this package is checked against this ID. Only a user with SYSADM or DBADM authority can specify an authorization identifier other than the user ID.

Use of DYNAMICRULES allows you to indicate whether the package user's privileges (the privileges of the user ID executing the package) are checked for dynamic SQL (default, DYNAMICRULES RUN), or whether the implicit or explicit value of the owner of the package should be checked for dynamic SQL (DYNAMICRULES BIND). Note that the value of the CURRENT SCHEMA special register will have no impact on the qualification of SQL in a package bound with DYNAMICRULES BIND.



For dynamic applications:

- To precompile to a bind file, CONNECT on database is required.
- To create a new package, CONNECT on database and BINDADD on database are required.
- To modify an existing package, CONNECT on database and BIND on package are required.
- To recreate an existing package, CONNECT on database and BIND on package are required.
- To execute a package, CONNECT on database and EXECUTE on package are required. In addition, if the package was bound with DYNAMICRULES RUN, privileges needed for executing each dynamic SQL statement are required. If the package was bound with DYNAMICRULES BIND, the binder would require privileges for executing each dynamic SQL statement.
- To drop a package, CONNECT on database and CONTROL on package or creator of package are required.

## ***Instructor Notes:***

**Purpose** — Identify the privileges required for programming.

**Details** — Here are some interesting options for binding packages. First, the qualifier indicates what the default qualifier should be for unqualified SQL in the package (static SQL). OWNER indicates the owner of the package.

DYNAMICRULES is an interesting BIND option. This option allows you to indicate whether the executor of the package must hold the privileges to run the dynamic SQL issued from within it, or if the privilege checking goes back to the binder (DYNAMICRULES BIND).

To illustrate how powerful or dangerous this could be, imagine that as DBADM, you bind the bind file for CLP against your database with DYNAMICRULES BIND. That would say that anyone using that program (CLP) would have the privileges that you have as DBADM for issuing dynamic SQL. This could be a scary prospect, but could be powerful too.

**Transition Statement** — Let's look at the BIND GRANT option.

## DB2 BIND GRANT option



RICK has  
BINDADD  
and  
underlying  
SQL  
privileges

db2 connect to sample  
db2 prep packa.sqc bindfile

db2 bind packa.bnd grant

[  
authid  
groupid  
public  
]

OR



SALLY  
has  
CONTROL  
on packa

grant execute, bind on  
package packa

to

[  
authid  
groupid  
public  
]

© Copyright IBM Corporation 2007

Figure 11-15. DB2 BIND GRANT option

CF238.3

### Notes:

The `db2 bind` command `grant` option requests that package EXECUTE and BIND privilege be granted to an additional userid or PUBLIC.

If the GRANT option is used when binding a package at a local DB2 database, both EXECUTE and BIND privileges are granted. If the GRANT option is used when binding a package at a remote DRDA AS database, only the EXECUTE privilege is granted.

## ***Instructor Notes:***

**Purpose** — Identify the GRANT option on the BIND command.

**Details** — In addition to using the GRANT statement explicitly, you can use the GRANT option on the BIND command. This will grant EXECUTE and BIND to the userid, groupid, or PUBLIC: whatever you specify.

**Additional Information** —

**Transition Statement** — Let's look at implicit privileges.

## 11.4 Implicit privileges

### Instructor topic introduction

**What students will do** — Examine implicit database privileges.

**How students will do it** — Lecture and lab.

**What students will learn** — Implicit privileges.

**How this will help students on their job** — Control access to the database through implicit database privileges.

## Implicit privileges (1 of 2)

### ■ Create database

- Internal GRANT of DBADM authority with BINDADD, CONNECT, CREATETAB, CREATE\_EXTERNAL\_ROUTINE, CREATE\_NOT\_FENCED\_ROUTINE, IMPLICIT\_SCHEMA, LOAD, and QUIESCE\_CONNECT privileges to creator (SYSADM or SYSCTRL)
- Internal GRANT of BINDADD, CREATETAB, CONNECT and IMPLICIT\_SCHEMA to PUBLIC
- BIND and EXECUTE privilege on each successfully bound utility to PUBLIC
- SELECT on system catalog tables and views to PUBLIC
- USE privilege on USERSPACE1 table space to PUBLIC
- EXECUTE WITH GRANT privilege to PUBLIC on all functions in SYSFUN schema; EXECUTE privilege to PUBLIC on all procedures in SYSIBM schema

© Copyright IBM Corporation 2007

Figure 11-16. Implicit privileges (1 of 2)

CF238.3

### **Notes:**

The privileges and authorities can be obtained either implicitly or explicitly. You should be aware of all the ways users can obtain access to objects in a database.

Some privileges are granted to users automatically (implicit) by DB2. Be aware of these internal GRANTS, so that you can REVOKE the privileges if needed.

When a user creates a database via SYSADM or SYSCTRL authorization, the user is granted DBADM authority for the database. If a user has their SYSADM or SYSCTRL authority removed, you must explicitly revoke DBADM authority for each database that this user created if you wish to prevent them from accessing a database as a DBADM. The creator is granted the privileges that would be given if DBADM had been granted explicitly.

***Instructor Notes:***

**Purpose** — Identify implicit privileges.

**Details** — Go through each point on the graphic. These are the privileges given implicitly. You can't change the fact they are given implicitly, but you can revoke them after the object has been created.

**Additional Information** —

**Transition Statement** — More implicit privileges next.

## Implicit privileges (2 of 2)

- GRANT DBADM
  - Internal GRANT of BINDADD, CONNECT, CREATETAB, CREATE\_EXTERNAL\_ROUTINE, CREATE\_NOT\_FENCED\_ROUTINE, IMPLICIT\_SCHEMA, LOAD and QUIESCE\_CONNECT
- Create object (table, index, package)
  - Internal GRANT of CONTROL to object creator
- Create view
  - Internal GRANT to intersection of creator's privileges on base tables to view creator

© Copyright IBM Corporation 2007

Figure 11-17. Implicit privileges (2 of 2)

CF238.3

### Notes:

When a user is granted DBADM authority on a database, the user is also granted BINDADD, CONNECT, CREATETAB, CREATE\_EXTERNAL\_ROUTINE, CREATE\_NOT\_FENCED\_ROUTINE, IMPLICIT\_SCHEMA, LOAD, and QUIESCE\_CONNECT privileges automatically. **If the DBADM authority is later revoked, the other privileges remain in effect unless they are separately revoked.**

When a user is granted CONTROL privilege on a table, the user is also granted all of the other table privileges automatically. **If CONTROL privilege is later revoked, the other privileges will remain in effect unless they are separately revoked. You may also use the ALL option to do so.**

Privileges granted to the creator of a view will never exceed the base table authority the creator possesses. As an example, consider two users, A and B.

A owns table ATABLE.

B has the SELECT authority on ATABLE.

A creates view AVIEW }



```
        } both views use SELECT * FROM ATABLE
        } WHERE (some condition) as the view definition
B creates view BVIEW }
```

User A could SELECT from AVIEW, as well as INSERT, UPDATE, and DELETE, through the view AVIEW. In addition, user A could GRANT authority against AVIEW since user A has CONTROL of the view.

User B could SELECT from BVIEW, but that is all. User B did not have INSERT, UPDATE, and DELETE against the table and did not have the authority to GRANT against the base table. The view BVIEW **DOES NOT** provide privileges beyond those that exist on the base table.

The privileges needed to create a view must be explicitly granted to the user or granted to PUBLIC. Group privileges, except for PUBLIC, are not used when a user's privileges to create a view are checked.

## ***Instructor notes:***

**Purpose** — Identify implicit privileges.

**Details** — Go through each point on the graphic. These are the privileges given implicitly. You can't change the fact they are given implicitly, but you can revoke them after the object has been created.

### **Additional Information** —

**Transition Statement** — Once you go through this page, say, "Does that all make sense?" If students say yes, then say, "Oh good, then it is time for a quiz!"

## Implicit privileges scenarios

- Scenario 1.
  - `ivo` is placed in SYSADM group.
  - `ivo` creates database DB1
  - `ivo` is removed from SYSADM group.
  - What privileges does `ivo` retain?
  
- Scenario 2.
  - `db2 grant dbadm on database to user mel`
  - `db2 revoke dbadm on database from user mel`
  - What privileges does `mel` retain?
  
- Scenario 3.
  - `db2 grant control on table t1 to user bill`
  - `db2 revoke control on table t1 from user bill`
  - What privileges does `bill` retain?
  - Write a command to revoke `bill`'s remaining privileges.

© Copyright IBM Corporation 2007

Figure 11-18. Implicit privileges scenarios

CF238.3

### Notes:

Can you solve the scenarios?

## ***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — Answers to scenarios:

1. Since ivo created a database before he was removed from the SYSADM group, he retains BINDADD, CONNECT, CREATETAB, CREATE\_EXTERNAL\_ROUTINE, CREATE\_NOT\_FENCED\_ROUTINE, IMPLICIT\_SCHEMA, LOAD, and QUIESCE\_CONNECT privileges on the DB1 database. These privileges must be explicitly revoked. A user in the SYSCTRL group can also create a database and the same would be true.
2. When DBADM is granted to a user, the user is implicitly granted BINDADD, CONNECT, CREATETAB, CREATE\_EXTERNAL\_ROUTINE, CREATE\_NOT\_FENCED\_ROUTINE, IMPLICIT\_SCHEMA, LOAD, and QUIESCE\_CONNECT on the database DB1. Mel will continue to have those privileges even after DBADM is revoked.
3. When CONTROL is granted to a user, the user is implicitly granted ALTER, DELETE, INDEX, INSERT, REFERENCES, SELECT, and UPDATE on the table.

These privileges may be explicitly dropped with the following SQL statement:

```
db2 revoke all on table t1 from bill
```

**Transition Statement** — Let's look at the next set of questions.

## Grant / Revoke scenarios

### ■ Scenario 1.

```
db2 create view v2 as select name, pay from t1 where dept=50
db2 grant select on view v2 to user doug
db2 grant select on table t1 to user pres
What is difference between doug's & pres's privileges?
```

### ■ Scenario 2.

```
db2 grant select on table tab1 to user gerhard
db2 grant select on table tab1 to group denmark
Who can create a view with tab1 as the base table?
db2 revoke select on table tab1 from group denmark
If gerhard in denmark group, may he select from tab1?
```

### ■ Scenario 3.

```
db2 grant select on table tab1 to mel
Then mel creates view:
  db2 create view v1 as select * from tab1
db2 revoke select on table tab1 from mel
What happens to the view v1 and packages that specify v1?
```

© Copyright IBM Corporation 2007

Figure 11-19. Grant / Revoke scenarios

CF238.3

### **Notes:**

Can you answer the above scenarios?

## ***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — The following are the answers to the GRANT/REVOKE scenarios.

1. State that the table T1 has more than the NAME and PAY columns. T1 has more departments than DEPT=50. Doug can only look at DEPT 50 and the NAME and PAY columns. Pres can look at all departments and all columns in T1. Note that a view can be used as a way to grant security.
2. Only Gerard can create a view. To create a view requires SYSADM, or DBADM, or explicit CONTROL, or explicit SELECT on the base table TAB1 to be granted to Gerard or public, but not a group ID. Even though the select authority is revoked from group denmark, Gerard can still select from tab1 since the userid was explicitly granted the authority in a separate grant. If an individual userid has been directly granted a privilege, it will keep it until the privilege is directly revoked.
3. When the privilege used to create a view is revoked, the view will become **inoperative**. To recover an inoperative view, determine the SQL statement that was initially used to create the view. You can obtain this information from the TEXT column of the SYSCAT.VIEW catalog view. Recreate the view by using the CREATE VIEW statement with the same view name and same definition. Use the GRANT statement to regrant all privileges that were previously granted on the view. Privileges granted on the inoperative view were revoked.

The package will be marked as invalid. Packages in an invalid state are implicitly rebound the next time they are accessed. They can also be explicitly rebound.

**Transition Statement** — Let's look at how you can query privileges.

## 11.5 Querying privileges/authorities

### Instructor topic introduction

**What students will do** — Query database privileges and authorities.

**How students will do it** — Lecture and lab.

**What students will learn** — How to query database privileges and authorities.

**How this will help students on their job** — They will be able to identify users' database privileges and check on their authorities.

## Query who has privileges

- Most of the information on authorizations is maintained in seven system catalog tables:

|                           |                                  |
|---------------------------|----------------------------------|
| <i>SYSCAT.DBAUTH</i>      | Database privileges              |
| <i>SYSCAT.COLAUTH</i>     | Table and View Column privileges |
| <i>SYSCAT.INDEXAUTH</i>   | Index privileges                 |
| <i>SYSCAT.PACKAGEAUTH</i> | Access Package privileges        |
| <i>SYSCAT.SCHEMAAUTH</i>  | Schema privileges                |
| <i>SYSCAT.TBAUTH</i>      | Table and view privileges        |
| <i>SYSCAT.TBSPACEAUTH</i> | Table space privileges           |

- SYSADM, SYSCTRL, SYSMAINT, and SYSMON authority and group membership are defined outside the Database Manager and not reflected in catalogs

© Copyright IBM Corporation 2007

Figure 11-20. Query who has privileges

CF238.3

### Notes:

During database creation, SELECT privilege on the system catalog views is granted to PUBLIC. In most cases, this does not present any security problems. For very sensitive data, it may be inappropriate, as these tables describe every object in the database. If this is the case, consider revoking the SELECT privilege from PUBLIC; then grant the SELECT privilege as required to specific users or groups. Granting and revoking SELECT on the system catalog views is done in the same way as for any view, but you must have either SYSADM or DBADM authority to do so.

At a minimum, you should consider restricting access to the SYSCAT.DBAUTH, SYSCAT.COLAUTH, SYSCAT.INDEXAUTH, SYSCAT.PACKAGEAUTH, SYSCAT.SCHEMAAUTH, SYSCAT.TBAUTH, and SYSCAT.TBSPACEAUTH catalog views. This would prevent information on user privileges, which could be used to target an ID for break-in from becoming available to everyone with access to the database.

You should examine the columns for which statistics are gathered. Some of the statistics recorded in the system catalog contain data values which could be sensitive information in your environment. If these statistics contain sensitive data, you may wish to revoke



SELECT privilege from PUBLIC for the SYSCAT.COLUMNS and SYSCAT.COLDIST catalog views.

If you wish to limit access to the system catalog views, you could define views on the views to let each authorization ID retrieve information about its own privileges. This may be done by using a *WHERE .... IN (USER, 'PUBLIC')* predicate.

Privileges granted to users by the system will have SYSIBM as the grantor. SYSADM, SYSMANT, SYSCTRL and SYSMON authorizations are not listed in the system catalog.

## ***Instructor Notes:***

**Purpose** — Identify where security information is stored.

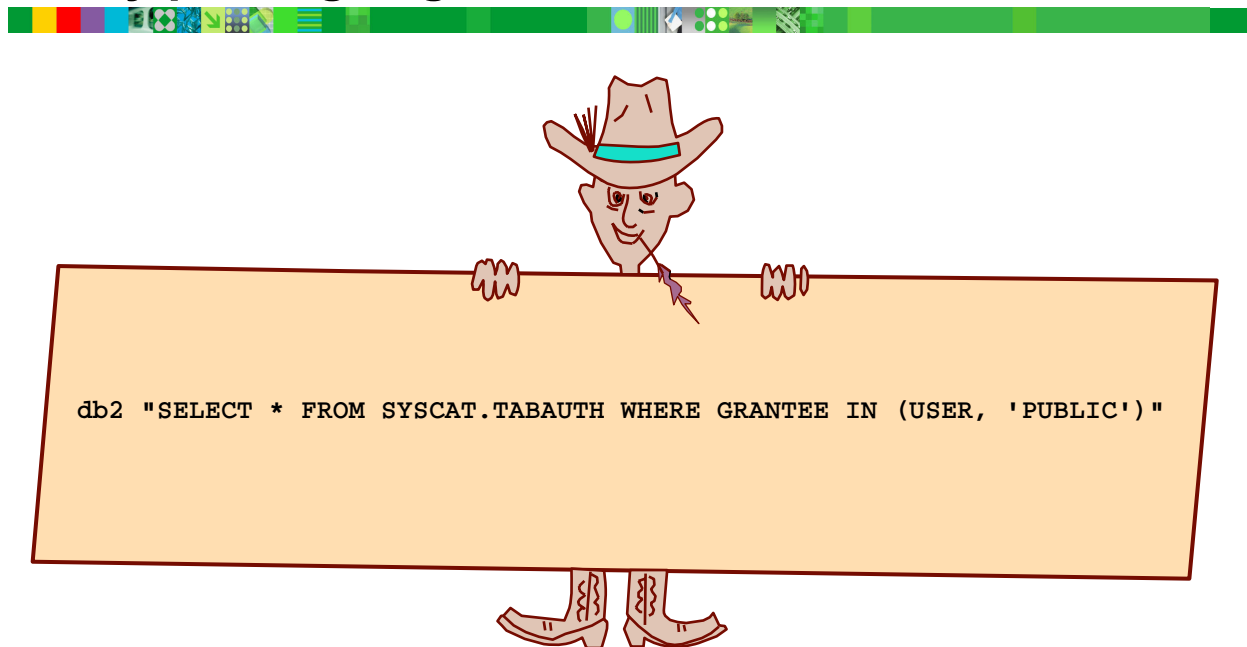
**Details** — Authorization information is stored inside the database in seven tables; the views that allow access to this information are listed.

Please note that SYSADM, SYSCTRL, SYSMANT, and SYSMON, as well as group membership, are defined outside the database manager, and are not reflected in the catalogs.

**Additional Information** —

**Transition Statement** — Let's look at how to query privileges.

## Query privileges granted to current ID



### *Rounding up Privileges*

© Copyright IBM Corporation 2007

Figure 11-21. Query privileges granted to current ID

CF238.3

#### **Notes:**

USER is a keyword that means the current *connection ID*. Do not place USER in quotation marks.

If you had a current operating system login of USER1 and then connected to database DB1 with “CONNECT TO DB1 USER MELANIE USING SCUBA”, USER would equal MELANIE.

**DB2 get authorizations** reports the authority and privileges that you have for the database. You must be connected to the database to get authorizations.

Direct authority or privileges are acquired by explicit commands that grant the authority or privilege to your username. Indirect authority or privileges are through implicit ways; that is, SYSADM has authority over the entire DB2 system.

## ***Instructor Notes:***

**Purpose** — Show an SQL statement for getting privilege information.

**Details** — This will return all the privileges granted to the currently connected userid or to PUBLIC.

**Additional Information** — You may want to mention that you would also need to list any groups that the user is a member of to get a full picture of the users privileges.

**Transition Statement** — Let's look at LBAC — Label Based Access Control capabilities.

## 11.6 Label Based Access Control (LBAC)

### Instructor topic introduction

**What students will do** — Learn the basic concepts of LBAC.

**How students will do it** — Listen to the lecture.

**What students will learn** — The basic concepts of LBAC.

**How this will help students on their job** — Give students the ability to judge whether or not LBAC would be the right choice to protect sensitive data.

## Label Based Access Control (LBAC) overview

- Label-Based Access Control (LBAC) is an implementation of Mandatory Access Control (MAC)
  - In the context of database systems, LBAC is commonly referred to as the ability to control access to data rows based on security labels.
- The DB2 LBAC solution is a *flexible* implementation of MAC at both:
  - The Row level
  - The Column level
- Row level and column level protection are independent and can be used either separately or combined
- Table level labeling can be easily simulated using column level labeling
- LBAC complements the traditional DB2 Discretionary Access Control (DAC)
  - When a protected table is accessed, DB2 enforces two (2) levels of access control:
    1. DAC at the table level, that is, does the user hold the required privilege to perform the requested operation on the table?
    2. LBAC, which can be at the row level, column level or both

© Copyright IBM Corporation 2007

Figure 11-22. Label Based Access Control (LBAC) overview

CF238.3

### Notes:

Label-Based Access Control (LBAC) is an implementation of Mandatory Access Control (MAC). In the context of database systems, LBAC is commonly referred to as the ability to control access to data rows based on security labels.

The DB2 LBAC solution is a flexible implementation of MAC that can provide row level and column level protection. Row level and column level protection are independent and can be used either separately or combined. Table level labeling can be easily simulated using column level labeling for each column in a table.

LBAC complements the traditional DB2 Discretionary Access Control (DAC). For access to tables protected using LBAC functions, DB2 enforces two levels of access control:

- First, the standard Discretionary Access Control permissions are checked. DB2 checks the authorities that have been granted to a user or user groups to see if the access is permitted. For example, does the user have SELECT authority for a table or view?
- Next, DB2 would check for LBAC authorization to access protected rows or columns in a table.

**Instructor notes:**

**Purpose** — To introduce LBAC as an additional level of security checking that complements the standard security checking. LBAC provides methods to protect rows and columns in a table that will be described in this presentation. A key point is that the DBA will still need to GRANT normal SELECT, INSERT, UPDATE and DELETE authority for user's to access tables.

**Details** —

**Additional information** — Certification objective: 7D Knowledge of the DB2 Security Infrastructure (for example, LBAC and Security Plug-ins).

**Transition statement** — Let's look at an important extension to the security model, necessary to support LBAC security.

## Security model extensions — SECADM Authority

- Security Administrator (SECADM) Authority
  - The authority required to
    - Create and drop security label components
    - Create and drop security policies
    - Create and drop security labels
    - Grant and revoke security labels
    - Grant and revoke exemptions
    - Grant and revoke the SETSESSIONUSER privilege
  - SYSADM is the only authority that can grant SECADM
  - SECADM has no inherent ability to access data within a protected table!
  - SECADM cannot grant any privileges to him/her self!
  - SYSADM authority also does not provide access to data that is protected by LBAC.

© Copyright IBM Corporation 2007

Figure 11-23. Security model extensions — SECADM Authority

CF238.3

### Notes:

Implementing Label Based Access Control security requires the use of a new user authority, the Security administration authority, or SECADM. In order to support this new higher level of security, it is necessary to establish one or more users who must perform all maintenance for the new LBAC objects.

SECADM authority can only be granted by the SYSADM user and can be granted to a user but not to a group. It gives these and only these abilities:

- Create and drop security label components
- Create and drop security policies
- Create and drop security labels
- Grant and revoke security labels
- Grant and revoke LBAC rule exemptions
- Grant and revoke setsessionuser privileges
- Execute the SQL statement TRANSFER OWNERSHIP on objects that you do not own

No other authority gives these abilities, not even SYSADM.



In DB2 databases, the SYSADM user has had *unlimited* permission to perform any command or data access. The abilities provided by the SECADM authority are not provided by any other authority. SYSADM authority also does not provide access to data that is protected by LBAC.

The SECADM user has no inherent ability to access data within a protected table, and a SECADM user cannot grant any privileges to him/her self. So the security administrator manages the new LBAC security objects for other users, but does not have any special authority to access data.

## ***Instructor notes:***

### **Purpose —**

**Details —** This explains that part of implementing LBAC security is the assignment of a new type of database user, the security administrator, SECADM. This allows the security for LBAC to be controlled by a specialized security administrator and does not allow the SYSADM user an unlimited access to all data. This is important for some highly secure environments.

**Additional information —** Certification objective: 7D Knowledge of the DB2 Security Infrastructure (for example, LBAC and Security Plug-ins).

**Transition statement —** Let's look at the summary.

# Checkpoint

## Exercise — Unit Checkpoint

- \_\_\_ 1. Can the right to grant table, view, or schema privileges to others be extended to other users using the WITH GRANT OPTION on the GRANT statement?

---

**Yes**

## Unit summary

Having completed this unit, you should be able to:

- Differentiate between authentication and authorization
- Use DB2 access control mechanisms to implement security within the database
- Use group ids to create a control hierarchy
- Describe Label Based Access Control (LBAC)
- Explore privileges in the database
- Describe privileges required for binding and executing a package
- Describe the difference between explicit privileges and implicit privileges
- Describe the different DB2 authorization levels

© Copyright IBM Corporation 2007

Figure 11-24. Unit summary

CF238.3

### Notes:

***Instructor notes:***

**Purpose —**

**Details —**

**Additional information —**

**Transition statement —** Now it's time to complete the Security lab.



# Appendix A. LOBs, UDFs, UDTs, and Relational Extenders

## What This Unit Is About

This unit provides information about LOBs, UDFs, UDTs, and Relational Extenders.

## What You Should Be Able to Do

After completing this unit, you should be able to:

- Describe LOBs, UDFs, UDTs, and Relational Extenders

## References

*Command Reference*

*Administration Guide*

## How You Will Check Your Progress

Accountability:

- Machine lab
- Checkpoint Questions

## Appendix objectives

After completing this appendix, you should be able to:

- Describe LOBs, UDFs, UDTs, and Relational Extenders

© Copyright IBM Corporation 2007

Figure A-1. Appendix objectives

CF488.3

### **Notes:**



***Instructor Notes:***

**Purpose** — Identify objectives for the unit.

**Details** — Level set.

**Transition Statement** —

## **A.1. LOBs**

### **Instructor Topic Introduction**

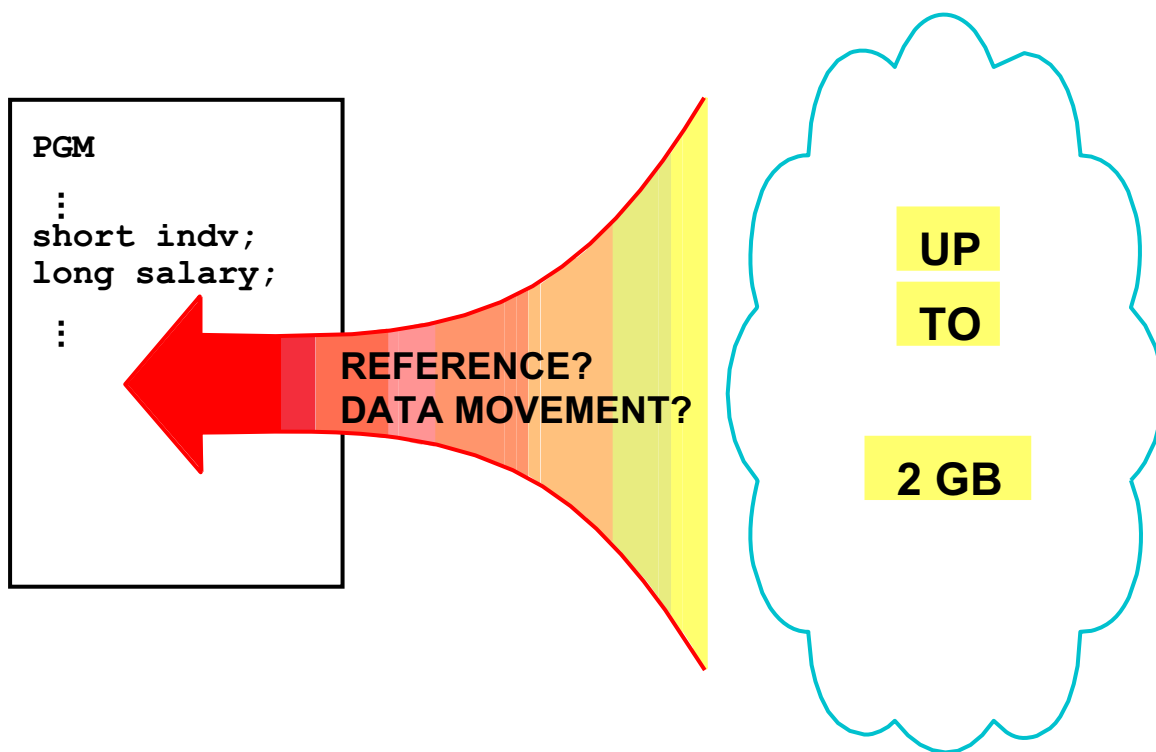
**What students will do —**

**How students will do it —** Listen in lecture.

**What students will learn —**

**How this will help students on their job —**

## Large objects — memory consideration



© Copyright IBM Corporation 2007

Figure A-2. Large objects — memory consideration

CF488.3

### Notes:

Storage of LOB data is now possible due to improvements in storage systems and technology.

The programmer should still be concerned with data movement of such large data volumes. For example, moving two gigabytes into an application when the entire LOB is not required would be wasteful in terms of resource consumption. In addition, using application memory for such a large object is not feasible.

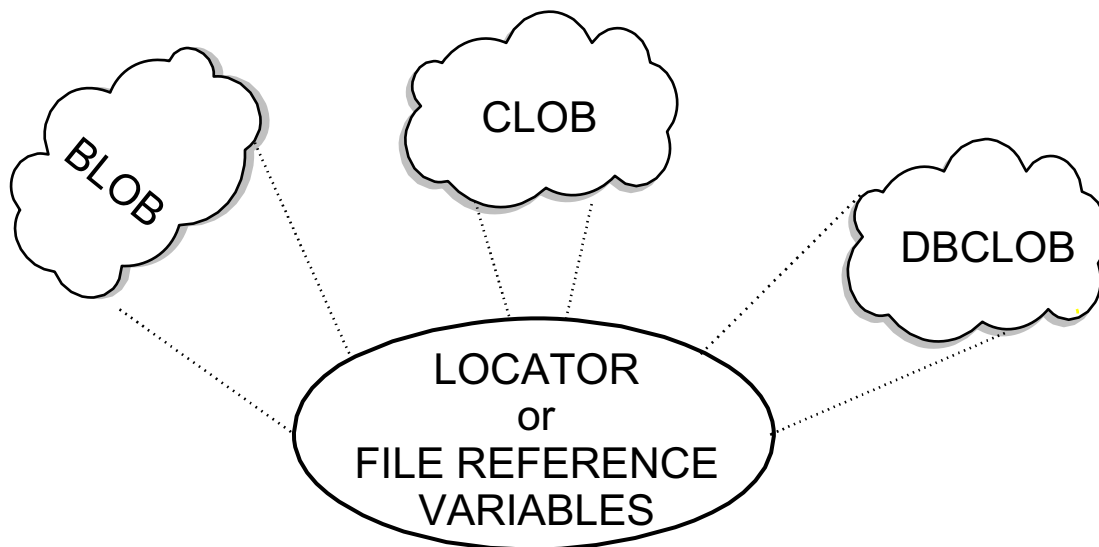
***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Transition Statement** — Let's look at the next page.

## Large objects — manipulation



- ✓ Reduce data transfer
- ✓ Defer evaluation for performance
- ✓ Direct file transfer
- ✓ CLI Functions

© Copyright IBM Corporation 2007

Figure A-3. Large objects — manipulation

CF488.3

### Notes:

DB2 provides the programmer with mechanisms to manipulate LOB data so that performance impact can be minimized.

A Locator Variable can be defined to represent the LOB or a portion of LOB data. A Locator Variable acts like a token within the application. The use of tokens allows the programmer to retrieve the LOB in parts, thus reducing the amount of data actually transferred to an application.

DB2 will permit several operations on LOB data:

- Retrieve value or partial value
- Replace value
- Use of the LIKE predicate
- Use of concatenation
- Use of the scalar functions SUBSTR, POSSTR, and LENGTH
- Use of UNION ALL

Evaluation of LOB data predicates is considered during the optimization process and will generally be deferred to enhance performance. The LOB data will not be accessed until other predicates are applied to reduce the number of candidate rows.

File reference variables provide the application programmer with a method to move LOB data directly between files and the database. There is no need to move the data into program storage when using file reference variables.

***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Transition Statement** — Let's look at the next page.

## DB2 Relational Extenders



- Help DB2 users
  - Handle complex data types
  - Improve application development productivity
  - Reduce development complexity
- Text Extender
- Image Extender
- Video Extender
- Audio Extender
- XML Extender
- Extra Charge:
  - Net Search Extender
  - Spatial Extender

© Copyright IBM Corporation 2007

Figure A-4. DB2 Relational Extenders

CF488.3

### **Notes:**

Text Extender components consist of:

- Administration CLP for preparing databases, tables, and columns for text search
- Building text indexes and changing text index characteristics
- Setting and changing default document characteristics such as language, format, and codepage
- Text extender UDFs for integrating full-text search into SQL queries, retrieving document characteristics, and refining text search arguments
- Programming API (C) for performing text search, browsing the resulting documents, including highlighting of matches

Image Extenders can be used to:

- Store and query images like traditional data
- Support a variety of image formats like GIF, JPEG, BMP, and TIFF
- Import and export images with attributes like size in bytes, format, height, width, and number of colors
- Convert the format of the images



- Query images based on related data or image attributes
- Generate and display thumbnail images
- Query By Image Content (QBIC) technology to search by what you see, match color, match texture, and match shape

Video Extenders can be used to:

- Store and query video clips like traditional data
- Import and export video clips with attributes like frame rate, compression format, and number of video tracks
- Query video clips based on related data or video attributes like name, number, description, format of video, and last update
- Play video clips
- Create an SQL statement within an application to query news databases for video clips
- Select a subject, list the playing times of selected clips, and allow application users to play clips
- Automatically segment a video clip into shots based on scene change detection when the recording camera changes its point of view

Audio Extenders can be used to:

- Store and query audio clips like traditional data
- Import and export audio clips with attributes like audio channels, transfer time, and sampling rate
- Query audio clips based on related data or audio attributes like name, number, description, format of the audio, and last update
- Play audio clips

With the XML Extender, your application can:

- Store entire XML documents as column data in an application table or externally as a local file, while extracting desired XML element or attribute values into side tables for search. Using the XML column method, you can:
  - Perform fast search on XML elements or attributes of SQL general data types that have been extracted into side tables and indexed
  - Update the content of an XML element or the value of an XML attribute
  - Extract XML elements or attributes dynamically using SQL queries
  - Validate XML documents during insertion and update
  - Perform structural-text search with the Text Extender
- Compose or decompose contents of XML documents with one or more relational tables, using the XML collection storage and access method.

Net Search Extender:

- Contains a DB2 stored procedure that adds the power of fast full-text retrieval to Net.Data, Java, or CLI applications.
- Offers application programmers a variety of search functions such as fuzzy search, stemming, Boolean operators, and section search.

Spatial Extender can be used to:

- Create a geographic information system: a complex of objects, data, and applications that allows you to generate and analyze spatial information about geographic features.
- Integrate spatial data with normal SQL data to give users access to new types of queries that could not previously be run.
- Store and index spatial data (coordinate information) and allow spatial queries to run against this data.

## ***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes. There are no customers using Fingerprint Extenders. The product went into beta 1995 and never shipped due to lack of interest.

**Additional Information** — The following information has been extracted from the Extender manuals and is included here for your reference.

From *Text Extender Administration and Programming*: The main part of the Text Extender product is installed on the same machine as the DB2 server. Only one Text Extender server instance can be installed with one DB2 server instance.

A Text Extender installation in a LAN is flexible and can comprise:

- One or several Text Extender LAN servers (on AIX, OS/2, Windows NT, SUN-Solaris, or HP-UX workstations)
- AIX, OS/2, Windows NT, Windows 95, Windows 98, Windows 3.1, SUN-Solaris, and HP-UX clients with access to one or several remote Text Extender servers
- AIX clients containing a local Text Extender server and having access to remote servers.

To run Text Extender from a client, you must first install a DB2 client and some Text Extender utilities. These utilities constitute the Text Extender client although it is not a client in the strict sense of the word. The client communicates with the Text Extender server via the DB2 client connection.

Text Extender has the following main components:

- A command line interpreter similar to the one offered by DB2 UDB for administration commands. Commands are available that let you prepare text in columns for searching, and maintain text indexes.
- User-defined functions (UDFs) that you can include in SQL queries for searching in text, and finding, for example, the number of times the search term occurs in the text.
- An application programming interface (API) consisting of functions that can be called in C programs for searching in text and displaying the search results.

From the *Image, Audio, and Video Extenders Administration and Programming*:

The Image, Audio, and Video Extenders create and use administrative support tables and handles to store and access image, audio, and video data. The Image Extender also creates and uses QBIC catalogs to access images by content. The Video Extender also uses index files and shot catalogs to access information about scene changes in a video.

**Administrative support tables**, also called metadata tables, contain the information that the extenders need to process user requests on image, audio, and video objects. The information in administrative support tables is often referred to as *metadata*.

Some of the administrative support tables identify user tables and columns that are enabled for an extender. These tables reference other administrative support tables that are created to hold attribute information about objects in enabled columns. In these tables, the extenders maintain information about attributes that are unique to a particular extender-defined data type, as well as information about attributes that are common across extender data types. For example, the Image Extender maintains information about the width, height, and number of colors in an image, as well as information about attributes common to image, audio, and video objects, such as the identification of the person who imported the object into the database or who last updated the object.

The administrative support tables can also contain the contents of stored objects in BLOB format. Alternatively, an object can be kept in a file and referenced by the administrative support tables. For example, a video clip can be stored as a BLOB in an administrative support table or kept in a file that is referenced by the table.

When you store an image, audio, or video object in a user table, the object is not actually stored in the table. Instead, an extender creates a character string called a **handle** to represent the object, and stores the handle in the table. The extender stores the object in an administrative support table, or stores a file identifier in an administrative support table if you keep the content of the object in a file. It also stores the object's attributes and handle in administrative support tables. In this way, the extender can link the handle stored in a user table with the object information stored in the administrative support tables.

A **QBIC catalog** is a set of files that hold data about the visual features of images. The Image Extender uses this data to search for images by content.

You create a QBIC catalog for each column of images in a user table that you want to make available for searching by content. When you create a QBIC catalog, you identify the features for which you want the Image Extender to analyze, store, and later query data. You can also add or drop features for a QBIC catalog after the catalog is created.

A **video index** is a file that the Video Extender uses to find a specific shot or frame in a video clip.

The Video Extender can detect scene changes in a video. A **scene change** is a point in a video clip where there is a significant difference between two successive frames. This happens, for example, when a camera changes its point of view while recording a video. The frames between two scene changes constitute a **shot**.

You can use the Video Extender's scene detection capabilities to find a shot, or even an individual frame, in a video clip. To do this, the extender needs indexing information for the shot or frame. This indexing information is stored in an **index file**.

A **shot catalog** is used to store data about shots in a video clip. The shot catalog can be stored in a database or in a file.

## A.2. User-Defined distinct types and User-Defined functions

### Instructor Topic Introduction

**What students will do** — Learn about UDTs and UDFs.

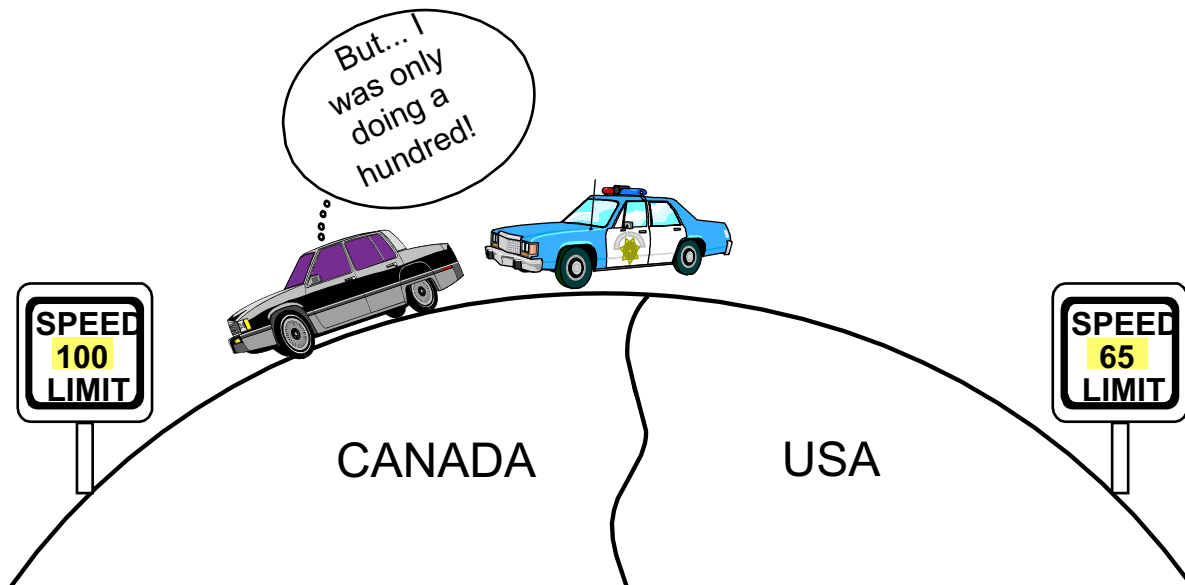
**How students will do it** — Listen to lecture.

**What students will learn** — A high-level view of creation of User-Defined Distinct Types and User-Defined Functions. They will also be introduced to relational extenders.

**How this will help students on their job** — They will be introduced to UDTs, UDFs and relational extenders.

## User-Defined Types — the need

- Need to establish context for values
- DB2 enforced typing



© Copyright IBM Corporation 2007

Figure A-5. User-Defined Types — the need

CF488.3

### Notes:

In the business world, typing is not limited to data types such as INTEGER or CHARACTER. Installations require data typing that specifically reflects the use of data.

For example, the above diagram represents a situation where the statement “**the speed limit is 100**” is ambiguous until the unit of measure is applied. 100 miles per hour is NOT the same as 100 kilometers per hour.

By permitting the installation to define types to the database manager, code to enforce typing is not required in applications. The database manager will enforce strong typing so that invalid comparisons are not allowed. These types are known as User-Defined Types (UDTs).

Strong typing is also valuable for adhoc access to data. The user taking advantage of a query tool may attempt to compare such things as *miles per hour* and *kilometers per hour* without realizing his or her error. The database manager can prevent such comparisons.

***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Transition Statement** — Let's look at the next page.

## User-Defined Types — definition

```
CREATE DISTINCT TYPE KPH AS INTEGER
WITH COMPARISONS
CREATE DISTINCT TYPE MPH AS INTEGER
WITH COMPARISONS
```

```
CREATE TABLE SPEED_LIMITS
  (ROUTE_NUM SMALLINT,
   CANADA_SL KPH NOT NULL,
   US_SL MPH NOT NULL )
```

```
SELECT ROUTE_NUM FROM SPEED_LIMITS
WHERE CANADA_SL > KPH(80)
SELECT ROUTE_NUM FROM SPEED_LIMITS
WHERE CANADA_SL > US_SL
```

**FAILS**

© Copyright IBM Corporation 2007

Figure A-6. User-Defined Types — definition

CF488.3

### Notes:

IMPLICIT\_SCHEMA authority or CREATEIN privilege is required to create a distinct type using a schema name that matches the current authorization ID of the statement. To create a distinct type in another schema, SYSADM or DBADM or CREATEIN privilege is needed.

The CREATE DISTINCT TYPE statement allows installations to create any number of data types that will be handled by the database manager.

The TYPE defined is **SOURCED** on an existing type, that is, one defined with the base product (such as INTEGER).

Once a type is defined, column definitions can reference the type when tables are created or altered. The database manager will enforce strong typing when such columns are used. For example, the SELECT statement shown at the bottom of the visual is invalid since KPH and MPH have been defined as DISTINCT types. They cannot be compared.

The use of the clause **WITH COMPARISONS** causes DB2 to create casting functions to convert the UDT to and from its source type. It is required syntax except when the base type is a LONG VARGRAPHIC, LONG VARCHAR, BLOB, CLOB, or DBCLOB. For



example, the definition of KPH in the illustration creates the function to cast that distinct type to and from the INTEGER source type. The use of **WITH COMPARISONS** and the automatic generation of the casting functions allows the following statement to be valid:

```
SELECT ROUTE_NUM FROM SPEED_LIMITS  
WHERE CANADA_SL > KPH(80)
```

where KPH(80) will cast the integer value of 80 to a type of KPH.

In DB2, structured types may also be created with a CREATE TYPE statement. A user-defined structured type may include zero or more attributes. A structured type may be a subtype allowing attributes to be inherited from a supertype.

***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Additional Information** — Two pages up at once.

**Transition Statement** — Let's look at the next page.

## Create Distinct Type — GUI

**Create Distinct Type**  
PATTI2 - DB2 - MUSICDB

Type schema: ADMIN

Type name: KPH

Source data type: INTEGER

**Datatype characteristic**

Length:  LOB unit: Bytes

Precision:  Scale:

☐ Bit data

Comparison operators will be generated

Comment: Kilometers per hour

OK Show SQL Cancel Help

© Copyright IBM Corporation 2007

Figure A-7. Create Distinct Type — GUI

CF488.3

### Notes:

To access the Create Distinct Type GUI from the Control Center:

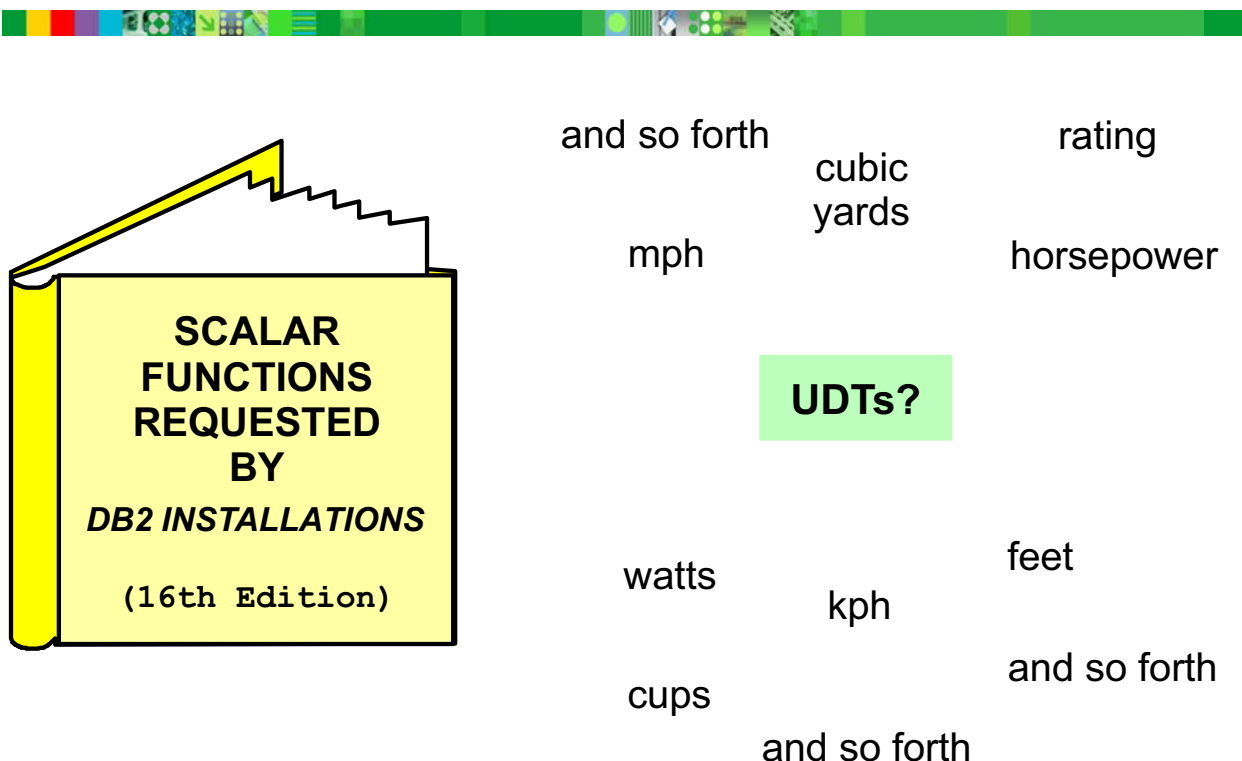
- Application Objects
  - Select User-Defined Distinct Types (right mouse button)
    - Select Create

The Create Distinct Type GUI allows the following to be indicated:

- Selection for the schema for the distinct type
- Specification of the name of the distinct type
- Select of the source data type
- Datatype characteristic for non-fixed source data types
- Comment

***Instructor Notes:***

## User-Defined Functions — the need



© Copyright IBM Corporation 2007

Figure A-8. User-Defined Functions — the need

CF488.3

### Notes:

The number of different business applications using DB2 as a database manager is wide and varied, and the number continues to grow. This diverse usage places demands on the database to support a wide variety of scalar functions.

The use of UDTs also increases the need for additional scalar functions. For example, a conversion may be required to convert from gallons to liters.

The introduction of support for User-Defined Functions (**UDFs**) allows the installation to define any number of site specific functions.

This support can also be exploited by software developers who write specialized applications that can be installed with DB2 as the database manager supporting data storage and retrieval. The software developer may provide UDTs and UDFs as part of the software package.

## ***Instructor Notes:***

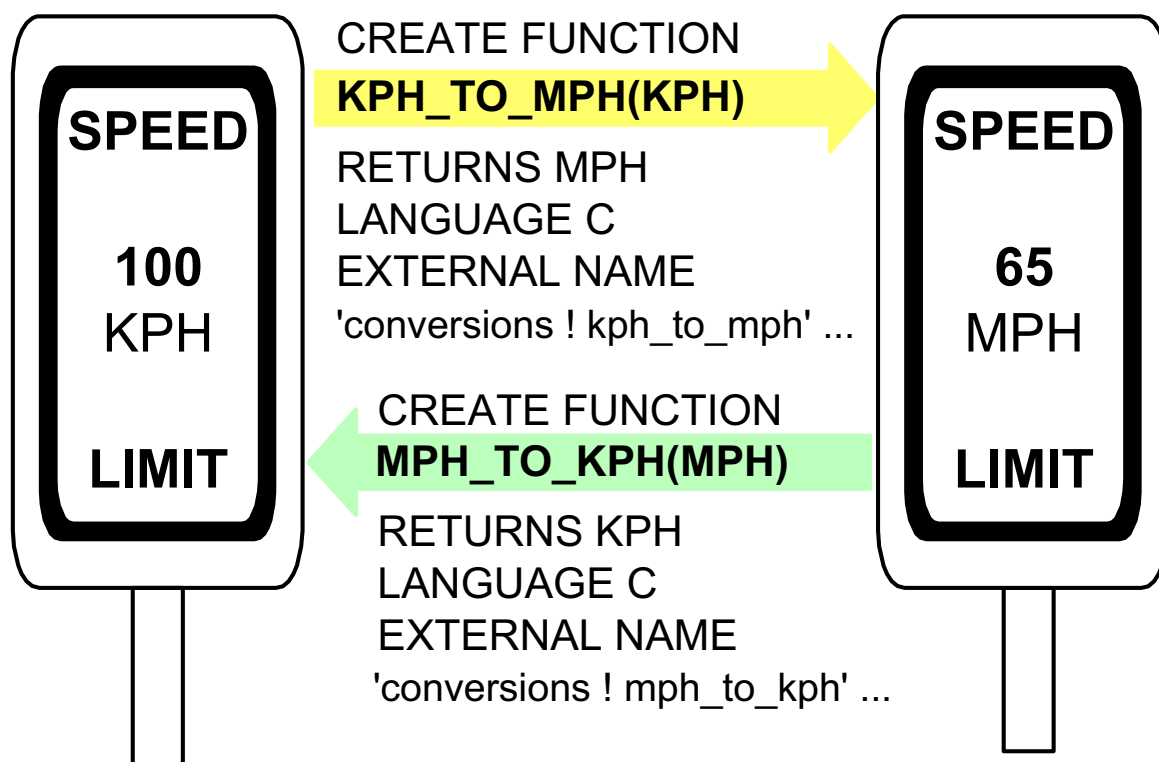
**Purpose** — Introduce topic.

**Details** — See student notes.

**Additional Information** — The book illustrated on the graphic is, of course, a fictional book.

**Transition Statement** — Let's look at the next page.

## User-Defined Functions — definition



© Copyright IBM Corporation 2007

Figure A-9. User-Defined Function — definition

CF488.3

### Notes:

The CREATE FUNCTION statement can be used to define a user-defined SQL scalar, table, or row function. A scalar function returns a single value each time it is invoked, and is generally valid wherever an SQL expression is valid. A table function may be used in a FROM clause and returns a table. A row function may be used as a transform function and returns a row.

The UDF is coded in a programming language such as C, Java or SQL PL. This allows the installation to take advantage of functions available in the language. The installation can take advantage of library management of such functions as well. For example, the functions defined in the visual are conversion functions. They reside in a library called **conversions**.

A UDF can be invoked anywhere in an SQL statement where an expression is currently permitted.

UDFs are considered by the optimizer during access path selection.

For an example of usage of the UDF defined as MPH\_TO\_KPH, consider resolution of the previously illustrated SELECT statement that needed to compare KPH to MPH.

```
SELECT ROUTE_NUM FROM SPEED_LIMITS
      WHERE CANADA_SL > MPH_TO_KPH(US_SL)
```

A CREATE FUNCTION statement can also be used to create a user defined scalar SQL function. If the returned value of a function can be determined via SQL PL statements, then an SQL scalar function could be used. However, if you need to access resources outside of DB2 (for example, read a file) which SQL PL does not support, then the UDF must be coded in an external language. For example, the function illustrated on the graphic could be coded as:

```
CREATE FUNCTION MPH_TO_KPH (MILES MPH)
  RETURNS KPH
  LANGUAGE SQL
  CONTAINS SQL
  RETURN KPH(INTEGER(1.6 * (1.0 * MILES)))
```

which would require no external code to be created.

Scalar function:

```
CREATE FUNCTION tan (x double) RETURNS double LANGUAGE SQL
  CONTAINS SQL NO EXTERNAL ACTION DETERMINISTIC
  RETURN sin(x) / cos(x)
```

Row function:

```
CREATE FUNCTION fromperson (p person)
  RETURNS ROW (name varchar(10), firstname varchar(10))
  LANGUAGE SQL CONTAINS SQL NO EXTERNAL ACTION DETERMINISTIC
  RETURN VALUES (p..name, p..firstname)
```

Table function:

```
CREATE FUNCTION deptemployees (deptno char(3))
  RETURNS TABLE (empno char(6), lastname varchar(15),
  firstname varchar(12))
  LANGUAGE SQL READS SQL DATA NO EXTERNAL ACTION DETERMINISTIC
  RETURN
  SELECT EMPNO, LASTNAME, FIRSTNAME FROM EMPLOYEE
  WHERE EMPLOYEE.WORKDEPT = DEPTEMPLOYEES.DEPTNO
```



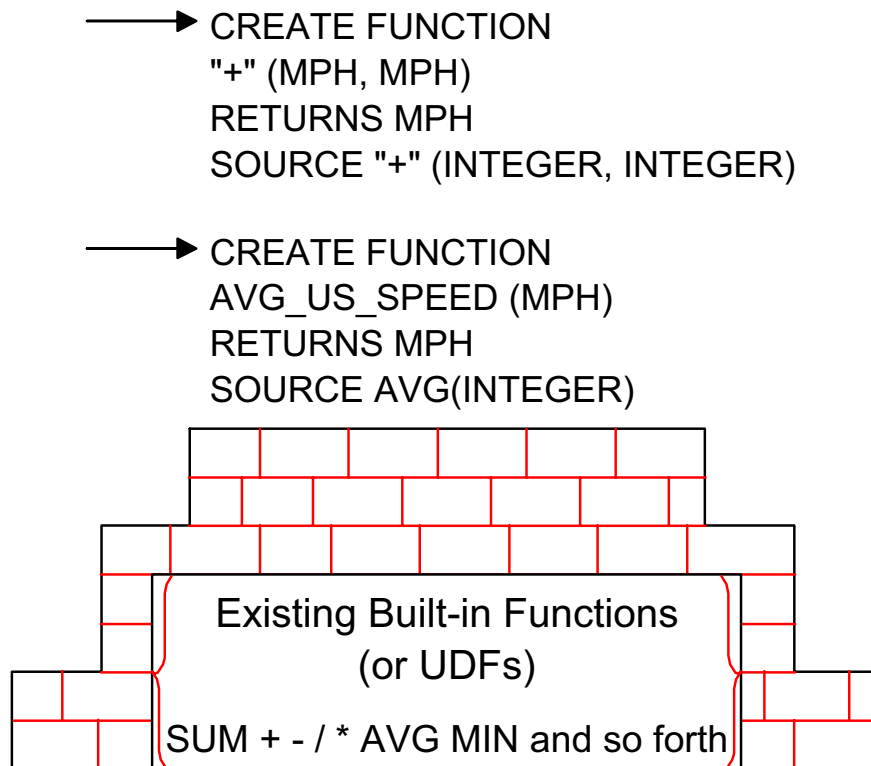
***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Transition Statement** — Let's look at the next page.

## User-Defined Functions — sourced



© Copyright IBM Corporation 2007

Figure A-10. User-Defined Function — sourced

CF488.3

### Notes:

Strong typing defines that data types can only be used in manners defined by the installation. Therefore, a UDT cannot be used in conjunction with functions provided by the database manager unless such usage is defined. To assist the installation in defining common functions, DB2 allows functions to be **sourced** on previously defined functions. These previously defined functions could be provided by the database manager or user-written.

**Note:** It may be advantageous **NOT** to define a function sourced on every function provided by the database manager for certain UDTs. For example, assume the employee number is defined as a distinct type based on the INTEGER type. The installation would probably not define the functions of SUM, AVG, +, -, \*, and / on this data type since such manipulations would be invalid.

Overloading of a function name is also permitted so that the same name can be used several times. For example, consider the function **AVG\_US\_SPEED** defined in the visual. This function could have been defined as **AVG**, even though such a function name already

exists. The database manager will invoke the function that matches the input parameters passed in an overloading situation.

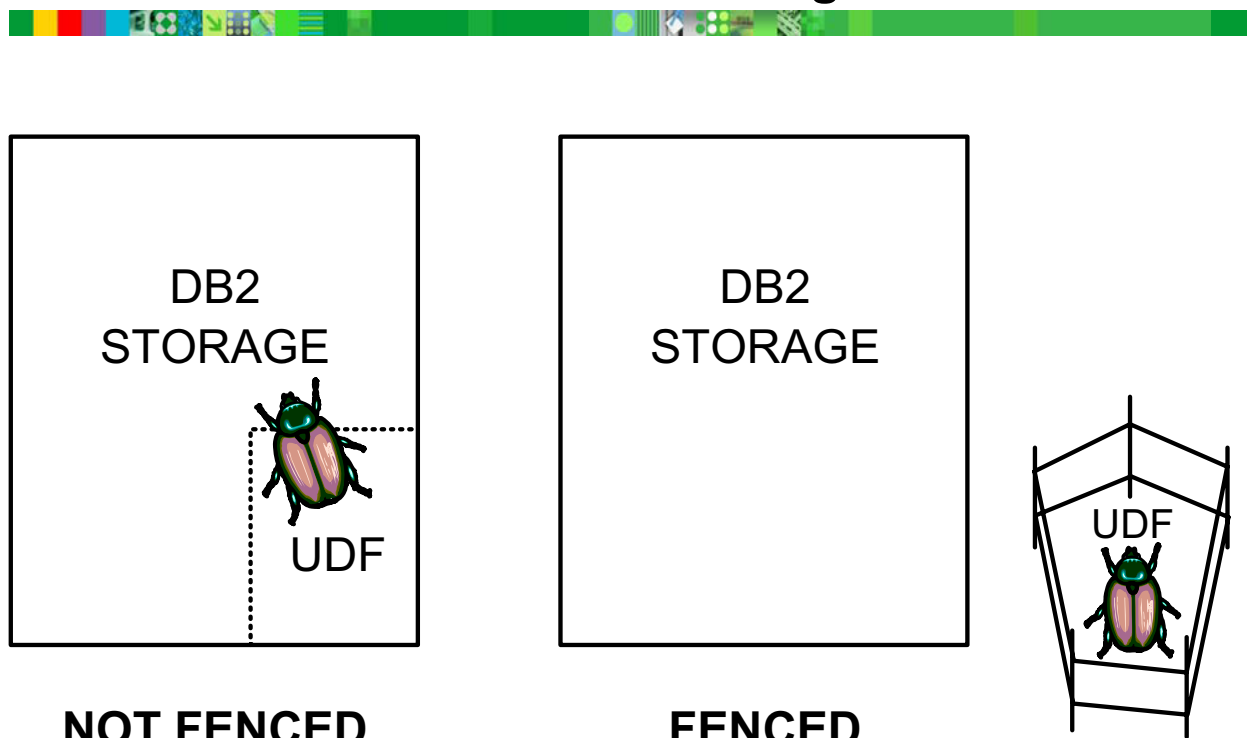
***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Transition Statement** — Let's look at the next page.

## User-Defined Functions — fencing



© Copyright IBM Corporation 2007

Figure A-11. User-Defined Functions — fencing

CF488.3

### Notes:

DB2 provides the installation with the capability to determine where user-defined functions execute. Where UDFs execute is set at the instance level and thus applies to all databases created within an instance.

A function that is *fenced* cannot impair the integrity of the database manager's storage area. If coding errors are present in the UDF, the database will be shielded from the errors.

A function that is not fenced executes as part of the database manager code. This may yield a performance improvement, but care must be taken to ensure that the UDF does not corrupt the database manager storage. The capability to create a function that is not fenced is controlled through DB2 authorization.

On UNIX systems, fenced UDFs can be run under a USERNAME different from the instance USERNAME. The UDF USERNAME must be specified on the DB2ICRT command when an instance is created.

Other clauses that may be defined when creating UDFs include:

- DETERMINISTIC or NOT DETERMINISTIC - function will or will not produce different results when called with the same arguments. For example, a randomizing function would be defined as NOT DETERMINISTIC.
- EXTERNAL ACTION - defines to the database manager that an action external to the database may be taken by the function. For example, a function that sends a message to a development team has an EXTERNAL ACTION.

***Instructor Notes:***

**Purpose** — Introduce topic.

**Details** — See student notes.

**Transition Statement** — Let's look at the next page.





# Checkpoint

## Exercise — Appendix A Checkpoint

\_\_\_ 1. What are the data types for large objects?

---

**BLOB, CLOB, and DBCLOB**

## Appendix summary



Having completed this appendix, you should be able to:

- Describe LOBs, UDFs, UDTs, and Relational Extenders

© Copyright IBM Corporation 2007

Figure A-12. Appendix summary

CF488.3

### **Notes:**

***Instructor Notes:***

**Purpose** — Review the unit objectives.

**Details** — Question the students on what they've learned.

**Transition Statement** —



# Appendix B. Checkpoint solutions

## Unit 4 Checkpoint

\_\_\_ 1. Name the two types of table spaces:

SMS  
DMS

\_\_\_ 2. Which type of table space must be used if you want to store the index data separately from the table data?

DMS

\_\_\_ 3. Which type of table space should be used to allow the operating system to allocate pages to the table space as needed?

SMS

## Unit 7 Checkpoint

\_\_\_ 1. State the three methods of recovery.

Crash, version, roll forward recovery

\_\_\_ 2. What is required to avoid the need for a roll forward?

Offline database backup  
Restore...without Rolling Forward

\_\_\_ 3. How might a database be placed in roll forward pending status?

RESTORE online database backup  
RESTORE offline database backup omitting WITHOUT ROLLING FORWARD

\_\_\_ 4. How might a table space be placed in roll forward pending status?

---

**RESTORE a table space backup. DB2 detects a media failure isolated to a table space.**

- \_\_\_ 5. What is the minimum time to which a database must be rolled forward?

---

**To the end of the time of the online backup.**

a table space?

---

**To the minimum roll forward time queried via LIST TABLESPACES.**

## Unit 8 Checkpoint

- \_\_\_ 1. You are evaluating the locking strategy of a program that reads and changes data. The program reads or changes a limited number of rows in the target tables and should run during the time when heavy use of the system is expected by other transactions. What lock modes best support this level of required concurrency at the table level?

---

**IS for reading and IX for changing.**

**Ref:** This question is meant to emphasize the most common table locking strategy expected for table spaces and tables in transaction processing.

- \_\_\_ 2. A programmer requests your advice. An application that will run during a batch cycle (no other concurrent requests against the target table) will be written to update a large portion of the table. Explain indicates an IX lock for the table space and table. Would you suggest any technique for the programmer to investigate, and if so, what is the technique?

---

**Include: LOCK TABLE name IN EXCLUSIVE MODE.**

**Ref:** Eliminating row locking may be beneficial in terms of performance for the application in question. However, the impact on concurrency needs to be carefully evaluated before using the LOCK TABLE statement.

- \_\_\_ 3. An airline requires an application that will determine available seats on a particular flight to support the reservation system. The application should list the seats available that meet a general grouping (for example, aisle seat toward the front of the plane) so that reservations can suggest a seat when booking a passenger. However, the application should not prevent update to the row concerning the seat while the passenger is considering possible options. (In other words, the seat is not held for the passenger while the passenger is thinking; in fact, the cursor is closed after producing the answer set.) Assume that the airline uses conversational

transactions (that is, the transaction does not commit prior to displaying a screen). What isolation level would be most suitable?

---

### Cursor Stability, CS.

**Ref:** The isolation levels of Repeatable Read or Read Stability are not appropriate because they would maintain S locks on rows accessed. This is in conflict with the business requirement for other applications to be able to update the rows examined.

Uncommitted read is not appropriate, since rows in the process of being updated should not be considered *available*. (It is possible that some students may disagree with this interpretation of the term *available*. As long as a student demonstrates understanding of the difference between UR and CS, the interpretation of the term is not a concern.)

Cursor Stability permits obtaining a list of seats that are *available* at the time they were examined, but does not prevent changes to the status when the rows are presented to the reservation agent.

- \_\_\_ 4. Assume that the airline described in the previous question now wants to ensure that any seat being shown to the user will be available, should that user wish to reserve it. What isolation level would be most suitable?

---

### Read Stability.

**Ref:** Some people may suggest repeatable read, but this is not appropriate as it will maintain S locks on all rows accessed, not just the ones being presented to the user. Cursor Stability will not lock all of the rows that have been read; in fact, since the cursor was previously closed, none of the rows will be held.

- \_\_\_ 5. You are the database administrator for a large financial institution. Most processing requirements of the business are accomplished by using an isolation level of Cursor Stability. However, assume that government regulations dictate that a status report of an account table must be generated at 12:00 p.m. on the last day of business each month. Furthermore, the regulations stipulate that the state of all rows is not permitted to change from the start of the report application until it is completed. How would you suggest meeting this requirement?

---

### Include: LOCK TABLE name IN SHARE MODE.

**Ref:** Many students will tend to suggest repeatable read as a solution to this requirement. However, repeatable read does not obtain and hold row locks until the row is accessed. It is therefore possible for other applications to change data that has not yet been read by the report application. The only way to guarantee the stability of an entire table is to lock the table before processing the rows.

- \_\_\_ 6. You receive several phone calls from users of your database complaining about a high number of deadlocks and timeouts. You begin investigating by taking a snapshot with the monitor and discover a high number of lock escalations. You know that you have configured the locklist and maxlocks parameters to handle your normal application requirements. What would be your next area of investigation?

---

**Look for programs not committing frequently.**

**Ref:** An inappropriate number of lock escalations can be caused by a lock list that is not large enough to handle the application load or a maxlocks parameter that does not support the largest unit of work generally required by some applications.

Assuming that these areas have been eliminated as a possible source, the most likely cause is an application that exceeds the normal design case concerning lock requirements. Students may also respond with other valid arguments as well. Again, the absolute answer is not important. The assumptions and thought process to arrive at a conclusion are the key.

## Unit 9 Checkpoint

- \_\_\_ 1. What is the database manager configuration parameter that defines the level of error logging done to the administration notification log?

---

**NOTIFYLEVEL**

- \_\_\_ 2. What tools can be used to monitor DB2 activity?

---

**Snapshot Monitor, Event Monitor, Health Monitor**

- \_\_\_ 3. What type of tool is used to determine the access strategy DB2 will use to execute an SQL statement?

---

**Explain**

## Unit 10 Checkpoint

- \_\_\_ 1. You are the only DBA permitted to create new packages on the production system. You are called at 2:00 a.m. by a member of the operations staff, who informs you that an application is ending with an SQLCODE of -805. What is your response?

---

**Analyze the function of the new application and BIND the new package if justified.**



**Ref:** Since it is given that the student is the only DBA with the authority to add new packages to the system, it follows that some degree of security is implied. The -805 code indicates that the package does not exist. Therefore, a new application is being put into production or the package for an existing application has been dropped. In either case, examining the function of the application is warranted.

- \_\_\_ 2. Assume that there is another DBA with the authority to BIND any existing application. Three nights after your previous call from the operations staff, you receive another call. This time, the SQLCODE is a -818. What would you advise the DBA on duty to do?

---

### Precompile, Bind, Compile, and Link the application.

**Ref:** The -818 indicates a consistency token (timestamp) mismatch between the executable and the package. In this particular case, several responses are valid. If the DBA has the capability to *trace* the preparation steps that have been completed, it may be possible to determine the specific steps required to resolve the problem. For example, perhaps all that is required is to bind the package from a bind file. The answer provided assumes that the *trace* capability does not exist and also favors the *easy* solution.

- \_\_\_ 3. Chris is a programmer who will automate the creation of backups for your DB2 system. In a meeting to determine the proper interface, she asks for your advice on how to proceed. What do you tell her?

---

### Use an API.

- \_\_\_ 4. Mike is a programmer who will write an application to support part of a point-of-sales application. His program will need to check on the credit history of customers attempting to use a credit card. The input to the program is the credit card number. The credit history is in a 40-million row DB2 table, which has an index by credit card number. What interface would you suggest?

---

### Embedded Static SQL.

**Ref:** Also, it is assumed that this transaction will be requested on a repetitive basis and that dynamic bind would not be desirable.

- \_\_\_ 5. Your consulting services are requested at a planning meeting of a major software vendor. The software product being discussed must be able to perform ad hoc queries against a number of different database products. In order to prevent having to provide specific code for each database product supported, you contribute a suggestion. What is it?

---

### Use CLI.

**Ref:** Also, a student may also suggest ODBC. This is a valid answer as well, but is targeted by the next question.

- \_\_\_ 6. The software application described in the prior question is going to be implemented using Microsoft Windows. What architecture might you recommend?

---

### ODBC

**Ref:** Also, ODBC is not limited only to the Windows environment, but historically has been a solution popular with Windows developers. Visigenic Software Inc. provides an ODBC Software Development Kit for non-Microsoft platforms. See the Call Level Interface Guide for details.

- \_\_\_ 7. As the database administrator, you have been constantly requested to supply samples of production data for a test machine. The test machine has enough security so that the content of the data can be a sampling of actual production data; however, only 5% of the data needs to be periodically sampled for the test environment. In the past, you have manually retrieved the sample and would like to eliminate this task from your busy schedule. You would like staff leaders from the programming group to be able to request a random 5% sampling of any table for which they have SELECT authority. The application supporting this function would sample the data and optionally transmit the data to a target machine. How would you suggest implementing this application?

---

### Use a stored procedure and dynamic SQL.

**Ref:** Since sampling the data would require server-intensive activity, the stored procedure approach is desired.

- \_\_\_ 8. A programmer is writing an application against a DB2 database. Some of the columns desired and the predicate is not totally known until execution time. What would you advise the programmer to use?

---

### Dynamic SQL.

- \_\_\_ 9. You are attempting to determine the best cluster sequence for a large customer table. During your initial analysis, you have determined that online access to the table is generally via a unique CUSTNUMB. You also have many *batch* applications that require data to be sorted on LASTNAME. There are indexes on both of these columns. Given these brief requirements, what would you choose as the index to specify when performing a REORG, if either?

---

### The index on LASTNAME.

**Ref:** Students may debate the correct answer to this question based on assumptions they may make concerning the access to this table. The response given reflects most accesses to data via unique keys. Clustering the data by such a key does not improve performance because access tends to be via equal predicates. There doesn't tend to be an inherent relationship between rows with *adjacent* values for the unique key. (Of course, there are exceptions to this general rule.)

The LASTNAME column has been indicated as one necessary to support ordering requirements, which in turn indicates that more than one data row will tend to be accessed. If the data is in cluster sequence according to this column, DB2 may choose to use the index to avoid actual sorts and also limit the I/O required to retrieve the data.

- \_\_\_ 10. The table analyzed for the prior question is examined more closely. Many online applications examine the data according to a range of ENTRY\_DATE values. There is an index on ENTRY\_DATE. Would you change your choice of cluster?

---

**Yes. Change the clustering according to the index on ENTRY\_DATE.**

**Ref:** Supporting this answer is the assumption that MANY online applications access the data by a range of values against the ENTRY\_DATE column. By changing the clustering of the table, the online access for these applications could be improved by reducing the I/O requirements of access. Of course, the batch applications mentioned in the prior question that require sorting on LASTNAME may be negatively impacted. However, the DBA should generally favor online performance over batch applications. Certainly, there are exceptions to this general rule.

- \_\_\_ 11. An application programmer asks for your advice concerning a program that produces a simple business report from the table previously described. The program accesses the server from a client workstation and appears to run for longer than necessary. It simply FETCHES about 20% of the data and writes the rows to a report. EXPLAIN indicates that the index on ENTRY\_DATE is being used to support a range predicate in the query. The programmer assures you that the same statement runs fine in CLP, but takes much longer inside the static program. What would be a PRECOMPILE / BIND parameter that you should examine as one, if not the first, possible cause of the problem?

---

**Blocking. Determine if it is enabled.**

**Ref:** Certainly, students may respond with other plausible causes of the problem. However, the symptoms described indicate a strong possibility that blocking (or lack of it) could be the problem.

- \_\_\_ 12. A programmer presents you with an application that has a very complex SQL statement that accesses many tables. Assume that the statement is logically correct and that your database design is proper. However, the programmer implies that the strategy externalized by EXPLAIN accesses the *wrong* table first. Assume that you have plenty of time and CPU for program preparation. What PRECOMPILE / BIND parameter might you experiment with to influence DB2?

---

### QUERYOPT

**Ref:** A basic assumption for this answer is that the installation does not always use a QUERYOPT level that would cause DB2 to consider all possible access strategies.

- \_\_\_ 13. You have just finished analyzing the SQL in a package that has been identified as a performance problem. You determined that an index to support the application was required, so you created the index. Assume that the new index was also based on the columns of the table by which the data should be clustered. What are your next three steps required for the package to take advantage of the new index?

---

### REORG RUNSTATS REBIND

**Ref:** The REORG puts the table in the correct sequence according to the key specified, which is assumed to be the new index. The RUNSTATS command would gather the statistics for the new index and physical structure of the data, which is required for the REBIND to function properly.

## Unit 11 Checkpoint

- \_\_\_ 1. Can the right to grant table, view, or schema privileges to others be extended to other users using the WITH GRANT OPTION on the GRANT statement?

---

Yes.

## Appendix A Checkpoint

- \_\_\_ 1. What are the data types for large objects?

---

### BLOB, CLOB, and DBCLOB

# Index

## A

access control 11-16  
 access strategy 10-70  
 access to database objects 11-34, 11-56  
 administration authorities 11-19  
 Administration Tools 1-11, 1-12  
 Advisors 2-42  
 Alert Center 1-11  
 ALTER 11-22  
 APIs 10-16  
 application process 10-51  
 Applications 1-12  
 archival logging 7-29  
 aslheapsz 10-88  
 asynchronous page cleaner 10-82  
 authorities 11-19, 11-22  
 authority 11-16  
 authorization for backup 7-60  
 avg\_appls 10-58

## B

backup considerations 7-60  
 Backup Database GUI 7-68  
 backup file naming conventions 7-74  
 backup, offline versus online 7-71  
 BIND 10-96, 10-107, 11-22  
 bind 10-31, 10-45  
 bind command syntax 10-45  
 BIND options 11-53  
     11-53  
 BINDADD 11-22  
 BLOBs 5-79  
 blocking 10-88  
 BUFFPAGE 10-82  
 buffpage 10-58

## C

candidates for table space backup strategy 7-87  
 catalog directory 4-94  
 catalog views, packages 10-48  
 change directory 4-94  
 check constraints - definition 5-64  
 check constraints - the need 5-62  
 CHNGPGS\_THRESH 10-82  
 circular logging 7-26  
 CLOBs 5-79  
 CLP 1-11  
 Command Center 1-11, 1-12, 2-33  
 Command Line Processor 1-11  
 compile 10-31

Configuration Assistant 2-52, 2-55  
 configuration files 3-4  
 configuration parameters for logging 7-41  
 Configuration Tool 1-11  
 CONNECT 11-22  
 consistency token 10-31, 10-34  
 CONTROL 11-22  
 Control Center 1-11, 1-12, 2-30  
 coordinated universal time (CUT) 7-100  
 cpuspeed 10-58  
 CREATE BUFFERPOOL statement 4-35  
 create table 5-56  
 Create Table Space Wizard 4-58  
 create view 5-33  
 CREATETAB 11-22  
 CS 1-15  
 CURRENT QUERY OPTIMIZATION special register  
     10-61, 10-65  
 cursor stability 8-32

## D

DAS 2-62  
 data fragmentation 10-114  
 data placement considerations 4-86  
 data processing problems 7-9  
 database 3-4, 4-6  
 database access 11-34, 11-56  
 database configuration file 3-4  
 Database Configuration files 3-4  
 Database Configuration window 4-90  
 database directory 4-94  
 Database Engine 1-11  
 Database Managed Space 4-46  
 Database Manager APIs 10-16  
 database manager configuration file 3-4  
 Database Manager Configuration files 3-4  
 database monitoring 9-32  
 Database Services 1-11  
 Database System Monitor authorization 9-39  
 Database System Monitor basic information 9-43  
 Database System Monitor commands 9-45  
 Database System Monitor functional groups 9-43  
 Database System Monitor interfaces 9-39  
 Database System Monitor, when active 9-41  
 database\_consistent 7-37  
 db cfg 3-4  
 DB2 1-20  
 DB2 Administration Server 2-62  
 DB2 Communication Support 1-15  
 DB2 configuration files 3-4  
 DB2 Connect 1-15

DB2 Connect Enterprise Edition 1-19  
DB2 Connect Personal Edition 1-19  
DB2 Database Partitioning Feature 1-9  
DB2 Enterprise - Extended Edition 1-8  
DB2 Enterprise Edition 1-8  
DB2 Enterprise Server Edition 1-9  
DB2 Express Edition 1-8  
DB2 Family 1-15  
DB2 get authorizations 11-69  
DB2 Personal Edition 1-8  
DB2 Physical Environment 3-4  
DB2 prep syntax 10-41  
DB2 Product Components 1-11  
DB2 utilities 6-20, 6-29, 6-32  
DB2 Workgroup Edition 1-8  
DB2 Workgroup Server Edition 1-8  
db2diag.log 9-9, 9-24  
DB2GROUPS 11-40  
db2rbind 10-73  
db2trc 9-75  
db2untag 4-54  
DBADM 11-16, 11-19, 11-22  
DBCLOBs 5-79  
dbm cfg 3-4  
DBM Configuration 4-92  
deadlock causes 8-67  
deadlock common causes 8-70  
deadlock definition 8-67  
deadlock detection interval 8-67  
deadlock detector 8-67  
default database configuration 4-37  
defining table spaces 4-56  
DELETE 11-22  
Development Center 1-12  
DFT\_EXTENT\_SZ 4-49  
Diagnostics  
    data required 9-9  
    db2diag.log 9-9  
directories 4-94  
Directory Tool 1-11  
DMS 4-46  
DMS table space - minimum requirements 4-54  
DRDA Application Server 1-22  
dynamic versus static SQL 10-13

## E

enforcing business rules 5-69  
enforcing data rules 5-62  
escalation of locks 8-55  
Event Analyzer 1-11  
event monitoring 9-35  
EXECUTE 11-22  
EXECUTE option 11-53  
EXPLAIN 9-80, 9-82  
explicit authorization 11-34

EXPORT utility 6-10, 6-20  
extent 4-49  
EXTENTSIZ 4-49

## F

file reference variables A-7  
First-failure data capture 9-16  
fragmentation 10-114

## G

get authorizations 11-69  
GET MONITOR SWITCHES 9-45  
GET SNAPSHOT 9-48  
grant specification 11-53  
grant/revoke scenarios 11-63  
grants 11-42  
group IDs 11-40  
groupids 11-42  
groups 11-40

## H

Health Center 1-12, 2-46  
health monitoring 9-35  
history file for recovery 7-54

## I

implicit authorization 11-56  
implicit privileges scenarios 11-61  
implicit rebind 10-70  
IMPORT utility 6-10, 6-29, 6-32  
independent trace facility 9-75  
INDEX 11-22  
index clustering 10-100  
index statistics 10-103, 10-111  
individual ids 11-42  
inoperative 10-70  
INSERT 11-22  
instance 3-4, 4-6  
instance owner privileges 11-16  
isolation level 8-32

## J

Journal 1-11, 1-12, 2-49

## L

Large object types 5-79  
large objects - data manipulation concern A-5  
large objects - tokens A-7  
large table space 4-46  
Launchpads 2-42  
License Center 1-12

link 10-31  
 list authorizations 11-69  
 Load wizard 6-47  
 LOBs 5-79  
 lock conversion 8-52  
 lock escalation 8-55  
 lock isolation level 8-32  
 lock mode compatibility 8-23  
 lock modes for rows 8-20  
 lock modes for tables 8-16  
 LOCK TABLE statement 8-49  
 lock wait 8-60  
 Locking  
     explicit 8-46  
 locking application scenarios 8-16  
 locking at table level only 8-49  
 locking influenced by temporary tables 8-32  
 locking objects 8-11  
 locking strategies 8-13  
 locklist configuration parameter 8-55  
 locktimeout configuration parameter 8-60  
 log buffer 7-21  
 log file location 7-37  
 log file naming convention 7-103  
 log retention logging 7-29  
 logbufsz 7-41  
 logfilsiz 7-41  
 Logging  
     dual/mirrored 7-33  
 logging and ACTIVATE DATABASE 7-29  
 logging configuration parameters 7-41  
 logging/backup requirements summary 7-111  
 loghead 7-35  
 logpath 7-37  
 logprimary 7-41  
 logretain 7-41  
 logsecond 7-41

## M

max\_coordagents 10-79  
 maxagents 10-79  
 maxappls 10-79  
 maxcagents 10-79  
 maxlocks configuration parameter 8-55  
 mincommit 7-41  
 modes of locks for rows 8-20  
 modes of locks for tables 8-16  
 monitoring activation 9-41

## N

newlogpath 7-37, 7-41  
 nextactive 7-35  
 NLEAF 10-100  
 NLEVELS 10-100

NOTIFYLEVEL 9-11  
 NUM\_IOCLEANERS 10-82  
 num\_poolagents 10-79  
 numdb 10-79

## O

objects of locks 8-11  
 optimization class 10-61, 10-65  
 optimizer functions 10-58  
 optimizer, factors influencing 10-58  
 OWNER option on BIND 10-39

## P

package 10-70  
 packages 10-34  
 packages catalog views 10-48  
 PAGEFREESPACE 6-88  
 pckcachesz 10-31  
 PCTFREE 6-88  
 PDE 1-15  
 performance 10-114  
 performance benefits of dynamic SQL 10-13  
 performance benefits of static SQL 10-13  
 Physical Environment 3-4  
 point-in-time roll forward 7-100  
 precompile 10-31  
 precompile syntax 10-41  
 precompiler options 10-41  
 prep 10-31  
 prep syntax 10-41  
 privilege 11-16  
 privileges 11-22, 11-42  
 Problem  
     description 9-7  
     solving 9-6  
 product components 1-15  
 program preparation steps 10-31  
 programming education 10-128

## Q

QUALIFIER option on BIND 10-39  
 query authorizations 11-69  
 query privileges 11-69

## R

read stability 8-32  
 REBIND 10-73  
 rebinding 10-70  
 REC\_HIS\_RETENTN 7-54  
 recovery history file 7-54  
 recovery methods 7-18  
 Recovery Tool 1-11  
 REFERENCES 11-22



referential integrity 5-56  
regular table space 4-46  
relational extenders A-10  
REORG 10-96, 10-107  
REORG TABLE 10-114  
reorganize check 10-96, 10-107, 10-111  
REORGCHK 10-96, 10-107, 10-111  
repeatable read 8-32  
Replication Center 1-12  
RESET MONITOR 9-45  
RESTORE command 7-80  
Restore Database GUI 7-84  
reuse of log files 7-103  
revoke/grant scenarios 11-63  
Roll Forward Database GUI 7-97  
Roll Forward pending 7-90  
rollforward 7-92  
ROLLFORWARD options 7-100  
ROLLFORWARD PENDING - clearing 7-100  
row lock compatibility 8-23  
row lock duration 8-32  
RUNSTATS 10-96, 10-103, 10-107, 10-114  
Run-Time Client 1-15

## S

Script Center 1-11  
SECADM 11-74  
security listing 11-69  
SELECT 11-22  
sheaphres configuration parameter 10-75  
SMS 4-46  
SNA Support Feature 1-15  
Snapshot monitoring 9-35  
snapshots 9-48  
softmax 7-41  
sort passes 10-77  
sortheap configuration parameter 10-75, 10-77  
sorts - piped vs. non-piped 10-75  
SQL Assist 2-36  
statistics 10-96, 10-103, 10-107, 10-111  
Stored Procedures 10-91  
strict table locking 8-49  
SYSADM 11-16, 11-19, 11-22  
syscat.packages.valid 10-70  
SYSCTRL 11-16, 11-19  
SYSMAINT 11-16, 11-19  
system authorities 11-19  
system catalog tables 4-39  
system catalog views 4-39  
System Managed Space 4-46

## T

table 4-6  
table lock compatibility 8-23

table space 4-6  
table space backup/restore considerations 7-87  
table statistics 10-103, 10-111  
Task Center 1-12  
temporary table impact on locking 8-32  
The 9-24  
timeouts 8-60  
timestamp 10-31  
timestamps 10-34  
Tool Settings 2-58  
Tools Settings 1-11, 1-12  
trace 9-75  
Transaction logging 7-21  
triggers - defining 5-75  
triggers - the need 5-69

## U

UDE 1-15  
uncatalog directory 4-94  
uncommitted read 8-32  
unqualified references 10-37  
updatable catalog views 10-118  
UPDATE 11-22  
UPDATE MONITOR SWITCHES 9-45  
updating statistics manually 10-118  
user-defined functions - defining A-25  
user-defined functions - deterministic A-31  
user-defined functions - external action A-31  
user-defined functions - fencing A-31  
user-defined functions - sourced on other functions A-28  
user-defined types - defining A-18  
user-defined types - the need A-16  
userexit 7-41  
utilities 6-20, 6-29, 6-32  
utility 6-20

## V

VALID 10-70  
Visual Explain 1-11, 9-84

## W

Wizards 2-42  
writing logs 7-21





